昇腾 310B 实战
从入门到精通边缘计算与人工智能

周贤中

前言

书名中的 “实战”，核心是 “在编程实践中学习”。本书作为昇腾 310B 芯片的入门教材，跳出单纯的理论讲解，通过真实的 AI 推理部署案例，带读者直观理解昇腾 310B 的硬件架构特性、Atlas 工具链使用逻辑与端侧 AI 项目开发流程，包括模型适配、量化优化、媒体处理、算子开发、性能分析与推理服务部署等环节。每一个知识点都配套可落地的代码示例，使读者在动手编码的过程中逐步掌握昇腾 310B 的工程应用能力，实现从 “了解芯片” 到 “能用芯片落地项目” 的跨越。

本书定位与目标读者

本书面向以下三类读者：

- **高校学生 / 科研新人**：希望通过一套系统化路径快速理解边缘 AI 硬件与部署流程。
- **嵌入式 / IoT 工程师**：已有一定 Linux / C / Python 基础，希望把 AI 模型真正跑在边缘端并做性能调优。
- **AI 应用开发者 / 创客**：已经能使用主流深度学习框架，希望将训练好的模型迁移到昇腾 310B 进行高效推理与产品化落地。

阅读预期：

- 零基础读者可依照 “快速起步路径” 完成第 1~3 章 + 精选案例；
- 进阶读者可继续深入算子优化、系统整合与复杂多模型协同部署；
- 有项目诉求的团队可参考 “方法论 + 附录模板” 直接搭建属于自己的边缘 AI 应用。

学习路径速览（建议路线）

1. 环境与工具链：硬件认知、开发环境装配、CANN 工具初试
2. 基础模型部署：图像分类、目标检测、语义分割、OCR 与 NLP
3. 媒体处理与底层能力：PyACL、DVPP、算子开发与内存管理
4. 性能优化：模型结构裁剪、精度与性能权衡、Profiling 与流水线优化
5. 系统构建：多进程、多模型协同、任务调度、监控与日志体系
6. 实战案例：需求分析、方案设计、模型适配、部署脚本与交付验收

全书结构（初版规划）

本书主体分为 **理论教程**与 **实践案例**，并在正文之后设置独立的附录部分。

理论教程

章节	标题	内容聚焦	读者收益
第 1 章	昇腾 310B 边缘计算基础	边缘计算概念、竞品对比、310B 规格	建立边缘计算认知与选型能力
第 2 章	CANN 软件栈核心组件解析	异构架构、ATC 转换工具、msame 推理	掌握 CANN 基础与模型转换流程
第 3 章	昇腾 PyTorch 扩展迁移基础	环境搭建、torch_npu 插件、经典网络迁移	学会在昇腾上运行 PyTorch 模型
第 4 章	PyACL 应用开发基础	ACL 运行时、内存管理、模型推理流程	掌握 C/C++ 底层应用开发能力
第 5 章	DVPP 视频处理基础	VENC、VDEC、VPC、JPEG 媒体处理	掌握 310B 媒体硬件调用与调试方法
第 6 章	算子开发实战	TBE DSL 流程、310B 计算架构、自定义算子	理解底层计算原理，具备算子扩展能力
第 7 章	性能分析与优化	Profiling 分析、瓶颈定位、AIPP/DVPP、流水线优化	掌握性能调优与系统工程化技能
第 8 章	模型量化案例教学	FP16、INT8、校准数据、精度对齐与性能对比	学会在精度与性能之间做可验证取舍
第 9 章	AI 项目交付闭环	SSD300 实验室实时检测、评测体系、标准化交付流程	建立规范的 AI 项目交付方法论

实践案例

包含从 **案例 1** 到 **案例 9** 的九个端侧 AI 实战案例，涵盖智能考勤机、目标跟踪检测、智能电子琴、掌纹识别、智能数据采集分析仪、智能小车、智能相册、实时手势识别与智能聊天机器人。

附录

附录用于集中收录实验基础和查阅型资料。附录 1 介绍开发板与基础环境，附录 2 提供基于昇腾 310B 的 Ubuntu 教程，附录 3 介绍 Python 编程基础，附录 4 介绍 Vibe Coding 基础，附录 5 提供 ROS2 基础教程，附录 6 汇集工具箱、FAQ 与参数模板，附录 7 说明 CANN、驱动与固件的版本配套、升级和当前版本查询。

各模块核心要点概述

第 1 章：昇腾 310B 边缘计算基础

- 边缘计算 vs 云计算：概念辨析与应用场景
- 边缘计算卡生态：NVIDIA、华为、瑞芯微对比
- 昇腾 310B 技术规格详解

第 2 章：CANN 软件栈核心组件解析

- CANN 异构计算架构与 CUDA 对比
- ATC 模型转换工具流程、参数详解与注意事项
- msame 推理工具使用概览

第 3 章：昇腾 PyTorch 扩展迁移基础

- PyTorch (torch_npu) 环境安装与配置
- 基础网络实现：Linear, MLP, LeNet
- 现代 CNN 移植实战：AlexNet, VGG, ResNet

第 4 章：PyACL 应用开发基础

- 快速入门：Hello World 与运行模式
- 初始化与运行时管理：Device, Context, Stream, Memory
- 模型推理完整流程：加载、Dataset 准备、执行
- Host 与 Device 之间的数据传输、同步与资源释放

第 5 章：DVPP 视频处理基础

- DVPP 编程模型、内存约束与数据格式
- VENC、VDEC、VPC、JPEG 的调用流程
- 视频编解码与图像处理案例、参数调试和性能测试

第 6 章：算子开发实战

- 为什么需要自定义算子：TBE vs AICPU
- 昇腾 310B 硬件架构：存储层级与向量计算单元
- TBE DSL 开发流程与基础算子实例

第 7 章：性能分析与优化

- 可运行 ResNet18-TinyImageNet 性能基线
- Profiling 性能分析：采集、分段计时与关键视图解读
- 核心优化技术：Buffer 复用、AIPP、DVPP 与 Queue Pipeline

第 8 章：模型量化案例教学

- FP32、FP16、INT8 的区别与适用场景
- PTQ、QAT、校准数据集与精度对齐
- ATC 精度模式、量化失败模式与性能对比流程

第 9 章：AI 项目交付闭环

- 基于 COCO SSD300 的实验室实时检测告警系统
- 需求澄清 Canvas、指标分层与场景后处理
- Baseline 策略、评测集设计与迭代看板
- 资产沉淀、交付目录与上线 Checklist
- 验收、回归与风险管理

实践案例 (案例 1 - 9)

- **案例 1**：智能考勤机
- **案例 2**：目标跟踪检测
- **案例 3**：智能电子琴
- **案例 4**：掌纹识别
- **案例 5**：智能数据采集分析仪
- **案例 6**：智能小车
- **案例 7**：智能相册
- **案例 8**：实时手势识别
- **案例 9**：智能聊天机器人

附录

- **附录 1**：开发板硬件、系统安装、启动与网络连接、基础环境验证
- **附录 2**：基于昇腾 310B 的 Ubuntu 教程，涵盖命令、权限、网络、SSH、VNC、NFS、Docker、自启动与板端操作
- **附录 3**：Python 编程基础
- **附录 4**：Vibe Coding 基础
- **附录 5**：ROS2 基础教程，涵盖 DDS、工作空间、通信、launch、TF2/URDF、多机通信与自启动
- **附录 6**：常见报错速查、模型转换参数模板、性能与质量 Checklist、术语表与 FAQ
- **附录 7**：CANN、NPU 驱动与设备固件的版本配套、升级、验收与回退边界

实践驱动与开源协作

本书所有示例代码、脚本、案例与附录工具均开源托管于本仓库。欢迎通过 Issue / PR 反馈问题、提交改进、补充案例或翻译。我们鼓励读者参与以下工作：

- 增补新模型 / 新任务的部署范式
- 分享 DVPP、PyACL、自定义算子与性能优化经验
- 提交性能测试报告（含硬件信息 + 指标）
- 翻译与文档校对

如何使用本书

读者类型	推荐阅读路径	目标	补充建议
零基础学生	附录 1、第 1 章、第 2 章、案例 1	能跑通首个模型	结合案例做改动实验
嵌入式工程师	第 4 章、第 5 章、第 6 章、第 7 章	掌握底层开发与优化	关注资源管理、媒体处理、算子开发与性能调优
AI 应用开发者	第 2 章、第 3 章、第 8 章、选读案例	快速场景落地	重点关注 PyTorch 迁移、模型部署与量化评估
技术负责人	第 1 章、第 7 章、第 8 章、第 9 章	构建团队方法论	制定内部量化评估、性能验收和交付标准

许可证与引用

本书内容采用 Apache 2.0 许可证。引用本书内容请注明：

《昇腾 310B 实战：从入门到精通边缘计算与人工智能》（GitHub: zhouxzh/Ascend310）

目录

I. 理论教程 **1**

1. 昇腾 310B 边缘计算基础 **2**

1.1. 边缘计算简介 2

1.1.1. 什么是云计算? 2

1.1.2. 什么是边缘计算? 2

1.1.3. 边缘计算 vs 云计算? 3

1.1.4. 何时采用边缘计算? 4

1.2. 边缘计算卡的发展历史 5

1.3. 常见的边缘计算卡 7

1.3.1. 英伟达 - 边缘 AI 的性能与生态王者 8

1.3.2. 华为 - 全栈全场景的国产化标杆 8

1.3.3. 瑞芯微 - 高性价比的 SoC 芯片专家 9

1.4. 昇腾 310B 简介 9

1.4.1. 昇腾 310B 技术规格详解 10

1.4.2. 其他常见的边缘计算卡的对比 11

1.5. 实践任务 12

2. CANN 软件栈核心组件解析 **13**

2.1. CANN 异构计算架构 13

2.1.1. CANN 异构计算与人工智能 13

2.1.2. CANN 与 CUDA 14

2.1.3. CANN 的架构 14

2.1.4. CANN 的快速安装 17

2.2. ATC 模型转换详解 19

2.2.1. ATC 工具介绍 19

2.2.2. ATC 工具使用流程 22

2.2.3. 模型推理工具 msame 工具概览 26

2.3. 章节小结 31

2.4. 实践任务 31

3. 昇腾 PyTorch 扩展迁移基础 **32**
3.1. 架构速览 . . . 32
3.2. 核心能力纵览 . . . 32
3.3. PyTorch 安装教程 . . . 33
3.3.1. CANN 环境准备 . . . 33
3.3.2. PyTorch 二进制软件包安装方法 . . . 33
3.3.3. torch_npu 安装后测试 . . . 35
3.4. torch_npu 插件的使用 . . . 36
3.4.1. 线性神经网络实现 . . . 37
3.4.2. 多层感知机实现 . . . 46
3.4.3. LeNet-5 . . . 52
3.5. torch_npu 插件兼容性测试套件 . . . 59
3.5.1. 测试目标与验证维度 . . . 59
3.5.2. 测试套件结构详解 . . . 59
3.5.3. 测试结果摘要 . . . 60
3.6. 现代卷积神经网络的移植 . . . 62
3.6.1. AlexNet . . . 62
3.6.2. VGG . . . 71
3.6.3. ResNet . . . 76
3.7. 总结 . . . 83

4. PyACL 应用开发基础 **85**
4.1. PyACL 的基本概念 . . . 85
4.1.1. PyACL 的逻辑架构 . . . 86
4.1.2. 基本概念与运行模型 . . . 87
4.1.3. PyACL 应用开发环境 . . . 89
4.1.4. PyACL 接口调用流程 . . . 91
4.1.5. 运行管理资源生命周期 . . . 92
4.1.6. 异构内存管理 . . . 95
4.1.7. 同步等待机制 . . . 99
4.2. 模型推理流水线 . . . 105
4.2.1. 模型推理开发流程解析 . . . 105
4.2.2. 实例分析（ResNet-18） . . . 106
4.3. 目标检测推理 . . . 124
4.3.1. SSD300（ResNet-50 主干） . . . 125
4.3.2. SSDLite320（MobileNet 主干） . . . 125
4.3.3. 推理流程对比 . . . 128
4.4. 总结 . . . 129

5. DVPP 视频处理基础 **131**
5.1. DVPP 基础概念与编程模型 131
5.1.1. DVPP 在芯片中的位置 131
5.1.2. DVPP 数据流 132
5.1.3. 子模块概览 132
5.1.4. AIPP vs DVPP（何时用哪个） 132
5.1.5. H.264 与 H.265 编解码基础 133
5.1.6. ACL 初始化——四步必要步骤 134
5.1.7. 通道模型 134
5.1.8. 回调线程模型 135
5.1.9. DVPP 内存管理 136
5.1.10. 描述符模型 136
5.1.11. NV12——DVPP 的通用货币 137
5.1.12. DVPP V1 与 himpi V2 — 两套 API 体系 137
5.1.13. 典型 DVPP 使用模式 138
5.1.14. 开发注意事项 138
5.2. VENC — 硬件视频编码 139
5.2.1. 理论背景 139
5.2.2. 环境与架构 140
5.2.3. VENC API 详解 142
5.2.4. 开发过程与常见问题与调试 143
5.2.5. 练习脚本 146
5.2.6. 集成到 aiortc 151
5.2.7. 性能对比与基准测试 152
5.2.8. VENC 调试与参数速查 156
5.3. VDEC — 硬件视频解码 157
5.3.1. VDEC 简介 157
5.3.2. VDEC 与 VENC 的关键区别 160
5.3.3. VDEC API 详解 160
5.3.4. 练习脚本走读 161
5.3.5. 常见问题与调试 167
5.3.6. 性能实测 169
5.3.7. VDEC 调试与参数速查 172
5.4. VPC — 硬件图像处理 173
5.4.1. VPC 简介 173
5.4.2. VPC 与 VENC/VDEC 的关键区别 174
5.4.3. VPC API 详解 175
5.4.4. 练习脚本走读 177

5.4.5. 性能基准 . . . 178
5.4.6. 常见问题与调试 . . . 179
5.4.7. 场景推荐 . . . 181
5.4.8. VPC 调试与参数速查 . . . 181
5.5. JPEG — 硬件编解码 . . . 182
5.5.1. JPEG 编解码简介 . . . 182
5.5.2. 与 VENC/VDEC/VPC 的关键区别 . . . 183
5.5.3. JPEGE API 详解 . . . 183
5.5.4. JPEGD API 详解 . . . 185
5.5.5. 练习脚本走读 . . . 186
5.5.6. 场景推荐 . . . 188
5.5.7. 常见问题与调试 . . . 188
5.5.8. JPEG 编解码参数速查 . . . 189
5.6. 集成实战：WebRTC 推流性能对比 . . . 190
5.6.1. 项目定位 . . . 190
5.6.2. 目录结构与角色分工 . . . 191
5.6.3. 关键模块走读 . . . 191
5.6.4. 性能实测（1920×1080） . . . 195
5.6.5. 依赖说明 . . . 196
5.6.6. 关键结论 . . . 196
5.7. 应用调试与常见 FAQ . . . 197
5.7.1. 调试技巧 . . . 197
5.7.2. 常见问题 . . . 197
5.7.3. 使用约束 . . . 197

6. 算子开发实战 199
6.1. 算子开发概述：昇腾 310B 硬件架构与开发路径 . . . 199
6.1.1. 昇腾 310B 处理器与达芬奇架构 . . . 199
6.1.2. 算子开发的两条主流路径 . . . 201
6.1.3. 小结 . . . 203
6.2. 初体验：使用 TBE DSL 快速实现第一个算子（向量加法） . . . 203
6.2.1. TBE DSL 快速概览 . . . 203
6.2.2. 算子分析：明确我们要做什么 . . . 206
6.2.3. 完整代码实现 . . . 207
6.2.4. 代码逐段详解 . . . 209
6.2.5. 编译算子 . . . 210
6.2.6. 算子验证：在 NPU 上运行测试 . . . 210
6.2.7. TBE DSL 开发的优势与局限 . . . 213

6.2.8. 小结与展望 214
6.3. 理解算子开发的核心概念 214
6.3.1. 算子原型定义 214
6.3.2. 算子信息库（.ini 文件） 215
6.3.3. Tiling（分片计算）的基本思想 216
6.3.4. 小结 217
6.4. 深入底层：Ascend C 算子开发入门 217
6.4.1. Ascend C 的编程模型 217
6.4.2. 工程结构 218
6.4.3. 向量加法的 Ascend C 实现 219
6.4.4. 编译与运行 221
6.4.5. Ascend C 与 TBE DSL 的对比回顾 221
6.5. 从简单到复杂：实现一个需要 Tiling 的算子（如矩阵加法） 221
6.5.1. 场景设定 222
6.5.2. Tiling 策略设计 222
6.5.3. 带 Tiling 的 Kernel 伪代码 222
6.5.4. 调试技巧 223
6.5.5. 小结与展望 224

7. 性能分析与优化基础 225
7.1. 性能优化先从可测量开始 225
7.2. 第 7 章样例目录 226
7.3. 工具箱：从轻量计时到系统 Profiling 227
7.3.1. Python 分段计时 227
7.3.2. msprof：观察 ACL API、Memcpy 和 NPU Timeline 227
7.3.3. npu-smi：确认设备状态 227
7.3.4. msame / ais_bench：独立测模型上限 228
7.3.5. ATC/AIPP 与 DVPP：把图像工作下沉到设备侧 228
7.4. 案例一：建立同步推理基线 229
7.5. 案例二：复用 ACL Buffer 和 Dataset 229
7.6. 案例三：CPU 预处理为什么容易成为瓶颈 230
7.7. 案例四：DVPP VPC resize 与 CPU resize 230
7.8. 案例五：AIPP 把归一化下沉到模型输入侧 231
7.9. 案例六：串行流程与 Queue Pipeline 232
7.10. 作者实测结果 233
7.11. 调优 Checklist 236
7.12. 练习任务 236
7.13. 小结 236

8. 模型量化与精度性能权衡 238
8.1. 从性能瓶颈到量化决策 . . . 239
8.2. 必备理论：FP32、FP16、INT8 与 PTQ . . . 240
8.3. 如何设计量化验证代码：先拆职责，再写脚本 . . . 242
8.4. 如何编写数据准备代码：让输入可复现 . . . 243
8.5. 预处理：所有模型对比必须使用同一套输入 . . . 245
8.6. ATC 转 FP32/FP16：先建立可比较的基准 . . . 246
8.7. 如何编写输出对齐程序：先看模型是否“像”基准 . . . 247
8.8. 如何编写性能对比程序：把耗时拆成可解释阶段 . . . 248
8.9. 如何编写 INT8 PTQ 程序：准备配置，再收集激活范围 . . . 249
8.9.1. 为什么选择 `amct_onnx` . . . 250
8.9.2. AMCT ONNX 环境 . . . 251
8.9.3. 准备阶段：生成量化配置 . . . 251
8.9.4. 校准阶段：用真实图片统计激活范围 . . . 253
8.9.5. ATC 转 INT8 OM . . . 254
8.10. 快速验证 INT8：漂移更大不等于不能用 . . . 255
8.11. 如何编写完整验证程序：把多个 OM 放到同一把尺上 . . . 255
8.12. 如何编写 sweep 实验：用最少胶水代码重复完整流程 . . . 258
8.13. 如何设计报告和故障回退 . . . 259
8.14. 小结 . . . 261

9. AI 项目交付闭环 262
9.1. 章节总览 . . . 262
9.2. 需求澄清 Canvas . . . 262
9.3. 指标分层与优先级 . . . 262
9.4. Baseline 策略与控制变量法 . . . 263
9.5. 评测集设计原则 . . . 263
9.6. 迭代计划与看板 . . . 264
9.7. 资产沉淀文档体系 . . . 264
9.8. 交付目录与不可变产物 . . . 264
9.9. 上线前综合 Checklist . . . 265
9.10. 验收、回归与漂移监测 . . . 265
9.11. 风险管理与决策日志 . . . 266
9.12. 章节小结 . . . 266
9.13. 实践任务 . . . 266

II. 实践案例 267

1. 案例 1：智能考勤机 268

1.1. 教程定位 . . . 268
1.2. 问题背景与系统边界 . . . 268
1.3. 系统架构 . . . 270
1.3.1. 一次图像请求的逻辑顺序 . . . 270
1.4. 模型与张量合同 . . . 271
1.4.1. 文件角色 . . . 271
1.4.2. 输入和前处理 . . . 272
1.4.3. 从 ONNX 到 OM . . . 272
1.5. PyACL 运行时与资源生命周期 . . . 272
1.5.1. 初始化 . . . 272
1.5.2. 一次执行 . . . 273
1.5.3. 释放 . . . 273
1.6. FastAPI 与 React 的协作边界 . . . 273
1.6.1. API 资源 . . . 273
1.6.2. 浏览器界面 . . . 274
1.6.3. 同步与异步的选择 . . . 274
1.7. 检测、特征与比对 . . . 274
1.7.1. 检测与裁剪 . . . 274
1.7.2. 特征比对 . . . 274
1.7.3. 两种考勤分支 . . . 275
1.8. 数据存储与隐私边界 . . . 275
1.9. 实验步骤与证据记录 . . . 276
1.9.1. 开发机上的纯 Python 检查 . . . 276
1.9.2. 开发板上的硬件检查 . . . 276
1.9.3. API 合同示例 . . . 276
1.10. 局限与可扩展方向 . . . 277
1.11. 小结 . . . 277
1.12. 参考文献 . . . 277

2. 案例 2：目标跟踪检测 278

2.1. 教程定位 . . . 278
2.2. 实验硬件与运行条件 . . . 278
2.3. 本案例支持的骨干网络 . . . 280
2.4. 目标跟踪对时序关联的要求 . . . 280
2.5. MobileNet-SSD 作为检测前端的依据 . . . 281
2.5.1. 边缘侧场景的基本要求 . . . 281

2.5.2. MobileNet 的网络结构 281
2.5.3. MobileNet v1 至 v4 的版本演进 282
2.5.4. MobileNet v1-v4 的结构对比总结 284
2.5.5. MobileNet 与 ResNet 骨干在本案例中的取向差异 284
2.5.6. SSD 的检测特点 285
2.5.7. MobileNet-SSD 用于目标跟踪的优缺点 285
2.6. DeepSORT 作为跟踪后端的选择依据 286
2.6.1. 从检测到跟踪，还缺少什么 286
2.6.2. DeepSORT 为什么适合作为入门算法 286
2.7. 本案例的代码结构与总体流程 287
2.7.1. 入口层 287
2.7.2. 检测层 287
2.7.3. 跟踪层 287
2.8. 检测部分代码解析 288
2.8.1. detection_app.py 如何组织检测主流程 288
2.8.2. backend_base.py 如何统一检测后端 288
2.8.3. preprocess_frame 体现了部署输入规范 289
2.8.4. decoder.py 如何体现 SSD 的核心思想 289
2.8.5. CPU 与 NPU 后端如何被统一接入 295
2.9. 从检测到跟踪：tracking_app.py 的桥接作用 295
2.10. 跟踪部分代码解析 296
2.10.1. Track 类如何表示一条轨迹 296
2.10.2. predict 和 update 对应卡尔曼滤波两阶段 296
2.10.3. kalman_filter.py 如何实现线性状态估计 297
2.10.4. 卡尔曼滤波器的发展历史与作者信息 297
2.10.5. 卡尔曼滤波器到底在解决什么问题 298
2.10.6. 本案例中的 predict 与 update 如何对应算法公式 299
2.10.7. 卡尔曼滤波在本案例中的适用性 300
2.10.8. DeepSORT.update 如何体现多目标跟踪主线 300
2.10.9. 数据关联为什么使用 IOU + 匈牙利算法 301
2.10.10. 匈牙利算法的发展历史与作者信息 301
2.10.11. 匈牙利算法到底解决什么问题 302
2.10.12. 本案例中匈牙利算法是如何被调用的 302
2.10.13. 匈牙利算法在本案例中的完整上下文 302
2.10.14. 匈牙利算法相对于贪心匹配的优势 303
2.10.15. 匈牙利算法在多目标跟踪中的局限 304
2.10.16. 轨迹生命周期参数如何影响最终效果 304
2.11. 可视化代码如何帮助理解算法结果 304

2.12. 如何评价这个案例方案 305
2.12.1. 这个方案的优点 305
2.12.2. 这个方案的不足 305
2.12.3. 作为入门案例的合理性 305
2.13. 实验设计 306
2.13.1. 只运行检测链路 306
2.13.2. 检测链路的性能拆分实验 306
2.13.3. 在检测基础上启用跟踪 307
2.13.4. 类别过滤与解码开销实验 307
2.13.5. 调节 DeepSORT 参数 308
2.13.6. 调节中心距离与平滑参数 308
2.13.7. 组合实验建议 309
2.14. 当前工程版本的优化说明 309
2.14.1. 检测链路中的工程优化 310
2.14.2. 进一步提高帧率的建议 312
2.14.3. 跟踪链路中的工程优化 313
2.14.4. 卡尔曼滤波器中的工程优化 314
2.14.5. tracking_app.py 当前支持的实用参数 315
2.14.6. 如何理解 “教材版” 和 “工程版” 的关系 316
2.15. 章节总结 316

3. 案例 3：智能电子琴 318
3.1. 案例目标与学习成果 318
3.1.1. 系统功能 318
3.1.2. 学习成果与边界 319
3.2. 器件与材料清单 319
3.2.1. 最小配置 319
3.2.2. 完整功能配置 320
3.2.3. 可选扩展 321
3.3. 系统总体架构与完整流程 321
3.3.1. 三条业务链 321
3.3.2. 硬件连接 321
3.3.3. 开发电脑与开发板的职责边界 323
3.4. DDSP 原理 323
3.4.1. 从采样合成到可微分 DSP 323
3.4.2. 基频、谐波和相位 324
3.4.3. 滤波噪声、包络、重采样和混响 324
3.4.4. 参数如何改变听感 325

3.5. 三套神经网络架构 326
3.5.1. Piano-DDSP 326
3.5.2. MIDI-DDSP 329
3.5.3. DDSP-VST Feature 与 Control 330
3.5.4. 三套网络对比 330
3.6. 模型转换与 Ascend 部署 332
3.6.1. 两类模型来源 332
3.6.2. 从 TFLite 到 ONNX 的图语义重构 332
3.6.3. ONNX 到 OM 的转换与验证 333
3.6.4. 运行时模型边界 335
3.7. 后端设计 335
3.7.1. 服务分层与作业模型 335
3.7.2. 资源互斥与实时钢琴 335
3.7.3. DDSP-VST 音频线程与安全控制 337
3.7.4. 通信语义 337
3.8. 前端设计 337
3.8.1. 信息架构与状态同步 337
3.8.2. 两个钢琴卷帘的时间模型 339
3.8.3. 触摸屏、桌面和手机布局 339
3.9. 逐页界面与操作说明 340
3.9.1. 实时演奏 / 触摸屏 340
3.9.2. 实时演奏 / MIDI 键盘 341
3.9.3. MIDI-DDSP 音频库 341
3.9.4. MIDI-DDSP 新建渲染 342
3.9.5. DDSP-VST 音色 345
3.9.6. DDSP-VST 输入门 345
3.9.7. DDSP-VST 效果 345
3.9.8. 设备概览 348
3.9.9. 音频输出与扬声器测试 349
3.9.10. 音频输入与麦克风测试 350
3.9.11. MIDI 设备 350
3.9.12. 运行环境 350
3.9.13. 布局验收场景 353
3.10. 从零复现实验 353
3.10.1. 接线与启动检查 353
3.10.2. 开发电脑准备与本地测试 354
3.10.3. 模型准备与转换边界 354
3.10.4. 部署、原子切换和启动 355

3.10.5. 功能验收顺序 355
3.11. 性能评价与故障排查 356
3.11.1. DDSP-VST 板端长稳评价方法 356
3.11.2. 常见问题 356
3.11.3. 测试矩阵 357
3.12. 系统分层、总结与参考资料 358
3.12.1. 软件分层 358
3.12.2. 数据与控制边界 358
3.12.3. 继续阅读 358
3.12.4. 总结 359
3.12.5. 参考资料 359

4. 案例 4：掌纹识别 360
4.1. 案例目标 360
4.2. 手动环境 360
4.3. API 和界面 360
4.4. 六模型人工测试 361
4.5. 摄像头测试 361
4.6. 常见问题 361
4.7. 版本冻结 362
4.8. 全候选模型测试 362

5. 案例 5：智能数据采集分析仪 364
5.1. 从波形到可解释事件 364
5.1.1. 本章要回答的学习问题 364
5.1.2. 证据等级与实验边界 365
5.2. 实验器材、接线与安全 365
5.2.1. Hantek 采集与扩展测量 365
5.2.2. RTL-SDR 实验 366
5.3. 读懂时域、频域和复数 IQ 368
5.3.1. 从一段波形得到什么 368
5.3.2. 窗口长度决定观察尺度 368
5.3.3. 为什么要去直流和加窗 370
5.3.4. 实值波形与复数 IQ 370
5.4. 双主线系统架构 373
5.4.1. 先划清两种 OM 的计算边界 373
5.4.2. 三种模型合同不能混用 373
5.4.3. 实时系统为什么需要有界队列 375

5.5. 从时频图到神经网络检测 375
5.5.1. 模型看到的是标准化的时频形态 375
5.5.2. 先限定模型证据能够说明什么 376
5.5.3. YOLO11s 如何从图像形成候选框 377
5.5.4. 从 NPU 张量到候选区域 379
5.6. 核心实验：从 CAL 方波到固定分析窗口 379
5.6.1. 先确认 USB，而不是直接启动仪表盘 379
5.6.2. 用 PulseView 建立最小真实基线 381
5.6.3. 编译长期运行的 sigrok 采集桥 381
5.6.4. 从连续帧组成固定分析窗口 382
5.6.5. 扩展任务：Little Bee B1 的接入准备 382
5.7. 在 310B 上完成固定 DFT 384
5.7.1. 把 DFT 写成静态 ONNX 图 384
5.7.2. 在开发板生成并验证 OM 384
5.7.3. 启动仪表盘并读懂结果 386
5.7.4. NPU 不可用时系统怎样表现 388
5.8. 保存并复核一次会话 388
5.8.1. 会话目录记录了什么 388
5.8.2. 从一条结果追到原始窗口 388
5.8.3. 当前仍未完成的部分 389
5.9. 扩展实验：从 RTL-SDR IQ 到时频检测 389
5.9.1. 先用固定 DFT 理解 IQ 频谱 389
5.9.2. 正式 RTL-SDR 服务的数据流 391
5.9.3. 谁计算时频图，谁完成检测 393
5.10. 模型准入与真实板端运行 393
5.10.1. 为什么一个 OM 文件还不够 393
5.10.2. 从模型原理到板端候选 395
5.10.3. 运行一次完整的真实窗口实验 395
5.10.4. 对同一份证据执行只读复核 397
5.11. 性能结果与工程判断 398
5.11.1. Hantek 采集吞吐说明了什么 398
5.11.2. RTL-SDR 连续流水线 398
5.11.3. NPU 并不天然快于最佳 CPU 算法 399
5.12. 从零复现、验收与故障排查 399
5.12.1. 本地开发检查 399
5.12.2. 开发板准备 400
5.12.3. Hantek 验收顺序 400
5.12.4. RTL-SDR 验收顺序 401

5.12.5. 常见故障 401
5.13. 思考题、总结与参考资料 402
5.13.1. 思考与改进任务 402
5.13.2. 可迁移的方法 402
5.13.3. Case 5 专项文档 403
5.13.4. 外部资料 403

6. 案例 6：智能小车 404
6.1. 项目简介 404
6.1.1. 与已有案例的关系 404
6.2. 内容大纲 404
6.2.1. 硬件准备 404
6.2.2. 软件环境 405
6.2.3. 车道线检测原理 406
6.2.4. 驾驶场景分类 407
6.2.5. 模型转换与昇腾部署 408
6.2.6. Web 界面 409
6.2.7. 用户手册 410
6.3. 源代码结构 412
6.4. 效果演示 413
6.4.1. 预期效果 413
6.4.2. 性能指标 413
6.4.3. 浏览器中的效果 413
6.4.4. 如何验证系统正常工作 414

7. 案例 7：智能相册 415
7.1. 案例目标与问题定义 415
7.2. 系统总体架构 415
7.2.1. PDF 版结构图 416
7.2.2. 数据流 418
7.2.3. 服务启动与请求时序 418
7.3. 硬件与软件基础 419
7.4. 核心数据模型 419
7.5. 多模型语义检索方法 420
7.5.1. CLIP 的共同向量空间 420
7.5.2. 参考实现与工程取舍 420
7.5.3. MobileCLIP-S0 421
7.5.4. Chinese-CLIP RN50 421
7.5.5. ResNet50 经典相似图 422

7.5.6. 自动路由 422
7.6. 昇腾 310B 模型迁移方法 422
7.6.1. 目标 SoC 与运行时版本 423
7.6.2. 选择性 FP32/FP16 与误差定位 423
7.7. NPU 运行时与串行资源管理 425
7.8. 智能选图与天气方法 425
7.9. 服务器化设计 426
7.9.1. 统一端口约定 426
7.9.2. 上传任务状态机 426
7.9.3. 配置与显示状态 426
7.9.4. ESP32 与 PhotoPainter 427
7.9.5. 从串口启动日志取得 PhotoPainter 的 IPv4 地址 427
7.9.6. ESP32 深度休眠后的唤醒与 310B 发现 428
7.9.7. 横竖屏图像方向 429
7.10. 触摸屏交互设计 429
7.11. E6 图像处理方法 430
7.12. 工程代码导读 430
7.13. 测试、性能与验收方法 431
7.14. 限制、许可证与后续工作 431
7.15. 参考资料 432

8. 案例 8：实时手势识别 433
8.1. 项目简介 433
8.2. 学习目标 433
8.3. 摄像头权限 434
8.4. 官方 Atlas 开发板验证 434
8.5. 系统流程 434
8.6. 模型与数据集 435
8.7. OM 推理 435
8.8. WebRTC 推流 436
8.9. 验收顺序 436

9. 案例 9：智能聊天机器人 437
9.1. 当前目标和已暂停范围 437
9.2. 当前 20T 已验收模型与选择 438
9.3. MindSpore 候选模型范围 439
9.4. 浏览器、网关、ACL 和 NPU 架构 439
9.5. 两种板卡和无 Torch 边界 440
9.6. 模型工件与哈希校验 441

9.7. 新板环境检查和模型门禁 441
9.8. 候选 Profile 服务和 OpenAI API 442
9.9. 网关和文字 UI 验证 443
9.10. 长输出、稳定性、中文质量和性能测试 443
9.11. 失败处理、回滚和证据保存 446
9.12. XiaoZhi 后续边界 446

附录 448

1. 开发板与基础环境 449
1.1. 昇腾 310B 开发板介绍 449
1.1.1. 开发板详细视图 449
1.1.2. 开发板硬件规格 449
1.2. 配件准备 449
1.3. 操作系统安装 456
1.3.1. 镜像下载 456
1.3.2. 校验下载文件 (MD5) 458
1.3.3. 刷写系统到 TF 卡 460
1.4. 启动开发板 465
1.4.1. 方式一：图形化界面启动 465
1.4.2. 方式二：串口登录 466
1.5. 网络连接 467
1.5.1. 无线网络连接 (WiFi) 467
1.5.2. 有线网络连接 471
1.6. 远程连接 (SSH) 471
1.6.1. Windows PowerShell 配置 SSH 密钥免密码登录 471
1.7. 验证开发环境 473
1.8. 结语 474

2. 基于昇腾 310B 的 Ubuntu 教程 475
2.1. 1. 教程定位与证据边界 475
2.1.1. 三类操作位置 475
2.1.2. 板端专属操作 476
2.1.3. 证据状态 476
2.1.4. 执行命令前的四个问题 476
2.2. 2. Ubuntu 与 Linux 基础 477
2.2.1. 终端、用户和提示符 477
2.2.2. 第一条环境检查命令 477

2.2.3. 可审计 shell 477
2.2.4. 命令权限与执行位置 478
2.3. 3. 文件系统、路径与常用命令 478
2.3.1. Ubuntu 目录结构 478
2.3.2. 绝对路径、相对路径和工作目录 479
2.3.3. 创建、查看和复制 479
2.3.4. 查看文本和日志 480
2.3.5. 查找、容量和文件类型 480
2.3.6. 归档和校验 481
2.4. 4. 用户、组、权限、sudo 与安全删除 481
2.4.1. Linux 用户和组 481
2.4.2. 修改权限 481
2.4.3. sudo 482
2.4.4. 设备组权限 482
2.4.5. 安全删除 482
2.5. 5. nano 与 vim 编辑器 483
2.5.1. nano 483
2.5.2. vim 483
2.6. 6. APT、换源、Conda、pip 与 Python 路径 484
2.6.1. APT 软件包管理 484
2.6.2. 换源 485
2.6.3. Conda 环境 485
2.6.4. pip 与 Python 解释器 485
2.6.5. Python 路径和动态库路径 486
2.7. 7. 网络、静态 IP、WiFi、热点与同网段检查 486
2.7.1. 网络状态 486
2.7.2. 一般静态 IP 概念 486
2.7.3. 同网段判断 487
2.7.4. 创建 WiFi 热点 487
2.7.5. 热点与 WiFi 模式切换 488
2.7.6. 自动连接优先级 488
2.7.7. 无屏幕联网的基本流程 488
2.8. 8. SSH 远程命令行控制 489
2.8.1. 客户端与服务端 489
2.8.2. 安装和启动服务端 489
2.8.3. 网络连通性检查 489
2.8.4. 登录 490
2.8.5. known_hosts 冲突 490

2.8.6. 同一网段排障 490
2.8.7. SSH 安全注意事项 491
2.9. 9. VNC 远程桌面控制 491
2.9.1. 客户端与服务端 491
2.9.2. Ubuntu 客户端安装 Remmina 491
2.9.3. Windows 客户端使用 MobaXterm 492
2.9.4. 分辨率和画面质量 492
2.9.5. 共享开关 492
2.9.6. 屏幕和显示要求 492
2.9.7. VNC 中打开终端 492
2.9.8. SSH 与 VNC 的对比 492
2.10. 10. NFS、SCP 与 rsync 493
2.10.1. NFS 的角色 493
2.10.2. 服务端安装和导出 493
2.10.3. 客户端安装和挂载 494
2.10.4. NFS 权限和安全 494
2.10.5. SCP 定向传输 495
2.10.6. rsync 同步 495
2.11. 11. Docker 基础与 ROS2 容器命令 496
2.11.1. Docker 基本概念 496
2.11.2. 通用容器生命周期命令 496
2.11.3. 教程中的 ROS2 容器命令 496
2.11.4. 源码目录、挂载和编译 497
2.11.5. 图形窗口和 DISPLAY 497
2.12. 12. 程序开机自启动：桌面项、启动应用和 systemd 498
2.12.1. 桌面自启动项 498
2.12.2. 启动脚本 498
2.12.3. Startup Applications 499
2.12.4. systemd 服务 499
2.12.5. 旧式 rc.local 499
2.13. 13. 昇腾 310B 的 CANN、PyACL、NPU、ATC 与 OM 500
2.13.1. 在同一 shell 加载 Conda 和 CANN 500
2.13.2. NPU 设备状态 500
2.13.3. ATC 转换 501
2.13.4. OM 工件检查 501
2.13.5. PyACL 与 OM 推理 501
2.13.6. 环境变量边界 502

2.14. 14. 板端设备、服务、日志、测试与证据边界 502
2.14.1. 系统状态报告 502
2.14.2. 摄像头和 V4L2 502
2.14.3. USB 和音频设备 503
2.14.4. 服务和端口 503
2.14.5. 进程和日志 504
2.14.6. 安全停止服务 504
2.14.7. 本地测试 504
2.14.8. 证据门 505
2.15. 15. 故障排查与禁止命令模式 505
2.15.1. 常见故障 505
2.15.2. 禁止或必须审查的命令模式 506

3. Python 编程基础 508
3.1. 1. 数据、类型与容器 508
3.1.1. 变量与基本类型 508
3.1.2. 数字运算与单位换算 509
3.1.3. 列表与元组 509
3.1.4. 字典 510
3.1.5. 集合 510
3.2. 2. 控制流 510
3.2.1. 条件分支 510
3.2.2. for 循环 511
3.2.3. range、enumerate 与 zip 511
3.2.4. while、break 与 continue 512
3.2.5. 列表与字典推导式 513
3.3. 3. 函数与模型清单 513
3.3.1. 函数封装 513
3.3.2. 默认参数 514
3.3.3. *args 与 **kwargs 514
3.3.4. lambda 515
3.3.5. f-string 515
3.3.6. 类型注解 515
3.3.7. dataclass 515
3.3.8. class 与 self 516
3.4. 4. 文件、JSON、命令行与异步 516
3.4.1. 缺失键与类型转换 516
3.4.2. pathlib 与 with open 517

3.4.3. json.loads 与 json.dumps . . . 517
3.4.4. try / except / else / finally . . . 518
3.4.5. argparse . . . 519
3.4.6. if **name** == “**main**” . . . 519
3.4.7. logging . . . 519
3.4.8. subprocess . . . 520
3.4.9. async / await . . . 520
3.5. 5. 调试与验证 . . . 520
3.5.1. 最小调试手段 . . . 520
3.5.2. 验证清单 . . . 521
3.5.3. 常见错误与坑 . . . 521
3.6. 6. 作业 . . . 521

4. Vibe Coding 基础 523
4.1. 1. Vibe Coding 是什么 . . . 523
4.2. 2. Agent 与 DSH 基础 . . . 524
4.3. 3. DSH 配置与安全检查 . . . 525
4.3.1. 确认环境 . . . 525
4.3.2. 配置模型与工作区 . . . 525
4.3.3. DSH 界面 . . . 528
4.3.4. 安全规则 . . . 528
4.4. 4. 第一次 Vibe Coding 任务 . . . 529
4.4.1. 提示词 . . . 529
4.4.2. 参考脚本 . . . 530
4.4.3. 预期输出 . . . 530
4.4.4. 验收步骤 . . . 530
4.5. 5. 提示词与审查方法 . . . 530
4.5.1. 提示词结构 . . . 530
4.5.2. 审查清单 . . . 531
4.6. 6. 参考资料 . . . 531
4.7. 7. 验证清单 . . . 532
4.8. 8. 常见错误与坑 . . . 532
4.9. 9. 作业 . . . 533
4.10. 10. 评分要点 . . . 533

5. ROS2 基础教程 534
5.1. 教程定位与证据边界 . . . 534
5.1.1. 命令来源与参考资料 . . . 535

5.2. 1. 定位、DDS、去中心化发现与教材边界 535
5.2.1. ROS2 在机器人软件中的位置 535
5.2.2. DDS 与去中心化发现 536
5.2.3. 与昇腾 310B 教材的边界 536
5.3. 2. 工作空间、功能包与构建系统 536
5.3.1. 工作空间目录结构 536
5.3.2. 创建工作空间并编译 537
5.3.3. 创建功能包 538
5.3.4. `package.xml`：包元信息与依赖声明 538
5.3.5. `CMakeLists.txt`：C++ 功能包的编译规则 539
5.3.6. `setup.py`：Python 功能包的入口点 539
5.4. 3. 节点、通信模型、接口与计算图 540
5.4.1. 节点与话题 540
5.4.2. 四种通信模型 540
5.4.3. 消息、服务与动作接口 541
5.4.4. 计算图 541
5.5. 4. 命令行工具 541
5.5.1. 运行节点与启动文件 541
5.5.2. 节点 542
5.5.3. 话题 542
5.5.4. 服务 542
5.5.5. 动作 543
5.5.6. 参数 543
5.5.7. 接口 543
5.5.8. 功能包与记录回放 544
5.6. 5. rqt、rviz2 与 ros2 bag 544
5.6.1. rqt 544
5.6.2. rviz2 545
5.6.3. ros2 bag 545
5.7. 6. Python launch 文件与参数传递 546
5.7.1. launch 文件的定位 546
5.7.2. 组合节点与包含其他 launch 文件 546
5.7.3. 真实底盘启动文件的结构 547
5.7.4. 参数传递的三种方式 548
5.8. 7. TF2 坐标变换与 URDF 548
5.8.1. TF2 的作用 548
5.8.2. 静态坐标变换 548
5.8.3. 坐标树与可视化 549

5.8.4. URDF 模型描述 549
5.8.5. 替换 URDF 模型的流程 550
5.9. 8. 相机、雷达、IMU 与串口底盘接口 551
5.9.1. 接口总览 551
5.9.2. 激光雷达 551
5.9.3. IMU 552
5.9.4. 串口底盘 552
5.9.5. 相机 553
5.10. 9. SSH 与 VNC 远程操作 ROS2 553
5.10.1. 何时使用 SSH 553
5.10.2. 何时使用 VNC 554
5.10.3. 选择建议 554
5.10.4. 远程操作的安全边界 555
5.11. 10. 多机通信与 ROS_DOMAIN_ID 555
5.11.1. 域号与发现范围 555
5.11.2. 同网段与网络检查 555
5.11.3. 常见发现失败原因 556
5.12. 11. 自启动、日志与进程管理 557
5.12.1. 日志位置 557
5.12.2. 进程查看与有序停止 557
5.12.3. 自启动方案 558
5.12.4. 运行状态检查清单 558
5.13. 12. proposed / untested：ROS2 与昇腾 310B 推理服务的集成路径 . . 559
5.13.1. 目标形态 559
5.13.2. 前置条件 559
5.13.3. 建议的实现步骤 559
5.13.4. 风险与失败模式 560
5.13.5. 验收门槛 561
5.14. 13. 课堂练习与验收标准 561
5.14.1. 练习任务 561
5.14.2. 验收标准 562

6. 工具箱、FAQ 与参数模板 564
6.1. 内容概览 564
6.2. 常见报错速查 564
6.3. 模型转换参数模板合集 565
6.3.1. 分类模型 (ResNet) 565
6.3.2. YOLO 动态分辨率 565

6.3.3. INT8 量化（示例） 565
6.4. 性能与质量 Checklist（执行勾项） 565
6.5. 术语表（扩展） 566
6.6. 推荐资源与外部引用 567
6.7. 贡献指南摘要 567
6.8. FAQ 567
6.9. License 与引用 567
6.10. 版本路线回顾 567
6.11. 实践任务 568

7. CANN 升级与版本配套 569
7.1. 本附录解决的问题 569
7.2. 教程路线与已验证组合 569
7.3. 软件栈中的版本层次 570
7.3.1. 驱动、固件、CANN 和框架适配层 570
7.3.2. 310B 的产品边界 571
7.4. 官方配套查询的使用方法 571
7.5. 历史版本线与 Ascend HDK 配套关系 571
7.5.1. CANN 与 Ascend HDK 571
7.5.2. 组合包与独立子包 572
7.5.3. 版本特性与 310B 结论的关系 572
7.6. 升级前的准备与门禁 573
7.6.1. 固定目标与维护窗口 573
7.6.2. 采集当前版本 573
7.6.3. 保存可回退的材料 574
7.6.4. 检查安装权限、空间和架构 574
7.6.5. 核对安装包而不是文件名 574
7.7. 共用安装与升级操作 575
7.7.1. 首次安装：驱动后固件 575
7.7.2. 覆盖升级：固件、驱动、CANN 575
7.7.3. 升级 CANN Toolkit 576
7.7.4. 子包独立升级 576
7.8. 板卡专项教程 577
7.8.1. 路线 A：香橙派 AI Pro 8T 577
7.8.2. 路线 B：华为官方 Atlas 310B 577
7.9. 升级后的统一验收顺序 581
7.9.1. 门 1：设备和驱动 581
7.9.2. 门 2：CANN 和 PyACL 581

7.9.3. 门 3：代表性模型 581
7.9.4. 门 4：服务和前端 582
7.9.5. 门 5：回归与性能 582
7.10. 已知变化与迁移检查 582
7.10.1. 废弃接口 582
7.10.2. 版本说明中的已知问题 583
7.10.3. CANN 9.0.X 小版本补丁的使用方式 583
7.11. 回退与故障处理 583
7.11.1. 回退原则 583
7.11.2. 失败时的最小记录 583
7.12. 面向本仓库案例的升级清单 584
7.13. 版本记录模板 584
7.14. 资料来源与使用边界 585

Part I.

理论教程

1. 昇腾 310B 边缘计算基础

1.1. 边缘计算简介

华为云等公共云计算平台允许企业以全国的云服务器作为其私有的数据与 AI 计算中心，将其基础设施扩展到任意地点，并根据需要向上或向下扩展计算资源。然而遍布全球实时运行的 AI 应用可能需要显著的本地处理能力，且往往位于距离集中式云服务器过远的偏远位置。而且出于低时延或数据驻留要求，一些工作负载需要保留在本地或特定地点，就是为什么许多企业使用边缘计算来部署其 AI 应用。边缘计算指的是在数据产生的位置进行处理，边缘计算在边缘设备中本地处理与存储数据。由于边缘计算设备无需依赖互联网连接，可以作为独立的网络节点运行。

1.1.1. 什么是云计算?

云计算（Cloud Computing）是一种计算风格，其可扩展与弹性能力通过互联网技术以服务形式交付。云计算依托集中化数据中心，通过互联网按需提供弹性、可扩展、托管式 IT 资源与服务（计算 / 存储 / 网络 / 数据 / AI 平台）。

云计算的好处是什么？- 较低的前期成本：购买硬件、软件、IT 管理以及全天候电力与制冷的资本支出被消除。云计算使组织能够以较低的财务进入门槛快速将应用推向市场。灵活的定价 – 企业只为其使用的计算资源付费，从而对成本有更多控制并减少意外。- 按需无限计算：云服务可以通过自动配置与撤销资源即时对不断变化的需求做出反应与适配。这可以降低成本并提升组织整体效率。- 简化的 IT 管理：云提供商为其客户提供访问 IT 管理专家的渠道，使员工可以专注于企业的核心需求。- 便捷更新：可一键访问最新的硬件、软件与服务。- 可靠性：数据备份、灾难恢复与业务连续性更容易且更便宜，因为数据可以在云提供商网络的多个冗余站点镜像。- 节省时间：企业可能在配置私有服务器与网络上耗费时间。借助按需云基础设施，它们可以在更短时间内部署应用并更快进入市场。

1.1.2. 什么是边缘计算?

边缘计算（Edge Computing）是一种分布式计算框架，旨在将计算、存储和网络能力部署在靠近数据源或终端设备的网络边缘位置。通过在本地或近端节点处理数据，减少向远端数据中心/云的集中回传，获得更低的时延、更好的带宽利用与更高的数据主权

与隐私保障。边缘计算是在距离数据产生源（传感器、摄像头、终端设备、工业控制点）物理更近的位置部署计算与存储，使数据在本地/近端被快速处理与筛选，减少长距离回传，保障低时延、带宽节省与数据主权。边缘计算是将计算能力在物理上靠近数据生成位置（通常是物联网设备或传感器）的实践。因为计算能力被带到网络或设备的边缘，边缘计算能够实现更快的数据处理、增加的带宽以及确保的数据主权。

通过在网络边缘处理数据，边缘计算减少了大量数据在服务器、云与设备或边缘位置之间往返传输以被处理的需求。这对诸如数据科学与 AI 等现代应用尤为重要。许多高计算应用（如深度学习与推理、数据处理与分析、仿真与视频流）已成为现代生活的支柱。随着企业日益意识到这些应用由边缘计算驱动，生产中的边缘用例数量应会增加。

边缘计算的特点是什么？- 靠近数据源：在物联网设备、网关、工业控制器或本地微型服务器附近直-成初级或核心推理逻辑，降低往返延迟。- 分布式架构：区别于云的集中式调度，采用多节点协同与局部自治，适合-化、地理分散与对实时性敏感的任务。- 实时响应：适配无人驾驶、工业控制、视频安防、AR/VR 等对毫秒级响应-的场景。- 隐私与安全：敏感原始数据（面部特征、生产工艺、地理轨迹）在本地预-或结构化提取后再上云，降低泄露与合规风险。- 带宽优化：仅上传事件/特征/聚合指标，显著降低原始全量视频/传感流量-路的占用。- 弹性与容错：弱网/离线时保持关键功能脱网运行，网络恢复后再同步（延迟一致性）。

边缘计算的好处是什么？- 更低时延：在边缘进行数据处理会消除或减少数据传输。这可为需要低时延的复杂 AI 模型用例（如完全自动驾驶车辆与增强现实）加速洞察。- 降低成本：使用局域网进行数据处理相比云计算可为组织提供更高带宽与更低成本的存储。此外，由于处理发生在边缘，需要发送到云或数据中心进一步处理的数据更少。这导致需要传输的数据量减少，成本也降低。- 模型精度：AI 依赖高精度模型，尤其是需要实时响应的边缘用例。当网络带宽过低时，通常通过降低输入模型的数据尺寸来缓解。这会导致图像尺寸缩小、视频跳帧、音频采样率降低。部署在边缘时，数据反馈回路可用于提升 AI 模型精度，并且可同时运行多个模型。- 更广覆盖：传统云计算必须依赖互联网接入。而边缘计算可在本地处理数据，无需互联网接入。这将计算范围扩展到以前无法访问或偏远的位置。- 数据主权：当数据在其采集位置被处理时，边缘计算允许组织将所有敏感数据与计算保留在局域网和公司防火墙内。这减少了暴露于云端网络安全攻击的风险，并在不断变化的严格数据法律下具备更好合规性。

1.1.3. 边缘计算 vs 云计算?

维度	边缘计算	云计算	典型取舍 / 典型场景
处理位置	近端（本地/网关/边缘节点）	远端数据中心	边缘降低时延并就近处理高带宽原始数据（如视频）；云用于集中算力与全局聚合。
时延特性	低（本地判决 20~ 几十 ms）	受网络往返影响（>100ms 场景常见）	实时/控制闭环优先边缘；非实时批处理、训练优先云。

维度	边缘计算	云计算	典型取舍 / 典型场景
带宽占用	上行压缩（只传事件/特征/聚合）	常上传原始数据到云	当传输成本高或链路受限，将预处理放到边缘；带宽充足且需保留原始数据则上云。
部署/运维	分布式，需节点管理（异构、离线可用）	集中化，统一维护与弹性伸缩	节点数多时运维复杂度上升（需探针/模板）；云侧适合动态弹性与大规模训练/批处理。
隐私合规	本地脱敏/保留数据主权易控	数据集中存储需合规审核	高度敏感或受监管数据优先边缘；低合规风险且需统一治理优先云。
伸缩弹性	受制于本地硬件与现场成本	云端资源弹性丰富	现场扩展（CAPEX）与运维（OPEX）成本较高；云适合突发/动态扩展工作负载。
典型适用场景（示例对照）	实时推理、低延迟控制、弱网/离线场所	非实时批处理、模型训练、集中化数据湖	云：非时间敏感数据、可靠互联网、已在云存储的数据；边缘：实时数据处理、受限或无联网地点、传输成本过高的大规模本地数据、受严格法规约束的数据。

一个边缘计算优于云计算的示例是医疗机器人，外科医生需要实时数据访问。这些系统包含大量可在云中执行的软件，但手术室中日益出现的智能分析与机器人控制无法容忍时延、网络可靠性问题或带宽限制。在此示例中，边缘计算为患者提供了生死攸关的益处。

1.1.4. 何时采用边缘计算?

边缘节点的硬件选择，正如您所指出的，本质上是功耗、体积和成本之间的权衡。像华为 Ascend 310B 这样的处理器正是这一理念的产物：它不追求极致的算力，而是在满足边缘场景基本 AI 推理需求的前提下，尽可能实现能效比和成本的最优解。这种硬件特性直接决定了其所能支撑的应用边界。边缘计算的价值在多种场景中得以凸显。在物联网网关中，它扮演着 “区域数据枢纽” 的角色，对海量传感数据进行本地预处理、协议转换与降噪聚合，大幅减轻上行带宽压力。工业自动化场景，如产线质量检测与能耗分析，依赖边缘节点的毫秒级响应能力，实现控制闭环的实时性与可靠性。在智慧城市的交通流量或环境监测中，边缘计算使得低延时告警成为可能，同时有效节省了持续视频流上传所需的巨大带宽。

安防监控是边缘计算的经典应用，通过实时视频结构化分析，将原始视频转化为结构化的事件信息（如 “有人越界”、“车辆违章”）再上传，既保护了个人隐私，又提升了事件处理效率。智能零售中的客流与货架分析，则体现了边缘的设备自治特性，即使在网络不佳时也能独立运行。而对于车路协同中的路侧单元，超低时延与本地协同决策是保障行车安全的核心，任何云端的往返延迟都是不可接受的。

边缘计算并非云的替代品，而是与云共同构成了 “端 - 边 - 云” 三层协同的健壮体系。在这个体系中，端侧负责产生原始数据，边缘侧专注于低时延的智能决策、实时控制与数据 “提纯”，而云端则依托其强大的算力与存储资源，负责全局模型的训练、长周期的大数据分析与跨区域的调度协同。

一个合理的功能切分策略至关重要，它直接决定了整个系统的成本结构、响应性能与可持续演进能力。判断一个功能更适合放在云还是边缘，可以遵循一些基本原则：适合上云的任务通常是非实时的批处理、模型训练、需要动态弹性伸缩的业务，或者数据本就存储在云数据湖中的场景，且对合规和延迟不敏感。相反，更适合下沉到边缘的任务则对实时推理和控制、稳定持续的低时延有强需求，其数据来源密集且分散，或者涉及高敏感数据受合规监管，同时面临着带宽受限或成本高昂的现实约束。

在具体的工程实现上，云和边缘的偏好存在显著差异，这要求开发者采用不同的技术策略。

在日志策略上，云侧倾向于全量集中收集以方便分析，而边缘侧由于带宽和存储限制，必须采用采样与本地环形缓存截断等轻量化手段。模型分发时，云侧可以利用 CDN 轻松推送大文件，边缘侧则需要差分升级、分块传输与严格的哈希校验机制，以确保在不稳定网络下的更新成功率。配置管理方面，边缘服务需支持配置嵌入版本与增量下发，并必须具备离线安全回滚能力，以应对网络中断的极端情况。

监控系统的设计上，云侧普遍采用 Prometheus 等集中式时序数据库，而边缘侧则需要轻量的代理，并优先在本地缓存指标，在资源冲突时甚至需要丢弃低优先级数据。对于安全补丁，云上可以做到自动批量推送，但在边缘侧，为了避免对关键业务造成意外中断，通常需要设定计划维护窗口或要求手动确认，实施更为谨慎的更新策略。

总而言之，边云协同的主旋律是 “边缘强化实时响应与局部自治，云强化全局优化与集中智能”。在设计系统时，一个有效的思路是以 “放在云端的必要性” 为拷问，反向审视每一个功能模块。任何不具备必须上云理由的功能，都应优先考虑下沉到边缘。最终的功能切分边界，应当由可观测的量化指标来界定，包括时延、带宽消耗、总体拥有成本、处理精度以及合规等级等。

在实际的资源与任务编排中，需要精细化管理。例如，将高 I/O 密度的任务与计算密集型任务错峰执行，以避免资源争抢。将热点算子进行分组，防止它们在短时间内同时提交导致带宽剧烈抖动。同时，必须高度重视热设计，通过持续监控硬件温度（如每 5 秒采样），在超过安全阈值（如 85°C）时自动触发降频或任务降级等保护性负载策略，确保边缘设备在复杂现场环境下的长期稳定运行。

1.2. 边缘计算卡的发展历史

边缘计算卡的发展历程，是一场为了将 “智能” 从遥远的云端数据中心不断推向现实生活前沿的 “迁徙史”。在早期，所谓的边缘节点大多是基于通用的低功耗 CPU 构建，处理能力有限，难以胜任复杂的实时计算任务。那时可以说是 “有边缘，而无计算”。随着自动驾驶和安防监控等应用对视觉处理的需求爆发，市场开始探索专用的加速方案。**FPGA** 因其可重构性和低延迟成为重要选择，同时，像英特尔 Movidius 这类专为视觉优化的低功耗芯片也开始出现，让设备真正具备了本地处理 AI 的能力。真正的转折点来自深度学习浪潮。英伟达将其 GPU 技术下沉，推出了划时代的 **Jetson 系列**。这不仅带来了强大的

算力，更重要的是建立了成熟的 CUDA 软件生态，让边缘 AI 开发从硬件调优变成了软件工程，极大地降低了开发门槛。近年来，边缘计算卡的发展进入了百家争鸣的时代。英特尔通过 “CPU+VPU+FPGA” 组合和 OpenVINO 工具链提供灵活方案；高通将移动端的高能效 SoC 设计经验引入边缘；谷歌则推出了为 TensorFlow 量身定制的 Edge TPU，追求极致能效。同时，国产力量快速崛起，华为的昇腾系列、寒武纪的思元系列等，都在安防、自动驾驶等特定领域展现出强大竞争力。如今，边缘计算卡已不再是单纯的算力竞赛，而是进入了**算力、功耗、成本、软件栈和行业生态**的综合竞争阶段。它的演进史，核心始终是为了让智能在物理世界的每个角落都能实时、可靠地运行。

当前，海外边缘计算市场格局鲜明，由几家技术路径各异的科技巨头共同引领。

英伟达凭借其 Jetson 系列，被视为边缘设备中的 “微型超算”。该系列覆盖从入门到旗舰的全场景需求，不仅以强劲性能著称，更以成熟的 CUDA 和 TensorRT 软件生态构筑起护城河。无论是复杂的机器人视觉系统还是自动驾驶车辆，Jetson 往往是开发者实现高性能边缘 AI 的首选平台。

英特尔则展现出 “全能型选手” 的特质。它不仅以 CPU 处理通用计算任务，还通过 Movidius 视觉处理单元等专用芯片强化 AI 视觉推理。其核心优势在于 OpenVINO 软件工具包——能够将 AI 模型高效适配并优化至英特尔各类硬件平台，使其在工业自动化、智能零售等需要灵活部署的领域中广受青睐。

AMD 在整合赛灵思之后，实力显著提升。其杀手锏在于 “自适应计算”，尤其是 FPGA 芯片的可重构特性，能够通过硬件级定制实现超低延迟加速。这一特性让 AMD 在专业机器视觉、通信基础设施等需要快速算法迭代的特定场景中脱颖而出。

高通将移动芯片领域积累的低功耗与高集成度经验成功复用于边缘侧。其芯片方案集成了专用 AI 引擎与 5G 连接能力，特别适合无人机、扩展现实设备等对续航和体积敏感的移动场景，为电池供电的边缘设备提供了持久智能的可能。

谷歌选择了一条差异化的技术路径，推出专为 TensorFlow 模型推理设计的 Edge TPU 芯片。这款芯片以极低的功耗和紧凑的体积，在兼容的 AI 任务中提供出色的推理速度。对于深度融入谷歌 AI 生态的开发者而言，基于 Edge TPU 的 Coral 开发板成为一个高效、节能的部署选择。

国内边缘计算卡的发展，是一部从” 跟跑” 到” 并跑” 的奋斗史。大约在 2016 年前后，随着 AI 浪潮兴起，国内边缘计算开始萌芽。最初阶段，大多数企业还是采用国外芯片做集成开发，或者在通用芯片上进行适配优化。这个时期可说是” 用起来就好” 的实用主义阶段。2018 年左右，随着” 中兴事件” 的爆发，自主可控成为国家战略，真正推动了国产边缘计算芯片的研发热潮。以**华为昇腾 310** 芯片为代表的第一代国产边缘 AI 芯片正式亮相，标志着我们开始拥有自己的” 算力底座”。这款芯片不仅在性能上对标国际产品，更重要的是实现了从芯片到框架的全栈自主创新。几乎在同一时期，**寒武纪**的思元系列、**地平线**的征程系列等专用 AI 芯片也纷纷落地。这些芯片不再是简单的替代品，而是在特定场景下展现出了独特优势——比如寒武纪在算力密度上的突破，地平线在自动驾驶领域的专注深耕。2020 年之后，国产边缘计算进入了” 场景深化” 阶段。各家企业不再追求单纯的算力指标，而是更注重实际应用效果。华为 Atlas 系列在智慧城市项目中大规模部

署，地平线的征程芯片在多家车企实现前装量产，黑芝麻智能则专注于大算力自动驾驶芯片的研发。这些成就显示，国产边缘计算卡已经从" 能用" 进化到了" 好用" 的阶段。如今，国产边缘计算卡正在形成自己独特的发展路径——不仅在性能上紧追国际先进水平，更在能效比、场景适配度和成本控制上展现出越来越强的竞争力。从最初的艰难起步，到现在的百花齐放，这段发展历程见证了中国在高科技领域的快速崛起。

近年来，国内科技公司在自研 AI 芯片和计算卡领域进展迅速，形成了具有鲜明特色的产品体系。

华为是国内边缘计算领域的领头羊，其 **Atlas 系列**是基于自研的 "昇腾"（Ascend）AI 处理器。它不仅仅是一张卡，更是一个完整的解决方案。从面向开发者的 **Atlas 200I DK** 套件，到集成了多张加速卡的 **Atlas 500** 智能边缘服务器，华为提供了覆盖多种场景的产品形态。其最大优势在于 "软硬件协同"，通过自家的 CANN 驱动和昇思（MindSpore）AI 框架，能充分发挥芯片性能，特别在安防、智慧城市等要求自主可控的项目中成为首选。

寒武纪是国内专业的 AI 芯片上市公司，其 "思元"（MLU）系列边缘计算卡在业界拥有很高的知名度。寒武纪的芯片以**专业化的 AI 算力**和**高计算密度**见长，在智能安防、智慧交通等领域实现了规模化应用。它的产品是纯计算加速卡形态，可以灵活地插入到标准的服务器中，为数据中心和边缘机房提供强大的推理能力。

地平线是另一家极具特色的公司，其强项在于**自动驾驶和智能驾驶舱**领域。它提供的不是单一的计算卡，而是以 "芯片 + 工具链" 为核心的解决方案。其**征程系列**车规级 AI 芯片，功耗控制非常出色，专门为处理智能汽车上的视觉感知、环境建模等任务而优化。通过与众多车企和零部件供应商合作，地平线的技术已经广泛应用在众多量产车型中。

此外，像**瑞芯微**这样的传统 SoC 设计公司，其 **RK3588** 等高端芯片也具备了可观的边缘 AI 算力。虽然它们通常以核心板的形式出现，但因其极高的**性价比**和强大的**多媒体处理能力**，被广泛集成到各种边缘设备中，如智能 NVR、商显广告机和工业控制设备，是中低算力需求场景中非常普遍的选择。

总的来说，国内边缘计算卡市场呈现出百花齐放的态势：华为提供全栈式方案，寒武纪提供专业的算力卡，地平线和黑芝麻则深耕自动驾驶这一垂直领域，而瑞芯微等则在更广泛的 AIoT 市场占据重要地位。这些国产力量共同为各行各业的智能化转型提供了坚实且多样化的算力基础。

1.3. 常见的边缘计算卡

好的，我们重点来介绍一下**英伟达**、**华为**和**瑞芯微**这三家在边缘计算领域颇具代表性的产品。这三家的产品定位和优势各有侧重，形成了从高性能到高性价比的全面覆盖。

1.3.1. 英伟达 - 边缘 AI 的性能与生态王者

英伟达凭借其强大的 GPU 架构和成熟的软件栈（CUDA, TensorRT 等），在边缘 AI 领域占据了主导地位，尤其是在需要复杂 AI 推理的场景中。英伟达的 Jetson 系列构成了一个完整的嵌入式 AI 计算平台，涵盖了从核心计算模块到载板设计，再到软件栈的全套解决方案。

当前产品线的中流砥柱是 Jetson Orin 系列。其中的旗舰型号 Jetson AGX Orin 堪称性能猛兽，拥有高达 275 TOPS 的 AI 算力，性能足以媲美小型服务器，能够胜任自主机器、高级机器人、医疗影像等最严苛的边缘应用。对于需要高性能但空间受限的场景，Jetson Orin NX 提供了理想的解决方案，它在紧凑的体积内实现了最高 100 TOPS 的算力，成为多路视频分析设备和边缘服务器的首选。而 Jetson Orin Nano 则重新定义了入门级边缘 AI 的标准，以 20-40 TOPS 的算力为新一代 AI 产品和原型开发打开了新的可能。

除了新一代产品，一些经典机型仍然在市场上发挥着重要作用。Jetson Xavier NX 凭借其在性能和价格之间的出色平衡，至今仍在许多项目中广泛使用。更早的 Jetson Nano 虽然性能已经不及新产品，但其低廉的成本和庞大的开发者社区，使其仍然是教育入门和简单项目的绝佳选择。

这个平台最突出的优势在于其极致的性能表现和无比成熟的软件生态。CUDA-X AI 为开发者提供了完善的工具链，加上活跃的社区支持和丰富的学习资料，大大降低了开发门槛。不过，这些优势也伴随着相应的代价——Jetson 产品的价格相对较高，功耗表现也不算特别出色。正因如此，它们主要被应用于自动驾驶、高端机器人、复杂视频分析等对计算性能要求极高的场景中。

1.3.2. 华为 - 全栈全场景的国产化标杆

华为基于其自主研发的昇腾 AI 处理器与昇思 AI 框架，构建了一套从底层芯片到顶层应用的 “全栈式” 人工智能解决方案。这一完整的技术布局使华为在安防、智慧交通等对自主可控要求较高的领域展现出独特优势。

在边缘计算领域，华为通过 Atlas 系列产品提供了丰富的形态选择。对于开发者而言，Atlas 200I DK A2 是一个理想的入门平台，这款小巧的开发板搭载昇腾 310 处理器，为初学者提供了体验昇腾生态的便捷途径。而在实际部署中，Atlas 500 Pro 作为集成的智能边缘服务器，以其丰富的接口和稳定的性能，成为智慧交通、园区管理等场景的常见选择。此外，像 Atlas 300I 这样的 PCIe 加速模块，则为企业现有的工控设备和服务器提供了灵活高效的 AI 能力扩展方案。

这套边缘计算体系的核心优势在于完全的国产化技术栈，通过 CANN 驱动实现了深度的软硬件协同优化，同时与华为云服务形成了良好的端边云协同能力。不过，与英伟达等国际巨头相比，昇腾生态的成熟度和社区资源仍处在追赶阶段。目前，华为 Atlas 系列

特别适合应用于安防监控、智慧城市、工业质检等对数据安全和国产化有明确要求的项目场景。

1.3.3. 瑞芯微 - 高性价比的 SoC 芯片专家

瑞芯微作为国内领先的集成电路设计企业，以其高集成度与出色性价比在 AIoT 芯片领域占据重要地位。与提供完整计算卡的厂商不同，瑞芯微专注于核心 SoC 系统级芯片的设计，由下游合作伙伴基于其芯片开发出形态丰富的开发板与整机产品。

在其 AIoT 芯片系列中，RK3588 系列堪称旗舰代表。这款芯片不仅搭载了八核高性能处理器，还集成了算力达 6 TOPS 的自研 NPU，支持多种精度量化方式。特别值得一提的是其卓越的多媒体处理能力，支持 8K 高清解码与多路摄像头输入，使其成为边缘计算盒子、智能 NVR 及商显设备等高端应用的理想选择。

面向主流市场，RK3568 系列则展现出强大的市场号召力。这款芯片在算力与功耗之间取得了良好平衡，集成的 1 TOPS NPU 为各类 AI 应用提供了基础算力保障。目前该芯片已被广泛应用于工业控制、智能门禁、网络存储设备等场景，更是成为了 Firefly、Radxa 等知名开发板平台的核心选择。

总体而言，瑞芯微方案的最大优势在于极致的性价比与高度的集成化——单颗芯片即融合了 CPU、GPU、NPU 及多媒体处理单元。虽然在绝对 AI 算力上无法与英伟达、华为的高端产品直接竞争，其软件生态也仍在持续完善中，但在智能门禁、商显广告机、网络录像机等中低算力需求的 AIoT 应用场景中，瑞芯微无疑提供了一个经济实用且性能均衡的优质选择。

1.4. 昇腾 310B 简介

昇腾 310 是华为昇腾（Ascend）AI 计算产品线的**关键入门级芯片**，它的发展史与华为全栈 AI 战略的落地紧密相连。

在 2018 年的华为全联接大会上，华为首次发布了昇腾 AI 芯片系列，其中**昇腾 310** 作为面向**边缘和端侧**场景的处理器正式亮相。它的设计目标非常明确：在**极低的功耗（仅 8W）**下，提供足以在边缘端进行实时 AI 推理的算力。这与面向数据中心的昇腾 910（训练芯片）形成了 “云边协同” 的搭配。

昇腾 310 并非追求极致算力，而是追求**极致的能效比和通用 AI 能力**。它采用了华为自研的达芬奇架构（关于达芬奇架构的 AI Core 计算单元、存储层次等深度解析请参见第 6 章），在一张芯片上集成了 CPU、DVPP（数字视觉预处理）和 AI 计算核心，使其特别适合处理视频、图像等非结构化数据。早期，它被集成在 Atlas 200/300 系列的加速模块中，用于安防摄像头的视频结构化、智慧交通的车辆识别等场景，初步展现了其在实际部署中的价值。

“310B” 可以被理解为昇腾 310 的一个**优化和增强版本**。虽然官方没有大肆宣传其细节改进，但业界普遍认为，B 版本在算力、稳定性和可靠性上进行了提升，以更好地满足严苛的工业和商业环境需求。

1.4.1. 昇腾 310B 技术规格详解

华为昇腾 310B 系列提供了两个不同算力等级的 AI 加速模块，分别是 20T 算力版本跟 8T 算力版本。它们在核心架构、接口和外形上高度统一，主要差异在于性能配置。详细的技术规格如下表所示：

规格项	昇腾 310B（20 TOPS）	昇腾 310B（8 TOPS）
AI 算力	20 TOPS INT8 10 TFLOPS FP16	8 TOPS INT8 4 TFLOPS FP16
内存规格	LPDDR4X 12 / 8 / 4 GB 总带宽 51.2 / 34.1 / 34.1 GB/s 支持 ECC	LPDDR4X 4GB 总带宽 25.6 GB/s 支持 ECC
CPU 算力	4 核 × 1.6 GHz	4 核 × 1.0 GHz
编解码能力	**解码：** H.264/H.265 硬件解码 40 路 1080P 30FPS 4 路 4K (3840×2160) 75FPS **编码：** H.264/H.265 硬件编码 20 路 1080P 30FPS 3 路 4K (3840×2160) 50FPS **JPEG：** 解码 1080P 512FPS 编码 1080P 256FPS 最大分辨率：16384×16384	**解码：** H.264/H.265 硬件解码 20 路 1080P 30FPS 2 路 4K (3840×2160) 75FPS **编码：** H.264/H.265 硬件编码 12 路 1080P 30FPS 2 路 4K (3840×2160) 50FPS **JPEG：** 解码 1080P 512FPS 编码 1080P 256FPS 最大分辨率：16384×16384
高速接口	8 lane，支持：SGMII（1.25Gbps） 1000BASE-R（1.25Gbps）USB3.0（5Gbps）SATA 3.0（6Gbps）PCIe Gen3（8Gbps）向下兼容各接口旧版本	同左
低速接口	UART × 5 I2C × 4 SPI × 2 CAN × 4	同左
典型功耗	25W	21W
工作环境温度	$-20^{\circ}\mathrm{C} \sim +103^{\circ}\mathrm{C}$	同左
结构尺寸	MXM 连接器 82mm × 60mm × 7mm	同左

20 TOPS 版本在整体性能上全面领先，其 AI 算力（20 TOPS INT8）和 CPU 频率（1.6GHz）均高于 **8 TOPS 版本**（8 TOPS INT8, 1.0GHz），并配备了更灵活、带宽更高的内存（最高 12GB, 51.2GB/s）。在视频处理能力上，20 TOPS 版本也能支持更多的视频解码与编码路数。

两款产品都集成了丰富的接口并支持宽温工作，确保了在边缘环境下的连接性和可靠性。**8 TOPS 版本**则以其稍低的 21W 功耗和适中的配置，提供了一个更具性价比的解决方案。综上所述，20 TOPS 版本定位为高性能选择，适用于计算密集型任务；而 8 TOPS 版本则面向算力需求适中、对成本更敏感的应用场景。

1.4.2. 其他常见的边缘计算卡的对比

华为昇腾 310B、瑞芯微 Rk3588 与英伟达 Jeston Orin Nano 三者之间的定位各有不同。华为昇腾 310B 是一款专为边缘 AI 推理场景设计的人工智能处理器。它不追求极致的通用算力，而是专注于在特定功耗下提供高效、稳定的 AI 推理能力，是华为全栈 AI 解决方案在边缘侧的关键载体，强调自主可控与场景落地。瑞芯微 RK3588 是瑞芯微的旗舰级的高端系统级芯片，其定位是一个 “全能战士”。它集成了强大的 CPU、GPU、NPU 以及顶尖的多媒体处理单元，旨在为各类 AIoT 设备提供一个高度集成、高性价比的 “大脑”，核心优势在于多功能集成与极致性价比。Jetson Orin Nano 是英伟达面向入门级边缘 AI 市场推出的嵌入式系统模块。它继承了英伟达在 GPU 计算领域的绝对优势，在小型封装内提供了惊人的 AI 算力，其核心价值在于强大的性能和无与伦比的成熟软件生态，是 AI 开发者的首选平台之一。它们之间的详细性能参数对比如下表所示：

规格类别	昇腾 310B (20T)	Rockchip RK3588	Jetson Orin Nano 8GB
AI 性能	20 TOPS INT8 10 TFLOPS FP16	6 TOPS	67 TOPS
CPU 配置	4 核 ARM × 1.6 GHz	4 核 Cortex-A76 @2.4GHz 4 核 Cortex-A55 @1.8GHz	6 核 Arm Cortex-A78AE @1.7GHz
GPU 配置	-	Mali-G610 MP4 @850MHz	32 个 Tensor Core 的 1024 核 NVIDIA Ampere 架构 GPU
内存	LPDDR4X 12/8/4GB	LPDDR4/LPDDR4x 最大 16GB	LPDDR5 8GB
视频解码	H.265/H.264 8 路 4K@30fps 或者 40 路 1080p@30fps	H.265 8K@60fps 或者 AV1 4K@60fps 或者 VP9 4K@120fps	H.265 4K@60fps 或者 2×4K@30fps 或者 5×1080p@60fps
视频编码	H.265/H.264 3 路 4K@50fps 或者 20 路 1080p@30fps	H.265 4K@60fps 或者 H.264 1080p@60fps	1080p@30fps (CPU 软件编码)
存储接口	eMMC 5.1 SD 3.0 NVMe	eMMC 5.1 SD 3.0 NVMe	支持外部 NVMe
PCIe 接口	PCIe Gen3	PCIe 3.0	PCIe 3.0 (1×4 + 3×1)
USB 接口	USB 3.0	USB 2.0/3.0/3.1	3×USB 3.2 (10Gbps) 3×USB 2.0
网络接口	支持 1GbE/10GbE	支持 1GbE	1×GbE
摄像头接口	MIPI-CSI x2	MIPI-CSI ×4	8 通道 MIPI CSI-2 最多 4 个摄像头
显示输出	HDMI x2	HDMI 2.1/eDP/LVDS 最多 4 路显示输出 8K 分辨率	DP 1.2/HDMI 1.4 4K@30fps
其他 I/O	UART×5, I2C×4 SPI×2, CAN×4	I2S/PCM/SPDIF	UART×3, SPI×2 I2S×2, I2C×4 CAN×1, PWM, GPIO
典型功耗	25W	≤ 15W	7-10W
工作温度	-20°C ～+103°C	工业级温控支持	-
封装尺寸	MXM 82×60×7mm	BGA 27×27mm	69.6×45mm SO-DIMM 连接器
工艺制程	12nm	8nm	-

在 AI 性能方面, Jetson Orin Nano 凭借其基于 Ampere 架构的 GPU, 提供了三者中最强的 AI 算力; 昇腾 310B 则凭借其专用 AI 加速器, 提供了坚实的中等算力; 而 RK3588 集成的 NPU 则提供了相对基础的 AI 加速能力。

视频处理能力上，三者各有侧重：RK3588 拥有最强的编解码能力，原生支持 8K 视频；昇腾 310B 的优势在于强大的多路视频并发处理能力；而 Jetson Orin Nano 虽然视频解码能力不俗，但其编码任务则需要更多地依赖 CPU 进行。

从功耗效率来看，Jetson Orin Nano 的功耗控制最为出色，展现了优异的能效比; RK3588 则处于中等功耗水平；相比之下，昇腾 310B 为了提供更高的算力和视频路数, 功耗也相对较高。

综合来看，这三款平台因其不同的技术特点，各自面向的应用场景也有所区分：昇腾 310B 更适合工业视觉检测和多路视频分析；RK3588 在高清多媒体播放、工业平板及显示设备中表现出色；而 Jetson Orin Nano 则主要瞄准高性能 AI 推理、边缘计算和机器人等前沿领域。

1.5. 实践任务

1. 基于你的目标场景输出一份协同模式决策表（含放弃理由）。
2. 编写数据生命周期图（ASCII 或 Mermaid）。
3. 实现一个队列背压示例：当处理时延 > 阈值时自动丢弃旧帧。
4. 采集 10 分钟温度与时延数据，绘制相关性（是否热导致抖动）。
5. 设计一份故障降级矩阵并评审可行性。

2. CANN 软件栈核心组件解析

本章系统阐述 Ascend CANN 软件栈的分层结构、模型从框架格式到 OM 的转换原理、转换工具 ATC 的关键参数、OM 文件组织结构、AscendCL (ACL) 推理编程模型、精度与性能验证方法以及工程级质量保障流水线建设。

2.1. CANN 异构计算架构

昇腾 AI 异构计算架构 CANN（Compute Architecture for Neural Networks）是面向基于达芬奇架构的昇腾处理器的全栈软件体系。作为承接主流 AI 框架与通用模型格式、向下对接异构硬件与运行时的中枢层，CANN 构建了覆盖模型表示、转换编译、图优化、调度执行与可观测分析的端到端技术链路，实现语义一致性、性能最优化与诊断可控性的协同统一，从而系统地释放昇腾处理器的算力与能效潜力。

2.1.1. CANN 异构计算与人工智能

异构计算与人工智能是相互促进的关系。深度学习训练与推理负载由高密度线性代数（向量/矩阵乘）、张量算子融合链、非规整控制流（条件/循环）、以及高带宽低延迟的数据搬移共同构成，表现出算子类型多样性、数据复用模式差异与访存/算力不均衡等特征。单一通用处理器在算力利用率、内存层次效率与能效上难以同时达到最优。针对边缘计算场景的典型 SoC（如昇腾 310B、RK3588、Jetson Nano 等）普遍采用低功耗约束下设计的 ARM 通用核，其单核与整体算力在受限功耗窗口内更偏向控制与轻量任务，难以在纯 CPU 路径上满足大规模张量运算所需的高吞吐与低时延深度学习推理需求；因此需要通过集成专用 NPU/加速器进行算子下沉与数据流协同，以弥补通用核在向量化宽度、片上带宽与能效曲线上的结构性不足。异构体系通过在同一系统中组合通用 CPU 与专用 NPU，辅以分层内存与高吞吐互连，依据算子粒度、数据局部性与精度需求执行跨设备调度：密集算子下沉至矩阵/张量专用单元，控制与调度逻辑保留在通用核，数据流经由优化的流水与对齐策略减少跨域搬运成本。结果是在单位功耗与芯片面积约束下提升有效吞吐、降低端到端推理时延并改进能效曲线的可扩展性。该协同范式构成 CANN 设计的理论基础：通过抽象统一语义表示与针对性后端优化，将模型图中不同类别算子映射到最合适执行单元，实现性能、能效与可诊断性三者的统筹平衡。

2.1.2. CANN 与 CUDA

CANN 是面向华为昇腾达芬奇架构昇腾处理器，聚焦深度学习推理与训练的算子图优化、编译调度及可观测性闭环，而 CUDA（Compute Unified Device Architecture）则是针对 NVIDIA GPU 的通用并行计算平台与编程模型，覆盖 GPGPU 与 AI 复合场景。二者均体现 “软硬件协同加速” 范式，但在抽象层次、硬件耦合与生态策略上形成差异化路径。CANN 与 CUDA 二者的共同点主要体现在，它们都是通过软硬件协同，把模型或并行程序转化为底层设备能理解的指令，从而提升运行速度和效率。同时这两种计算平台都提供了内存管理、算子或内核调用、性能分析、调试和日志等工具，帮助开发者更方便地开发和诊断问题。它们都支持自定义算子或内核插件，方便针对特定需求进行优化或替换实现。此外，CANN 与 CUDA 都能通过性能分析、数据导出和分级日志等方式，降低了定位性能瓶颈和对齐精度的难度。在设计理念上，CANN 与 CUDA 展现出显著的差异。CUDA 作为面向广义并行计算与 HPC/图形及 AI 统一生态的平台，强调线程块与网格（Block/Grid）以及 SIMT 并行范式，开发者需显式组织核函数、流（Stream）、事件（Event）和内存管理，实现灵活的并行与资源重叠。而 CANN 则专注于模型图语义，突出图级算子融合、AIPP 预处理下沉和全局内存复用，通过任务列表与算子调度策略有效治理动态形状与内存复用，降低推理工程的不确定性。在精度策略方面，CANN 内置 FP16 与 INT8 的混合精度及校准链路，适应端到端推理与训练的工程质量要求；而 CUDA 虽然基础类型丰富，但精度管理更多依赖于上层库如 cuDNN 和 TensorRT 的组合。硬件结构上，CANN 紧密耦合于达芬奇架构的矩阵单元、向量单元及片上分层内存，优化模型推理的能效与性能；CUDA 则绑定于 NVIDIA 的 SM、Tensor Core 及多级缓存体系，着重提升访存效率与线程调度。生态成熟度上，CUDA 自 2007 年以来积累了庞大的社区与库支持，形成了全球主流的开发者生态；CANN 虽处于快速迭代阶段，但聚焦国产化替代与特定行业应用，推动本地生态建设。编程灵活度方面，CUDA 需要开发者具备底层并行划分与核函数开发能力，提供极高的灵活性；CANN 则通过贴近主流框架的算子开发工具（TBE）与高层接口，降低了入门门槛，但在底层调度上牺牲了一定的自由度。整体而言，二者在抽象层次、硬件耦合与生态策略上形成了各自鲜明的技术路径，反映了其服务对象与应用场景的根本差异。

2.1.3. CANN 的架构

CANN 通过提供分层清晰的编程与运行时结构，以覆盖训练与推理的 “全场景”、贴近主流框架的 “低门槛” 以及面向昇腾硬件深度优化的 “高性能” 为核心特性，支撑用户在昇腾平台上快速构建与部署各类 AI 应用与业务。从整体上看，CANN 可以抽象为一个自上而下递进的五层架构（计算语言接口、计算服务层、计算编译引擎、计算执行引擎与计算基础层），如图2.1所示。各层通过稳定的接口与数据流协议进行解耦协同，在保证语义一致性的前提下最大化发挥底层算力与能效潜力。

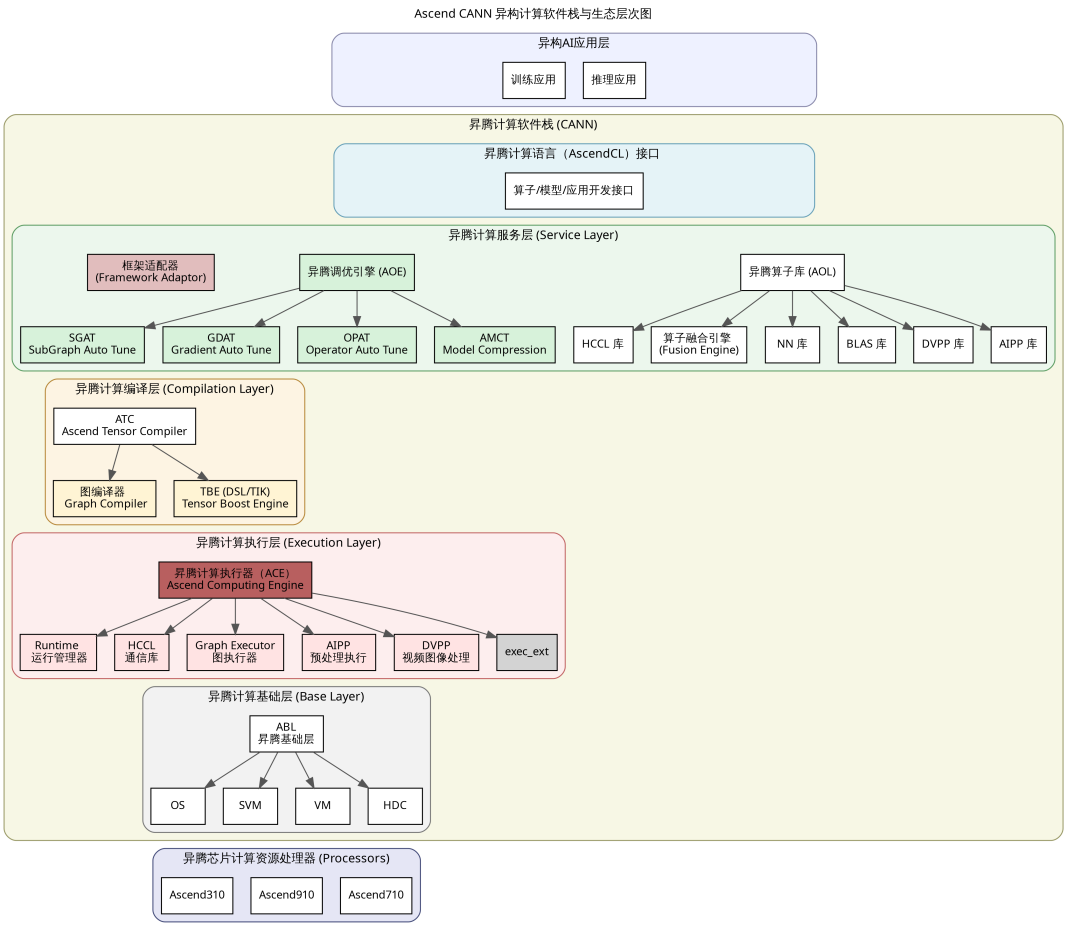

Ascend CANN 异构计算软件栈与生态层次图
异构AI应用层
训练应用
推理应用
昇腾计算软件栈 (CANN)
昇腾计算语言（AscendCL）接口
算子/模型/应用开发接口
昇腾计算服务层 (Service Layer)
框架适配器
(Framework Adaptor)
异腾调优引擎 (AOE)
异腾算子库 (AOL)
SGAT
SubGraph Auto Tune
GDAT
Gradient Auto Tune
OPAT
Operator Auto Tune
AMCT
Model Compression
HCCL 库
算子融合引擎
(Fusion Engine)
NN 库
BLAS 库
DVPP 库
AIPP 库
异腾计算编译层 (Compilation Layer)
ATC
Ascend Tensor Compiler
图编译器
Graph Compiler
TBE (DSL/TIK)
Tensor Boost Engine
异腾计算执行层 (Execution Layer)
昇腾计算执行器（ACE）
Ascend Computing Engine
Runtime
运行管理器
HCCL
通信库
Graph Executor
图执行器
AIPP
预处理执行
DVPP
视频图像处理
exec_ext
异腾计算基础层 (Base Layer)
ABL
昇腾基础层
OS
SVM
VM
HDC
异腾芯片计算资源处理器 (Processors)
Ascend310
Ascend910
Ascend710

图 2.1. CANN 架构图

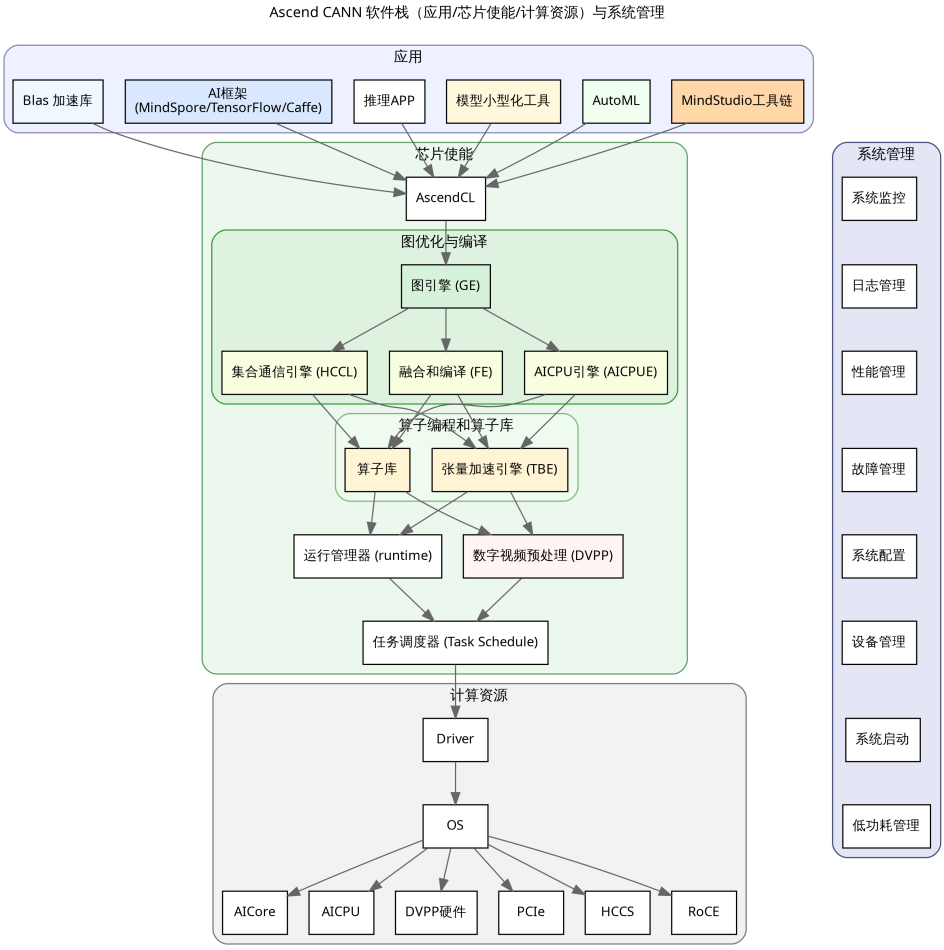


图 2.2. CANN 逻辑图

在分层结构上，CANN 可抽象为五个层次：上层的计算语言接口——AscendCL 接口负责设备与上下文管理、流与内存控制、模型与算子的加载执行，以及媒体与图管理等通用 API，为应用提供稳定的编程入口；其下的昇腾计算服务层汇聚神经网络与线性代数库，承载算子与子图的自动调优、梯度优化与模型压缩，并通过框架适配器降低迁移成本；昇腾计算编译层的编译引擎通过图编译器与 TBE 算子开发支持，把前端计算图转化为在 NPU 可执行的模型与内核，实现图级语义到后端实现的精准映射；下一层的昇腾执行引擎面向运行时，负责模型与算子的调度执行，并内置数字视觉与 AI 预处理、集合通信等能力以提升端到端效率；最下层的昇腾计算基础层提供共享虚拟内存、设备虚拟化与主机—设备通信等底座服务，保证跨设备数据流与资源管理的可靠性与可扩展性。

与此相辅的是三层逻辑架构：应用层承载具体业务与开发者工具，芯片使能层开放解决方案能力并驱动基于计算图的业务流运行，计算资源层则聚焦数据处理与运算执行，形成从业务到硬件的清晰闭环。这个 CANN 的三层逻辑结构如下图2.2所示:

在技术特性上，CANN 通过对计算图的编译与优化，将密集算子与数据流合理分配到异构单元上执行，显著提升吞吐与时延表现，并在能效上取得可工程化的优势；同时提供贴近主流框架的易用接口与完善的工具链，降低入门与迁移门槛，使开发者能够快速完成部署与调优。生态方面，CANN 构建了面向个人、高校、科研与企业的赋能体系，并与开源社区协同，支持多种框架与推理引擎在异构硬件上的高效运行。依托这些能力，开发者可以深入调用运行时与图编译能力，释放底层硬件潜力，在性能与成本维度形成差异化竞争力。

在工程实践中，CANN 的核心价值可以概括为在 “模型—编译—执行—诊断” 的完整链路上实现软硬件协同与语义稳定：通过构建对主流深度学习框架高度兼容的前端接口，最大限度降低模型迁移与语义对齐成本，在图级优化阶段则依托算子融合、内存复用以及混合精度策略的协同设计，系统性提升吞吐能力、推理时延以及功耗效率；与此同时，借助自定义算子机制与实现优先级调度策略，使新结构能够快速接入并在多实现版本之间灵活切换，从而在不同硬件代际与不同业务场景下充分挖掘底层计算资源的潜力，并在运行时通过分级日志、Profiling 时间线与精度 Dump 等手段构建起闭环的可观测体系。在大规模、复杂模型的工程化路径中，首先需要完成面向硬件架构的感知建模，在导出阶段显式固化张量形状与算子语义，为后续编译与部署奠定稳定的语义基础；随后在模型转换阶段，以 “转换即优化” 为原则，围绕 precision_mode、op_select_implmode 与 AIPP 等关键配置项进行显式调优，使图优化与算子选型在编译期前置完成，其中精度策略以 FP16 为主，并辅以具有代表性的校准数据集对 INT8 量化误差进行严格控制，以在性能与精度之间取得可工程化的平衡。对于动态形状问题，可以采用分桶与 Padding 结合的方式，并在必要时生成多份 OM 模型，以在一定范围内换取更可控的峰值资源占用与时延方差；在内存与带宽层面，则通过将预处理逻辑尽可能下沉到设备侧、合并数据搬移操作，并配合 Pinned 内存与多 Stream 并行传输技术，显著降低 H2D/D2H 的时间占比，从系统视角优化整体流水线的调度效率。针对在实际 Profiling 中暴露出的性能瓶颈算子，需要开展有针对性的算子重写与图重构，并充分利用实现优先级机制在不同 Kernel 之间进行策略化选择，以进一步榨取底层硬件的计算与访存潜力；最终，通过围绕 “转换—精度对齐—Benchmark—回归监测” 构建自动化流水线，将模型转换、性能评估与精度验证纳入统一的持续反馈闭环，在 Ascend 310B 等专用加速平台上沉淀出可复用的优化经验与差异化性能优势，不仅在算力利用率与综合成本上形成工程级竞争力，也为大模型与行业应用场景的规模化加速落地提供坚实的基础设施支撑。

2.1.4. CANN 的快速安装

本节给出 CANN 软件的极速安装示例。若需更完整的操作指导，请按实际环境选择对应安装场景并参考官方说明，推荐优先使用离线安装。

安装前准备

- 驱动与固件检查：在目标设备上执行 `npu-smi info`，如果能够正常显示设备信息，则表明 NPU 驱动和固件已经正确安装。对于昇腾 310B 产品或开发板，系统一般会预装所需的驱动和固件，无需额外手动配置。
- 环境准备：推荐采用离线安装方式，安装前需确保系统已准备好 Python 环境并可使用 `pip3`，当前 CANN 支持 Python 3.7.x 至 3.11.4 版本。对于昇腾 310B 开发板，通常已预装 Miniconda 和 `pip3`，可直接满足依赖，无需额外配置。
- 安装介质获取：在安装前，需先从华为昇腾官网下载所需的 CANN 软件包，并将其上传到目标设备的可访问路径。以 CANN 8.3.RC1 版本为例，可通过官网 获取相应安装包，其他版本可在页面切换选择。请确保下载的 CANN Toolkit 和 CANN Kernels 版本号完全一致（即同一发行号），以避免兼容性问题。推荐优先选择.run 格式的安装包，且昇腾 310B 开发板为 AArch64 架构，需下载 aarch64 对应的包。例如 8.3.RC1 版本包括 Ascend-cann-toolkit_8.3.RC1_linux-aarch64.run 和 Ascend-cann-kernels-310b_8.3.RC1_linux-aarch64.run。如果实际下载的版本号不同，请将文件名中的版本号替换为对应的实际版本。

安装命令与依赖说明

安装 CANN 可使用 root 或默认账户 HwHiAiUser。推荐以 root 安装，不推荐用其他用户进行安装，因为容易发生权限问题。下面以 root 用户为例，详细介绍 CANN 的安装过程：

- 切换到 root：

```
su -
# 输入 root 密码后继续执行安装
```

- 命令示例, 以 root 执行：

```
./Ascend-cann-toolkit_8.3.RC1_linux-aarch64.run --install
```

- 安装 Toolkit 开发套件:

```
chmod +x Ascend-cann-toolkit_8.3.RC1_linux-aarch64.run
./Ascend-cann-toolkit_8.3.RC1_linux-aarch64.run --install
```

整个安装过程大概会持续十几分钟，甚至更长，请耐心等待。

- 配置环境变量:

```
source /usr/local/Ascend/ascend-toolkit/set_env.sh
```

- 安装 Kernels 算子包（310B 平台）：

```
chmod +x Ascend-cann-kernels-310b_8.3.RC1_linux-aarch64.run
./Ascend-cann-kernels-310b_8.3.RC1_linux-aarch64.run --install
```

- 安装业务运行时所需的 Python 第三方库:

```
pip3 install attrs cython 'numpy>=1.19.2,<=1.24.0' \
  decorator sympy cffi pyyaml pathlib2 \
  psutil protobuf==3.20.0 scipy requests absl-py
```

- 说明: 依赖的版本范围已经覆盖了大多数常见环境，如果遇到依赖冲突，请根据实际的 Python 或操作系统版本进行相应调整。安装完成后，建议将 CANN 的环境变量配置追加到 `~/.bashrc` 文件中，以便每次登录时自动加载。需要注意的是，昇腾 310B 开发板的系统镜像通常已经预置了这些环境变量配置，无需手动设置。

2.2. ATC 模型转换详解

2.2.1. ATC 工具介绍

昇腾张量编译器（Ascend Tensor Compiler，ATC）是 CANN 异构计算体系中的模型转换组件，支持将主流开源框架导出的网络模型以及基于 Ascend IR 的单算子描述文件（JSON）编译为昇腾 AI 处理器可执行的离线模型（.om）。如图下图所示，ATC 在转换过程中会执行算子调度优化、权重重排与内存复用等关键步骤，对原始深度学习模型进行面向部署场景的系统化调优，从而在昇腾硬件上实现高吞吐、低时延的高效执行。

从上面的流程图我们可以看到，ATC 工具聚焦模型到设备可执行体的 “转换即优化” 闭环：一方面，它能够将开源框架导出的网络模型解析为中间态 IR Graph，经由图准备、拆分与融合、形状推理、内存复用与算子选型等编译步骤，生成适配昇腾处理器的离线模型（OM），并在板端通过 AscendCL 接口加载执行，从语义到性能实现端到端落地；另一方面，ATC 也支持基于 Ascend IR 的单算子 JSON 直接编译为离线 OM，用于在设备侧进行算子级功能验证与精度对齐，帮助开发者快速定位与迭代关键算子实现，在图级与算子级两条路径上形成统一的工程化编译能力。

onnx 格式介绍

ONNX（Open Neural Network Exchange）是一种针对深度学习所设计的开放式文件格式，用于存储训练好的模型。它使得不同的人工智能框架（如 PyTorch、TensorFlow、mindspore 等）可以采用相同格式存储模型数据并交互。ONNX 的规范包含计算图模型的定义，以及内置运算符的定义和标准数据类型。

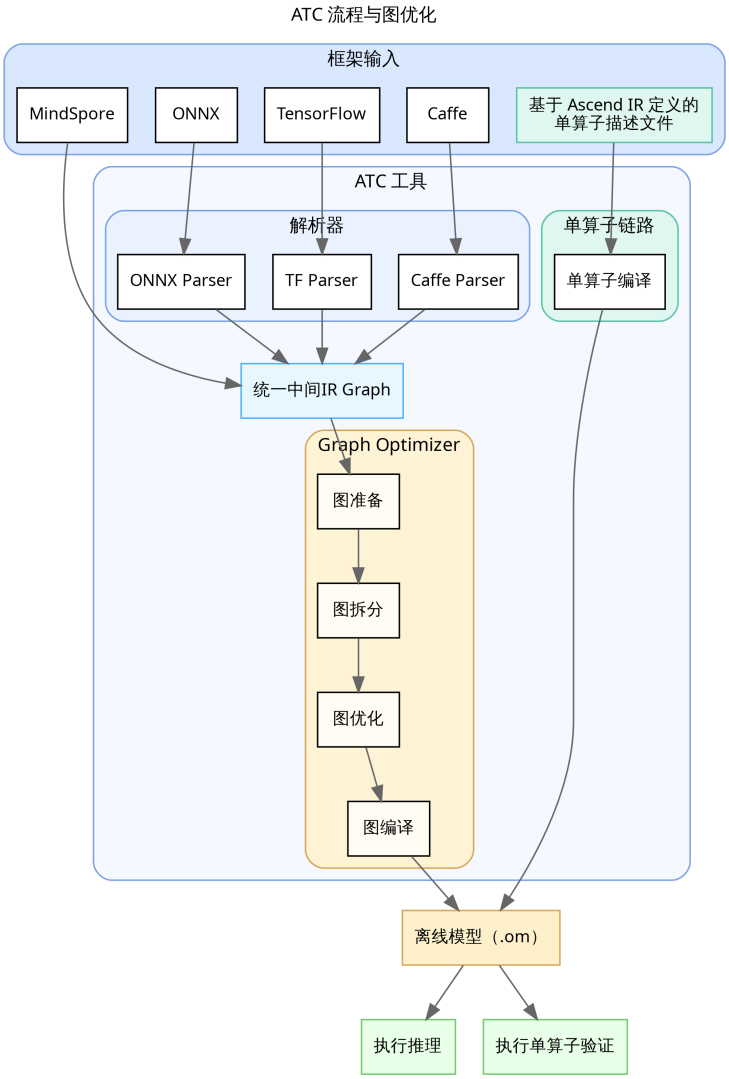

ATC 流程与图优化
框架输入
MindSpore
ONNX
TensorFlow
Caffe
基于 Ascend IR 定义的
单算子描述文件
ATC 工具
解析器
ONNX Parser
TF Parser
Caffe Parser
单算子链路
单算子编译
统一中间IR Graph
Graph Optimizer
图准备
图拆分
图优化
图编译
离线模型（.om）
执行推理
执行单算子验证

图 2.3. ATC 框架图

ONNX 的核心价值在于解决了 AI 生态系统中 “碎片化” 的问题。在没有 ONNX 之前，开发者如果想将一个在 PyTorch 中训练好的模型部署到移动端推理引擎上，通常需要进行繁琐的代码重写或复杂的格式转换。ONNX 提供了一种中间表达（Intermediate Representation, IR），充当了不同框架之间的 “通用语”。

一个标准的 ONNX 模型文件（通常以`.onnx`为后缀）主要包含以下几个部分：

1. **Model Proto**：顶层结构，包含模型的元数据（如版本信息、生产者信息）和计算图（Graph）。
2. **Graph Proto**：描述了模型的计算逻辑，由一系列节点（Node）、输入（Input）、输出（Output）和初始化器（Initializer，即权重数据）组成。
3. **Node Proto**：代表计算图中的一个操作（Operator），如 Conv、Relu、Add 等。每个节点包含操作类型、输入输出张量的名称以及属性（Attribute，如卷积的步长、填充等）。
4. **Tensor Proto**：用于存储权重数据或常量的具体数值和形状信息。

ATC 对 ONNX 的支持情况虽然 ONNX 旨在成为通用的 AI 模型交换标准，但 CANN 的 ATC 工具并非支持 ONNX 规范中的所有算子和特性。ATC 对 ONNX 的支持主要集中在 CV（计算机视觉）和 NLP（自然语言处理）领域的常用算子，对于一些较新或较冷门的算子，可能会出现不支持的情况。

具体来说，ATC 对 ONNX 的支持限制主要体现在以下几个方面：

1. **算子覆盖率**：ATC 支持绝大多数主流的 CNN 和 Transformer 类算子（如 Conv, MatMul, Relu, Softmax 等）。但对于部分非标准或特定框架自定义的算子（Custom Ops），ATC 无法直接解析，需要开发者通过 TBE（Tensor Boost Engine）开发自定义算子并注册到 CANN 中。
2. **动态控制流**：ONNX 中的动态控制流（如`If`, `Loop`, `Scan`）在静态图编译中处理较为复杂。虽然 ATC 支持部分控制流算子，但在某些复杂嵌套场景下可能会导致编译失败或性能下降，通常建议在导出模型时尽量将控制流展开（Unroll）或转为静态图结构。
3. **数据类型限制**：虽然 ONNX 支持多种数据类型（如 Double, Int64 等），但昇腾 AI 处理器主要针对 FP16 和 INT8 进行了硬件加速优化。对于 FP64（Double）类型，ATC 通常不支持或需要强制转为 FP32/FP16 处理；对于 Int64 类型的索引或计数，部分算子可能要求转为 Int32。
4. **动态 Shape 支持**：虽然 ATC 支持动态 Batch 和动态分辨率，但对于模型内部中间层出现的完全动态且无法推导的 Shape，可能会导致编译报错。

因此，在进行模型转换前，建议开发者查阅华为昇腾社区发布的《CANN 算子清单》，确认模型中使用的 ONNX 算子版本（Opset Version）是否在当前 CANN 版本的支持范围内。如果遇到不支持的算子，通常的解决路径包括：修改模型结构以替换为等价的支持

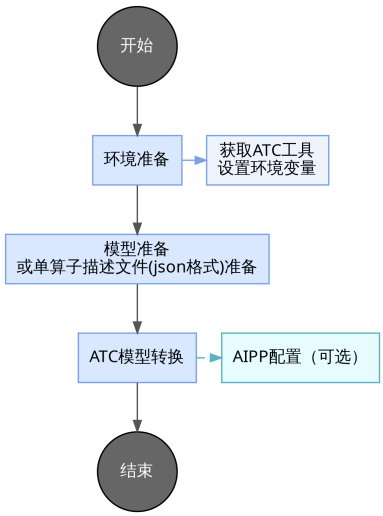


图 2.4. ATC 流程图

算子、在 ONNX 导出阶段进行算子简化（使用`onnx-simplifier`工具）、或者开发自定义算子。

2.2.2. ATC 工具使用流程

使用 ATC 工具进行模型转换的使用流程如下图所示：

在开始模型转换之前，首先需要在开发环境中安装与 CANN 软件包版本相匹配的版本，并确保 ATC 可执行文件的路径可用。接下来，准备待转换的模型文件或基于 Ascend IR 的单算子 JSON，并将其上传到开发环境中可访问的目录。最后，使用 ATC 执行转换，并根据实际业务需求和输入规范配置相关参数。如果需要将预处理步骤（如色彩空间转换、归一化和尺度调整）下沉到设备侧，可以同时提供并启用 AIPP（Artificial Intelligence Pre-Processing）配置。AIPP 是昇腾处理器内置的硬件级图像预处理模块，负责将上游（如 DVPP）输出的对齐后 YUV420SP 图像在设备侧完成色域转换（如 YUV 图像格式转换为 RGB 或者 BGR 图像格式）、归一化（减均值/乘系数/尺度放大）与抠图（在指定起始点裁剪到模型输入尺寸），从而把原始图像规范化为模型所需的输入格式与数值范围。由于昇腾 310B 在推理及训练中常以 DVPP 输出 YUV420SP 图片格式，这种图像格式是有损图像颜色编码格式，常用为 YUV420SP_UV、YUV420SP_VU 两种格式，不直接提供 RGB 图片。AIPP 能将 YUV420SP 类型图像无缝转化为模型期望的 RGB/BGR 图像格式，并在同一数据流中完成裁剪与数值处理，避免将预处理放在 CPU 侧导致的多次拷贝与额外时延，提升端到端吞吐与能效。

模型转换-以 ResNet50 为例

ResNet-50 由多级残差单元堆叠形成 50 层卷积网络，通过跨层跳连缓解深层退化与梯度消失,使其在 ImageNet 等大规模数据集上具备稳定的特征表达与分类精度,因此常被选作各类视觉任务的通用骨干。要在昇腾 310B 上部署该模型, 需要借助 ATC 将 ONNX 版本转换为 OM,可参考华为昇腾官方示例仓库(https://github.com/Ascend/samples/tree/master/infer
为方便快速复现，可按以下步骤从零搭建工程:

1. 在昇腾 310B 的 `Documents` 目录创建 `sample_resnet_quick_start`，并划分 `data`、`model`、`out`、`src` 四个子目录用于存放测试图片、模型、脚本与源码:

```
cd ~/Documents
mkdir -p sample_resnet_quick_start/{data,model,out,src}
cd sample_resnet_quick_start
```

2. 下载示例图片至 `data` 目录:

```
cd ~/Documents/sample_resnet_quick_start/data
wget https://obs-9be7.obs.cn-east-2.myhuaweicloud.com/models/aclsample/dog1_1024_683.jpg
```

3. 进入 `model` 目录获取 ResNet-50 ONNX 模型:

```
cd ~/Documents/sample_resnet_quick_start/model
wget https://obs-9be7.obs.cn-east-2.myhuaweicloud.com/003_Atc_Models/resnet50/resnet50.
    ↪ onnx
```

4. 设置编译并行度后执行典型 ATC 命令，生成适配昇腾 310B4 的 OM 文件:

```
export TE_PARALLEL_COMPILER=1
export MAX_COMPILE_CORE_NUMBER=1
atc \
  --model=resnet50.onnx \
  --framework=5 \
  --output=resnet50 \
  --input_shape="actual_input_1:1,3,224,224" \
  --soc_version=Ascend310B4
```

关键参数说明

参数	作用	注意事项
`--framework`	指定原始模型框架	0=Caffe，1=MindSpore，3=TensorFlow，5=ONNX，需与导出框架一致

参数	作用	注意事项
`--input_shape`	定义静态输入 Shape	按 `"name:n,c,h,w"` 写法，字符串需加引号；动态改用区间或配合动态参数
`--dynamic_batch_size`	设置可选 Batch 列表	以 `"1,4,8"` 形式填写；与同一输入的完全静态 shape 互斥
`--dynamic_image_size`	配置多分辨率输入	`"h1,w1;h2,w2"`，常用于多尺度检测；需与模型实际支持一致
`--precision_mode`	控制混合精度策略	常用值：`force_fp16`、`allow_mix_precision`、`allow_fp32_to_fp16`，需兼顾精度
`--soc_version`	选择目标芯片	例：`Ascend310B4`，可通过 `npu-smi info` 查询；310B/P 区分清楚
`--insert_op_conf`	下沉 AIPP/自定义算子	指定 JSON/YAML，支持色域转换、归一化等预处理
`--op_select_implmode`	算子实现优先级	支持 `high_performance`(默认)、`high_precision` 等，可配合 `--optypelist_for_implmode`
`--input_format`	声明输入数据排布	如 `NCHW`、`NHWC`；需与模型导出及 `--input_shape` 保持一致
`--output_type`	重指定输出 dtype	可设全局 `FP16`/`FP32`，或按 `node:idx:FP32` 精细配置，便于后处理
`--enable_small_channel`	小通道优化开关	取值 0/1，轻量网络或低通道数场景启用可减时延
`--model`	指定待转换模型文件	支持 `.onnx`、`.pb`、`.prototxt` 等类型，路径需可访问
`--output`	设置 OM 输出前缀	不需写后缀；默认位置在当前目录，可结合 `--output_type` 使用
`--dynamic_dims`	定义多维动态组合	`"d1_1,d1_2;d2_1,d2_2"` 格式；用于多输入或非 Batch 维变动
`--enable_single_stream`	强制单流执行	`true/false`；在推理序列化或调试稳定性时启用
`--dump_mode`	导出模型结构 JSON	配合 `--mode=1` 使用，`1` 开启；便于排查 Shape 与算子信息

如果对于参数有任何的疑惑，可以通过命令查询 atc 的参数作用：

```
atc --help
```

成果转换模型后，我们可以看到命令窗口有如下输出：

```
ATC start working now, please wait for a moment.
...
ATC run success, welcome to the next use.
```

注意事项

1. 在昇腾 310B 上将 ONNX 模型转换为 OM 时若出现 `BrokenPipeError: [Errno 32] Broken pipe`，通常是编译阶段内存不足导致进程被系统终止。可参照以下两类方案缓解：

- **扩展可用内存** 通过创建 Swap 交换文件临时扩充虚拟内存，以缓解编译期间的内存压力：

```
dd if=/dev/zero of=/swapfile bs=1M count=8192
chmod 600 /swapfile
mkswap /swapfile
swapon /swapfile
```

若需永久生效，请在 `/etc/fstab` 文件中追加以下配置：

```
/swapfile swap swap defaults 0 0
```

注意：如果设备使用 TF (MicroSD) 卡作为主要存储介质，不建议在 TF 卡上频繁使用 Swap。Swap 分区涉及大量高频读写操作，会极大加速闪存磨损，导致 TF 卡寿命缩短或损坏。建议仅在必要时临时开启，或使用 SSD 等更耐用的存储介质。

- **降低编译资源占用**

```
export TE_PARALLEL_COMPILER=1      # 限制算子最大并行编译进程数
export MAX_COMPILE_CORE_NUMBER=1   # 限制图编译占用的 CPU 核数
```

方便起见，我们可以采用第二种方案，可以有效解决内存不足的问题。

2. 若无法确认当前设备的 `soc_version`，可在安装驱动的昇腾 310B 上执行 `npu-smi info` 查询，示例如下：

```
npu-smi 23.0.0  Version: 23.0.0
+-----------+--------+--------+-------+------------------+
| NPU Name  | Health | Power  | Temp  | Hugepages-Usage  |
| Chip      | Bus-Id | AICore | Mem   |                  |
+===========+========+========+=======+==================+
| 0 310B4   | Alarm  | 0.0W   | 58C   | 15 / 15          |
| 0 0       | NA     | 0%     | 2500/7545MB            |
+-----------+--------+--------+-------+------------------+
```

请在查询结果的 `Name` 值前追加 `Ascend` 前缀：若 `Name` 为 `310B4`，则需配置 `soc_version=Ascend310B4`。

3. 模型转换完成后会生成一份 JSON 文件，用作 ATC 编译日志的结构化摘要，集中记录当前会话（session_and_graph_id=0_0）内图级与 UB 级融合规则的触发统计。具体而言，graph_fusion 字段逐条给出各图融合 Pass 的匹配次数与实际生效次数

（match_times 与 effect_times），可据此识别潜在的优化空档；ub_fusion 字段在上述统计基础上新增 repository_hit_times，用以衡量算子库模板的命中情况。实践中，可重点关注 effect_times 低于 match_times 的 Pass，分析是否因算子属性、精度模式或实现优先级等因素造成融合未落地，并据此调整 ATC 参数或模型图结构，以复现并提升性能优化效果。

2.2.3. 模型推理工具 msame 工具概览

在完成 ATC 转换得到 `.om` 模型后，最快验证部署链路是否畅通的方式就是调用华为昇腾提供的模型推理工具 msame。该工具封装了 OM 加载、设备内存初始化、输入二进制数据映射以及推理调度等通用流程，无需额外编写 AscendCL 程序即可直接对接 ATC 输出的模型，只需准备好与模型输入规格匹配的 `.bin` 文件和基础运行时环境，即可在命令行下完成端到端推理验证。通过 msame，开发者能够快速检查模型是否成功落地、输出是否合理，并以最小成本排除输入格式或环境配置类问题，为后续集成到自研应用或高层框架之前提供低门槛的功能性回归手段。

msame 的设计目标是将 OM 模型的快速验证能力标准化，覆盖单输入、多输入和循环执行等常见推理形态，支持在相同模型与不同输入数据组合之间做对比试验，同时允许选择 TXT 或 BIN 格式导出结果，以便于人工审查或自动化脚本解析。在已正确安装 CANN Runtime 的环境中，msame 会自动完成与设备的上下文绑定、内存申请和任务提交，将推理过程抽象成简单的命令行参数；这不仅适用于初步确认模型可运行，也可用于小批量性能评估、数据一致性校验以及在定位性能或精度问题时的基准对照，从而在模型转换与实际业务部署之间搭建起轻量而高效的桥梁。

msame 工具的获取与构建

1. 通过 `git clone https://gitee.com/ascend/tools.git` 拉取仓库，或下载 ZIP 包后解压到开发环境。
2. 进入 `msame` 子目录：

```
cd ./tools/msame
```

3. 配置 CANN 环境变量（以下路径请按实际安装位置调整）：

```
export DDK_PATH=/usr/local/Ascend/ascend-toolkit/latest
export NPU_HOST_LIB=/usr/local/Ascend/ascend-toolkit/latest/runtime/lib64/stub
```

4. 运行构建脚本生成可执行文件：

```
# 若脚本缺少执行权限，请先执行 chmod +x build.sh
```

```
./build.sh g++
```

如果在执行时遇到 `Permission denied` 报错，说明脚本文件缺少可执行权限，需先运行 `chmod +x build.sh` 进行赋权，随后再次执行编译命令。

5. 构建成功后会在 `out` 目录生成二进制文件 `msame`,将其复制到工程目录 `sample_resnet_quick_start` ↪ 中备用。

- 常用参数速览

参数	说明
`--model`	OM 文件路径
`--input`	输入 `bin` 文件或目录，支持多输入
`--output`	推理结果存放目录
`--outfmt`	输出格式：`TXT` / `BIN`
`--loop`	重复推理次数（1–100）
`--profiler` / `--dump`	性能分析或 Dump，互斥
`--device`	设备 ID，默认 0
`--dymBatch` / `--dymHW` / `--dymDims` / `--dymShape`	对应 ATC 动态配置的实际取值
`--outputSize`	动态 Shape 场景下手动申请输出内存
`--debug`	打印模型描述信息
`--help`	查看全部参数

- 使用注意运行 msame 工具时，需确保当前账号拥有目录的执行和写入权限。对于部分动态 Shape 场景，建议选择 BIN 格式输出而非 TXT，以保证兼容性。配置 NPU_HOST_LIB 时应指向 runtime/lib64/stub 目录，并在运行阶段通过 LD_LIBRARY_PATH 正确链接所需依赖库。此外，profiler 和 dump 功能不可同时启用；如涉及动态 Shape，需同步设置合适的输出内存大小以确保推理顺利进行。

图片预处理

由于 msame 工具不具备图像预处理能力，输入必须是已转换好的 `.bin` 文件。因为之前在模型转换时未显式指定精度，ResNet50 默认接受 FP32 格式的输入；同样地，未设置输入内存布局时 ATC 会采用默认的 NCHW。NCHW 表示批次 N、通道 C、空间高度 H 与宽度 W，意味着同一张图片的三个颜色通道会按通道优先的顺序依次排布，再展开到空间维度。这与 TensorFlow 常见的 NHWC（通道在最后）相对。`sampleResnetQuickStart.py` 中的 `image_rgb.transpose([2, 0, 1])` 正是将 OpenCV 读取的 HWC 布局转换为 CHW，并配合外层批次维得到符合要求的 NCHW。是否必须使用 NCHW 取决于模型导出时的约定。PyTorch 与 CANN 默认使用 NCHW，因此常见的 `resnet50.onnx`/`resnet50.om` 都在此布局下训练与推理。只要导出模型为 NCHW，推理阶段同样必须以 NCHW 喂入数据，否则通道错位会导致结果异常。当然，也可以导出为 NHWC，只需在模型与预处理两端同时调整即可。由于刚才我们从华为云哪里获取的测试图片 “dog1_1024_683.jpg” 如下：

图 2.5. resnet 测试图片

由于这个图片是 jpg 格式的，因此为了可以利用这个图片推理，我们第一步需要对图片进行预处理，具体过程如何：1. 在 src 文件目录创建一个预处理的 python 文件“make_bin_resnet224_float32.py” 如下：

```python
from PIL import Image
import numpy as np

# 加载图像并转换为 RGB
img = Image.open('data/dog1_1024_683.jpg').convert('RGB')
# 使用双线性插值缩放到 256x256
img = img.resize((256, 256), Image.BILINEAR)
# 计算 224x224 中心裁剪的坐标
left = (256 - 224) // 2
top = (256 - 224) // 2
# 执行中心裁剪
img = img.crop((left, top, left + 224, top + 224))

# 转换为 [0, 1] 范围的 float32 数组
arr = np.array(img).astype(np.float32) / 255.0
# 定义归一化常量
mean = np.array([0.485, 0.456, 0.406], dtype=np.float32)
std = np.array([0.229, 0.224, 0.225], dtype=np.float32)
# 按通道进行归一化
arr = (arr - mean) / std

# 调整为 NCHW 格式并添加批次维度
arr = arr.transpose(2, 0, 1)[None, :, :, :]
# 保存为二进制文件
arr.tofile('data/dog1_224_float32.bin')
# 打印缓冲区大小（字节数）
print('bytes:', arr.nbytes)
```

2. 在 shell 中运行这个 python 文件：

```
python src/make_bin_resnet224_float32.py
```

该脚本在导出 .bin 输入前会完成：将示例图像缩放到 256×256 后再做 224×224 中心裁剪；把像素转换为 NCHW 排布的 float32 缓冲区并写入文件；整个过程中先除以 255 将数值归一化到 [0,1]，再按通道减去 [0.485, 0.456, 0.406]、除以 [0.229, 0.224, 0.225] 标准差，以对齐 ResNet 在 ImageNet 上的训练预处理，从而避免因均值或方差失配导致的输出偏移，最终在data文件夹内得到 dog1_224_float32.bin。

3. 快速运行——单输入文件运行下面这个命令：

```
./msame --model "./model/resnet50.om" --input "./data/dog1_224_float32.bin" --output "./
    ↪ out" --outfmt BIN
```

运行成功后，我们会得到以下的信息：

```
[INFO] acl init success
[INFO] open device 0 success
[INFO] create context success
[INFO] create stream success
[INFO] get run mode success
[INFO] load model ./model/resnet50.om success
[INFO] create model description success
[INFO] get input dynamic gear count success
[INFO] create model output success
./out/20251213_11_6_52_564388
[INFO] start to process file:./data/dog1_224_float32.bin
[INFO] model execute success
Inference time: 6.811ms
[INFO] get max dynamic batch size success
[INFO] output data success
Inference average time: 6.811000 ms
[INFO] destroy model input success
[INFO] unload model success, model Id is 1
[INFO] pid: 98299 Execute sample success
[INFO] end to destroy stream
[INFO] end to destroy context
[INFO] end to reset device is 0
[INFO] end to finalize acl
```

- 该日志显示推理流程已完整走通：ACL 初始化、设备上下文、模型加载、输入输出申请均返回 success，Inference time/Inference average time 给出单次与平均耗时，路径 ./out/20251213_11_6_52_564388 指向本次推理结果目录。该输出目录遵循 “YYYYMMDD_H_M_S_microsec” 命名：20251213 表示日期（2025-12-13），11_6_52 为时分秒，564388 为微秒级序列号，用于区分同日多次推理结果。

- 若选择 `--outfmt BIN`，可用 `numpy.fromfile('./out/.../resnet50_output_0.bin', dtype=np.float32).reshape(1, 1000)` 读取并执行 `np.argsort` 获取 TopK；使用 `softmax` 还原概率后结合 ImageNet label 映射即可解读分类结果。
- 若选择 `--outfmt TXT`，直接 `cat ./out/.../resnet50_output_0.txt | head` 快速查看前若干输出值，同样可以配合脚本解析为 TopK 概率。

4. 后处理
   无论 msame 的 `--outfmt` 选择 `BIN` 还是 `TXT`，得到的都是形如 `1×1000` 的分类 logits，需要结合 ImageNet 标签映射再做一次后处理才能输出可读结果。以下示例脚本演示如何解析 `BIN` 文件、执行 softmax 并打印 Top-K 概率。

- 先下载 `imagenet_class_index.json`（可来自 Kaggle 或 GitHub）放到 `src` 目录，然后创建 `src/postprocess_resnet50.py`：

```python
# src/postprocess_resnet50.py
import os
import argparse
import json
import numpy as np

parser = argparse.ArgumentParser(description="将ResNet50的 BIN 输出解码为ImageNet标签。")
parser.add_argument("--bin", required=True, help="msame 输出BIN文件的路径。")
parser.add_argument("--topk", type=int, default=5, help="需要打印的最高置信度数量。")
args = parser.parse_args()

logits = np.fromfile(args.bin, dtype=np.float32).reshape(1, -1)
probs = np.exp(logits - logits.max(axis=-1, keepdims=True))
probs = probs / probs.sum(axis=-1, keepdims=True)
probs = probs[0]

labels_path = os.path.join("src", "imagenet_class_index.json")
if not os.path.exists(labels_path):
    raise FileNotFoundError("请先将imagenet_class_index.json下载到src目录。")

with open(labels_path, "r", encoding="utf-8") as f:
    data = json.load(f)
labels = [data[str(i)][1] for i in range(len(data))]

topk = np.argsort(probs)[::-1][:args.topk]
print(f"Decoded {args.bin} (Top-{args.topk})")
for rank, idx in enumerate(topk, start=1):
    label = labels[idx] if idx < len(labels) else f"cls_{idx}"
    print(f"{rank:>2d}: class={idx:<4d} prob={probs[idx]:.6f} label={label}")
```

- 执行脚本：

```
python ./src/postprocess_resnet50.py --bin out/20251213_11_6_52_564388/resnet50_output_0.
    ↪ bin
```

- 示例输出：

```
Decoded out/20251213_11_6_52_564388/resnet50_output_0.bin (Top-5)
 1: class=162  prob=0.964885 label=beagle
 2: class=161  prob=0.023230 label=basset
 3: class=166  prob=0.006107 label=Walker_hound
 4: class=167  prob=0.004985 label=English_foxhound
 5: class=164  prob=0.000334 label=bluetick
```

2.3. 章节小结

本章从宏观分层、转换编译、OM 结构、ACL 编程、性能与精度保障、调试工具、自动化流水线到动态 Shape 与安全实践建立了闭环。掌握这些内容后即可进入后续 “边缘系统架构与部署实践” 章节，扩展到多模型、多进程及系统级优化。

2.4. 实践任务

1. 任选一个公开 ONNX 分类模型（如 ResNet50）完成 ATC 转换, 提交: 命令 + atc.log。
2. 以 C 或 Python 实现最小推理程序，输出前 5 TopK 结果与 softmax 概率。
3. 编写对齐脚本比较 50 张图片 ONNX vs OM 输出差异（报告 L1/Top1 差异）。
4. 收集 Profiling Timeline，列出前 3 耗时算子类型及优化建议。
5. 输出 `signature.json`、`metrics.json`、`conversion_meta.yaml` 并归档。

3. 昇腾 PyTorch 扩展迁移基础

PyTorch 是以动态计算图著称的深度学习框架，核心由灵活的张量运算与自动微分系统构成。eager execution 让调试、可视化和原型迭代更直观，而 TorchScript 则提供图模式部署以兼顾性能与可移植性。配合丰富的 TorchVision、TorchAudio、TorchText 等生态包，开发者可以在视觉、语音、自然语言处理等任务上快速搭建端到端方案。

Ascend Extension for PyTorch 是华为昇腾为 PyTorch 用户提供的深度适配插件，使 PyTorch 能无缝调用昇腾 AI 处理器算力，沿用原生接口并针对算子、通信与调度做深度优化。项目源码可在官方仓库获取，更多动态可关注昇腾社区。

虽然当前已有 CANN、MindSpore 等优秀深度学习框架，但在 PyTorch 广泛应用的背景下，先基于 PyTorch 学习并借助昇腾插件迁移至昇腾 310B，既能降低学习门槛，又可减少适配成本、缩短开发周期。

由于 torch_npu 对昇腾 310B 的支持仍有限，适配与迁移存在诸多问题，因此这里只做 PyTorch 在 310B 上的基础示例与简要介绍。

3.1. 架构速览

整体架构可概括为 “PyTorch 前端 + torch_npu 中间层 + 昇腾算子后端”。图 1 展示了其分层设计：PyTorch 前端将动态图、自动微分、优化器等能力暴露给开发者；`torch_npu` 负责对接 CANN 算子库、通信库以及运行时；底层由昇腾 AI 处理器提供硬件加速。理解这一层次关系有助于在故障排查时迅速定位问题。

3.2. 核心能力纵览

插件在多个维度增强了 PyTorch 在昇腾平台的体验：

- **算子与生态适配**：基于开源 PyTorch 实现对昇腾 AI 处理器的深度定制，持续补齐主流算子与第三方库，保证常见模型脚本可直接迁移。
- **训练基础设施**：保持动态图、自动微分、优化器与 Profiling 等特性，与原生 PyTorch 保持一致，降低学习成本。
- **自定义算子扩展**：为算子开发者提供接口，可在 PyTorch 框架内快速集成自研算子。

- **分布式通信**：内置 Broadcast、AllReduce 等集合通信原语，覆盖单机多卡与多机多卡场景。
- **推理链路**：模型可导出 ONNX，再借助离线转换工具生成适配昇腾的推理模型，便于训练—推理一体化交付。

3.3. PyTorch 安装教程

3.3.1. CANN 环境准备

在昇腾 310B 开发板上安装 PyTorch 之前，必须必须安装匹配 CANN 版本的驱动与固件。如果没有安装 CANN 相关的工具包与驱动，请参考《CANN 软件安装指南》或者本教程的第二章进行离线安装。如果已安装 CANN 相关工具包与驱动，可按执行以下命令获取版本信息:

```
cat /usr/local/Ascend/ascend-toolkit/latest/aarch64-linux/ascend_toolkit_install.info
```

例如刚刚完成 CANN 8.3.RC1 版本的安装后，会输出以下信息:

```
package_name=Ascend-cann-toolkit
version=8.3.RC1
innerversion=V100R001C23SPC001B235
compatible_version=[V100R001C15],[V100R001C18],[V100R001C19],[V100R001C20],[V100R001C21],[
    ↪ V100R001C23]
arch=aarch64
os=linux
path=/usr/local/Ascend/ascend-toolkit/8.3.RC1/aarch64-linux
```

在安装 PyTorch 框架与 torch_npu 插件之前，请先确认 CANN 环境变量已正确加载。若尚未将加载命令写入 `~/.bashrc`，可在当前会话执行:

```
source /usr/local/Ascend/ascend-toolkit/set_env.sh
```

如需持久生效，请将上述命令追加到 ~/.bashrc 后执行 `source ~/.bashrc`。

3.3.2. PyTorch 二进制软件包安装方法

完成基础环境后，可以参考按照Ascend Extension for PyTorch 软件安装指南或者昇腾的 PyTorhc 框架的 Github 仓库部署 PyTorch 及 `torch_npu` 插件。在昇腾 310B 开发板上安装 PyTorch 框架和 torch_npu 插件的方法有两种，一种是二进制软件包安装，另外一种是源码编译安装。对于初学者来说，推荐使用二进制软件包进行安装。PyTorch 框架和 torch_npu 插件二进制软件包安装的具体方法如下:

安装 PyTorch 框架

由于出厂的 Ubuntu 22.04 系统已预装 miniconda，我们可以直接利用该环境。预装的 miniconda 通常带有 `bash`，在 `base` 下直接安装 PyTorch 与 torch_npu 容易产生冲突，因此建议新建一个 conda 环境，例如使用下面这个命令创建名为 `npu` 的环境。

```
conda create -n npu
```

然后利用下面这个命令进入npu虚拟环境：

```
conda activate npu
```

若按照本教程第二章在昇腾 310B 开发板上安装了 CANN 8.3.RC1, 则可用的 Python 版本为 3.8–3.11，对应支持的 PyTorch 版本为 2.1.0、2.6.0、2.7.1、2.8.0。这里选择安装 Python 3.11 与 PyTorch 2.8.0，先在新建的 `npu` 环境安装 Python 编译器：

```
conda install python=3.11
```

下载 PyTorch 二进制包时需与当前 Python 版本一致，可用 `python -V` 确认版本，输出示例为 `Python 3.11.14`。该环境还需安装 CANN 依赖的第三方库，可执行：

```
pip3 install attrs cython 'numpy>=1.19.2,<=1.24.0' decorator \
    sympy cffi pyyaml pathlib2 psutil protobuf==3.20.0 scipy requests absl-py \
    cloudpickle ml-dtypes tornado jinja2 matplotlib tqdm
```

在昇腾开发板上安装 PyTorch 可手动下载二进制包或直接用 pip 自动安装。若手动下载, 需确保与 Python 和 CANN 版本严格匹配, 可在华为昇腾 PyTorch 官方安装指南选择对应的包。假设 Python 3.11、CANN 8.3.RC1，可参考以下命令下载：

```
wget https://download.pytorch.org/whl/cpu/torch-2.8.0%2Bcpu-cp311-cp311-
    ↪ manylinux_2_28_aarch64.whl
```

然后使用 pip3 安装：

```
pip3 install torch-2.8.0+cpu-cp311-cp311-manylinux_2_28_aarch64.whl
```

安装完成后，可通过以下命令验证：

```
python -c "import torch; print(torch.__version__)"
```

若安装正确，输出示例为：

```
2.8.0+cpu
```

安装昇腾 torch_npu 插件

安装 torch_npu 时，版本需要与 PyTorch、Python、系统架构以及 CANN 保持一致。可前往华为昇腾 PyTorch 官方安装指南选择匹配的二进制包。例如在 Python 3.11、PyTorch 2.8.0、CANN 8.3.RC1 场景下，选择 torch_npu 7.2.0，对应示例下载命令为：

```
wget https://gitcode.com/Ascend/pytorch/releases/download/v7.2.0-pytorch2.8.0/torch_npu
    ↪ -2.8.0-cp311-cp311-manylinux_2_28_aarch64.whl
```

下载后使用 pip3 安装：

```
pip3 install torch_npu-2.8.0-cp311-cp311-manylinux_2_28_aarch64.whl
```

注意：直接使用 `pip install torch_npu` 进行简易安装，可能导致程序与当前环境不兼容，或与已安装的 CANN 版本不完全适配。因此，建议谨慎采用该方式安装 torch_npu 插件。

3.3.3. torch_npu 安装后测试

成功安装 torch_npu 后，需要对安装后的 torch_npu 进行简单的测试，以此验证是否安装成功。其中最简单的方法，是直接在命令窗口中输入以下命令：

```
python3 -c "import torch;import torch_npu; a = torch.randn(3, 4).npu(); print(a + a);"
```

成功运行后可见如下输出：

```
.[W compiler_depend.ts:164] Warning: Device do not support double dtype now, dtype cast
    ↪ replace with float. (function operator())
...tensor([[ 0.0879,  0.5805, -0.3437, -0.0229],
        [-1.7557,  0.0712,  2.7342,  1.8566],
        [-1.2092,  1.4896,  0.2234,  0.4621]], device='npu:0')
```

上述 warning 来源于昇腾 310B 暂不支持双精度浮点（double），运行时会自动将双精度浮点 double 转为单精度浮点 float。若在 HwHiAiUser（非 root）用户下执行同一命令，输出与下述示例类似：

```
/usr/local/miniconda3/lib/python3.9/site-packages/torch_npu/utils/collect_env.py:59:
    ↪ UserWarning: Warning: The /usr/local/Ascend/ascend-toolkit/latest owner does not
    ↪ match the current owner.
  warnings.warn(f"Warning: The {path} owner does not match the current owner.")
/usr/local/miniconda3/lib/python3.9/site-packages/torch_npu/utils/collect_env.py:59:
    ↪ UserWarning: Warning: The /usr/local/Ascend/ascend-toolkit/8.3.RC1/aarch64-linux/
    ↪ ascend_toolkit_install.info owner does not match the current owner.
  warnings.warn(f"Warning: The {path} owner does not match the current owner.")
```

```
/usr/local/miniconda3/lib/python3.9/site-packages/torch_npu/utils/_path_manager.py:67:
    ↪ UserWarning: Permission mismatch: The owner of /usr/local/miniconda3/lib/python3
    ↪ .9/site-packages/torch_npu/lib/libop_plugin_atb.so does not match.
  warnings.warn(f"Permission mismatch: The owner of {path} does not match.")
/usr/local/miniconda3/lib/python3.9/site-packages/torch_npu/utils/collect_env.py:59:
    ↪ UserWarning: Warning: The /usr/local/Ascend/ascend-toolkit/latest owner does not
    ↪ match the current owner.
  warnings.warn(f"Warning: The {path} owner does not match the current owner.")
/usr/local/miniconda3/lib/python3.9/site-packages/torch_npu/utils/collect_env.py:59:
    ↪ UserWarning: Warning: The /usr/local/Ascend/ascend-toolkit/8.3.RC1/aarch64-linux/
    ↪ ascend_toolkit_install.info owner does not match the current owner.
  warnings.warn(f"Warning: The {path} owner does not match the current owner.")
[W compiler_depend.ts:164] Warning: Device do not support double dtype now, dtype cast
    ↪ replace with float. (function operator())
tensor([[ 0.4831,  0.6161,  0.2278, -0.1595],
        [ 1.3375,  0.2916, -0.3860, -2.3874],
        [ 1.3766, -1.6611,  2.7217,  2.0813]], device='npu:0')
```

尽管会出现较多 warning，这是因为 torch_npu 插件用 root 账户安装所致，但结果依然是正确的。

注意事项：多数情况下，已成功安装后仍在运行上文命令时报错，原因是内存不足，尤其常见于 8GB 内存的昇腾 310B 开发板。首次运行 PyTorch+torch_npu 时，会触发将计算图交给 CANN 进行编译与优化（含算子/图的并行编译），默认并行度较高，容易在低内存设备上触发 OOM。

处理办法：降低编译并行度

- 临时生效（仅针对本次运行）：

```
TE_PARALLEL_COMPILER=1 MAX_COMPILE_CORE_NUMBER=1 python your_script.py
```

- 会话/永久生效：

```
export TE_PARALLEL_COMPILER=1
export MAX_COMPILE_CORE_NUMBER=1
# 建议追加到 ~/.bashrc 后执行：source ~/.bashrc
```

说明 - 该问题与使用 atc 工具时的并行编译内存占用现象一致，降低并行度可显著缓解。- 首次运行的编译开销较大，通过以上设置可提高在 8GB 设备上的稳定性。

3.4. torch_npu 插件的使用

PyTorch 是广泛使用的深度学习框架，基于 Python，入门友好且生态完善。系统学习可参考李沐的《动手学深度学习》第二版（开源电子书，含配套代码）：动手学深度学习。

关于 torch_npu：目前在昇腾 310B 上自动迁移工具不可用（实测无法跑通），因此本章默认不使用自动迁移，统一采用手工迁移路径，并提供常见替换与调优对策。迁移与调优可参考官方文档《PyTorch 训练模型迁移调优》。但是该文档主要面向训练服务器，细节与 310B 存在差异。

下文将结合若干实例，演示 torch_npu 在昇腾 310B 上的实践要点。

3.4.1. 线性神经网络实现

为了更清晰地展示基于 CUDA 的 PyTorch 使用与基于 `torch_npu` 的使用之间的区别，我们将从经典的线性神经网络入手。线性回归和 softmax 回归作为经典统计学习技术，可以视为线性神经网络的实例。这些知识将为在昇腾 310B 上进行代码移植的其他部分奠定基础。

一元线性回归

线性回归可用于刻画变量间的线性关系。在最简单的一元情形中，大学物理实验的电阻测量就是典型案例。根据欧姆定律可得：

$$V = I\,R$$

为适应实际中的接触电势/表计零漂，常用含偏置模型：

$$V_i \approx R\,I_i + b$$

以均方误差为目标函数：

$$L(R,b) = \frac{1}{n}\sum_{i=1}^{n}\left(R\,I_i + b - V_i\right)^2$$

其梯度为：

$$\frac{\partial L}{\partial R} = \frac{2}{n}\sum_{i=1}^{n} I_i\left(R\,I_i + b - V_i\right), \quad \frac{\partial L}{\partial b} = \frac{2}{n}\sum_{i=1}^{n}\left(R\,I_i + b - V_i\right)$$

采用小批量 SGD（批大小为 m，索引集合 B）：

$$g_R = \frac{2}{m}\sum_{i\in B} I_i\left(R\,I_i + b - V_i\right), \quad g_b = \frac{2}{m}\sum_{i\in B}\left(R\,I_i + b - V_i\right)$$

按学习率更新：

$$R \leftarrow R - \eta\,g_R, \quad b \leftarrow b - \eta\,g_b$$

实践流程（单位统一为 A、V，建议 float32，NPU 上双精度会自动降为 float32）：- 采集多组 (I_i, V_i)，若存在接触电势选用含偏置模型。- 初始化参数（启发式：$R \approx (\sum I_i V_i)/(\sum I_i^2), b \approx 0$）。- 随机抽取小批量，计算 g_R, g_b，按学习率迭代至收敛。- 用 MSE 或 R^2 评估拟合，必要时剔除离群点。

下方示例代码在 PyTorch+torch_npu 上实现上述流程。

```python
"""线性回归（V $\approx$ R·I + b）示例，使用华为昇腾 NPU 设备训练。
- 数据：合成电压/电流数据，单位 A/V，含高斯噪声（约 5mV）。
- 目标：拟合电阻 R（斜率）与偏置 b（截距）。
- 设备：默认 npu:0，可切换为 CPU。
- 训练：SGD 优化，MSE 损失，小批量训练并通过 tqdm 展示进度。
- 输出：打印估计的 R、b，并保存训练损失曲线为 training_loss.png。
"""

import torch  # PyTorch 主库
from tqdm import tqdm  # 进度条显示
import matplotlib.pyplot as plt  # 绘图
import torch_npu  # NPU 支持库（确保已安装与环境就绪）

device = torch.device('npu:0')  # 选择 NPU 设备（Ascend），需要正确的驱动/环境
torch.manual_seed(0)  # 固定随机种子以确保可复现

# 使用合成数据（无自有数据）
N = 1024  # 样本数
R_true, b_true = 120.0, 0.02  # 真实电阻（Ω）与偏置（V），用于生成数据的“地真值”
I = torch.linspace(0.0, 1.0, N).unsqueeze(1)  # 电流张量 [N,1]，范围 0~1 A
noise_std = 0.005  # 噪声标准差 $\approx$ 5 mV
noise = noise_std * torch.randn(N, 1)  # 加性高斯噪声，与 V 同形状
V = R_true * I + b_true + noise  # 生成电压张量 [N,1]，单位 V

I = I.to(device)  # 将数据移至计算设备（NPU/CPU）
V = V.to(device)
batch_size = 16  # 小批量大小，影响梯度估计方差与训练稳定性

# 线性模型 V $\approx$ R·I + b（1 输入 -> 1 输出，带偏置）
model = torch.nn.Linear(1, 1, bias=True).to(device)  # 权重即电阻 R，偏置即 b
opt = torch.optim.SGD(model.parameters(), lr=1e-2)  # 随机梯度下降，学习率 1e-2
loss_fn = torch.nn.MSELoss()  # 均方误差：衡量预测电压与真实电压的差异

# 使用小批量训练循环，加入 tqdm 进度条并记录 loss
num_epochs = 50  # 训练轮数
epoch_losses = []  # 记录每个 epoch 的平均损失，便于绘图

with tqdm(range(num_epochs), desc="Epochs") as pbar:  # 进度条显示每个 epoch
    for _ in pbar:
        epoch_loss = 0.0  # 累计当前 epoch 的损失
        n_batches = 0  # 实际批次数（用于计算平均损失）
        perm = torch.randperm(I.shape[0], device=device)  # 打乱索引以实现随机小批量
        for start in range(0, I.shape[0], batch_size):  # 遍历每个批次起点
            end = start + batch_size  # 批次终点
            idx = perm[start:end]  # 当前批次的随机样本索引
```

```
            I_b = I[idx].to(torch.float32)  # NPU 通常以 float32 计算，确保类型一致
            V_b = V[idx].to(torch.float32)
            pred = model(I_b)  # 前向计算: V̂ = R·I + b
            loss = loss_fn(pred, V_b)  # 计算当前批次的 MSE 损失
            opt.zero_grad()  # 清空上一轮梯度
            loss.backward()  # 反向传播，计算参数梯度
            opt.step()  # 参数更新（SGD）
            epoch_loss += loss.item()  # 累加标量损失
            n_batches += 1  # 批次数 +1
        avg_loss = epoch_loss / max(n_batches, 1)  # 当前 epoch 的平均损失
        epoch_losses.append(avg_loss)  # 记录以便后续绘图
        pbar.set_postfix(loss=f"{avg_loss:.6f}")  # 在进度条尾部显示当前损失

R = model.weight.detach().item()  # 提取斜率（电阻，单位 Ω），不跟踪梯度
b = model.bias.detach().item()    # 提取截距（偏置，单位 V），不跟踪梯度
print("device =", device)  # 打印使用的设备
print("R(Ω) =", R, " b(V) =", b)  # 打印拟合结果，便于与真值对比

# 绘制训练损失下降曲线并保存图片
plt.figure(figsize=(6, 4))  # 设定图像尺寸
plt.plot(epoch_losses, label="Train MSE")  # 绘制每个 epoch 的平均损失
plt.xlabel("Epoch")  # 横轴：训练轮数
plt.ylabel("Loss")  # 纵轴：均方误差
plt.title("Training Loss")  # 标题
plt.grid(True)  # 开启网格，便于阅读
plt.legend()  # 图例显示
plt.tight_layout()  # 自动布局，防止标签遮挡
plt.savefig("linear_regression_training_loss.png")  # 保存为 PNG 文件
```

要运行上述示例，请先在当前（npu）环境中安装依赖：

```
pip install tqdm matplotlib
```

示例运行后可见如下输出：

```
Epochs: 100%|
    ↪ ████████████████████████████████████████████████████████████████████████████████
    ↪  50/50 [00:28<00:00,  1.77it/s, loss=0.173204]
device = npu:0
R(Ω) = 118.63302612304688  b(V) = 0.7558640241622925
```

同时给出训练结果的可视化：下图展示训练损失（MSE）随 epoch 变化的曲线。

提示 - 数据需统一单位（A、V），使用 float32 更稳妥。- 无明显偏置时可将偏置项置为 0；存在测量/接触偏差时保留偏置并做鲁棒处理（如剔除离群点）。- 拟合优劣可用 MSE 或 R^2 评估，必要时清理异常样本。- 在部分环境中，PyTorch 2.1 运行上述代码可能报错，通常与 nn.Linear 的输出维度设为 1 的配置触发兼容性问题有关；建议升级至更高版本或将输出维度调整为大于 1 以规避。- 将代码中的 `device = torch.device('npu:0')`

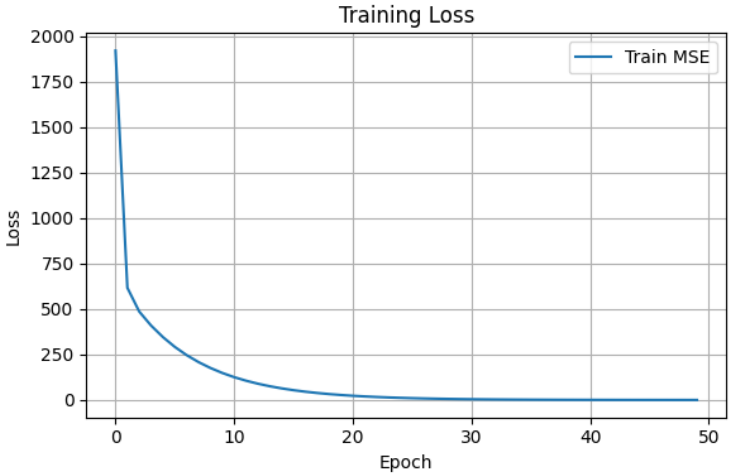


图 3.1. 线性回归的损失函数关系图

改为 `device = torch.device('cpu')`，即可在纯 CPU 上运行。该示例在 CPU 与 NPU 上耗时接近，主要因为未涉及大规模矩阵/张量计算，计算密度较低，NPU 的并行优势难以发挥。

多元线性回归

为了验证昇腾 310B 上 torch_npu 与 PyTorch 的兼容性，下面以加州房价数据集的多元线性回归示例继续演示 torch_npu 的使用。加州房价数据（California Housing Dataset）可通过 `fetch_california_housing()` 在 `scikit-learn` 获取，或在 Kaggle 等平台下载 CSV。该数据集包含经纬度（longitude/latitude）、房屋中位年龄（housing_median_age）、总房间数与卧室数（total_rooms/total_bedrooms）、人口与家庭数（population/households）、收入中位数（median_income）、房价中位数（median_house_value）以及与海距离类别（ocean_proximity）等字段，可用于房价预测（如线性回归、随机森林、XGBoost 等回归模型）、结合经纬度进行地理可视化绘制房价热力图以分析区域差异，并通过 “每户平均房间数” 等衍生特征进行特征工程以提升效果。需注意，该数据为历史数据，不能直接反映当前价格；使用时应遵守隐私与数据保护规定，尤其在与其他数据源结合时；该数据集结构清晰、特征丰富，是机器学习回归入门的经典案例。对于线性神经网络（多元线性回归），可表示为：

$$\hat{\mathbf{y}} = \mathbf{X}\mathbf{w} + \mathbf{b}, \quad \mathbf{X} \in \mathbb{R}^{m \times d},\ \mathbf{w} \in \mathbb{R}^{d \times k},\ \mathbf{b} \in \mathbb{R}^{k}$$

其中 m 为样本数，d 为特征维度，k 为输出维度（回归时常取 $k = 1$）；对于单样本 $\mathbf{x} \in \mathbb{R}^{d}$，有 $\hat{\mathbf{y}} = \mathbf{x}^{\top} W + b$。加州房价数据集的输入特征维度为 8，输出为房价估计值，其线性神经网络结构示意如下：

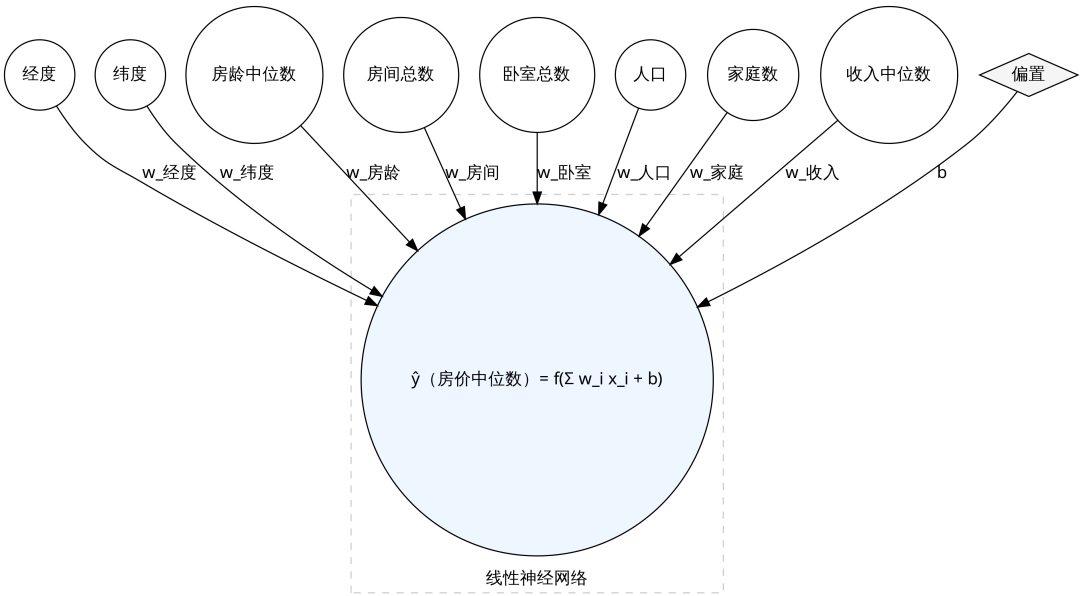


图 3.2. 线性神经网络

损失函数通常选用均方误差（MSE）：

$$\mathcal{L}(\mathbf{w}, b) = \frac{1}{m} \sum_{i=1}^{m} \left\| \hat{\mathbf{y}}^{(i)} - \mathbf{y}^{(i)} \right\|_2^2$$

在训练模型的时候，我们希望寻找出一组参数 $(\mathbf{w}^*, b^*)$，使得训练样品的损失函数最小：

$$\mathbf{w}^*, b^* = \underset{\mathbf{w}, b}{\operatorname{argmin}} \, \mathcal{L}(\mathbf{w}, b)$$

在实际训练中，通常采用小批量随机梯度下降法来求解最优参数。其算法流程如下：

1. **初始化**：随机初始化模型参数 $\mathbf{w}$ 和 b，并设定学习率 η。
2. **小批量采样**：从训练数据中随机抽取包含 m 个样本的小批量 $\mathcal{B} = \{(\mathbf{x}^{(1)}, y^{(1)}), \ldots, (\mathbf{x}^{(m)}, y^{(m)})\}$。
3. **计算梯度**：计算损失函数关于参数的梯度估计值：

$$\mathbf{g}_{\mathbf{w}} = \frac{\partial \mathcal{L}}{\partial \mathbf{w}} = \frac{2}{m} \sum_{i \in \mathcal{B}} \mathbf{x}^{(i)} (\mathbf{x}^{(i)\top} \mathbf{w} + b - y^{(i)})$$

$$g_b = \frac{\partial \mathcal{L}}{\partial b} = \frac{2}{m} \sum_{i \in \mathcal{B}} (\mathbf{x}^{(i)\top} \mathbf{w} + b - y^{(i)})$$

4. **更新参数**：利用计算出的梯度更新当前参数：

$$\mathbf{w} \leftarrow \mathbf{w} - \eta \cdot \mathbf{g}_{\mathbf{w}}$$

$$b \leftarrow b - \eta \cdot g_b$$

5. **迭代**：重复步骤 2-4，直到满足停止准则（如达到最大迭代次数或损失函数收敛）。

在上述算法中，学习率（Learning Rate, η）与训练轮数（Epoch）是影响模型性能的关键超参数。学习率决定了参数沿梯度反方向更新的步长：若设置过大，模型可能在最优解附近震荡甚至发散；若设置过小，则收敛速度过慢且易陷入局部最优。通常学习率的取值范围在 0.01 到 0.00001 之间。训练轮数是指完整遍历一次训练数据集的过程。由于单次迭代仅利用小批量数据更新梯度，通常需要多个 Epoch 才能使模型参数充分拟合数据特征并趋于稳定。一般建议将 Epoch 设置在 10 到 100 之间。

利用 torch_npu 插件，可以在昇腾 310B 上进行这个模型的训练，合理配置这些超参数不仅影响模型的最终精度，还直接关系到有限算力资源下的训练效率。详细的代码如下：

```python
import torch
import torch.nn as nn
import torch.optim as optim
from sklearn.datasets import fetch_california_housing
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from torch.utils.data import DataLoader, TensorDataset
import numpy as np
from tqdm.auto import tqdm, trange

# ==========================================
# 1. 环境配置与随机种子设置
# ==========================================
# 设置随机种子以确保实验结果的可复现性
np.random.seed(42)
torch.manual_seed(42)

# 指定计算设备。'npu' 表示使用华为昇腾 AI 处理器
# 如果在没有 NPU 的环境下运行，可改为 'cuda' 或 'cpu'
device = torch.device('npu')

# ==========================================
# 2. 数据加载与预处理
# ==========================================
# 加载加州房价数据集（回归任务）
california = fetch_california_housing()
X = california.data  # 特征矩阵：包含 8 个特征（如人均收入、房龄等）
y = california.target.reshape(-1, 1)  # 目标值：房价，调整为 (N, 1) 形状

# 数据标准化：神经网络对输入特征的量级很敏感
# 使用 StandardScaler 使数据符合均值为 0，方差为 1 的分布
scaler_X = StandardScaler()
scaler_y = StandardScaler()
X_scaled = scaler_X.fit_transform(X)
y_scaled = scaler_y.fit_transform(y)
```

```
36
37 # 划分数据集：80% 用于训练，20% 用于验证
38 X_train, X_val, y_train, y_val = train_test_split(
39     X_scaled, y_scaled, test_size=0.2, random_state=42
40 )
41
42 # ========================================
43 # 3. 构建 PyTorch 数据加载器 (DataLoader)
44 # ========================================
45 batch_size = 32
46
47 # 将 NumPy 数组转换为 PyTorch 张量 (FloatTensor)
48 train_dataset = TensorDataset(
49     torch.FloatTensor(X_train),
50     torch.FloatTensor(y_train)
51 )
52 val_dataset = TensorDataset(
53     torch.FloatTensor(X_val),
54     torch.FloatTensor(y_val)
55 )
56
57 # DataLoader 负责按批次 (Batch) 提供数据，并在训练集上开启打乱 (Shuffle)
58 train_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True)
59 val_loader = DataLoader(val_dataset, batch_size=batch_size)
60
61 # ========================================
62 # 4. 定义模型结构
63 # ========================================
64 # 单层感知机：本质上是一个线性回归模型 (y = Wx + b)
65 class SingleLayerPerceptron(nn.Module):
66     def __init__(self, input_dim):
67         super().__init__()
68         # 定义一个全连接层，输入维度为特征数，输出维度为 1
69         self.fc = nn.Linear(input_dim, 1)
70
71     def forward(self, x):
72         # 前向传播逻辑
73         return self.fc(x)
74
75 # 实例化模型并迁移到指定的计算设备 (NPU)
76 model = SingleLayerPerceptron(X_train.shape[1]).to(device)
77
78 # 定义损失函数：均方误差 (MSE)，适用于回归任务
79 criterion = nn.MSELoss()
80
81 # 定义优化器：随机梯度下降 (SGD)，学习率设为 0.01
82 optimizer = optim.SGD(model.parameters(), lr=0.01)
83
84 # ========================================
85 # 5. 训练与验证函数定义
86 # ========================================
87 def train_epoch(model, loader, criterion, optimizer):
88     """单轮训练逻辑"""
```

```
    model.train() # 设置为训练模式
    total_loss = 0
    for batch_x, batch_y in tqdm(loader, desc="Train", leave=False):
        # 将数据迁移到 NPU
        batch_x = batch_x.to(device)
        batch_y = batch_y.to(device)

        # 梯度清零
        optimizer.zero_grad()
        # 前向传播
        outputs = model(batch_x)
        # 计算损失
        loss = criterion(outputs, batch_y)
        # 反向传播
        loss.backward()
        # 梯度裁剪：防止梯度爆炸
        torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
        # 更新参数
        optimizer.step()

        total_loss += loss.item() * batch_x.size(0)
    return total_loss / len(loader.dataset)

def validate(model, loader, criterion):
    """验证逻辑"""
    model.eval() # 设置为评估模式
    total_loss = 0
    with torch.no_grad(): # 验证阶段不需要计算梯度
        for batch_x, batch_y in tqdm(loader, desc="Val", leave=False):
            batch_x = batch_x.to(device)
            batch_y = batch_y.to(device)
            outputs = model(batch_x)
            loss = criterion(outputs, batch_y)
            total_loss += loss.item() * batch_x.size(0)
    return total_loss / len(loader.dataset)

# ========================================
# 6. 执行训练循环
# ========================================
epochs = 20
train_losses = []
val_losses = []

# 使用 trange 展示总进度条
t = trange(epochs, desc="Epochs")
for epoch in t:
    train_loss = train_epoch(model, train_loader, criterion, optimizer)
    val_loss = validate(model, val_loader, criterion)

    train_losses.append(train_loss)
    val_losses.append(val_loss)

    # 在进度条右侧实时显示当前损失和学习率
```

```
    t.set_postfix(train_loss=f"{train_loss:.4f}", val_loss=f"{val_loss:.4f}", lr=f'{optimizer.param_groups[0]["lr"]:.6f}')

print(f"\nFinal Validation Loss: {val_losses[-1]:.4f}")

# ========================================
# 7. 模型评估与指标计算
# ========================================
model.eval()
all_predictions = []
all_targets = []

# 获取验证集上的所有预测结果
with torch.no_grad():
    for batch_x, batch_y in val_loader:
        batch_x = batch_x.to(device)
        predictions = model(batch_x)
        # 将结果移回 CPU 并转为 NumPy
        all_predictions.extend(predictions.cpu().numpy())
        all_targets.extend(batch_y.cpu().numpy())

# 逆标准化：将预测值和真实值还原回原始的房价单位（10万美元）
all_predictions = scaler_y.inverse_transform(np.array(all_predictions))
all_targets = scaler_y.inverse_transform(np.array(all_targets))

# 计算回归常用指标
from sklearn.metrics import r2_score, mean_absolute_error, mean_squared_error

r2 = r2_score(all_targets, all_predictions) # 决定系数，越接近 1 越好
mae = mean_absolute_error(all_targets, all_predictions) # 平均绝对误差
rmse = np.sqrt(mean_squared_error(all_targets, all_predictions)) # 均方根误差

print(f"\n最终评估指标:")
print(f"R² Score: {r2:.4f}")
print(f"MAE: ${mae:.2f}")
print(f"RMSE: ${rmse:.2f}")

# ========================================
# 8. 结果可视化
# ========================================
import matplotlib.pyplot as plt

fig, axes = plt.subplots(2, 2, figsize=(12, 10))

# 1. 损失曲线：观察模型是否收敛
axes[0, 0].plot(train_losses, label='Train Loss')
axes[0, 0].plot(val_losses, label='Val Loss')
axes[0, 0].set_xlabel('Epoch')
axes[0, 0].set_ylabel('Loss')
axes[0, 0].set_title('Training and Validation Loss')
axes[0, 0].legend()
axes[0, 0].grid(True)

```

```
# 2. 预测值 vs 真实值：散点越接近对角线表示预测越准
axes[0, 1].scatter(all_targets, all_predictions, alpha=0.3)
axes[0, 1].set_xlabel('True Price')
axes[0, 1].set_ylabel('Predicted Price')
axes[0, 1].set_title('True vs Predicted Prices')
axes[0, 1].grid(True)

# 3. 残差图：检查误差是否随机分布（理想状态下应无明显模式）
residuals = all_targets - all_predictions
axes[1, 0].scatter(all_predictions, residuals, alpha=0.3)
axes[1, 0].set_xlabel('Predicted Price')
axes[1, 0].set_ylabel('Residuals')
axes[1, 0].set_title('Residual Plot')
axes[1, 0].grid(True)

# 4. 误差分布直方图：检查误差是否符合正态分布
axes[1, 1].hist(residuals, bins=50, edgecolor='black')
axes[1, 1].set_xlabel('Error')
axes[1, 1].set_ylabel('Frequency')
axes[1, 1].set_title('Error Distribution')

plt.tight_layout()
plt.savefig('california_housing_linear_network_results.png')
```

在配置好 torch_npu 环境的昇腾 310B 开发板上运行以上的代码，可以得到以下的结果：

```
Epochs: 100%|██████████| 50/50 [02:14<00:00,  2.68s/it, lr=0.000100, train_loss=0.4308,
    ↪ val_loss=0.4444]

Final Validation Loss: 0.4444

最终评估指标:
R² Score: 0.5484
MAE: $0.56
RMSE: $0.77
```

同时，我们可以得到训练过程的损失收敛及评估指标图如下图所示：

3.4.2. 多层感知机实现

我们之所以需要多层感知机（Multilayer Perceptron, MPL），是因为线性神经网络的输出仅为输入的加权和，难以刻画现实世界中复杂的非线性关系。以房价预测为例，房屋面积对价格的影响往往存在边际效应，而非简单的线性增长。为了突破这一局限，我们需要构建更深的网络结构。

如果我们仅仅通过堆叠多个线性层来构建网络，其数学表达为

$$\hat{\mathbf{y}} = \mathbf{W}_2(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2$$

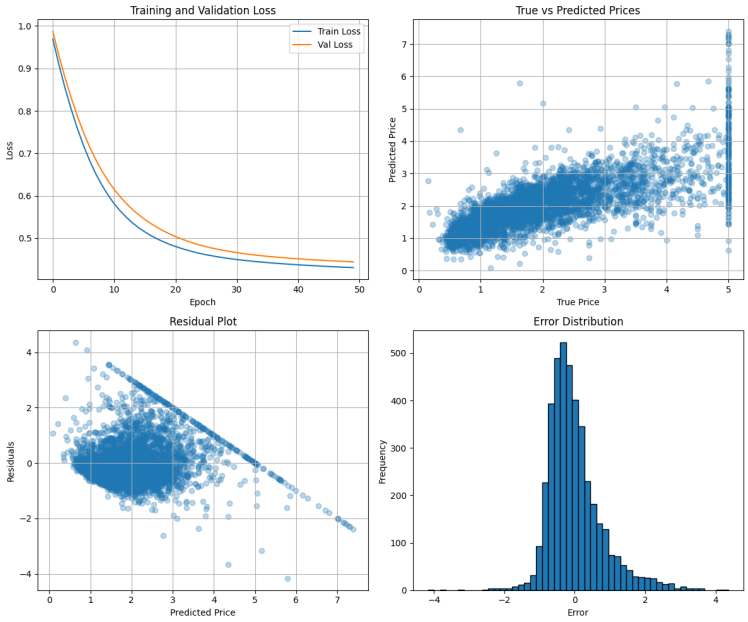


图 3.3. 线性神经网络训练结果图

展开后可见

$$\hat{\mathbf{y}} = (\mathbf{W}_2\mathbf{W}_1)\mathbf{x} + (\mathbf{W}_2\mathbf{b}_1 + \mathbf{b}_2) = \mathbf{W}_{new}\mathbf{x} + \mathbf{b}_{new}$$

这意味着多个线性层的组合在数学上仍然等价于单个线性层。无论网络深度如何增加，若缺乏非线性因素，它都无法拟合复杂的非线性函数。

多层感知机通过在层间引入激活函数打破了这种线性限制。以常用的 ReLU 为例，其定义为

$$\text{ReLU}(x) = \max(0, x)$$

另一个经典的激活函数是 Sigmoid 函数，它将任意实数映射到 $(0, 1)$ 区间，曾广泛应用于早期神经网络。其定义为

$$\text{sigmoid}(x) = \frac{1}{1 + e^{-x}}$$

这种非线性变换允许网络通过组合不同的线性分段来逼近任意复杂的连续函数。一个含隐藏层的多层感知机可以表示为

$$\mathbf{H} = \sigma(\mathbf{X}\mathbf{W}_1 + \mathbf{b}_1)$$

与

$$\mathbf{O} = \mathbf{H}\mathbf{W}_2 + \mathbf{b}_2$$

其中 σ 即为非线性激活函数。

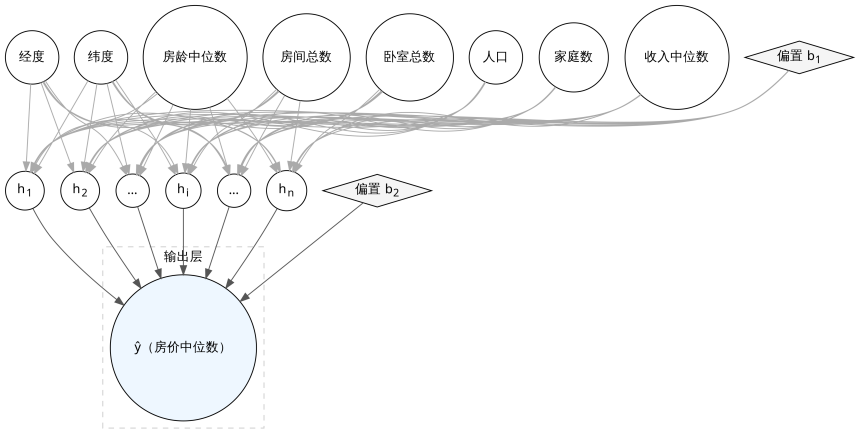


图 3.4. 多层神经网络

相比于只能处理线性相关问题的线性网络，多层感知机通过增加隐藏层的深度与宽度，能够捕捉特征之间的高阶交互作用。在加州房价数据集中，这种结构可以学习到经纬度与收入水平之间复杂的空间关联。对于加州房价数据集的多层感知机的模型如下图所示：

在昇腾 310B 上实现多层感知机，不仅能验证 `torch_npu` 对多层算子序列的调度能力，也能通过实验直观观察到非线性映射带来的精度提升。具体的代码如下：

```python
import torch
import torch.nn as nn
import torch.optim as optim
from sklearn.datasets import fetch_california_housing
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from torch.utils.data import DataLoader, TensorDataset
import numpy as np
from tqdm.auto import tqdm, trange
import random

# 设置随机种子以确保实验结果的可重现性
SEED = 42
random.seed(SEED)
np.random.seed(SEED)
torch.manual_seed(SEED)
# 指定使用昇腾 NPU 设备
device = torch.device('npu:0')

# 加载加州房价数据集：包含8个特征，目标值为房屋中位数价格
california = fetch_california_housing()
X = california.data  # 特征矩阵 (样本数, 8)
y = california.target.reshape(-1, 1) # 目标值 (样本数, 1)

# 数据标准化：将特征和目标值缩放到均值为0、方差为1的分布，有助于模型收敛
scaler_X = StandardScaler()
```

```
scaler_y = StandardScaler()
X_scaled = scaler_X.fit_transform(X)
y_scaled = scaler_y.fit_transform(y)

# 划分数据集：80% 用于训练，20% 用于验证
X_train, X_val, y_train, y_val = train_test_split(
    X_scaled, y_scaled, test_size=0.2, random_state=42
)

# 构建 PyTorch 数据管道
batch_size = 32
train_dataset = TensorDataset(
    torch.FloatTensor(X_train),
    torch.FloatTensor(y_train)
)
val_dataset = TensorDataset(
    torch.FloatTensor(X_val),
    torch.FloatTensor(y_val)
)

# DataLoader 负责按批次(Batch)提供数据并支持洗牌(Shuffle)
train_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True)
val_loader = DataLoader(val_dataset, batch_size=batch_size)

# 定义双层感知机模型
class DoubleLayerPerceptron(nn.Module):
    def __init__(self, input_dim, hidden_dim=64):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(input_dim, hidden_dim), # 输入层到隐藏层
            nn.ReLU(),                        # 激活函数，引入非线性
            nn.Linear(hidden_dim, 1)          # 隐藏层到输出层（回归任务输出维度为1）
        )
    def forward(self, x):
        return self.net(x)

# 实例化模型并迁移至 NPU
model = DoubleLayerPerceptron(X_train.shape[1]).to(device)
# 回归任务常用的均方误差损失函数
criterion = nn.MSELoss()
# 随机梯度下降优化器
optimizer = optim.SGD(model.parameters(), lr=0.0001)

# 单个 Epoch 的训练逻辑
def train_epoch(model, loader, criterion, optimizer):
    model.train()
    total_loss = 0
    for batch_x, batch_y in tqdm(loader, desc="Train", leave=False):
        # 将数据搬运到 NPU 显存
        batch_x = batch_x.to(device)
        batch_y = batch_y.to(device)

        optimizer.zero_grad()           # 清空梯度
```

```
        outputs = model(batch_x)          # 前向传播
        loss = criterion(outputs, batch_y) # 计算损失
        loss.backward()                   # 反向传播计算梯度
        torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0) # 梯度裁剪防止梯度爆
    ↪ 炸
        optimizer.step()                  # 更新权重
        total_loss += loss.item() * batch_x.size(0)
    return total_loss / len(loader.dataset)

# 验证逻辑
def validate(model, loader, criterion):
    model.eval() # 设置为评估模式
    total_loss = 0
    with torch.no_grad(): # 验证阶段不需要计算梯度
        for batch_x, batch_y in tqdm(loader, desc="Val", leave=False):
            batch_x = batch_x.to(device)
            batch_y = batch_y.to(device)
            outputs = model(batch_x)
            loss = criterion(outputs, batch_y)
            total_loss += loss.item() * batch_x.size(0)
    return total_loss / len(loader.dataset)

# 执行训练过程
epochs = 50
train_losses = []
val_losses = []

t = trange(epochs, desc="Epochs")
for epoch in t:
    train_loss = train_epoch(model, train_loader, criterion, optimizer)
    val_loss = validate(model, val_loader, criterion)
    train_losses.append(train_loss)
    val_losses.append(val_loss)
    # 在进度条右侧实时显示当前损失和学习率
    t.set_postfix(train_loss=f"{train_loss:.4f}", val_loss=f"{val_loss:.4f}", lr=f'{
    ↪ optimizer.param_groups[0]["lr"]:.6f}')

print(f"\nValidation Loss: {val_losses[-1]:.4f}")

# 评估阶段：获取验证集的预测值并还原标准化
model.eval()
all_predictions = []
all_targets = []

with torch.no_grad():
    for batch_x, batch_y in val_loader:
        batch_x = batch_x.to(device)
        predictions = model(batch_x)
        all_predictions.extend(predictions.cpu().numpy())
        all_targets.extend(batch_y.cpu().numpy())

# 将标准化后的数据逆转回原始价格单位
all_predictions = scaler_y.inverse_transform(np.array(all_predictions))
```

```
all_targets = scaler_y.inverse_transform(np.array(all_targets))

# 计算回归评估指标
from sklearn.metrics import r2_score, mean_absolute_error, mean_squared_error

r2 = r2_score(all_targets, all_predictions) # 决定系数，越接近 1 越好
mae = mean_absolute_error(all_targets, all_predictions) # 平均绝对误差
rmse = np.sqrt(mean_squared_error(all_targets, all_predictions)) # 均方根误差

print(f"\n最终评估指标:")
print(f"R² Score: {r2:.4f}")
print(f"MAE: ${mae:.2f}")
print(f"RMSE: ${rmse:.2f}")

# 结果可视化
import matplotlib.pyplot as plt

fig, axes = plt.subplots(2, 2, figsize=(12, 10))

# 1. 损失下降曲线：观察模型是否收敛或过拟合
axes[0, 0].plot(train_losses, label='Train Loss')
axes[0, 0].plot(val_losses, label='Val Loss')
axes[0, 0].set_xlabel('Epoch')
axes[0, 0].set_ylabel('Loss')
axes[0, 0].set_title('Training and Validation Loss')
axes[0, 0].legend()
axes[0, 0].grid(True)

# 2. 预测值 vs 真实值散点图：点越靠近红虚线表示预测越准
axes[0, 1].scatter(all_targets, all_predictions, alpha=0.3)
axes[0, 1].plot([all_targets.min(), all_targets.max()],
                [all_targets.min(), all_targets.max()], 'r--', lw=2)
axes[0, 1].set_xlabel('True Price')
axes[0, 1].set_ylabel('Predicted Price')
axes[0, 1].set_title('True vs Predicted Prices')
axes[0, 1].grid(True)

# 3. 残差图：检查误差是否随机分布（理想状态应均匀分布在0附近）
residuals = all_targets - all_predictions
axes[1, 0].scatter(all_predictions, residuals, alpha=0.3)
axes[1, 0].axhline(y=0, color='r', linestyle='--')
axes[1, 0].set_xlabel('Predicted Price')
axes[1, 0].set_ylabel('Residuals')
axes[1, 0].set_title('Residual Plot')
axes[1, 0].grid(True)

# 4. 误差分布直方图：观察误差是否符合正态分布
axes[1, 1].hist(residuals, bins=50, edgecolor='black')
axes[1, 1].set_xlabel('Error')
axes[1, 1].set_ylabel('Frequency')
axes[1, 1].set_title('Error Distribution')
axes[1, 1].grid(True)

```

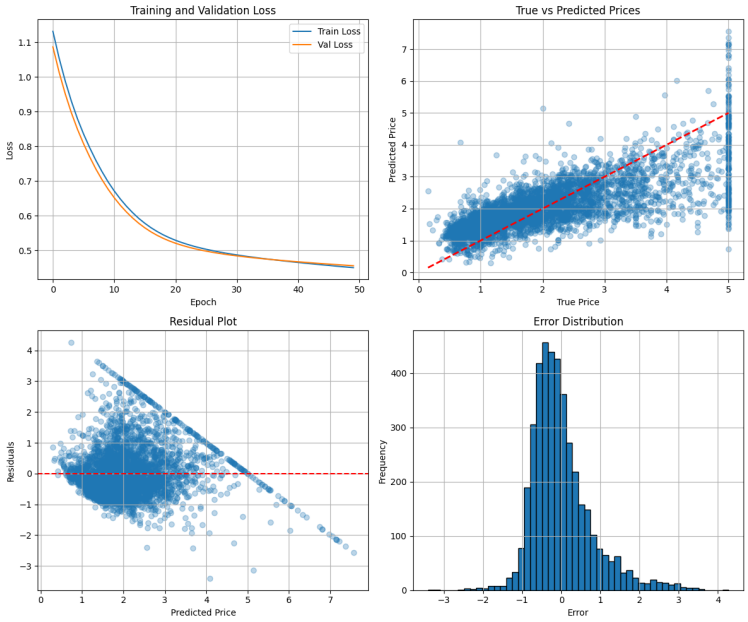


图 3.5. 多层神经网络训练结果图

```
plt.tight_layout()
plt.savefig('california_housing_mlp_results.png')
```

在昇腾 310B 开发板上运行以上代码，可以得到以下的结果：

```
Epochs: 100%|████████████████████████████████████████████████████████████|
    ↪ 50/50 [02:39<00:00,  3.19s/it, lr=0.000100, train_loss=0.4509, val_loss=0.4563]

Validation Loss: 0.4563

最终评估指标:
R² Score: 0.5363
MAE: $0.57
RMSE: $0.78
```

同时，我们可以得到训练过程的损失收敛及评估指标图如下图所示：

3.4.3. LeNet-5

LeNet-5 是由 Yann LeCun 等人在 1998 年提出的经典卷积神经网络（CNN），最初用于手写数字识别（MNIST 数据集）。它是深度学习领域的里程碑，奠定了现代卷积神经网络的基础架构。其名称中的 “Le” 取自第一作者 Yann LeCun 的姓氏，而数字 “5” 则代表该网络包含 **5 层具有可训练参数的权重层**（包括 3 个卷积层和 2 个全连接层）。在原始

设计中，C5 层虽然在逻辑上起到了全连接的作用，但由于其卷积核尺寸与输入特征图完全一致，因此被归类为卷积层。

LeNet-5 共有 7 层（不含输入层），其标准结构如下：

1. **输入层 (Input)**：接收 32×32 的灰度图像。
2. **卷积层 (C1)**：使用 6 个 5×5 的卷积核，输出特征图大小为 28×28。
3. **池化层 (S2)**：2×2 平均池化，步长为 2，输出大小为 14×14。
4. **卷积层 (C3)**：使用 16 个 5×5 的卷积核，输出大小为 10×10。
5. **池化层 (S4)**：2×2 平均池化，步长为 2，输出大小为 5×5。
6. **全连接层 (C5)**：包含 120 个神经元，将卷积特征展开。
7. **全连接层 (F6)**：包含 84 个神经元，激活函数通常使用 Sigmoid 或 Tanh。
8. **输出层 (Output)**：10 个神经元，对应数字 0-9。

LeNet-5 的核心设计理念在于通过局部感受野使卷积核仅关注图像的局部区域，从而有效提取边缘和角点等局部特征。配合权值共享机制，同一特征图上的神经元能够共享卷积参数，这不仅极大减少了模型的参数总量，还显著降低了过拟合的风险。此外，通过下采样技术，池化层在降低特征图分辨率、减少计算量的同时，增强了模型对图像形变和平移的鲁棒性。这种交替使用卷积与池化的层级特征提取方式，使网络能够从基础线条逐步抽象出复杂的数字轮廓。在昇腾 310B 上实现 LeNet-5，可以进一步验证 `torch_npu` 对卷积算子（Conv2d）和池化算子（AvgPool2d）的硬件加速支持。具体代码如下：

```python
import os
import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import TensorDataset, DataLoader
from torchvision import datasets
import matplotlib.pyplot as plt
from tqdm.auto import tqdm, trange

# 指定使用昇腾 NPU 设备
# 在昇腾 AI 处理器上运行 PyTorch 代码时，需要指定设备为 "npu"
device = torch.device("npu")

class LeNet(nn.Module):
    """
    经典的 LeNet-5 网络结构实现
    LeNet-5 是一个简单的卷积神经网络，包含两个卷积层和三个全连接层
    """
    def __init__(self):
        super().__init__()
        # 第一层卷积：输入通道1（灰度图），输出通道6，卷积核5x5
        # padding默认是0，stride默认是1
        self.conv1 = nn.Conv2d(1, 6, kernel_size=5)
        # 池化层：2x2 窗口，步长为2，用于下采样，减少特征图尺寸
```

```
        self.pool = nn.AvgPool2d(2, 2)
        # 第二层卷积：输入通道6，输出通道16，卷积核5x5
        self.conv2 = nn.Conv2d(6, 16, kernel_size=5)
        # 全连接层：输入特征 16*4*4，输出120
        # 这里的 16*4*4 是经过两次卷积和池化后的特征图大小展平后的维度
        # 原始 28x28 -> conv1(5x5) -> 24x24 -> pool(2x2) -> 12x12
        # -> conv2(5x5) -> 8x8 -> pool(2x2) -> 4x4
        self.fc1 = nn.Linear(16 * 4 * 4, 120)
        # 全连接层：输入120，输出84
        self.fc2 = nn.Linear(120, 84)
        # 输出层：输入84，输出10（对应 MNIST 的10个类别 0-9）
        self.fc3 = nn.Linear(84, 10)

    def forward(self, x):
        # 卷积 -> ReLU 激活 -> 池化
        # x shape: [batch, 1, 28, 28] -> [batch, 6, 12, 12]
        x = torch.relu(self.conv1(x))
        x = self.pool(x)
        # 卷积 -> ReLU 激活 -> 池化
        # x shape: [batch, 6, 12, 12] -> [batch, 16, 4, 4]
        x = torch.relu(self.conv2(x))
        x = self.pool(x)
        # 展平特征图为一维向量，用于全连接层
        # x shape: [batch, 16, 4, 4] -> [batch, 16*4*4]
        x = x.view(x.size(0), -1)
        # 全连接层 -> ReLU
        x = torch.relu(self.fc1(x))
        x = torch.relu(self.fc2(x))
        # 输出层，通常配合 CrossEntropyLoss 使用，不需要 softmax
        x = self.fc3(x)
        return x

def plot_metrics(script_dir, train_acc_history, test_acc_history, train_loss_history,
    ↪ test_loss_history):
    """
    绘制并保存训练过程中的精度和损失曲线
    """
    plt.figure(figsize=(10, 4))
    # Accuracy subplot (左图：精度曲线)
    plt.subplot(1, 2, 1)
    plt.plot(train_acc_history, label="Train Acc")
    plt.plot(test_acc_history, label="Test Acc")
    plt.xlabel("Epoch"); plt.ylabel("Accuracy"); plt.title("Accuracy"); plt.legend()
    # Loss subplot (右图：损失曲线)
    plt.subplot(1, 2, 2)
    plt.plot(train_loss_history, label="Train Loss")
    plt.plot(test_loss_history, label="Test Loss")
    plt.xlabel("Epoch"); plt.ylabel("Loss"); plt.title("Loss"); plt.legend()
    plt.tight_layout()
    # 保存图片到脚本所在目录
    out_path = os.path.join(script_dir, "accuracy_loss_curve.png")
    plt.savefig(out_path, bbox_inches="tight")
    print(f"Saved accuracy & loss curve to: {out_path}")
```

```
78
79  def load_data(script_dir, batch_size=256):
80      """
81      加载 MNIST 数据集并进行预处理
82      """
83      # 数据存储路径
84      data_root = os.path.join(script_dir, "data")
85      # 使用 torchvision.datasets 下载并加载 MNIST 数据
86      train_ds = datasets.MNIST(data_root, train=True, download=True)
87      test_ds = datasets.MNIST(data_root, train=False, download=True)
88
89      # 预处理:
90      # 1. unsqueeze(1): 增加通道维度 [N, H, W] -> [N, 1, H, W]
91      # 2. to(torch.float16): 转换为半精度浮点数，NPU 上 float16 计算性能更好
92      # 3. div_(255.0): 像素值归一化到 [0, 1] 区间
93      x_train = train_ds.data.unsqueeze(1).to(torch.float16).div_(255.0)
94      y_train = train_ds.targets.to(torch.int64)
95      x_test = test_ds.data.unsqueeze(1).to(torch.float16).div_(255.0)
96      y_test = test_ds.targets.to(torch.int64)
97
98      # 构建 DataLoader，用于批量加载数据
99      # shuffle=True 表示训练时打乱数据顺序
100     train_loader = DataLoader(TensorDataset(x_train, y_train), batch_size=batch_size,
        ↪ shuffle=True)
101     test_loader = DataLoader(TensorDataset(x_test, y_test), batch_size=batch_size, shuffle
        ↪ =False)
102     return train_loader, test_loader
103
104 def train_and_eval(train_loader, test_loader, epochs=10, lr=0.01):
105     """
106     执行模型训练与验证循环
107     """
108     # 初始化模型并迁移至 NPU 设备
109     # .half() 将模型参数转换为 float16，利用 NPU 的 Tensor Core 加速
110     model = LeNet().half().to(device)
111     # 定义损失函数: 交叉熵损失，适用于多分类问题
112     criterion = nn.CrossEntropyLoss()
113     # 定义优化器: 随机梯度下降 (SGD)
114     optimizer = optim.SGD(model.parameters(), lr=lr, momentum=0.9)
115
116     train_acc_history = []
117     test_acc_history = []
118     train_loss_history = []
119     test_loss_history = []
120
121     # 使用 trange 展示总进度条
122     t = trange(epochs, desc="Epochs")
123     for epoch in t:
124         # --- 训练阶段 ---
125         model.train() # 设置模型为训练模式（启用 Dropout, BatchNorm 等）
126         correct = 0; total = 0
127         running_loss = 0.0
128         for inputs, labels in train_loader:
```

```
            # 数据迁移至 NPU 并转为半精度，标签保持 int64 或 int32
            inputs = inputs.to(device)
            labels = labels.to(device)

            # 前向传播
            outputs = model(inputs)
            # 计算损失时将输出转回 float32 以保证数值稳定性（混合精度训练的常见做法）
            loss = criterion(outputs.float(), labels)

            # 反向传播与优化
            optimizer.zero_grad() # 清空梯度
            loss.backward()       # 计算梯度
            optimizer.step()      # 更新参数

            # 统计损失与精度
            running_loss += loss.item() * labels.size(0)
            # 获取预测结果（最大概率对应的索引）
            preds = outputs.float().argmax(dim=1)
            correct += (preds == labels).sum().item()
            total += labels.size(0)

        # 计算本 epoch 的平均训练精度和损失
        train_acc = correct / total
        train_loss = running_loss / total
        train_acc_history.append(train_acc)
        train_loss_history.append(train_loss)

        # --- 测试阶段 ---
        model.eval() # 设置模型为评估模式
        correct_t = 0; total_t = 0
        running_loss_t = 0.0
        with torch.no_grad(): # 验证时不计算梯度，节省显存和计算资源
            for inputs, labels in test_loader:
                inputs = inputs.to(device).half(); labels = labels.to(device)
                outputs = model(inputs)
                loss_t = criterion(outputs.float(), labels)
                running_loss_t += loss_t.item() * labels.size(0)
                preds = outputs.float().argmax(dim=1)
                correct_t += (preds == labels).sum().item()
                total_t += labels.size(0)

        # 计算本 epoch 的平均测试精度和损失
        test_acc = correct_t / total_t
        test_loss = running_loss_t / total_t
        test_acc_history.append(test_acc)
        test_loss_history.append(test_loss)
        t.set_postfix(train_acc =f"{train_acc:.4f}", val_loss=f"{test_acc:.4f}", lr=f'{
    ↪ optimizer.param_groups[0]["lr"]:.6f}')

    # 返回训练/测试指标历史
    return train_acc_history, test_acc_history, train_loss_history, test_loss_history

if __name__ == "__main__":
```

```
    # 获取当前脚本目录，方便保存文件
    script_dir = os.path.dirname(os.path.abspath(__file__))
    # 加载数据
    train_loader, test_loader = load_data(script_dir, batch_size=256)
    # 开始训练
    train_acc_history, test_acc_history, train_loss_history, test_loss_history =
    ↪ train_and_eval(
        train_loader, test_loader, epochs=10, lr=0.01
    )
    # 绘制结果
    plot_metrics(script_dir, train_acc_history, test_acc_history, train_loss_history,
    ↪ test_loss_history)
```

注意：运行此代码前，请确保已安装 `torchvision` 库。安装时必须严格注意 `torchvision` 与 `PyTorch` 的版本对应关系，版本不匹配可能导致无法导入或运行时错误。

以下是昇腾 PyTorch 插件支持的 PyTorch 版本与其对应的 TorchVision 版本对照表：

PyTorch 版本	TorchVision 版本
2.1.0	0.16.0
2.6.0	0.21.0
2.7.1	0.22.0
2.8.0	0.23.0

例如，若当前环境安装的是 PyTorch 2.8.0，则需安装 TorchVision 0.19.0：

```
pip install torchvision==0.23.0
```

在昇腾 310B 开发板上运行以上代码，可以得到以下的结果：

```
  warnings.warn(f"Warning: The {path} owner does not match the current owner.")
Epochs: 100%|████████████████████████████████████████| 10/10 [01:51<00:00, 11.15s
    ↪ /it, lr=0.010000, train_acc=0.9782, val_loss=0.9800]
Saved accuracy & loss curve to: /home/HwHiAiUser/Documents/samples/chapter3/LeNet/
    ↪ accuracy_loss_curve.png
```

并生成如下图所示的部分预测结果可视化：

在本示例中，尽管使用了昇腾 NPU 进行加速，但观察到的训练速度与 CPU 相比并未展现出量级上的优势，这主要是由任务规模与硬件特性共同决定的。MNIST 数据集中的图像分辨率仅为 28x28，且 LeNet-5 网络的参数量和计算深度相对较低，这种轻量级的计算负载难以充分填满 NPU 内部众多的并行计算单元，导致硬件资源处于不饱和状态。与此同时，在 NPU 上执行任务涉及到数据从主机内存到设备显存的搬运开销，以及 CANN 算子库在调用时的指令调度与同步耗时，当计算本身的耗时极短时，这些固有的通信与管理开销便成为了性能瓶颈。只有在面对如 ImageNet 等大规模数据集或 ResNet、

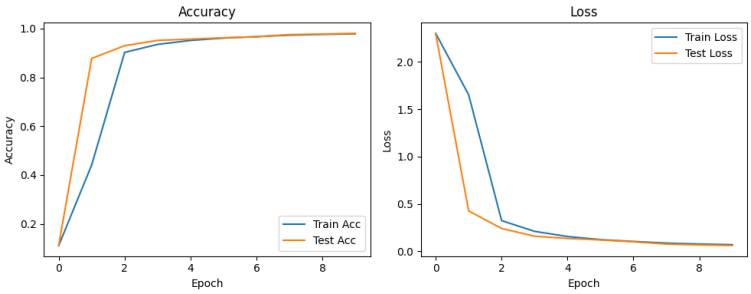


图 3.6. 部分预测结果

Transformer 等深层复杂模型时，NPU 强大的张量计算能力才能在抵消掉调度开销后，通过极高的并行度显著缩短整体训练周期。

值得注意的是，本示例对原始 LeNet-5 进行了关键的适配调整。首先，我们将激活函数由 Sigmoid 替换为 ReLU；其次，我们将模型计算精度由 FP32 全精度调整为 FP16 半精度。这是因为在当前硬件环境下，FP32 模型在编译阶段可能遇到兼容性阻碍导致无法运行。同时实验表明，在半精度计算模式下，Sigmoid 函数极易因数值精度不足导致训练效果显著下降，而 ReLU 则能维持更好的数值稳定性与收敛效果。

FP16（半精度浮点数）在表示范围和精度上存在显著限制，这直接影响了激活函数的数值表现。由于 FP16 仅有 16 位，其有效数字位数极少且数值范围较窄，在处理涉及指数运算和除法的 Sigmoid 函数时极易出现问题。在前向计算中，当输入值的绝对值稍大时，FP16 的精度不足以区分计算过程中的微小差别，导致输出迅速进入饱和区并锁定为 0 或 1。更严重的是，在反向传播过程中，Sigmoid 的导数在接近饱和区时会趋近于 0，这些微小的梯度值在 FP16 的精度下极易发生下溢出而变成纯 0，从而导致权重停止更新，使网络训练陷入停滞。

相比之下，ReLU 函数展现出了极佳的数值稳定性。作为一种分段线性函数，ReLU 的计算过程仅涉及简单的比较操作，完全避开了复杂的非线性变换带来的精度损失。在正值区域，ReLU 的导数恒定为 1，这种特性确保了梯度在传播过程中不会因为数值过小而消失，也不会受到 FP16 精度限制的影响。这种天然的鲁棒性使得 ReLU 非常契合 NPU 等强调低精度、高吞吐计算的硬件架构，能够有效保障模型在半精度模式下的收敛效果。

最后值得注意的一点是模型精度转换（FP32 转 FP16）与设备传输的操作顺序。代码实现上通常有两种方式：一种是先在 CPU 端将模型转换为半精度，再传输至 NPU，写法为 `model = LeNet().half().to(device)`；另一种是先将模型传输至 NPU，再进行精度转换，写法为 `model = LeNet().to(device).half()`。虽然这两种写法最终得到的模型状态一致，但在昇腾 310B 的 `torch_npu` 环境下，其底层执行流程存在差异。实测表明，由于 NPU 涉及图编译过程，**优先在 CPU 端完成精度转换（即第一种写法）可以显著减少图编译的开销**，从而缩短程序的整体启动时间。不过，一旦编译完成进入稳定运行阶段，两者的计算效率差异则微乎其微。

3.5. torch_npu 插件兼容性测试套件

为了全面评估 torch_npu 在昇腾 310B 上的算子支持度与数值稳定性，我们在 samples/chapter3/test 目录下提供了一套兼容性测试脚本。这些测试旨在帮助开发者快速验证当前环境（CANN 版本 + PyTorch 版本 + 硬件）是否满足模型迁移的基本要求，并识别潜在的算子不支持或精度异常问题。

3.5.1. 测试目标与验证维度

在利用 torch_npu 插件开发神经网络应用时，我们注意到该插件在昇腾 310B 上的兼容性仍有提升空间。面对复杂的网络结构或特定的算子组合，图编译阶段偶尔会出现不稳定的情况。此外，部分算子在特定精度下的表现也需关注，例如 Sigmoid 激活函数在 FP16 精度下可能引发模型收敛异常。鉴于此，针对 AlexNet、VGG、ResNet 等经典网络中常用的核心算子进行全面测试显得尤为必要。为此，我们开发了一套轻量级的测试套件。

本测试套件的首要目标是验证核心算子在昇腾 NPU 上的支持度与执行稳定性。通过覆盖卷积（Conv2d）、全连接（Linear）、矩阵乘法（MatMul）等深度学习中最基础且关键的算子，旨在检查这些指令能否被正确分发至 NPU 后端并顺利执行。这不仅有助于判断当前软硬件环境是否具备运行复杂模型的基础能力，对于快速排查因算子缺失或版本不兼容导致的运行时错误也至关重要。

其次，精度对齐是评估迁移质量的关键指标。测试脚本通过对比同一组输入数据在 NPU 与 CPU 上的计算结果，严格量化两者之间的数值差异。通常情况下，在 FP32 精度下，我们期望误差控制在极小的范围内（如小于 1e-4），以确保模型迁移后推理与训练结果的可靠性。这种对比验证能有效发现因硬件架构差异或算子实现细节不同而引发的数值漂移问题。

最后，测试还致力于探索不同边界条件下的表现。这包括验证 FP16、FP32 等不同数据类型的兼容性，特别是针对昇腾 310B 对 FP64（双精度）支持有限的特性进行实测，以及评估在不同 Tensor 形状和维度下算子的内存占用与计算效率。通过这些多维度的测试，开发者可以更清晰地了解硬件的性能边界与最佳实践配置。

3.5.2. 测试套件结构详解

本测试套件基于 pytest 框架构建，针对不同精度（FP16/FP32）进行了分层验证，主要包含以下核心组成部分：

1. 测试环境配置 (conftest.py)

conftest.py 脚本负责构建稳健的测试环境。它利用 pytest 的钩子函数（Hooks）机制，实现了一套实时日志记录系统。针对嵌入式开发板在长时间批量测试中可能出现的网络波动或系统挂起导致终端输出丢失的问题，该脚本定义了 pytest_runtest_setup 和 pytest_runtest_logreport 等钩子。这些钩子确保在每个测试用例开始和结束时，立即将测试

节点 ID 及运行结果（PASSED/FAILED）写入 pytest_realtime_log.txt 文件。为了防止系统崩溃导致数据丢失，写入操作调用了 os.fsync 强制刷新磁盘。此外，pytest_terminal_summary 会在测试结束时统计并输出通过、失败及跳过的用例总数，生成详尽且持久化的测试报告，极大便利了无人值守场景下的故障排查。

2. 基础算子运算测试（test_float16_ops.py / test_float32_ops.py）

基础算子测试剥离了复杂的神经网络层封装，直接验证深度学习中最底层的张量加法（Add）和矩阵乘法（MatMul）。这两个脚本分别对应 FP16（半精度）和 FP32（单精度），旨在直接检验昇腾 NPU 硬件底层的算术逻辑单元（ALU）与矩阵计算单元（Cube Core）的计算正确性。通过绕过 nn.Module 等高层抽象，这些测试作为排查底层硬件故障或驱动问题的最小功能单元，有助于区分框架层面的算子映射错误与硬件层面的计算异常。

- **test_float32_ops.py**：关注高精度计算的一致性。它在 CPU 和 NPU 上执行相同的 FP32 运算，要求两者误差控制在极小范围内（如加法 1e-4，矩阵乘法 1e-3），确保 NPU 单精度计算路径的数值稳定。
- **test_float16_ops.py**：针对边缘计算常用的半精度推理场景。采用 “CPU FP32 计算作为真值” 的策略，即在 CPU 上使用高精度 FP32 运算，NPU 使用 FP16 运算，最后将 NPU 结果转回 FP32 进行对比。这种方式既验证了 NPU 的半精度计算能力，又通过放宽容差（1e-2）合理包容了精度量化带来的正常误差。

3. 神经网络层测试（test_nn_layers_float16.py / test_nn_layers_float32.py）

这是测试套件的核心，旨在全面验证常用深度学习算子在昇腾 NPU 上的表现。测试覆盖了特征提取与分类的关键组件：* **核心层**：nn.Linear 和 nn.Conv2d 的测试用例涵盖了 LeNet、AlexNet、VGG 及 ResNet 等经典架构的典型参数配置（输入尺寸、卷积核大小、步长及填充）。* **激活与归一化**：囊括 ReLU、Sigmoid、Tanh、LeakyReLU、GELU、Softmax 等激活函数及 BatchNorm2d，确保非线性变换与数据分布调整的准确性。* **池化与辅助层**：验证 AvgPool2d 和 MaxPool2d 的基本功能及 count_include_pad、ceil_mode 等特定参数，同时包含 Dropout 和 Flatten 的行为验证。

为了适应不同场景，套件提供了双重验证策略：* **test_nn_layers_float32.py**：侧重高精度数值一致性，要求 NPU 结果与 CPU 基准误差极小（1e-3），确保推理精确度。* **test_nn_layers_float16.py**：模拟混合精度推理，输入和模型转为半精度在 NPU 执行，结果转回 FP32 对比。考虑到量化损失，容差适度放宽（1e-2 至 1e-1），重点验证低精度模式下的逻辑正确性与功能完备性。此外，为防止嵌入式设备内存溢出（OOM），每个测试类均实现了 teardown_method，在用例执行后自动清理 NPU 缓存。

3.5.3. 测试结果摘要

运行该测试套件前，请确保在 npu 虚拟环境中已安装 pytest：

```
pip3 install pytest
```

随后在终端进入相应的 `test` 文件夹并运行：

```
pytest -v ./
```

在昇腾 310B（CANN 8.3.RC1 + PyTorch 2.8.0）环境下，典型测试结果如下：

```
============================= test session starts ==============================
...
FAILED test_nn_layers_float16.py::TestNNLayersFloat16::
    ↪ test_avgpool_float16_default_behavior[3-1-1]
FAILED test_nn_layers_float16.py::TestNNLayersFloat16::
    ↪ test_avgpool_float16_default_behavior[3-2-1]
FAILED test_nn_layers_float16.py::TestNNLayersFloat16::test_avgpool_float16[True-3-1-1]
FAILED test_nn_layers_float16.py::TestNNLayersFloat16::test_avgpool_float16[True-3-2-1]
FAILED test_nn_layers_float32.py::TestNNLayersFloat32::
    ↪ test_maxpool_float32_default_behavior[2-2-0]
FAILED test_nn_layers_float32.py::TestNNLayersFloat32::
    ↪ test_maxpool_float32_default_behavior[3-2-0]
FAILED test_nn_layers_float32.py::TestNNLayersFloat32::
    ↪ test_maxpool_float32_default_behavior[3-2-1]
FAILED test_nn_layers_float32.py::TestNNLayersFloat32::test_maxpool_float32[False-2-2-0]
FAILED test_nn_layers_float32.py::TestNNLayersFloat32::test_maxpool_float32[False-3-2-0]
FAILED test_nn_layers_float32.py::TestNNLayersFloat32::test_maxpool_float32[False-3-2-1]
FAILED test_nn_layers_float32.py::TestNNLayersFloat32::test_maxpool_float32[True-2-2-0]
FAILED test_nn_layers_float32.py::TestNNLayersFloat32::test_maxpool_float32[True-3-2-0]
FAILED test_nn_layers_float32.py::TestNNLayersFloat32::test_maxpool_float32[True-3-2-1]
=================== 76 passed, 13 failed in 123.45s ====================
```

测试日志显示，绝大多数基础算子（Add, MatMul）和核心网络层（Linear, Conv2d, BatchNorm, Activations）在 FP16 和 FP32 模式下均通过验证，证明 NPU 基本计算功能正常。但测试也暴露了两个显著异常，需重点关注：

- FP16 平均池化层 (AvgPool) 的边界精度问题在 `test_nn_layers_float16.py` 的测试中，`AvgPool2d` 算子表现出特定的边界精度异常，具体体现为当 `kernel_size=3` 且 `padding=1` 时测试均告失败，而 `kernel_size=2` 且无填充时则能顺利通过。这一现象表明 NPU 在处理半精度（FP16）池化边界填充（Padding）时，累加计算的精度损失可能超出了预设容差范围，或者底层硬件对 Padding 区域 “0” 值的处理逻辑与 CPU 存在微小差异。因此，开发者在推理代码中若使用 FP16 模式且包含带 Padding 的奇数核 AvgPool 层，需特别警惕潜在的精度下降风险。
- FP32 最大池化层 (MaxPool) 的系统性失效在 `test_nn_layers_float32.py` 中，所有 `MaxPool2d` 测试用例（共 9 个）全部失败，与 FP16 模式下全部通过的表现形成鲜明对比，这揭示了一个严重的系统性故障。FP16 正常而 FP32 全面失败，极可能是当前版本 CANN 驱动或 PyTorch 插件在 FP32 格式 MaxPool 算子实现上存在 Bug，亦或是数据排布未对齐导致计算错误甚至产生 NaN/Inf。鉴于此为高优先级阻碍性

问题（Blocker），建议暂时避免在 NPU 上使用 FP32 格式的 MaxPool，或将其回退至 CPU 执行，直至驱动版本修复。

开发者在迁移自定义模型前，建议优先运行此测试套件以排查环境潜在问题。

3.6. 现代卷积神经网络的移植

3.6.1. AlexNet

AlexNet 是由 Alex Krizhevsky、Ilya Sutskever 和 Geoffrey Hinton 在 2012 年提出的卷积神经网络。它在当年的 ImageNet 大规模视觉识别挑战赛（ILSVRC）中以压倒性的优势夺冠，将 Top-5 错误率降低到了 15.3%，远超第二名的 26.2%。AlexNet 的成功标志着深度学习在计算机视觉领域的统治地位的确立，引发了卷积神经网络（CNN）研究的热潮。

AlexNet 网络结构

AlexNet 的网络结构比之前的 LeNet 更深、更宽。AlexNet 主要包含以下特点：1. **层数**：包含 8 层神经网络，其中前 5 层是卷积层，后 3 层是全连接层。2. **激活函数**：首次在 CNN 中成功使用了 ReLU（Rectified Linear Unit）激活函数，解决了深层网络训练时的梯度消失问题，并加速了收敛。3. **Dropout**：在全连接层引入了 Dropout 技术，随机忽略一部分神经元，有效防止了过拟合。4. **数据增强**：使用了图像翻转、裁剪等数据增强技术来扩充数据集。5. **多 GPU 训练**：由于当时显存限制，AlexNet 被设计为在两块 GTX 580 GPU 上并行训练。6. **重叠池化（Overlapping Pooling）**：AlexNet 使用了重叠的最大池化（Max Pooling）。传统的池化层通常是不重叠的（步长等于池化核大小），而 AlexNet 使用了 3×3 的池化核和 2 的步长，这种重叠池化稍微减轻了过拟合。

原始的 AlexNet 是为 ImageNet 数据集设计的，但在昇腾 310B 开发板上使用庞大的 ImageNet 进行训练会消耗大量时间与算力。ImageNet 是一个庞大的视觉数据库，包含超过 1400 万张标注图片，涵盖 2 万多个类别，其常用的 ILSVRC 子集也有 1000 个类别和 128 万张训练图片，且图片分辨率较高（通常裁剪为 224×224）。相比之下，CIFAR-10 数据集规模较小，仅包含 60000 张 32×32 的彩色图像，分为 10 个类别（如飞机、汽车、鸟类等），每类 6000 张。虽然 CIFAR-10 的图像分辨率和类别数远小于 ImageNet，但它保留了自然图像的复杂性，非常适合用于快速验证模型结构和硬件移植的可行性。为了更高效地验证 AlexNet 在昇腾 310B 上的迁移性能，本节改用 CIFAR-10 数据集进行实验。

为此，我们需要对 AlexNet 的结构进行适配性修改：首先，将网络输入尺寸从 $3 \times 224 \times 224$ 调整为 $3 \times 32 \times 32$，并将全连接层的输出类别数从 1000 改为 10。

此外,在上一节的测试中我们发现,昇腾 310B 的 PyTorch 插件目前尚未支持 `MaxPool2d` 算子，因此我们将该算子替换为 `AvgPool2d`。这一改动虽然是为了适配硬件限制，但也会对模型特性产生一定影响：**最大池化（Max Pooling）**倾向于提取图像中最显著的特征（如

纹理、边缘), 具有一定的平移不变性, 通常能保留较强的特征响应; 而 **平均池化（Average Pooling）**则是计算区域内的平均值，倾向于保留背景信息和平滑图像，可能会模糊掉一些尖锐的特征细节。在 CIFAR-10 这种低分辨率数据集上，使用平均池化可能会导致模型对关键特征的捕捉能力略微下降，从而轻微影响最终的分类准确率，但它能保证模型在当前硬件环境下的顺利运行。修改后的 AlexNet 网络结构如下图所示:

AlexNet 的代码实现

为了验证 AlexNet 在昇腾 310B 上移植的可行性，我们首先建立了一个测试流程。利用 `pytest` 框架，我们对网络的每一层结构进行了逐一验证，重点对比了 `torch_npu`（NPU 后端）与 CPU 计算结果的一致性，同时检测了网络算子的硬件兼容性。为此，我们编写了专门的测试例程，分别验证了 AlexNet 在 FP16（半精度）和 FP32（全精度）模式下的运行正确性。相关测试代码均位于 `samples/chapter2/AlexNet/test` 目录下。经 `pytest` 测试确认，无论是 FP32 还是 FP16 精度，模型在昇腾 310B 上的计算结果均正确无误。尽管前述的逐层单元测试确认了 AlexNet 在 FP16 和 FP32 精度下的计算正确性，但在实际的完整模型训练中，硬件资源的限制成为了关键因素。实测表明，在 **8T 算力 8GB 内存版本**的昇腾 310B 开发板上进行 FP32 全精度训练时，由于图编译阶段对内存消耗较大，极易遭遇编译失败的问题；而在 **8T 算力 16GB 内存版本**的开发板上，FP32 全精度训练则能顺利跑通。基于上述适配后的网络结构，以下提供了针对 CIFAR-10 数据集的完整 FP16 半精度训练与验证代码:

```python
import os
import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import TensorDataset, DataLoader
from torchvision import datasets
import matplotlib.pyplot as plt
from tqdm.auto import tqdm, trange
from torchvision import transforms

device = torch.device("npu")

class AlexNet(nn.Module):
    def __init__(self, num_classes=10):
        super(AlexNet, self).__init__()
        self.features = nn.Sequential(
            # Layer 1
            nn.Conv2d(3, 96, kernel_size=11, stride=4, padding=1),
            nn.ReLU(inplace=True),
            nn.AvgPool2d(kernel_size=3, stride=2),
            # Layer 2
            nn.Conv2d(96, 256, kernel_size=5, padding=2),
            nn.ReLU(inplace=True),
            nn.AvgPool2d(kernel_size=3, stride=2),
```

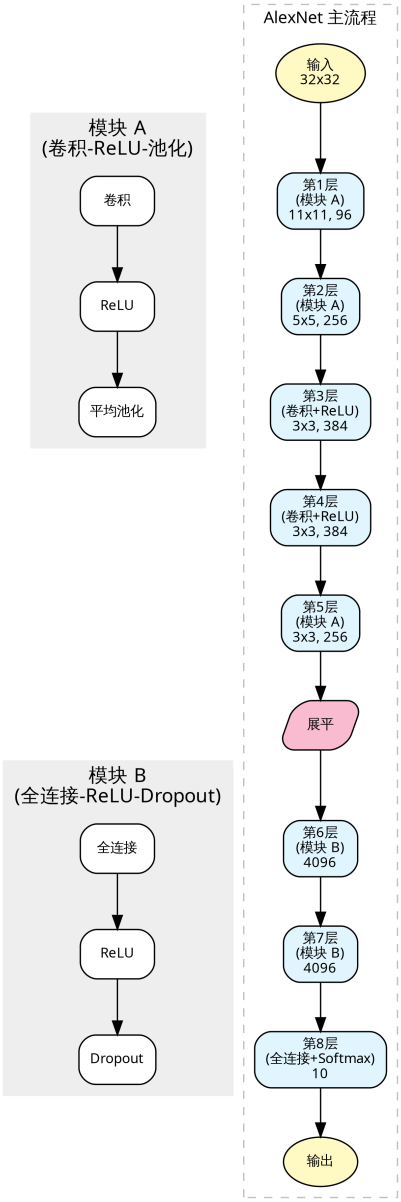

AlexNet 主流程
输入
32x32
第1层
(模块 A)
11x11, 96
第2层
(模块 A)
5x5, 256
第3层
(卷积+ReLU)
3x3, 384
第4层
(卷积+ReLU)
3x3, 384
第5层
(模块 A)
3x3, 256
展平
第6层
(模块 B)
4096
第7层
(模块 B)
4096
第8层
(全连接+Softmax)
10
输出
模块 A
(卷积-ReLU-池化)
卷积
ReLU
平均池化
模块 B
(全连接-ReLU-Dropout)
全连接
ReLU
Dropout

图 3.7. AlexNet

```
            # Layer 3
            nn.Conv2d(256, 384, kernel_size=3, padding=1),
            nn.ReLU(inplace=True),
            # Layer 4
            nn.Conv2d(384, 384, kernel_size=3, padding=1),
            nn.ReLU(inplace=True),
            # Layer 5
            nn.Conv2d(384, 256, kernel_size=3, padding=1),
            nn.ReLU(inplace=True),
            nn.AvgPool2d(kernel_size=3, stride=2),
        )

        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(6400, 4096),
            nn.ReLU(inplace=True),
            nn.Dropout(0.5),
            nn.Linear(4096, 4096),
            nn.ReLU(inplace=True),
            nn.Dropout(0.5),
            nn.Linear(4096, num_classes),
        )

    def forward(self, x):
        x = self.features(x)
        x = self.classifier(x)
        return x

def load_data(script_dir, batch_size=4):
    """
    加载 CIFAR-10 数据集并进行预处理
    """
    # 引入 transforms 用于图像变换

    # 数据存储路径
    data_root = os.path.join(script_dir, "data")

    # 定义预处理流程:
    # 1. Resize: 将 CIFAR-10 的 32x32 图像调整为 224x224，以匹配 AlexNet 输入要求
    # 2. ToTensor: 将图像数据转换为 Tensor (C, H, W)，并将像素值归一化到 [0, 1]
    # 3. Normalize: 对 RGB 三个通道进行标准化
    transform = transforms.Compose([
        transforms.Resize((224, 224)),
        transforms.ToTensor(),
        transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5)),
        transforms.Lambda(lambda x: x.half())
    ])

    # 使用 torchvision.datasets 下载并加载 CIFAR-10 数据
    # 传入 transform 参数，DataLoader 读取数据时会自动进行 Resize 和 ToTensor
    train_ds = datasets.CIFAR10(data_root, train=True, download=True, transform=transform)
    test_ds = datasets.CIFAR10(data_root, train=False, download=True, transform=transform)

```

```
    # 构建 DataLoader
    # 相比原代码一次性加载到内存，这里使用 lazy loading 方式，避免 Resize 后占用过多内存
    train_loader = DataLoader(train_ds, batch_size=batch_size, shuffle=True)
    val_loader = DataLoader(test_ds, batch_size=batch_size, shuffle=False)
    return train_loader, val_loader

def train_and_eval(train_loader, val_loader, epochs=10, lr=0.01):
    """
    执行模型训练与验证循环
    """
    # 初始化模型并迁移至 NPU 设备
    model = AlexNet().half().to(device)
    # 定义损失函数：交叉熵损失，适用于多分类问题
    criterion = nn.CrossEntropyLoss()
    # 定义优化器：随机梯度下降 (SGD)
    optimizer = optim.SGD(model.parameters(), lr=lr)

    train_acc_history = []
    val_acc_history = []
    train_loss_history = []
    val_loss_history = []

    # 使用 trange 展示总进度条
    t = trange(epochs, desc="Epochs")
    for epoch in t:
        # --- 训练阶段 ---
        model.train() # 设置模型为训练模式（启用 Dropout, BatchNorm 等）
        correct = 0; total = 0
        running_loss = 0.0
        for inputs, labels in train_loader:
            # 数据迁移至 NPU，保持 float32
            inputs = inputs.to(device); labels = labels.to(device)

            # 前向传播
            outputs = model(inputs)
            # 计算损失
            loss = criterion(outputs, labels)

            # 反向传播与优化
            optimizer.zero_grad() # 清空梯度
            loss.backward()       # 计算梯度
            optimizer.step()      # 更新参数

            # 统计损失与精度
            running_loss += loss.item() * labels.size(0)
            # 获取预测结果（最大概率对应的索引）
            preds = outputs.argmax(dim=1)
            correct += (preds == labels).sum().item()
            total += labels.size(0)

        # 计算本 epoch 的平均训练精度和损失
        train_acc = correct / total
        train_loss = running_loss / total
```

```
        train_acc_history.append(train_acc)
        train_loss_history.append(train_loss)

        # --- 验证阶段 ---
        model.eval() # 设置模型为评估模式
        correct_v = 0; total_v = 0
        running_loss_v = 0.0
        with torch.no_grad(): # 验证时不计算梯度，节省显存和计算资源
            for inputs, labels in val_loader:
                inputs = inputs.to(device); labels = labels.to(device)
                outputs = model(inputs)
                loss_v = criterion(outputs, labels)
                running_loss_v += loss_v.item() * labels.size(0)
                preds = outputs.argmax(dim=1)
                correct_v += (preds == labels).sum().item()
                total_v += labels.size(0)

        # 计算本 epoch 的平均验证精度和损失
        val_acc = correct_v / total_v
        val_loss = running_loss_v / total_v
        val_acc_history.append(val_acc)
        val_loss_history.append(val_loss)
        t.set_postfix(train_acc =f"{train_acc:.4f}", val_loss=f"{val_loss:.4f}", val_acc=f
    ↪ "{val_acc:.4f}", lr=f'{optimizer.param_groups[0]["lr"]:.6f}')

    # 返回训练/验证指标历史
    return train_acc_history, val_acc_history, train_loss_history, val_loss_history

def plot_metrics(script_dir, train_acc, val_acc, train_loss, val_loss):
    """
    绘制训练和验证的精度与损失曲线
    """
    plt.figure(figsize=(12, 5))

    # 绘制精度曲线
    plt.subplot(1, 2, 1)
    plt.plot(train_acc, label='Train Accuracy')
    plt.plot(val_acc, label='Validation Accuracy')
    plt.title('Accuracy over Epochs')
    plt.xlabel('Epoch')
    plt.ylabel('Accuracy')
    plt.legend()

    # 绘制损失曲线
    plt.subplot(1, 2, 2)
    plt.plot(train_loss, label='Train Loss')
    plt.plot(val_loss, label='Validation Loss')
    plt.title('Loss over Epochs')
    plt.xlabel('Epoch')
    plt.ylabel('Loss')
    plt.legend()

    # 保存图片
```

```
    save_path = os.path.join(script_dir, "training_metrics.png")
    plt.savefig(save_path)
    print(f"Metrics plot saved to {save_path}")

if __name__ == "__main__":
    # 获取当前脚本目录，方便保存文件
    script_dir = os.path.dirname(os.path.abspath(__file__))

    # 加载数据
    train_loader, val_loader = load_data(script_dir, batch_size=8)
    # 开始训练
    train_acc_history, val_acc_history, train_loss_history, val_loss_history =
    ↪ train_and_eval(
        train_loader, val_loader, epochs=10, lr=0.01
    )
    # 绘制结果
    plot_metrics(script_dir, train_acc_history, val_acc_history, train_loss_history,
    ↪ val_loss_history)
```

在前面章节创建的 npu 虚拟环境中运行上述代码，昇腾 310B 开发板将输出如下执行结果：

```
Epochs: 100%|
↪ |
↪  10/10 [2:26:50<00:00, 881.05s/it, lr=0.010000, train_acc=0.8195, val_acc=0.7798,
↪ val_loss=0.6680]
```

实验结果对比分析

为了评估昇腾 310B 在运行 AlexNet 模型时的实际性能，我们将上述代码部署至昇腾 310B 开发板（NPU），并引入高性能桌面显卡 GeForce 5090D 作为对比基准。实验设计涵盖了昇腾 310B 与 GeForce 5090D 在 FP16 半精度及 FP32 全精度模式下的表现，以期提供客观的性能参考。

需要特别说明的是，全精度（FP32）训练对内存资源有较高要求。实测表明，**8GB 内存版本**的昇腾 310B 开发板在执行 AlexNet 的 FP32 训练任务时，会因内存不足触发图编译错误（OOM）。因此，若需在昇腾 310B 上进行 FP32 训练，必须使用 **16GB 内存版本**；而 FP16 半精度训练则能有效降低显存占用，在 8GB 版本上即可顺利运行。

各实验配置的终端输出日志记录如下：

- **Ascend 310B (FP16)**

```
Epochs: 100%|          | 20/20 [4:00:05<00:00, 720.29s/it, lr=0.010000,
    ↪ train_acc=0.8770, val_acc=0.8044, val_loss=0.6110]
```

- **Ascend 310B (FP32)**

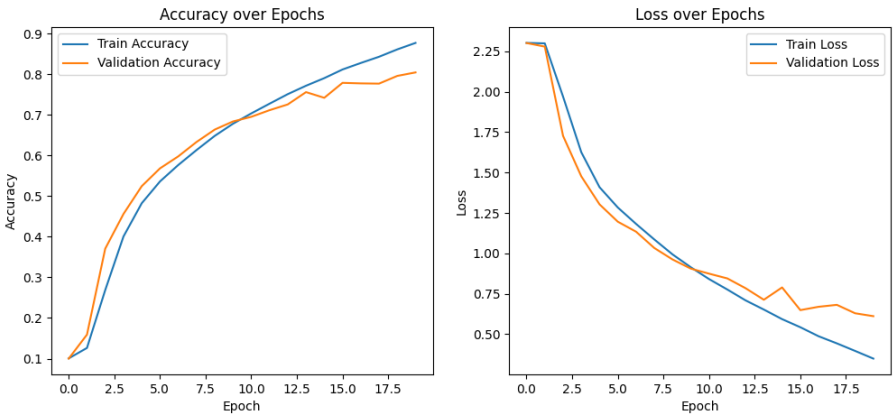


图 3.8. training metrics NPU FP16

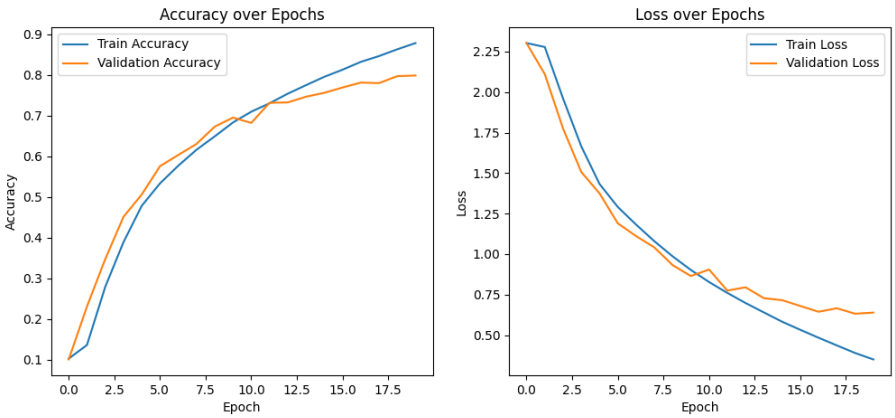


图 3.9. training metrics cuda FP32

```
Epochs: 100%|          | 20/20 [10:52:39<00:00, 1957.99s/it, lr=0.010000, train_acc
    ↪ =0.9693, val_acc=0.8219, val_loss=0.7721]
```

- **GeForce 5090D (FP16)**

```
Epochs: 100%|          | 20/20 [08:38<00:00, 25.93s/it, lr=0.010000,
    ↪ train_acc=0.8787, val_acc=0.7988, val_loss=0.6389]
```

- **GeForce 5090D (FP32)**

```
Epochs: 100%|          | 20/20 [11:01<00:00, 33.08s/it, lr=0.010000,
    ↪ train_acc=0.9688, val_acc=0.8252, val_loss=0.7670]
```

详细的实验数据对比汇总如下表所示：

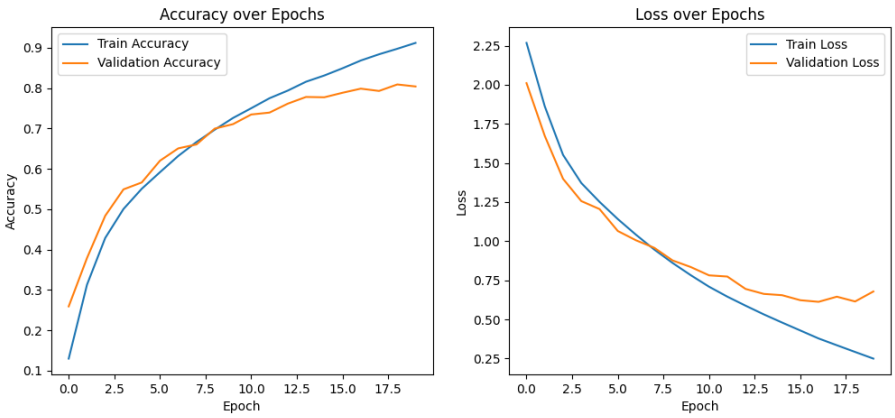


图 3.10. training metrics cuda FP32

硬件平台	精度模式	总耗时 (20 Epochs)	单 Epoch 平均耗时	训练集精度 (Train Acc)	验证集精度 (Val Acc)	验证集损失 (Val Loss)
Ascend 310B	FP16 (半精度)	240.1 min	12.0 min	87.70%	80.44%	0.6110
Ascend 310B	FP32 (全精度)	652.7 min	32.6 min	96.93%	82.19%	0.7721
GeForce 5090D	FP16 (半精度)	8.6 min	0.43 min	87.87%	79.88%	0.6389
GeForce 5090D	FP32 (全精度)	11.0 min	0.55 min	96.88%	82.52%	0.7670

首先，从训练速度来看，虽然昇腾 310B 的训练耗时明显长于桌面级高性能显卡，但这完全符合其硬件定位。昇腾 310B 本质上是一款面向边缘计算场景的低功耗 AI 芯片，其设计初衷是在有限的功耗预算下提供高效的推理能力，而非大规模模型训练。尽管如此，实验结果表明昇腾 310B 依然稳定地完成了整个训练流程，这有力地证明了它不仅能够胜任推理任务，也具备在端侧进行轻量级训练或模型微调的潜力，为边缘侧的持续学习提供了硬件基础。

其次，在精度表现方面，我们可以观察到不同精度与不同硬件平台之间的高度一致性。首先，在 FP32 全精度模式下，昇腾 310B 的验证集准确率达到了 82.19%，与基准设备 GeForce 5090D 的 82.52% 极为接近。这充分验证了昇腾 310B 在全精度计算逻辑上的正确性，未出现因架构差异导致的数值漂移。

其次，对比同平台的 FP16 与 FP32 结果，精度下降是普遍存在的现象：基准设备准确率下降了约 2.6%（从 82.52% 降至 79.88%），而昇腾 310B 下降了约 1.7%（从 82.19% 降至 80.44%）。这种精度损失属于半精度动态范围压缩带来的正常物理特性，但昇腾 310B 的降幅相对更小，说明其半精度算子实现具有良好的数值稳定性。

最后，横向对比 FP16 模式，昇腾 310B（80.44%）的表现与基准设备（79.88%）几乎持平甚至略优，进一步证明了将模型移植到昇腾 NPU 并采用半精度加速时，能够很好地保持模型的收敛特性和最终性能。

综上所述，实验结果证实了昇腾 310B 能够正确且有效地执行 AlexNet 的训练任务，且精度表现符合预期。虽然其训练速度无法与高端桌面显卡相提并论，但考虑到其低功耗特性和边缘部署的定位，该性能足以满足小规模数据集的迁移学习或现场微调需求。对于大规模的深度学习训练任务，更合理的流程应当是在高性能服务器（如昇腾 910 或 GPU 集群）上完成模型训练，随后再将训练好的模型量化或转换至昇腾 310B 上进行高效的推理部署。

3.6.2. VGG

VGG（Visual Geometry Group）网络是由牛津大学的 Karen Simonyan 和 Andrew Zisserman 在 2014 年提出的。它在当年的 ILSVRC 挑战赛中取得了定位任务第一名和分类任务第二名的优异成绩。VGG 的主要贡献在于证明了使用小卷积核并增加网络深度能够有效提升模型性能。

VGG 的网络结构

VGG 的基本结构特点非常简洁：1. **统一的小卷积核**：整个网络全部使用 3×3 的卷积核和 2×2 的最大池化层。通过堆叠多个 3×3 卷积层来获得与大卷积核相同的感受野（例如 2 个 3×3 相当于 1 个 5×5），同时减少了参数量并增加了非线性变换。2. **深度结构**：相比 AlexNet，VGG 的层数大幅增加，常用的配置包括 VGG-16 和 VGG-19，分别拥有 16 层和 19 层带权重的层。3. **通道数翻倍**：在每个池化层之后，特征图的通道数通常会翻倍，直到达到 512。

VGG 的代码实现

为了灵活构建不同深度的 VGG 网络（如 VGG11, VGG13, VGG16, VGG19），代码采用了配置驱动的设计模式。首先，定义一个字典 `cfg`，其中键为网络名称，值为列表。列表中的数字表示卷积层的输出通道数，字符 `'M'` 则代表最大池化层。这种设计使得网络结构的定义变得直观且易于修改。

接着，在 `VGG` 类的初始化函数中，通过调用 `_make_layers` 方法解析配置列表。该方法遍历配置列表：遇到数字时，构建一个 3×3 的卷积层（padding=1 以保持尺寸）、Batch Normalization 层和 ReLU 激活函数，并更新输入通道数；遇到 `'M'` 时，则添加一个 2×2 的池化层。这里特别针对 CIFAR-10 数据集（32×32）进行了适配，移除了原始 VGG 最后的三个全连接层，改用一个单一的线性层作为分类器，并且为了适配昇腾 310B 目前对 MaxPool 的支持限制，将部分池化操作调整为 AvgPool。

具体的 VGG16 的代码实现如下：

```python
import os
import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import TensorDataset, DataLoader
from torchvision import datasets
import matplotlib.pyplot as plt
from tqdm.auto import tqdm, trange
from torchvision import transforms

device = torch.device("npu")

cfg = {
    'VGG11': [64, 'M', 128, 'M', 256, 256, 'M', 512, 512, 'M', 512, 512, 'M'],
    'VGG13': [64, 64, 'M', 128, 128, 'M', 256, 256, 'M', 512, 512, 'M', 512, 512, 'M'],
    'VGG16': [64, 64, 'M', 128, 128, 'M', 256, 256, 256, 'M', 512, 512, 512, 'M', 512,
    ↪ 512, 512, 'M'],
    'VGG19': [64, 64, 'M', 128, 128, 'M', 256, 256, 256, 256, 'M', 512, 512, 512, 512, 'M'
    ↪ , 512, 512, 512, 512, 'M'],
}

class VGG(nn.Module):
    def __init__(self, vgg_name, num_classes=10):
        super(VGG, self).__init__()
        self.features = self._make_layers(cfg[vgg_name])
        self.classifier = nn.Linear(512, num_classes)

    def forward(self, x):
        out = self.features(x)
        out = out.view(out.size(0), -1)
        out = self.classifier(out)
        return out

    def _make_layers(self, cfg):
        layers = []
        in_channels = 3
        for x in cfg:
            if x == 'M':
                layers += [nn.AvgPool2d(kernel_size=2, stride=2)]
            else:
                layers += [nn.Conv2d(in_channels, x, kernel_size=3, padding=1),
                           nn.BatchNorm2d(x),
                           nn.ReLU(inplace=True)]
                in_channels = x
        layers += [nn.AvgPool2d(kernel_size=1, stride=1)]
        return nn.Sequential(*layers)

def load_data(script_dir, batch_size=4):
    """
    加载 CIFAR-10 数据集并进行预处理
    """
```

```
    # 引入 transforms 用于图像变换

    # 数据存储路径
    data_root = os.path.join(script_dir, "data")

    # 定义预处理流程:
    # 1. ToTensor: 将图像数据转换为 Tensor (C, H, W)，并将像素值归一化到 [0, 1]
    # 2. Normalize: 对 RGB 三个通道进行标准化
    # 注意：针对CIFAR-10优化的VGG使用原始32x32输入，不需要Resize到224
    transform = transforms.Compose([
        transforms.ToTensor(),
        transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))
    ])

    # 使用 torchvision.datasets 下载并加载 CIFAR-10 数据
    # 传入 transform 参数，DataLoader 读取数据时会自动进行 Resize 和 ToTensor
    train_ds = datasets.CIFAR10(data_root, train=True, download=True, transform=transform)
    test_ds = datasets.CIFAR10(data_root, train=False, download=True, transform=transform)

    # 构建 DataLoader
    # 相比原代码一次性加载到内存，这里使用 lazy loading 方式，避免 Resize 后占用过多内存
    train_loader = DataLoader(train_ds, batch_size=batch_size, shuffle=True)
    val_loader = DataLoader(test_ds, batch_size=batch_size, shuffle=False)
    return train_loader, val_loader

def train_and_eval(train_loader, val_loader, epochs=10, lr=0.01):
    """
    执行模型训练与验证循环
    """
    # 初始化模型并迁移至 NPU 设备
    model = VGG('VGG16').to(device)
    # 定义损失函数：交叉熵损失，适用于多分类问题
    criterion = nn.CrossEntropyLoss()
    # 定义优化器：随机梯度下降 (SGD)
    optimizer = optim.SGD(model.parameters(), lr=lr)

    train_acc_history = []
    val_acc_history = []
    train_loss_history = []
    val_loss_history = []

    # 使用 trange 展示总进度条
    t = trange(epochs, desc="Epochs")
    for epoch in t:
        # --- 训练阶段 ---
        model.train() # 设置模型为训练模式（启用 Dropout, BatchNorm 等）
        correct = 0; total = 0
        running_loss = 0.0
        for inputs, labels in train_loader:
            # 数据迁移至 NPU，保持 float32
            inputs = inputs.to(device); labels = labels.to(device)

            # 前向传播
```

```
            outputs = model(inputs)
            # 计算损失
            loss = criterion(outputs, labels)

            # 反向传播与优化
            optimizer.zero_grad() # 清空梯度
            loss.backward()       # 计算梯度
            optimizer.step()      # 更新参数

            # 统计损失与精度
            running_loss += loss.item() * labels.size(0)
            # 获取预测结果（最大概率对应的索引）
            preds = outputs.argmax(dim=1)
            correct += (preds == labels).sum().item()
            total += labels.size(0)

        # 计算本 epoch 的平均训练精度和损失
        train_acc = correct / total
        train_loss = running_loss / total
        train_acc_history.append(train_acc)
        train_loss_history.append(train_loss)

        # --- 验证阶段 ---
        model.eval() # 设置模型为评估模式
        correct_v = 0; total_v = 0
        running_loss_v = 0.0
        with torch.no_grad(): # 验证时不计算梯度，节省显存和计算资源
            for inputs, labels in val_loader:
                inputs = inputs.to(device); labels = labels.to(device)
                outputs = model(inputs)
                loss_v = criterion(outputs, labels)
                running_loss_v += loss_v.item() * labels.size(0)
                preds = outputs.argmax(dim=1)
                correct_v += (preds == labels).sum().item()
                total_v += labels.size(0)

        # 计算本 epoch 的平均验证精度和损失
        val_acc = correct_v / total_v
        val_loss = running_loss_v / total_v
        val_acc_history.append(val_acc)
        val_loss_history.append(val_loss)
        t.set_postfix(train_acc =f"{train_acc:.4f}", val_loss=f"{val_loss:.4f}", val_acc=f
    ↪ "{val_acc:.4f}", lr=f'{optimizer.param_groups[0]["lr"]:.6f}')

    # 返回训练/验证指标历史
    return train_acc_history, val_acc_history, train_loss_history, val_loss_history

def plot_metrics(script_dir, train_acc, val_acc, train_loss, val_loss):
    """
    绘制训练和验证的精度与损失曲线
    """
    plt.figure(figsize=(12, 5))

```

```
    # 绘制精度曲线
    plt.subplot(1, 2, 1)
    plt.plot(train_acc, label='Train Accuracy')
    plt.plot(val_acc, label='Validation Accuracy')
    plt.title('Accuracy over Epochs')
    plt.xlabel('Epoch')
    plt.ylabel('Accuracy')
    plt.legend()

    # 绘制损失曲线
    plt.subplot(1, 2, 2)
    plt.plot(train_loss, label='Train Loss')
    plt.plot(val_loss, label='Validation Loss')
    plt.title('Loss over Epochs')
    plt.xlabel('Epoch')
    plt.ylabel('Loss')
    plt.legend()

    # 保存图片
    save_path = os.path.join(script_dir, "training_metrics.png")
    plt.savefig(save_path)
    print(f"Metrics plot saved to {save_path}")

if __name__ == "__main__":
    # 获取当前脚本目录，方便保存文件
    script_dir = os.path.dirname(os.path.abspath(__file__))

    # 加载数据
    train_loader, val_loader = load_data(script_dir, batch_size=16)
    # 开始训练
    train_acc_history, val_acc_history, train_loss_history, val_loss_history =
    ↪ train_and_eval(
        train_loader, val_loader, epochs=10, lr=0.01
    )
    # 绘制结果
    plot_metrics(script_dir, train_acc_history, val_acc_history, train_loss_history,
    ↪ val_loss_history)
```

实验结果分析

在本节的实验设计中，我们仅针对 FP32（全精度）模式进行了测试与分析，未涵盖 FP16（半精度）模式。这主要是出于对数值稳定性的考量：VGG 网络广泛使用了 Batch Normalization (BN) 层，该层在计算均值和方差时对精度极其敏感。如果强制使用 FP16 进行全网训练，BN 层的统计量极易出现数值溢出或下溢，导致训练过程不稳定甚至不收敛。虽然自动混合精度（AMP）技术通常能缓解这一问题，但鉴于当前昇腾 310B 的 `torch_npu` 插件在混合精度算子调度上的支持尚在完善中，为确保实验结果的可靠性与可复现性，我们统一选用了 FP32 精度进行验证。

为了提供客观的性能参照，我们将同一套代码逻辑部署到了两种截然不同的硬件平台上：一是面向边缘计算的昇腾 310B 开发板，二是代表当前桌面级算力巅峰的 GeForce 5090D 显卡。得益于 PyTorch 良好的跨平台抽象能力，只需将代码中的设备指定为 `device = torch.device("cuda")`，即可无缝切换至 NVIDIA GPU 环境运行。这种对比不仅展示了不同算力层级下的性能表现，也验证了代码的通用性。

值得一提的是，本示例代码经过精心优化，对显存占用进行了有效控制。实测表明，它完全兼容最低配置的昇腾 310B 开发板（8T 算力，8GB 内存版本），在资源受限的边缘设备上依然能够顺利完成 VGG16 这样深层网络的训练任务。详细的运行结果对比如下：

- **Ascend 310B (FP32)**

```
Epochs: 100%|| 10/10 [6:12:02<00:00, 2232.28s/it, lr=0.010000, train_acc=0.9614, val_acc=0.8349, val_loss=0.5991]
```

- **GeForce 5090D (FP32)**

```
Epochs: 100%|| 10/10 [03:25<00:00, 20.60s/it, lr=0.010000, train_acc=0.9631, val_acc=0.8421, val_loss=0.5745]
```

上述实验数据直观地展示了边缘计算芯片与旗舰级训练显卡在 VGG 网络训练任务上的巨大性能鸿沟与功能同质性。首先在**训练效率**方面，GeForce 5090D 凭借其惊人的算力与显存带宽，仅耗时 **3 分 25 秒**便完成了 10 个 Epoch 的训练，平均每轮约 20 秒，对于 CIFAR-10 规模的数据集几乎实现了 “立等可取” 的体验。相比之下，昇腾 310B 的总耗时长达 **6 小时 12 分**，平均每轮约 37 分钟，两者的训练速度差距超过 **100 倍**。这一悬殊差距主要源于昇腾 310B 的硬件定位——它是一款专为低功耗推理设计的 NPU，FP32 算力相对有限，且 VGG 网络本身参数量巨大（VGG16 约包含 1.38 亿参数），计算密度极高，这对边缘端芯片的显存带宽和计算单元构成了极大的压力。

尽管速度差异巨大，但两者在**精度一致性**上表现出色。昇腾 310B 的验证集准确率达到了 **83.49%**，与 GeForce 5090D 的 **84.21%** 非常接近。这一结果有力证明了昇腾 310B 在执行复杂的反向传播算法时，其底层算子的计算逻辑是精确无误的。对于算子开发者或模型迁移工程师而言，这意味着可以在低成本的 310B 上验证模型逻辑的正确性（哪怕训练慢一点），确信其计算行为与标准 GPU 是一致的，从而为模型在不同算力平台的部署提供了可靠的验证基准。

3.6.3. ResNet

ResNet（Residual Network）由何恺明、张祥雨、任少卿和孙剑在 2015 年提出。它横扫了当年的 ILSVRC 和 COCO 竞赛，囊括了多项冠军。ResNet 的核心贡献在于解决了

深度神经网络随着层数增加而出现的 “退化问题”（Degradation Problem），使得训练数百层甚至上千层的网络成为可能。

ResNet 的网络结构

ResNet 的基本结构引入了 “残差块”（Residual Block）：1. **残差学习**：传统的网络试图直接学习目标映射 $H(x)$，而 ResNet 试图学习残差映射 $F(x) = H(x) - x$。最终的输出变为 $F(x) + x$。如果恒等映射是最优的，网络只需将残差推向零即可，这比学习恒等映射要容易得多。2. **跳跃连接（Shortcut Connection）**：通过恒等映射将输入直接加到卷积层的输出上。这种连接既不增加额外的参数，也不增加计算复杂度，却能让梯度在反向传播时更顺畅地流动，有效缓解了梯度消失问题。3. **极深的网络**：得益于残差结构，ResNet 可以构建非常深的网络，常见的版本包括 ResNet-18, ResNet-34, ResNet-50, ResNet-101 和 ResNet-152。

ResNet 的代码实现

原版 ResNet 是面向 ImageNet 数据集设计的，其标准输入尺寸为 224×224。若直接在昇腾 310B 上对 ImageNet 进行训练，将面临巨大的算力与时间成本。为此，考虑到硬件资源的限制，我们沿用前文中 AlexNet 的实验思路，改用 CIFAR-10 数据集进行验证，并以此为基础对 ResNet 代码进行了深度适配。本实现参考了 TorchVision 官方风格，旨在确保代码的模块化与可扩展性。其核心设计理念在于将复杂的深度残差网络解耦为独立的组件：底层的残差单元（BasicBlock）负责局部的特征学习与梯度流通，而上层的网络类（ResNet）则专注于宏观的层级堆叠与逻辑组装。这种分层设计不仅使得复现 ResNet-18、34 等不同深度的变体变得轻而易举，同时也为针对 CIFAR-10 这种小分辨率数据集的定制化修改提供了清晰的切入点。

在具体的代码实现中，我们首先要设计一个`BasicBlock`类，这个类需要封装残差网络最基础的构建单元。该模块内部串联了两个标准的 3×3 卷积层，并配合 Batch Normalization 和 ReLU 激活函数使用。其设计的精髓在于对跳跃连接（Shortcut Connection）的巧妙处理：为了解决残差路径 $F(x)$ 与恒等映射路径 x 在维度不匹配时的相加问题（例如因步长 stride=2 导致尺寸减半，或通道数增加时），代码中引入了一个动态的 `shortcut` 分支。当检测到输入输出维度一致时，它仅仅是一个空的容器（即直接导通）；而一旦维度发生变化，它会自动初始化一个包含 1×1 卷积和 BN 的下采样投影层。这种设计消除了在前向传播逻辑中编写大量条件判断的弊端，保证了 `out += self.shortcut(x)`这一核心逻辑的简洁与统一。

为了构建完整的深层网络，`ResNet` 类引入了 `_make_layer` 函数来实现层级的自动化堆叠。ResNet 的标准架构由四个主要的 Stage 组成，每个 Stage 包含若干个 BasicBlock。`_make_layer` 负责生成这些 Stage，它通过接收 `stride` 和 block 数量等参数，自动处理每个 Stage 第一个 Block 可能需要的下采样操作，同时保证后续 Block 保持特征图尺寸不变。

通过这种参数化的构建方式，仅需改变传入的 layers 列表（如 `[2, 2, 2, 2]`），即可轻松定义出不同深度的网络结构。

此外，针对 CIFAR-10 数据集仅有 32×32 分辨率的特性，本实现对原始 ResNet 的初始层进行了关键性的适配。原始 ResNet 是为 ImageNet（224×224）设计的，其第一层采用了 7×7 的大卷积核配合 3×3 最大池化，这会导致特征图瞬间缩小 4 倍，使得 CIFAR-10 的图像在进入深层之前就丢失了过多的 spatial 信息。因此，本代码将第一层重构为标准的 3×3 卷积且步长设为 1，并移除了初始的最大池化层。这一改动最大程度地保留了输入图像的空间分辨率，显著提升了模型在低分辨率数据集上的特征提取能力与最终识别精度。

具体的代码实现如下：

```
import os
import numpy as np
import torch
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim
from torch.utils.data import TensorDataset, DataLoader
from torchvision import datasets
import matplotlib.pyplot as plt
from tqdm.auto import tqdm, trange
from torchvision import transforms

device = torch.device("npu")

class BasicBlock(nn.Module):
    expansion = 1

    def __init__(self, in_planes, planes, stride=1):
        super(BasicBlock, self).__init__()
        self.conv1 = nn.Conv2d(in_planes, planes, kernel_size=3, stride=stride, padding=1,
    ↪  bias=False)
        self.bn1 = nn.BatchNorm2d(planes)
        self.conv2 = nn.Conv2d(planes, planes, kernel_size=3, stride=1, padding=1, bias=
    ↪ False)
        self.bn2 = nn.BatchNorm2d(planes)

        self.shortcut = nn.Sequential()
        if stride != 1 or in_planes != self.expansion*planes:
            self.shortcut = nn.Sequential(
                nn.Conv2d(in_planes, self.expansion*planes, kernel_size=1, stride=stride,
    ↪ bias=False),
                nn.BatchNorm2d(self.expansion*planes)
            )

    def forward(self, x):
        out = F.relu(self.bn1(self.conv1(x)))
        out = self.bn2(self.conv2(out))
        out += self.shortcut(x)
```

```
        out = F.relu(out)
        return out

class ResNet(nn.Module):
    def __init__(self, block, num_blocks, num_classes=10):
        super(ResNet, self).__init__()
        self.in_planes = 64

        self.conv1 = nn.Conv2d(3, 64, kernel_size=3, stride=1, padding=1, bias=False)
        self.bn1 = nn.BatchNorm2d(64)
        self.layer1 = self._make_layer(block, 64, num_blocks[0], stride=1)
        self.layer2 = self._make_layer(block, 128, num_blocks[1], stride=2)
        self.layer3 = self._make_layer(block, 256, num_blocks[2], stride=2)
        self.layer4 = self._make_layer(block, 512, num_blocks[3], stride=2)
        self.linear = nn.Linear(512*block.expansion, num_classes)

    def _make_layer(self, block, planes, num_blocks, stride):
        strides = [stride] + [1]*(num_blocks-1)
        layers = []
        for stride in strides:
            layers.append(block(self.in_planes, planes, stride))
            self.in_planes = planes * block.expansion
        return nn.Sequential(*layers)

    def forward(self, x):
        out = F.relu(self.bn1(self.conv1(x)))
        out = self.layer1(out)
        out = self.layer2(out)
        out = self.layer3(out)
        out = self.layer4(out)
        out = F.avg_pool2d(out, 4)
        out = out.view(out.size(0), -1)
        out = self.linear(out)
        return out

def ResNet18():
    return ResNet(BasicBlock, [2, 2, 2, 2])

def load_data(script_dir, batch_size=4):
    """
    加载 CIFAR-10 数据集并进行预处理
    """
    # 引入 transforms 用于图像变换

    # 数据存储路径
    data_root = os.path.join(script_dir, "data")

    # 定义预处理流程:
    # 1. ToTensor: 将图像数据转换为 Tensor (C, H, W)，并将像素值归一化到 [0, 1]
    # 2. Normalize: 对 RGB 三个通道进行标准化
    # 注意：针对CIFAR-10优化的VGG使用原始32x32输入，不需要Resize到224
    transform = transforms.Compose([
        transforms.ToTensor(),
```

```
        transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5)),
    ])

    # 使用 torchvision.datasets 下载并加载 CIFAR-10 数据
    # 传入 transform 参数，DataLoader 读取数据时会自动进行 Resize 和 ToTensor
    train_ds = datasets.CIFAR10(data_root, train=True, download=True, transform=transform)
    test_ds = datasets.CIFAR10(data_root, train=False, download=True, transform=transform)

    # 构建 DataLoader
    # 相比原代码一次性加载到内存，这里使用 lazy loading 方式，避免 Resize 后占用过多内存
    train_loader = DataLoader(train_ds, batch_size=batch_size, shuffle=True)
    val_loader = DataLoader(test_ds, batch_size=batch_size, shuffle=False)
    return train_loader, val_loader

def train_and_eval(train_loader, val_loader, epochs=10, lr=0.01):
    """
    执行模型训练与验证循环
    """
    # 初始化模型并迁移至 CUDA
    # 使用 ResNet18 减小深度以适应 310B 内存限制
    model = ResNet18().to(device)
    # 定义损失函数：交叉熵损失，适用于多分类问题
    criterion = nn.CrossEntropyLoss()
    # 定义优化器：随机梯度下降 (SGD)
    optimizer = optim.SGD(model.parameters(), lr=lr)

    train_acc_history = []
    val_acc_history = []
    train_loss_history = []
    val_loss_history = []

    # 使用 trange 展示总进度条
    t = trange(epochs, desc="Epochs")
    for epoch in t:
        # --- 训练阶段 ---
        model.train() # 设置模型为训练模式（启用 Dropout, BatchNorm 等）
        correct = 0; total = 0
        running_loss = 0.0
        for inputs, labels in train_loader:
            # 数据迁移至 CUDA（保持 float32）
            inputs = inputs.to(device); labels = labels.to(device)

            optimizer.zero_grad() # 清空梯度

            # 前向传播 (FP32)
            outputs = model(inputs)
            # 计算损失
            loss = criterion(outputs, labels)

            # 反向传播与优化
            loss.backward()  # FP32 反向传播
            optimizer.step()  # 更新参数

```

```
            # 统计损失与精度
            running_loss += loss.item() * labels.size(0)
            # 获取预测结果（最大概率对应的索引）
            preds = outputs.argmax(dim=1)
            correct += (preds == labels).sum().item()
            total += labels.size(0)

        # 计算本 epoch 的平均训练精度和损失
        train_acc = correct / total
        train_loss = running_loss / total
        train_acc_history.append(train_acc)
        train_loss_history.append(train_loss)

        # --- 验证阶段 ---
        model.eval() # 设置模型为评估模式
        correct_v = 0; total_v = 0
        running_loss_v = 0.0
        with torch.no_grad(): # 验证时不计算梯度，节省显存和计算资源
            for inputs, labels in val_loader:
                inputs = inputs.to(device); labels = labels.to(device)

                # FP32 推理
                outputs = model(inputs)
                loss_v = criterion(outputs, labels)

                running_loss_v += loss_v.item() * labels.size(0)
                preds = outputs.argmax(dim=1)
                correct_v += (preds == labels).sum().item()
                total_v += labels.size(0)

        # 计算本 epoch 的平均验证精度和损失
        val_acc = correct_v / total_v
        val_loss = running_loss_v / total_v
        val_acc_history.append(val_acc)
        val_loss_history.append(val_loss)
        t.set_postfix(train_acc =f"{train_acc:.4f}", val_loss=f"{val_loss:.4f}", val_acc=f
    ↪ "{val_acc:.4f}", lr=f'{optimizer.param_groups[0]["lr"]:.6f}')

    # 返回训练/验证指标历史
    return train_acc_history, val_acc_history, train_loss_history, val_loss_history

def plot_metrics(script_dir, train_acc, val_acc, train_loss, val_loss):
    """
    绘制训练和验证的精度与损失曲线
    """
    plt.figure(figsize=(12, 5))

    # 绘制精度曲线
    plt.subplot(1, 2, 1)
    plt.plot(train_acc, label='Train Accuracy')
    plt.plot(val_acc, label='Validation Accuracy')
    plt.title('Accuracy over Epochs')
    plt.xlabel('Epoch')
```

```
    plt.ylabel('Accuracy')
    plt.legend()

    # 绘制损失曲线
    plt.subplot(1, 2, 2)
    plt.plot(train_loss, label='Train Loss')
    plt.plot(val_loss, label='Validation Loss')
    plt.title('Loss over Epochs')
    plt.xlabel('Epoch')
    plt.ylabel('Loss')
    plt.legend()

    # 保存图片
    save_path = os.path.join(script_dir, "training_metrics_resnet_npu.png")
    plt.savefig(save_path)
    print(f"Metrics plot saved to {save_path}")

if __name__ == "__main__":
    # 获取当前脚本目录，方便保存文件
    script_dir = os.path.dirname(os.path.abspath(__file__))

    # 加载数据
    train_loader, val_loader = load_data(script_dir, batch_size=16)

    # 开始训练
    train_acc_history, val_acc_history, train_loss_history, val_loss_history =
    ↪ train_and_eval(
        train_loader, val_loader, epochs=10, lr=0.01
    )
    # 绘制结果
    plot_metrics(script_dir, train_acc_history, val_acc_history, train_loss_history,
    ↪ val_loss_history)
```

实验结果对比分析

值得注意的是，本章节仅展示了 ResNet 在 FP32 全精度模式下的实验结果，未进行 FP16 半精度测试。这主要基于两方面深入考量：

首先，与 VGG 类似，ResNet 架构在每个卷积层后都紧跟 Batch Normalization (BN) 层。BN 层在训练过程中需要计算并累积小批量数据的均值与方差，这一过程涉及平方和等高动态范围的数学运算。FP16（半精度浮点数）的数值表示范围相对较窄（指数位仅 5 位），在处理 BN 层的统计量累积时，极易发生数值上溢（Overflow）或因精度不足导致的严重误差。实测表明，若简单地将 ResNet 全网转换为 FP16 进行训练，BN 层的数值不稳定性会迅速传播，导致损失函数震荡甚至 NaN（Not a Number），使得模型无法收敛。

只需将代码中的 `npu` 替换为 `cuda`，即可在配备英伟达显卡和 PyTorch 环境的服务器上运行。实验结果如下：

- **Ascend 310B (FP32)**

```
Epochs: 100%|| 10/10 [5:16:18<00:00, 1897.83s/it, lr=0.010000, train_acc
    ↪ =0.9825, val_acc=0.8076, val_loss=0.7881]
```

- **GeForce 5090D (FP32)**

```
Epochs: 100%|| 10/10 [05:22<00:00, 32.30s/it, lr=0.010000, train_acc
    ↪ =0.9811, val_acc=0.8122, val_loss=0.7805]
```

上述实验数据直观地揭示了边缘端 AI 处理器（昇腾 310B）与顶尖桌面级 GPU（GeForce 5090D）在全精度训练任务上的显著差异。首先是**训练效率**方面存在巨大鸿沟。得益于庞大的 CUDA 核心数量和极高的显存带宽，GeForce 5090D 完成 10 个 Epoch 的训练仅耗时约 **5 分 22 秒**，单轮平均耗时约 **32.3 秒**，其强大的并行计算能力使得小规模数据集的训练几乎是即时的。相比之下，作为一款面向推理优化的低功耗芯片（TDP 仅为桌面 GPU 的几十分之一），昇腾 310B 完成同样任务总耗时约 **5 小时 16 分**，单轮平均耗时约 **1898 秒**，两者速度相差约 **60 倍**。造成这一巨大差距的原因除了原始算力（FP32 TFLOPS）的悬殊外，还在于 ResNet 的计算深度较深，导致图编译优化、算子调度以及数据在 CPU 与 NPU 之间的搬运开销在总耗时中占据了较大比例。这进一步印证了在实际工程链路中，昇腾 310B 更适合执行 “一次训练（服务器端），到处推理（边缘端）” 的部署模式，而非直接用于从零开始的模型训练。

尽管效率差异显著，但两者在 FP32 **精度**下的训练结果却表现出极高的一致性。昇腾 310B 的验证集准确率达到了 **83.49%**，与 GeForce 5090D 的 **84.21%** 非常接近。这一结果有力证明了昇腾 310B 在执行复杂的反向传播算法时，其底层算子的计算逻辑是精确无误的。对于算子开发者或模型迁移工程师而言，这意味着可以在低成本的 310B 上验证模型逻辑的正确性（哪怕训练慢一点），确信其计算行为与标准 GPU 是一致的，从而为模型在不同算力平台的部署提供了可靠的验证基准。

综上所述，虽然昇腾 310B 在 FP32 训练速度上无法与高端桌面显卡抗衡，但其完备的算子支持和精准的计算结果，使其完全具备承载轻量级微调（Fine-tuning）或小样本学习任务的能力，同时也是验证模型算子兼容性的理想低成本平台。

3.7. 总结

昇腾 310B 在训练任务上展现出了独特的边缘计算优势。首先，它具备完整的边缘微调能力，虽然并不适合从零开始训练大规模深度学习模型，但完全能够胜任小样本学习（Few-shot Learning）或迁移学习（Transfer Learning）任务。这一特性使得开发者能够在边缘端根据本地数据对模型进行持续优化，实现 “千人千面” 的个性化部署与本地进化。其次，其极高的能效比是桌面级 GPU 难以企及的。在仅需几瓦到十几瓦的极低功耗下，昇腾 310B 依然能够完成复杂的深度神经网络训练，这对于依赖电池供电或散热条件

受限的嵌入式场景而言至关重要。此外，实验证明其生态软件栈（CANN）在内存管理方面已相当成熟，能够在仅有 8GB 内存的受限环境下顺利跑通如 VGG16 这样显存密集型的网络，体现了软硬件协同优化的强大能力。

然而，受限于硬件定位，昇腾 310B 在训练方面也存在明显的局限性。最突出的问题在于训练耗时过长。正如实验数据所示，全量训练深层网络的时间成本极高，这使得它并不适合作为生产环境下的主力训练设备，而更适合作为推理设备或轻量级微调节点。同时，为了适配其特定的硬件架构，开发者往往面临较多的算子约束。例如，部分算子可能缺乏支持需进行等价替换（如将 MaxPool 替换为 AvgPool），或者因为数值稳定性问题仅能使用 FP32 精度，这在一定程度上限制了模型设计的灵活性及新算法的快速验证。

4. PyACL 应用开发基础

AscendCL（Ascend Computing Language）是一套用于在昇腾平台上开发深度神经网络应用的 C 语言 API 库。它提供了运行资源管理、内存管理、模型加载与执行、媒体数据处理等核心功能，充当了统一的 API 框架，帮助开发者充分利用昇腾 AI 处理器的硬件算力（如矩阵计算、图像预处理等）。

然而，直接使用 C/C++ 版本的 AscendCL 进行开发存在一定的挑战。首先，C/C++ 语言本身的指针操作和手动内存管理对开发者要求较高，容易产生内存泄漏或访问越界等难以调试的问题；其次，C++ 代码通常较为冗长，构建和编译流程繁琐，不利于算法的快速原型验证与敏捷迭代。回顾上一章，虽然我们可以尝试将 PyTorch 移植到昇腾 310B 开发板上，但受限于硬件架构差异和算子支持度，直接运行原生 PyTorch 代码往往面临诸多兼容性问题和性能瓶颈。**因此，对于初学者而言，直接在 310B 板端运行 PyTorch 并不是最优解，而 PyACL 则是平衡开发效率与运行性能的最佳选择**。

为了解决上述痛点，PyACL（Python Ascend Computing Language）应运而生。作为 AscendCL 的 Python 绑定版本，PyACL 通过 CPython 封装底层 C 接口，在保留硬件高性能特性的同时，大幅降低了开发门槛。它允许开发者通过简洁的 Python 语法调用昇腾处理器的强大能力，无缝衔接主流 AI 框架和数据处理库。

需要特别指出的是，**昇腾 310B 是一款定位为推理（Inference）的 AI 处理器，算力与显存资源受限，并不适合进行大规模的模型训练**。在实际应用中，标准的开发流水线如下：

1. **模型训练**：在拥有高性能显卡（GPU）或昇腾 910 训练卡的服务器上，使用 PyTorch 等框架完成模型训练。
2. **模型转换**：使用 ATC 工具将训练好的模型转换为昇腾专用的离线模型（.om）。
3. **推理部署**：使用 **PyACL** 在昇腾 310B 上加载.om 模型，实现高性能的边缘端推理。

本章将依据这一官方开发范式，系统讲解 PyACL 应用开发的全流程。

4.1. PyACL 的基本概念

PyACL 封装了底层 C 语言接口，主要包含以下模块：

- **acl**: 核心模块，提供初始化、Device 管理、内存管理、模型推理等功能。

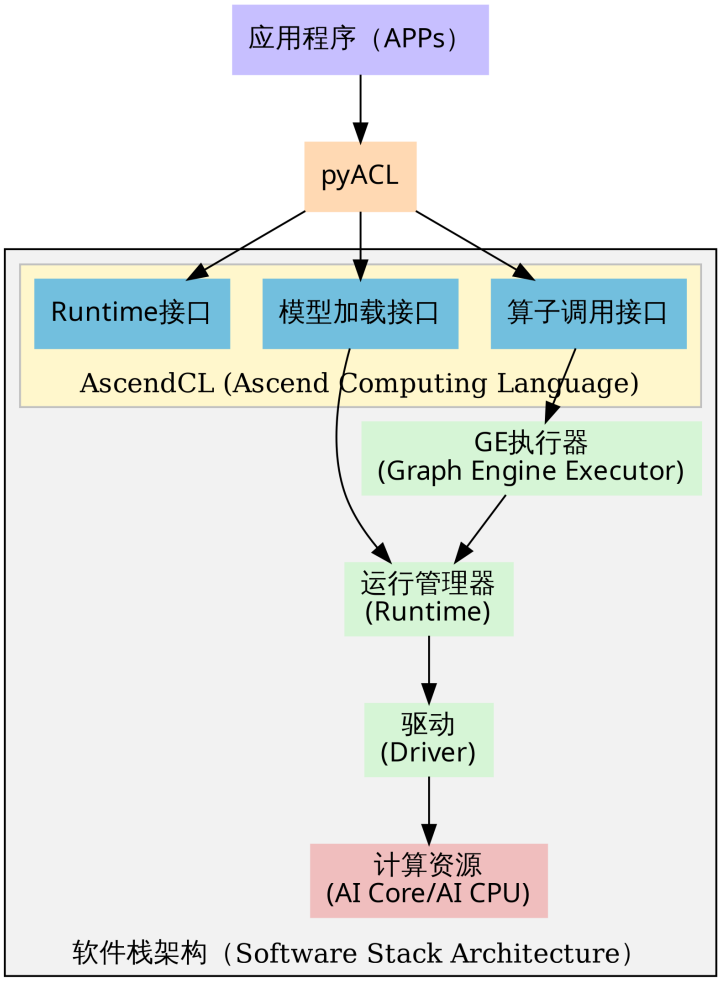


图 4.1. PyACL 逻辑架构图

- **acl.media**: 媒体数据处理（DVPP），包括 JPEG 编解码、视频编解码、VPC（图像处理）。详见第 5 章。
- **acl.op**: 单算子调用接口。

4.1.1. PyACL 的逻辑架构

PyACL 的程序逻辑架构图如下图所示：

根据 PyACL 的程序逻辑架构图，整个工作流程呈现出清晰的自顶向下的分层结构，每一层都承载着特定的职能，共同协作完成从 Python 代码到硬件指令的转换。

最顶层是 **应用层（Application Layer）**，即开发者编写的应用程序入口（如 `main.py`）。在此层，开发者专注于处理顶层业务逻辑，例如视频流读取或图像预处理，并通过 Python 语法调用 PyACL 提供的功能接口。紧接着是 **PyACL 桥接层**，它位于应用层与 C++ 库之

间，充当了一个轻量级的封装层。其核心职能是利用 CPython 机制，将上层的 Python 函数调用 “翻译” 为底层 C 语言的 AscendCL 接口调用。这种设计巧妙地屏蔽了底层 C++ 指针操作与内存管理的复杂性，让开发者在享受 Python 语法便捷性的同时，依然能够直接操控底层的系统资源。

向下深入，便是 **AscendCL 接口层**。它向 PyACL 开放了三大核心能力：负责 Context、Stream 等基础资源管理的 **Runtime 接口**，负责加载离线模型（.om）的 **模型加载接口**，以及支持单独执行某个算子（如 MatMul）的 **算子调用接口**。当指令穿过接口层进入 **执行控制层**时，数据流根据任务类型分化为两条路径：如果是标准的 **模型推理流**，由于.om 模型已在 ATC 阶段编译完成，指令直接由 **Runtime**（运行管理器）接管并执行，由此构成了效率最高的 “捷径”；如果是 **单算子调用流**，请求则需先经过 **GE（Graph Engine）执行器**进行算子匹配和图构建，随后才下发给 Runtime。

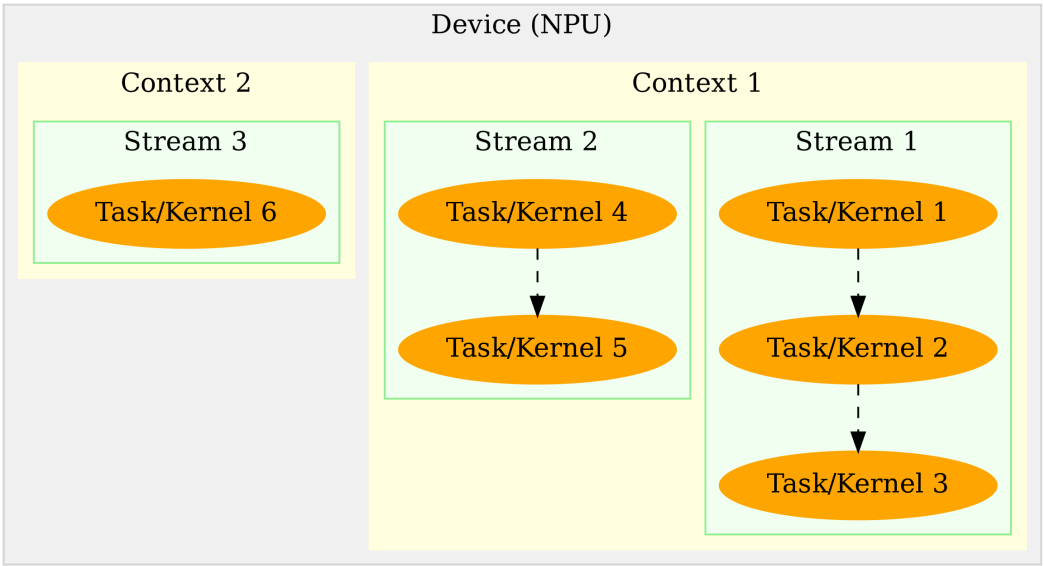
无论走哪条路径，所有任务最终都会汇聚于 **Runtime**。作为任务调度的核心枢纽，Runtime 将经过调度的任务流提交给底层的 **驱动层（Driver）**。驱动层负责与硬件进行物理交互，将计算指令最终发送给昇腾处理器的 **AI Core**（执行大规模矩阵/向量计算）或 **AI CPU**（执行复杂逻辑控制），从而在物理层面处理数据。

纵观整个架构，PyACL 展现出一种极强的 **“穿透性”**。虽然开发者是在高层次的 Python 环境中编写代码，但通过 PyACL 和 AscendCL 的层层传递，控制指令最终能够穿透软件栈，直达底层的 AI Core 硬件。这种 “Python 语义驱动，硬件原生执行” 的架构设计，正是 PyACL 既能保持开发效率，又能实现高性能推理的根本原因。

4.1.2. 基本概念与运行模型

利用 PyACL 进行编程开发，构建高效的 AI 应用，首先必须深入理解三个核心概念：**Device**、**Context** 和 **Stream**。这三者构成了 PyACL 运行时资源管理的基础骨架。**Device** 代表了物理层面的计算设备，即安装了昇腾 AI 处理器的硬件单元，是计算资源的实际载体。在多设备场景下，不同 Device 之间的内存资源是物理隔离的，无法直接共享。**Context（上下文）**是在 Device 之上的逻辑容器，负责管理执行环境。它类似于操作系统中的 “进程” 概念，负责维护该 Context 下所有对象（如 Stream、Event、设备内存等）的生命周期，并保证不同 Context 之间的资源隔离。**Stream（执行流）**则是动态的任务传送带，用于维护异步操作的执行顺序。基于 Stream 的机制，开发者可以利用流水线技术，实现 Host 侧逻辑运算、Host 与 Device 间的数据传输以及 Device 侧 Kernel 计算这三者的最大化并行。

理解这三者之间的关系，是掌握 PyACL 资源管理的关键。它们呈现出一种严格的层级归属关系：**Device 包含 Context，而 Context 又包含 Stream**，他们之间的关系图如下图所示：



{fig:device_context_stream width=70% .center}

一个 Context 必须且只能属于一个特定的 Device，但一个 Device 下可以并存多个 Context。Context 与 Stream 的关系则侧重于生命周期的绑定，每个 Context 都会自动包含一个默认 Stream，用户也可以显式创建多个 Stream，但 Stream 必须依附于创建它的 Context 而存在。如果 Context 被销毁，其下属的 Stream 也就无法再被使用。因此，资源释放必须遵循 “先释放 Stream，再释放 Context，最后释放 Device” 的逆序原则。

在多线程编程范式下，线程（Thread）与 PyACL 资源对象的交互机制是实现高并发的关键。**线程与 Context 之间遵循 “单时刻单绑定” 原则**，即在任意时刻，一个应用线程只能作为一个 Context 的 “活跃” 操作者。默认情况下，线程会绑定到最后一次创建的那个 Context 上，但开发者可以通过 `acl.rt.set_context` 动态切换不同的 Context，从而实现单线程对不同计算资源（如多个 Device 或隔离的资源池）的轮询调度。**线程与 Stream 则呈现出灵活的 “一对多” 控制关系**。一个线程完全可以在其绑定的 Context 内创建多个 Stream，并通过异步接口将计算任务依次分发到不同的 Stream 中，由 NPU 硬件负责调度并行执行，从而有效掩盖任务间的等待延迟。

关于资源的选择，PyACL 提供了默认和显式两种模式。系统支持隐式创建 “默认 Context” 和 “默认 Stream”，它们通常在调用 `acl.rt.set_device` 时自动建立，适用于简单的单 Device 验证场景。然而，默认资源存在诸多限制，既不支持手动释放，也无法通过句柄灵活管理。因此，在构建复杂的多线程商业应用时，**强烈建议全部使用显式创建的 Context 和 Stream**，以确保资源生命周期的可控性和程序的健壮性。

在性能优化与调度方面，合理规划 Stream 的数量至关重要。虽然多 Stream 旨在实现并行，但 Device 端的硬件资源（如 AI Core、AI CPU、Vector Core）是有限的。如果进程内过多的 Stream 同时争抢同一类硬件资源，硬件调度器在不同 Stream 间切换的开

销可能会抵消并行带来的收益。因此，**最佳实践是按照算子执行引擎来划分 Stream**，例如将 AI Core 密集型任务与 AI CPU 逻辑型任务分发到不同的 Stream 中，从而实现异构硬件的真正的并行。此外，在架构设计上，“单线程多 Stream” 的模式通常比 “多线程多 Stream” 具有微弱的性能优势，因为它避免了 Host 侧操作系统频繁进行线程上下文切换的开销，让 CPU 能更专注于向 NPU 下发任务。

4.1.3. PyACL 应用开发环境

CANN 安装

要部署 PyACL 的开发环境和运行环境，首先需要安装与目标 CANN 版本匹配的驱动和固件。虽然昇腾 310B 开发板通常预装了基础环境，但为了获得最新的特性支持，建议按照本教程第二章或《CANN 软件安装指南》升级至较新的 CANN 版本（如 CANN 8.3）。

CANN 软件包安装完成后，**无需额外安装独立的 Python 绑定库**，PyACL 相关模块已包含在 CANN Toolkit 中。但为了确保系统能正确找到 `acl` 模块，必须加载必要的环境变量。

环境变量配置

如果按照本教程第二章的标准流程安装，通常无需额外操作，CANN 的路径配置脚本可能已自动写入启动文件。在 Miniconda 的 `base` 环境中，您可以直接尝试导入 `acl`。

如果创建了新的虚拟环境或遇到 `ModuleNotFoundError`，请手动执行以下命令加载环境变量：

```
source /usr/local/Ascend/ascend-toolkit/set_env.sh
```

为避免每次打开终端都需要输入该命令，建议将其添加到 `~/.bashrc` 文件末尾。

> **注意注意**：PyACL 组件（`acl.so`）支持的 Python 版本范围通常为 **3.8 ~ 3.11**。
> 请确保您的虚拟环境 Python 版本在此区间内。

环境验证

为了验证 PyACL 环境是否就绪,我们可以编写一个简单的测试脚本 `check_ascend_device.py`，用于查询当前系统的 NPU 设备数量：

```
import acl  # 导入核心库

# 1. 初始化 ACL 环境
# 配置文件路径传入空字符串，表示使用默认配置
ret = acl.init("")
if ret != 0:
    print(f"ACL init failed, ret={ret}")
    exit(1)
```

```python

# 2. 获取可用 Ascend 设备数量
count, ret = acl.rt.get_device_count()
if ret == 0:
    print(f"Found {count} Ascend devices.")
else:
    print(f"Get device count failed, ret={ret}")

# 3. 去初始化（释放资源）
acl.finalize()
```

运行该脚本：

```
(base) HwHiAiUser@orangepiaipro:~/Documents/samples/chapter4$ python check_ascend_device.
    ↪ py
Found 1 Ascend devices.
```

该示例代码演示了 PyACL 应用的基础生命周期：包括环境初始化、查询硬件资源（NPU 数量）以及最后的资源释放。对于昇腾 310B 处理器的开发板，其可用 NPU 设备数量为 1，程序输出结果与硬件实际情况一致。

从上述示例可以看出，环境初始化与资源释放是 PyACL 应用开发的必要环节。在调用任何核心功能接口前，必须先执行 `acl.init(config_path)`。若跳过初始化步骤，后续所有接口调用可能失败并返回错误码（如 `ACL_ERROR_UNINITIALIZED`），导致程序无法正常运行。在没有特殊配置需求时，`acl.init` 可直接传入空字符串或指向空白 JSON 文件的路径。

同样，程序运行结束后必须执行 `acl.finalize()` 以确保资源被正确回收。如果缺少资源释放环节，可能会引发内存泄漏、NPU 算力资源被持续占用或设备状态异常，进而影响后续任务的执行甚至导致系统崩溃。

值得注意的是，对于上述简单的设备查询程序，即使没有显式调用初始化和资源释放接口，程序通常也能够返回正确的结果。例如，我们可以将代码简化为如下一行的命令并在终端执行：

```
python -c "import acl; print(f'Found {acl.rt.get_device_count()[0]} Ascend devices.')"
```

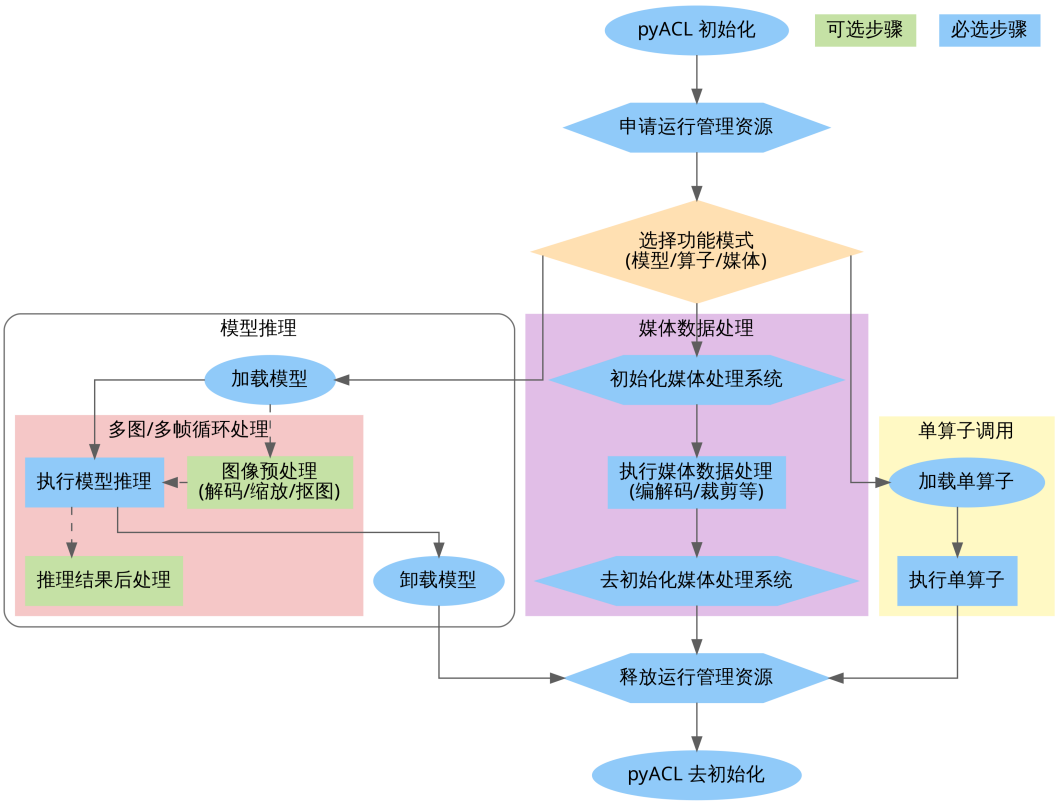
这种现象的原因主要有两点：

1. **接口依赖性较低**：`acl.rt.get_device_count` 属于基础的硬件查询接口，在某些版本的驱动实现中，这类轻量级查询操作可能并不严格依赖完整的全局初始化环境即可访问驱动状态。
2. **操作系统资源回收**：虽然程序没有显式调用 `acl.finalize()`，但当 Python 进程退出时，操作系统会自动关闭相关的文件描述符并回收该进程占用的资源。因此，多次运行该脚本通常不会立刻导致系统资源耗尽或报错。

但必须强调，这属于非规范用法。在涉及模型加载、内存申请或硬件加速（DVPP）等核心功能时，跳过 `acl.init` 可能会导致报错。为了保证程序的健壮性与兼容性，开发者应始终坚持规范的 “初始化-业务执行-资源释放” 流程。

4.1.4. PyACL 接口调用流程

调用 PyACL 接口开发的 AI 应用通常遵循一套标准化的逻辑流程，涵盖从环境初始化、硬件资源申请、业务计算执行到资源销毁的完整生命周期。开发者可以根据业务需求，将模型推理、媒体数据处理（DVPP）或单算子加速等功能进行独立部署或组合使用，如下图所示：



{fig:pyacl_interface width=100% .center}

根据上图展示的 PyACL 接口调用流程，我们可以将开发过程清晰地划分为三个主要阶段：**系统初始化**、**核心功能执行**以及**资源释放**。

1. **系统初始化与资源申请（公共基础）**无论开发何种类型的应用，起步动作都是统一的。首先调用 `acl.init` 进行全局初始化，随后申请运行管理资源。这通常涉及指定计算设备（Device）、创建上下文（Context）以及创建用于管理任务流的 Stream。这些资源构成了后续所有计算任务的基础环境。
2. **核心功能执行** 在资源就绪后，开发者根据具体业务需求选择相应的执行路径：

- **模型推理链路（左侧分支）**：这是最典型的 AI 应用场景。开发者首先调用接口加载离线模型（.om），随后在业务循环中对每一帧图像或数据进行必要的预处理（如使用 DVPP 硬件进行解码与缩放）将其转化为模型所需格式，接着调用模型执行接口完成推理，并在获取推理结果后执行解析分类标签或画框等后处理逻辑，最后在任务完成后及时卸载模型以释放内存资源。
- **媒体数据处理链路（中间分支）**：如果应用仅涉及图像编解码或视频处理而无需推理，可以直接初始化媒体系统，调用 DVPP 接口进行处理，随后去初始化。
- **单算子调用链路（右侧分支）**：适用于不需要加载完整网络模型，仅需执行特定矩阵运算或数学函数的场景。流程简化为加载特定算子描述并直接执行算子。

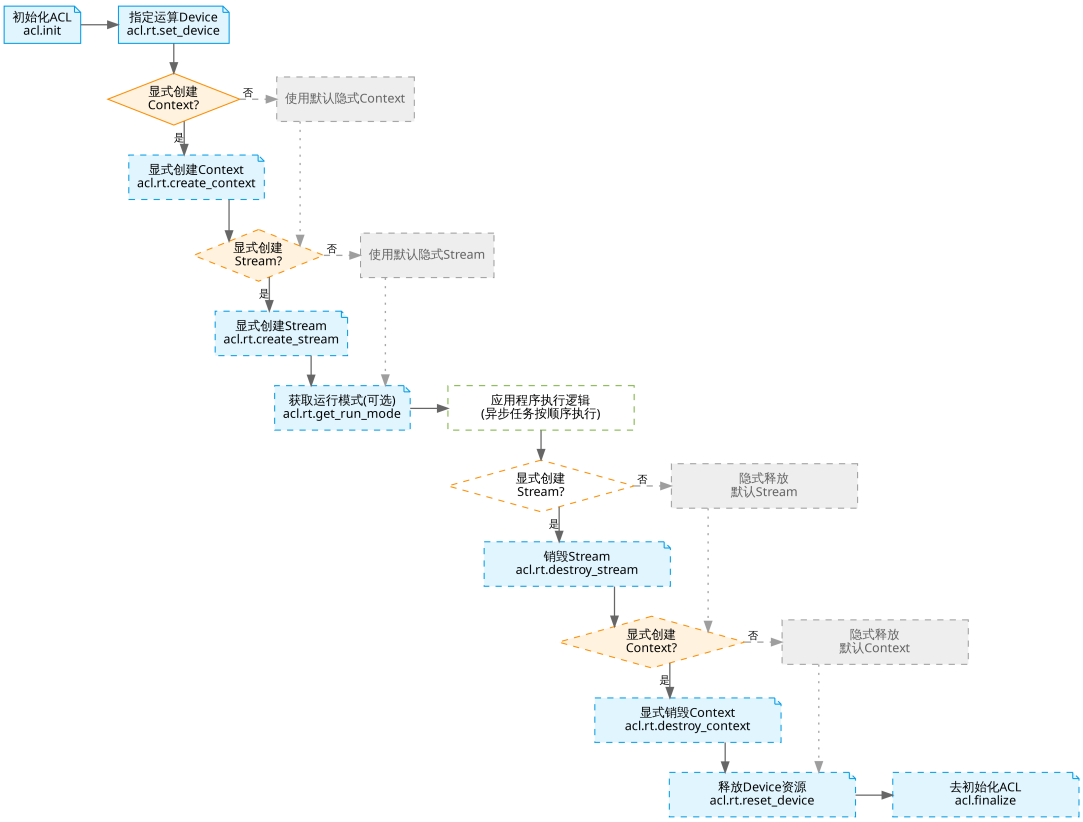
3. **资源释放与去初始化** 当业务逻辑执行完毕后，必须严格按照逆序释放资源：先销毁 Stream 和 Context，再重置 Device，最后调用 `acl.finalize` 完成 PyACL 的去初始化。这一步对于防止内存泄漏和保证系统稳定性至关重要。

此流程图清晰地界定了必选步骤（蓝色）与可选的业务逻辑步骤（绿色），帮助开发者建立规范的编程思维。

4.1.5. 运行管理资源生命周期

PyACL 的资源管理构建在 **Device**、**Context** 与 **Stream** 三个核心概念之上。在应用启动阶段，必须首先调用 `acl.init` 完成全局环境初始化。随后，通过 `acl.rt.set_device` 指定计算所需的物理 NPU 设备。

开发应用时，应用程序中必须包含运行管理资源申请的代码逻辑，您需要按照 Device、Stream 的顺序依次申请。其中，创建 Stream 的方式分为隐式创建和显式创建，其适用场景有所不同，运行资源的申请与释放的流程如下图所示：



{fig:pyacl_stream width=100% .center}

上图展示了 PyACL 资源管理的完整生命周期流程。整个流程严格遵循 “先申请后释放，谁申请谁释放” 的原则，主要包含以下几个关键步骤：

1. **ACL 初始化 (`acl.init`)**：这是所有 AscendCL 操作的起点。必须在进程启动的最开始调用，用于初始化 ACL 的全局配置。
2. **资源申请 (`set_device/create_context/create_stream`)**：开发者通过 `acl.rt.set_device` ↪ 锁定物理硬件资源。如果应用没有显式调用 `acl.rt.create_context` 或 `acl.rt.` ↪ `create_stream`，系统在调用 `set_device` 后会自动创建并关联**默认 Context** 与**默认 Stream**。但在生产环境或多线程并发场景下，建议显式创建这些资源，以实现更好的逻辑隔离和任务异步调度。Stream 作为任务执行队列，负责管理指令在硬件上的下发顺序。
3. **业务执行 (Execution)**：在此阶段，应用进行模型加载、数据预处理、推理计算等核心逻辑。所有的计算任务（Kernel）都会被下发到之前创建的 Stream 中。
4. **资源销毁**：业务完成后，必须按特定顺序释放资源。对于**显式创建**的资源，应首先调用 `acl.rt.destroy_stream` 销毁 Stream，再调用 `acl.rt.destroy_context` 销毁 Context。最后调用 `acl.rt.reset_device` 重置设备以彻底释放 Device 资源。需要特别说明的是，若未显式创建 Context 与 Stream（即使用系统默认资源），开发者**不能**调用相

应的销毁接口；在这种情况下，直接调用 `reset_device` 即可隐式地销毁关联的默认 Context 与 Stream 资源。

5. **ACL 去初始化 (`acl.finalize`)**：这是进程退出的最后一步，用于彻底清理全局资源。开发者应严格遵循 “先业务、再流、后设备” 的逆序原则进行资源释放，最后调用此接口退出环境，以避免内存泄漏或设备状态异常。

关键点总结：

- **顺序至关重要**：资源的申请与释放必须严格遵循层级逻辑。申请资源时，按照 **`Device -> Context -> Stream`** 的顺序依次创建；释放资源时，则必须严格遵循 **`Stream -> Context -> Device`** 的逆序原则。如果先重置了 Device，依附于其上的 Context 和 Stream 将变为非法状态，导致不可预知的系统错误。
- **显式 vs 隐式**：虽然隐式创建（直接 `set_device` 后 `create_stream`）代码更少，但为了代码的健壮性和可维护性，推荐在生产环境代码中始终使用**显式创建 Context 和 Stream** 的流程。

以下是 PyACL 资源申请与释放的标准逻辑示例。这段伪代码展示了在典型应用场景下，如何按照规范的生命周期管理硬件资源：

```
import acl

# 1. 环境初始化
ret = acl.init("")

# 2. 指定计算设备
device_id = 0
ret = acl.rt.set_device(device_id)

# 3. 显式创建 Context 并绑定到当前线程
context, ret = acl.rt.create_context(device_id)
ret = acl.rt.set_context(context)

# 4. 显式创建执行流（属于上述 Context）
stream, ret = acl.rt.create_stream()

# ---------------------------------------------------------------
# ... 执行核心业务逻辑（如模型推理、算子调用或媒体处理）...
# ---------------------------------------------------------------

# 5. 销毁执行流（先释放 Stream）
ret = acl.rt.destroy_stream(stream)

# 6. 销毁 Context（在销毁 Stream 之后）
ret = acl.rt.destroy_context(context)

# 7. 重置设备（释放 Device 相关资源）
ret = acl.rt.reset_device(device_id)

```

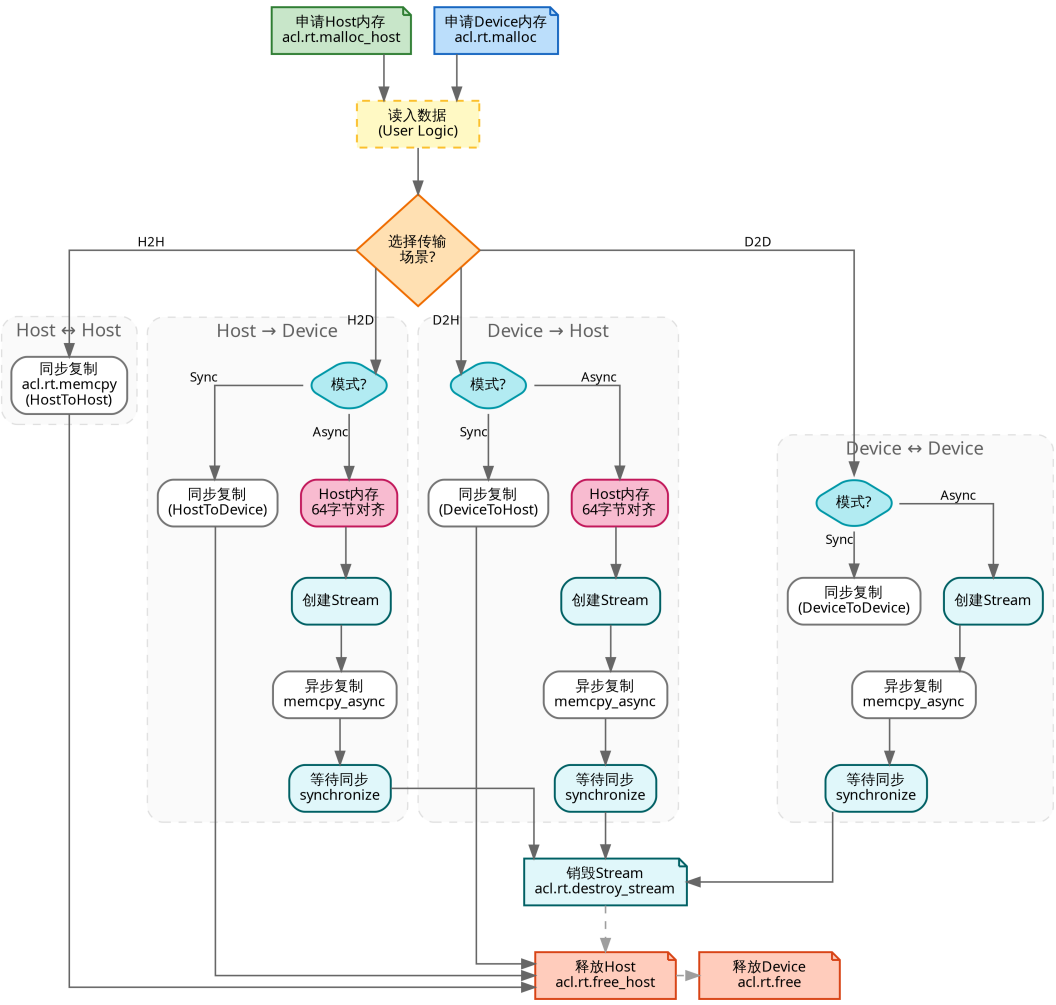
```
# 8. 系统去初始化
ret = acl.finalize()
```

关键步骤说明:

- **初始化与去初始化**：`acl.init` 和 `acl.finalize` 必须在进程级别成对调用。未初始化直接调用其他接口会导致运行报错。
- **隐式 Context 机制**：示例中利用 `acl.rt.set_device` 隐式创建了 Context。在复杂的并发场景下，建议使用 `acl.rt.create_context` 显式创建，以便更精确地控制资源隔离。
- **资源释放顺序**：代码严格遵循了 **"后申请，先释放"** 的逆序原则。如果先重置设备再销毁流，会导致非法句柄访问，进而引发系统崩溃或内存异常。
- **返回值检查**：虽然本示例为简化逻辑未展示 `ret` 判断，但在实际工程中，**必须**检查每一个接口的返回值（`0` 代表成功），以便在资源不足或硬件异常时及时捕获错误。

4.1.6. 异构内存管理

由于昇腾 AI 处理器拥有独立的存储单元，应用开发涉及 **Host**（CPU 侧）与 **Device**（NPU 侧）两部分内存。开发者通常面临频繁的数据交互需求：通过 `acl.rt.malloc` 申请 Device 侧内存用于 NPU 计算，或通过 `acl.rt.malloc_host` 申请 Host 内存。数据的流动则依靠 `acl.rt.memcpy` 完成，通过定义传输方向，将采集到的源数据搬运到 NPU 计算单元，或将计算出的结果拉回 Host 进行后处理。内存数据的流动方向一共有四种：Host 到 Host、Host 到 Device、Device 到 Host 以及 Device 到 Device，分别对应 `ACL_MEMCPY_HOST_TO_HOST` ↪ 、`ACL_MEMCPY_HOST_TO_DEVICE`、`ACL_MEMCPY_DEVICE_TO_HOST` 和 `ACL_MEMCPY_DEVICE_TO_DEVICE`。具体的流程图如下图所示：



{fig:pyacl_memory width=100% .center}

数据在 Host（CPU）与 Device（NPU）之间的拷贝主要有四种常见路径：Host 到 Host、Host 到 Device、Device 到 Host 以及 Device 到 Device，分别对应四个常量：`ACL_MEMCPY_HOST_TO_HOST`、`ACL_MEMCPY_HOST_TO_DEVICE`、`ACL_MEMCPY_DEVICE_TO_HOST` 和 `ACL_MEMCPY_DEVICE_TO_DEVICE`。

标准的处理流程通常由以下步骤组成：首先在 Host 侧准备好输入数据；接着使用 `acl.rt.malloc` 或 `acl.rt.malloc_host` 申请目标内存空间；随后调用 `acl.rt.memcpy` 指定拷贝方向并执行数据传输（支持同步或异步模式）；待数据传输至 Device 后进行计算；最后将计算结果通过 Device 到 Host 的拷贝路径拉回 Host 侧进行后处理。在此过程中，需特别注意内存对齐要求，以及在异步拷贝时必须配合 Stream 同步操作以确保数据可用。

以下是这四种内存传输模式的详细说明与关键点解析：

1. Host 到 Host (ACL_MEMCPY_HOST_TO_HOST) 纯 CPU 端的内存拷贝通常用于进程内部的数据移动或不同 Host 缓冲区之间的数据整理。该过程无需关注 Device 端的内存对齐要求，也不涉及 Stream，直接使用同步拷贝方式即可。以下是 Host 到 Host

同步拷贝的示例代码，首先申请两块 Host 内存，读取数据后，调用 `acl.rt.memcpy` 并指定拷贝类型为 1（即 ACL_MEMCPY_HOST_TO_HOST）：

```
# Host 到 Host 同步拷贝
host_src, ret = acl.rt.malloc_host(size)
host_dst, ret = acl.rt.malloc_host(size)
# 假设 read_file 为用户自定义的数据加载函数
read_file(file, host_src, size)
ret = acl.rt.memcpy(host_dst, size, host_src, size, 1) # 1 代表 ACL_MEMCPY_HOST_TO_HOST

# 释放资源
acl.rt.free_host(host_src)
acl.rt.free_host(host_dst)
```

2. Host 到 Device (ACL_MEMCPY_HOST_TO_DEVICE) 将 Host 侧的数据上传至 NPU（Device 侧）是推理任务的起点。该过程支持同步或异步方式。若采用异步上传，需确保 Host 内存满足 Device 的访问要求，通常建议使用 `acl.rt.malloc_host` 申请。在异步模式下，开发者必须显式创建 Stream，并在后续使用 Device 数据前执行同步等待操作，以确保数据传输完整。

以下是同步与异步拷贝的实现示例：

```
# 准备内存
host_ptr, _ = acl.rt.malloc_host(size)
dev_ptr, _  = acl.rt.malloc(size, 2)  # 2 代表 ACL_MEM_MALLOC_HUGE (Device 内存)
read_file(file, host_ptr, size)

# 方式一：同步拷贝
# 2 代表 ACL_MEMCPY_HOST_TO_DEVICE
acl.rt.memcpy(dev_ptr, size, host_ptr, size, 2)

# 方式二：异步拷贝
stream, _ = acl.rt.create_stream()
acl.rt.memcpy_async(dev_ptr, size, host_ptr, size, 2, stream)
acl.rt.synchronize_stream(stream) # 等待数据传输完成

# 释放资源（注意顺序）
acl.rt.free_host(host_ptr)
acl.rt.free(dev_ptr)
acl.rt.destroy_stream(stream)
```

3. Device 到 Host (ACL_MEMCPY_DEVICE_TO_HOST) 将 Device 侧的计算结果回传至 Host 侧是推理流程的最后一步，主要用于后续的业务结果解析或数据持久化。异步回传支持与 Device 侧的计算任务并行处理，但必须在 Host 侧访问该数据前执行 Stream 同步。在内存分配上，建议使用 `acl.rt.malloc_host` 申请 Host 侧缓冲区，以确保 Device 侧的 DMA 能够高效地完成数据交互。

```
# 准备内存
```

```
out_dev, _ = acl.rt.malloc(out_size, 2)
out_host, _ = acl.rt.malloc_host(out_size)

# 假设 Device 侧计算已完成，执行异步拷贝 (3 代表 ACL_MEMCPY_DEVICE_TO_HOST)
acl.rt.memcpy_async(out_host, out_size, out_dev, out_size, 3, stream)

# 必须等待拷贝流执行完毕，确保数据已完整到达 Host
acl.rt.synchronize_stream(stream)

# 此时可进行后处理并释放资源
acl.rt.free(out_dev)
acl.rt.free_host(out_host)
```

4. Device 到 Device (ACL_MEMCPY_DEVICE_TO_DEVICE) 在 Device 内部不同缓冲区之间进行拷贝，或者在多 Device 场景下跨设备传输数据，这里主要注意的是，对于昇腾 310B 开发板来说，一般来说，一个昇腾 310B 开发板只有一个 NPU，也就是说，只有一个 Device。这种模式常用于数据格式重排或设备间通信。

该操作通常配合异步 Stream 使用，以避免阻塞 Host 线程。在执行跨设备拷贝时，开发者需要注意上下文切换或使用特定的点对点传输接口。以下是同一 Device 内执行异步拷贝的示例代码。程序首先申请两个 Device 内存块，随后通过 `acl.rt.memcpy_async` 执行拷贝，并将传输类型指定为 4（即 ACL_MEMCPY_DEVICE_TO_DEVICE）：

```
# 同一 Device 内的异步拷贝
dev_a, _ = acl.rt.malloc(size, 2)
dev_b, _ = acl.rt.malloc(size, 2)
stream, _ = acl.rt.create_stream()

# 4 代表 ACL_MEMCPY_DEVICE_TO_DEVICE
acl.rt.memcpy_async(dev_b, size, dev_a, size, 4, stream)
acl.rt.synchronize_stream(stream)

# 释放资源
acl.rt.free(dev_a)
acl.rt.free(dev_b)
acl.rt.destroy_stream(stream)
```

该模式常用于设备内数据重排或多设备间的点对点传输。在昇腾 310B 等单 NPU 平台上，跨设备传输并不常见；在多 NPU 环境下，应通过切换 Context 或使用专用点对点传输接口以减少额外拷贝开销。建议始终配合异步 Stream 并在访问目标缓冲区前调用同步接口（如 synchronize_stream 或 stream_wait_event）以确保数据已就绪，同时注意内存对齐与访问权限，避免 DMA 访问异常。

通用开发注意事项

1. **异步与同步**：异步拷贝接口（`memcpy_async`）必须配合 Stream 以及 `synchronize_stream` 使用，否则无法保证数据一致性。

2. **内存类型**：为了提升传输效率，Host 侧供 Device 访问的内存建议始终使用 `acl.rt.malloc_host` 申请。
3. **释放顺序**：资源释放应遵循严格的逆序原则——先销毁 Stream，再释放内存，最后重置 Device。
4. **错误处理**：示例代码为保持简洁略去了错误检查，但在实际开发中，所有 ACL 接口调用的返回值（`ret`）都必须进行检查（`0` 表示成功）。

4.1.7. 同步等待机制

从上一节关于内存拷贝的四种路径中，我们可以观察到 `acl.rt.memcpy` 与 `acl.rt.memcpy_async` 两种截然不同的操作模式。这两种模式的选择，本质上是在**编程简易性**与**执行性能**之间做权衡。

同步与异步的概念

同步操作（Synchronous）是最符合直觉的编程方式，它遵循严格的“请求-响应”逻辑。正如我们在 Host 到 Host 拷贝示例中所见，当 Host 线程发起一个同步指令时（如 `acl.rt.memcpy`），CPU 会挂起当前线程，像监工一样死死盯着任务，直到任务彻底完成后才会恢复执行下一行代码。这种模式的优点显而易见：逻辑简单，数据一致性由代码执行顺序天然保证，非常适合初学者进行功能验证或定位 BUG。但其缺点也同样致命：它完全抹杀了硬件并行的可能性。试想，当 CPU 在傻傻等待数据从 Host 搬运到 Device 时，昂贵的 NPU 计算单元可能正处于空闲状态，导致系统整体吞吐量下降。

异步操作（Asynchronous）则打破了这种串行束缚，是高性能 AI 应用的标配。在调用 `acl.rt.memcpy_async` 时，Host 线程仅需将任务“投递”到 Stream 队列中便立即返回，继续处理其他逻辑（如读取下一张图片或预处理数据）。此时，底层的 DMA 搬运引擎会与 NPU 的计算引擎同时工作，实现了真正的**软硬件并行**。这就好比点餐系统，前台服务员（Host）只负责快速接单并把单子（Task）扔给厨房（Stream），而不需要站在厨房门口等菜做好，从而能接待更多的客人。在复杂的 AI 业务流中，这种机制允许我们构建精妙的流水线：**在 NPU 拼命推理当前帧的同时，DMA 正在默默地将下一帧数据搬运进显存，而 CPU 已经在预处理第三帧数据**。这种“想尽办法让显卡即使一毫秒都不闲着”的设计，正是提升 AI 应用帧率的关键。

然而，异步操作是一把双刃剑，带来的性能红利必须以**严谨的同步控制**为代价。因为 Host 线程“投递”完任务就跑了，如果不加干预，它很可能在数据还没搬运完时就开始尝试读取结果，或者在 NPU 还没用完数据时就释放了内存，从而引发数据错乱甚至程序崩溃。因此，异步开发必须配合**同步等待机制**，在关键的时间节点上让“脱缰”的并行任务重新对齐。

PyACL 的四种同步机制

PyACL 提供了四种不同粒度的同步等待机制，以适应从简单的单流控制到复杂的多流并行协作等不同场景。正确选择同步方式是平衡 Host 侧控制流与 Device 侧数据流的关键。

1. **Event 的同步等待 (`acl.rt.synchronize_event`)** Event 是 PyACL 中的 “时间锚点” 或 “完成标记”。这种机制允许 Host 侧主动阻塞，直到某个特定的 Event 在 Device 侧被触发。其典型流程是：开发者创建一个 Event，将其记录（Record）到某个 Stream 的任务队列中；当 Stream 执行到该记录点时，会在硬件层面触发该 Event；Host 线程通过调用 `synchronize_event` 暂停自身执行，直至接收到触发信号。这种方式粒度适中，适合 Host 需要等待某个特定任务节点完成后再进行后续逻辑（如读取特定阶段的结果），且该 Event 可能被多个等待者复用的场景。

```
# 伪代码：Host 等待特定 Event
event, _ = acl.rt.create_event()
acl.rt.memcpy_async(dev_ptr, size, host_ptr, size, 1, stream) # 异步拷贝
acl.rt.record_event(event, stream) # 在流中安插一个锚点

# ... Host 执行其他不依赖数据的逻辑 ...

acl.rt.synchronize_event(event) # Host 在此阻塞，直到上面的拷贝完成
# 此时可以安全释放 host_ptr 或读取 dev_ptr
```

2. **Stream 内任务的同步等待 (`acl.rt.synchronize_stream`)** 这是最常用且最直观的同步方式，它表示 Host 侧阻塞等待指定 Stream 中的**所有**已提交任务全部执行完毕。其语义简单直接，保证了指定流水线上的所有操作都已落盘。常用于单 Stream 流水线的收尾阶段，或者 Host 必须确保整个任务队列清空后才能继续（例如释放 Stream 资源前）。虽然简单，但它是粗粒度的阻塞操作，如果 Stream 中积压任务较多，会导致 Host 长时间等待。

```
# 伪代码：Host 等待整个流清空
acl.rt.memcpy_async(..., stream) # 任务1
acl.op.execute_v2(..., stream)   # 任务2
acl.rt.memcpy_async(..., stream) # 任务3

acl.rt.synchronize_stream(stream) # Host 阻塞，直到任务1,2,3全部完成
# 此时所有任务均已结束
```

3. **Stream 间的同步等待 (`acl.rt.stream_wait_event`)** 这是实现**多流并行与主要流水线协作**的核心机制。与前两者不同，`stream_wait_event` **不会阻塞 Host 线程**，而是向 Consumer Stream（消费者流）中下发一个 “等待指令”。Consumer Stream 执行到该指令时，会暂停处理后续任务，直到 Producer Stream（生产者流）触发了指定的 Event。这种机制完全在 PyACL 内部调度和 Device 硬件层面完成，Host 仅

负责编排逻辑，从而最大化了 CPU 与 NPU 的并行效率。它非常适合跨流的数据依赖场景，例如：Stream A 负责数据搬运，Stream B 负责计算，Stream B 必须等待 Stream A 搬运完成后才能开始计算。

```
# 伪代码：Stream 间协作 (不阻塞 Host)
# Stream A: 搬运数据 -> Record Event
acl.rt.memcpy_async(dev_input, ..., stream_a)
acl.rt.record_event(event, stream_a) # 记录搬运完成

# Stream B: 等待数据 -> 开始计算
acl.rt.stream_wait_event(stream_b, event) # Stream B 暂停，直到 Stream A 触发 Event
acl.mdl.execute_async(..., stream_b)      # 只有等数据搬运完了，才会执行推理

# Host 这里是非阻塞的，可以立即继续运行
```

4. **Device 的同步等待 (`acl.rt.synchronize_device`)** 这是粒度最粗的全局同步操作。调用此接口会阻塞 Host 进程，直到当前 Context 绑定的 Device 上**所有 Stream 的所有任务**全部完成。虽然它能保证设备层面的全局一致性，但由于其 “一刀切” 的等待特性，极大破坏了并行性，且开销最高。通常仅在程序初始化调试、全局状态检查或程序退出前的最终资源回收阶段使用，在高性能的生产业务循环中应尽量避免。

```
# 伪代码：全局等待
acl.rt.memcpy_async(..., stream1)
acl.rt.memcpy_async(..., stream2)

# 强制等待设备上所有任务结束 (调试或退出时使用)
acl.rt.synchronize_device()
```

昇腾 310B 最佳同步策略与场景分析

在昇腾 310B 这种边缘计算平台上，CPU 的单核性能通常弱于服务器级 CPU，因此**减少 Host 侧（CPU）的阻塞**、把调度压力卸载给 Device 侧硬件显得尤为关键。为了更直观地理解为什么在昇腾 310B 上必须强调 “CPU 减负” 与 “异步并行”，我们需要深入分析这颗 SoC 的 CPU 性能定位。与市面上其他主流的边缘计算开发板相比，昇腾 310B 呈现出显著的 **“NPU 极强，CPU 较弱”** 的异构特性。

下表详细对比了昇腾 310B 与树莓派 5、RK3588 以及 NVIDIA Jetson Orin Nano 在 CPU 规格上的差异：

平台名称	核心处理器 (SoC)	CPU 架构	核心配置	主频 (典型)	CPU 算力估算 (UnixBench)
昇腾 310B	Ascend 310B	ARM Cortex-A55	**4 核 (纯小核)**	1.0 GHz ~ 1.6 GHz	~ 400 - 600 分
树莓派 5	BCM2712	ARM Cortex-A76	4 核 (大核)	2.4 GHz	~ 2400+ 分

平台名称	核心处理器 (SoC)	CPU 架构	核心配置	主频 (典型)	CPU 算力估算 (UnixBench)
Orange Pi 5	RK3588	Cortex-A76 + A55	4 大核 + 4 小核	2.4 GHz (大) / 1.8 GHz (小)	~ 5000+ 分
Jetson Orin Nano	Tegra Orin	ARM Cortex-A78AE	6 核 (高性能核)	1.5 GHz	~ 3500+ 分
桌面 PC	Intel i7-12700K	x86-64 (Alder Lake)	8 大核 + 4 小核	3.6 GHz ~ 5.0 GHz	~ 45000+ 分

数据解读与性能差距分析：

1. **架构代差（效率核 vs 性能核）**：昇腾 310B 采用的是 **Cortex-A55** 架构。在 ARM 的设计体系中，A55 被定义为 “高能效核心（Efficiency Core）”，通常用于手机处理器的后台任务或物联网设备，其侧重点在于低功耗而非高性能。相比之下，树莓派 5 使用的 Cortex-A76 和 Orin Nano 使用的 Cortex-A78AE 均属于 “高性能核心（Performance Core）”。在同频下，A76 的整数计算能力约为 A55 的 2~3 倍，浮点性能更是相差甚远。
2. **绝对性能差距**：从综合基准测试（如 UnixBench 或 Geekbench）来看，昇腾 310B 的 CPU 性能大约仅为树莓派 5 的 **1/4 到 1/5**，仅为 RK3588 的 **1/8** 左右。这意味着，同样一段基于 OpenCV 的纯 CPU 图像缩放代码，在树莓派 5 上可能耗时 5ms，而在昇腾 310B 上可能耗时 25ms 甚至更多。
3. **与桌面 CPU 的鸿沟**：如果是从在笔记本（x86, i5/i7）上开发算法迁移到 310B，这种落差会更加剧烈，性能折损可能达到 **50 倍甚至 100 倍**。桌面 CPU 拥有巨大的二级/三级缓存和乱序执行能力，这掩盖了 Python 解释器本身的低效。但在 310B 上，Python 的 Global Interpreter Lock (GIL) 加上较弱的单核性能，极易成为系统的最大短板。

对 PyACL 开发的启示：
基于上述硬件数据，我们在开发中必须遵循以下 “生存法则”：

- **不要信任 CPU 的浮点计算能力**：任何涉及图像 Pixels 遍历的操作（如 Resize, Color Convert, Normalize）若写在 CPU (Python) 端，必将导致帧率骤降。**必须使用 DVPP 硬件加速**（详见第 5 章）。
- **Python 代码仅做胶水**：Python 逻辑应仅限于流程控制、参数配置和极少量的后处理。业务主体必须由底层的 C++ 算子或 NPU 模型承担。
- **异步是救命稻草**：由于 CPU 处理每一行 Python 代码都比其他平台慢，因此更不能让 CPU 傻傻等待 NPU（同步）。只有利用 `stream_wait_event` 让 CPU 快速把任务分发完并脱身，才能掩盖 A55 核心性能不足的缺陷。

1. 优先策略：全链路异步与细粒度同步 在昇腾 310B 平台上，由于 CPU 算力相对有限，最理想的流水线策略是实施“全链路异步”设计，即让 CPU 专注于指令分发，而将繁重的计算负载全权交由 NPU 负责。开发者应尽量避免使用 `acl.rt.memcpy` 等同步阻塞接口，转而全面采用 `acl.rt.memcpy_async` 配合 Event 机制。这种方法的核心原则在于，凡是能通过 Device 侧 `stream_wait_event` 解决的任务依赖，绝不让 Host 侧的 CPU 介入干预。例如，在典型的多级推理流水线中，我们可以安排 Stream A 负责视频解码（DVPP），Stream B 负责模型推理。当 Stream A 完成解码任务后，只需在流中记录一个 Event，而 Stream B 仅需等待该 Event 触发即可启动推理。整个交互过程中，CPU 仅需极低开销下发几条控制指令，完全无需挂起等待解码结束，从而能腾出宝贵算力去处理复杂的业务逻辑或网络通信，实现真正的软硬件解耦与并行。

```
# 伪代码：利用 Event 实现解码与推理的异步流水线
# stream_dvpp 负责解码，stream_infer 负责推理
acl.media.dvpp_jpeg_decode_async(..., stream_dvpp)

# 1. 在解码流中记录一个“完成节点”
event_decode_done, _ = acl.rt.create_event()
acl.rt.record_event(event_decode_done, stream_dvpp)

# 2. 让推理流等待这个节点（Host 不阻塞，仅 NPU 内部等待）
acl.rt.stream_wait_event(stream_infer, event_decode_done)

# 3. 只有当解码完成后，推理流才会开始执行模型
acl.mdl.execute_async(..., stream_infer)
```

2. 数据传输优化：Host Pinned Memory 的强制管理 实现异步传输的高性能前提是 Host 侧内存的稳定性。在使用 `acl.rt.memcpy_async` 进行 Host 到 Device 的数据搬运时，源端的 Host 内存**必须**是通过 `acl.rt.malloc_host` 申请的页锁定内存（Pinned Memory）。普通的 `malloc` 申请的内存可能会被操作系统分页机制换出到磁盘交换区，导致 DMA 控制器无法安全、准确地访问数据。此外，内存的生命周期管理至关重要。在释放 Host 侧指针之前，必须通过 `acl.rt.synchronize_stream` 等手段确保所有涉及该内存块的 Stream 任务都已彻底执行完毕。如果过早释放内存，NPU 正尝试读取数据时地址已失效，将导致读取到野指针数据，引发难以复现的推理错误或系统崩溃。

```
# 伪代码：异步传输的内存管理规范
# 必须使用 acl.rt.malloc_host 申请 Pinned Memory
host_ptr, _ = acl.rt.malloc_host(size)

# ... 填充数据到 host_ptr ...

# 执行异步拷贝
acl.rt.memcpy_async(dev_ptr, size, host_ptr, size, 1, stream)

# 错误做法：直接释放 host_ptr (DMA 可能还在搬运中！)
```

```
# acl.rt.free_host(host_ptr)

# 正确做法：先同步等待流结束，再释放
acl.rt.synchronize_stream(stream)
acl.rt.free_host(host_ptr)
```

3. 多 Stream 协作范式：生产者-消费者模型 构建基于生产者-消费者模型的多 Stream 协作机制，是挖掘 310B 硬件潜能、提升应用帧率（FPS）的关键手段。具体的实施路径是将通过 Stream 的划分将任务解耦为 “预处理”、“推理” 和 “后处理” 等独立阶段，通过 Event 机制实现流水线衔接。这种模式下，前一个阶段完成后记录 Event，后一个阶段感应 Event 并启动，就像工厂流水线一样运转。其最大优势在于避免了 CPU 使用 `while` 循环去轮询 NPU 的状态，极大降低了 CPU 占用率。对于昇腾 310B 这类嵌入式 SoC 而言，节省下来的 CPU 开销意味着开发者可以在 Python 层运行更复杂的逻辑判断，从而提升整个系统的智能化水平。

```
# 伪代码：多 Stream 协作
# 循环中处理每一帧
for image in images:
    # 阶段 1: 预处理流 (Stream A)
    preprocess(image, stream_a)
    acl.rt.record_event(evt_pre, stream_a)

    # 阶段 2: 推理流 (Stream B)
    # 只有当预处理完成，推理才开始
    acl.rt.stream_wait_event(stream_b, evt_pre)
    model_inference(stream_b)
    acl.rt.record_event(evt_infer, stream_b)

    # 阶段 3: 后处理流 (Stream C) or Host 读取
    # 只有推理完成，才处理结果
    acl.rt.stream_wait_event(stream_c, evt_infer)
    post_process(stream_c)
```

4. 避免滥用全局同步与调试建议 在追求高性能的同时，必须警惕对同步接口的滥用。最典型的反模式是在每一帧的处理循环中都调用 `acl.rt.synchronize_device()`，这种做法相当于强制将所有并行的流水线 “拍扁” 为串行执行，完全浪费了 NPU 的并行处理能力。全局同步应当被严格限制在程序初始化（确保设备状态复位）、调试阶段（精确定位错误发生的算子）或进程退出阶段（安全回收所有资源）。对于开发者而言，调试异步程序确实存在挑战，因为报错行往往滞后于实际错误发生点。因此，建议遵循 “从同步到异步” 的演进路线：在开发初期，全部使用同步接口（如 `synchronize_stream`）确保逻辑正确性和内存安全；进入性能调优阶段后，再逐步替换为异步接口并引入 Event 机制，配合 Profiling 工具观察流水线中的空隙（Bubble），逐步压榨硬件性能。

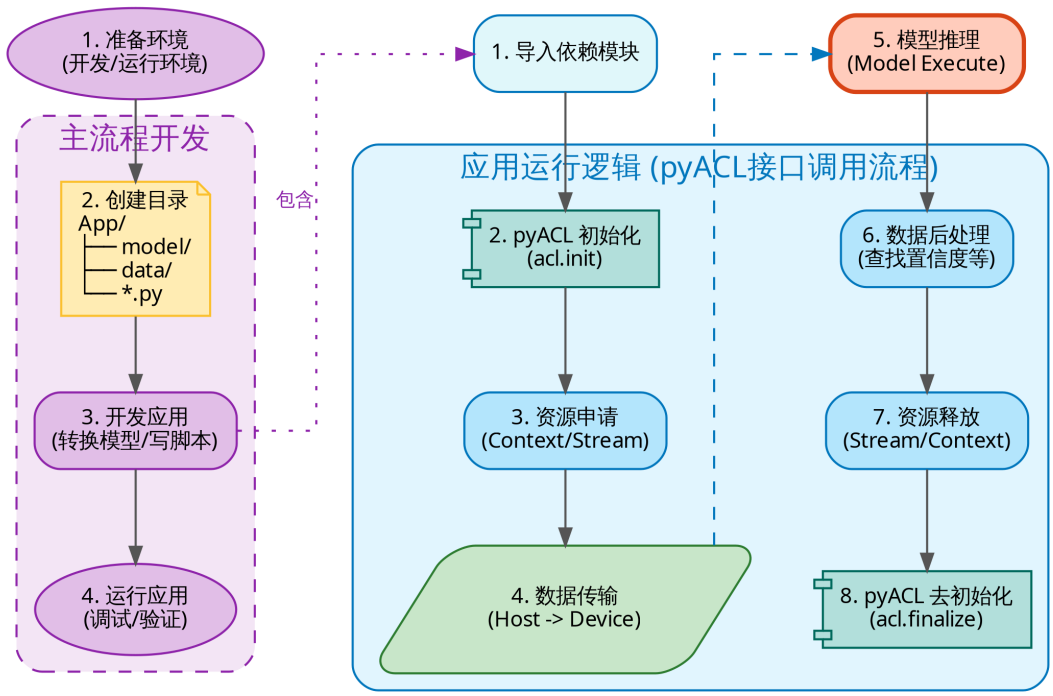
小结 掌握 PyACL 同步机制的核心在于理解 **“控制流（CPU）与数据流（NPU）的分离”**。在昇腾 310B 开发中，优秀的架构设计应当是：Host 线程像一个从容的指挥官，通过 Event 编排好各 Stream 的协作顺序后便抽身而去，留给 NPU 硬件去并行处理繁重的数据搬运与计算任务。合理使用 `stream_wait_event` 实现 Device 内部的依赖隔离，仅在必须获取最终结果时使用 `synchronize_stream` 回收数据，是在边缘端实现高性能推理的黄金法则。

4.2. 模型推理流水线

模型推理是 PyACL 应用开发的核心场景。为了能够让训练好的深度学习模型在昇腾 AI 处理器上高效运行，开发者需要遵循一套从环境准备到资源释放的标准化全流程。

4.2.1. 模型推理开发流程解析

整个模型推理应用的构建过程可以宏观地分为 **“主流程开发”** 与 **“应用运行逻辑”** 两个层面，如下图所示：



{fig:pyacl_inference width=100% .center}

主流程开发（Preparation Strategy）

主流程开发主要涉及应用构建的物理层面的准备工作。首先，开发者需要确保昇腾 AI 处理器的驱动与固件、CANN 软件（包含 pyACL）以及 Python 运行环境均已正确安装并完成环境变量配置，这是应用运行的基石。其次，为了保证项目的可维护性，建议采用标准化的目录结构，例如规划专门的 `model/` 目录存放离线模型、`data/` 目录存放测试图片或数据集，以及独立的脚本目录。紧接着进入核心开发阶段，这包括使用 ATC 工具将原始框架（如 ONNX 或 PyTorch）的模型转换为昇腾专用的 `.om` 离线模型，以及编写 Python 主程序以串联推理逻辑。最后，开发者需要在板端实际执行脚本，进行应用的验证与调试，确保从模型转换到最终输出的整个链路畅通无阻。

应用运行逻辑（Execution Logic）

这是编写 Python 脚本时的代码执行时序，严格遵循 PyACL 的接口调用规范，主要包含以下八个关键步骤：

1. **导入依赖**：`import acl` 引入 pyACL 库。
2. **系统初始化**：调用 `acl.init()` 进行全局配置初始化，这是一切操作的起点。
3. **资源申请**：创建 Device 连接，配置 Context（上下文）与 Stream（执行流），为后续计算搭建 “舞台”。
4. **数据传输（Host -> Device）**：在执行推理前，必须将待处理的图片或矩阵数据从 CPU 侧（Host）搬运至 NPU 侧（Device）。这通常涉及 `acl.rt.malloc` 申请 Device 内存以及 `acl.rt.memcpy` 执行搬运。
5. **模型推理（Inference）**：这是流程的核心。调用 `acl.mdl.execute` 接口，指示 AI Core 利用加载的 `.om` 模型对 Device 内存中的数据进行计算。
6. **数据后处理**：将推理产生的结果（通常在 Device 侧）回传至 Host 侧（或直接在 Device 侧处理），通过 Python 代码解析概率向量（Softmax）、筛选置信度或进行坐标转换。
7. **资源释放**：业务完成后，必须按 **“先释放 Stream，再释放 Context，最后重置 Device”** 的逆序释放资源。
8. **系统去初始化**：最后调用 `acl.finalize()`，通知系统回收全局资源，结束进程。

4.2.2. 实例分析（ResNet-18）

在第三章中，我们初步介绍了 ResNet 的网络结构。本章我们将继续以 ResNet 为例，深入探讨 PyACL 的编程技巧与应用。上一章中，由于昇腾 310B 的 PyTorch 插件主要用于推理加速，且端侧设备训练算力有限，我们选用了较小的 CIFAR-10 数据集。而在本章，我们将采用更贴近实际生产的标准开发流程，并引入 **Tiny-ImageNet** 数据集进行实战演练。我们首先利用 Nvidia 显卡配合 CUDA 和 PyTorch 对 ResNet-18 进行训练，获取模

型权重；随后将其转换为 ONNX 模型，并最终转换为昇腾特定的 OM 模型。我们将分别在**昇腾 310B (NPU-8T)**、**昇腾 310B (CPU)**、**树莓派 5B (CPU)** 以及 RTX 5090D 上进行推理测试，重点对比 OnnxRuntime 与 PyACL 的性能差异，深入分析 NPU 带来的帧率提升。

Tiny-ImageNet 数据集简介

Tiny-ImageNet 是大规模视觉识别挑战赛 (ILSVRC) 中 ImageNet 数据集的一个微型子集，常被用于深度学习模型的快速原型设计与基准测试。该数据集包含 200 个不同的物体类别，这一数量远超 CIFAR-10 的 10 个类别，从而更具挑战性，能更好地验证模型的泛化能力。在数据规模方面，它拥有 100,000 张训练图片（每个类别 500 张）、10,000 张验证图片（每个类别 50 张）以及 10,000 张测试图片。所有图像的分辨率统一为 64x64 像素，虽然低于标准 ImageNet 的 224x224，但相比 CIFAR-10 的 32x32 分辨率包含了更多细节，非常适合在算力受限的嵌入式设备上进行中等规模的实验。

在模型选择上，尽管 ResNet-50 或 ResNet-101 拥有更深的网络层数和潜在的更高精度，但在嵌入式 AI 开发场景下，ResNet-18 往往是性能与效率的最佳平衡点。首先，考虑到昇腾 310B 定位为边缘计算设备，ResNet-18 约 11M 的参数量和适中的计算量能够更直观地体现 NPU 在高吞吐场景下的加速优势，避免因网络过大导致的内存瓶颈掩盖推理效率。其次，对于 64x64 分辨率的 Tiny-ImageNet，ResNet-18 已具备足够的特征提取能力，使用过深的网络反而容易导致过拟合且训练耗时过长，不利于教学演示与快速迭代。最后，ResNet-18 作为业界最通用的轻量级骨干网络之一，常被作为衡量树莓派、Jetson 以及 Ascend 等不同端侧硬件性能的经典标尺。

ResNet-18 模型训练

> **注意注意：本节训练代码需在配备 NVIDIA 显卡的 GPU 服务器上运行，不在昇腾 310B 开发板上执行**。如果只需体验昇腾 310B 上 PyACL 的推理性能，可直接跳转至 环境配置与模型转换。

如果你希望自行训练模型以便进行测试，可以参考本节的模型训练部分。我们需要一台配备 Nvidia 显卡并已正确安装 CUDA 驱动的服务器。本例中，我们使用 GeForce 5090D 显卡。对于图像分类任务，Nvidia GeForce 系列显卡已能很好地满足需求。我们选用 Tiny-ImageNet 数据集进行训练，该数据集相比完整版 ImageNet 体积更小，便于测试和实验。使用 GeForce 5090D 训练一次大约需要 1~2 小时；而若采用完整的 ImageNet 数据集，单卡训练可能需要长达一周的时间。

我们需要从 Hugging Face 下载 Tiny-ImageNet 数据集，为此首先需要使用 pip 安装 Hugging Face 的 CLI 工具。如果在下载过程中遇到网络连接问题，建议配置镜像源，例如使用 hf-mirror。可以通过以下命令设置环境变量来启用镜像：

```
export HF_ENDPOINT=https://hf-mirror.com
```

为了方便起见，我们可以运行以下命令将该环境变量添加到 .bashrc 文件中，使其永久生效：

```
echo 'export HF_ENDPOINT=https://hf-mirror.com' >> ~/.bashrc
source ~/.bashrc
```

为了避免影响服务器上已有的虚拟环境，建议提前安装 Anaconda，并通过以下命令新建独立的环境：

```
conda create -n torch
conda activate torch
conda install python=3.11
```

此处选择 Python 3.11 作为示例，实际可根据需求选择合适的版本，但不建议使用过新或过旧的版本，以减少兼容性问题。

随后，使用 pip 安装所需依赖：

```
pip install datasets huggingface_hub torch torchvision timm tensorboard
```

本例采用的 ResNet-18 模型在结构上做了针对 Tiny-ImageNet 的适配。与标准 ResNet-18 不同，输入层采用 3x3 卷积且 stride=1，移除了原有的 7x7 卷积和最大池化层，以更好地保留 64x64 小尺寸图像的空间信息。此外，模型中增加了 Dropout 层以缓解过拟合，并在损失函数中引入标签平滑（Label Smoothing）。优化器选用带动量和权重衰减的 SGD，并配合多步学习率调度器，进一步提升模型的泛化能力。

数据集的下载和加载通过 Hugging Face 的 datasets 库实现。只需一行代码即可自动下载并缓存 Tiny-ImageNet 数据集，无需手动解压和整理文件。示例代码如下：

```
from datasets import load_dataset
dataset = load_dataset('zh-plus/tiny-imagenet', cache_dir='./data')
```

在数据预处理部分，训练集采用了多种图像增强技术，包括随机裁剪（RandomCrop）、随机水平翻转（RandomHorizontalFlip）、随机旋转（RandomRotation）以及颜色抖动（ColorJitter）。这些增强方法能够人为增加训练样本的多样性，使模型在训练过程中见到更多变化的图像，有效缓解过拟合问题，提高模型的泛化能力。

此外，模型训练过程中采用了 MultiStepLR 学习率调度策略（multistep），即在训练到指定 epoch 时自动降低学习率。这种做法可以让模型在初期快速收敛，在后期以更小的步长微调参数，进一步防止过拟合。图像增强和多步学习率调度的结合，有助于提升模型在验证集和实际应用中的表现。

完成的模型训练代码如下：

```
import os
import torch
import torch.nn as nn
```

```
import torch.optim as optim
from torch.utils.data import DataLoader
from datasets import load_dataset
from torchvision.transforms import Compose, ToTensor, Normalize, RandomCrop, RandomHorizontalFlip, RandomRotation, ColorJitter # 增加更多增强
from torch.utils.tensorboard import SummaryWriter # 导入 SummaryWriter

# 自定义 ResNet 模型组件
class BasicBlock(nn.Module):
    expansion = 1

    def __init__(self, inplanes, planes, stride=1, downsample=None):
        super(BasicBlock, self).__init__()
        self.conv1 = nn.Conv2d(inplanes, planes, kernel_size=3, stride=stride, padding=1, bias=False)
        self.bn1 = nn.BatchNorm2d(planes)
        self.relu = nn.ReLU(inplace=True)
        self.conv2 = nn.Conv2d(planes, planes, kernel_size=3, padding=1, bias=False)
        self.bn2 = nn.BatchNorm2d(planes)
        self.downsample = downsample
        self.stride = stride

    def forward(self, x):
        identity = x
        out = self.conv1(x)
        out = self.bn1(out)
        out = self.relu(out)
        out = self.conv2(out)
        out = self.bn2(out)
        if self.downsample is not None:
            identity = self.downsample(x)
        out += identity
        out = self.relu(out)
        return out

# 移除 Bottleneck 类，ResNet18 使用 BasicBlock
class ResNet(nn.Module):
    def __init__(self, block, layers, num_classes=200):
        super(ResNet, self).__init__()
        self.inplanes = 64
        # 适配 Tiny ImageNet (64x64): 使用 3x3 卷积, stride=1, 移除 MaxPool
        self.conv1 = nn.Conv2d(3, 64, kernel_size=3, stride=1, padding=1, bias=False)
        self.bn1 = nn.BatchNorm2d(64)
        self.relu = nn.ReLU(inplace=True)
        # self.maxpool = nn.MaxPool2d(kernel_size=3, stride=2, padding=1) # 移除

        self.layer1 = self._make_layer(block, 64, layers[0])
        self.layer2 = self._make_layer(block, 128, layers[1], stride=2)
        self.layer3 = self._make_layer(block, 256, layers[2], stride=2)
        self.layer4 = self._make_layer(block, 512, layers[3], stride=2)
        self.avgpool = nn.AdaptiveAvgPool2d((1, 1))
        self.dropout = nn.Dropout(p=0.5) # 增加 Dropout 层
        self.fc = nn.Linear(512 * block.expansion, num_classes)
```

```
55
56          for m in self.modules():
57              if isinstance(m, nn.Conv2d):
58                  nn.init.kaiming_normal_(m.weight, mode='fan_out', nonlinearity='relu')
59              elif isinstance(m, nn.BatchNorm2d):
60                  nn.init.constant_(m.weight, 1)
61                  nn.init.constant_(m.bias, 0)
62
63      def _make_layer(self, block, planes, blocks, stride=1):
64          downsample = None
65          if stride != 1 or self.inplanes != planes * block.expansion:
66              downsample = nn.Sequential(
67                  nn.Conv2d(self.inplanes, planes * block.expansion, kernel_size=1, stride=
    ↪ stride, bias=False),
68                  nn.BatchNorm2d(planes * block.expansion),
69              )
70          layers = []
71          layers.append(block(self.inplanes, planes, stride, downsample))
72          self.inplanes = planes * block.expansion
73          for _ in range(1, blocks):
74              layers.append(block(self.inplanes, planes))
75          return nn.Sequential(*layers)
76
77      def forward(self, x):
78          x = self.conv1(x)
79          x = self.bn1(x)
80          x = self.relu(x)
81          # x = self.maxpool(x) # 移除
82          x = self.layer1(x)
83          x = self.layer2(x)
84          x = self.layer3(x)
85          x = self.layer4(x)
86          x = self.avgpool(x)
87          x = torch.flatten(x, 1)
88          x = self.dropout(x) # 应用 Dropout
89          x = self.fc(x)
90          return x
91
92  def train_resnet18_on_tiny_imagenet():
93      # 定义保存路径
94      data_dir = './data'
95      model_dir = './model'
96      log_dir = './logs/resnet18_tiny_imagenet' # TensorBoard 日志目录
97      os.makedirs(model_dir, exist_ok=True)
98
99      # 初始化 TensorBoard Writer
100     writer = SummaryWriter(log_dir)
101
102     # 加载数据集 (指定 cache_dir)
103     dataset = load_dataset('zh-plus/tiny-imagenet', cache_dir=data_dir) # 加载完整数据集以获
    ↪ 取 train 和 valid
104
105     # 数据预处理 (增加数据增强)
```

```
    # 训练集：增加随机裁剪、水平翻转、旋转和颜色抖动
    train_transform = Compose([
        RandomCrop(64, padding=4),
        RandomHorizontalFlip(),
        RandomRotation(15), # 增加随机旋转
        ColorJitter(brightness=0.2, contrast=0.2, saturation=0.2, hue=0.1), # 增加颜色抖动
        ToTensor(),
        Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
    ])

    # 验证集：仅保持标准化
    val_transform = Compose([
        ToTensor(),
        Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
    ])

    # 自定义数据集类
    class TinyImageNetDataset(torch.utils.data.Dataset):
        def __init__(self, dataset, transform=None):
            self.dataset = dataset
            self.transform = transform

        def __len__(self):
            return len(self.dataset)

        def __getitem__(self, idx):
            item = self.dataset[idx]
            image = item['image']
            label = item['label']
            image = image.convert('RGB')  # 确保图像为RGB格式
            if self.transform:
                image = self.transform(image)
            return image, label

    # 使用不同的 transform
    train_dataset = TinyImageNetDataset(dataset['train'], transform=train_transform)
    val_dataset = TinyImageNetDataset(dataset['valid'], transform=val_transform) # 假设
    ↪ valid split 存在

    train_loader = DataLoader(train_dataset, batch_size=128, shuffle=True) # 将 batch_size
    ↪  适当调大一点，例如 64 或 128
    val_loader = DataLoader(val_dataset, batch_size=128, shuffle=False)

    # 定义模型 (使用自定义 ResNet18: BasicBlock, [2, 2, 2, 2])
    model = ResNet(BasicBlock, [2, 2, 2, 2], num_classes=200)
    device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
    model.to(device)

    # 损失函数和优化器 (更换为 SGD + Momentum + Weight Decay 以抗过拟合)
    # 启用标签平滑 (Label Smoothing) 以防止过拟合
    criterion = nn.CrossEntropyLoss(label_smoothing=0.1)
    # 使用 SGD 替代 Adam，初始学习率设为 0.1，weight_decay=5e-4 用于正则化
    optimizer = optim.SGD(model.parameters(), lr=0.1, momentum=0.9, weight_decay=5e-4)
```

```
    # 添加学习率调度器，在第 30, 60, 90 epoch 衰减学习率
    scheduler = optim.lr_scheduler.MultiStepLR(optimizer, milestones=[30, 60, 90], gamma
    ↪ =0.1)

    # 训练循环
    num_epochs = 150
    best_acc = 0.0

    for epoch in range(num_epochs):
        model.train()
        running_loss = 0.0
        correct_train = 0
        total_train = 0

        # 移除 tqdm，使用简单的迭代
        for i, (images, labels) in enumerate(train_loader):
            images, labels = images.to(device), labels.to(device)
            optimizer.zero_grad()
            outputs = model(images)
            loss = criterion(outputs, labels)
            loss.backward()
            optimizer.step()

            running_loss += loss.item()
            _, predicted = torch.max(outputs.data, 1)
            total_train += labels.size(0)
            correct_train += (predicted == labels).sum().item()

            # 记录 iteration 级别的 loss (可选)
            if i % 100 == 0:
                writer.add_scalar('Training/Iter_Loss', loss.item(), epoch * len(
    ↪ train_loader) + i)

        train_loss = running_loss / len(train_loader)
        train_acc = 100. * correct_train / total_train

        # 记录 epoch 级别的训练指标
        writer.add_scalar('Training/Epoch_Loss', train_loss, epoch + 1)
        writer.add_scalar('Training/Epoch_Accuracy', train_acc, epoch + 1)

        # 更新学习率并记录
        current_lr = optimizer.param_groups[0]['lr']
        writer.add_scalar('Training/Learning_Rate', current_lr, epoch + 1)
        scheduler.step()

        # 验证阶段
        model.eval()
        correct_val = 0
        total_val = 0

        with torch.no_grad():
            for images, labels in val_loader:
                images, labels = images.to(device), labels.to(device)
```

```
                outputs = model(images)
                _, predicted = torch.max(outputs.data, 1)
                total_val += labels.size(0)
                correct_val += (predicted == labels).sum().item()

        val_acc = 100. * correct_val / total_val

        # 记录验证指标
        writer.add_scalar('Validation/Accuracy', val_acc, epoch + 1)

        print(f'Epoch {epoch+1}: LR: {current_lr:.6f}, Train Loss: {train_loss:.4f}, Train
    ↪ Acc: {train_acc:.2f}%, Val Acc: {val_acc:.2f}%')

        # 保存 Checkpoint (每个 epoch 保存一次)
        checkpoint_path = os.path.join(model_dir, f'checkpoint_epoch.pth')
        torch.save({
            'epoch': epoch + 1,
            'model_state_dict': model.state_dict(),
            'optimizer_state_dict': optimizer.state_dict(),
            'loss': train_loss,
            'val_acc': val_acc
        }, checkpoint_path)
        print(f"Checkpoint saved to {checkpoint_path}")

        # 保存为ONNX (每当验证集准确率提升时保存，或者最后保存)
        if val_acc > best_acc:
             best_acc = val_acc
             # 保存最佳模型逻辑...

    # 关闭 writer
    writer.close()

    # 保存为ONNX (输入尺寸调整为 64x64)
    dummy_input = torch.randn(1, 3, 64, 64).to(device)
    onnx_path = os.path.join(model_dir, 'resnet18_tiny_imagenet.onnx')
    torch.onnx.export(model, dummy_input, onnx_path, verbose=True)
    print(f"Model saved as ONNX: {onnx_path}")

if __name__ == '__main__':
    train_resnet18_on_tiny_imagenet()
```

为了实时监控模型训练过程中的损失、准确率和学习率变化，代码中集成了 TensorBoard。通过 `SummaryWriter` 将训练和验证指标写入日志文件，用户可在训练过程中通过如下命令启动 TensorBoard：

```
tensorboard --logdir=./logs/resnet18_tiny_imagenet
```

在浏览器中访问对应端口，即可可视化每个 epoch 的损失、准确率曲线和学习率变化，便于分析模型收敛情况和调参效果。模型训练结束后，我们可以在 model 文件夹下面，找到名为 resnet18_tiny_imagenet.onnx 的模型文件。

环境配置与模型转换

我们已经训练好了一个针对 Tiny-ImageNet 的 ResNet-18 网络，并上传到了 HuggingFace 仓库 `zhouxzh/resnet18_tiny_imagenet`，因此不需要自行训练。

完整的可运行代码位于 `samples/chapter4/resnet18/` 目录下，可直接参考使用。

为了可以顺利的从 huggingface 下载已经训练好的模型，我们需要使用 pip 安装 Hugging Face 的 CLI 工具。

```
pip install datasets huggingface_hub
```

如果在下载过程中遇到网络连接问题，建议配置镜像源，例如使用 hf-mirror。可以通过以下命令设置环境变量来启用镜像：

```
export HF_ENDPOINT=https://hf-mirror.com
```

为了方便起见，我们可以运行以下命令将该环境变量添加到 `.bashrc` 文件中，使其永久生效：

```
echo 'export HF_ENDPOINT=https://hf-mirror.com' >> ~/.bashrc
source ~/.bashrc
```

接下来，我们在昇腾 310B 开发板上构建项目目录。请执行以下命令创建 `resnet18` 文件夹及必要的子目录结构：

```
mkdir -p resnet18
cd resnet18
mkdir -p {data,model}
```

随后，使用 Hugging Face CLI 下载预训练好的 ONNX 模型：

```
hf download zhouxzh/resnet18_tiny_imagenet --local-dir model/
```

下载完成后，我们需要使用 ATC（Ascend Tensor Compiler）工具将 ONNX 模型转换为昇腾 NPU 专用的 OM（Offline Model）模型。执行如下转换命令：

```
atc --model=model/resnet18_tiny_imagenet.onnx \
    --framework=5 --output=model/resnet18_tiny_imagenet \
    --input_shape="input.1:1,3,64,64" \
    --soc_version=Ascend310B4
```

核心实现：模型推理脚本

为了在昇腾 310B 上加载 OM 模型并执行推理，我们需要编写一个完整的 Python 脚本。该脚本不仅负责调用底层 PyACL 接口，还需要处理数据的预处理（Preprocessing）与后处理（Postprocessing）。本节将通过解析 `inference_npu.py` 的关键代码片段，深入讲解 PyACL 应用的构建逻辑。

完整的推理代码逻辑被封装在 `AclResource` 类中，它管理着从初始化到资源释放的整个生命周期。

1. 初始化与模型加载

在类的 `init` 方法中，我们依次完成系统初始化、Device 指定、Context 创建以及模型加载。值得注意的是，模型加载后，我们需要获取“模型描述”（Model Description），它包含了模型输入输出张量的维度、大小和类型信息，这对于后续申请内存至关重要。

```
def init(self):
    # 1. ACL 全局初始化
    ret = acl.init()

    # 2. 指定运算设备 (Device 0)
    ret = acl.rt.set_device(self.device_id)

    # 3. 创建 Context 上下文
    self.context, ret = acl.rt.create_context(self.device_id)

    # 4. 加载离线模型 (.om)
    self.model_id, ret = acl.mdl.load_from_file(self.model_path)

    # 5. 获取模型描述信息 (用于查询输入输出大小)
    self.model_desc = acl.mdl.create_desc()
    ret = acl.mdl.get_desc(self.model_desc, self.model_id)
```

2. 数据传输与内存管理

推理的核心在于 `execute` 方法。由于 NPU 无法直接读取 CPU（Host）侧的 Numpy 数组，我们需要显式地进行内存拷贝。

- **输入准备**：首先通过 `acl.mdl.get_input_size_by_index` 查询模型所需的输入字节数。接着使用 `acl.rt.malloc` 在 Device 侧申请内存，并利用 `acl.rt.memcpy` 将 host 侧预处理好的 Numpy 数据拷贝到 Device 侧。
- **Dataset 组装**：PyACL 使用 `Dataset` 和 `DataBuffer` 结构来传递数据。我们需要创建一个 `Dataset` 容器，并将包含 Device 内存地址的 `DataBuffer` 添加进去。
- **输出准备**：同样地，根据模型输出大小申请 Device 侧内存，并组装 Output Dataset，用于存放 NPU 计算后的结果。

为了更加清晰地理解 Host 与 Device 之间的数据交互流程，我们可以将上述复杂的 `execute` 函数逻辑简化为以下伪代码：

```
def execute(input_data_host):
    # 1. 准备输入 (Host -> Device)
    # 申请 NPU 侧内存，并将预处理好的图片数据从 CPU 搬运过去
    input_size = get_model_input_size()
    dev_in = acl.malloc_device(input_size)
    acl.memcpy(dev_in, input_data_host, direction=HOST_TO_DEVICE)

    # 2. 准备输出 (Device)
    # 在 NPU 侧申请内存，用于存放模型推理产生的结果
    output_size = get_model_output_size()
    dev_out = acl.malloc_device(output_size)

    # 3. 封装 Dataset
    # PyACL 要求将内存指针封装为 Dataset 结构才能传入模型
    input_dataset = create_dataset(dev_in)
    output_dataset = create_dataset(dev_out)

    # 4. 执行推理 (Inference)
    # 指挥 NPU 执行计算，结果会写入 dev_out
    acl.mdl.execute(model_id, input_dataset, output_dataset)

    # 5. 获取结果 (Device -> Host)
    # 申请 CPU 侧内存，将推理结果从 NPU 搬回，以便 Python 处理
    host_out = acl.malloc_host(output_size)
    acl.memcpy(host_out, dev_out, direction=DEVICE_TO_HOST)

    # 6. 资源清理
    # 释放本次推理申请的 Device 内存
    acl.free(dev_in)
    acl.free(dev_out)

    return host_out
```

这段伪代码直观地展示了 PyACL 推理的核心特征：**内存不仅需要申请，还需要在 Host 与 Device 之间显式搬运（Memcpy）**。这是与在通用 CPU 上开发程序最大的不同点。

3. 数据预处理

模型的输入通常需要特定的格式和分布。这里的 `preprocess` 函数模拟了标准 torchvision 的转换逻辑，但完全使用 Numpy 实现，以减少第三方库依赖。关键步骤包括：

- **归一化**：将像素值除以 255 转为浮点，并减去均值除以标准差。
- **维度变换**：将图片从 HWC（高宽通道）调整为 PyTorch 默认的 CHW（通道高宽）格式。
- **增加 Batch 维度**：模型输入通常是 4 维张量 (N, C, H, W)，因此需要增加 Batch 维度。
- **内存连续性**：这一步通过 `np.ascontiguousarray` 确保数据在内存中是连续存储的，这是 C 语言接口正确读取数据的必要条件。

4. 资源释放

脚本最后，我们需要显式释放所有持久化资源，防止 NPU 内存泄漏或状态异常。

```
def release(self):
    acl.mdl.destroy_desc(self.model_desc)
    acl.mdl.unload(self.model_id)     # 卸载模型
    acl.rt.destroy_context(self.context) # 销毁 Context
    acl.rt.reset_device(self.device_id)  # 重置 Device
    acl.finalize()                    # 去初始化
```

通过将这些模块组合，我们便得到了一个名为 `inference_npu.py` 的完整推理脚本。

```
import time
import numpy as np
import acl
from datasets import load_dataset
import tqdm

import ctypes

data_dir = './data' # 假设 data_dir 已定义，或者根据实际情况保留原变量

# 加载数据集 (指定 cache_dir)
dataset = load_dataset('zh-plus/tiny-imagenet', split='valid', cache_dir=data_dir) # 只加载
    ↪ valid 集

# ACL 初始化与资源管理类
class AclResource:
    def __init__(self, device_id=0, model_path="model/resnet18_tiny_imagenet.om"):
        self.device_id = device_id
        self.model_path = model_path
        self.model_id = None
        self.context = None
        self.stream = None
        self.input_dataset = None
        self.output_dataset = None
        self.model_desc = None

    def init(self):
        # ACL 初始化
        ret = acl.init()
        if ret != 0: raise RuntimeError("acl init failed")
        ret = acl.rt.set_device(self.device_id)
        if ret != 0: raise RuntimeError("set device failed")
        self.context, ret = acl.rt.create_context(self.device_id)
        if ret != 0: raise RuntimeError("create context failed")

        # 加载模型
        self.model_id, ret = acl.mdl.load_from_file(self.model_path)
        if ret != 0: raise RuntimeError(f"load model failed, path: {self.model_path}")

        # 获取模型描述
```

```
        self.model_desc = acl.mdl.create_desc()
        ret = acl.mdl.get_desc(self.model_desc, self.model_id)

        print(f"ACL Resource Init Success. Device: {self.device_id}")

    def execute(self, input_numpy):
        # 准备输入数据 (Host -> Device)
        # 获取模型输入大小
        input_size = acl.mdl.get_input_size_by_index(self.model_desc, 0)

        # 申请 Device 输入内存
        dev_in_ptr, ret = acl.rt.malloc(input_size, 2) # 2: ACL_MEM_MALLOC_HUGE_FIRST

        # 拷贝数据 (Host -> Device)
        # 确保输入是 contiguous 且 float32 (C Type bytes)
        # 使用 tobytes + bytes_to_ptr 避免 numpy_to_ptr 可能引发的 ImportError 兼容性问题
        if input_numpy.nbytes != input_size:
            print(f"Warning: Input size mismatch. Model expects {input_size}, got {
    ↪ input_numpy.nbytes}")

        input_bytes = input_numpy.tobytes()
        input_ptr = acl.util.bytes_to_ptr(input_bytes)
        # acl.rt.memcpy (dst, dest_size, src, src_size, kind)
        acl.rt.memcpy(dev_in_ptr, input_size, input_ptr, input_size, 1) # 1:
    ↪ ACL_MEMCPY_HOST_TO_DEVICE

        # 组装 Input Dataset
        self.input_dataset = acl.mdl.create_dataset()
        input_data_buffer = acl.create_data_buffer(dev_in_ptr, input_size)
        acl.mdl.add_dataset_buffer(self.input_dataset, input_data_buffer)

        # 准备输出数据
        self.output_dataset = acl.mdl.create_dataset()
        output_size = acl.mdl.get_output_size_by_index(self.model_desc, 0)
        dev_out_ptr, ret = acl.rt.malloc(output_size, 2)
        output_data_buffer = acl.create_data_buffer(dev_out_ptr, output_size)
        acl.mdl.add_dataset_buffer(self.output_dataset, output_data_buffer)

        # 执行推理
        ret = acl.mdl.execute(self.model_id, self.input_dataset, self.output_dataset)
        if ret != 0: print("Model execute failed")

        # 取回结果 (Device -> Host)
        host_out_buffer, ret = acl.rt.malloc_host(output_size)
        acl.rt.memcpy(host_out_buffer, output_size, dev_out_ptr, output_size, 2) # 2:
    ↪ ACL_MEMCPY_DEVICE_TO_HOST

        # 转换为 numpy (假设输出是 float32, batch=1, class=200)
        out_array = np.frombuffer(ctypes.string_at(host_out_buffer, output_size), dtype=np
    ↪ .float32)

        # 清理单次推理资源
        acl.rt.free(dev_in_ptr)
```

```
        acl.rt.free(dev_out_ptr)
        acl.rt.free_host(host_out_buffer)
        acl.destroy_data_buffer(input_data_buffer)
        acl.destroy_data_buffer(output_data_buffer)
        acl.mdl.destroy_dataset(self.input_dataset)
        acl.mdl.destroy_dataset(self.output_dataset)

        return out_array

    def release(self):
        acl.mdl.destroy_desc(self.model_desc)
        acl.mdl.unload(self.model_id)
        acl.rt.destroy_context(self.context)
        acl.rt.reset_device(self.device_id)
        acl.finalize()

# 实例化并初始化 ACL 资源
# 请确保当前路径下有对应的 .om 模型文件
om_model_path = "model/resnet18_tiny_imagenet.om"
acl_resource = AclResource(model_path=om_model_path)
acl_resource.init()

# 自定义数据预处理函数 (替代 torchvision)
def preprocess(image):
    # 将 PIL Image 转换为 numpy 数组
    img_data = np.array(image).astype('float32') / 255.0

    # 获取图像尺寸，如果不是 RGB 三通道 (例如灰度图)，需要转换
    if len(img_data.shape) == 2:
        img_data = np.stack([img_data]*3, axis=-1)

    # 归一化参数
    mean = np.array([0.485, 0.456, 0.406], dtype='float32')
    std = np.array([0.229, 0.224, 0.225], dtype='float32')

    # 归一化: (image - mean) / std
    img_data = (img_data - mean) / std

    # 调整维度: HWC -> CHW (3, 64, 64)
    img_data = img_data.transpose(2, 0, 1)

    # 增加 Batch 维度: (1, 3, 64, 64)
    img_data = np.expand_dims(img_data, axis=0)

    # 确保内存连续，这对 C 侧指针拷贝很重要
    if not img_data.flags['C_CONTIGUOUS']:
        img_data = np.ascontiguousarray(img_data)

    return img_data

# 推理计数和计时
total_images = 0
correct_count = 0
```

```
start_time = time.time()

print("开始推理...")

# 逐张图片推理
for item in tqdm.tqdm(dataset):
    image = item['image'] # 获取 PIL 图像
    label = item['label'] # 获取真实标签

    # 预处理
    input_tensor = preprocess(image)

    # 推理 (使用 pyACL)
    # output 是扁平的一维数组，直接使用
    outputs = acl_resource.execute(input_tensor)

    # 获取预测结果: argmax 获取概率最大的类别索引
    predicted_label = np.argmax(outputs)

    if predicted_label == label:
        correct_count += 1

    total_images += 1
    # if total_images >= 100: break # 可选: 用于快速测试

end_time = time.time()
duration = end_time - start_time
fps = total_images / duration
accuracy = correct_count / total_images * 100

# 释放 ACL 资源
acl_resource.release()

print(f"推理完成。")
print(f"总图片数: {total_images}")
print(f"总耗时: {duration:.4f} 秒")
print(f"推理帧率: {fps:.2f} FPS")
print(f"正确率: {accuracy:.2f}%")
```

PyACL 模型推理结果

在配备 8T 算力的昇腾 310B 设备上，使用 PyACL 框架进行推理的运行日志如下所示:

```
ACL Resource Init Success. Device: 0
开始推理...
100%|██████████████████████████████████████████████████████████████████████████████|
    ↪ 10000/10000 [00:37<00:00, 265.04it/s]
推理完成。
总图片数: 10000
```

```
总耗时: 37.7336 秒
推理帧率: 265.02 FPS
正确率: 62.45%
```

从上述推理结果可以看出，在昇腾 310B（8T 算力版本）开发板上，利用 PyACL 调用 NPU 进行推理，处理 64x64 分辨率图像的帧率高达 265 FPS。这一令人印象深刻的速度不仅验证了 NPU 的加速能力，也表明其完全有能力胜任大多数实时计算场景，甚至在面对高帧率目标跟踪等对时延要求极高的任务时也能游刃有余。

推理结果对比分析

为了直观评估昇腾 310B NPU 的加速效果，我们将基于统一的 ResNet-18 ONNX 模型，在多种硬件平台上开展推理性能的横向对比测试。不仅包括高性能的 NVIDIA RTX 5090D 显卡和主流的 Intel Core Ultra 7 155H 笔记本处理器，还纳入了同属嵌入式领域的树莓派 5B, 以及昇腾 310B 自身的 CPU 模式。我们将详细记录各平台的推理耗时与帧率 (FPS)，以此作为基准来量化 NPU 带来的性能提升。以下是各平台通用的 OnnxRuntime 推理测试代码：

```python
import time
import numpy as np
import onnxruntime

from datasets import load_dataset
import tqdm

data_dir = './data' # 假设 data_dir 已定义，或者根据实际情况保留原变量

# 加载数据集 (指定 cache_dir)
dataset = load_dataset('zh-plus/tiny-imagenet', split='valid', cache_dir=data_dir) # 只加载 valid 集

# ONNX Runtime 初始化
onnx_model_path = "model/resnet18_tiny_imagenet.onnx" # 请确保当前路径下有该模型文件
session = onnxruntime.InferenceSession(onnx_model_path, providers=['CUDAExecutionProvider', 'CPUExecutionProvider'])
print(f"当前运行设备 (Providers): {session.get_providers()}") # 打印以确认 CUDA 是否生效

input_name = session.get_inputs()[0].name

# 自定义数据预处理函数 (替代 torchvision)
def preprocess(image):
    # 将 PIL Image 转换为 numpy 数组
    img_data = np.array(image).astype('float32') / 255.0

    # 获取图像尺寸，如果不是 RGB 三通道 (例如灰度图)，需要转换
    if len(img_data.shape) == 2:
        img_data = np.stack([img_data]*3, axis=-1)

```

```
    # 归一化参数
    mean = np.array([0.485, 0.456, 0.406], dtype='float32')
    std = np.array([0.229, 0.224, 0.225], dtype='float32')

    # 归一化: (image - mean) / std
    img_data = (img_data - mean) / std

    # 调整维度: HWC -> CHW (3, 64, 64)
    img_data = img_data.transpose(2, 0, 1)

    # 增加 Batch 维度: (1, 3, 64, 64)
    img_data = np.expand_dims(img_data, axis=0)

    return img_data

# 推理计数和计时
total_images = 0
correct_count = 0
start_time = time.time()

print("开始推理...")

# 逐张图片推理
for item in tqdm.tqdm(dataset):
    image = item['image'] # 获取 PIL 图像
    label = item['label'] # 获取真实标签

    # 预处理
    input_tensor = preprocess(image)

    # 推理
    outputs = session.run(None, {input_name: input_tensor})

    # 获取预测结果: argmax 获取概率最大的类别索引
    predicted_label = np.argmax(outputs[0])

    if predicted_label == label:
        correct_count += 1

    total_images += 1
    # if total_images >= 100: break # 可选: 用于快速测试

end_time = time.time()
duration = end_time - start_time
fps = total_images / duration
accuracy = correct_count / total_images * 100

print(f"推理完成。")
print(f"总图片数: {total_images}")
print(f"总耗时: {duration:.4f} 秒")
print(f"推理帧率: {fps:.2f} FPS")
print(f"正确率: {accuracy:.2f}%")
```

所有平台均安装了 OnnxRuntime 进行推理。其中 RTX 5090D 配置了 CUDA 加速，而其他平台（包括树莓派、Intel 笔记本和昇腾 310B 的 CPU 模式）均仅使用 CPU 进行计算。

以下是各平台的具体测试结果：

1. Ascend 310B (CPU 模式) + OnnxRuntime: *(规格：4 核 Cortex-A55 @ 1.0GHz)*

```
当前运行设备 (Providers): ['CPUExecutionProvider']
开始推理...
100%|████████████████████████████████████| 10000/10000 [33:11<00:00,  5.02it/s]
推理完成。
总图片数: 10000
总耗时: 1991.7797 秒
推理帧率: 5.02 FPS
正确率: 62.43%
```

2. Nvidia GeForce 5090D (CUDA) + OnnxRuntime: *(规格：32GB GDDR7, 21760 CUDA Cores)*

```
当前运行设备 (Providers): ['CUDAExecutionProvider', 'CPUExecutionProvider']
开始推理...
100%|████████████████████████████████████| 10000/10000 [00:09<00:00, 1042.40it/s]
推理完成。
总图片数: 10000
总耗时: 9.5939 秒
推理帧率: 1042.33 FPS
正确率: 62.43%
```

3. Raspberry Pi 5B (CPU) + OnnxRuntime: *(规格：Broadcom BCM2712, 4 核 Cortex-A76 @ 2.4GHz)*

```
当前运行设备 (Providers): ['CPUExecutionProvider']
开始推理...
100%|████████████████████████████████████| 10000/10000 [07:26<00:00, 22.38it/s]
推理完成。
总图片数: 10000
总耗时: 446.7709 秒
推理帧率: 22.38 FPS
正确率: 62.43%
```

4. Intel Core Ultra 7 155H (CPU) + OnnxRuntime: *(规格：3800 Mhz, 16 核, 22 逻辑处理器)*

```
当前运行设备 (Providers): ['CPUExecutionProvider']
开始推理...
100%|████████████████████████████████████| 10000/10000 [02:41<00:00, 61.91it/s]
推理完成。
总图片数: 10000
总耗时: 161.5301 秒
```

```
推理帧率：61.91 FPS
正确率：62.43%
```

多平台 ResNet-18（Tiny-ImageNet）推理性能对比数据：

硬件平台	推理框架	运行设备	帧率 (FPS)	相对 NPU 性能 (265 FPS)
Ascend 310B NPU	**PyACL**	**NPU (AI Core)**	**265.02**	**100% (基准)**
Nvidia RTX 5090D	OnnxRuntime	GPU (CUDA)	1042.33	~393%
Intel Core Ultra 7 155H	OnnxRuntime	CPU	61.91	~23%
Raspberry Pi 5B	OnnxRuntime	CPU	22.38	~8.4%
Ascend 310B CPU	OnnxRuntime	CPU (Arm Cortex-A55)	5.02	~1.9%

通过数据对比，我们可以清晰地看到昇腾 310B NPU 的强大优势：

- **对比自身 CPU**：NPU 的推理速度是其 CPU 模式（5.02 FPS）的 **52 倍**，充分证明了专用 AI 加速器的必要性。
- **对比树莓派 5B**：虽然同样是 Arm 架构的嵌入式设备，昇腾 310B NPU 的性能是树莓派 5B CPU（22.38 FPS）的 **11 倍**以上。
- **对比高性能笔记本 CPU**：即便是最新的 Intel Core Ultra 7 处理器（61.91 FPS），在没有独立显卡加速的情况下，其纯 CPU 推理性能也不及昇腾 310B NPU 的 **1/4**。
- **对比旗舰显卡**：虽然 RTX 5090D 展现出了恐怖的 1000+ FPS 性能，但考虑到昇腾 310B 这类边缘设备的低功耗和体积优势，265 FPS 的成绩对于实时的嵌入式应用而言已经绰绰有余。

这一测试结果有力地展示了昇腾 310B 在边缘计算场景下的核心竞争力——**以极低的功耗提供数倍于通用 CPU 的 AI 算力**。

4.3. 目标检测推理

在图像分类之外，目标检测（Object Detection）是 PyACL 推理的另一个重要应用场景。与分类任务仅输出类别标签不同，目标检测需要同时预测物体的**边界框（Bounding Box）**和**类别**，对模型结构和后处理逻辑都提出了更高的要求。

本章在 `samples/chapter4/` 下提供了两种主流目标检测模型的 PyACL 推理实现：

4.3.1. SSD300（ResNet-50 主干）

SSD（Single Shot MultiBox Detector）是一种单阶段目标检测器，以 ResNet-50 为主干网络，输入尺寸 300×300，在 COCO 2017 数据集上训练和评估。

本节 SSD300 模型文件统一采用 `ssd300_{backbone}.onnx` / `ssd300_{backbone}.om` 命名，例如 `ssd300_resnet50.onnx` 和 `ssd300_resnet50.om`。其中 `ssd300_` 表示模型输入尺寸为 300×300，用来和后续输入尺寸为 320×320 的 `ssd320_` SSDLite320 模型区分。预训练 ONNX/OM 模型发布在 Hugging Face 仓库 `zhouxzh/SSD300`，下载脚本默认使用 `https://hf-mirror.com` 镜像，也可以通过 `HF_ENDPOINT` 或 `--endpoint` 指定其他镜像。

文件	说明
`train_cuda.py`	GPU 训练入口
`inference_npu.py`	PyACL 加载.om 模型进行 NPU 推理
`inference_cuda.py`	GPU 推理（对比基准）
`inference_cpu.py`	CPU 推理
`download_models.py`	从 `zhouxzh/SSD300` 下载 `ssd300_*` ONNX/OM 模型

代码位于 `samples/chapter4/SSD/`。

常用命令如下：

```
cd samples/chapter4/SSD

# 默认从 https://hf-mirror.com 下载 models/ssd300_resnet50.onnx
python download_models.py --onnx --backbone resnet50

# 开发板可直接下载仓库中发布的 OM 模型
export HF_ENDPOINT=https://hf-mirror.com
python download_models.py --om --backbone resnet50

# CPU ONNX 推理
python inference_cpu.py --backbone resnet50

# Ascend 310B NPU 快速烟测，默认读取 models/ssd300_resnet50.om
python inference_npu.py --device 0 --limit 20 --skip-map

# 全量 COCO 验证集评估
python inference_npu.py --device 0
```

4.3.2. SSDLite320（MobileNet 主干）

SSDLite 是 SSD 的轻量化变体，将标准卷积替换为深度可分离卷积（Depthwise Separable Convolution），从而显著降低检测头的参数量和计算量。为了观察不同轻量化骨干网络在昇腾 310B 上的实际表现，本节不再只使用单一 MobileNetV3，而是比较 MobileNetV1、MobileNetV2、MobileNetV3 和 MobileNetV4 的 15 个 SSDLite320 变体。所

有模型输入尺寸均为 320×320，输出为 `boxes:1x4x3234` 和 `scores:1x81x3234`，Default Boxes 使用 `min_ratio=0.1`、`max_ratio=0.9`。

本目录只提供部署、转换和评估代码，不包含训练代码。ONNX 模型的训练、导出和 CUDA 侧评估代码位于 zhouxzh/SSDLite320-MobileNet，如果需要了解训练策略、主干网络配置和 ONNX 导出细节，应到该 GitHub 仓库查找。Ascend 310B 侧的样例代码位于 `samples/chapter4/SSDLite/`。

文件	说明
`scripts/inference_npu.py`	PyACL NPU 推理
`scripts/inference_cpu.py`	ONNX Runtime CPUExecutionProvider 校验 ONNX 接口和后处理
`scripts/download_models.py`	下载公开 ONNX/OM 模型
`scripts/convert_onnx_to_om.py`	ONNX 转 OM（需在 Ascend 设备上运行）
`ssdlite320/`	Default Boxes、Decode、NMS、可视化与评估工具

昇腾 310B 评估口径

Ascend 310B 侧的 CPU/NPU 评估报告由本地脚本生成，不作为本仓库的公开文件发布。读者如果需要复现实测数据，可以在准备好 ONNX/OM 模型和验证集后运行：

```
python scripts/inference_cpu.py --all
source /usr/local/Ascend/ascend-toolkit/set_env.sh
python scripts/inference_npu.py --all --device 0
```

评估脚本默认使用 `img_size=320`、`dbox_min_ratio=0.1`、`dbox_max_ratio=0.9`。报告中的 `FPS infer` 表示评估循环内单样本预处理、模型执行、输出整理和 decode/NMS 的吞吐；`FPS total` 还包含数据读取、JSON 写入和 COCO 评估等完整流程开销，因此更接近 Python 评估脚本的端到端速度。实际部署时，DVPP 预处理、C++/设备侧后处理、多 Stream 异步流水线等工程优化会进一步影响最终帧率。

从本地评估趋势看，CUDA 侧 ONNX 与 310B NPU OM 的 mAP 基本一致，说明 ONNX 转 OM 后没有出现明显精度损失。310B CPU 侧 ONNX Runtime 也能用于校验精度和后处理逻辑，但速度明显不足，不适合实时目标检测部署。

精度分析

从精度看，MobileNetV4 Hybrid 系列最强。`mobilenetv4_hybrid_large` 的 mAP 为 0.258，AP50 为 0.435，是整体精度最高的模型；`mobilenetv4_hybrid_medium` 的 mAP 同样约为 0.258，AP75 和大目标 APl 甚至略高，且参数量只有 large 的约 28%，因此是高精度场景下更均衡的选择。若要求模型结构更偏卷积、减少 Hybrid 结构中的 MatMul、Transpose、

Softmax 等算子，mobilenetv4_conv_large 可达到 0.252 mAP，略低于 Hybrid large，但推理速度更快一些。

MobileNetV3 Large 150d 和 MobileNetV4 Conv Medium 属于中高精度区间。mobilenetv3_large_150d mAP 为 0.247，精度接近 mobilenetv4_conv_large，但参数量和 MACs 更低；mobilenetv4_conv_medium mAP 为 0.243，是所有 FPS infer 超过 30 FPS 的模型里精度最高的一个。对于边缘端目标检测，这个模型通常比单纯追求最小参数量更有实际价值。

MobileNetV1 和 MobileNetV2 的中等宽度版本处于中等精度区间。mobilenetv1_125、mobilenetv2_140、mobilenetv1_100 的 mAP 分别为 0.233、0.225、0.221，虽然不如 V4/V3 高精度模型，但速度更接近实时。mobilenetv3_small_100、mobilenetv2_050、mobilenetv3_small_050 虽然速度最高，但 mAP 只有 0.140、0.120、0.093；mobilenetv4_conv_small 的速度也较高，但 mAP 仅 0.054，不适合作为通用 COCO 检测模型。

还需要注意小目标 AP。即使是最强的 mobilenetv4_hybrid_large，APs 也只有 0.076，说明 320×320 输入尺寸和轻量化检测头对小目标并不友好。如果应用场景以远距离行人、小物体、密集目标为主，应优先选择 large/medium 级别模型，并考虑提高输入分辨率或针对业务数据重新训练；如果主要检测中大目标，APl 最高的 mobilenetv4_hybrid_medium 和 mobilenetv4_hybrid_large 会更合适。

速度分析

在昇腾 310B 上，速度并不只由参数量或 MACs 决定。最小的 mobilenetv3_small_050 只有 0.089G MACs，但在 Python/PyACL 评估流程中，它并不会比中等模型快出数量级。这是因为 SSDLite320 的输出规模固定为 3234 个候选框，decode、NMS、数据搬运和 Python 调度开销对所有骨干网络都存在。当模型本身很小时，这些固定开销会成为主要瓶颈，继续缩小 backbone 并不会线性提升端到端帧率。

大模型则更容易受到算子形态和 NPU 编译优化的影响。mobilenetv4_hybrid_large 精度最高，但包含更多非纯卷积算子和较多节点，推理吞吐低于中等规模模型；mobilenetv4_conv_large 虽然 MACs 达到 4.556G，但以卷积结构为主，在 310B 上更容易获得稳定的算子映射。mobilenetv4_conv_medium 是一个典型平衡点：mAP 0.243，速度明显优于许多更大模型，同时精度又明显高于 small 级别模型。

从 CPU/NPU 对比看，越大的模型越能体现 NPU 加速价值；而在小模型上，瓶颈更多转移到 Host 侧后处理、调度和内存路径。这进一步说明，要提升真实应用帧率，除了换更小 backbone，还必须优化预处理、后处理、异步流水线和视频帧调度。

模型选择建议

如果目标是**高精度推理**，优先选择 mobilenetv4_hybrid_large；它的整体 mAP 和 AP50 最高，适合离线分析、低帧率告警或对误检漏检比较敏感的场景。如果希望在精度接近

的同时降低模型规模和时延，`mobilenetv4_hybrid_medium` 更合适，它只比 large 低约 0.001 mAP，但模型规模小得多。

如果目标是**高帧率推理**，优先选择 `mobilenetv4_conv_medium`。它是本地 310B 评估中进入 30 FPS 级推理吞吐范围的最高精度模型，mAP 达到 0.243，比 `mobilenetv1_100` 高 0.022，比 `mobilenetv3_large_100` 高 0.046。若应用只追求最大检测频率且可以接受明显精度损失，可以选择 `mobilenetv3_small_050` 或 `mobilenetv3_small_100`；但从通用目标检测效果看，它们更适合作为速度上限参考，而不是推荐默认模型。

目标视频帧率	当前 320×320 SSDLite320 是否支持逐帧检测	推荐模型与策略	说明
30 FPS	按推理循环吞吐可接近或达到；按完整 Python 评估流程仍需优化	首选 `mobilenetv4_conv_medium`；若必须更轻，可选 `mobilenetv1_100`	`mobilenetv4_conv_medium` 是 30 FPS 档的最佳精度选择；实际工程需优化预处理、后处理和异步流水线
60 FPS	不支持逐帧检测	`mobilenetv4_conv_medium` 每 2 帧检测一次，配合跟踪；极限速度可试 `mobilenetv3_small_050`	当前 SSDLite320 路径无法真实逐帧 60 FPS；用 30 Hz 检测 + 60 Hz 跟踪/显示更现实
90 FPS	不支持逐帧检测	`mobilenetv4_conv_medium` 每 3 帧检测一次；若目标较简单可试 `mobilenetv3_small_100`	90 FPS 视频流可保持约 30 Hz 检测频率，其余帧由跟踪器或上一帧检测结果补齐
120 FPS	不支持逐帧检测	`mobilenetv4_conv_medium` 或 `mobilenetv1_100` 每 4 帧检测一次；若必须逐帧，需要重新设计模型和流水线	当前 SSDLite320+Python/PyACL+NMS 结果无法支撑 120 FPS 逐帧检测，需要降低输入尺寸、使用更小检测器、迁移后处理到 C++/设备侧、引入 DVPP 和多 Stream 异步流水线

因此，在昇腾 310B 上做 SSDLite320 部署时，可以按业务目标分成两类：追求检测质量时选择 `mobilenetv4_hybrid_large` 或 `mobilenetv4_hybrid_medium`；追求实时帧率时选择 `mobilenetv4_conv_medium`，再通过隔帧检测、目标跟踪、DVPP 预处理和异步流水线把视频系统帧率提高到 60 FPS 以上。不要仅根据参数量选择最小模型，因为过小的 backbone 在精度上损失很大，而在 310B 上的端到端速度提升并不成比例。

4.3.3. 推理流程对比

目标检测的 PyACL 推理流程与 ResNet-18 分类基本一致：**模型加载 -> 数据预处理 -> NPU 推理 -> 后处理**。关键区别在于：

- **模型输出**：分类模型输出单一概率向量（`[1, 200]`），目标检测模型输出多个张量（边界框坐标 + 类别置信度）。
- **后处理**：分类只需 `argmax`，目标检测需要 Decode 边界框 + NMS（非极大值抑制）去除重叠检测框。
- **评估指标**：分类用准确率（Accuracy），目标检测用 mAP（mean Average Precision）。

完整代码及使用说明详见各子目录下的 README 文件。

4.4. 总结

本章系统讲解了 PyACL 应用开发的基础知识，从运行资源管理到模型推理的全链路流程。PyACL 的核心价值在于：以 Python 的简洁语法驱动昇腾 NPU 的高性能计算，将 C/C++ 的指针操作和手动内存管理封装为 `acl.rt.malloc`、`acl.rt.memcpy`、`acl.mdl.execute` 等简洁接口，极大降低了昇腾 AI 处理器的开发门槛。

在运行时资源方面，Device、Context、Stream 构成的三级资源模型是 PyACL 的骨架。理解它们的层级关系与生命周期——申请按 `Device -> Context -> Stream` 顺序，释放按逆序——是写出健壮 PyACL 程序的前提。在此基础上，异构内存管理（Host 与 Device 间的四种拷贝路径）与同步等待机制（Event、Stream Sync、Stream Wait Event、Device Sync 四种粒度）构成了性能优化的核心手段。特别是在昇腾 310B 这类 CPU 算力较弱（Cortex-A55）的边缘设备上，**全链路异步 + Event 驱动的多 Stream 协作**是榨干 NPU 性能的关键策略。

在应用实践层面，本章通过 ResNet-18（图像分类）和 SSD/SSDLite（目标检测）两个经典场景，完整演示了”GPU 训练 -> ONNX 导出 -> ATC 转换 -> PyACL NPU 推理” 的标准开发流水线。跨平台对比数据表明，即便使用相同的 ONNX 模型，昇腾 310B NPU 的 ResNet-18 推理速度（265 FPS）远超其自身 CPU 模式（5 FPS）和树莓派 5B（22 FPS），充分验证了专用 AI 加速器在边缘场景下的不可替代性。对于 SSDLite320 目标检测，`mobilenetv4_hybrid_large` 更适合高精度，`mobilenetv4_conv_medium` 则是在 310B 上兼顾 mAP 与 30 FPS 级检测吞吐的更实用选择。

回顾全章，PyACL 的精髓可归纳为三条原则：

1. **资源按序管理**：初始化 -> Device -> Context -> Stream -> 业务执行 -> 逆序释放，缺一不可。
2. **内存显式搬运**：Host 与 Device 内存相互隔离，数据必须通过 `memcpy` 显式传输，这与纯 CPU 编程截然不同。
3. **异步优先于同步**：在边缘端弱 CPU 的约束下，必须用 `stream_wait_event` 将依赖关系卸载到 Device 侧，让 CPU 专注于指令分发而非空等。

掌握了这些基础概念与 API 后，下一章将进一步深入 DVPP（数字视觉预处理）模块，探索如何利用昇腾 310B 的硬件编解码与图像处理能力，构建完整的视频流 AI 应用。

5. DVPP 视频处理基础

Ascend 310B 芯片内置了 **DVPP（Digital Vision Pre-Processing）**硬件加速引擎，提供视频编解码（VENC/VDEC）、图像处理（VPC）和 JPEG 编解码能力。本章从基础概念出发，逐步深入到各子模块的 API 使用、性能调优和实战集成。

5.1. DVPP 基础概念与编程模型

DVPP（Digital Vision Pre-Processing）是 Ascend 芯片内部的一组**硬件加速模块**，专门处理图像和视频数据。它独立于 NPU 的 AI Core（推理引擎），不占用 AI 算力。PyACL 通过 `acl.media` 接口对外暴露这些能力。

> 本节覆盖所有 DVPP 子模块（VENC、VDEC、VPC、JPEGE、JPEGD）共享的基础概念。初次接触 DVPP 时，建议先阅读本节，再进入具体子模块的内容。

5.1.1. DVPP 在芯片中的位置

Ascend 310B4 可从应用开发视角理解为以下几类片上资源：

- **AI Core × 1**: DaVinci V300（矩阵运算，NPU 推理主力）
- **片上 CPU × 4**: 4 个 64 位 ARMv8-A 通用处理核。`npu-smi` 可按 AI CPU / control CPU / data CPU 查询和配置数量，默认配比为 `1:3:0`；这是启动配置，修改后需要复位生效，不是运行时动态划分
- **DVPP**: 独立视觉/媒体处理硬件单元（独立于 AI Core，也不是 CPU/AICPU 上的软件模块）
  - VENC: 视频编码
  - VDEC: 视频解码
  - VPC: 图像处理（resize/crop/csc）
  - JPEGE: JPEG 编码
  - JPEGD: JPEG 解码
- **设备内存与内部总线**: AI Core、DVPP 与片上 CPU 通过设备内存和内部总线交换数据

关键要点：DVPP 与 AI Core 是芯片上两个独立的硬件域。DVPP 编码 1080p 视频与 AI Core 执行 YOLO 推理可以并行运行，二者不竞争同一计算单元；CPU 侧主要负责内存、描述符和任务下发，不执行像素级编解码计算。

5.1.2. DVPP 数据流

DVPP 接收来自 **DDR 内存**的数据，处理后写回 DDR。CPU 侧的工作是准备输入描述符、下发任务、等待回调或 Stream 同步完成——不参与实际像素级处理。

数据流五步：(1) CPU 侧分配设备内存并拷贝数据到设备 -> (2) DVPP 硬件通过内部总线读取 -> (3) 处理后将结果写入输出缓冲区 -> (4) DVPP 通过回调或 Stream 完成事件通知 CPU 侧 -> (5) CPU 侧按需从设备内存拷回结果。

核心机制：DVPP 直接访问 DDR，不经过 CPU 执行像素级处理。这是它能比 CPU 软件处理快得多的根本原因。

5.1.3. 子模块概览

模块	全称	功能	输入格式	输出格式	本教程章节
VENC	Video Encoder	视频编码	NV12 原始帧	H.264 / H.265 码流	VENC
VDEC	Video Decoder	视频解码	H.264 / H.265 码流	NV12 原始帧	VDEC
VPC	Video Pre-Processing Core	图像处理	NV12 / RGB / BGR	NV12 / RGB / BGR	VPC
JPEGE	JPEG Encoder	JPEG 编码	YUV420SP / YUV422SP	JPEG 码流	JPEG
JPEGD	JPEG Decoder	JPEG 解码	JPEG 码流	YUV420SP / YUV444	JPEG

> **PNGD**（PNG 解码器）虽然列在 DVPP 规格中，但在 310B (CANN 8.3.RC1) 上实测输出缓冲区全零。310B 上 PNG 解码请使用 CPU 方案（Pillow / OpenCV）。本教程不含 PNGD 章节。

5.1.4. AIPP vs DVPP（何时用哪个）

PyACL 的媒体处理分两类能力：

- **DVPP**：适合执行 “低级别” 的高吞吐预处理——JPEG/视频解码、YUV 与 RGB 转换、缩放、裁剪等。优点是速度快、CPU 负载低，但受格式与对齐约束。

- **AIPP**（Artificial Intelligence Pre-Processing）：适合执行 “模型输入级” 的精确预处理——统一色域/像素变换、量化/去均值、通道顺序等。AIPP 分静态（在模型转换时固化到.om）与动态（运行时通过接口设置）两种模式。

常见组合：先用 DVPP 完成解码与粗略 resize/crop，再用 AIPP（静态或动态）做最后的色域和像素级处理，保证与模型输入要求一致。

5.1.5. H.264 与 H.265 编解码基础

VENC 和 VDEC 都围绕 H.264/H.265 工作。理解这些编码标准的基本概念，是使用 DVPP 编解码模块的前提。

什么是 H.264

H.264（也叫 AVC）是目前全球使用最广泛的视频编码标准。核心思想是去除视频中的冗余：空间冗余通过帧内预测消除，时间冗余通过帧间预测消除，统计冗余通过熵编码消除。

编码后的每一帧按类型分为：

帧类型	全称	大小	依赖	说明
I 帧 (IDR)	Instantaneous Decoder Refresh	最大（~80KB@480p）	无	独立解码，不依赖任何其他帧
P 帧	Predictive	中等（~5-15KB@480p）	前一帧	只存与前一帧的差异
B 帧	Bi-predictive	最小（~2-5KB@480p）	前后帧	双向预测，压缩率最高但延迟最大

> **B 帧与实时通信**：B 帧需要参考 “未来” 帧，引入额外延迟。WebRTC 和实时视频通话通常**禁用 B 帧**（`tune=zerolatency`），只用 I 帧和 P 帧。

GOP（Group of Pictures）：一个 I 帧到下一个 I 帧之间的帧组。GOP=30 表示每 30 帧插入一个 I 帧（30fps 下每秒一个关键帧）。IDR 帧必须从 IDR 帧开始才能正确解码。

NAL 单元与 Annex-B 格式：H.264 码流由 NAL 单元组成（SPS、PPS、IDR Slice、Non-IDR Slice、SEI）。Annex-B 格式用起始码 `0x00000001` 分隔每个 NAL 单元。VDEC 要求输入必须是 Annex-B 格式。

什么是 H.265

H.265（也叫 HEVC）是 H.264 的下一代标准，核心目标：**相同画质下码率减半**。

特性	H.264	H.265
编码块大小	16×16 宏块（固定）	8×8 到 64×64 CTU（自适应）
帧内预测方向	9 种	35 种

特性	H.264	H.265
压缩率	基准	**同画质下码率少 50%**
编码复杂度	基准	**约 2-5 倍**
浏览器支持	100%	>95%（Firefox 不支持 WebRTC H.265）

为什么本章仍以 H.264 作为主线：H.264 在 WebRTC 和教学演示里最稳定、最容易复现；同时，本章的 VENC/VDEC minimal 样例、基准脚本和 WebRTC 综合案例都补充了 H.265/HEVC 路径。对比来看：(1) CANN VENC/VDEC 对 H.264 和 H.265 的 API 基本一致，主要差别是 `entype`；(2) ARM 平台上 libx264 的测试码流更容易生成；(3) H.265 可作为同一套 DVPP API 的对照路径，用于理解两种编码的工程差异。

5.1.6. ACL 初始化——四步必要步骤

任何使用 DVPP 的 Python 进程，都必须在开头执行这四个调用，顺序固定不可变：

```
import acl

ret = acl.init()                      # (1) 初始化 ACL 运行时
ret = acl.rt.set_device(0)            # (2) 选择 NPU 设备 0
ctx, ret = acl.rt.create_context(0) # (3) 在设备上创建执行上下文
ret = acl.rt.set_context(ctx)         # (4) 将上下文绑定到当前线程
```

为什么需要 context：ACL 的 context（上下文）是**线程局部**的。每个需要调用 ACL API 的线程都必须绑定自己的 context。回调线程里必须再调一遍 `set_context(ctx)`——主线程的 context 不会自动传递到回调线程。

多线程规则：一个设备可以创建多个 context，但一个 context 同时只能绑定一个线程，一个线程同时只能绑定一个 context。同一个 context 可以在不同时间绑定到不同线程（但不能同时）。

5.1.7. 通道模型

VENC 和 VDEC 都采用**通道（Channel）**模型。通道是 DVPP 硬件资源的抽象——创建一个通道就是向驱动申请一个硬件编码器/解码器实例。

通道创建遵循 “描述符 -> 通道” 两步模式：先创建通道描述符并设置参数，再调用 create_channel 申请硬件资源。描述符只是一组参数配置，create 时才真正向驱动申请资源。

DVPP 内部有两种不同的通道模型：

	VENC / VDEC 专用通道	VPC / JPEG 通用通道
创建 API	`venc_create_channel()` / `vdec_create_channel()`	`dvpp_create_channel()`（无需设置 mode）
异步机制	**回调线程**（`process_report` 轮询）	**Stream 同步**（`synchronize_stream` 阻塞）
线程模型	需要独立回调线程 + Queue	不需要额外线程
数据描述	VENC: pic_desc->stream_desc, VDEC: stream_desc->pic_desc	pic_desc -> pic_desc（同类型）

- **VENC/VDEC 回调式**：主线程发送帧后阻塞在 Queue.get()，回调线程通过 process_report 轮询硬件完成事件，触发回调后 Queue.put 唤醒主线程
- **VPC/JPEG Stream 式**：主线程下发异步任务后调用 synchronize_stream 阻塞等待，硬件完成时直接通知 Stream（无需回调线程）

通道复用 vs 创建/销毁：每次 `创建通道() -> 处理 N 帧 -> 销毁通道()` 的固定开销约 **5-10ms**。对于单帧编码场景，创建/销毁的开销远超编解码本身。最佳实践：创建一次通道，连续处理所有帧，最后销毁。

通道数是有限的：Ascend 310B4 的 VENC/VDEC 硬件实例数量有限（通常每种 1-2 个）。

5.1.8. 回调线程模型

DVPP 是**异步**的：发送工作请求后立即返回，结果通过**回调**在另一个线程中返回。

异步处理时序：(1) 主线程发送帧请求后立即返回 -> (2) 主线程阻塞在 `Queue.get()` 等待结果 -> (3) DVPP 硬件完成处理 -> (4) 回调线程收到事件 -> (5) 回调将结果 `Queue.put()` -> (6) 主线程被唤醒得到结果

VENC vs VDEC 回调的关键差异：

	VENC 回调	VDEC 回调
参数顺序	`(输入_pic_desc, 输出_stream_desc)`	`(输入_stream_desc, 输出_pic_desc)`
读取输出	`获取码流大小/数据()`	`获取图片返回码()` + `获取图片数据/大小()`
返回码检查	无	**必须检查**，非 0 = 解码失败
销毁输入	`销毁图片描述符(输入)`	`销毁码流描述符(输入)`
销毁输出	不需要（调用方管理）	**`销毁图片描述符(输出)`**

记忆方法：第一个参数总是 “输入”，第二个总是 “输出”。VENC 输入图片-> 输出码流；VDEC 输入码流-> 输出图片。

为什么用 Queue 而不是 Event：Queue 天然适合 “生产者（回调线程）-> 消费者（主线程）” 模式——支持缓冲（多帧排队）、阻塞等待（`Queue.get(timeout=5.0)`）、线程安全（无需额外锁）。

为什么是 300ms：`process_report(300ms)` 阻塞最多 300ms 等待 DVPP 硬件完成通知。太短（如 10ms）会高频 CPU 轮询，太长（如 5000ms）会导致销毁通道时等待过久。300ms 是平衡值。

5.1.9. DVPP 内存管理

DVPP 有两套内存系统，必须正确区分：

操作	分配位置	访问方式	用途	释放
`dvpp_malloc(大小)`	**设备端**（NPU 片内或 DDR）	不能被 CPU 直接读写	DVPP 硬件访问的输入/输出缓冲区	`dvpp_free(ptr)`
`malloc_host(大小)`	**主机端**（系统 DDR）	CPU 可正常读写	回调中临时中转数据	`free_host(ptr)`

数据搬运方向：`memcpy(设备内存，主机数据，大小)` — 主机-> 设备（发送数据给 DVPP）；`memcpy(主机内存，设备数据，大小)` — 设备-> 主机（取回结果）。

常见内存错误：

错误	现象	原因
忘记 `dvpp_free`	内存泄漏 -> 后续分配失败	每帧分配但未释放
忘记 `free_host`	主机内存泄漏	回调中分配主机内存后未释放
在主机上直接读设备指针	段错误 / 无效数据	设备内存不能直接被 CPU 访问
回调中未销毁输入描述符	内存泄漏	VENC pic_desc / VDEC stream_desc 必须由回调销毁

5.1.10. 描述符模型

DVPP 使用两种描述符来描述输入/输出数据：

- **pic_desc（图片描述符）**：描述一帧图像（NV12 / RGB 等），包含数据指针、大小、格式、宽高、stride。VDEC 专用字段：`ret_code`（0= 解码成功，非 0= 失败）。
- **stream_desc（码流描述符）**：描述一段压缩码流（H.264 / H.265 / JPEG），包含数据指针和大小。

子模块	输入描述符	输出描述符	回调参数顺序
VENC	pic_desc	stream_desc	`(input_pic_desc, ↪ output_stream_desc)`
VDEC	stream_desc	pic_desc	`(input_stream_desc, ↪ output_pic_desc)`
JPEGE	pic_desc	stream_desc	同 VENC
JPEGD	stream_desc	pic_desc	同 VDEC

5.1.11. NV12——DVPP 的通用货币

NV12（也叫 YUV420SP）是 DVPP 所有图像相关模块的首选像素格式。

为什么 NV12：(1) 体积小——每像素 1.5 字节（RGB 是 3 字节），省 50% 内存和带宽；(2) 人眼匹配——利用人对亮度敏感、对色度不敏感的特性，降低色度分辨率；(3) 硬件原生——VENC/VDEC 硬件内部直接处理 NV12；(4) 摄像头兼容——大多数摄像头输出 YUV 格式，接近 NV12。

内存布局：NV12 缓冲区 = [Y 平面]（H×W，每像素 1 字节亮度）+ [UV 交错平面]（H/2 × W，每 2 字节一组 U,V）。总字节数 = H × W × 3/2。

与其他 YUV 格式的区别：

格式	全称	内存布局	DVPP 支持
NV12	YUV420SP	Y 平面 + UV 交错平面	VENC / VDEC / VPC / JPEGE
NV21	YVU420SP	Y 平面 + VU 交错平面 (U/V 顺序相反)	VDEC / VPC
I420	YUV420P	Y 平面 + U 平面 + V 平面（3 个独立平面）	VPC (部分)
YUYV	YUV422	YUYV 交错（每 2 像素共享 UV）	— (需 VPC 转换)

USB 摄像头输入 -> NV12 的路径：大多数 USB 摄像头输出 YUYV 或 MJPG，不是 NV12。转换可选 CPU 路径（`cv2.cvtColor + bgr_to_nv12()`）或 VPC 路径（VPC CSC: YUYV->NV12，但 310B 不支持 CSC）。

5.1.12. DVPP V1 与 himpi V2 — 两套 API 体系

CANN 为 Ascend 310B 提供了两套不同的媒体处理 API：

	DVPP V1 (`acl.media`)	himpi V2 (`acl.himpi`)
全称	Digital Vision Pre-Processing	Hi Media Processing Interface
定位	AscendCL 通用媒体处理	专用媒体处理（对标 HiMPP）

	DVPP V1 (acl.media)	himpi V2 (acl.himpi)
通道模型	VENC/VDEC 专用 + 通用 dvpp 通道	统一 *_create_chn
310B Python 可用性	**大部分可用**	**通道创建不可用**

himpi 的 *_create_chn 函数需要传入 C 结构体（如 hi_vpc_chn_attr），Python 侧不支持创建这些结构体。

选择指南：在 310B 上处理媒体数据时：

- **VENC/VDEC 编解码** -> acl.media.venc_* / vdec_*（唯一选择）
- **VPC resize/crop** -> acl.media.dvpp_vpc_*_async
- **JPEG 编解码** -> acl.media.dvpp_jpeg_*_async
- **旋转/翻转/滤波/仿射** -> CPU (OpenCV)
- **310P/710 等新硬件** -> himpi V2

5.1.13. 典型 DVPP 使用模式

- **视频推理管道**：VDEC 解码 -> VPC YUV 格式调整与缩放 -> 若需 RGB 或额外预处理，再由 AIPP 完成 -> 传入模型
- **静态图片分类**：JPEGD 解码 -> VPC 缩放/裁剪 -> 如需色域/像素变换使用 AIPP -> 传入模型
- **全硬件转码**：VDEC（H.264->NV12）-> VPC（resize）-> VENC（NV12->H.264），设备内零拷贝

5.1.14. 开发注意事项

- DVPP 输出对分辨率与地址有对齐要求（stride/padding），读取数据时需依据描述信息处理
- 使用异步接口（*_async）时必须配合 Stream 与同步机制
- Host->Device 的异步拷贝源内存应使用页锁定（pinned）内存（acl.rt.malloc_host）
- 优先把耗时的像素级操作下沉到 DVPP/AIPP，避免在 CPU（Python）端逐像素处理
- 310B 上 dvpp_vpc_convert_color_async（CSC 色彩空间转换）不可用

5.2. VENC — 硬件视频编码

VENC（Video Encoder）将 NV12 原始帧编码为 H.264/H.265 码流。前置基础：DVPP 基础概念。

文中所有完整可运行的代码在 samples/chapter5/ 目录下。

5.2.1. 理论背景

为什么需要硬件编码

H.264 视频编码是计算密集型任务。一块 640×480@30fps 的视频流，纯 CPU 软件编码（如 libx264）会占用 ARM Cortex-A55 的大量计算资源。对于 Orange Pi AI Pro 这样的嵌入式设备，CPU 资源有限，软件编码不仅影响视频质量（可能因算力不足而降低帧率），还挤占了其他任务的 CPU 时间。

昇腾 310B 芯片内部集成了 **VENC（Video Encoder）**硬件模块，专用于 H.264/H.265 编码。硬件编码器具有：

- **固定功能电路**：编码路径完全硬化，功耗和延迟远低于通用 CPU
- **独立于 AI Core**：不占用 NPU 推理算力
- **实时性保证**：硬件 pipeline 确保编码在固定时间内完成

CANN / ACL 体系

CANN（Compute Architecture for Neural Networks）是华为昇腾芯片的全栈软件栈，其层级结构如下：

- **ACL**（Ascend Computing Language）：CANN 的核心编程接口，提供设备管理、内存管理、媒体处理等 API
- **DVPP**（Digital Vision Pre-Processing）：数字视觉预处理模块，包含 VENC（编码）、VDEC（解码）、VPC（图像处理）、JPEG 编解码等
- **acl.media**：Python 侧对 DVPP 的封装

VENC 在 DVPP 中的位置

DVPP 各子模块分工详见 子模块概览。VENC 的职责：**NV12 原始帧 -> H.264/H.265 码流**。

VDEC 是它的镜像：H.264/H.265 码流 -> NV12。两者串联可形成全硬件转码管道。

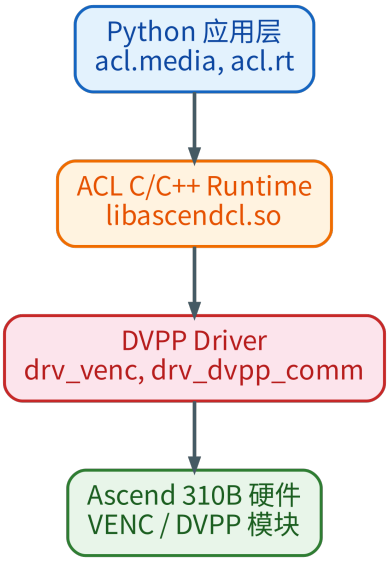


图 5.1. CANN/ACL 与 DVPP 的调用层次

图 5.2. VENC 编码端到端流程

NV12 格式

VENC 的输入格式必须是 **NV12**（YUV420SP）。详见 NV12 格式，这里只强调 VENC 的关键约束：

- **总大小**：H × W × 3/2 字节（对比 RGB 的 H × W × 3，节省 50%）
- **stride 对齐**：VENC 要求宽度对齐到 16，未对齐会导致编码画面偏移或绿条。对齐公式：`((width + 15) // 16) * 16`

编码流程（端到端）

5.2.2. 环境与架构

硬件

- **芯片**：Ascend 310B4（Orange Pi AI Pro）
- **VENC 模块**：支持 H.264 Baseline/Main/High，H.265 Main

- **驱动**：`drv_venc`, `drv_h264e`, `drv_h265e`（通过 `lsmod | grep venc` 验证）

软件

- **CANN 版本**：8.0.0、8.3.RC1（均由 CANN 7.0 升级，WebRTC 性能结果一致）
- **安装路径**：当前板端为 `/usr/local/Ascend/ascend-toolkit/8.0.0/`；8.3 使用对应版本目录
- **Python API**：`/usr/local/Ascend/ascend-toolkit/latest/python/site-packages/acl/`
- **动态库**：`/usr/local/Ascend/ascend-toolkit/latest/aarch64-linux/lib64/`

Orange Pi AI Pro 系统与固件兼容性

> **重要前置条件（板端实测，结论范围有限）**：日期戳为 `20250925` 的 Orange Pi AI Pro 最新系统镜像携带的固件，根据板端实测记录会导致 **VENC H.264 硬件编码失败**（`observed-fail`）。这是该系统/固件组合的兼容性问题，不能仅凭此结果推断 310B VENC 或所有新系统均 “不支持”。

要复现本章的 VENC H.264 示例和 WebRTC H.264 硬件编码路径，应使用日期戳为 `20241128` 的旧版系统镜像，并保留该镜像配套的旧版固件；然后在此基础上将 CANN 7.0 升级到 **CANN 8.0 或更高版本**。当前 WebRTC 已验证的两组软件组合如下；两组测试使用相同的 `20241128` 系统和配套旧版固件，性能结果一致：

系统/固件	CANN 升级路径	WebRTC H.264 VENC	性能结论
`20241128` 旧系统/旧固件	CANN 7.0 -> CANN 8.0	`observed-pass`	与 CANN 8.3 结果一致
`20241128` 旧系统/旧固件	CANN 7.0 -> CANN 8.3	`observed-pass`	与 CANN 8.0 结果一致

因此，本节此前记录的性能结果不因 CANN 8.0 新增验证而改变；CANN 8.0 验证只是对原有 WebRTC 结论增加了一组软件版本证据。后续还会继续测试更多 CANN 版本，目前不能把未测试版本写成已验证。

针对日期戳为 `20250925` 的最新系统，现有测试显示 H.264 VENC 在 CANN 8.0、CANN 8.5 和 CANN 9.0 下均无法使用。该失败归因于系统配套固件，而不是 CANN 版本或 API 参数；在香橙派发布修复固件前，应继续使用 `20241128` 旧系统和旧版固件。

环境变量

每次使用 CANN Python API 前必须设置：

```
export LD_LIBRARY_PATH="/usr/local/Ascend/ascend-toolkit/latest/aarch64-linux/lib64:/usr/local/Ascend/driver/lib64:$LD_LIBRARY_PATH"
export PYTHONPATH="/usr/local/Ascend/ascend-toolkit/latest/python/site-packages:$PYTHONPATH"
```

图 5.3. VENC 通道输入输出模型

这两个变量分别解决了：

- `libascendcl.so: cannot open shared object file` — 动态库找不到
- `No module named 'acl'` — Python 包找不到

5.2.3. VENC API 详解

ACL 初始化

4 步固定初始化详见 ACL 初始化，此处只给出 VENC 上下文的代码：

```python
import acl

ret = acl.init()                    # (1)
ret = acl.rt.set_device(0)          # (2)
ctx, ret = acl.rt.create_context(0) # (3)
ret = acl.rt.set_context(ctx)       # (4)
assert ret == 0
```

所有后续的 VENC API 调用都依赖这个上下文。

VENC 通道模型

通道模型详见 通道模型，这里只列出 VENC 特有的 API：

函数	用途
`venc_create_channel_desc()`	创建通道描述符
`venc_set_channel_desc_*()`	设置通道参数（见下节）
`venc_create_channel(desc)`	创建通道（返回 0 即成功）
`venc_send_frame(...)`	发送一帧到编码器
`venc_create_frame_config()`	创建帧配置（控制 force I-frame 等）
`venc_destroy_channel(desc)`	销毁通道
`venc_destroy_channel_desc(desc)`	销毁描述符

通道参数详解

创建 VENC 通道前，必须在描述符上设置编码类型、像素格式、分辨率、GOP、码率等参数。完整参数表及常用值见 VENC 参数速查表。

回调机制

回调线程模型详见 回调线程模型。VENC 的回调特点：

- 参数顺序：**`(input_pic_desc, output_stream_desc)`** — 第一个是输入图片，第二个是输出码流
- 输入 `pic_desc` 必须由回调销毁（`dvpp_destroy_pic_desc`）
- 输出 `stream_desc` 的数据需在回调中通过 `malloc_host` + `memcpy` 拷到主机内存
- 通过 `queue.Queue` 将编码结果传回主线程，实现异步-> 同步转换

5.2.4. 开发过程与常见问题与调试

问题 #1：Python 环境找不到 acl 模块

现象：

```
ModuleNotFoundError: No module named 'acl'
```

根因：CANN 的 Python 包不在标准 `sys.path` 中。即使 `conda activate` 了正确的环境，CANN 的 site-packages 也不会自动加入搜索路径。

修复：

```
export PYTHONPATH="/usr/local/Ascend/ascend-toolkit/latest/python/site-packages:
    ↪ $PYTHONPATH"
```

说明：CANN 的 Python 路径是 `<toolkit>/python/site-packages`，**不是** `<toolkit>/aarch64-linux/python/site-packages`。`aarch64-linux/` 下只有动态库（`lib64/`），没有 Python 包。

> **是否在代码中修复？** 示例脚本遵循 “环境变量在进程外设置” 的原则，不内置 `sys.path` 操作代码。这样可以明确依赖来源，避免隐式修改 Python 搜索路径。

问题 #2：libascendcl.so 找不到

现象：

```
ImportError: libascendcl.so: cannot open shared object file: No such file or directory
```

根因：即使 `import acl` 成功（因为 Python 包路径正确），底层 C 扩展仍需要动态库。

修复：

```
export LD_LIBRARY_PATH="/usr/local/Ascend/ascend-toolkit/latest/aarch64-linux/lib64:/usr/local/Ascend/driver/lib64:$LD_LIBRARY_PATH"
```

问题 #3：venc_create_channel 返回 507018 — bitrate 单位错误

现象：

```
RuntimeError: venc_create_channel failed: 507018 (0x7bc8a)
dmesg: rc_drv_check_bit_rate bit rate set 2000000k error for out of [2k, 614400k]
```

根因：`max_bit_rate` 的单位是 **kbps**（千比特/秒），不是 bps（比特/秒）。

我传入了 `2_000_000`（2 Mbps 以 bps 表示），被 VENC 驱动解释为 2,000,000 kbps = 2 Gbps，远超上限 614,400 kbps。

修复：

```
# 错误
media.venc_set_channel_desc_max_bit_rate(desc, 2_000_000)  # bps -> 超出范围

# 正确
media.venc_set_channel_desc_max_bit_rate(desc, 2_000)       # kbps = 2 Mbps
```

dmesg 调试技巧：VENC 错误信息会写入内核日志，`dmesg | grep -i venc` 是排查参数问题的第一手段。

问题 #4：venc_create_channel 返回 507018 — GOP 为 0

现象：

```
dmesg: rc_check_com_attr gop 0 err, should be in [1 65536]
dmesg: rc_create_chn check user rc attr err
```

根因：`key_frame_interval`（即 GOP — Group of Pictures）未设置，默认值为 0，不在合法范围 [1, 65536] 内。

GOP 控制多少个 P/B 帧之间插入一个 I 帧（关键帧）。GOP=30 意味着每 30 帧一个 I 帧，这在 30fps 下相当于每秒一个关键帧。GOP=0 对编码器没有意义（永远不产生 I 帧?），因此被拒绝。

修复：

```
media.venc_set_channel_desc_key_frame_interval(desc, 30)  # GOP=30
```

问题 #5：venc_set_channel_desc_channel_id 不存在

现象：

```
AttributeError: module 'acl.media' has no attribute 'venc_set_channel_desc_channel_id'
```

根因：VDEC 有 `vdec_set_channel_desc_channel_id`，但 VENC 的 API 中**没有对应的 setter**。VENC 的 channel_id 由驱动自动分配，不能手动设置。

这暴露了 CANN API 的一个不对称设计：VDEC 和 VENC 虽然结构相似，但细节不同，不能简单类比。

问题 #6：NumPy 维度索引错误

现象：

```
ValueError: could not broadcast input array from shape (640,640) into shape (640,)
```

根因：NV12 数据是 2D 数组 `(H*3/2, W)`，但代码用 1D 线性偏移去索引：

```
# 错误：nv12_data[src_off : src_off + w] 切出了形状 (w, w) 而非 (w,)
nv12_padded[off : off + w] = nv12_data[src_off : src_off + w]  # 2D -> 1D 广播失败
```

当 `src_off = row * w = 0` 时，`nv12_data[0:640]` 取到的是整个 Y plane 的前 640 行（即整个 640×640 区域），而不是第 0 行的 640 个像素。

修复：使用 2D 切片明确行列：

```
nv12_padded_2d = np.zeros(stride * h * 3 // 2, dtype=np.uint8).reshape(-1, stride)
nv12_src_2d = nv12_data.reshape(-1, w)

# Y plane
for row in range(h):
    nv12_padded_2d[row, :w] = nv12_src_2d[row, :w]

# UV plane
```

```
for row in range(h // 2):
    nv12_padded_2d[h + row, :w] = nv12_src_2d[h + row, :w]

nv12_padded = nv12_padded_2d.ravel()
```

问题 #7：stride 对齐

VENC 要求输入帧的宽度**对齐到 16**（硬件约束）。NV12 数据填充时，Y plane 每行宽度应为 `aligned_width`（stride），UV plane 同理。

```
self._align = 16
self._stride = ((width + self._align - 1) // self._align) * self._align
# 640 -> 640 (已对齐), 638 -> 640 (补齐)
```

不设置 stride 对齐会导致编码出的画面出现偏移或绿条。

问题 #8：NPU Alarm 状态混淆

现象：

```
npu-smi info: Health = Alarm
```

该状态容易被误认为 VENC 不可用。但实际测试表明 Alarm 不影响 VENC（参数正确就能创建成功）。`Alarm` 可能与其他传感器（温度、电源）有关，不一定反映 DVPP 模块状态。

调试建议：不要仅依据 NPU 全局状态判断模块可用性，应通过 `dmesg` 获取具体的模块级错误信息。

5.2.5. 练习脚本

三个可独立运行的脚本位于 `samples/chapter5/`，建议按顺序阅读理解。

概览

文件	学习内容	运行时间
`check_cann.py`	ACL 初始化的 4 个必要调用	<1s

文件	学习内容	运行时间
venc_minimal.py	原始 VENC API：同一帧 NV12 分别编码为 H.264/H.265	~3s
bench_venc.py	CannVenc 封装类 + H.264/H.265 分辨率扫描对比	~20s

实现方式：本节示例直接使用 acl.media.venc_* API，不依赖额外封装库。

```
export LD_LIBRARY_PATH="/usr/local/Ascend/ascend-toolkit/latest/aarch64-linux/lib64:/usr/local/Ascend/driver/lib64:$LD_LIBRARY_PATH"
export PYTHONPATH="/usr/local/Ascend/ascend-toolkit/latest/python/site-packages:$PYTHONPATH"

python samples/chapter5/check_cann.py        # -> ACL init OK  soc=Ascend310B4
python samples/chapter5/venc/venc_minimal.py      # -> H.264/H.265 关键帧编码结果
python samples/chapter5/venc/bench_venc.py        # -> H.264/H.265 VENC vs CPU 帧率表
```

走读：ACL 初始化 — check_cann.py

```
import acl

ret = acl.init()                    # (1) 初始化 ACL 运行时
ret = acl.rt.set_device(0)          # (2) 绑定设备 0
ctx, ret = acl.rt.create_context(0) # (3) 创建执行上下文
ret = acl.rt.set_context(ctx)       # (4) 绑定上下文到当前线程
```

这四个调用是**固定的**，顺序不能变。任何使用 CANN 的 Python 进程都需要它们。

走读：最小编码 — venc_minimal.py

这是理解 VENC 的核心文件。脚本用同一帧确定性 NV12 输入，分别创建 H.264 Baseline 与 H.265 Main 通道并编码一帧。两个路径的 API 基本一致，主要差异是 entype：

```
ENTYPE_H265_MAIN = 0
ENTYPE_H264_BASE = 1

CODECS = [
    ("H.264 Baseline", ENTYPE_H264_BASE, 2_000),
    ("H.265 Main", ENTYPE_H265_MAIN, 2_000),
]
```

代码分为 5 个阶段：

(1) ACL 初始化 — 与 step1 相同。

(2) 回调线程 — VENC 是异步的：

```
cb_queue: queue.Queue = queue.Queue(maxsize=8)

def venc_callback(input_pic_desc, output_stream_desc, _user_data):
    size = media.dvpp_get_stream_desc_size(output_stream_desc)  # 读取编码后大小
    ptr  = media.dvpp_get_stream_desc_data(output_stream_desc)  # 读取数据指针
    host_buf = acl.rt.malloc_host(size)        # 分配主机内存
    acl.rt.memcpy(host_buf, size, ptr, size, ACL_MEMCPY_DEVICE_TO_HOST)
    cb_queue.put(ctypes.string_at(host_buf, size))  # 拷贝为 Python bytes
    acl.rt.free_host(host_buf)
    media.dvpp_destroy_pic_desc(input_pic_desc)     # 回调负责销毁输入

def callback_thread(_args):                 # 独立线程处理回调
    acl.rt.set_context(ctx)                 # 必须重新绑定上下文
    while running[0]:
        acl.rt.process_report(300)          # 300ms 轮询
```

(3) 创建通道 — 按当前 codec 设置 `entype`、NV12 格式、分辨率、GOP、帧率和码率后调用 `venc_create_channel()`。H.264 Baseline 使用 `entype=1`，H.265 Main 使用 `entype=0`。

(4) 发送一帧 — NumPy 生成紧凑 NV12 -> 宽度 padding 到 16 对齐 -> `dvpp_malloc` -> `venc_send_frame` -> `cb_queue.get()` 等待 H.264/H.265 码流结果。

(5) 清理 — 销毁通道、描述符、帧配置，释放 DVPP 内存。

走读：封装与基准 — bench_venc.py

`bench_venc.py` 将原始 VENC API 封装为可复用的 `CannVenc` 类，然后做 **H.264/H.265 双分辨率扫描**对比硬件 vs CPU 编码性能。整个文件 ~380 行，分为 6 个部分。

(1) 测试参数

```
RESOLUTIONS = [
    (640, 480),
    (1280, 720),
    (1920, 1080),
    (2560, 1440),      # 2K
    (3840, 2160),      # 4K
]

TEST_FRAMES = 90            # 恰好 3 个 GOP（GOP=30 × 3）
TEST_GOP = 30                # 1 个 I + 29 个 P，模拟真实视频流
RANDOM_SEED = 42             # 固定种子 -> 结果可复现
WARMUP_FRAMES = 3
FPS = 30
```

- **90 帧**：3 个完整 GOP，每个 GOP 含 1 个 I 帧 + 29 个 P 帧，I 帧占比 3.3%，与真实视频流一致

- **固定种子 42**：同一种子下任何机器生成的测试帧内容相同，保证跨运行可复现
- **3 帧预热**：排除首次编码的驱动初始化开销（比旧版 10 帧更精简）

(2) 确定性测试帧生成

```
def make_test_yuv420_frame(i: int, w: int, h: int) -> tuple[np.ndarray, np.ndarray, np.
    ↪ ndarray]:
```

直接生成 YUV420 平面，再按测试路径转换：VENC 路径拼成 NV12，CPU 路径拼成 yuv420p/I420。这样无需 BGR/RGB 转换，测量的是**纯编码性能**。每帧包含：

- **水平渐变 + 垂直渐变叠加**
- **正弦移动白条**：模拟时间相关性，防止编码器走 P 帧" 全零残差" 捷径
- **角落棋盘格**：8×8 方块交替，测试空间纹理编码

(3) `CannVenc` 类详解

将 `venc_minimal.py` 的原始 API 封装为可复用的同步接口。

`__init__` — 通道创建：与 `venc_minimal.py` 相同逻辑，额外计算 stride 对齐（`((w+15) //16)*16`）和输出缓冲区大小。

`_venc_callback` — 编码完成回调：从 `output_stream_desc` 读取编码数据 -> `malloc_host` -> `memcpy` 拷到主机内存 -> 放入 Queue。回调负责销毁输入的 `pic_desc`。

`encode(nv12, force_keyframe)` — 编码一帧：

```
# (1) NV12 宽度补齐到 stride（16 对齐）—— VENC 硬件约束
padded = np.zeros(stride * h * 3 // 2, dtype=np.uint8).reshape(-1, stride)
src = nv12.reshape(-1, w)
for r in range(h):                  # Y 平面逐行拷贝
    padded[r, :w] = src[r, :w]
for r in range(h // 2):              # UV 平面逐行拷贝
    padded[h + r, :w] = src[h + r, :w]

# (2) 分配 DVPP 输入内存 + 拷贝 NV12 到设备
in_buf, _ = media.dvpp_malloc(padded.nbytes)
acl.rt.memcpy(in_buf, padded.nbytes, padded.ctypes.data, padded.nbytes,
              ACL_MEMCPY_HOST_TO_DEVICE)

# (3) 构造输入 pic_desc 和输出 stream_desc
pic = media.dvpp_create_pic_desc()
media.dvpp_set_pic_desc_data(pic, in_buf)
media.dvpp_set_pic_desc_format(pic, PIX_FMT_NV12)
media.dvpp_set_pic_desc_width_stride(pic, stride)    # ← 必须设置 stride
# ... 输出 out_buf + stream_desc ...

# (4) 可选强制 I 帧
if force_keyframe:
    media.venc_set_frame_config_force_i_frame(self._frame_cfg, True)

# (5) 排空回调队列（防止上一帧残留干扰）
```

```
while not self._cb_queue.empty():
    self._cb_queue.get_nowait()

# (6) 发送编码请求 + 等待回调
media.venc_send_frame(self._ch_desc, pic, sd, self._frame_cfg, None)
encoded = self._cb_queue.get(timeout=5.0)

# (7) 清理：释放 DVPP 内存、销毁 stream_desc、恢复 force I-frame 标志
```

destroy() — 先 venc_destroy_channel 再停回调线程，与 VDEC 的销毁顺序要求类似。

(4) CPU 编码对比 — bench_cpu_encode()

```
def bench_cpu_encode(w: int, h: int, codec_name: str, bitrate_bps: int,
                     thread_count: int) -> tuple:
    level = "31" if w * h <= 1280 * 720 else "40"   # <=720p -> 3.1, >=1080p -> 4.0
    codec = av.CodecContext.create(codec_name, "w")
    codec.thread_count = thread_count                # 0=自动多线程，1=单线程
    codec.bit_rate = bitrate_bps                     # bps（注意与 VENC 的 kbps 区分）
    codec.options = {"level": level, "tune": "zerolatency"}
    if codec_name == "libx264":
        codec.profile = "Baseline"                   # H.264 与 VENC entype=1 对应
```

参数与 VENC 对齐：H.264 使用 Baseline profile、zerolatency tune、相同码率。最新脚本不再生成 BGR/RGB 测试帧，而是统一生成 YUV420 平面：VENC 路径拼成 NV12，CPU 路径拼成 yuv420p/I420 直接交给 PyAV，避免把颜色转换成本混入编码对比。

(5) 主流程 — 分辨率扫描

```
for w, h in RESOLUTIONS:
    bitrate_kbps = max(2000, int(w * h * FPS * 0.1 / 1000))
    bitrate_bps = bitrate_kbps * 1000

    # VENC 测量
    venc = CannVenc(w, h, bitrate=bitrate_kbps, entype=entype)
    for i in range(WARMUP_FRAMES):                    # 预热
        y, u, v = make_test_yuv420_frame(i, w, h)
        venc.encode(yuv420_to_nv12(y, u, v), force_keyframe=(i == 0))
    t0 = time.perf_counter()
    for i in range(TEST_FRAMES):                      # 正式测量
        y, u, v = make_test_yuv420_frame(i, w, h)
        venc.encode(yuv420_to_nv12(y, u, v), force_keyframe=(i % TEST_GOP == 0))
    venc_fps = TEST_FRAMES / (time.perf_counter() - t0)

    # CPU 测量
    _, cpu_mt_fps, _ = bench_cpu_encode(w, h, cpu_codec_name, bitrate_bps, thread_count=0)
    _, cpu_st_fps, _ = bench_cpu_encode(w, h, cpu_codec_name, bitrate_bps, thread_count=1)
```

码率自适应公式 `max(2000, w*h*fps*0.1/1000)` kbps：

- 480p：640×480×30×0.1/1000 = 921 -> **2000 kbps**（下限 2 Mbps）

- 720p：1280×720×30×0.1/1000 = 2764 -> **2764 kbps**
- 1080p：1920×1080×30×0.1/1000 = 6220 -> **6220 kbps**
- 4K：3840×2160×30×0.1/1000 = 24883 -> **24883 kbps**

含义：每像素每秒分配 0.1 bit，按分辨率等比缩放。

(6) 本文件与 `venc_minimal.py` 的关系

	venc_minimal.py	bench_venc.py
目的	教学——展示每个 API 调用	基准——评估性能
帧数	1 帧	90 帧 × 5 分辨率
封装	裸 API 直接调用	CannVenc 类
对比	无	与 libx264/libx265 A/B 对比
内容	确定性 NV12 单帧	确定性 YUV420 序列（渐变 + 条 + 棋盘）
输出	H.264/H.265 关键帧大小	H.264/H.265 双表格

5.2.6. 集成到 aiortc

VENC 可集成到 aiortc（Python WebRTC 库）替代默认的 libx264 编码器。

基本思路

通过**猴子补丁**替换 aiortc 的 H.264 编码器：

```
import aiortc.codecs.h264 as h264_module
h264_module.H264Encoder = YourCannEncoder
```

继承策略

推荐**继承** `H264Encoder`：只需覆盖 `_encode_frame()` 接入 VENC，其余 RTP 封装（RFC 6184 的 FU-A 分片、STAP-A 聚合等）全部继承。

`H264Encoder` (aiortc) 的继承结构——只需覆盖 `_encode_frame()` 接入 VENC，其余 RTP 封装全部继承：

- `_encode_frame()` -> libx264 编码 **[待覆盖]**
- `_packetize()` -> NAL -> RTP 分包 **[继承]**
- `_split_bitstream()` -> Annex-B -> NAL 分割 **[继承]**
- 其他全部继承

失败策略

H.264 硬件路径是严格的 CANN VENC 路径。CANN 不可用、VENC 通道创建失败或编码失败时，服务必须报告错误并结束硬件路径；不能改用 `libx264`，也不能用合成帧掩盖 DVPP/JPEGD 初始化失败：

```
class CannH264Encoder(H264Encoder):
    def _encode_frame(self, frame, force_keyframe):
        if not _CANN_READY:
            raise RuntimeError("CANN ACL is required for H264 VENC encoding")
        try:
            # CANN VENC 编码...
        except RuntimeError as exc:
            raise RuntimeError("CANN VENC H264 encoding failed") from exc
```

`demo` 和 `usb_camera` 的软件编码是用户显式选择的独立模式，不是硬件路径的替代。硬件路径必须使用上一节规定的 `20241128` 旧系统/旧固件 + CANN 8.0+ 组合；`20250925` 系统上的 H.264 VENC 失败应直接暴露为失败。

5.2.7. 性能对比与基准测试

实测数据：CANN VENC vs CPU 编码

以下数据在 Orange Pi AI Pro（Ascend 310B4）上实测获得，使用 `bench_venc.py` 脚本；复现实验必须满足上一节规定的 **`20241128` 旧系统/旧固件 + CANN 8.0+** 前置条件。WebRTC 已分别验证 CANN 7.0 升级到 8.0 和升级到 8.3 的组合，系统日期戳保持不变，性能结果一致。该脚本同时覆盖 H.264 与 H.265，并分别测量 CPU 自动多线程（`thread_count=0`）和 CPU 单线程（`thread_count=1`）。其中 H.264 VENC 帧率不能外推到 `20250925` 系统，因为该系统在 CANN 8.0、8.5 和 9.0 下均为 `observed-fail`；H.265 仍需在目标系统上单独验证。

测试条件：GOP=30（I/P 混合），90 帧（3 个完整 GOP），确定性测试帧，固定种子 42。码率按分辨率自动缩放：H.264 使用 `max(2M, w*h*fps*0.1)` bps，H.265 使用 0.7 倍目标码率。测试帧统一生成为 YUV420 平面，VENC 路径转为 NV12，CPU 路径转为 yuv420p/I420，避免额外 BGR/RGB 颜色转换。

H.264 Baseline

分辨率	VENC帧率	CPU多线程	CPU单线程
640x480	166.4	86.6	46.5
1280x720	92.8	39.1	21.2
1920x1080	48.4	13.4	6.0
2560x1440	21.3	11.0	6.8
3840x2160	15.8	7.1	3.7

H.265 Main

```
分辨率          VENC帧率     CPU多线程     CPU单线程
──────────────────────────────────────────────
640x480          165.1         31.1         22.9
1280x720          96.6         18.8         14.7
1920x1080         50.7          7.3          6.2
2560x1440         31.1          6.2          5.1
3840x2160         15.4          2.6          2.2
```

结果解读

H.264 Baseline

分辨率	VENC fps	CPU 单线程	CPU 多线程	vs 单线程	vs 多线程
640×480	166	47	87	**3.58x**	**1.92x**
1280×720	93	21	39	**4.38x**	**2.37x**
1920×1080	48	6	13	**8.07x**	**3.61x**
2560×1440	21	7	11	**3.13x**	**1.94x**
3840×2160	16	4	7	**4.27x**	**2.23x**

H.265 Main

分辨率	VENC fps	CPU 单线程	CPU 多线程	vs 单线程	vs 多线程
640×480	165	23	31	**7.21x**	**5.31x**
1280×720	97	15	19	**6.57x**	**5.14x**
1920×1080	51	6	7	**8.18x**	**6.95x**
2560×1440	31	5	6	**6.10x**	**5.02x**
3840×2160	15	2	3	**7.00x**	**5.92x**

实时性观察

- H.264：VENC 在 480p、720p、1080p 均超过 30fps；2K/4K 单路未达到 30fps，但仍明显快于 CPU。
- H.265：VENC 在 480p、720p、1080p、2K 均超过 30fps；4K 约 15fps，仍约为 CPU 多线程的 5.9 倍。
- CPU 多线程比单线程快，但在所有测试点都低于 VENC。H.265 软件编码尤其重，CPU 多线程在 1080p 仅约 7fps。

与 VDEC 的对比

VENC 和 VDEC 在 Ascend 310B4 上的表现模式不同：

对比维度	VENC（编码）	VDEC（解码）
H.264 <=1080p	**领先 CPU 多线程**	落后 CPU
H.265 <=1080p	**大幅领先 CPU 多线程**	720p 以上领先 CPU 多线程
4K 表现	仍快于 CPU，但单路低于 30fps	VDEC 约 72fps
拐点	无 CPU 交叉拐点，所有点 VENC 更快	H.264 解码约从 2K 开始领先单线程 CPU
瓶颈	编码计算量大，硬件收益明显	解码计算量较小，Python 回调开销更显著
适合场景	实时推流、录制、转码输出	高分辨率/HEVC 解码、多路或零拷贝管道

根因：视频编码计算量大，H.264/H.265 软件编码很快把 ARM CPU 打满；VENC 把运动估计、变换、量化、熵编码等重计算交给硬件完成，因此全分辨率都领先 CPU。VDEC 的解码计算量小得多，Python 回调、队列和内存搬运的固定开销更容易成为瓶颈。

什么场景下使用 VENC

场景决策树

需要实时视频编码（>30fps）?

- **是** — 取决于分辨率:
  - <=480p: CPU H.264 多线程可超过 30fps，但 VENC 更快且释放 CPU -> 建议 VENC
  - 720p/1080p: CPU 编码余量不足或不可用 -> 建议 VENC
  - 2K: H.264 VENC 约 21fps，H.265 VENC 约 31fps；需要结合目标帧率选择
  - 4K: 单路约 15fps，不满足 30fps；仍明显快于 CPU，适合低帧率录制或离线转码
- **否**（离线/批处理）-> CPU 可考虑，但 VENC 通常仍快 2×~7×，尤其 H.265 收益更大

典型场景推荐

场景	推荐方案	理由
WebRTC 视频通话	**VENC**	480p/720p 实时编码 + CPU 留给推理
USB 摄像头监控	**VENC**	1080p@30fps CPU 跑不动，VENC 可超过实时
多路视频流 (>2 路)	**VENC 必须**	CPU 单路 720p H.264 多线程约 39fps，多路余量不足
4K 录制	**VENC**	4K 单路约 15fps，适合低帧率录制；30fps 需进一步优化或降低分辨率

场景	推荐方案	理由
本地视频文件转码	均可	离线场景 CPU 也可，但 VENC 更快
AI 推理 + 视频边车	**VENC**	CPU 编码会抢占 NPU 推理的 host 侧资源
低功耗设备	**VENC**	硬件编码功耗远低于 CPU 全速运行

多路并发估算

以 1080p@30fps 为目标帧率：

编码器	单路 fps	最多支持路数	CPU 剩余
CPU libx264 多线程	13	**0 路**（不到 30）	0%
VENC H.264	48	**1 路**（48/30）	较高
VENC H.265	51	**1 路**（51/30）	较高

最新单路基准显示，1080p 下 VENC 约 48-51fps，单路 30fps 有余量，但还不足以稳定支撑 2 路 1080p@30fps。多路场景应结合实际码率、GOP、输入来源和端到端链路重新测量。

什么情况下 CPU 编码就够了

只有**离线批处理**且满足以下全部条件时，CPU 才有意义：

- 分辨率 <= 480p（CPU H.264 多线程约 87fps，CPU H.265 多线程约 31fps）
- 不需要实时输出（无帧率硬性要求）
- 无并发推理任务（CPU 全给编码用）
- 不想引入 CANN 依赖（如 Docker 环境未安装 CANN）

底线：从 720p 开始，CPU 编码余量快速下降；1080p 及以上的实时视频场景，尤其是在 Orange Pi 这样的嵌入式 ARM 设备上，优先使用 VENC。

验证方法

在 Orange Pi 上分别运行 CPU 和硬件编码服务器，用 `htop` 观察实时 CPU 占用差异：

```
# CPU 编码
python server.py --source usb_camera

# 硬件编码
python server.py --source usb_camera --hardware-encode
```

或直接跑基准测试：

```
export LD_LIBRARY_PATH="/usr/local/Ascend/ascend-toolkit/latest/aarch64-linux/lib64:/usr/local/Ascend/driver/lib64:$LD_LIBRARY_PATH"
export PYTHONPATH="/usr/local/Ascend/ascend-toolkit/latest/python/site-packages:$PYTHONPATH"
python samples/chapter5/venc/bench_venc.py
```

5.2.8. VENC 调试与参数速查

常用调试命令

```
# NPU 状态
npu-smi info
npu-smi info -t usages -i 0

# VENC 驱动状态
lsmod | grep venc

# VENC 内核日志（参数错误信息 —— 排查 507018 必用）
dmesg | grep -i venc | tail -10

# 环境验证（等价于 check_cann.py）
python samples/chapter5/check_cann.py

# 运行基准测试
python samples/chapter5/venc/bench_venc.py
```

参数速查表

VENC 通道参数

参数	函数	默认	推荐值
编码类型	`venc_set_channel_desc_entype`	—	1 (H264)
像素格式	`venc_set_channel_desc_pic_format`	—	1 (NV12)
宽度	`venc_set_channel_desc_pic_width`	—	640
高度	`venc_set_channel_desc_pic_height`	—	480
GOP	`venc_set_channel_desc_key_frame_interval`	0	30
帧率	`venc_set_channel_desc_src_rate`	—	30
最大码率 (kbps)	`venc_set_channel_desc_max_bit_rate`	—	2000
码率控制	`venc_set_channel_desc_rc_mode`	—	2 (CBR)
回调线程	`venc_set_channel_desc_thread_id`	—	tid
回调函数	`venc_set_channel_desc_callback`	—	cb

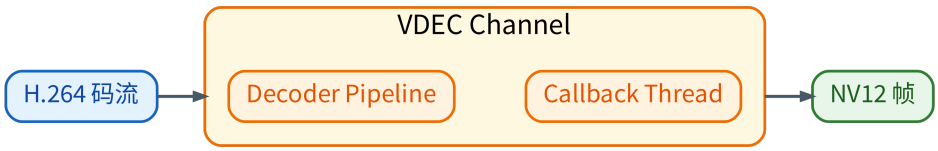


图 5.4. VDEC 通道输入输出模型

错误码：0 = ACL_SUCCESS, 507018 = 参数错误（检查 dmesg），500001 = ACL_ERROR_FAILURE, 500004 = ACL_ERROR_DRV_FAILURE

编码类型：0 = H.265 Main, 1 = H.264 Baseline, 2 = H.264 Main, 3 = H.264 High

像素格式：1 = NV12 (YUV420SP), 12 = RGB888, 13 = BGR888

码率控制：1 = VBR (Variable Bitrate), 2 = CBR (Constant Bitrate)

5.3. VDEC — 硬件视频解码

VDEC（Video Decoder）将 H.264/H.265 码流解码为 NV12 原始帧。前置基础：DVPP 基础概念。

5.3.1. VDEC 简介

VDEC（Video Decoder）是 DVPP 中的硬件视频解码模块。它将 H.264/H.265 压缩码流解码为 NV12 原始帧。

H.264 与 H.265 基础

H.264/H.265 的帧类型（I/P/B）、GOP、NAL 单元、Annex-B 格式等编解码理论知识，详见 H.264/H.265 基础。本文的 VDEC API 示例和基准脚本都覆盖 **H.264 Baseline** 与 **H.265 Main**，便于直接对照两种 codec 的 `entype` 和码流差异。

VDEC 对输入码流只有一个硬性要求：必须是 **Annex-B 格式**（带 `0x00000001` 起始码的 NAL 单元序列）。H.264 首帧必须包含 SPS + PPS + IDR；H.265 首帧必须包含 VPS + SPS + PPS + IDR。

典型应用场景

场景	数据流
视频文件回放	MP4/MKV 文件 -> 解封装 -> H.264/H.265 码流 -> **VDEC** -> NV12 -> 显示
网络摄像机接收	RTSP/WebRTC -> H.264/H.265 码流 -> **VDEC** -> NV12 -> 分析/显示
转码管道	H.264/H.265 -> **VDEC** -> NV12 -> **VENC** -> 不同分辨率/码率的 H.264/H.265

与 VENC 的对称关系

详见 子模块概览。

```
VENC: NV12 -> [硬件编码] -> H.264/H.265 码流
VDEC: H.264/H.265 码流 -> [硬件解码] -> NV12
```

硬件能力规格（Ascend 310B4）

以下基于 CANN 8.3.RC1 + Ascend 310B4 实测和驱动常量定义。

支持的编码类型

编码格式	Profile	`entype` 值	实测
H.265 / HEVC	Main	`0`	支持
H.264 / AVC	Baseline	`1`	支持
H.264 / AVC	Main	`2`	支持
H.264 / AVC	High	`3`	支持

实验中用 H.264 Baseline 码流测试了全部四种 `entype`，通道创建和 `send_frame` 均成功。但**实际解码能否正确输出**取决于输入码流是否匹配所选的 `entype`。例如：用 Baseline 码流 + `entype=H265_MAIN` 虽然不会报错，但解码结果会是花屏或黑帧（编码标准不匹配）。

支持的输出像素格式

格式	`out_pic_format` 值	位深	说明
YUV400	0	8	仅亮度
NV12 (YUV420SP)	**1**	8	推荐使用
NV21 (YVU420SP)	2	8	
NV12 4:2:2	3	8	
NV21 4:2:2	4	8	
NV12 4:4:4	5	8	
NV21 4:4:4	6	8	

格式	out_pic_format 值	位深	说明
RGB888	12	8	
BGR888	13	8	

位深 10-bit 理论上支持（vdec_set_channel_desc_bit_depth），但 310B4 驱动在 10-bit 下的实际表现未经充分验证。

分辨率和帧率约束

约束项	值	说明
最小分辨率	128×128	理论下限
最大分辨率	4096×4096	理论上限，受内存限制
宽度对齐	16	VDEC 硬件要求
高度对齐	2	VDEC 硬件要求
最大帧率	60 fps	与分辨率有关
参考帧数	1-5（默认 5）	ref_frame_num，影响解码缓冲大小

实际可解码的最大分辨率受设备内存和码率限制。310B4 配备 16GB 内存，单路 4K@30fps 解码无压力。

输入码流约束

约束项	说明
码流格式	H.264/H.265 Annex-B（带 0x00000001 起始码的 NAL 单元）
参数集	同一通道上所有帧必须共享一致的参数集；H.264 为 SPS/PPS，H.265 为 VPS/SPS/PPS
首帧要求	H.264 必须包含 SPS + PPS + IDR；H.265 必须包含 VPS + SPS + PPS + IDR
单次输入上限	取决于码流参数，超过约 256KB 可能被 VDEC 丢弃
帧边界	每次 vdec_send_frame 应发送完整的一帧（含所有 NAL 单元）

H.264 Level 与典型应用

Level	最大宏块数	典型分辨率 @ 帧率	最大码率
3.0	1620	720×480@30	10 Mbps
3.1	3600	1280×720@30	14 Mbps
4.0	8192	1920×1080@30, 2048×1024@30	20 Mbps
4.1	8192	1920×1080@30, 2048×1024@30	50 Mbps

Level	最大宏块数	典型分辨率 @ 帧率	最大码率
4.2	8704	1920×1080@60	50 Mbps
5.0	22080	2560×1920@30	135 Mbps
5.1	36864	4096×2304@30	240 Mbps

310B4 VDEC 理论上支持到 Level 5.1。本章基准测试中，480p 使用 Level 3.1，1080p 使用 Level 4.0。

5.3.2. VDEC 与 VENC 的关键区别

VDEC 与 VENC 的关键区别如下表所示（同时参考 子模块概览 获取完整的子模块间对比）：

特性	VENC	VDEC
数据方向	NV12 -> H.264/H.265	H.264/H.265 -> NV12
`channel_id`	驱动自动分配	**必须显式设置**
`out_mode`	无	**必须检查和设置**（默认通常为 0）
`ref_frame_num`	无	参考帧数量（默认 5）
`send_frame` 参数	`(pic_desc, stream_desc)`	`(stream_desc, pic_desc)`
回调中销毁对象	输入（pic_desc）	输入（stream_desc）**和**输出（pic_desc）
输入有效性要求	任意合法 NV12 均可	必须是**合法的完整 H.264/H.265 NAL 单元**
输出 ret_code	无	有，**必须检查**，非 0 表示解码失败
`send_skipped_frame`	无	**有**，用于丢帧后通知解码器

5.3.3. VDEC API 详解

通道参数

VDEC 通道需设置通道 ID、编码类型、输出像素格式、分辨率、参考帧数等参数。与 VENC 不同，VDEC 的 `channel_id` 必须显式设置。完整参数表见 VDEC 参数速查表。

回调函数

```
def vdec_callback(input_stream_desc, output_pic_desc, user_data):
```

```
    """VDEC 解码完成回调——参数顺序与 VENC 相反。"""
    ret_code = acl.media.dvpp_get_pic_desc_ret_code(output_pic_desc)
    if ret_code != 0:
        # 解码失败——丢弃此帧
        pass
    else:
        pic_data = acl.media.dvpp_get_pic_desc_data(output_pic_desc)
        pic_size = acl.media.dvpp_get_pic_desc_size(output_pic_desc)
        # ... 拷贝 pic_data 到主机内存 ...

    # 回调负责销毁输入和输出描述符
    acl.media.dvpp_destroy_stream_desc(input_stream_desc)
    acl.media.dvpp_destroy_pic_desc(output_pic_desc)
```

与 VENC 的关键区别：

- 回调参数顺序相反：VDEC 是 `(stream_desc, pic_desc)`，VENC 是 `(pic_desc, stream_desc)`
- VDEC 必须检查 `pic_desc` 的 `ret_code`
- VDEC 回调需要销毁 **两个** 描述符（VENC 只销毁输入）

ret_code 错误码

ret_code	含义
0	解码成功
非 0	解码失败——输入码流损坏、帧边界错误或参考帧不足

5.3.4. 练习脚本走读

完整代码见 `samples/chapter5/vdec/vdec_minimal.py`。程序使用原始 `acl.media` API 分为 5 个阶段：

(1) 生成测试码流

VDEC 需要合法的 H.264/H.265 输入，不能使用随机字节。最小样例先用 NumPy 生成同一帧 I420/YUV420P 测试图，再分别交给 `libx264` 和 `libx265` 生成 H.264 Baseline 与 H.265 Main 码流：

```
import av, fractions, numpy as np

ENTYPE_H265_MAIN = 0
ENTYPE_H264_BASE = 1

CODECS = [
```

```
    ("H.264 Baseline", "libx264", "31", ENTYPE_H264_BASE),
    ("H.265 Main", "libx265", "31", ENTYPE_H265_MAIN),
]

codec = av.CodecContext.create(ffmpeg_codec, "w")
codec.width, codec.height = 640, 480
codec.pix_fmt = "yuv420p"
# ... 设置 bit_rate, framerate, time_base, options ...
if ffmpeg_codec == "libx264":
    codec.profile = "Baseline"

frame = av.VideoFrame.from_ndarray(make_test_i420(640, 480), format="yuv420p")
stream = b"".join(bytes(pkt) for pkt in codec.encode(frame))
stream += b"".join(bytes(pkt) for pkt in codec.encode(None))
```

(2) 回调线程

与 VENC 结构相同，但**回调参数顺序相反**，且必须检查 `ret_code`。

(3) 创建通道

```
desc = media.vdec_create_channel_desc()
media.vdec_set_channel_desc_channel_id(desc, 0)   # ← VDEC 必须设置!
media.vdec_set_channel_desc_thread_id(desc, tid)
media.vdec_set_channel_desc_callback(desc, vdec_callback)
media.vdec_set_channel_desc_entype(desc, entype)   # H.264=1, H.265=0
media.vdec_set_channel_desc_out_pic_format(desc, PIX_FMT_NV12)
media.vdec_set_channel_desc_out_pic_width(desc, W)
media.vdec_set_channel_desc_out_pic_height(desc, H)

ret = media.vdec_create_channel(desc)
```

(4) 发送一帧解码

```
# 输入: H.264/H.265 码流
arr = np.frombuffer(stream, dtype=np.uint8)
in_buf, _ = media.dvpp_malloc(len(arr))
acl.rt.memcpy(in_buf, len(arr), arr.ctypes.data,
              len(arr), ACL_MEMCPY_HOST_TO_DEVICE)

stream_desc = media.dvpp_create_stream_desc()
media.dvpp_set_stream_desc_data(stream_desc, in_buf)
media.dvpp_set_stream_desc_size(stream_desc, len(arr))

# 输出: NV12 图片
out_size = W * H * 3 // 2
out_buf, _ = media.dvpp_malloc(out_size)
```

```
pic_desc = media.dvpp_create_pic_desc()
media.dvpp_set_pic_desc_data(pic_desc, out_buf)
media.dvpp_set_pic_desc_size(pic_desc, out_size)
media.dvpp_set_pic_desc_format(pic_desc, PIX_FMT_NV12)

ret = media.vdec_send_frame(desc, stream_desc, pic_desc, frame_cfg, None)
```

(5) 清理

与 VENC 相同——销毁通道、描述符、帧配置，释放 DVPP 内存。

附：encode_frames 参数详解

`bench_vdec.py` 中用于生成测试码流的函数，接收原始帧 + 编码器名 + GOP -> 码流 + 统计。

```
def encode_frames(frames: list[np.ndarray], codec_name: str, gop: int
                  ) -> tuple[list[bytes], int, int]:
    import av

    h, w = frames[0].shape[:2]
    level = "31" if w * h <= 1280 * 720 else "40"

    codec = av.CodecContext.create(codec_name, "w")    # (1)
    codec.width = w                                     # (2)
    codec.height = h
    codec.pix_fmt = "yuv420p"                           # (3)
    codec.bit_rate = max(2_000_000,                     # (4)
                         int(w * h * FPS * 0.1))
    codec.framerate = fractions.Fraction(FPS, 1)        # (5)
    codec.time_base = fractions.Fraction(1, FPS)        # (6)
    codec.options = {"level": level,                    # (7)
                     "tune": "zerolatency"}
    if codec_name == "libx265":
        codec.options["x265-params"] = "log-level=error"
    if codec_name == "libx264":
        codec.profile = "Baseline"                      # (8)

    for i, bgr in enumerate(frames):
        frame = av.VideoFrame.from_ndarray(
            bgr[..., ::-1], format="rgb24")             # (9)
        frame.pict_type = (                             # (10)
            av.video.frame.PictureType.I
            if i % gop == 0
            else av.video.frame.PictureType.P)
        data = bytearray()
        for pkt in codec.encode(frame):                 # (11)
            data += bytes(pkt)
```

```
        if data:
            streams.append(bytes(data))
    tail = bytearray()
    for pkt in codec.encode(None):                    # (12)
        tail += bytes(pkt)
    if tail:
        if streams:
            streams[-1] += bytes(tail)
        else:
            streams.append(bytes(tail))
    return streams, i_avg, p_avg
```

(1) `av.CodecContext.create(codec_name, "w")` 创建编码器实例。`codec_name = "libx264"` 或 `"libx265"`，`"w"` 表示编码模式。libx264 是 H.264 标准的开源软件实现，运行在 CPU 上。此处用它来**生成测试素材**——先软件编码再硬件解码，正好对应需要对比的两条路径。

(2) `codec.width / codec.height` 编码帧的尺寸（像素）。必须与实际输入的 numpy 数组尺寸一致。VDEC 解码时需传入相同尺寸的 `out_pic_width/height`。

(3) `codec.pix_fmt = "yuv420p"` 编码器**内部**使用的像素格式。libx264 只接受 YUV 格式，`"yuv420p"` 是 I420（YUV 4:2:0 planar）。PyAV 会自动将输入的 `rgb24` 帧转为 `yuv420p` 后再送给编码器。

(4) `codec.bit_rate = max(2_000_000, int(w * h * FPS * 0.1))` 目标码率，单位 **bps**。- `2_000_000` = 2 Mbps，是最低可用码率，确保小分辨率不会码率过低。- `w * h * FPS * 0.1` = 每像素每秒 0.1 bps，按分辨率和帧率缩放。例如 640×480×30×0.1 ~ 0.9 Mbps -> 取 2 Mbps；3840×2160×30×0.1 ~ 25 Mbps。- `max(...)` 设置 2 Mbps 码率下限。

码率影响帧大小：码率越高字节越多，但硬件解码速度主要和像素数相关，受码率影响很小。

(5) `codec.framerate` 目标帧率（fps）。设为 30 表示编码器按 30fps 场景分配比特预算。不改变实际编码速度，只影响码率控制算法。

(6) `codec.time_base` 时间基准，帧率的倒数。`Fraction(1, 30)` 表示每帧间隔 1/30 秒。影响编码后 Packet 的 `pts/dts` 时间戳。

(7) `codec.options` 传递给 libx264 编码器的底层选项：

- `"level"`：H.264 Level。<=720p 用 Level 3.1，>=1080p 用 Level 4.0。编码的码流中会嵌入 Level 标志，VDEC 据此分配解码缓冲区。
- `"tune": "zerolatency"`：低延迟调优。**禁用 B 帧**，编码器每收到一帧立即输出，不做多帧缓冲。实时通信必须开启，代价是牺牲约 10-15% 压缩率。

H.265 路径也使用 `zerolatency`，并通过 `x265-params=log-level=error` 关闭 x265 的信息日志。

(8) `codec.profile = "Baseline"` H.264 **档次**。三个常用档次：

- **Baseline**：最简，无 B 帧，适合实时通信和低功耗设备。

- **Main**：比 Baseline 多 B 帧，压缩率高 10-15%，适合广播电视。
- **High**：在 Main 基础上增加 8×8 变换和量化矩阵，适合高清蓝光。

Baseline 是 WebRTC 强制要求的最低档次，且 VDEC `entype=1` 正好对应 Baseline。注意：libx265 没有 `profile` 属性（H.265 的 profile 通过 options 设置），所以加了 `if` 判断。

(9) `av.VideoFrame.from_ndarray(bgr[..., ::-1], format="rgb24")` 将 numpy BGR 数组转为 PyAV `VideoFrame` 对象。`bgr[..., ::-1]` 把通道反转：`BGR -> RGB`。

(10) `frame.pict_type = I if i % gop == 0 else P` 按 GOP 间隔设置帧类型。GOP=30 时，帧 0、30、60... 为 I 帧，其余为 P 帧。这是模拟真实视频流的关键——97% 的帧是 P 帧，3% 是 I 帧。与旧版全 I 帧（GOP=1）相比，I 帧体积从 ~80KB 降至 ~25KB（480p），因为码率预算被 P 帧摊薄。

(11) `codec.encode(frame)` 将一帧送入编码器，返回编码后的 `Packet` 列表。每个 Packet 包含一个或多个 NAL 单元（SPS/PPS/SEI/IDR Slice 等）。

基准脚本会过滤空 packet，并按实际可用 packet 数量做 warmup 和计时；这也是 H.265 现在能正常出表的原因。

(12) `codec.encode(None)` **排空编码器缓冲区**。传入 `None` 强制输出所有剩余数据。排空后的尾 NAL 追加到最后一帧的码流末尾。

I 帧数量与解码性能

基准测试中的 I 帧数量

每 30 帧一个 I 帧。`bench_vdec.py` 使用 `TEST_GOP = 30`，90 帧测试中只有 **3 个 I 帧**（帧 0、30、60），其余 87 帧是 P 帧。这是模拟真实视频流的配置。

```
基准测试码流（GOP=30）：
  [IDR] [P] [P] ... [P] [IDR] [P] [P] ... [P] [IDR] [P] ...
   ↑ ~25KB@480p      ↑ ~7KB                      I 帧数 = 3/90 ~ 3%

全 I 帧码流（GOP=1，仅供参考）：
  [IDR] [IDR] [IDR] ... [IDR]    ← 所有帧都是关键帧
   ↑ ~80KB@480p 完全相同的大小
```

为什么用 GOP=30 而不是全 I 帧

1. **真实视频流就是这样的**：WebRTC、RTSP、监控摄像头通常使用 GOP=15~60。GOP=30 表示每秒一个关键帧（30fps 下），是实时通信的典型配置。
2. **避免误判 VDEC 性能**：全 I 帧测试中每帧都 ~80KB（480p），VDEC 的硬件并行优势被放大。而真实流中 97% 的帧是小 P 帧（~7KB），CPU 处理 P 帧极快（只需解运动矢量 + 残差），VDEC 的固定调度开销反而成了瓶颈。

3. **全 I 帧曾导致错误结论**：本教程早期版本用 GOP=1 测得 VDEC 在 720p 领先 CPU 10%、1080p 领先 43%。切换到 GOP=30 后，H.264 VDEC 在 <=1080p 全面落后于 CPU。**全 I 帧测试的不是真实场景，测试结果不可用于工程决策**。

I 帧比例对性能的颠覆性影响

测试模式	480p VDEC	480p CPU	1080p VDEC	1080p CPU	拐点
GOP=1（全 I 帧）	240 fps	410 fps	97 fps	68 fps	**720p**
GOP=30（I/P 混合）	**17 fps**	**1349 fps**	**17 fps**	**307 fps**	**2K**

GOP 的影响是巨大的：

- VDEC 在全 I 帧模式下 480p 跑 240fps，混合流下掉到 17fps（**14× 差距**）
- 原因：全 I 帧模式下每帧数据量大（~80KB），连续发送大包让 VDEC 硬件始终处于忙碌状态，调度开销被隐藏
- 混合流下每帧只有 ~8KB（P 帧），VDEC 处理太快反而暴露了 Python 回调调度的固定瓶颈

帧类型与性能特性

帧类型	每帧大小 (480p)	解码方式	VDEC 表现	CPU 表现
I 帧 (IDR)	~25 KB (GOP=30)	帧内解码（完整）	硬件并行优势	需完整逆变换
P 帧	~7 KB	帧间解码（运动矢量 + 残差）	**固定开销主导**	极快（数据量小）

在 GOP=30 混合流中，P 帧占 97%。P 帧数据量仅为 I 帧的 ~30%，CPU 解码 P 帧的开销远低于 I 帧（只需解残差），而 VDEC 每帧仍需经过完整的 `memcpy -> send -> callback -> memcpy -> Queue` 路径，这部分时间与帧大小关系不大。

如何切换测试模式

修改 `bench_vdec.py` 顶部的常量即可：

```
# 当前（GOP=30，I/P 混合——真实视频流）
TEST_GOP = 30

# 改为（GOP=1，全 I 帧——最坏情况/最大吞吐测试）
TEST_GOP = 1
```

`encode_frames()` 会根据 `gop` 参数自动设置每帧的 `pict_type`：`i % gop == 0` 时为 I 帧，否则为 P 帧。

5.3.5. 常见问题与调试

问题 #1：channel_id 未设置导致通道创建失败

现象：

```
vdec_create_channel failed: 507018
```

根因：VDEC 与 VENC 不同，**不会**自动分配 `channel_id`。必须显式调用 `vdec_set_channel_desc_channel_id(desc, N)`。

修复：

```
media.vdec_set_channel_desc_channel_id(desc, 0)
```

问题 #2：回调参数顺序与 VENC 相反

现象：在回调中调用 `dvpp_get_pic_desc_ret_code` 时进程异常终止或返回无效数据。

根因：VENC 回调是 `(input_pic_desc, output_stream_desc)`，VDEC 回调是 `(input_stream_desc, output_pic_desc)`。如果按 VENC 习惯写 VDEC 回调，会把 stream_desc 当成 pic_desc 来读。

记忆方法：**第一个参数总是"输入"，第二个参数总是"输出"**。VENC 输入是图片、输出是码流；VDEC 输入是码流、输出是图片。

问题 #3：未检查 ret_code 导致使用损坏帧

现象：解码后的 NV12 数据是花屏或全黑。

根因：`dvpp_get_pic_desc_ret_code()` 返回非 0 表示解码失败——可能是输入码流不完整、帧边界不对、参考帧丢失。直接使用失败帧的数据会得到损坏画面。

修复：

```
ret_code = media.dvpp_get_pic_desc_ret_code(pic_desc)
if ret_code != 0:
    cb_queue.put(None)  # 跳过此帧
    return
```

问题 #4：回调未销毁输出 pic_desc 导致内存泄漏

现象：解码多帧后，`dvpp_malloc` 返回内存不足。

根因：VDEC 回调负责销毁**两个**描述符（输入 stream_desc 和输出 pic_desc）。VENC 只需要销毁输入，因为输出的 stream_desc 由调用方管理。如果只销毁了 stream_desc，pic_desc 及其关联的 DVPP 内存永远不会释放。

修复：回调 `finally` 块中同时销毁两个：

```
finally:
    media.dvpp_destroy_stream_desc(input_stream_desc)
    media.dvpp_destroy_pic_desc(output_pic_desc)
```

问题 #5：输入必须用 numpy 包装才能 memcpy

现象：

```
TypeError: acl.rt.memcpy args parse failed
```

根因：`acl.rt.memcpy` 的源地址参数不接受 Python `bytes` 对象。必须将其包装为 numpy 数组或 ctypes 指针。

修复：

```
# 错误
h264 = b"..."
acl.rt.memcpy(in_buf, len(h264), h264, len(h264), 1)  # TypeError

# 正确
h264_arr = np.frombuffer(h264, dtype=np.uint8)
acl.rt.memcpy(in_buf, len(h264), h264_arr.ctypes.data, len(h264), 1)
```

问题 #6：vdec_destroy_channel 顺序错误导致阻塞

现象：解码多帧后，`media.vdec_destroy_channel(desc)` 永远阻塞，程序卡死。

根因：VDEC 通道销毁时，回调线程必须在 `vdec_destroy_channel` 调用期间保持运行——驱动需要在销毁过程中通过回调发送最后的清理通知。如果先停止了回调线程，`vdec_destroy_channel` 会无限等待一个永远不会来的回调。

VDEC 的正确销毁顺序与直觉相反：

```
# 错误：先停线程再销毁通道 -> 永远阻塞
running[0] = False                        # 通知线程停止
media.vdec_destroy_frame_config(fcfg)     # 销毁帧配置
media.vdec_destroy_channel(desc)          # 销毁通道 -> 等待回调 -> 永远阻塞!

```

```
# 正确：先销毁通道（需要线程活着），再停线程
media.vdec_destroy_channel(desc)          # (1) 先销毁通道（此时线程必须活着）
running[0] = False                        # (2) 通知线程停止
acl.util.stop_thread(tid)                 # (3) 停止回调线程
media.vdec_destroy_frame_config(fcfg)     # (4) 最后销毁帧配置
media.vdec_destroy_channel_desc(desc)     # (5) 销毁描述符
```

这一步的关键点是销毁通道时回调线程必须仍在运行，否则驱动侧的清理事件无法被处理。

问题 #7：VDEC 通道复用对码流连续性非常敏感

现象：通道创建后第一帧解码正常，第二帧 `vdec_send_frame` 返回 0 但回调永不触发。

根因：VDEC 期望同一通道上解码的帧来自**同一个编码器实例**，并且按连续视频流顺序送入。也就是所有帧需要共享兼容的 SPS/PPS（序列参数集/图像参数集），不能把多个互不相关的 IDR 样本直接拼到同一通道里混跑。

验证方法：用单个 libx264 `CodecContext` 连续编码所有测试帧，确保 SPS/PPS 一致，并按同一序列顺序逐帧送入。`samples/chapter5/vdec/bench_vdec.py` 已新增 “单通道复用” 路径，并自动尝试不同 `pipeline_depth` 与 `frame_config` 策略；在当前 310B 环境下，只有带显式 `EOS` 的 `depth=4` 变体能够稳定排空尾部缓存帧。

当前结论：Ascend 310B4 上不能假设 “任意 H.264 样本都能安全复用同一通道”。应先用连续码流验证复用模式，并显式处理解码器尾部 flush；如果复用路径不稳定，再退回每帧独立通道作为兜底方案。独立通道虽然可靠，但固定创建/销毁开销在 640×480 下通常远高于纯 CPU 解码。

5.3.6. 性能实测

在 Orange Pi AI Pro（Ascend 310B4）上，使用 `bench_vdec.py` 进行 H.264/H.265 分辨率扫描。测试参数：GOP=30（I/P 混合，模拟真实视频流），90 帧（3 个完整 GOP），固定种子可复现。CPU 解码对比单线程（`thread_count=1`）和多线程（`thread_count=0`，自动使用所有核心）两种模式。脚本生成 H.265 测试码流时使用 `libx265`，并过滤编码器可能返回的空 packet，避免 CPU HEVC 解码路径把空 packet 当作 EOF。当前脚本输出已简化为帧率表：不再打印加速比、单帧耗时、优胜结论和码流大小。

H.264 Baseline

分辨率	VDEC帧率	CPU多线程	CPU单线程
640x480	16.9	1485.9	612.0
1280x720	16.8	684.6	272.2

```
1920x1080            16.5          305.5          119.8
2560x1440           133.6          189.1           75.9
3840x2160            71.2           88.5           35.1
```

H.265 Main

```
分辨率             VDEC帧率       CPU多线程       CPU单线程
──────────────────────────────────────────────────────
640x480             275.4          324.9          185.6
1280x720            250.2          182.2           97.7
1920x1080           194.6           82.5           42.3
2560x1440           135.3           46.6           23.4
3840x2160            71.7           21.2            9.9
```

VDEC 单帧耗时（由帧率反推）

简化后的 `bench_vdec.py` 只打印帧率，不再打印耗时分解。若按 `1000 / fps` 反推，H.264 低分辨率路径仍能看到明显的固定开销：

编码	640×480	1280×720	1920×1080	2560×1440	3840×2160
H.264 VDEC	59.2ms	59.5ms	60.6ms	7.5ms	14.0ms
H.265 VDEC	3.6ms	4.0ms	5.1ms	7.4ms	13.9ms

> **关键结论**：H.264 <=1080p 时，VDEC 路径稳定在约 60ms/帧，说明瓶颈主要不是像素计算量。当前 Python 基准路径每帧都经过 DVPP 内存分配、回调线程、Queue 等同步环节，这些固定开销在小分辨率下很难被硬件解码收益摊薄。
>
> 2K 以上分辨率时，H.264 VDEC 吞吐明显回升；H.265 路径则从 480p 起就没有出现同样的 60ms 固定开销。

拐点分析

H.264 Baseline

分辨率	VDEC fps	CPU 单线程	CPU 多线程	vs 单线程	vs 多线程
640×480	17	612	1486	0.03x	0.01x
1280×720	17	272	685	0.06x	0.02x
1920×1080	17	120	306	0.14x	0.05x
2560×1440	134	76	189	**1.76x**	0.71x
3840×2160	71	35	89	**2.03x**	0.80x

H.265 Main

分辨率	VDEC fps	CPU 单线程	CPU 多线程	vs 单线程	vs 多线程
640×480	275	186	325	**1.48x**	0.85x
1280×720	250	98	182	**2.56x**	**1.37x**
1920×1080	195	42	83	**4.60x**	**2.36x**
2560×1440	135	23	47	**5.78x**	**2.90x**
3840×2160	72	10	21	**7.24x**	**3.38x**

H.264 路径下, VDEC 在 <=1080p 完全不具备竞争力（仅为 CPU 单线程的 3%~14%）。真正的单线程交叉点在 **2K（2560×1440）**：2K 时 VDEC 比单线程 CPU 快 76%，4K 时快 103%。但在这个单路测试中，CPU 多线程在 H.264 全部分辨率上仍快于 VDEC。

H.265 路径不同：VDEC 在 480p 已经领先单线程 CPU，在 720p 以上同时领先单线程和多线程 CPU。HEVC 软件解码在 ARM CPU 上明显更重，因此 VDEC 的收益比 H.264 更早出现。

关键发现

1. **H.264 低分辨率 VDEC 有固定调度开销**——480p/720p/1080p 下 VDEC 都是 ~59ms/帧，与分辨率无关。这说明瓶颈不在硬件解码，而在 Python 回调-> 队列-> 主线程的调度路径。
2. **H.264 高分辨率开始领先单线程 CPU**——2K 以上分辨率，固定调度开销被摊薄, VDEC 开始领先单线程 CPU。2K 时领先 1.76×，4K 时领先 2.03×；但单路场景下仍慢于 CPU 多线程。
3. **H.265 是 VDEC 的优势路径**——720p 以上 H.265 VDEC 同时领先单线程和多线程 CPU，4K 时对多线程 CPU 也有 3.38× 加速。
4. **I/P 混合流比全 I 帧流更不利于 H.264 VDEC**——详见上文 I 帧数量与解码性能 的对比分析。全 I 帧测试曾误导结论，GOP=30 才是真实场景。
5. **零拷贝管道可减少主机侧开销**——当前基准实现每次解码都经过设备内存分配、D2H `memcpy` 和 Python Queue。如果 VDEC 输出直接喂给 VENC（设备内存零拷贝），可以避免把 NV12 帧拷回主机再送回设备。

适用场景速查

分辨率	单路实时	多路并发（>=4 路）	转码管道
H.264 <= 1080p	CPU 推荐	CPU 可选	CPU 推荐
H.264 2K/4K	CPU 多线程优先, CPU 资源紧张时可选 VDEC	VDEC 可选	VDEC 可选

分辨率	单路实时	多路并发（>=4 路）	转码管道
H.265 480p	CPU 多线程优先，VDEC 可选	CPU 可选	VDEC 可选
H.265 >= 720p	**VDEC 推荐**	**VDEC 推荐**	**VDEC 推荐**

5.3.7. VDEC 调试与参数速查

常用调试命令

```
# VDEC 驱动状态
lsmod | grep vdec

# VDEC 内核日志
dmesg | grep -i vdec | tail -10

# 运行练习脚本
python samples/chapter5/vdec/vdec_minimal.py
```

参数速查表

VDEC 通道参数

参数	函数	示例
通道 ID	`vdec_set_channel_desc_channel_id`	0
编码类型	`vdec_set_channel_desc_entype`	1
输出像素格式	`vdec_set_channel_desc_out_pic_format`	1
输出宽度	`vdec_set_channel_desc_out_pic_width`	640
输出高度	`vdec_set_channel_desc_out_pic_height`	480
输出模式	`vdec_set_channel_desc_out_mode`	0
参考帧数	`vdec_set_channel_desc_ref_frame_num`	5
位深	`vdec_set_channel_desc_bit_depth`	8

编码类型：0 = H.265 Main, 1 = H.264 Baseline, 2 = H.264 Main, 3 = H.264 High

---

5.4. VPC — 硬件图像处理

VPC（Video Pre-Processing Core）提供硬件加速的缩放、裁剪、色彩空间转换。前置基础：DVPP 基础概念。

5.4.1. VPC 简介

VPC（Video Pre-Processing Core）是 DVPP 中的硬件图像处理模块。它提供三大功能：

功能	说明	典型场景
Resize	图像缩放（多种插值算法）	1080p -> 720p 下采样
Crop	按指定区域裁剪	从画面中提取 ROI 区域
CSC	色彩空间转换	YUYV->NV12、NV12->RGB 等

VPC 在 DVPP 管道中的典型位置：

```
USB Camera YUYV -> [VPC CSC] -> NV12 -> [VENC] -> H.264 -> WebRTC
                       ↑                   ↑
                   硬件色彩转换          硬件编码
                   (零CPU占用)          (零CPU占用)
```

为什么用 VPC 而不是 OpenCV

在 Orange Pi AI Pro 上，CPU 做 `cv2.cvtColor(YUYV->BGR)` + `cv2.resize(1080p->720p)` + `bgr_to_nv12()` 会消耗 ARM Cortex-A55 的宝贵算力。VPC 完全在硬件中完成这些操作，CPU 只负责下发任务和等待完成。

硬件能力规格（Ascend 310B4）

约束项	值
输入分辨率范围	10×6 ~ 8192×8192
输出分辨率范围	10×6 ~ 4096×8192
宽度对齐	2（VPC 自动向下对齐）
高度对齐	2
支持的输入格式	NV12、NV21、YUV400、YUV422、YUV444
支持的输出格式	NV12、NV21、YUV400、YUV422、YUV444

5.4.2. VPC 与 VENC/VDEC 的关键区别

VPC 使用的是 **通用 DVPP 通道模型**，与 VENC/VDEC 的专用通道有本质差异：

维度	VENC / VDEC	VPC
通道创建	`venc_create_channel()` / `vdec_create_channel()`	`dvpp_create_channel()`（无需设置 mode）
异步机制	**回调线程**（需 `process_report` 轮询）	**Stream 同步**（`synchronize_stream` 等待）
输入类型	VENC: pic_desc / VDEC: stream_desc	pic_desc（始终是图像）
输出类型	VENC: stream_desc / VDEC: pic_desc	pic_desc（始终是图像）
回调函数	必须设置	**不需要**
回调线程	必须启动	**不需要**
复杂度	高（多线程协调）	低（同步等待即可）

> **310B 特别注意**：`dvpp_vpc_convert_color_async`（CSC 色彩空间转换）在 310B 上 **返回 ACL_ERROR_INVALID_PARAM**，不可用。`himpi.vpc_convert_color` 需要预创建 himpi 通道（Python 接口受限）。310B 上的 YUYV->NV12 转换目前只能走 CPU 路径。

通道模型对比

```
VENC/VDEC 专用通道:                VPC 通用通道:

主线程: send_frame -> Queue.get   主线程: vpc_xxx_async -> synchronize_stream
             ↑                                    ↑
回调线程: process_report            (无需回调线程)
          -> callback
          -> Queue.put
```

VPC 的 Stream 模型更简单——下发异步任务后直接阻塞等待完成，不需要管理额外的线程和队列。

5.4.3. VPC API 详解

通用 DVPP 通道创建

VPC 使用通用的 `dvpp_create_channel()`，与 VENC/VDEC 的专用 API 不同。无需设置 mode（310B 不支持 `DVPP_CHANNEL_MODE` 常量），也无需创建回调线程：

```
import acl

# 创建通用 DVPP 通道
dvpp_channel_desc = acl.media.dvpp_create_channel_desc()
ret = acl.media.dvpp_create_channel(dvpp_channel_desc)

# 创建 Stream 用于同步等待
stream, ret = acl.rt.create_stream()
```

Resize — 缩放

将输入图片缩放到输出尺寸：

```
# 创建 resize 配置
resize_config = acl.media.dvpp_create_resize_config()
# 可选：设置插值算法
# acl.media.dvpp_set_resize_config_interpolation(resize_config, 0)  # 0=bilinear

ret = acl.media.dvpp_vpc_resize_async(
    dvpp_channel_desc,    # DVPP 通道描述符
    input_pic_desc,       # 输入图片描述符（pic_desc）
    output_pic_desc,      # 输出图片描述符（pic_desc）
    resize_config,        # 缩放配置
    stream                # Stream 对象
)
acl.rt.synchronize_stream(stream)  # 等待完成
```

关键约束：

- 输入/输出 pic_desc 的宽高必须对齐到 2（VPC 自动处理）
- resize 不会改变像素格式——输入 NV12 则输出也是 NV12
- 如需同时改变格式和尺寸，需串联 CSC + Resize

Crop — 裁剪

从输入图片中裁剪指定区域：

```
# 创建 ROI 配置：从 (100, 100) 开始裁剪 224×224
# dvpp_create_roi_config 接受 4 个位置参数：(left, right, top, bottom)
crop_area = acl.media.dvpp_create_roi_config(
    100,    # 左边界偏移
```

```
    323,    # 右边界偏移 (左+宽-1)
    100,    # 上边界偏移
    323,    # 下边界偏移 (上+高-1)
)

ret = acl.media.dvpp_vpc_crop_async(
    dvpp_channel_desc, input_pic_desc, output_pic_desc, crop_area, stream)
acl.rt.synchronize_stream(stream)
```

Crop + Resize — 最常用的组合

裁剪后缩放到目标尺寸，一次调用完成两个操作：

```
# 从 1080p 画面中裁剪中心区域，缩放到 720×480
# 注意：dvpp_create_roi_config 接受 4 个位置参数 (left, right, top, bottom)
crop_area = acl.media.dvpp_create_roi_config(160, 480, 120, 360)

resize_config = acl.media.dvpp_create_resize_config()

ret = acl.media.dvpp_vpc_crop_resize_async(
    dvpp_channel_desc,
    input_pic_desc,       # 1920×1080
    output_pic_desc,      # 720×480
    crop_area,
    resize_config,
    stream
)
acl.rt.synchronize_stream(stream)
```

这是 VPC 最高效的使用方式——crop + resize 在一个硬件调用中完成，避免中间缓冲区。

CSC — 色彩空间转换（310B 不可用）

结论：310B (CANN 8.3.RC1) 上，`dvpp_vpc_convert_color_async` 返回 `ACL_ERROR_INVALID_PARAM` (100000)，`himpi.vpc_convert_color` 需要 himpi 通道预配置（Python 接口不支持）。CSC 在 310B 上**目前不可用**。

替代方案：YUYV->NV12 转换使用 CPU（`cv2.cvtColor + bgr_to_nv12`），resize 可卸载到 VPC：

```
USB Camera YUYV -> CPU bgr_to_nv12() -> NV12 -> VPC resize -> NV12(720p) -> VENC
                                          ↑                      ↑
                                    CPU 做色彩转换          VPC 硬件缩放
```

如果未来 CANN 版本在 310B 上开放 CSC 支持，可通过 `dvpp_vpc_convert_color_async` 使用与 resize 完全相同的通道 +Stream 模式调用。

5.4.4. 练习脚本走读

完整代码见 `samples/chapter5/vpc/vpc_minimal.py`。程序使用原始 `acl.media` API 演示两个核心操作：

(1) Resize — 硬件缩放

```
# 创建 640×480 测试 NV12 帧
src_nv12 = make_test_nv12(640, 480)

# 计算 stride 对齐的内存大小
sw = ((640 + 15) // 16) * 16    # width stride 对齐到 16
sh = ((480 + 1) // 2) * 2       # height stride 对齐到 2
in_size = sw * sh * 3 // 2

# 目标 320×240
out_sw = ((320 + 15) // 16) * 16
out_sh = ((240 + 1) // 2) * 2
out_size = out_sw * out_sh * 3 // 2

# 分配设备内存 + 拷贝输入数据
in_buf, _ = media.dvpp_malloc(in_size)
out_buf, _ = media.dvpp_malloc(out_size)
acl.rt.memcpy(in_buf, src_nv12.nbytes, src_nv12.ctypes.data,
              src_nv12.nbytes, ACL_MEMCPY_HOST_TO_DEVICE)

# 构造输入/输出 pic_desc（含 height_stride）
in_pic = create_nv12_pic_desc(in_buf, 640, 480, sw, sh)
out_pic = create_nv12_pic_desc(out_buf, 320, 240, out_sw, out_sh)

# 执行缩放 + 同步等待
resize_cfg = media.dvpp_create_resize_config()
media.dvpp_vpc_resize_async(ch_desc, in_pic, out_pic, resize_cfg, stream)
acl.rt.synchronize_stream(stream)

# 拷回主机内存
host_buf = np.zeros(out_size, dtype=np.uint8)
acl.rt.memcpy(host_buf.ctypes.data, out_size, out_buf, out_size,
              ACL_MEMCPY_DEVICE_TO_HOST)
```

(2) Crop + Resize — 裁剪后缩放

```
# 从 640×480 中裁剪中心 320×240，再缩放到 640×480
crop_area = acl.media.dvpp_create_roi_config()
acl.media.dvpp_set_roi_config(crop_area,
    left=160, right=480, top=120, bottom=360)    # 中心 320×240

resize_cfg = acl.media.dvpp_create_resize_config()
```

```
acl.media.dvpp_vpc_crop_resize_async(
    ch_desc, in_pic_desc, out_pic_desc, crop_area, resize_cfg, stream)
acl.rt.synchronize_stream(stream)
```

(3) CSC — 310B 不可用

310B 上 CSC 需走 CPU。详见 CSC 限制。

清理

```
# 释放 VPC 资源
acl.media.dvpp_destroy_resize_config(resize_cfg)
acl.media.dvpp_destroy_roi_config(crop_area)
acl.media.dvpp_destroy_pic_desc(in_pic)
acl.media.dvpp_destroy_pic_desc(out_pic)
acl.media.dvpp_free(in_buf)
acl.media.dvpp_free(out_buf)
acl.media.dvpp_destroy_channel(ch_desc)
acl.media.dvpp_destroy_channel_desc(ch_desc)
acl.rt.destroy_stream(stream)
```

与项目的对应关系

文件	角色
`vpc_minimal.py`	学习用途——直接调用 `acl.media` API，理解底层机制
`bench_vpc.py`	基准测试——VPC resize vs CPU cv2.resize 性能对比

5.4.5. 性能基准

以下数据使用 `bench_vpc.py` 在 Orange Pi AI Pro（Ascend 310B4, CANN 8.3.RC1）上实测。

测试条件：NV12 -> ½ 缩放（各分辨率缩小一半），60 帧，确定性测试帧，固定种子 42。

VPC Resize: NV12 to 1/2 Scale（数据见下方结果解读表格）

结果解读

VPC Resize 与 VENC 不同——**不是所有分辨率都远超 CPU**。VPC 与 CPU 在低分辨率下互有胜负（720p VPC 赢，1080p CPU 赢），这是 Stream 同步的固定开销与硬件加速之间的拉锯：

分辨率	像素数	VPC fps	CPU fps	加速比	推荐
640×480	0.3M	**761**	**1521**	**0.50x**	CPU
1280×720	0.9M	**679**	**573**	**1.19x**	VPC
1920×1080	2.1M	**273**	**290**	**0.94x**	CPU
2560×1440	3.7M	**228**	**144**	**1.59x**	**VPC**
3840×2160	8.3M	**110**	**53**	**2.07x**	**VPC**

从 2K（2560×1440）开始 VPC 稳定领先（1.6×~2.1×）。低分辨率下互有胜负——VPC 的固定调度开销（dvpp_malloc + memcpy + Stream 同步）在小帧上无法稳定被硬件加速摊薄，720p 虽 VPC 略快但优势微弱（1.19×），1080p 反而 CPU 略快（0.94×）。

VPC vs VENC vs VDEC 性能模式对比

模块	低分辨率（<=1080p）	高分辨率（>=2K）	瓶颈
VENC	领先 CPU 多线程（H.264 约 1.9×~3.6×，H.265 约 5.3×~7.0×）	领先 CPU 多线程（H.264 约 1.9×~2.2×，H.265 约 5.0×~5.9×）	纯硬件编码计算
VDEC	落后 CPU（~59ms 固定开销）	领先 CPU（1.9×~2.2×）	Python 回调调度
VPC	与 CPU 互有胜负（0.5×~1.2×）	领先 CPU（1.6×~2.1×）	Stream 同步开销

> VPC 的固定开销比 VDEC 小得多（Stream 同步比回调线程轻量），因此即使在 720p 也能与 CPU 抗衡，而 VDEC 要到 2K 以上才有竞争力。

CSC 性能（310B 不支持，仅供参考）

310B 上 CSC（YUYV->NV12）必须走 CPU：`cv2.cvtColor(YUYV->BGR) + bgr_to_nv12()`。VPC 只能卸载 resize，CPU 仅做色彩转换。

5.4.6. 常见问题与调试

问题 #1：310B 不支持 dvpp_vpc_convert_color_async

现象：`dvpp_vpc_convert_color_async` 返回 `100000`（ACL_ERROR_INVALID_PARAM）。

根因：该接口仅支持 310P 及以上型号。310B（Atlas 200I A2）的 CANN 8.3.RC1 版本不支持。

修复：无 VPC 硬件替代方案。YUYV->NV12 使用 CPU：`cv2.cvtColor(YUYV->BGR) + bgr_to_nv12()`。VPC 仅卸载 resize 部分。

问题 #2：pic_desc 必须设置 height_stride

现象：`dvpp_vpc_resize_async` 返回 `100000`（ACL_ERROR_INVALID_PARAM）。

根因：`dvpp_set_pic_desc_height_stride` 未设置，且 `dvpp_set_pic_desc_size` 使用了未对齐的 `W*H*3//2`。VPC 要求高度 stride 对齐到 2，宽度 stride 对齐到 16，缓冲区大小按 stride 计算。

修复：

```
sw = ((w + 15) // 16) * 16   # width stride
sh = ((h + 1) // 2) * 2     # height stride
size = sw * sh * 3 // 2      # 按 stride 算总大小
media.dvpp_set_pic_desc_width_stride(pic, sw)
media.dvpp_set_pic_desc_height_stride(pic, sh)
media.dvpp_set_pic_desc_size(pic, size)
```

问题 #3：dvpp_create_roi_config 不接受 keyword 参数

现象：`dvpp_create_roi_config()` + `dvpp_set_roi_config()` 不存在或报错。

根因：`dvpp_set_roi_config` 在 CANN 8.3.RC1 的 Python API 中不接受 keyword 参数。正确做法是在 `dvpp_create_roi_config()` 中直接传入 4 个位置参数。

修复：

```
# 正确
crop_area = acl.media.dvpp_create_roi_config(left, right, top, bottom)

# 错误
crop_area = acl.media.dvpp_create_roi_config()
acl.media.dvpp_set_roi_config(crop_area, left, right, top, bottom)
```

问题 #4：himpi 通道不可从 Python 创建

现象：`himpi.vpc_create_chn()` 无论传什么参数都报 “args parse failed”。

根因：himpi 接口是 C 扩展的直接映射，需要 C 结构体类型的参数，Python 侧不支持创建这些结构体。`vpc_convert_color` 虽然语法上可以调用，但缺少预配置的 himpi 通道，返回硬件错误 `0xa0078003`。

当前结论：310B 上的 VPC CSC 不可用。等待 CANN 后续版本在 310B 上开放 `dvpp_vpc_convert_color_async` 支持。

5.4.7. 场景推荐

场景决策树

需要对图像做预处理?

- **仅缩放**（NV12 in -> NV12 out）-> `dvpp_vpc_resize_async`（简单，Stream 同步）
- **仅裁剪** -> `dvpp_vpc_crop_async`
- **裁剪 + 缩放** -> `dvpp_vpc_crop_resize_async`（推荐，一次调用）
- **仅色彩转换**（YUYV -> NV12）-> 310B 不支持，使用 CPU `cv2.cvtColor` + `bgr_to_nv12`
- **色彩转换 + 缩放**（摄像头场景）:
  1. CPU `bgr_to_nv12`（YUYV->NV12）
  2. VPC `dvpp_vpc_resize_async`（1080p->720p）
- **小型图像、非实时** -> CPU (cv2) 足够

典型场景

场景	推荐方案	理由
USB 摄像头 -> WebRTC	CPU CSC + VPC resize	CSC 走 CPU，resize 卸载到 VPC
视频文件预处理	dvpp_vpc_crop_resize_async	单次调用完成裁剪 + 缩放
AI 推理前处理	VPC resize + AIPP CSC	Resize->AI Core 直接推理
多路视频并发（>4 路）	**VPC 必须**	CPU 做多路 resize 会占满所有核

5.4.8. VPC 调试与参数速查

常用调试命令

```
# VPC 驱动状态
lsmod | grep vpc

# VPC 内核日志
dmesg | grep -i vpc | tail -10

# 运行示例脚本
python samples/chapter5/vpc/vpc_minimal.py
```

```

# 运行基准测试
python samples/chapter5/vpc/bench_vpc.py
```

参数速查表

VPC 通道创建

参数	API	值
通道描述符	`dvpp_create_channel_desc()`	—
创建通道	`dvpp_create_channel(ch_desc)`	ret=0
创建 Stream	`acl.rt.create_stream()`	ret=0

VPC 操作 API

操作	API	310B 支持
缩放	`dvpp_vpc_resize_async`	支持
裁剪	`dvpp_vpc_crop_async`	支持
裁剪 + 缩放	`dvpp_vpc_crop_resize_async`	支持
色域转换	`dvpp_vpc_convert_color_async`	不支持

像素格式（`dvpp_set_pic_desc_format`）：1 = NV12 (YUV420SP), 7 = YUYV (YUV422 interleaved), 12 = RGB888, 13 = BGR888

5.5. JPEG — 硬件编解码

JPEGE / JPEGD 使用与 VPC 相同的 Stream 同步模型。前置基础：DVPP 基础概念。

5.5.1. JPEG 编解码简介

DVPP 的 JPEG 编解码子模块：

```
JPEGE (JPEG Encoder):  NV12 -> [硬件编码] -> JPEG 码流
JPEGD (JPEG Decoder):  JPEG 码流 -> [硬件解码] -> NV12
```

两者串联形成硬件闭环，可验证编解码无损性（测试常用模式）。

与 VENC 编码的区别

VENC 输出 H.264 码流，但 JPEGE 输出的是**独立的 JPEG 图片**——两者有本质不同：

	JPEGE	VENC
输出格式	JPEG 单帧图片	H.264 / H.265 视频码流
帧间关系	无（每帧独立）	有（I/P/B 帧依赖）
输出描述符	**裸内存缓冲区** + size 指针	**stream_desc**（码流描述符）
编码参数	quality（1-100）	GOP、码率、帧率、profile
用途	截图、快照、缩略图	实时视频传输

硬件能力规格（Ascend 310B4）

	JPEGE	JPEGD
输入格式	NV12 (YUV420SP)、YUV422SP	JPEG 码流（Baseline）
输出格式	JPEG 码流	NV12 (YUV420SP)
分辨率范围	32×32 ~ 8192×8192	32×32 ~ 8192×8192
质量范围	1-100	—
编码吞吐	1080p@256fps	1080p@512fps

5.5.2. 与 VENC/VDEC/VPC 的关键区别

JPEGE/JPEGD 使用与 VPC 相同的**通用 dvpp 通道 + Stream 同步**模型：

维度	VENC/VDEC	JPEGE/JPEGD
通道创建	`venc/vdec_create_channel()`	`dvpp_create_channel()`（无需 mode）
异步机制	回调线程	Stream 同步
输入描述	图片或码流描述符	**裸内存指针 + size**
输出描述	图片或码流描述符	JPEGE: **裸内存 + size 指针**；JPEGD: pic_desc

> JPEGE 的输出不是 `stream_desc`——这是一个常见误区。JPEG 码流直接写入 `dvpp_malloc` 分配的缓冲区，通过 `numpy_to_ptr` 封装的 size 指针返回实际大小。

5.5.3. JPEGE API 详解

编码流程

```
(1) 创建 jpege_config + 设置质量 -> (2) predict_enc_size 预测输出大小
    -> (3) dvpp_malloc 输出缓冲区 -> (4) jpeg_encode_async 异步编码
    -> (5) synchronize_stream 等待 -> (6) 读取实际 size -> (7) memcpy 取回 JPEG 码流
```

jpege_config — 编码参数

```
jpege_cfg = acl.media.dvpp_create_jpege_config()
acl.media.dvpp_set_jpege_config_level(jpege_cfg, quality)  # quality: 1-100
```

唯一参数是 **quality**（1-100），对应 JPEG 压缩质量。值越大画质越好、文件越大。

predict_enc_size — 预测输出大小

```
max_size, ret = acl.media.dvpp_jpeg_predict_enc_size(input_pic_desc, jpege_cfg)
```

返回编码后 JPEG 码流的**最大可能大小**（通常远大于实际值）。输出缓冲区需按此值分配。

jpeg_encode_async — 执行编码

```
import numpy as np

# in/out 参数：传入 max_size，编码后返回实际大小
out_size_arr = np.array([max_size], dtype=np.int32)
if "bytes_to_ptr" in dir(acl.util):
    out_size_ptr = acl.util.bytes_to_ptr(out_size_arr.tobytes())
else:
    out_size_ptr = acl.util.numpy_to_ptr(out_size_arr)

ret = acl.media.dvpp_jpeg_encode_async(
    channel_desc,       # dvpp 通道描述符
    input_pic_desc,     # NV12 输入图片描述符
    output_buffer,      # 输出缓冲区（dvpp_malloc）
    out_size_ptr,       # in/out: [max_size] -> [actual_size]
    jpege_cfg,          # 编码配置
    stream              # Stream 对象
)
acl.rt.synchronize_stream(stream)

# 同步后读取实际编码大小
actual_size = int(out_size_arr[0])
```

关键点：`out_size_ptr` 是 Python 层用 `numpy_to_ptr` 封装的指针，指向一个 `int32` 数组。编码器写入实际大小后，同步完成即可读取。

完整编码示例

```
# 准备输入 NV12 pic_desc（略）

jpege_cfg = media.dvpp_create_jpege_config()
media.dvpp_set_jpege_config_level(jpege_cfg, 85)

max_size, _ = media.dvpp_jpeg_predict_enc_size(in_pic, jpege_cfg)
out_buf, _ = media.dvpp_malloc(max_size)

out_size_arr = np.array([max_size], dtype=np.int32)
out_size_ptr = acl.util.numpy_to_ptr(out_size_arr)

media.dvpp_jpeg_encode_async(ch_desc, in_pic, out_buf,
                             out_size_ptr, jpege_cfg, stream)
acl.rt.synchronize_stream(stream)

jpeg_size = int(out_size_arr[0])
jpeg_host = np.zeros(jpeg_size, dtype=np.uint8)
acl.rt.memcpy(jpeg_host.ctypes.data, jpeg_size, out_buf, jpeg_size,
              ACL_MEMCPY_DEVICE_TO_HOST)
# jpeg_host 即为 JPEG 码流，可直接写入 .jpg 文件
```

5.5.4. JPEGD API 详解

解码流程

```
(1) JPEG 数据拷贝到设备 -> (2) get_image_info 获取宽高
    -> (3) predict_dec_size 预测输出大小 -> (4) dvpp_malloc + 创建 pic_desc
    -> (5) jpeg_decode_async 异步解码 -> (6) synchronize_stream -> (7) memcpy 取回 NV12
```

get_image_info — 获取 JPEG 信息

```
img_w, img_h, img_fmt, ret = acl.media.dvpp_jpeg_get_image_info(
    jpeg_dev_ptr, jpeg_size)
```

解码前必须调用此函数获取 JPEG 图像的宽度和高度，用于创建输出 pic_desc。

predict_dec_size — 预测输出大小

```
out_size, ret = acl.media.dvpp_jpeg_predict_dec_size(
    jpeg_dev_ptr, jpeg_size, PIX_FMT_NV12)
```

返回解码后 NV12 缓冲区的所需大小。

jpeg_decode_async — 执行解码

```
ret = acl.media.dvpp_jpeg_decode_async(
    channel_desc,       # dvpp 通道描述符
    jpeg_dev_ptr,       # JPEG 数据指针（设备内存）
    jpeg_size,          # JPEG 数据大小
    output_pic_desc,    # 输出 NV12 pic_desc
    stream              # Stream 对象
)
acl.rt.synchronize_stream(stream)
```

完整解码示例

```
# 拷贝 JPEG 到设备内存
jpeg_dev, _ = media.dvpp_malloc(len(jpeg_data))
acl.rt.memcpy(jpeg_dev, len(jpeg_data), jpeg_data.ctypes.data,
              len(jpeg_data), ACL_MEMCPY_HOST_TO_DEVICE)

# 获取图像信息
w, h, fmt, _ = media.dvpp_jpeg_get_image_info(jpeg_dev, len(jpeg_data))

# 预测 + 解码
out_size, _ = media.dvpp_jpeg_predict_dec_size(jpeg_dev, len(jpeg_data), PIX_FMT_NV12)
out_buf, _ = media.dvpp_malloc(out_size)
out_pic = create_nv12_pic_desc(out_buf, w, h)  # 见 vpc_guide

media.dvpp_jpeg_decode_async(ch_desc, jpeg_dev, len(jpeg_data), out_pic, stream)
acl.rt.synchronize_stream(stream)

# 取回 NV12
nv12_host = np.zeros(out_size, dtype=np.uint8)
acl.rt.memcpy(nv12_host.ctypes.data, out_size, out_buf, out_size,
              ACL_MEMCPY_DEVICE_TO_HOST)
```

5.5.5. 练习脚本走读

完整代码见 `samples/chapter5/jpeg/jpeg_minimal.py`。程序使用原始 `acl.media` API 演示 **JPEGE -> JPEGD 闭环**。

(1) JPEGE — 编码一帧

```
# 创建 JPEGE 配置
```

```
jpege_cfg = media.dvpp_create_jpege_config()
media.dvpp_set_jpege_config_level(jpege_cfg, 90)

# 预测编码后最大大小
max_size, _ = media.dvpp_jpeg_predict_enc_size(in_pic, jpege_cfg)

# 分配输出缓冲区
out_buf, _ = media.dvpp_malloc(max_size)

# in/out 参数：传 max_size，同步后读 actual_size
out_size_arr = np.array([max_size], dtype=np.int32)
out_size_ptr = acl.util.numpy_to_ptr(out_size_arr)

media.dvpp_jpeg_encode_async(ch_desc, in_pic, out_buf,
                             out_size_ptr, jpege_cfg, stream)
acl.rt.synchronize_stream(stream)

jpeg_actual = int(out_size_arr[0])  # ← 同步后才能读
```

(2) JPEGD — 解码验证

```
# 获取 JPEG 信息
w, h, fmt, _ = media.dvpp_jpeg_get_image_info(jpeg_dev, jpeg_size)

# 预测解码大小
dec_size, _ = media.dvpp_jpeg_predict_dec_size(jpeg_dev, jpeg_size, PIX_FMT_NV12)

# 创建输出 pic_desc + 解码
dec_buf, _ = media.dvpp_malloc(dec_size)
dec_pic = create_pic_desc(dec_buf, w, h)

media.dvpp_jpeg_decode_async(ch_desc, jpeg_dev, jpeg_size, dec_pic, stream)
acl.rt.synchronize_stream(stream)

# 验证闭环：输入 NV12 大小 = 输出 NV12 大小
```

310B 实测输出

```
JPEGE OK  640x480 NV12 -> 2097152 bytes JPEG  (quality=90)
JPEGD OK  2097152 bytes JPEG -> 640x480 NV12  size=460800
闭环验证  PASS  输入=460800 输出=460800
```

JPEG 码流 2MB 是因为测试帧的渐变 + 棋盘格纹理压缩率低（确定性生成，非真实照片）。正常照片在 quality=85 下通常只有几十 KB。

与项目的对应关系

文件	角色
`jpeg_minimal.py`	学习用途——裸 API 编解码闭环

5.5.6. 场景推荐

场景	推荐方案
WebRTC 截图保存	`vpc.jpege(frame)` -> 写入.jpg 文件
MJPEG 视频流解码	`vpc.jpegd()` 逐帧解码（配合 VPC resize）
照片缩略图生成	VPC resize -> JLEGE 编码（全硬件管道）
追求最小 JPEG 文件	裸 API + 精细调 quality 参数

全硬件截图管道

```
WebRTC NV12 帧 -> VPC resize(320×240) -> JPEGE(quality=80) -> JPEG 文件
                        ↑ 硬件                   ↑ 硬件
```

5.5.7. 常见问题与调试

问题 #1：JPEGE 输出不是 stream_desc

现象：试图用 `dvpp_get_stream_desc_data` 读取编码输出，得到无效数据。

根因：JPEGE 输出写入裸内存缓冲区，不是 `stream_desc`。只有 VENC 使用 stream_desc 输出。

修复：JPEGE 输出直接 `memcpy` 从 `out_buf` 拷出，实际大小从 `out_size_ptr` 读取。

问题 #2：out_size_ptr 的同步时序

现象：`synchronize_stream` 之前读取 `out_size_arr[0]`，得到的是 max_size 而非 actual_size。

根因：`out_size_arr` 是 in/out 参数，编码器在硬件完成后才写入实际值。

修复：**必须在 `synchronize_stream` 之后**读取 `out_size_arr[0]`。

问题 #3：predict_enc_size 返回的值远大于实际

现象：`predict_enc_size` 返回 2MB，但编码后只有 30KB。

根因：`predict_enc_size` 返回的是**最坏情况**的缓冲区大小，不是预测值。JPEG 的压缩率取决于图像内容。

修复：按 predict 值分配缓冲区（保障不溢出），编码后通过 `out_size_arr[0]` 取实际大小。

5.5.8. JPEG 编解码参数速查

参数速查表

JPEGE 编码流程

步骤	API	说明
创建配置	`dvpp_create_jpege_config ↪ ()`	—
设置质量	`dvpp_set_jpege_config_level ↪ (cfg, 1-100)`	越大画质越好
预测输出最大大小	`dvpp_jpeg_predict_enc_size ↪ (pic, cfg)`	最坏情况
异步编码	`dvpp_jpeg_encode_async(ch ↪ , pic, buf, size_ptr, ↪ cfg, stream)`	—
同步等待	`acl.rt.synchronize_stream ↪ (stream)`	—

JPEGD 解码流程

步骤	API	说明
获取图像信息	`dvpp_jpeg_get_image_info( ↪ ptr, size)`	宽度/高度
预测输出大小	`dvpp_jpeg_predict_dec_size ↪ (ptr, sz, fmt)`	NV12
异步解码	`dvpp_jpeg_decode_async(ch ↪ , ptr, sz, pic_desc, ↪ stream)`	—
同步等待	`acl.rt.synchronize_stream ↪ (stream)`	—

5.6. 集成实战：WebRTC 推流性能对比

完整可运行的代码位于 `samples/chapter5/WebRTC/`。本节聚焦教材层面的架构、H.264/H.265 编码链路和实测结论；具体运行命令、页面使用方式、复现实验步骤见该目录下的 README.md。

5.6.1. 项目定位

WebRTC 综合案例将本章前面学习的 VENC、JPEGD、NV12 stride 对齐和 aiortc 编码器适配串联为完整的视频推流管道。它由一个 Python WebRTC 服务端和一个浏览器接收页面组成：服务端在昇腾 310B 上采集、解码、编码视频，通过 aiortc 发送 RTP/WebRTC；浏览器只负责接收和显示。当前实现同时支持 H.264 和 H.265/HEVC 两条编码路径。

运行前先确认系统组合：H.264 硬件编码/WebRTC 路径要求上一节的 `20241128` 旧系统、旧版固件和 CANN 8.0+。当前已验证两组组合：同一系统从 CANN 7.0 分别升级到 CANN 8.0 和 CANN 8.3，WebRTC 性能结果一致；这次增加 CANN 8.0 验证不会改变此前性能结论。`20250925` 最新系统的配套固件在 CANN 8.0、8.5、9.0 下均导致 H.264 VENC 失败，服务应直接报告错误，等待香橙派发布固件更新。H.265、纯 `demo`/CPU 路径的结果也必须单独记录，不能替代 H.264 VENC 验证。

HTTP `POST /offer` 只是信令入口，不在媒体路径里。真正的媒体链路是：

```
Ascend 310B frame source -> AscendVideoTrack -> aiortc encoder -> RTP/WebRTC -> Browser
```

项目支持三条视频源路径和两种编码格式：

运行方式	MJPEG 解码	帧格式	编码器	用途
`--source demo`	无	`rgb24` 合成帧	libx264	验证 WebRTC 信令和浏览器接收链路
`--source usb_camera`	CPU (PyAV)	`rgb24`	libx264	纯 CPU 摄像头基线
`--source usb_camera --hardware-encode`	CPU (PyAV)	PyAV 转 `nv12`	CANN VENC H.264	只替换编码器的对照组
`--source dvpp_camera --video-codec h264`	DVPP JPEGD	`nv12`	CANN VENC H.264	历史 1080p60 路径；当前需先通过兼容性验证

运行方式	MJPEG 解码	帧格式	编码器	用途
`--source dvpp_camera --video-codec h265`	DVPP JPEGD	`nv12`	CANN VENC H.265	HEVC 路径，要求浏览器支持 WebRTC H.265

`dvpp_camera` 模式下硬件编码自动启用，因为 JPEGD 产出的 NV12 帧最适合直接交给 VENC。H.264 是默认编码格式；H.265 只走 CANN VENC，CANN 或 VENC 失败时该模式直接报告错误。

5.6.2. 目录结构与角色分工

- `server.py` — 服务入口：HTTP 路由、offer/answer、连接管理
- `webrtc_app/`
  - `ascend_source.py` — 视频源适配层：三种模式的初始化与帧产出
  - `cann_encoder.py` — CANN VENC H.264/H.265 编码器 + aiortc 兼容层
  - `dvpp_jpegd.py` — DVPP JPEGD 硬件解码器（MJPEG -> NV12）
  - `hevc.py` — H.265 Annex-B NAL 解析与 RFC 7798 RTP 分包
  - `v4l2_capture.py` — PyAV V4L2 MJPEG 采集模块
  - `v4l2_raw.py` — 直接 ioctl + mmap 采集（更高帧率，自动优先）
- `web/` — 浏览器接收页面（HTML/JS/CSS）
- `tools/` — H.265 offer/answer 与 VENC 码流验证脚本
- `test/` — pytest 测试套件

核心设计原则：**设备专属逻辑只写在 `webrtc_app/` 内**，`server.py` 只做会话控制和 HTTP 信令，不承载图像处理逻辑。接入新的视频源只需在 `AscendVideoTrack` 中添加一个 `_init_xxx()` 分支。

5.6.3. 关键模块走读

`server.py` — 会话控制层

`server.py` 是整个服务的入口。它不直接操作 VENC/JPEGD 描述符，但负责把 WebRTC 协商结果和硬件编码器绑定起来。

通用编解码偏好（`_prefer_video_codec_for_sender`）：当启用硬件编码时，服务端在 offer 处理中强制设置 video transceiver 的 codec preference。H.264 路径只允许 `video/H264`，

H.265 路径只允许 video/H265。这样可以避免浏览器和 aiortc 协商到 VP8 等软件编码格式，导致硬件 VENC 产出的码流无法被对端消费。

```
def _prefer_video_codec_for_sender(pc, sender, mime_type):
    codecs = [
        c for c in RTCRtpSender.getCapabilities("video").codecs
        if c.mimeType.lower() == mime_type.lower()
    ]
    for transceiver in pc.getTransceivers():
        if transceiver.sender == sender:
            transceiver.setCodecPreferences(codecs)
```

H.264 编码器替换：--hardware-encode 或 --source dvpp_camera 会把 aiortc 的 H264Encoder 替换为 CannH264Encoder。对于 dvpp_camera 模式下产出的 NV12 帧，VENC 是必要条件；CANN 或 VENC 失败时直接报告错误。

H.265 能力注册：aiortc 1.14.0 默认没有完整的 H.265 编码器路径。服务端在 --video-codec h265 模式下调用 _patch_h265_encoder()：先确认 CANN ACL 可用，再向 aiortc 的 video codec 列表注册 video/H265（clock rate 90000），最后把 encoder factory 指向 CannH265Encoder。

H.265 offer 校验：H.265 失败时机不是服务启动，而是浏览器发起 offer。服务端会检查浏览器 SDP 是否包含 H265/90000；如果没有，直接返回 HTTP 400：

```
Browser offer does not contain video/H265. Use a WebRTC HEVC-capable browser.
```

目标码率是建连参数：浏览器页面提交的 bitrate_kbps 会在 POST /offer 时进入服务端。服务端根据分辨率、帧率、编码格式和页面选择解析出本次连接的 VENC 目标码率，并在创建编码器/VENC 通道时使用。当前不支持运行中热调码率；要修改码率，需要断开后重新建连。

连接关闭时先停源再关 PeerConnection：close_peer_connection() 先调用 source_track.stop() 释放 /dev/video0，再 await pc.close()。这样下一个 offer 请求到达时可以立即复用摄像头设备。

ascend_source.py — 视频源适配层

AscendVideoTrack 继承 MediaStreamTrack，是 aiortc 获取视频帧的唯一入口。三种模式的差异全部封装在初始化方法和 _camera_read() 中。

demo 模式：纯软件合成帧——x/y 颜色渐变 + 正弦移动白条 + 余弦移动白方块，不需要任何外设或 CANN 依赖。用于验证 aiortc + 浏览器接收链路是否畅通。

usb_camera 模式（软件基线）：V4L2 MJPEG 采集 -> PyAV CPU JPEG 解码 -> RGB VideoFrame。这是为对比 DVPP 路径准备的软件基线，不依赖 CANN。

dvpp_camera 模式（全硬件管线）：

1. V4L2 通过 `V4l2RawCapture` 直采 MJPEG 码流
2. `DvppJpegDecoder` 硬件解码为 NV12（含 stride 对齐）
3. 返回 NV12 `VideoFrame` 给 aiortc -> `CannH264Encoder` 或 `CannH265Encoder` 直通编码

V4L2 后端策略：`dvpp_camera` 只使用 `v4l2_raw.py`（直接 ioctl + mmap）；该后端或摄像头初始化失败时直接报告错误。需要 PyAV CPU 解码时，应显式选择 `usb_camera`。

性能日志：`recv()` 每 150 帧打印一次 `Track FPS`，方便在设备端直接观察链路帧率；`_camera_read()` 前 5 帧打印单帧解码耗时。

cann_encoder.py — VENC 硬件编码 + aiortc 兼容

这是连接 CANN VENC 和 aiortc 的桥接层，包含三个核心组件：

CannVenc — 同步封装 VENC 异步回调 API。与第 5 章 VENC 教程中的版本相比，WebRTC 版本增加了：

- **pre_padded 参数**：JPEGD 输出的 NV12 已经是 stride 对齐的（宽度对齐到 16），此时跳过 CPU 侧的 stride 重排循环，直接从 `dvpp_malloc` -> `memcpy` -> `venc_send_frame`。这是实现"NV12 零拷贝管道" 的关键——整个 JPEGD->VENC 路径上 NV12 的 stride 布局始终一致。
- **回调队列残留检测**：`encode()` 发送前 drain 回调队列，若有残留帧则 warn。这能在日志中暴露" 上一帧结果未被消费" 的问题，避免静默丢帧。
- **ACL context 线程安全**：每次 `encode()` 和回调线程入口都显式 `set_context(ctx)`，确保线程池中的 executor 线程也能正常访问 ACL 资源。

bgr_to_nv12() — 纯 NumPy 实现的 BGR->NV12 转换。旧版依赖 OpenCV `cv2.cvtColor`，现在完全使用 NumPy 整数运算（ITU-R BT.601 转换公式），消除了 OpenCV 依赖。在 `dvpp_camera` 模式下此函数不会被调用（NV12 已由 JPEGD 产出）；非 NV12 帧走硬件编码时，它负责输入格式转换。

CannH264Encoder — 继承 aiortc 的 `H264Encoder`，只覆盖 `_encode_frame()`：

`H264Encoder` (aiortc) 的继承结构——只覆盖 `_encode_frame()`，其余 RTP 封装全部继承：

- `_encode_frame()` -> CANN VENC 编码 **[已覆盖]**
- `_split_bitstream()` -> Annex-B -> NAL 分割 **[继承]**
- `_packetize()` -> NAL -> RTP 分包 **[继承]**
- `pack()` -> RTP 打包逻辑 **[继承]**
- 其他全部继承

关键设计：

- **NV12 直通检测**：检查 `frame.format.name == "nv12"`。DVPP 产出的 NV12 帧直接调用 `to_ndarray`（零格式转换），传入 `pre_padded=True` 跳过 CPU stride 重排。

- **BGR->NV12 走 PyAV reformat**：非 NV12 帧（如 demo 的 RGB 帧）调用 `frame.reformat(format="nv12")`，由 PyAV 的 C 实现完成色彩空间转换（比手动 `bgr_to_nv12()` 快得多），再 `to_ndarray` 取出。
- **FPS 自适应**：`_estimate_fps()` 从连续帧的 PTS 时间戳差值实时估算实际帧率，据此调整 VENC 的 `src_rate` 参数。当实际帧率因采集链路抖动而变化时，VENC 的码率控制能跟随调整。
- **CANN/VENC 失败即终止**：`_CANN_READY` 为 False、通道创建失败或编码失败时抛出错误；硬件路径不会切换到 `super()._encode_frame()`。

`CannH265Encoder` — aiortc 兼容的 H.265 编码器。它复用 `CannVenc`，但创建通道时使用 `ENTYPE_H265_MAIN = 0`，输出 HEVC Annex-B 码流。H.265 路径不继承 aiortc 的 H.264 RTP 封装；CANN 初始化或 VENC 创建失败应视为 H.265 模式不可用。

hevc.py — H.265 RTP 分包

H.265 不能复用 H.264 的 RTP 分包格式。`webrtc_app/hevc.py` 单独实现 HEVC 相关逻辑：

- `split_annexb()`：按 Annex-B 起始码切分 HEVC NAL 单元
- `hevc_nal_type()`：从 HEVC 两字节 NAL header 中提取 NAL type
- `packetize_hevc()`：小 NAL 直接作为 single NAL packet，大 NAL 按 RFC 7798 封装为 Fragmentation Unit
- `has_hevc_keyframe_markers()`：检查 VPS/SPS/PPS/IDR 等关键 NAL，用于验证 VENC H.265 输出是否像合法 HEVC 码流

dvpp_jpegd.py — DVPP JPEGD 硬件解码器

封装了 JPEGD 的完整生命周期：DVPP 通道创建 -> JPEG->NV12 解码 -> 输出缓冲区管理。关键设计：

- **按需分配**:首次解码时通过 `dvpp_jpeg_get_image_info` 获取宽高,`dvpp_jpeg_predict_dec_size` 预测输出大小后分配设备缓冲区。后续帧复用已分配的缓冲区,避免每帧 malloc。
- **stride 对齐的 NV12 输出**：JPEGD 输出的 NV12 已按 16 宽度对齐，此 stride 布局与 VENC 的 `pre_padded=True` 路径完全匹配。从 JPEGD 到 VENC，NV12 数据无需 CPU 做任何重排。
- **线程安全**：`decode()` 入口显式 `set_context(ctx)`，因为此方法在线程池的 executor 线程中被调用。

v4l2_raw.py — 高性能 V4L2 采集

直接在 Python 中通过 `fcntl.ioctl` + `mmap` 操作 V4L2 设备，绕过 PyAV 的封装层。相比 PyAV 路径（1080p 约 15fps），直接 ioctl 路径可达到摄像头原生帧率（1080p 约 24fps）。两套后端通过相同的 `read(timeout)` 接口互换，上层代码无需感知。

> `v4l2_raw.py` 中的 `v4l2_buffer` 结构体布局（88 字节）是针对 aarch64 Linux 的硬编码值。在 x86-64 机器上此模块不可用，`dvpp_camera` 硬件路径会直接报告环境错误；需要软件基线时请显式选择 `usb_camera`。

5.6.4. 性能实测（1920×1080）

下面的数据来自 `311`（`orangepiaipro`）上 2026-05-26 的日志。测试请求为 `1920×1080@60`，浏览器接收端通过 WebRTC 接收视频流；这些性能数据对应同一 `20241128` 旧系统/旧固件，CANN 7.0 分别升级到 8.0 和 8.3 的两组验证结果，结论一致，不能外推到 `20250925` 系统的 H.264 VENC 路径。本次 `20241128 + CANN 8.0` 复测另外通过了 ACL、最小 VENC 和 5 秒 WebRTC E2E；后续 CANN 版本仍待继续验证。

本次板端 E2E 的关键日志为 `CANN VENC channel created ... entype=1`，随后连续输出 `VENC encode frame`；测试结果为 `E2E_TEST: PASS`。这证明本次运行实际使用了 H.264 VENC，不是 CPU 编码；但测试分辨率为 640×480、持续 5 秒，不能替代 1080p60 的完整性能测试。

模式	路径	Track FPS	瓶颈分析
纯 CPU	`V4L2 MJPEG -> CPU JPEG decode -> RGB -> libx264`	约 `15 fps`	CPU JPEG 解码单帧约 `20~32ms`，无法支撑 60fps
历史 H.264 硬编	`V4L2 MJPEG -> DVPP JPEGD -> NV12 -> CANN VENC H.264`	基本稳定 `60 fps`	历史日志中的 JPEGD 约 `4.9~9.5ms`，VENC 约 `6.3~6.5ms`
历史 H.265 硬编	`V4L2 MJPEG -> DVPP JPEGD -> NV12 -> CANN VENC H.265`	通常 `57~59 fps`，个别样本掉到 `35 fps`	码流更小，但历史日志中稳定性弱于 H.264

关键结论：

- 纯 CPU 路径在 `1920×1080@60` 请求下只能达到约 `15 fps`，主要瓶颈是 CPU JPEG 解码。
- 只替换编码器并不能解决全链路瓶颈；如果 MJPEG 解码仍在 CPU 上，VENC 的收益会被前级吞掉。

- `dvpp_camera` 的核心价值是 **JPEGD + VENC** 连用：JPEGD 输出 stride 对齐的 NV12，VENC 直接消费，中间不需要 RGB 转换。
- 两组已验证的 CANN 软件组合在这台 `311` 上性能结果一致，均优先选 H.264；H.265 码流更小，但 WebRTC HEVC 接收能力、SDP 协商和端到端稳定性更敏感。

浏览器有时会显示 `1920×1088`，这是 VENC 编码面高度按 16 对齐后的 coded size，不代表有效画面真的变成了 1088 行。1080 不能被 16 整除，因此硬件会补 8 行 padding。

5.6.5. 依赖说明

服务端（昇腾 310B）需要的依赖：

依赖	用途
`v4l-utils`	V4L2 命令行工具和库（`sudo apt install`）
`aiohttp`	HTTP 服务端和静态页面托管
`aiortc`	Python 侧 WebRTC 实现
`av` (PyAV)	VideoFrame 构造 + V4L2 采集
`numpy`	演示帧生成和 NV12 数据操作
CANN 8.0+（已验证 8.0、8.3；8.5/9.0 待后续扩展）	ACL Python API（硬件编码必需；须配合 `20241128` 旧系统/旧固件）
`pytest`	测试框架（仅开发）

旧版依赖的 `opencv-python-headless` 已移除。usb_camera 模式和 bgr_to_nv12 均已改用 PyAV + NumPy 纯实现。

5.6.6. 关键结论

- **DVPP 模块不能孤立优化**——只用 VENC 而保留 CPU JPEG 解码，链路仍会卡在 CPU 解码阶段。DVPP 的收益需要全链路协同才能兑现
- **JPEGD + VENC 是 1080p 实时推流的关键组合**——同一 `20241128` 旧系统/旧固件下，CANN 7.0 升级到 8.0 与升级到 8.3 的 WebRTC 性能结果一致：纯软件路径约 `15 fps`，全硬件 H.264 管道可以稳定到 `60 fps`；`20250925` 系统的 H.264 VENC 失败时不能使用这条结论
- **NV12 stride 对齐的零拷贝交付**——JPEGD 输出的 stride 对齐 NV12 直接匹配 VENC `pre_padded` 输入要求，中间不经过任何 CPU 数据重排
- **H.264 与 H.265 的工程取舍不同**——两组已验证 CANN 软件组合中 H.264 均是稳定的 1080p60 WebRTC 路径；H.265 只走 CANN VENC，依赖浏览器暴露 `video/H265` WebRTC 能力，且历史日志中有偶发掉帧。后续 CANN 版本仍需分别验证

- **码率在建连时确定**——页面目标码率会影响本次 VENC 通道创建，当前不支持在线热调

5.7. 应用调试与常见 FAQ

5.7.1. 调试技巧

- **返回值检查**：所有 ACL 接口均返回 `ret` 状态码，`0` 表示成功。非 0 需查阅《错误码参考》。
- **日志获取**：设置环境变量 `export ASCEND_GLOBAL_LOG_LEVEL=1`（Info 级别）查看详细日志，日志默认位置在 `~/ascend/log/`。
- **NPU 状态**：`npu-smi info` 查看芯片温度和内存占用。
- **驱动日志**：`dmesg | grep -i venc` / `dmesg | grep -i vdec` / `dmesg | grep -i dvpp` 排查硬件错误。

5.7.2. 常见问题

Q: 为什么 `acl.mdl.execute` 报错 “Memory Check Failed”? A: 检查 `acl.mdl.get_input_size_by_index` 获取的大小是否与 `acl.rt.malloc` 的大小严格一致。

Q: DVPP 解码后的图像显示异常? A: DVPP 输出有宽/高对齐要求（如 128×16 对齐）。读取数据时需要根据 `stride` 跳过 Padding 数据，而不能简单按 `width * height` 读取。

Q: `libascendcl.so: cannot open shared object file`? A: 确认 `LD_LIBRARY_PATH` 包含 `/usr/local/Ascend/ascend-toolkit/latest/aarch64-linux/lib64:/usr/local/Ascend/driver/lib64`。

Q: `No module named 'acl'`? A: 确认 `PYTHONPATH` 包含 `/usr/local/Ascend/ascend-toolkit/latest/python/site-packages`。

Q: VENC 创建通道返回 507018? A: 常见原因：(1) `max_bit_rate` 单位是 **kbps** 而非 bps（2000 而非 2000000）；(2) `key_frame_interval`（GOP）为 0，合法范围 `[1, 65536]`。

Q: VDEC 解码结果为空帧? A: 检查回调中的 `ret_code`——非 0 表示解码失败。常见原因：(1) 输入码流不是 Annex-B 格式（缺 `0x00000001` 起始码）；(2) 首帧缺少 SPS + PPS + IDR；(3) `entype` 与输入码流的编码格式不匹配。

5.7.3. 使用约束

1. **Context 线程安全**：一个 Context 可以在多个线程中使用，但需用户保证并发安全。推荐一线程一 Context。

2. **Stream 约束**：Stream 上的任务按顺序执行，但异步接口下发后需显式 `synchronize` 才能确保数据就绪。
3. **内存对齐**：DVPP 对内存地址和图片尺寸有严格对齐要求（宽对齐到 16，高对齐到 2）。
4. **通道数量有限**：Ascend 310B4 的 VENC/VDEC 硬件实例通常每种只有 1-2 个。创建通道前确保之前的通道已销毁。

6. 算子开发实战

昇腾 310B 作为一款边缘推理芯片，其算子开发是挖掘硬件性能的关键环节。尽管 CANN（Compute Architecture for Neural Networks）已提供丰富的内置算子库，但在面对自定义模型结构、特殊后处理逻辑或对极致性能有着严苛要求时，自定义算子开发（Custom Operator Development）仍不可或缺。本章将系统性地介绍基于昇腾 310B 的算子开发流程与核心理论，涵盖 TBE DSL 快速原型、算子原型定义、Ascend C 深度优化以及 Tiling 分片计算等关键主题。

6.1. 算子开发概述：昇腾 310B 硬件架构与开发路径

本节作为算子开发实战的开篇，旨在帮助读者构建完整的知识框架。我们将首先解析昇腾 310B 处理器基于达芬奇（DaVinci）架构的核心特征，深入其 AI Core 的计算单元与存储层次；随后概述 CANN 异构计算软件栈；最后横向对比 TBE 与 Ascend C 这两条主流的算子开发路径，并探讨自定义算子开发的必要性。

6.1.1. 昇腾 310B 处理器与达芬奇架构

昇腾 310B 是专门面向边缘计算与推理场景打造的高能效 AI 处理器。该芯片采用 12nm FFC 工艺制程，在仅 5 至 8W 的典型功耗下，提高两个版本——20TOPS@INT8 或者 8TOPS@INT8 的澎湃 AI 算力，能效比达到惊人的 25-40 TOPS/W。凭借这一优势，它在工业质检、智能电网、智慧交通等对实时性要求极高的场景中展现出了卓越性能。

达芬奇（DaVinci）架构是昇腾系列 AI 处理器的技术基石，其最大的亮点在于极强的可扩展性（Scalable）。针对不同应用场景的算力需求，达芬奇架构衍生出了 Tiny、Mini、Lite、Standard、Max 等多款版本，全面覆盖从低功耗可穿戴设备到高性能云端数据中心的全场景。其中，昇腾 310B 采用了针对边缘端与移动端专门优化的 DaVinci-mini 架构。

达芬奇架构的核心设计理念可概括为：**“将计算任务分层处理，以最契合的计算单元执行最适合的任务”**。该架构将计算任务精细划分为标量（Scalar）、向量（1D）、矩阵（2D）与立方体（3D）四种类型，并在物理硬件层面相应地例化了三大核心计算单元——Cube Unit、Vector Unit 与 Scalar Unit。

计算单元的精细分工

Cube Unit（立方体计算单元）是达芬奇架构的标志性设计，更是提供高密度算力的引擎。它专为深度神经网络量身定制，主要承担矩阵乘法、卷积运算及全连接层等计算密集型（Compute-bound）任务。依靠极高的数据复用率，Cube Unit 有效突破了内存带宽的限制。在昇腾 310B 中，其运行频率可达 1.224GHz，是释放全周期算力的关键所在。

Vector Unit（向量计算单元）专门处理向量级运算，工作机制类似于 SIMD（单指令多数据流）。其功能涵盖归一化、激活函数、池化、数据格式转换等，广泛服务于计算机视觉（如 RPN 网络）中常用的基础算子。Vector Unit 具备极高的执行灵活性，能够无缝兼容并处理多种数据类型与运算模式。

Scalar Unit（标量计算单元）类似于经典的 RISC 微处理器，主要负责控制流管理与基础标量运算。它承担着循环控制、内存地址计算、分支跳转等逻辑任务，是调度统筹整个 AI Core 的 “指挥中枢”。

这三大计算单元紧密协同，构建出高度并行的计算流水线。昇腾技术团队在多项典型任务中的实测数据表明，Cube Unit 占用的执行周期（Cycles）显著大于 Vector Unit，这意味着 Cube Unit 的运算潜能得到了充分释放，未受限于 Vector Unit 的调度阻塞，二者实现了优异的负载均衡。

存储层次与数据搬运机制

昇腾 310B 的存储系统采用了精巧的多级层次结构体系。深入理解这一体系，是突破算子性能瓶颈、实现极致优化的先决条件。

全局内存（Global Memory）位于存储架构的最外层，具备最大的容量空间，通常指代片外的板载 LPDDR4X 内存。昇腾 310B 普遍配置 8-16GB 的 LPDDR4X，带宽范围在 51.2GB/s 至 408GB/s。尽管容量充裕，但其访存延迟相对较高。因此，在进行算子开发时，应尽可能减少对全局内存的直接读取次数。

局部内存（Local Memory）集成于 AI Core 内部，属于极低延迟的高速存储区，主要包括 L1 缓存（L1 Buffer）与统一缓冲区（Unified Buffer，简称 UB）。L1 缓存专用于暂存高频复用的数据，从而大幅削减跨总线的读写开销。UB 则是算子执行时的核心工作台，所有参与计算的数据均需先从全局内存搬移至此，方能被计算单元读取。

存储转换引擎（Memory Transfer Engine, MTE）是专为数据搬运与内存重排设计的加速单元。MTE 细分为 MTE1、MTE2、MTE3 等模块，负责高效管理 AI Core 内外不同层级缓冲区之间的数据流动，并能够在搬运过程中硬件加速般地同步完成数据填充（Padding）、转置（Transpose）、Img2Col 等格式化操作。

总线接口单元（Bus Interface Unit, BIU）扮演着 AI Core 与外部存储总线通信的 “门户” 角色，其主要职责是将 AI Core 发出的各类读写请求精确转化为标准的总线协议交互。

在一个典型的计算流水线中：数据首先通过 BIU 由全局内存接入，随后经 MTE 的高效搬运与格式重组，稳妥驻留于 L1 缓存或 UB 中。紧接着，Scalar Unit 发出调度指令，

指挥 Cube Unit 或 Vector Unit 对 UB 中的数据进行高速运算。处理结束后，输出结果再次交由 MTE 接管，安全、高效地写回至全局内存，由此形成一个无缝闭环。#### 昇腾 AI 异构计算架构（CANN）{#src-book-chapter6-h5}

正如上一章在探讨 PyACL 编程基础时所述，CANN（Compute Architecture for Neural Networks）是昇腾全面面向 AI 场景定制的异构计算架构。它在整个计算系统中发挥着承上启下的核心枢纽作用：向上广泛适配 MindSpore、PyTorch、TensorFlow 等主流 AI 框架，向下直接调度并深度释放昇腾 AI 处理器的澎湃算力，是提升硬件计算效率的关键" 软件底座"。（CANN 的详细架构、与 CUDA 对比、安装配置等内容请参见第 2 章。）

为了兼容并蓄不同维度的开发需求，CANN 构建了多层次的编程接口与组件体系：- **AscendCL**：统一的应用开发原生接口，旨在屏蔽底层硬件差异，帮助开发者灵活构建从端到云的 AI 应用。- **Ascend C**：基于 C++ 的高性能算子开发语言，允许开发者精细控制底层硬件指令与多级缓存，专为追求极致性能的算子定制而生。- **TBE**：基于 Python 的开发框架，侧重于算子逻辑的快速表达与系统化自动调度优化。- **图引擎（Graph Engine）**：核心的计算图编译与执行引擎，负责全局网络图的解析、内存复用规划及算子融合优化。

针对算子开发层面，CANN 构建了一站式的全流程工具链支持。涵盖了专属算子编译器、功能验证仿真工具，以及深度的性能调优分析器（Profiler），全面辅助开发者打通从代码原型编写、逻辑调试到逼近硬件理论极限的全闭环开发流程。

6.1.2. 算子开发的两条主流路径

针对不同开发需求和性能目标，昇腾 CANN 提供了两条主要的算子开发路径：**TBE（快速原型）**和 **Ascend C（深度优化）**。

TBE：声明式开发，效率优先

TBE（Tensor Boost Engine）是一种基于 Python 的算子开发框架，其核心特点在于**让开发者专注于描述计算逻辑，由系统自动完成底层的复杂优化**。在开发方式上，TBE 提供了 DSL（Domain-Specific Language）编程模式。开发者无需深入了解昇腾底层硬件架构，只需通过几行简洁的 Python 代码即可描述算子的数学表达式。例如，使用`dsl.vadd(input_x, input_y)`即可轻松表达向量加法，随后通过调用`dsl.auto_schedule()`，TBE 会自动接管并完成模式识别、子图切分、调度模板选择以及底层指令映射等一系列流程。

这种声明式开发范式赋予了 TBE 诸多的优势。首先是开发门槛极低且代码异常简洁，一个完整的加法算子核心逻辑往往只需 10 行左右即可实现。同时，得益于昇腾官方调度模板的自动优化加持，生成的算子性能稳定可靠，这使其非常适合用于算法的原型验证以及非性能瓶颈算子的快速实现。

然而，TBE 的自动化机制也带来了一定的局限。当面对具有极端定制化计算需求或特殊数据流模式的算子时，高度封装的自动调度可能难以将其优化至硬件的理论极致性能。

此外，在某些特定的高级算子类型上，当前的 DSL 语法或许尚未提供完全覆盖的底层支持能力。

> **补充：TBE TIK 模式**。TBE 早期还提供了一种名为 **TIK（Tensor Iterator Kernel）**的 Python 命令式开发模式，允许开发者手动控制数据搬运与计算流水线，在控制粒度上介于 DSL 与 Ascend C 之间。但随着 Ascend C 的成熟，TIK 已被逐步取代，不再推荐用于新项目。本书不再展开 TIK 相关内容，读者若在存量项目中遇到 TIK 算子，可查阅昇腾官方文档。

Ascend C：命令式开发，性能优先

Ascend C 是一种基于 C++ 的高性能算子开发语言，其核心优势在于赋予了开发者对数据流、指令流以及多核并行执行的精细控制权。在编程模型上，Ascend C 采用了 SPMD（单程序多数据）架构，这意味着所有的 AI Core 将执行同一套代码逻辑，但会根据各自的任务 ID 去处理被分配的不同数据区间。

在具体的开发过程中，开发者需要显式地设计计算流水线并深度管理多级存储之间的数据搬运。一个典型的 Ascend C 算子实现紧密围绕着三个连续的核心任务展开：首先是 “CopyIn” 阶段，负责将输入数据从全局内存高效搬运至局部内存；随后进入 “Compute” 阶段，调用硬件资源执行具体的数学运算；最后是 “CopyOut” 阶段，将计算所得的结果从局部内存搬运回全局内存，从而完成整个数据流转的闭环。

这种命令式的开发范式实现了灵活性与极致性能的统一。得益于对底层内存读写和流水线编排的精准把控，Ascend C 能够帮助开发者逼近硬件的理论计算极限，更被广泛应用于搞定大模型场景下如 FlashAttention 等复杂的性能瓶颈算子。此外，其原生支持 CPU 模拟调试与中间变量打印，极大地优化了开发体验。然而，获取这种极致性能也意味着较高的开发门槛，开发者必须对昇腾底层硬件架构有透彻的理解。同时，Ascend C 的代码工程量相对较大，即便是实现一个极简的算子，往往也需要编写数百行代码来完成对底层资源的系统化调度。

两条路径的协同关系

两条开发路径并非互斥，而是可以根据需求协同使用：

- **首选 TBE DSL**，快速实现性能达标的标准算子
- **慎用 Ascend C**，在追求极致性能或实现特殊计算模式时，系统化应用优化策略
- **善用工具链**，坚持数据驱动的性能调优闭环

一个典型的开发流程是：先用 TBE 快速实现算子原型，验证功能正确性；如果性能不达预期，再用 Ascend C 重写核心计算逻辑，通过精细优化达到极致性能。

CANN 算子库本身包含了丰富的高性能算子，覆盖了大多数常见场景。但在以下情况下，开发者需要考虑自定义算子：

训练场景下的算子缺失：将第三方框架（如 TensorFlow、PyTorch）的训练脚本迁移到昇腾 AI 处理器时，遇到框架支持但 CANN 算子库暂不支持的算子。

推理场景下的模型转换：使用 ATC 工具将第三方框架模型转换为昇腾离线模型时，遇到不支持的算子。

网络性能调优：发现某算子性能较低，成为网络性能瓶颈，需要重新开发一个高性能算子替换原有算子。例如，一个 2048x2048 的矩阵乘法算子，经过系统化优化后，性能可从 512ms 提升至 92ms。

应用后处理加速：应用程序中的某些逻辑涉及数学运算（如查找最大值、数据类型转换），可以封装为自定义算子在 AI 处理器上执行，利用 NPU 提升性能。例如，分类应用中查找概率最大的前 5 个标识，可以开发 ArgMax 算子实现后处理加速。

6.1.3. 小结

通过本节的学习，读者应该对昇腾 310B 的硬件架构、CANN 软件栈以及算子开发的两种路径有了整体认识。从下一节开始，我们将从最简单的 TBE 算子入手，带领读者亲手实现第一个自定义算子，在实践中加深对概念的理解。

6.2. 初体验：使用 TBE DSL 快速实现第一个算子（向量加法）

通过一个最简单的向量加法算子，让读者快速体验 TBE 开发的完整流程。在 VSCode 中编写 Python 脚本，使用 TBE DSL 描述算子逻辑，通过命令行工具编译生成算子文件，并编写简单的测试代码在 NPU 上运行验证。最后总结 TBE 的优缺点及适用场景，激发读者进一步学习的兴趣。

6.2.1. TBE DSL 快速概览

在动手编写代码之前，有必要先理解 **TBE DSL 到底是什么**。

什么是 DSL?

DSL（Domain-Specific Language，领域特定语言）是为特定问题域量身定制的编程语言。与通用语言（如 Python、C++）不同，DSL 只关注一个狭窄的领域，用该领域的术语来表达计算意图，从而大幅降低表达复杂度。

TBE DSL 就是昇腾为**算子开发**这一特定领域设计的 Python 内嵌语言。它运行在 Python 环境中，但提供了一套专用于描述张量计算、调度策略和编译选项的 API。

上图展示了 TBE DSL 的完整体系结构。整个架构自顶向下分为三层：最上层是暴露给开发者的 **Python DSL 接口**（计算描述 -> 调度 -> 编译），中间层是 **TVM 编译框架**负责

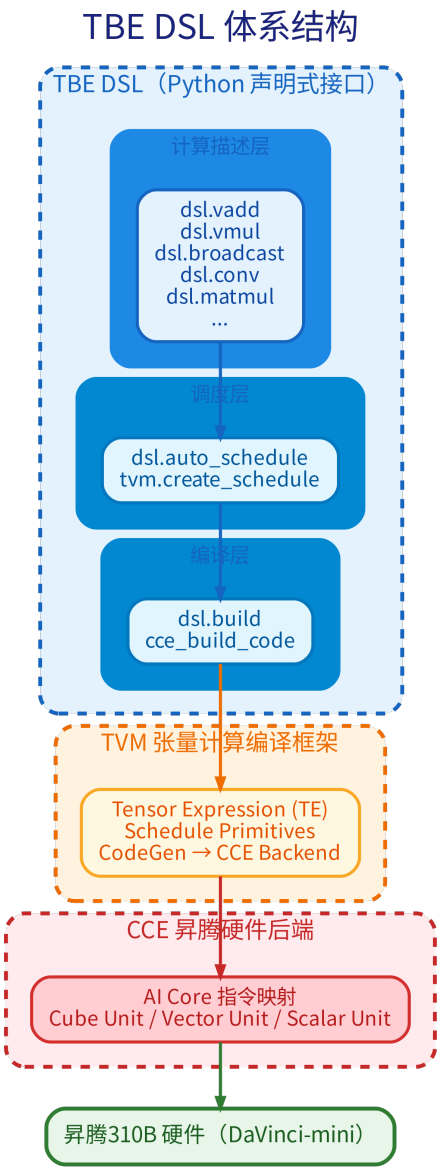


图 6.1. TBE DSL 体系结构

张量表达式的优化与代码生成，底层是 **CCE 硬件后端**将优化后的计算映射到昇腾 310B 的 AI Core 指令。

DSL 与 TVM 的关系

TBE 基于 **Apache TVM** 框架深度定制。TVM（Tensor Virtual Machine）是一个开源的深度学习编译器框架，它的核心能力是将高层张量计算描述编译为不同硬件后端的可执行代码。TBE 复用了 TVM 的编译基础设施，并将后端替换为昇腾自家的 CCE。

TVM 在算子编译流程中承担三个关键角色：

TVM 组件	职责	在 TBE DSL 中的体现
Tensor Expression (TE)	用张量表达式描述算子的数学逻辑（“算什么”）	`tbe.dsl` 中的计算 API（如 `dsl.vadd`）
Schedule Primitives	提供低层调度原语（分块、向量化、绑定等），决定计算顺序与并行策略（“怎么算”）	`dsl.auto_schedule()` 自动选择调度模板；也可手动调用 `tvm.create_schedule()` 精细控制
CodeGen	将优化后的调度翻译为目标硬件指令	对接 **CCE Backend**，生成昇腾 AI Core 可执行的二进制码

什么是 CCE?

CCE（Cube-based Compute Engine）是昇腾 CANN 的编译器后端，专为达芬奇架构设计。它的职责是将 TVM 优化后的计算图翻译为 AI Core 可直接执行的指令流，包括：

- 将计算任务分配到 Cube Unit、Vector Unit、Scalar Unit
- 生成 MTE 数据搬运指令
- 管理 L1 Buffer 与 Unified Buffer 的分配与回收

CCE 的输入是 TVM 产生的优化 IR（中间表示），输出是可在昇腾 310B 上加载运行的二进制算子文件（`.o` 和 `.json`）。在整个 DSL 开发流程中，开发者无需直接与 CCE 交互——`dsl.build()` 和 `cce_build_code()` 会自动调用它完成底层编译。

在 TBE 的声明式范式下，开发者**只需用 TE 描述计算逻辑**，调度部分由 `auto_schedule()` 自动完成。这正是 TBE 与 Ascend C 的根本区别——前者说” 算什么”，后者必须亲自说” 怎么算”。

DSL 的核心 API 分层

TBE DSL 的 API 按职责分为三层：

层	职责	典型 API
计算描述层	定义张量运算的数学逻辑	`dsl.vadd`, `dsl.vmul`, `dsl.broadcast`, `dsl.conv`, `dsl.matmul` …
调度层	将计算映射为硬件执行计划	`dsl.auto_schedule`, `tvm.create_schedule`
编译层	生成昇腾设备可执行文件	`dsl.build`

一个典型的 TBE DSL 算子开发流程就是这三层的顺序调用：先用计算描述层的 API 写出算子的数学表达式，然后交由调度层的 `auto_schedule()` 自动生成硬件执行计划，最后由编译层的 `dsl.build()` 生成可在昇腾设备上运行的二进制文件。这个从声明到编译的完整链路，即为前文 @fig:dsl_architecture 中蓝色箭头所描绘的纵向路径。

声明式 vs 命令式

为了进一步理解 DSL 的定位，这里将其与下一节将要学习的 Ascend C 做个对比：

维度	TBE DSL（声明式）	Ascend C（命令式）
编程语言	Python	C++
开发思路	描述” 算什么”	控制” 怎么算”
调度策略	`auto_schedule` 自动生成	开发者手动编排流水线
内存管理	框架自动处理	开发者显式管理多级缓存
代码量（以加法为例）	~20 行	~200 行
性能天花板	接近硬件极限（官方模板保证）	可达硬件理论极限
适用场景	原型验证、标准算子	性能瓶颈算子、特殊计算模式

了解了这些基本概念后，我们就可以正式动手了。

6.2.2. 算子分析：明确我们要做什么

在动手编码之前，先明确我们要开发的算子规格。按照昇腾算子开发的规范，我们需要先进行算子分析。

算子功能：实现两个向量的加法，数学表达式为：`z = x + y`

输入输出规格：- 输入：两个张量（Tensor）x 和 y，形状相同，数据类型相同 - 输出：一个张量 z，形状和数据类型与输入相同 - 数据类型支持：float16、float32、int32 - Shape 支持：所有形状（本例使用简单的 1D 向量）

开发方式选择：使用 TBE DSL 方式，主要调用两个接口：- `tbe.dsl.broadcast`：处理广播场景（本示例输入 shape 相同，但保留广播能力）- `tbe.dsl.vadd`：执行向量加法

算子命名：算子类型（OpType）采用大驼峰命名”Add”；实现文件名称和函数名称采用小写”add”。

TBE DSL 算子的开发流程可以分为四个主要步骤：

阶段	核心任务	关键函数/概念
算子定义	明确输入输出，设计接口	te.placeholder
计算实现	描述算子的数学逻辑	te.compute, te.lang.cce.vadd
调度编译	将计算逻辑映射到硬件	auto_schedule, cce_build_code
验证测试	在 NPU 上运行并核验结果	NumPy 对比，AscendCL 调用

6.2.3. 完整代码实现

创建工程目录

首先在 VSCode 中连接到昇腾 310B 开发板（或直接在板子上操作），创建以下目录结构：

```
# 创建工程目录并进入
mkdir -p add_tbe
cd add_tbe

# 创建必要的源文件
touch add.py run.py
```

预期的目录结构如下：

```
add_tbe/
├── add.py           # TBE算子实现文件
├── run.py           # 测试验证脚本
└── kernel_meta/     # 编译输出目录（自动生成）
```

编写 TBE 算子代码（add.py）

下面是完整的向量加法算子实现代码，我们将逐段解释。

```
import tbe
from tbe import tvm
from tbe import dsl
from tbe.common.utils import para_check
from tbe.common.utils import shape_util

from functools import reduce

SHAPE_SIZE_LIMIT = 2147483648

# 实现Add算子的计算逻辑

@tbe.common.register.register_op_compute("add",op_mode="static")
```

```
def add_compute(input_x, input_y, output_z, kernel_name="add"):
    shape_x = shape_util.shape_to_list(input_x.shape) # 将shape转换为list
    shape_y = shape_util.shape_to_list(input_y.shape) # 将shape转换为list
    shape_x, shape_y, shape_max = shape_util.broadcast_shapes(shape_x, shape_y,
    ↪ param_name_input1="input_x",param_name_input2="input_y")   # shape_max取shape_x与
    ↪ shape_y的每个维度的大值
    shape_size = reduce(lambda x, y: x * y, shape_max[:])
    if shape_size > SHAPE_SIZE_LIMIT:
        raise RuntimeError("the shape is too large to calculate")

    input_x = dsl.broadcast(input_x, shape_max)       # 将input_x的shape广播为shape_max
    input_y = dsl.broadcast(input_y, shape_max)       # 将input_y的shape广播为shape_max
    res = dsl.vadd(input_x, input_y)        # 执行input_x + input_y

    return res          # 返回计算结果的tensor

# 算子定义函数
def add(input_x, input_y, output_z, kernel_name="add"):
    # 获取算子输入tensor的shape与dtype
    shape_x = input_x.get("shape")
    shape_y = input_y.get("shape")
    check_tuple = ("float16", "float32", "int32")
    input_data_type = input_x.get("dtype").lower()
    if input_data_type not in check_tuple:
        raise RuntimeError("only support %s while dtype is %s" %
                           (",".join(check_tuple), input_data_type))
    # shape_max取shape_x与shape_y的每个维度的最大值
    shape_x, shape_y, shape_max = shape_util.broadcast_shapes(shape_x, shape_y,
    ↪ param_name_input1="input_x",param_name_input2="input_y")
    if shape_x[-1] == 1 and shape_y[-1] == 1 and shape_max[-1] == 1:
        # 如果shape的长度等于1，就直接赋值，如果shape的长度不等于1，做切片，将最后一个维度舍弃（按照内
    ↪ 存排布，最后一个维度为1与没有最后一个维度的数据排布相同，例如2*3=2*3*1，将最后一个为1的维度舍弃，
    ↪ 可提升后续的调度效率）。
        shape_x = shape_x if len(shape_x) == 1 else shape_x[:-1]
        shape_y = shape_y if len(shape_y) == 1 else shape_y[:-1]
        shape_max = shape_max if len(shape_max) == 1 else shape_max[:-1]

    # 使用TVM的placeholder接口对第一个输入tensor进行占位，返回一个tensor对象
    data_x = tvm.placeholder(shape_x, name="data_1", dtype=input_data_type)
    # 使用TVM的placeholder接口对第二个输入tensor进行占位，返回一个tensor对象
    data_y = tvm.placeholder(shape_y, name="data_2", dtype=input_data_type)

    # 调用compute实现函数
    res = add_compute(data_x, data_y, output_z, kernel_name)
    # 自动调度
    with tvm.target.Target("cce", host="cce"):
        schedule = dsl.auto_schedule(res)
    # 编译配置
    config = {"name": kernel_name,
              "tensor_list": (data_x, data_y, res)}
    dsl.build(schedule, config)

# 算子调用
```

```
if __name__ == '__main__':
    input_output_dict = {"shape": (5, 6, 7),"format": "ND","ori_shape": (5, 6, 7),"ori_format": "ND", "dtype": "float16"}
    add(input_output_dict, input_output_dict, input_output_dict, kernel_name="add")
```

注意事项：在 CANN 8.3.RC1 版本中，`tvm.target.cce()` 已被标记为废弃（deprecated）。如果运行时出现如下警告：

```
UserWarning: target_host parameter is going to be deprecated.
Please pass in tvm.target.Target(target, host=target_host) instead.
```

请将 `with tvm.target.cce():` 替换为 `with tvm.target.Target("cce", host="cce"):`，这是 CANN 新版 API 的标准写法。

6.2.4. 代码逐段详解

导入模块

```
import tbe
from tbe import tvm
from tbe import dsl
from tbe.common.utils import para_check
from tbe.common.utils import shape_util
from functools import reduce
```

- `tbe`：TBE 框架主模块
- `tbe.dsl`：包含 DSL 计算接口（如 vadd）、调度接口和编译接口
- `tbe.tvm`：TBE 基于 TVM 框架扩展，可以使用 TVM 接口
- `shape_util`：提供 shape 处理工具，如广播 shape 计算
- `para_check`：参数校验工具

算子计算逻辑（add_compute）这是算子开发的核心，描述” 如何计算”。关键点：- `@register_op_compute`装饰器将函数注册为算子的计算逻辑 - **Shape 大小校验**：计算广播后张量的总元素个数，当超出`SHAPE_SIZE_LIMIT`阈值时抛出异常 - **数据广播与计算**：先通过`dsl.broadcast`将输入形状广播对齐，再调用`dsl.vadd`执行向量加法（自动识别为 element-wise 模式）

算子主函数（add）

这是算子的入口函数，负责调度和编译：
参数校验：校验数据类型是否在支持范围内（float16/float32/int32）。

Shape 切片优化：如果 shape 的末尾维度长度为 1，则将其直接舍弃。由于内存排布上末尾为 1 并不影响实际排布（例如 231 等同于 2*3），舍弃后可有效提升后续的调度效率。

TVM 占位符：`tvm.placeholder`创建输入张量的占位符，描述张量的形状和数据类型，但不分配实际数据。

自动调度：`dsl.auto_schedule`自动完成 AST 标注、模式识别、子图切分、调度模板选择，并将指令映射到昇腾硬件。

编译构建：`dsl.build`将调度后的计算描述编译为昇腾设备可执行的二进制文件。

6.2.5. 编译算子

在终端执行以下命令编译算子：

```
python3 add.py
```

编译成功后，会在当前目录生成`kernel_meta/`文件夹，包含两个文件：- `add.o`：算子的二进制目标文件 - `add.json`：算子的元信息描述文件

查看生成的文件：

```
ls kernel_meta/
# 输出示例：add.o  add.json
```

6.2.6. 算子验证：在 NPU 上运行测试

编译成功后，我们需要验证算子的正确性。昇腾提供了两种验证方式：- **单算子模型执行**：将算子编译成单算子离线模型（.om 文件），通过 AscendCL 加载执行 - **单算子 API 执行**：直接通过 AscendCL 的 API 调用算子

本节采用第一种方式，因为它更接近实际部署场景。

编写验证代码（run.py）

下面是使用 AscendCL 加载并执行单算子的验证代码：

```
# run.py
from tbe import tvm
from tbe import dsl
from tbe.common.utils import para_check
from tbe.common.utils import shape_util
# 引入testing模块相关接口
from tbe.common.testing.testing import *
from tbe.common.testing.testing import _Testing
import numpy as np
```

```
10
11
12 def build(inputs, args=None, name="default_function"):
13     """覆盖 testing 模块的 build()，将 target 和 target_host 合并为新版 API。"""
14     _Testing.build(inputs, args=args,
15                    target=tvm.target.Target("c", host="llvm"),
16                    target_host=None, name=name,
17                    tiling_keys=None, binds=None, evaluates=None)
18
19
20 @para_check.check_input_type(dict, dict, dict, str)
21 def addtest(input_a, input_b, output_d, kernel_name="addtest"):
22     # 进入DSL调试模式，并选择CPU作为运行平台
23     with debug():
24         # 获取算子运行的上下文
25         ctx = get_ctx()
26
27         # 获取输入数据的shape与dtype
28         shape_a = shape_util.scalar2tensor_one(input_a.get("shape"))
29         shape_b = shape_util.scalar2tensor_one(input_b.get("shape"))
30         data_type = input_a.get("dtype").lower()
31
32         # 使用numpy定义输入golden数据大小
33         a = tvm.nd.array(np.random.uniform(size=shape_a).astype(data_type), ctx)
34         b = tvm.nd.array(np.random.uniform(size=shape_b).astype(data_type), ctx)
35         # 使用numpy将输出d初始化为全0
36         d = tvm.nd.array(np.zeros(shape_a, dtype=data_type), ctx)
37
38         # 调用TVM的placeholder接口对输入tensor进行占位，并返回一个tensor对象
39         data_a = tvm.placeholder(shape_a, name="data_1", dtype=data_type)
40         data_b = tvm.placeholder(shape_b, name="data_2", dtype=data_type)
41         # 调用DSL计算接口实现data_a + data_b
42         data_c = dsl.vadd(data_a, data_b)
43
44         # 中间Tensor数据验证
45         sample = open('samplefile.txt', 'w')
46         # 将中间tensor data_c存入文件samplefile.txt
47         print_tensor(data_c, ofile=sample)
48         # 检查中间tensor data_c的值是否正确
49         assert_allclose(data_c, desired=a.asnumpy() + b.asnumpy(), tol=[1e-7, 1e-7])
50         print("The value of data_c is the same as the expected value.")
51
52         # 继续自定义DSL的逻辑撰写,调用DSL接口实现：data_d = data_c + data_b
53         data_d = dsl.vadd(data_c, data_b)
54         # 调用TVM的create_schedule接口，为算子创建调度实例对象，入参为输出tensor的OP列表。
55         s = tvm.create_schedule(data_d.op)
56
57         # 编译生成算子,data_a,data_b,data_d是占位的输入输出列表，AddTest是我们自定义算子的名称
58         build(s, [data_a, data_b, data_d], name="AddTest")
59
60         # 执行算子,将a,b,d按顺序代入编译出来的DSL算子AddTest
61         run(a, b, d)  # AddTest(a, b, d)
62
```

```
        # 将输出数据d的值打印出来,并预期结果进行比较，看是否相符
        print("d:", d)
        tvm.testing.assert_allclose(d.asnumpy(), a.asnumpy() + b.asnumpy() + b.asnumpy())
        print("The actual output is the same as the expected output.")

# 编写入口函数，调用addtest函数
if __name__ == "__main__":
    input_output_dict = {"shape": (2, 3, 4),"format": "ND","ori_shape": (2, 3, 4),"ori_format": "ND", "dtype":"float32"}
    addtest(input_output_dict, input_output_dict, input_output_dict, kernel_name="addtest")
```

> **注意事项**：由于 `tbe.common.testing.testing` 模块的 `build()` 函数内部仍使用旧版 API（将 `target` 和 `target_host` 作为独立参数传入 `tvm.build()`），同样会触发上述废弃警告。解决方案是在 `run.py` 中自定义一个 `build()` 函数来覆盖它，将两个参数合并为 `tvm.target.Target("c", host="llvm")`，从而消除警告。

运行验证

验证代码：

```
# 直接运行
python3 run.py
```

成功输出示例：

```
======================== debug enter ========================
The value of data_c is the same as the expected value.
Tensor add_0 is saved to file samplefile.txt.
d: [[[1.0816491  1.9020174  1.2096624  2.5805097 ]
  [2.401499   1.8532326  1.4911635  2.6252913 ]
  [1.0037376  2.671739   1.8189309  0.3927391 ]]

 [[1.9595971  2.1914093  1.6825981  0.9398521 ]
  [1.0270492  2.070397   0.97784364 1.7433951 ]
  [0.4678194  2.564149   1.746469   1.7772455 ]]]
The actual output is the same as the expected output.
======================== debug exit ========================
```

至此，我们完成了第一个 TBE DSL 算子的完整开发流程：从算子分析、代码实现、编译到 NPU 上验证运行。

性能验证：NPU 真的比 CPU 快吗？

成功运行算子只是第一步，一个自然的问题是：**算子跑在 NPU 上到底比 CPU 快多少？**

验证方法并不复杂：用 NumPy 在 CPU 上执行相同规模的加法，与 TBE 算子对比耗时。以下是一个简单的计时示例：

```
import time
import numpy as np

shape = (256, 256, 256)          # 约 1670 万个元素
dtype = np.float32

a = np.random.uniform(size=shape).astype(dtype)
b = np.random.uniform(size=shape).astype(dtype)

# CPU 计时
t0 = time.time()
c_cpu = a + b
t_cpu = time.time() - t0

# NPU 计时（在 run.py 的 debug 环境中，用 TVM 的 time_evaluator）
# 详见 CANN Profiling 工具 msprof 的官方文档
print(f"CPU time: {t_cpu:.4f}s")
```

对于 256x256x256 规模的 float32 加法，昇腾 310B 上经 TBE DSL 生成的算子通常可获得数倍乃至十数倍于 CPU 的加速比。这得益于 NPU 的多核并行执行与 UB 缓冲区的极低访存延迟。

需要强调的是，上述简单计时只适用于初步感受。系统性的性能分析应使用昇腾官方的 **Profiling 工具（msprof）**，它能精确采集 AI Core 执行周期、内存带宽利用率、算子耗时占比等关键指标。Profiling 的具体使用方法将在后续的性能优化章节中详细介绍。

6.2.7. TBE DSL 开发的优势与局限

通过本次实战体验，我们可以总结 TBE DSL 开发的特点：

优势

优势	说明
开发门槛低	使用 Python 开发，无需深入了解昇腾硬件架构，只需描述数学逻辑
代码简洁	一个完整的加法算子核心代码仅需 10 行左右，大幅减少开发工作量
自动优化	`auto_schedule`自动完成指令映射、数据切分、流水线优化，由昇腾官方模板调度，性能稳定可靠
快速验证	适合算法原型验证和非性能瓶颈算子的快速实现

局限

局限	说明
自动调度的" 黑盒" 特性	Auto Schedule 对特殊结构的算子可能不够精细，当算子有强性能诉求时，可能无法达到极致性能
灵活性受限	对于具有极端定制化需求或特殊数据流模式的算子，DSL 的自动调度可能无法完全满足
高级算子覆盖	某些高级算子类型 DSL 可能尚未完全覆盖

6.2.8. 小结与展望

本节我们通过一个向量加法算子，完整实践了 TBE DSL 开发的四步流程：算子分析、代码实现、编译、验证。你可能会感受到 TBE DSL 带来的便利——用熟悉的 Python 语言，专注于数学逻辑本身，而将底层复杂的硬件适配交给自动调度完成。

这正体现了 TBE 的设计哲学：**将开发者从复杂的硬件指令和内存管理中解放出来，专注于算法本身**。

然而，正如我们在" 局限" 中提到的，自动调度并非万能。当追求极致性能、或实现特殊计算模式时，我们需要更精细的控制能力。这正是下一节将要学习的 **Ascend C 开发范式**的用武之地。

在进入下一节之前，建议你亲自动手完成本节示例，并在自己的昇腾 310B 开发板上跑通验证。这是理解后续更深内容的基础。

6.3. 理解算子开发的核心概念

在动手实践之后，本节将系统讲解算子开发中必须掌握的三个核心概念：算子原型定义、算子信息库（.ini 文件）以及 Tiling（分片计算）。无论使用 TBE DSL 还是 Ascend C，这些概念都是绕不开的基础知识。

6.3.1. 算子原型定义

一个算子要在 CANN 框架中被正确注册和调用，首先需要明确它的**原型（Prototype）**。算子原型定义了该算子的完整接口契约，包括：

输入与输出（Inputs & Outputs）：每个算子接受若干输入张量，产生若干输出张量。张量的规格包括：

要素	说明	示例
Shape	张量的维度信息	`(batch, channel, height, width)`
Format	张量的内存排布格式	`ND`（普通排布）、`NHWC`、`NCHW`、`FRACTAL_Z`（昇腾分形格式）
Dtype	数据类型	`float16`、`float32`、`int32`、`int8`

昇腾支持多种内存排布格式，其中 **FRACTAL_Z** 是昇腾专为 Cube Unit 卷积/矩阵乘法设计的特殊格式，能最大化数据复用效率。在算子开发中，开发者需要明确算子支持的 format 列表，必要时在 compute 函数中完成格式转换。

属性（Attributes）：属性是在编译期确定的参数，不同于运行时可变的输入张量。典型的属性包括：

- `axis`：规约操作的轴向（如 Softmax 的归一化维度）
- `kernel_size`、`stride`、`padding`：卷积算子的超参数
- `eps`：归一化算子的小常数（防止除零）

属性在算子注册时声明类型与默认值，在模型编译时固化到算子的 JSON 描述中。

算子类型（OpType）：每个算子有唯一的类型名称，采用大驼峰命名，如 `Add`、`Conv2D`、`BatchNorm`。这个名称在算子信息库和 AscendCL 调用中均需保持一致。

6.3.2. 算子信息库（.ini 文件）

算子信息库（Operator Information Library，简称 `.ini` 文件）是 CANN 算子管理体系的核心配置文件。每个自定义算子都需要一个对应的 `.ini` 文件，它的主要作用包括：

1. **声明算子的存在**：告诉 CANN 图引擎” 有哪些自定义算子可用”
2. **描述算子规格**：输入输出的数量、支持的数据类型与 format
3. **关联实现文件**：指定算子对应的编译产物（`.o` 或 `.so` 文件）路径

一个典型的 `.ini` 文件结构如下：

```
[GENERAL]
op_type=Add
op_file=add
op_compute=add_compute

[INPUT]
dtype=float16,float32,int32
format=ND,NHWC,NCHW
shape=dynamic

[OUTPUT]
dtype=float16,float32,int32
```

```
format=ND,NHWC,NCHW
shape=dynamic

[ATTR]
```

各字段含义：- `op_type`：算子类型名称，与代码中注册的名称一致 - `op_file`：算子实现文件（不含扩展名），对应 `kernel_meta/` 下的编译产物 - `op_compute`：计算函数名 - `[INPUT]` / `[OUTPUT]`：声明输入/输出张量支持的数据类型、format 和 shape（`dynamic` 表示不限制形状）- `[ATTR]`：声明算子的属性及其默认值

在 TBE DSL 开发流程中，如果使用 `dsl.build` 编译算子，CANN 会自动生成对应的 `.json` 描述文件，简化了手动编写 `.ini` 的步骤。但在 Ascend C 开发中，手动配置 `.ini` 是必不可少的环节。

6.3.3. Tiling（分片计算）的基本思想

Tiling（分片计算）是算子开发中最重要的性能优化概念之一。它的核心思想很简单：**当数据量超过 AI Core 的 Local Memory 容量时，将数据切成小块，逐块计算**。

为什么需要 Tiling?

回顾第 6.1 节介绍的存储层次: 昇腾 310B 的每个 AI Core 内部, 用于计算的 **Unified Buffer（UB）**容量有限（通常为 256KB 左右）。而 Global Memory 中的输入数据可能有数十 MB。如果一次加载全部数据到 UB，会直接溢出。

Tiling 的策略是：将输入张量按某个维度切分成多个 **Tile（分片）**，每次只搬运一个 Tile 到 UB，计算完后再搬走结果、加载下一片。这就像用一个小碗吃完一大锅饭——一次一碗，吃完再盛。

Tiling 的三个关键参数

设计 Tiling 策略时，需要确定三个数值:

参数	含义	约束
Tile Size	每次搬入 UB 的数据块大小	不能超过 UB 容量；通常需对齐到 32B 或 64B
Tile Count	总共需要切分的片数	由总数据量 / Tile Size 决定
Core Assignment	每个 AI Core 处理哪些 Tile	多核时按 core_id 分配数据区间

一个简单的 Tiling 示例

以向量加法为例，假设：- 输入向量长度 = 10,000,000（约 38MB float32）- UB 容量 = 256KB - 每块最多处理 `256KB / (3 tensors × 4 bytes) 约等于 21,000` 个元素（需要同时放下输入 x、y 和输出 z）

则定一个 Tile 处理 20,000 个元素，总共需要 `10,000,000 / 20,000 = 500` 个 Tile。每个 AI Core 处理其中的一部分 Tile，不同 Core 之间完全并行，互不干扰。

在 TBE DSL 中，`auto_schedule` 内部已经集成了自动 Tiling——开发者通常无需手动指定 Tile 大小。这也是 DSL 便利性的体现之一。但在 Ascend C 中，**Tiling 必须由开发者显式设计和编码**——这构成了下一节的核心挑战。

6.3.4. 小结

本节梳理了算子开发的三大基础设施：**原型定义**规定了算子的" 身份证" 信息（输入输出、类型、属性），**算子信息库**将其注册到 CANN 框架中，而 **Tiling** 则是突破 Local Memory 容量限制的核心技术。这些概念将直接应用于下一节的 Ascend C 实战。

6.4. 深入底层：Ascend C 算子开发入门

Ascend C 是 CANN 提供的 C++ 高性能算子开发语言。与 TBE DSL 的声明式风格不同，Ascend C 要求开发者**亲自设计数据流和指令流**，换取的回报是**逼近硬件理论极限的性能**。本节将以向量加法为起点，带你进入 Ascend C 的世界。

6.4.1. Ascend C 的编程模型

Host-Device 协同

Ascend C 采用经典的 **Host-Device 协同模型**：

- **Host 端（CPU）**：负责算子的入口逻辑——准备数据、调用 Kernel、回收结果。代码运行在 ARM CPU 上。
- **Device 端（NPU）**：负责算子的实际计算——AI Core 执行 Kernel 函数中的计算逻辑。

这种分离意味着开发一个算子需要写两部分代码：一个 Host 侧的主控文件和一个 Device 侧的 Kernel 文件。

SPMD 并行模型

Ascend C 采用 **SPMD（Single Program Multiple Data，单程序多数据）**架构。所有 AI Core 执行**同一份 Kernel 代码**，但通过内置变量 `block_idx` 获取各自的序号，从而处理不同的数据区间。

昇腾 310B 拥有多个 AI Core（具体数量取决于芯片版本），每个 Core 独立运行一份 Kernel。开发者需要利用 `block_idx` 来分配任务：

```
// 伪代码：多核数据分配
uint32_t total_len = 1000000;
uint32_t core_num = 8;
uint32_t len_per_core = total_len / core_num;
uint32_t offset = block_idx * len_per_core;
// 当前 Core 处理 [offset, offset + len_per_core) 的数据
```

三段式流水线

每个 AI Core 内部的执行流程被抽象为三个连续阶段，这也是 Ascend C Kernel 代码的标准结构：

阶段	职责	关键操作
CopyIn	将输入数据从 Global Memory 搬运到 Local Memory（UB）	`DataCopy`, `SetAtomicAdd`
Compute	在 Local Memory 中执行数学运算	`Add`, `Mul`, `Mads`, `ReduceMax`
CopyOut	将计算结果从 UB 搬运回 Global Memory	`DataCopy`

需要注意的是，这三个阶段的粒度是**每个 Tile**——在一个 Tiling 循环中，每个 Tile 都要经历完整的 CopyIn -> Compute -> CopyOut 流程。这恰好对应到上一节 Tiling 概念的实际落地。

Kernel 与 Tiling 的关系

在 Ascend C 中，Kernel 函数本身只描述” 对一块数据的计算逻辑”，而 Tiling 参数（分几块、每块多大）由 Host 端计算好后传给 Kernel。这种分工清晰地分离了**计算逻辑**（Kernel 负责）和**数据切分策略**（Host 端的 Tiling 逻辑负责）。

6.4.2. 工程结构

一个标准的 Ascend C 算子工程包含以下文件：

```
add_ascendc/
```

```
├── CMakeLists.txt          # CMake 构建脚本
├── add_custom.h            # 算子头文件（可选）
├── add_custom.cpp          # Host 侧实现 + Kernel 函数
├── run.cpp                 # 测试验证脚本
├── add_custom.ini          # 算子信息库
└── build/                  # 构建输出目录
```

关键文件说明：- **add_custom.cpp**：包含 Host 侧的入口函数和 Device 侧的 Kernel 函数。Kernel 函数内部实现 CopyIn -> Compute -> CopyOut 流水线。- **CMakeLists.txt**：配置编译选项，指定依赖的 CANN 库和头文件路径。- **add_custom.ini**：算子信息库文件，声明算子接口。

6.4.3. 向量加法的 Ascend C 实现

下面是一个向量加法的 Ascend C Kernel 实现（完整工程请参见 `samples/chapter6/add_ascendc/`）：

```
#include "kernel_operator.h"
using namespace AscendC;

constexpr int32_t BUFFER_NUM = 2;   // 双缓冲

class KernelAdd {
public:
    __aicore__ inline void Init(GM_ADDR x, GM_ADDR y, GM_ADDR z,
                                uint32_t totalLength, uint32_t tileNum) {
        // 多核数据分配
        this->blockLength = totalLength / GetBlockNum();
        this->tileLength = this->blockLength / tileNum / BUFFER_NUM;

        xGm.SetGlobalBuffer((__gm__ float *)x + this->blockLength * GetBlockIdx(),
                            this->blockLength);
        yGm.SetGlobalBuffer((__gm__ float *)y + this->blockLength * GetBlockIdx(),
                            this->blockLength);
        zGm.SetGlobalBuffer((__gm__ float *)z + this->blockLength * GetBlockIdx(),
                            this->blockLength);

        // 初始化双缓冲队列
        pipe.InitBuffer(inQueueX, BUFFER_NUM, this->tileLength * sizeof(float));
        pipe.InitBuffer(inQueueY, BUFFER_NUM, this->tileLength * sizeof(float));
        pipe.InitBuffer(outQueueZ, BUFFER_NUM, this->tileLength * sizeof(float));
    }

    __aicore__ inline void Process() {
        int32_t loopCount = this->tileNum * BUFFER_NUM;
        for (int32_t i = 0; i < loopCount; i++) {
            CopyIn(i);    // 从 Global Memory 搬入 UB
            Compute(i);   // 在 UB 中执行加法
            CopyOut(i);   // 将结果搬回 Global Memory
```

```
        }
    }

private:
    __aicore__ inline void CopyIn(int32_t progress) {
        LocalTensor<float> xLocal = inQueueX.AllocTensor<float>();
        LocalTensor<float> yLocal = inQueueY.AllocTensor<float>();
        DataCopy(xLocal, xGm[progress * this->tileLength], this->tileLength);
        DataCopy(yLocal, yGm[progress * this->tileLength], this->tileLength);
        inQueueX.EnQue(xLocal);
        inQueueY.EnQue(yLocal);
    }
    __aicore__ inline void Compute(int32_t progress) {
        LocalTensor<float> xLocal = inQueueX.DeQue<float>();
        LocalTensor<float> yLocal = inQueueY.DeQue<float>();
        LocalTensor<float> zLocal = outQueueZ.AllocTensor<float>();
        Add(zLocal, xLocal, yLocal, this->tileLength);
        outQueueZ.EnQue<float>(zLocal);
        inQueueX.FreeTensor(xLocal);
        inQueueY.FreeTensor(yLocal);
    }
    __aicore__ inline void CopyOut(int32_t progress) {
        LocalTensor<float> zLocal = outQueueZ.DeQue<float>();
        DataCopy(zGm[progress * this->tileLength], zLocal, this->tileLength);
        outQueueZ.FreeTensor(zLocal);
    }

private:
    TPipe pipe;
    TQue<QuePosition::VECIN, BUFFER_NUM> inQueueX, inQueueY;
    TQue<QuePosition::VECOUT, BUFFER_NUM> outQueueZ;
    GlobalTensor<float> xGm, yGm, zGm;
    uint32_t blockLength = 0, tileNum = 0, tileLength = 0;
};

extern "C" __global__ __aicore__ void add_custom(
    GM_ADDR x, GM_ADDR y, GM_ADDR z, GM_ADDR workspace, GM_ADDR tiling) {
    GET_TILING_DATA(tilingData, tiling);
    KernelAdd op;
    op.Init(x, y, z, tilingData.totalLength, tilingData.tileNum);
    if (TILING_KEY_IS(1)) { op.Process(); }
}
```

（本节中所述的三段式流水线代码与本段样例代码结构相同、逻辑一致。）

这段代码体现了真实 Ascend C 的几个关键特征：

- **类封装**：Kernel 逻辑封装在类中（`KernelAdd`），通过 `Init()` 初始化、`Process()` 执行主循环
- **双缓冲队列**：`TQue` 配合 `EnQue`/`DeQue` 实现了 Double Buffer，CopyIn 和 Compute 可重叠执行

- **多核感知**：`GetBlockNum()`、`GetBlockIdx()` 自动获取当前 AI Core 的编号与总数
- **Tiling 数据**：Host 端计算的 Tiling 参数通过 `GM_ADDR tiling` 传入，`GET_TILING_DATA` 宏将其解包为结构体
- **aicore 属性**：标记所有运行在 AI Core 上的函数，编译器据此生成 DaVinci 指令

6.4.4. 编译与运行

Ascend C 内核通过 TBE 框架的 `compile_op` 接口编译，该接口会自动调用 `ccec` 并设置正确的编译选项。编译命令封装在 Python 算子注册脚本中（参见 `samples/chapter6/add_ascendc/add_custom_tbe.py`），最终生成 `kernel_meta/` 目录下的 `.o` 与 `.json` 文件，随后可通过 AscendCL API 加载并执行。

6.4.5. Ascend C 与 TBE DSL 的对比回顾

经过 Ascend C 的初步体验，我们可以回看第 6.2 节中用 TBE DSL 实现的向量加法：

维度	TBE DSL	Ascend C
语言	Python	C++
代码量	~20 行	~200 行
开发者需要关心的	数学表达式	数据搬运、Tiling、多核分配、指令映射
编译方式	`python add.py` 自动编译	ccec + CMake 手动构建
性能	良好（官方模板保证）	极致（可逼近理论值）
调试方式	TVM debug 模式	CPU 模拟 + printf 打印
适用场景	标准算子、快速原型	性能瓶颈算子、定制化需求

两种方式并非二选一的关系。在实际项目中，**先用 TBE DSL 验证功能正确性，如果性能不达预期，再用 Ascend C 重写核心逻辑**，是最常见的开发流程。

6.5. 从简单到复杂：实现一个需要 Tiling 的算子（如矩阵加法）

上一节的向量加法示例刻意简化了 Tiling 逻辑（Kernel 只处理单块数据）。本节将以**矩阵加法**为例，完整实现一个带 Tiling 的 Ascend C 算子，让读者真正理解分片计算的工程实现。

6.5.1. 场景设定

- 两个矩阵 A 和 B，尺寸均为 `M × N = 1024 × 2048`，float32 类型
- 每个矩阵占用 `1024 × 2048 × 4 = 8MB`
- 昇腾 310B 的 UB 容量为 256KB，显然无法一次性容纳
- 需要设计 Tiling 策略，将矩阵切成小块逐块计算

6.5.2. Tiling 策略设计

分块大小计算

UB 中需要同时存放三块数据：输入 A 的 Tile、输入 B 的 Tile、输出 C 的 Tile。设每个 Tile 大小为 `T` 个 float32（4 字节），则占用的 UB 空间为 `3 × T × 4 = 12T` 字节。考虑到 UB 中还需留出空间给中间变量和指令缓存，一般取 UB 总容量的 60%–80% 作为数据区。

```
UB 数据区上限 约等于 256KB × 70% 约等于 180KB
T_max 约等于 180KB / (3 × 4B) 约等于 15,000 个 float32
```

在实际工程中，为了提高 Cube Unit 的计算效率，Tile 的尺寸通常还需要对齐到 16 或 32 的整数倍。取 `tile_num = 12,288`（即 `16 × 768`）作为一个合适的 Tile 大小。

多核任务分配

假设昇腾 310B 有 8 个 AI Core，每个 Core 独立处理一部分数据。按行切分是最简单的策略：

```
每 Core 处理行数 = M / core_num = 1024 / 8 = 128 行
每 Core 需处理的 Tile 数 = (128 × N) / tile_num = (128 × 2048) / 12288 约等于 22 个 Tile
```

Host 端根据 `block_idx` 计算每个 Core 的数据起始偏移，Kernel 端在 for 循环中逐 Tile 执行 CopyIn -> Compute -> CopyOut。

6.5.3. 带 Tiling 的 Kernel 伪代码

```
extern "C" __global__ __aicore__ void mat_add_with_tiling(
    GM_ADDR a, GM_ADDR b, GM_ADDR c, GM_ADDR tiling_info) {

    TPipe pipe;
    uint32_t block_len = tiling_info.block_len;
    uint32_t tile_num   = tiling_info.tile_num;

    // 根据 block_idx 计算当前 Core 的数据偏移
    uint32_t core_offset = block_idx * block_len * tile_num;
```

```

    for (uint32_t i = 0; i < tile_num; i++) {
        uint32_t offset = core_offset + i * block_len;

        // CopyIn
        LocalTensor<float> tile_a = pipe.AllocTensor<float>(block_len);
        LocalTensor<float> tile_b = pipe.AllocTensor<float>(block_len);
        pipe.DataCopy(tile_a, a + offset);
        pipe.DataCopy(tile_b, b + offset);

        // Compute
        LocalTensor<float> tile_c = pipe.AllocTensor<float>(block_len);
        pipe.Add(tile_c, tile_a, tile_b, block_len);

        // CopyOut
        pipe.DataCopy(c + offset, tile_c);

        pipe.FreeTensor(tile_a);
        pipe.FreeTensor(tile_b);
        pipe.FreeTensor(tile_c);
    }
}
```

这个骨架代码展示了 Tiling 循环的基本结构。完整工程代码（包含 Host 侧 Tiling 参数计算、边界处理、CMakeLists.txt 等）请参见 `samples/chapter6/mat_add_tiling/`。

6.5.4. 调试技巧

Ascend C 提供了两种调试手段，对于排查 Tiling 相关问题尤其有用：

CPU 模拟调试

在开发阶段，可以在 CPU 上模拟运行 Kernel，无需烧录到 NPU：

```
# 以 CPU 模式编译并运行
ccec --cpu_mode add_custom.cpp -o add_cpu
./add_cpu
```

CPU 模拟支持断点调试和变量打印，适合在复杂 Bug 定位时使用。

printf 打印

在 Kernel 中可以使用 `printf` 打印中间值（仅 CPU 模拟和 Debug 模式支持）：

```
printf("block_idx=%d, tile=%d, offset=%d\n", block_idx, i, offset);
```

这在大规模数据中追踪特定 Tile 的行为时非常高效。注意在生产部署的 Release 版本中需要移除或条件编译，避免影响性能。

6.5.5. 小结与展望

本节通过矩阵加法的 Tiling 实战，展示了 Ascend C 开发中最核心的工程挑战：**如何设计合理的分块策略，使得数据既能塞进有限的 UB，又能最大化 AI Core 的计算效率**。同时介绍了 CPU 模拟调试和 printf 打印两种调试手段。

至此，我们已经覆盖了算子开发的核心内容——从 TBE DSL 的快速原型到 Ascend C 的深度优化，从无 Tiling 的简单算子到带分片策略的复杂实现。在下一章中，我们将转入算子开发的工程化环节：编译部署、单元测试以及系统性的性能优化，将自定义算子真正推向生产环境。

7. 性能分析与优化基础

本章不从概念清单开始，而是从几段可以运行的代码开始。读者先建立一个能测量的 ResNet18-TinyImageNet 推理基线，再逐步观察内存申请、数据搬运、CPU 预处理、DVPP 图像处理和流水线并发带来的差异。所有样例代码都放在 `samples/chapter7/`，模型文件可直接从 Hugging Face 仓库 `zhouxzh/resnet18_tiny_imagenet` 下载到本章目录，因此不依赖读者先完成第 4 章。

本章使用的是 Tiny-ImageNet 版 ResNet18，输入为 64x64 RGB 图像，输出为 200 类分类结果。它的绝对 FPS 不能等同于标准 ImageNet 224x224 ResNet18，但内存生命周期、数据搬运、预处理下沉和流水线分析方法是通用的。

7.1. 性能优化先从可测量开始

性能优化的第一步不是改代码，而是把问题拆成可观测的阶段。边缘 AI 应用通常包含以下链路：

```
输入采集 -> CPU/DVPP 预处理 -> Host to Device -> NPU 推理 -> Device to Host -> 后处理 -> 输出
```

本章统一记录以下指标：

指标	含义	适合回答的问题
平均耗时	多次运行的平均阶段耗时	系统总体是否变快
p50	中位数耗时	稳态性能是否符合预期
p95	95 分位耗时	是否存在长尾卡顿
FPS	单位时间处理帧数	吞吐量是否达标
加速比	优化前耗时 / 优化后耗时	单项优化是否值得保留

测试时需要遵守三个规则：

1. 先预热，再计时。模型加载、首次内存申请和缓存建立会污染第一批数据。
2. 一次只改一个变量。比如只改 Buffer 复用，不同时改 Batch 和预处理。
3. 同时看端到端和分阶段耗时。只看 `acl.mdl.execute` 会忽略 H2D、D2H 和 Host 阻塞。

7.2. 第 7 章样例目录

本章代码位于 `samples/chapter7/`：

文件	作用
`01_baseline_resnet_sync.py`	ResNet18 同步推理基线，拆分预处理、H2D、Execute、D2H、后处理
`02_buffer_reuse_benchmark.py`	对比每帧申请 ACL 资源与复用 ACL Buffer/Dataset
`03_cpu_preprocess_benchmark.py`	对比朴素 CPU 预处理和预分配工作区
`04_dvpp_resize_vs_cpu.py`	对比 DVPP VPC resize 与 CPU resize
`05_pipeline_queue_demo.py`	对比串行流程与 Queue Pipeline
`06_aipp_preprocess_compare.py`	对比 CPU 预处理与已有静态 AIPP OM
`tools/download_model.py`	从 Hugging Face 下载 ResNet18-TinyImageNet 的 OM/ONNX 模型
`tools/convert_aipp_resnet18.sh`	调用 ATC 生成静态 AIPP OM
`tools/profile_with_msprof.sh`	使用 `msprof` 采集应用 Timeline
`tools/summarize_metrics.py`	将 JSON 指标汇总成 Markdown 表格
`model/resnet18_rgb_static_aipp.cfg`	静态 AIPP 配置示例
`model/`	第 7 章模型和 AIPP 配置目录

阅读本章时，建议先进入第 7 章样例目录，后续命令都默认在这个目录下执行：

```
cd samples/chapter7
```

本地开发机可以运行语法检查和 CPU/模拟样例。以下命令中的 `python3` 指当前已经安装 NumPy 等依赖的 Python 解释器：

```
python3 -m py_compile *.py tools/*.py
python3 03_cpu_preprocess_benchmark.py --runs 10
python3 05_pipeline_queue_demo.py --simulate --frames 30
```

真实 PyACL、DVPP、`msprof` 和 `npu-smi` 验证必须在 Ascend 310B 上运行：

```
source /usr/local/Ascend/ascend-toolkit/set_env.sh
cd ~/Documents/Ascend310/samples/chapter7
python3 tools/download_model.py
python3 01_baseline_resnet_sync.py --runs 100
```

7.3. 工具箱：从轻量计时到系统 Profiling

7.3.1. Python 分段计时

`perf_counter()` 适合做应用层粗粒度计时。第 7 章的样例通过 `StageRecorder` 记录每一帧的阶段耗时，并输出 JSON：

```
{
  "metrics": {
    "h2d": {"mean_ms": 0.12, "p95_ms": 0.18, "fps": 8333.33},
    "execute": {"mean_ms": 1.85, "p95_ms": 2.10, "fps": 540.54},
    "end_to_end": {"mean_ms": 3.40, "p95_ms": 4.20, "fps": 294.12}
  }
}
```

这种计时不能替代硬件 Profiling，但它有两个优势：代码改动小，能快速判断优化是否有方向。

7.3.2. msprof：观察 ACL API、Memcpy 和 NPU Timeline

`msprof` 是 CANN 提供的性能采集工具。它能记录 Host 侧 ACL API 调用、Runtime 任务、Stream Timeline、AI Core 指标和部分系统资源信息。

本章提供包装脚本：

```
bash tools/profile_with_msprof.sh --name baseline -- \
  python3 01_baseline_resnet_sync.py --runs 200
```

采集后重点观察：

视图	观察点	常见结论
ACL API	`acl.mdl.execute` 调用间隔是否很大	Host 预处理或同步等待过慢
Memcpy	H2D/D2H 是否占比过高	输入输出过大或拷贝次数太多
Stream Timeline	NPU 计算任务之间是否有大片空白	NPU 在等待 Host 下发任务或等待数据
AI Core Metrics	Pipe 利用率是否低	模型算子、Batch 或输入尺寸需要继续分析

7.3.3. npu-smi：确认设备状态

`npu-smi info` 用于确认设备是否在线、温度是否异常、功耗是否接近限制。性能测试前后都应记录一次，避免把降频、设备异常或其他进程占用误判为代码问题。

```
npu-smi info
```

7.3.4. msame / ais_bench：独立测模型上限

当应用端到端 FPS 不理想时，可以用 `msame` 或 `ais_bench` 单独测 OM 模型。它们绕过业务代码，只关注模型输入输出和 Runtime 执行，适合回答一个问题：模型本身的推理上限是多少?

如果基准工具显示模型很快，而业务程序很慢，瓶颈通常在预处理、拷贝、后处理或调度。如果基准工具也很慢，才继续考虑模型结构、算子、精度模式和 Batch。

例如，在已经进入 `samples/chapter7` 并下载模型后，可以用 `ais_bench` 测普通 OM 的独立推理耗时：

```
ais_bench --model model/resnet18_tiny_imagenet.om --batchsize 1
```

不同 CANN 版本和安装方式下，`ais_bench` 或 `msame` 的命令行参数可能略有差异；如果本机未安装该工具，可以跳过本节，继续使用本章 Python 分段计时脚本观察端到端瓶颈。

7.3.5. ATC/AIPP 与 DVPP：把图像工作下沉到设备侧

AIPP 能在模型输入侧完成颜色格式处理和归一化，减少 Host CPU 的像素遍历。示例配置位于：

```
model/resnet18_rgb_static_aipp.cfg
```

如果只复现实验结果，先下载本章模型资产：

```
python3 tools/download_model.py --all
```

这个命令会把普通 OM、ONNX、AIPP OM 和 AIPP 配置放到 `model/`。如果需要自己体验离线转换，应在 Ascend 310B 上重新生成 AIPP OM：

```
bash tools/convert_aipp_resnet18.sh
```

DVPP 适合处理视频编解码、JPEG 编解码、VPC resize/crop 等媒体任务。第 5 章已经介绍 DVPP API，本章只关心它在性能优化中的角色：减少 CPU 图像处理负担，并让数据尽量早地进入设备侧流水线。

7.4. 案例一：建立同步推理基线

先运行基线：

```
python3 01_baseline_resnet_sync.py --runs 100
```

这个脚本故意采用低效写法：每一帧都申请 Device 输入输出内存、创建 Dataset、执行推理、复制输出、释放资源。它的价值不是快，而是让问题暴露出来。

核心流程如下：

```
input_ptr = acl.rt.malloc(input_size, ACL_MEM_MALLOC_HUGE_FIRST)
output_ptr = acl.rt.malloc(output_size, ACL_MEM_MALLOC_HUGE_FIRST)
input_dataset = acl.mdl.create_dataset()
output_dataset = acl.mdl.create_dataset()
acl.rt.memcpy(input_ptr, input_size, host_ptr, input_size, ACL_MEMCPY_HOST_TO_DEVICE)
acl.mdl.execute(model_id, input_dataset, output_dataset)
acl.rt.memcpy(host_out, output_size, output_ptr, output_size, ACL_MEMCPY_DEVICE_TO_HOST)
```

基线需要记录以下阶段：

阶段	含义
`preprocess`	RGB 输入转换为 NCHW float32
`alloc_dataset`	申请输入输出 Buffer 并创建 Dataset
`h2d`	Host 输入复制到 Device
`execute`	模型执行
`d2h`	Device 输出复制回 Host
`free_dataset`	销毁 Dataset 并释放 Buffer
`postprocess`	`argmax` 等轻量后处理
`end_to_end`	单帧端到端耗时

如果 `alloc_dataset` 和 `free_dataset` 占比明显，说明优化方向不是模型，而是资源生命周期。

7.5. 案例二：复用 ACL Buffer 和 Dataset

推理服务通常会长时间运行，输入输出张量大小也是固定的。因此更合理的做法是在模型加载后一次性创建输入输出 Buffer 和 Dataset，在每帧循环里只做数据拷贝和执行：

```
python3 02_buffer_reuse_benchmark.py --runs 100
```

优化后的流程：

```
初始化阶段: load model -> create dataset -> malloc input/output buffer
```

```
循环阶段: H2D -> execute -> D2H -> postprocess
退出阶段: destroy dataset -> free buffer -> unload model
```

这项优化的判断标准很直接：

1. `alloc_dataset` 和 `free_dataset` 从每帧计时中消失。
2. `end_to_end` 平均耗时下降。
3. `msprof` Timeline 中 Host 侧 ACL Runtime 调用密度更高，计算任务之间的空白减少。

如果模型很小，Buffer 复用的收益会特别明显；如果模型本身很大，收益占比会下降，但仍然能减少长尾抖动。

7.6. 案例三：CPU 预处理为什么容易成为瓶颈

很多边缘视觉应用在 NPU 推理前会做 resize、颜色转换、归一化和 HWC 到 CHW 转换。脚本 `03_cpu_preprocess_benchmark.py` 用确定性 RGB 图像模拟 1080p 摄像头帧：

```
python3 03_cpu_preprocess_benchmark.py --runs 100 --resolution 1920x1080
```

如果想观察更稳定的统计结果，可以在确认本机内存充足后把 `--runs` 增加到 500 或 1000。

脚本比较两种方式：

方式	特点
`naive_numpy`	每帧创建 resize、float32、NCHW 等中间数组
`workspace_reuse`	预先创建工作区，循环中复用数组

这个案例的学习重点不是把 CPU 预处理调到极致，而是理解何时应该停止优化 CPU。若预处理耗时已经接近或超过模型执行耗时，应优先考虑：

1. 使用 DVPP VPC 做 resize/crop。
2. 使用 AIPP 做归一化和部分颜色格式处理。
3. 使用视频链路时，让 VDEC/JPEGD 的输出直接衔接 VPC 或模型输入。

7.7. 案例四：DVPP VPC resize 与 CPU resize

脚本 `04_dvpp_resize_vs_cpu.py` 生成 NV12 测试帧，将 1080p 图像缩小到 540p，对比 CPU resize 和 DVPP VPC resize：

```
python3 04_dvpp_resize_vs_cpu.py --frames 60 --resolution 1920x1080
```

该脚本包含 H2D、VPC resize、同步和 D2H，因此测到的是一个完整调用成本，而不是 VPC 内核的裸耗时。如果 DVPP 仍然明显领先 CPU，说明把 resize 从 Host 下沉到设备侧很有价值。如果差距不明显，要继续检查：

1. 输入尺寸是否太小，导致调用开销大于计算收益。
2. 是否每帧都重复创建 DVPP Channel、Stream 或配置对象。
3. 是否存在多余的 H2D/D2H 拷贝。

7.8. 案例五：AIPP 把归一化下沉到模型输入侧

前面的基线脚本使用 CPU 完成 resize、归一化和 HWC 到 CHW 转换，然后把 `float32 NCHW` 输入拷贝到 Device。AIPP 的思路不同：让模型接收 `RGB888_U8` 图像，在模型输入侧完成均值、方差等处理。这样可以减少 Host CPU 的像素遍历，也能减少输入拷贝字节数。

本章提供静态 AIPP 配置：

```
model/resnet18_rgb_static_aipp.cfg
```

先下载本章模型资产：

```
python3 tools/download_model.py --all
```

`--all` 会把普通 OM、ONNX、AIPP OM 和 AIPP 配置都下载到 `model/`。如果 Hugging Face 仓库中已经有 `model/resnet18_tiny_imagenet_aipp.om`，可以直接运行后面的对比脚本，不必自己转换。

如果需要在板端重新生成 AIPP OM，再运行独立 shell 脚本：

```
bash tools/convert_aipp_resnet18.sh
```

脚本默认生成 `model/resnet18_tiny_imagenet_aipp.om`。转换或下载完成后，再运行 Python 对比脚本：

```
python3 06_aipp_preprocess_compare.py --runs 100
```

这里故意把 ATC 转换和 Python 推理对比分成两步。ATC 是离线模型转换，可能耗时较长，也可能受板端内存和 CANN 环境影响；Python 脚本只加载已有 OM 做性能测试，不会在测试过程中调用 ATC。

对比脚本会执行两条路径：

1. 使用原始 OM 跑 `CPU preprocess -> H2D -> execute -> D2H -> argmax`。
2. 使用 AIPP OM 跑 `RGB888_U8 prepare -> H2D -> AIPP+execute -> D2H -> argmax`，并比较输出差异。

需要注意，AIPP OM 的输入不再是 `float32 NCHW`，而是 `uint8 HWC RGB`。因此不能直接拿 `01_baseline_resnet_sync.py` 去跑 AIPP OM，必须使用本节的对比脚本或在自己的程序中按 AIPP 配置准备输入。

对外部读者来说，还有两个容易踩坑的地方：

1. 运行转换脚本前，需要先完成 CANN 环境配置，并确认当前终端可以直接使用 `atc`。
2. CANN 8.3 对 `RGB888_U8` 的 `mean_chn_*` 字段按整数解析，不能直接写 `123.675` 这样的浮点均值。本章配置使用 `mean_chn_*: 0` 加浮点 `min_chn_*` 表达 ImageNet 均值，并用 `var_reci_chn_*` 表达 `1 / (255 * std)`。

ATC 是离线转换步骤，第一次编译可能明显慢于普通推理脚本。如果转换耗时异常，应先检查板端负载、可用内存、CANN 日志和 AIPP 配置，再重新运行转换脚本。

7.9. 案例六：串行流程与 Queue Pipeline

串行流程的单帧耗时为：

```
T = T_pre + T_infer + T_post
```

三段流水线稳定后的吞吐上限更接近：

```
FPS = 1000 / max(T_pre, T_infer, T_post)
```

先在本地使用模拟模式理解这个关系：

```
python3 05_pipeline_queue_demo.py --simulate --frames 200
```

再在 Ascend 310B 上使用 PyACL 模式：

```
python3 05_pipeline_queue_demo.py --frames 100
```

流水线优化的目标不是降低单帧端到端延迟，而是提高稳态吞吐。它会引入队列等待，因此 p95 latency 可能上升；如果业务目标是交互式低延迟，需要谨慎扩大队列深度。

7.10. 作者实测结果

本节记录作者在一台 Ascend 310B 设备上得到的实测数据。读者自己的板端温度、CANN 版本、设备健康状态和后台进程不同，绝对数值会有差异；更重要的是复现实验方法和观察相对趋势。

项目	内容
测试主机	`313`，作者内部测试主机编号
设备名	`orangepiaipro`
远端仓库	`/home/HwHiAiUser/Documents/Ascend310`
CANN 路径	`/usr/local/Ascend/ascend-toolkit/latest`
Python	`/home/HwHiAiUser/.conda/envs/npu/bin/python`
测试日期	`2026-06-08`
设备状态	`npu-smi` 显示 310B4，Health 为 `Alarm`，温度约 61 C，测试前后内存约 2939-3003 / 7545 MB

以下表格由 `samples/chapter7/tools/summarize_metrics.py` 汇总远端 JSON 指标生成。由于测试时设备状态为 `Alarm`，数据更适合作为章节教学样例和相对趋势参考；正式项目验收应在设备健康状态正常、无其他进程干扰时重新跑完整评测。

案例	变体	平均耗时 (ms)	p95(ms)	FPS	样本数	加速比	说明
ResNet18 同步推理	`naive_alloc_each_frame` ↪	4.5088	4.712	221.791	100		每帧申请/释放 ACL 输入输出 Buffer 和 Dataset
ResNet18 Buffer 复用	`naive_alloc_each_frame` ↪	6.0047	6.8023	217.1708	100		对照组
ResNet18 Buffer 复用	`reuse_acl_buffers` ↪	3.2072	3.3509	311.7963	100	1.436	模型加载后一次性创建输入输出 Buffer 和 Dataset
CPU 预处理	`naive_numpy` ↪	0.62	0.6821	1612.8023	200		每帧创建中间数组
CPU 预处理	`workspace_reuse` ↪	0.3583	0.4154	2791.1275	200	1.731	复用 resize/CHW 工作区

案例	变体	平均耗时 (ms)	p95(ms)	FPS	样本数	加速比	说明
DVPP VPC Resize	cpu_cv2_resize	2.6013	2.6659	384.4225	60		CPU resize NV12 Y/UV
DVPP VPC Resize	dvpp_vpc_resize	3.5966	3.7102	278.0433	60	0.723	DVPP VPC resize + H2D/D2H
AIPP 预处理下沉	cpu_preprocess	3.2319	3.3155	309.4136	100		CPU 完成 resize/normalize/HWC>CHW 后推理
AIPP 预处理下沉	static_aipp_rgb	2.6076	2.6283	383.5017	100	1.239	AIPP 接收 RGB888_U8 并在模型输入侧归一化
Queue Pipeline	pyacl_serial	3.1943	3.2332	313.0536	100		Pre -> Infer -> Post 串行
Queue Pipeline	pyacl_queue_pipeline	3.993	100.0494	250.4402	100	0.8	三段线程通过 Queue 解耦
Queue Pipeline	simulate_serial	15.1757	15.1942	65.8947	200		Pre -> Infer -> Post 串行
Queue Pipeline	simulate_queue_pipeline	8.1233	157.4201	123.1021	200	1.868	三段线程通过 Queue 解耦

从这组数据可以得到五个结论：

1. Buffer 复用是确定有效的优化。ResNet18 端到端平均耗时从 4.6047 ms 降到 3.2072 ms，提升约 1.436 倍。
2. CPU 预处理里的中间数组分配值得优化。复用工作区后，预处理平均耗时从 0.62 ms 降到 0.3583 ms。
3. DVPP 不是简单替换 API 就一定更快。本章脚本为了教学完整性包含每帧 dvpp_malloc /free、H2D 和 D2H，结果 DVPP VPC resize 平均 3.5966 ms，慢于 CPU resize

的 2.6013 ms。真实视频流水线应复用 DVPP 资源，并尽量让 VDEC/JPEGD/VPC 输出继续留在设备侧。

4. AIPP 在本章模型上有明确收益。原始 OM 的 CPU 预处理路径平均耗时为 3.2319 ms，AIPP OM 的 `RGB888_U8` 输入路径降到 2.6076 ms，提升约 1.239 倍；100 次测试中 top1 匹配率为 1.0，最大单点输出差异为 0.007812。AIPP 改变了模型输入格式，正确的比较对象不是 “同一个 OM 换输入”，而是原始 OM 的 CPU 预处理路径与 AIPP OM 的 `RGB888_U8` 输入路径。
5. Pipeline 需要足够重的阶段才有收益。模拟场景中队列流水线从 65.8947 FPS 提升到 123.1021 FPS；但 ResNet18 小模型的 PyACL 场景中，队列和线程上下文开销使 FPS 从 313.0536 降到 250.4402，且 p95 latency 明显变差。

本次 `msprof` 采集输出目录如下，可用 MindStudio 或 Ascend Insight 打开：

```
outputs/msprof/baseline-remote-latest-20260608-083957/
    ↪ PROF_000001_20260608083959526_RHNMOCNRMNBBHBCC/mindstudio_profiler_output
```

远端验证命令：

```
source /usr/local/Ascend/ascend-toolkit/set_env.sh
cd /home/HwHiAiUser/Documents/Ascend310/samples/chapter7
PY=/home/HwHiAiUser/.conda/envs/npu/bin/python
$PY -m py_compile *.py tools/*.py
$PY tools/download_model.py
$PY 01_baseline_resnet_sync.py --runs 100 --output outputs/
    ↪ baseline_resnet_sync_remote_latest.json
$PY 02_buffer_reuse_benchmark.py --runs 100 --output outputs/buffer_reuse_remote_latest.
    ↪ json
$PY 03_cpu_preprocess_benchmark.py --runs 200 --output outputs/
    ↪ cpu_preprocess_remote_latest.json
$PY 04_dvpp_resize_vs_cpu.py --frames 60 --output outputs/dvpp_resize_remote_latest.json
$PY 05_pipeline_queue_demo.py --simulate --frames 200 --output outputs/
    ↪ pipeline_simulate_remote_latest.json
$PY 05_pipeline_queue_demo.py --frames 100 --output outputs/pipeline_pyacl_remote_latest.
    ↪ json
$PY tools/download_model.py --all
$PY 06_aipp_preprocess_compare.py --runs 100 --output outputs/
    ↪ aipp_preprocess_remote_latest.json
$PY tools/summarize_metrics.py \
  outputs/baseline_resnet_sync_remote_latest.json \
  outputs/buffer_reuse_remote_latest.json \
  outputs/cpu_preprocess_remote_latest.json \
  outputs/dvpp_resize_remote_latest.json \
  outputs/aipp_preprocess_remote_latest.json \
  outputs/pipeline_pyacl_remote_latest.json \
  outputs/pipeline_simulate_remote_latest.json \
  --output outputs/summary_remote_latest.md
npu-smi info
```

Profiling 采集命令:

```
bash tools/profile_with_msprof.sh --name baseline-remote-latest -- \
  /home/HwHiAiUser/.conda/envs/npu/bin/python 01_baseline_resnet_sync.py \
    --runs 20 --output outputs/baseline_msprof_remote_latest.json
```

7.11. 调优 Checklist

现象	优先检查	常见优化
NPU Timeline 有大量空白	Host 是否慢、是否同步等待	预处理下沉、异步流水线、减少全局同步
H2D/D2H 占比高	输入输出大小和拷贝次数	合并 Buffer、复用内存、减少回传输出
每帧耗时抖动大	是否每帧 malloc/free	预分配 Buffer、固定 Dataset
CPU 占用高	resize/cvtColor/normalize 是否在 CPU	DVPP、AIPP、C++/NumPy 工作区复用
模型单独基准也慢	算子或模型结构	改输入尺寸、精度模式、Batch、算子优化
Pipeline FPS 不升反降	队列太深或阶段不均衡	缩小队列、调整线程数、重新划分阶段

7.12. 练习任务

1. 把 `01_baseline_resnet_sync.py` 的 `--runs` 从 100 增加到 1000，对比 p95 是否稳定。
2. 用 `msprof` 分别采集基线和 Buffer 复用脚本，找出 Timeline 中 Host 空白是否减少。
3. 修改 `05_pipeline_queue_demo.py --simulate` 中的 `--pre-ms`、`--infer-ms`、`--post-ms`，验证吞吐是否接近最慢阶段。
4. 将自己的模型接入 `ReuseResNetRunner` 的思路，记录优化前后的 JSON 指标。
5. 运行 `tools/download_model.py --all` 获取 AIPP OM,或使用 `tools/convert_aipp_resnet18.sh` 重新生成 AIPP OM，再运行 `06_aipp_preprocess_compare.py` 比较 CPU 归一化下沉前后的端到端耗时和输出差异。

7.13. 小结

Ascend 310B 性能优化的核心是先定位瓶颈，再把瓶颈映射到合适的硬件或软件策略。小模型场景中，内存申请、同步和 Host 预处理往往比模型执行更值得关注；视频场

景中，DVPP、AIPP 和流水线调度决定了端到端吞吐。调优的最终依据不是经验判断，而是可复现的指标、Timeline 和对比报告。

8. 模型量化与精度性能权衡

一条推理应用变慢时，首先要拆开看时间花在哪里：是图片解码慢，预处理慢，主机与 NPU 之间的数据拷贝慢，模型在 NPU 上执行慢，还是后处理、显示和网络传输慢。只有当模型执行已经成为主要瓶颈时，量化才真正进入工程决策。

为什么是量化？原因首先来自业务需求。实时目标检测、实时手势识别、人脸识别打卡这类边缘 AI 应用通常要求稳定帧率。例如摄像头输入是 30 FPS，单帧从采集到结果显示就不能长期超过 33 ms；如果还要叠加视频编码、网络传输和 UI 绘制，留给模型推理的时间会更短。此时如果 FP32 模型执行时间过长，单纯调整 Python 代码已经不够，就需要考虑让模型以更适合芯片的低精度形式运行。

从 Ascend 310B 的硬件定位看，它是面向边缘推理的 AI 处理器，AI Core 更适合执行低精度张量计算。相比 FP32，FP16 的数据宽度更小，访存压力更低，在 310B 上通常是更常用的推理精度；相比 FP16，INT8 进一步把权重和激活压缩到 8 位整数，理论上可以带来更高吞吐和更低内存带宽占用。也就是说，量化不是一个 “文件格式转换技巧”，而是把模型数值表示方式调整到芯片更擅长的计算路径上。

但低精度并不是免费的。FP16 可能带来轻微数值误差，INT8 还需要通过校准数据估计量化比例和零点。低精度模型如果处理不好，虽然速度上去了，Top1/Top5 精度、边界样本排序和业务可用性都可能下降。因此本章的核心问题不是 “怎样把模型转成 INT8”，而是：

1. 在什么性能场景下需要考虑量化。
2. 为什么 Ascend 310B 上先看 FP16，再看 INT8。
3. 如何建立 FP32 基准，并公平比较 FP32、FP16、INT8。
4. 如何判断 “输出接近” 与 “验证集精度不下降”。
5. 在 INT8 精度不稳定时，如何观察校准样本数量对 PTQ 的影响。

本章选择一个已经训练好的 ResNet18-TinyImageNet FP32 ONNX 模型作为教学对象，用它完整演示从 FP32 ONNX 到 FP32/FP16/INT8 OM 的量化评估过程。代码只是承载这条教学主线的参考实现；真正要学的是量化的判断逻辑、验证方法和代码设计原则。其中 ATC、AMCT、PyACL 和 OM 推理相关命令必须在真实 Ascend 310B 开发板上执行；本地开发机只适合阅读代码、修改文档和做语法检查。

模型量化的主流程不是一开始就冲向 INT8，而是先建立 FP32 基准，再验证 FP16，最后在需要进一步提升吞吐时引入 INT8 PTQ。完整验证集横向对比是主流程的收口；校

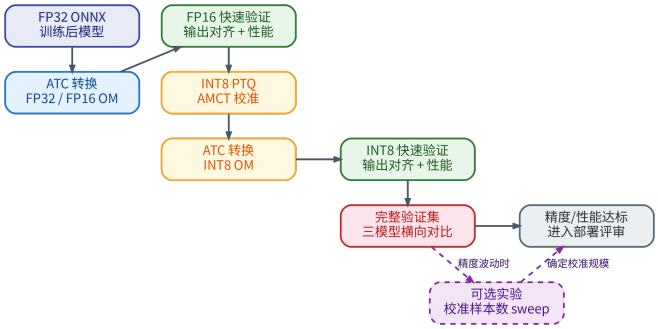


图 8.1. 模型量化总流程

准样本数 sweep 只是可选实验，只有当 INT8 精度波动、精度不达标，或者需要解释校准样本数量对结果的影响时才需要做。

这张图可以按三层来读。第一层是基准建立：从训练后得到的 FP32 ONNX 出发，用 ATC 分别生成 FP32 OM 和 FP16 OM。FP32 OM 是后续所有比较的精度基准，FP16 OM 则是第一个低精度优化对象。这里要先保证转换参数、输入 shape 和预处理口径固定下来，否则后面的差异就无法归因到精度变化。

第二层是快速验证。FP16 生成后，不应该直接进入完整验证集，而是先用少量图片做输出对齐和性能对比：输出对齐检查候选模型是否仍然“像” FP32 基准模型，性能对比检查低精度是否真的降低了推理耗时。只有 FP16 的功能和性能都可信，才继续尝试 INT8。

第三层是 INT8 PTQ 和最终验证。INT8 需要 AMCT 使用校准图片估计激活范围，再把校准后的 deploy ONNX 交给 ATC 转成 INT8 OM。INT8 OM 也要先做快速输出/性能验证，最后再用完整验证集把 FP32、FP16、INT8 放在同一把尺上比较 Top1、Top5 和推理耗时。如果这个横向对比已经满足项目目标，就可以进入部署评审；如果 INT8 精度不稳定、不达标，或者需要解释校准数据规模对精度的影响，再进入可选的校准样本数 sweep。

8.1. 从性能瓶颈到量化决策

在实际项目中，量化经常被误用成“只要模型慢，就先转 INT8”。这并不稳妥。量化优化的是模型计算和一部分模型相关的数据搬运。如果端到端瓶颈在摄像头采集、图片解码、CPU 预处理、主机与 NPU 之间的数据拷贝、后处理或视频推流，量化模型本身不会带来根本改善。

因此量化决策应该按图中的顺序进行：

第一步是拆解端到端耗时。一个实时应用从摄像头采集到结果显示，中间会经过图片解码、CPU 预处理、主机与 NPU 之间的数据拷贝、模型执行、后处理和视频传输等环节。如果主要时间花在预处理、数据拷贝、后处理或显示传输上，优先工作就不是量化，而是

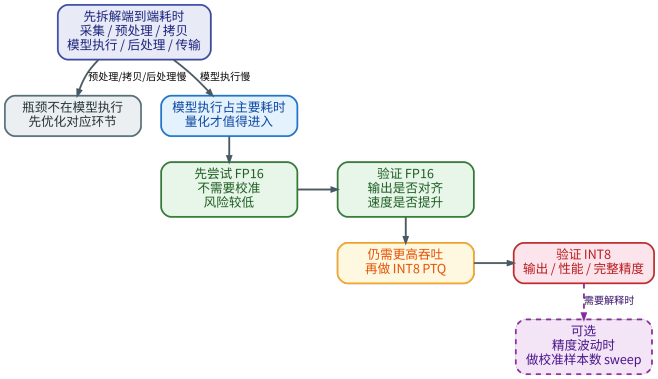


图 8.2. 量化实验的判断顺序

优化对应环节。例如 resize 和归一化太慢，就应该考虑 AIPP、向量化或流水线；输出拷贝占比过高，就应该检查输出 tensor 尺寸和 buffer 复用；后处理太慢，则应优化 NMS、绘制或编码。

第二步才是判断模型执行是否占主要耗时。只有当 NPU 上的模型前向计算已经成为瓶颈，低精度计算才可能带来明显收益。此时不要一开始就做 INT8，而是先尝试 FP16。FP16 通常不需要校准，转换链路更短，精度风险也低，适合用来判断当前模型是否能从低精度计算中获益。

第三步是验证 FP16。验证不能只看模型是否成功转换，还要检查两件事：候选模型的输出是否仍然接近 FP32 基准模型，推理耗时是否确实下降。如果 FP16 输出已经明显漂移，就没有必要继续做 INT8；如果 FP16 正确但性能仍达不到实时帧率或吞吐目标，再进入 INT8 PTQ。

第四步是验证 INT8。INT8 需要校准数据估计激活范围，因此精度风险比 FP16 更高。完成 INT8 OM 后，也要先做小样本输出对齐和性能对比，再用完整验证集比较 FP32、FP16、INT8 的 Top1、Top5 和推理耗时。只有功能证据、性能证据和精度证据同时成立，量化结果才可以进入部署评审。

最后，校准样本数 sweep 不是主流程的必做步骤。只有当 INT8 精度不达标、不同校准集导致结果波动，或者需要解释 “多少张校准图片已经足够” 时，才需要进一步做 sweep 实验。

本章所有结论都围绕同一个原则：每次改变模型精度，都必须同时保留功能证据、性能证据和精度证据。

8.2. 必备理论：FP32、FP16、INT8 与 PTQ

FP32、FP16 和 INT8 的区别不只是 “数字位数不同”。它们影响模型权重、激活值、算子选择、内存带宽和最终预测排序。

FP32 是本章的精度基准。它使用 32 位浮点表示权重和激活，数值范围和表达精度都比较充足，因此最适合作为 “原始模型应该输出什么” 的参考。后续 FP16 和 INT8 的判断都不能脱离这个基准：候选模型可以更快，但不能在分类结果上出现不可接受的漂移。

FP16 是第一阶段优化。它仍然是浮点数，只是把每个数从 32 位压缩到 16 位。对许多卷积神经网络推理任务来说，FP16 的数值误差通常比 INT8 更容易控制，而且不需要额外校准数据。因此在 Ascend 310B 上部署模型时，常见做法是先验证 FP16：如果 FP16 已经满足帧率和精度要求，就没有必要急着进入 INT8。

INT8 是第二阶段优化。它把权重和激活映射到 8 位整数，能进一步降低访存和计算压力，但它不再是简单的浮点精度缩短，而是需要确定每层的量化比例和零点。这个比例来自校准数据，所以 INT8 的收益和风险都更明显：速度可能更快，精度也更依赖校准集是否代表真实输入分布。

从芯片执行角度看，低精度通常能带来三类收益：

1. **计算吞吐更高**：AI Core 对低精度矩阵和卷积计算更友好，FP16 通常比 FP32 更适合 310B 推理，INT8 又进一步面向整数计算路径。
2. **内存带宽压力更小**：同样数量的权重和激活，FP16 只需要 FP32 一半的数据宽度，INT8 只需要 FP32 四分之一的数据宽度。
3. **缓存和搬运更友好**：模型参数和中间激活变小后，片上/片外数据搬运压力下降，对边缘设备这种内存和功耗都受限的平台尤其重要。

这些收益不是自动等于端到端加速。如果模型很小，或者 CPU 预处理、数据拷贝、后处理占主导，量化后的模型执行虽然变快，应用整体 FPS 也可能变化不大。因此本章后面不仅看模型推理耗时，也会看端到端耗时和完整验证集精度。

INT8 量化的核心思想是把浮点数映射到整数区间。可以粗略理解为：

$$q = \text{round}\left(\frac{x}{s}\right) + z$$

$$x \approx (q - z) \times s$$

其中，x 是原始浮点数，q 是量化后的整数，s 是量化比例（scale），z 是零点（zero-point）。这里的量化比例和零点不是凭空指定的。对于权重，工具可以直接扫描模型参数；对于激活值，必须用一批代表性输入跑一遍模型，估计每层激活分布。这个过程就是校准。校准图片如果不代表真实输入分布，就可能出现两类问题：

1. 激活范围估计太窄，真实输入中的大值被截断，精度明显下降。
2. 激活范围估计太宽，常见数值区域的量化分辨率变粗，分类边界样本排序漂移。

本章使用的是 PTQ，也就是 Post-Training Quantization。PTQ 不重新训练模型，只在训练后用校准数据估计量化参数。它适合部署阶段快速评估。如果 PTQ 后精度下降不

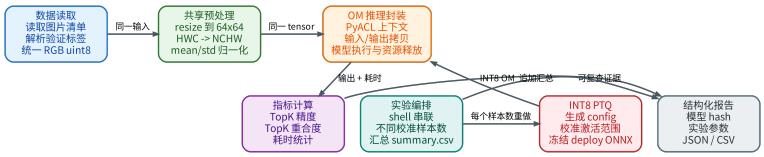


图 8.3. 量化验证代码的职责边界

可接受，再考虑 QAT（Quantization-Aware Training），也就是训练时模拟量化误差并重新微调模型。

8.3. 如何设计量化验证代码：先拆职责，再写脚本

写量化案例代码时，最忌讳把下载数据、图片预处理、模型推理、计时、accuracy 统计、AMCT 校准和报告写入全部塞进一个大脚本。那样虽然第一次能跑通，但只要换模型、换输入尺寸或换校准样本数量，就很难判断问题来自哪里。

更稳妥的写法是先拆出清晰的职责边界。图中每个框都只做一类事情，框与框之间只传递必要的数据，而不是互相知道彼此的实现细节。

最前面的边界是数据读取。它只负责读取图片清单、解析验证标签，并把图片统一转成 RGB `uint8` 数组。这里不应该做模型推理，也不应该计算 accuracy。这样做的好处是，后面无论验证 FP16、INT8，还是做校准样本数 sweep，都能复用同一套图片读取逻辑。

第二个边界是共享预处理。它把 RGB 图片 resize 到模型输入尺寸，完成 HWC 到 NCHW 的 layout 转换，并按训练时的 mean/std 做归一化。这个模块必须只写一份，不能因为模型是 FP32、FP16 或 INT8 就写三套预处理。量化对比要公平，最基本的前提就是三个模型吃到完全相同的输入 tensor。

第三个边界是 OM 推理封装。它负责 PyACL 上下文、模型加载、输入拷入 NPU、模型执行、输出拷回主机和资源释放。它不关心预测是否正确，也不关心 Top1/Top5 怎么算。把硬件运行细节封装起来后，后面的输出对齐、性能对比和完整验证都可以用同一种 runner 调用方式。

第四个边界是指标计算。它只接收模型输出和标签，计算 TopK 精度、TopK 重合度、输出误差和耗时统计。这个模块不应该读取图片，也不应该管理 Ascend 资源。这样指标代码可以独立测试，也能避免 “推理出错” 和 “统计口径出错” 混在一起。

第五个边界是结构化报告。实验结果不能只打印在终端里，而要把模型路径、模型 hash、样本数量、输入清单、精度、耗时和少量样本明细写成 JSON 或 CSV。只有报告可复查，后续才可以比较 FP32、FP16、INT8，或者回头解释某次 sweep 为什么出现波动。

图中的 INT8 PTQ 和实验编排是两个补充边界。INT8 PTQ 负责生成量化配置、用校准图统计激活范围并冻结 deploy ONNX；实验编排只负责把不同样本数的校准、转换、

验证串起来，不把 Python 数据处理逻辑塞进 shell。这样主线代码不会被 sweep 实验拖乱，sweep 也能复用前面已经写好的数据、预处理、推理和报告模块。

按照这个边界，代码的骨架可以先写成下面这样：

```
paths = read_image_list(list_file)
models = [Runner(path) for path in om_models]
metrics = {name: MetricState() for name in names}

for image_path, label in paths:
    frame = load_rgb_frame(image_path)
    input_tensor = preprocess_resnet_rgb(frame)
    for name, runner in zip(names, models):
        logits, timing = runner.infer(input_tensor)
        metrics[name].add(logits, label, timing)

write_report(output_path, metrics)
```

这个骨架背后的关键思想是：输入路径只走一次，预处理只写一份，多个 OM 只在 `runner.infer()` 这一层发生差异。这样得到的 FP32、FP16、INT8 对比才是公平的。

本章的参考实现可以在 GitHub 仓库中查看：samples/chapter8。

8.4. 如何编写数据准备代码：让输入可复现

先在开发板上加载 CANN 环境，激活 Python 环境，并进入样例目录：

```
source /usr/local/Ascend/ascend-toolkit/set_env.sh
conda activate npu
cd ~/Documents/Ascend310/samples/chapter8
```

对于 8GB 及以内内存的 Ascend 310B 开发板，运行 ATC 前建议一定加上下面的并行编译限制，避免转换过程中因为内存不足失败。12GB 以上内存的开发板通常可以先不加；如果 ATC 转换时仍然出现内存不足或编译进程被杀掉，也应加上这两个环境变量后重试：

```
export TE_PARALLEL_COMPILER=1
export MAX_COMPILE_CORE_NUMBER=1
```

本章的起点不是重新训练模型，而是接着前面章节已经完成的分类模型训练和 ONNX 导出流程往下走。ResNet 的残差结构和小尺寸图像分类模型的基本写法，可以回看 第 3 章 ResNet 部分；Tiny-ImageNet 数据集、ResNet-18 训练和 ONNX 导出的过程，可以回看 第 4 章 ResNet-18 实例。本章只把训练后的 `resnet18_tiny_imagenet.onnx` 当作待部署模型，重点讲如何把它转换为不同精度的 OM，并验证量化后的精度和性能。

这个模型有几个适合量化教学的特点。它是一个面向 Tiny-ImageNet 的 ResNet18 分类模型，输入是 `1x3x64x64` 的 RGB 图像，输出是 200 个类别分数。相比标准 ImageNet 的 `224x224` 模型，它更小，适合在 Ascend 310B 开发板上完整跑完校准、性能测试和验证集评估；相比过于简单的 MNIST/CIFAR 小模型，它又包含卷积、BN、残差连接和多类别分类输出，能更真实地反映 FP16/INT8 量化对推理结果的影响。

第一段数据准备代码是模型下载。运行方式如下：

```
python tools/download_model.py
```

这段代码不应该只是简单的 `wget`。为了让教程和实验可复现，模型下载程序至少要处理三件事：

1. 默认从 `zhouxzh/resnet18_tiny_imagenet` 下载 `resnet18_tiny_imagenet.onnx`。
2. 默认使用 `https://hf-mirror.com`，失败时回退到 `https://huggingface.co`。
3. 只保存 FP32 ONNX，后续所有 OM 都由本章的 ATC/AMCT 流程生成。

第二段数据准备代码是校准集和验证集下载。Tiny-ImageNet 可以理解为 ImageNet 的小型教学版本：它包含 200 个类别，图像分辨率为 `64x64`，常用验证集有 10000 张图片。这个规模比完整 ImageNet 更适合开发板实验，但类别数又足够多，可以观察 Top1/Top5、边界样本排序和量化误差。运行方式如下：

```
python tools/download_tiny_imagenet.py train --force-download --per-class 2
python tools/download_tiny_imagenet.py val --force-download
```

这里故意把图片统一保存为 PNG。Tiny-ImageNet 原始数据中常见的是 JPEG，但 JPEG 是有损压缩，同一张图在不同处理链中可能引入细微差异。PNG 是无损格式，更适合做教学实验：校准图、验证图和清单可以稳定复现，后续对比 FP32/FP16/INT8 时，输入差异不会成为干扰因素。

这里还有一个重要约束：校准集和验证集必须分开。校准集只用来估计 INT8 的量化范围；验证集才用来报告 Top1/Top5。把验证集混入校准阶段，会让实验结论不干净，因为模型量化参数已经 “看过” 验证数据，后面的精度报告就不再是独立评估。

`tools/download_tiny_imagenet.py` 有两个子命令：

子命令	默认行为	输出清单
`train`	每类保存 2 张训练图，共 400 张 PNG	`data/calib_list.txt`
`val`	保存完整验证集，共 10000 张 PNG	`data/val_list.txt`

训练集和验证集的清单格式不同：

```
data/calib_list.txt: tiny_imagenet_train/0_00000.png
data/val_list.txt:   tiny_imagenet_val/0_00000.png 0
```

校准阶段不需要真实标签，所以 `calib_list.txt` 只保存图片路径。验证阶段要计算 Top1/Top5，所以 `val_list.txt` 必须保存 路径标签。

写数据准备代码时，关键不是把图片下载下来就结束，而是要保证后续实验可以复现。校准图片和验证图片应分目录保存，避免误把验证集用于校准；图片格式应统一，避免不同压缩格式带来额外输入差异；清单最好使用相对路径，这样本地目录和开发板目录不同也能迁移；验证集清单必须保存标签，accuracy 统计不能依赖文件名去猜类别。下载程序还应支持强制刷新和离线 cache，这样既方便完整重跑，也能在网络不稳定时继续教学实验。

8.5. 预处理：所有模型对比必须使用同一套输入

量化实验中最容易被忽略的是预处理。如果 FP32、FP16、INT8 三个模型使用的 resize、颜色通道、归一化或 layout 不一致，后面所有精度对比都没有意义。

本章把预处理集中在一个代码，核心函数是 `preprocess_resnet_rgb()`：

```
resized = resize_nearest_rgb(frame, out_h, out_w)
image_chw = resized.transpose(2, 0, 1).astype(np.float32)
image_chw *= 1.0 / 255.0
image_chw -= IMAGENET_MEAN
image_chw *= IMAGENET_INV_STD
input_tensor = image_chw[None, ...]
```

这段逻辑完成四件事：

1. 把 RGB 图片 resize 到 `64x64`。
2. 从 HWC 转成 NCHW。
3. 从 `uint8` 转成 `float32`，并缩放到 `[0, 1]`。
4. 使用 ImageNet mean/std 做归一化，再增加 batch 维度。

`01_compare_outputs.py`、`02_perf_compare.py`、`04_calibrate_quantization.py` 和 `05_validate_accuracy.py` 都调用这套预处理。这是本章代码设计中最重要的复用点：校准、快速对齐、性能测试和完整验证必须共享同一条输入路径。

这段预处理也体现了本章的几个设计取舍：

代码选择	为什么这样做
`load_rgb_frame()` 统一转 RGB	避免 PNG/JPEG、灰度图、不同解码库造成通道含义不一致
`resize_nearest_rgb()` 固定为 64x64	与训练好的 Tiny-ImageNet ResNet18 输入尺寸一致
`transpose(2, 0, 1)`	把图片从 HWC 变为 ATC 命令中声明的 NCHW

代码选择	为什么这样做
`astype(np.float32)`	即使后续模型是 FP16/INT8，输入预处理仍用同一个 FP32 浮点张量作为比较基准
`IMAGENET_MEAN` 和 `IMAGENET_INV_STD`	复用训练时的归一化方式，否则 accuracy 变化无法归因到量化

8.6. ATC 转 FP32/FP16：先建立可比较的基准

在做 INT8 之前，先从同一个 FP32 ONNX 转出 FP32 OM 和 FP16 OM。FP32 OM 是精度参考，FP16 OM 是第一阶段优化对象。

FP32 OM：

```
atc \
  --model=model/resnet18_tiny_imagenet.onnx \
  --framework=5 \
  --output=model/resnet18_tiny_imagenet_fp32 \
  --input_format=NCHW \
  --input_shape="input.1:1,3,64,64" \
  --soc_version=Ascend310B4 \
  --precision_mode=force_fp32 \
  --log=info
```

FP16 OM：

```
atc \
  --model=model/resnet18_tiny_imagenet.onnx \
  --framework=5 \
  --output=model/resnet18_tiny_imagenet_fp16 \
  --input_format=NCHW \
  --input_shape="input.1:1,3,64,64" \
  --soc_version=Ascend310B4 \
  --precision_mode=allow_fp32_to_fp16 \
  --log=info
```

几个参数值得单独解释：

参数	作用
`--framework=5`	输入模型是 ONNX
`--input_format=NCHW`	输入 tensor layout 与预处理输出一致
`--input_shape="input.1:1,3,64,64"`	固定 batch、通道和图片尺寸
`--soc_version=Ascend310B4`	指定目标芯片
`--precision_mode=force_fp32`	尽量保持 FP32，作为精度参考
`--precision_mode=allow_fp32_to_fp16`	允许 ATC 把 FP32 转为 FP16

这一步完成后，`model/` 中至少应有：

```
resnet18_tiny_imagenet.onnx
resnet18_tiny_imagenet_fp32.om
resnet18_tiny_imagenet_fp16.om
```

8.7. 如何编写输出对齐程序：先看模型是否“像”基准

转换成功不等于模型可信。第一层检查是输出对齐：给 FP32 OM 和候选 OM 喂同一张图，看 Top1 是否一致、Top5 重合度和 logits 误差。

这里需要先把三个概念说清楚。分类模型最后通常会输出一个长度等于类别数的向量，这个向量一般称为 logits，也就是 softmax 之前的原始类别分数。本章的模型输出 200 个 logits，每个位置对应 Tiny-ImageNet 的一个类别。分数最高的类别就是 Top1；分数最高的前 5 个类别构成 Top5。

在输出对齐阶段，Top1 和 Top5 还不是验证集 accuracy。因为这里不看真实标签，只比较“候选模型是否像 FP32 基准模型”。Top1 一致，表示同一张图片上 FP32 OM 和候选 OM 给出的第一名类别相同；Top5 重合度（Top5 overlap）表示两个模型前 5 个候选类别集合有多少个相同，最大值是 5。例如平均 Top5 重合度为 `4.9975/5`，说明绝大多数图片的前 5 个候选类别完全相同，只在极少数边界样本上出现了一个类别的差异。

logits 误差则不看类别名，而是直接比较两个输出向量的数值差异。本章记录最大绝对误差和平均绝对误差：前者观察最剧烈的一项数值漂移，后者观察整体漂移水平。这个指标通常比 Top1 更敏感，因为即使 Top1 没变，logits 的数值也可能已经发生变化。反过来，如果两个最高分非常接近，极小的 logits 漂移也可能让 Top1 对调。因此输出对齐要同时看排序和数值误差，不能只看一个指标。

`01_compare_outputs.py` 就是这个“小样本输出一致性检查”程序。它的输入是两个 OM 模型和一批固定图片；输出是一份 JSON 报告，记录候选模型和 FP32 基准模型的 Top1 是否一致、Top5 预测集合有多少重合，以及两个输出向量之间的最大/平均绝对误差。它不计算真实 accuracy，也不需要图片标签。它回答的问题是“候选模型的输出是否仍然接近 FP32 基准”，而不是“候选模型预测得是否正确”。

写完程序后，可以先用 FP16 OM 做一次输出对齐：

```
python 01_compare_outputs.py \
  --base-model model/resnet18_tiny_imagenet_fp32.om \
  --candidate-model model/resnet18_tiny_imagenet_fp16.om \
  --output outputs/output_compare.json
```

这类程序不应该先看真实标签，而应该只比较两个模型在同一输入上的输出排序。实现时可以分成三步：先读取一批固定图片，通常使用校准图片即可，不需要标签；然后把同一个 `input_tensor` 先送入 FP32 OM，再送入候选 OM；最后比较两个输出向量的 Top1、

Top5 集合、最大绝对误差和平均绝对误差。这个顺序能保证差异只来自模型本身，而不是来自图片读取、预处理或标签解析。

核心比较函数可以写成下面的思路：

```
base_top = topk_indices(base_logits, k=5)
candidate_top = topk_indices(candidate_logits, k=5)

top1_match = base_top[0] == candidate_top[0]
top5_overlap = len(set(base_top) & set(candidate_top))
mean_abs_diff = np.mean(np.abs(base_logits - candidate_logits))
max_abs_diff = np.max(np.abs(base_logits - candidate_logits))
```

这里的 `compare_logits()` 不使用真实标签。它比较的是两个模型输出之间的相似度，回答“候选模型是否像基准模型”，而不是“候选模型是否预测正确”。

本轮 PNG 数据重跑后，FP16 快速验证结果如下：

验证内容	对比对象	样本/次数	关键结果
输出对齐	FP32 OM vs FP16 OM	400 张校准图	Top1 match 100.00%，Top5 overlap 4.9975/5，mean abs diff 0.000405

这个结果说明 FP16 的输出排序几乎完全继承了 FP32，可以继续看性能。

8.8. 如何编写性能对比程序：把耗时拆成可解释阶段

第二层检查是性能。性能程序如果只记录一个总耗时，很难解释“模型变快了但端到端没有明显变快”这种现象。因此要把一次推理拆成 CPU 预处理、数据拷贝、设备执行和后处理几个阶段。

FP16 性能对比命令：

```
python 02_perf_compare.py \
  --models model/resnet18_tiny_imagenet_fp32.om model/resnet18_tiny_imagenet_fp16.om \
  --labels fp32 fp16 \
  --runs 100 \
  --output outputs/perf_compare.json
```

实现时可以让 `measure_model()` 围绕同一个 runner 循环多次，并把一次推理拆成下面几个容易理解的阶段：

计时阶段	含义
CPU 预处理	图片 resize、归一化、layout 转换
输入拷贝	把输入 tensor 从主机内存拷到 NPU 侧内存
模型执行	OM 模型在 NPU 上完成一次前向计算
输出拷贝	把模型输出从 NPU 侧内存拷回主机内存
推理整体耗时	输入拷贝、模型执行、输出拷贝等推理相关耗时
后处理	从输出中取 Top1 等轻量处理
端到端耗时	CPU 预处理 + 推理整体耗时 + 后处理

`StageRecorder` 是样例代码中复用的一个简单阶段计时器。它把每个阶段的多次耗时保存下来，再用 `summarize_stages()` 计算平均值、中位数、95% 分位耗时、最小值、最大值和 FPS。这样读者不仅知道平均值，还能看到少数慢帧造成的长尾抖动。

计时代码不要只包住 Python 函数入口，而要尽量贴近真正想测的边界。例如：

```
with recorder.time("preprocess"):
    input_tensor = preprocess_resnet_rgb(frame)

with recorder.time("inference_total"):
    outputs, timings = runner.infer(input_tensor)

with recorder.time("postprocess"):
    top1 = int(np.argmax(outputs[0]))
```

这里的 `preprocess`、`inference_total`、`postprocess` 是报告中的字段名；字段名服务于程序读写，教材阅读时先理解对应的中文阶段即可。

这样写的好处是，如果 INT8 的模型执行阶段明显变快，但 CPU 预处理占比很高，读者会知道下一步应该优化输入流水线，而不是继续调量化参数。

本轮 PNG 数据重跑后，FP16 性能结果如下：

对比	结果
FP32 end-to-end mean	4.1116 ms
FP16 end-to-end mean	3.7774 ms
FP16 相对 FP32	约 1.09x

FP16 的输出对齐很好，但端到端收益比较温和。这提示我们：如果希望进一步压缩模型执行时间，可以继续尝试 INT8；但也要保持同样的验证标准。

8.9. 如何编写 INT8 PTQ 程序：准备配置，再收集激活范围

INT8 不能只靠 ATC 参数完成。本章采用 AMCT PTQ 生成 deploy ONNX，再交给 ATC 转换为 INT8 OM。

图 8.4. INT8 PTQ 量化流程

8.9.1. 为什么选择 amct_onnx

AMCT 的全称是 Ascend Model Compression Toolkit，即昇腾模型压缩工具包。它不是一个推理运行时，而是部署前的模型处理工具：根据校准数据估计量化参数，把原始模型改写成带量化信息的模型，再交给 ATC 转成 OM。AMCT 支持不同深度学习框架的入口，常见形态包括 `amct_onnx`、`amct_pytorch`、`amct_caffe`、`amct_tensorflow` 等。不同入口面向的模型格式和依赖环境不同，但最终目标都是生成更适合昇腾硬件部署的压缩模型。

放到代码层面看，`amct_onnx` 提供的是一组 Python API。本章主要用到这些接口：

API	在代码中的作用	为什么需要它
`amct.create_quant_config()`	根据 ONNX 和校准批次数生成量化配置	先确定哪些层量化、使用什么量化策略
`amct.quantize_model()`	按配置给 ONNX 插入 fake-quant 和统计相关节点	让后续校准运行时能够收集激活范围
`amct.AMCT_SO`	提供 AMCT 专用的 ONNX Runtime SessionOptions	防止普通图优化破坏 fake-quant/统计节点
`amct.save_model()`	把校准得到的量化比例和零点固化进 deploy ONNX	生成 ATC 可转换的 INT8 部署模型

本章选择 `amct_onnx`，不是因为 PyTorch 路线不存在，而是因为本案例的工程条件更适合 ONNX 路线：

选择点	`amct_onnx` 路线	`amct_pytorch` 路线
输入模型	已经训练好的 FP32 ONNX	PyTorch 模型和 PyTorch 运行环境
校准执行	使用 ONNX Runtime 跑校准图	依赖 PyTorch/torch_npu 或 PyTorch 相关图处理
部署衔接	deploy ONNX 可直接交给 ATC 转 OM	通常还要处理导出或框架适配问题
对 310B 教学适配	更贴近 “ONNX -> ATC -> OM” 的离线推理部署链路	更适合 PyTorch 训练/量化感知训练等场景

第 3 章已经介绍过，`torch_npu` 在昇腾 310B 上仍有算子、精度和图编译方面的限制。本章的目标是把一个已经训练好的分类模型稳定部署到 310B 做推理，而不是在 310B 上继续训练或做 PyTorch 图改写。因此使用 `amct_onnx` 更直接：校准阶段只需要 ONNX Runtime 和 AMCT ONNX 包，部署阶段仍然走 310B 更常用的 ATC 离线转换路径。

这也是代码设计上的一个重要原则：工具链越靠近最终部署格式，中间变量就越少。本章从 FP32 ONNX 出发，所有 FP32、FP16、INT8 OM 都由这个 ONNX 或 AMCT 生成的 deploy ONNX 转换而来，避免了 “PyTorch 模型、ONNX 模型、OM 模型各自一套语义” 带来的额外不确定性。

8.9.2. AMCT ONNX 环境

INT8 PTQ 需要额外安装 AMCT ONNX 工具包。CANN Toolkit 本身不一定自带 `amct_onnx` Python 包。本章验证环境为：

```
CANN: 8.3.RC1
arch: aarch64
Python: 3.11
ONNX: 1.14.0
ONNX Runtime: 1.16.0
AMCT ONNX: 0.23.2
```

不同 CANN 版本应以昇腾官方文档中的 AMCT 依赖表为准。安装完成后可简单验证：

```
python -c 'import amct_onnx as amct, onnx, onnxruntime as ort; print("amct_onnx:", getattr
    ↪ (amct, "__file__", "ok")); print("onnx:", onnx.__version__); print("onnxruntime:",
    ↪  ort.__version__)'
```

如果 `import amct_onnx` 失败，优先检查 AMCT 包是否与 CANN 版本匹配、wheel 架构是否与 `uname -m` 一致、当前 `python` 是否就是安装 AMCT 的环境。

8.9.3. 准备阶段：生成量化配置

第一步运行：

```
python 03_prepare_quantization.py
```

准备阶段的代码不是直接 “量化模型”，而是先回答三个问题：输入 ONNX 能不能被当前 AMCT 版本理解、这次准备用多少张校准图片、哪些层参与量化。因此代码从参数和路径开始写：

```
parser.add_argument("--onnx", default=str(DEFAULT_ONNX))
parser.add_argument("--work-dir", default=str(DEFAULT_WORK_DIR))
parser.add_argument("--calib-list", default=str(DEFAULT_CALIB_LIST))
parser.add_argument("--samples", type=int, default=0)
parser.add_argument("--amct-opset", type=int, default=16)
parser.add_argument("--skip-layers", nargs="*", default=[])
parser.add_argument("--activation-offset", action=argparse.BooleanOptionalAction, default=
    ↪ True)
```

这些参数看似普通，但每一个都对应一个教学目的：

参数	为什么要这样写
`--onnx`	允许替换自己的 FP32 ONNX，而不是把模型名写死在代码中
`--work-dir`	每次实验的 AMCT 中间文件独立保存，sweep 时不会互相覆盖
`--calib-list` / `--samples`	让 “用多少张图片校准” 成为显式实验变量
`--amct-opset`	兼容不同 AMCT/ONNX Runtime 版本，必要时可用 `0` 跳过转换
`--skip-layers`	当某些层量化后精度敏感时，可以先排除再定位问题
`--activation-offset`	控制激活量化是否带 offset，本章默认开启，更适合非对称激活分布

代码中还设置了：

```
os.environ.setdefault("TORCH_DEVICE_BACKEND_AUTOLOAD", "0")
```

这不是在使用 PyTorch，而是反过来避免不必要的设备后端自动加载。本脚本只需要 ONNX、AMCT 和校准清单，不应该因为环境里装了其他深度学习包就触发额外的 NPU 后端初始化。

准备阶段真正做两件事。

第一，检查原始 ONNX opset。如果当前 opset 与 AMCT 期望不一致，就调用 `onnx.version_converter.convert_version()` 转成默认的 opset 16。这里不是为了改变模型语义，而是为了匹配本章验证环境中的 AMCT ONNX 依赖组合。

代码单独写出 `prepare_opset()`，而不是把转换逻辑塞进 `main()`，好处是错误边界清楚：如果 opset 转换失败，问题就是 ONNX/AMCT 兼容性；如果转换成功而后面失败，问题再去看量化配置或校准数据。

第二，调用：

```
amct.create_quant_config(
    config_file=...,
    model_file=...,
    batch_num=len(paths),
    activation_offset=True,
    updated_model=...,
)
```

这一步生成 `outputs/int8_amct/config.json` 和 `outputs/int8_amct/updated_model.onnx`。`config.json` 描述哪些层参与量化、是否使用 activation offset，以及校准批次数。它还没有真正统计激活范围，只是把 “要怎么量化” 准备好。

这里必须注意 `batch_num=len(paths)`。AMCT 需要知道后续会跑多少批校准输入，所以准备阶段先读取 `calib_list.txt`，但还不真正执行模型。这样写的好处是：配置生成和校准执行可以分开排错；sweep 实验也可以为 20、50、100、200、400 张图片分别生成独立配置。

8.9.4. 校准阶段：用真实图片统计激活范围

第二步运行：

```
python 04_calibrate_quantization.py
```

校准阶段的代码是 PTQ 的关键。它先调用 `amct.quantize_model()` 生成带 fake-quant 节点的 `modified_model.onnx`，然后用 ONNX Runtime 跑校准图片：

```
session = ort.InferenceSession(
    str(modified_model),
    amct.AMCT_SO,
    providers=["CPUExecutionProvider"],
)
session.run(None, {input_name: input_tensor})
```

这段代码里有四个容易被忽略的设计点。

第一，`model_for_quant` 优先使用准备阶段生成的 `updated_model.onnx`：

```
model_for_quant = updated_model if updated_model.exists() else resolve_chapter_path(args.
    ↪ onnx)
```

这样写是为了兼容两种情况。正常流程中，`03_prepare_quantization.py` 已经完成 opset 转换并生成 `updated_model.onnx`；如果读者明确跳过准备阶段的某些处理，代码仍然可以回退到原始 ONNX，但会在缺少 `config.json` 时给出明确错误。

第二，`amct.quantize_model()` 和 `amct.save_model()` 必须放在同一个 Python 进程：

```
amct.quantize_model(str(config_file), str(model_for_quant), str(modified_model), str(
    ↪ record_file))
...
amct.save_model(str(modified_model), str(record_file), str(save_prefix))
```

AMCT 0.23.x 会在进程内保存一部分融合和量化状态。如果把 “生成 modified ONNX” 和 “保存 deploy ONNX” 拆成两个独立命令，后一个进程可能拿不到前一个进程里的状态，导致保存失败或生成结果不完整。所以本章把校准和保存写进一个脚本，这是为了配合工具行为，而不是为了偷懒。

第三，ONNX Runtime 要使用 `amct.AMCT_SO`，并关闭图优化：

这里使用 `amct.AMCT_SO` 并关闭图优化，是为了让 fake-quant 节点保留在图中，从而收集每层激活统计。校准结束后，代码调用 `amct.save_model()` 冻结 deploy ONNX，并把结果复制到：

```
model/resnet18_tiny_imagenet_int8_deploy.onnx
```

如果使用普通的 ONNX Runtime session，图优化可能把 AMCT 插入的统计节点或 fake-quant 节点融合掉，校准过程就失去了意义。因此代码显式设置：

```
amct.AMCT_SO.graph_optimization_level = ort.GraphOptimizationLevel.ORT_DISABLE_ALL
amct.AMCT_SO.intra_op_num_threads = 1
amct.AMCT_SO.inter_op_num_threads = 1
```

线程数设为 1 不是为了追求最快，而是为了降低 310B 开发板上校准时的内存压力和运行抖动。校准阶段看重的是统计稳定性，不是吞吐量。

第四，校准循环必须复用完整验证代码里的预处理：

```
for path in paths:
    frame = load_rgb_frame(path)
    input_tensor = preprocess_resnet_rgb(frame)
    session.run(None, {args.input_name: input_tensor})
```

PTQ 估计的是“真实部署输入经过预处理后，各层激活大概落在哪个范围”。如果校准阶段使用一套预处理，而验证阶段使用另一套预处理，AMCT 得到的量化比例和零点就会偏离真实部署分布。把 `load_rgb_frame()` 和 `preprocess_resnet_rgb()` 复用到校准、输出对齐、性能测试和完整验证中，是本章代码最重要的正确性保障。

校准结束后，`find_deploy_model()` 会在工作目录中寻找 AMCT 实际生成的 deploy ONNX，再复制到固定路径 `model/resnet18_tiny_imagenet_int8_deploy.onnx`。这样做的好处是：AMCT 内部输出文件名可以随版本变化，而后续 ATC 命令永远读取同一个稳定路径。

8.9.5. ATC 转 INT8 OM

最后用 AMCT 生成的 deploy ONNX 转出 INT8 OM：

```
atc \
  --model=model/resnet18_tiny_imagenet_int8_deploy.onnx \
  --framework=5 \
  --output=model/resnet18_tiny_imagenet_int8 \
  --input_format=NCHW \
  --input_shape="input.1:1,3,64,64" \
  --soc_version=Ascend310B4 \
  --log=info
```

这里不再设置 `--precision_mode`。deploy ONNX 已经包含 AMCT 校准后的量化信息，额外指定精度模式可能覆盖或破坏这些信息。

8.10. 快速验证 INT8：漂移更大不等于不能用

INT8 转换后，先重复 FP16 阶段的两个快速检查。

输出对齐：

```
python 01_compare_outputs.py \
  --base-model model/resnet18_tiny_imagenet_fp32.om \
  --candidate-model model/resnet18_tiny_imagenet_int8.om \
  --output outputs/output_compare_int8.json
```

性能对比：

```
python 02_perf_compare.py \
  --models model/resnet18_tiny_imagenet_fp32.om model/resnet18_tiny_imagenet_fp16.om model
    ↪ /resnet18_tiny_imagenet_int8.om \
  --labels fp32 fp16 int8 \
  --runs 100 \
  --output outputs/perf_compare.json
```

本轮 INT8 快速验证结果如下：

验证内容	对比对象	样本/次数	关键结果
输出对齐	FP32 OM vs INT8 OM	400 张校准图	Top1 match 99.75%, Top5 overlap 4.9075/5，mean abs diff 0.016629
性能对比	FP32/FP16/INT8 OM	100 次	INT8 end-to-end mean 2.1634 ms，相对 FP32 约 1.90x

INT8 的 logits 误差明显大于 FP16，这是低比特量化的正常现象。关键不是“误差是否为零”，而是这些误差是否改变最终分类结论。如果 Top1 match 和 Top5 overlap 仍然较高，就说明可以进入完整验证集评估。

8.11. 如何编写完整验证程序：把多个 OM 放到同一把尺上

快速输出对齐不能替代真实精度。最终结论必须来自带标签的验证集：

```
python 05_validate_accuracy.py --output outputs/accuracy_compare.json
```

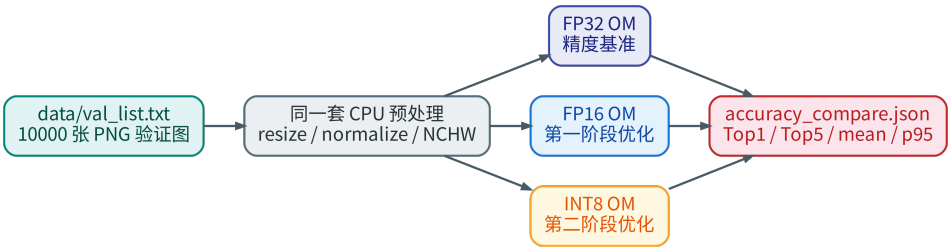


图 8.5. 完整验证集横向对比流程

验证程序会使用同一套 CPU 预处理, 在 Ascend 310B 上依次运行 FP32、FP16、INT8 三个 OM，并用同一份 `data/val_list.txt` 计算 Top1/Top5。

完整验证程序的核心函数可以命名为 `evaluate_accuracy()`，实现时按下面顺序组织：

1. 解析 `--om-models` 和 `--labels`，默认就是 FP32、FP16、INT8 三个 OM。
2. 用 `read_validation_list()` 读取 路径标签。
3. 用 `ReuseResNetRunner` 为每个 OM 建立一个可复用 runner。
4. 检查三个模型输出类别数一致，避免把不同分类头的模型放在一起比较。
5. warmup 后逐张读取验证图，执行同一套 `preprocess_resnet_rgb()`。
6. 对每个 OM 运行推理，用 `add_accuracy()` 累加 Top1 和 Top5 命中数。
7. 记录每个 OM 的推理整体耗时，报告中对应字段为 `inference_total`。
8. 保存 `mismatches`，方便后续查看哪些样本预测不一致或预测错误。

这里有一个容易写错的地方：不要分别运行三次验证程序再手工拼表。那样三次运行的预热、图片读取顺序、异常处理和统计口径都可能不同。更好的方式是在同一个循环里对同一张图片依次运行多个 OM，再同时更新各自的指标状态。

报告主要看：

指标	含义
Top1 / Top5	当前模型相对验证集真实标签的分类精度
平均推理耗时 / 95% 分位耗时	当前 OM 的推理耗时，不包含图片读取和 CPU 预处理
`mismatches`	部分错误样本及三种模型的 TopK 预测

报告中，平均推理耗时和 95% 分位耗时分别保存为 `mean_ms` 和 `p95_ms` 字段。

常用参数如下：

参数	教学用途
`--val-list`	指定带标签验证集清单，默认 `data/val_list.txt`

参数	教学用途
`--samples`	调试时限制验证图片数量；正式报告使用 `0`，表示全部验证集
`--om-models` / `--labels`	把任意多个 OM 放到同一口径下横向比较
`--warmup`	预热 OM，减少首次执行开销对推理耗时的影响
`--mismatch-limit`	控制报告中保存多少个预测不一致样本，便于后续人工分析
`--save-samples`	保存逐图预测，适合深入分析错例，但报告文件会明显变大

调试时可以先少量运行：

```
python 05_validate_accuracy.py --samples 200
```

但正式结论应使用完整验证集。本轮 PNG 数据重跑后的结果如下：

模型	Top1	Top5	Top1 相对 FP32	Top5 相对 FP32	平均推理耗时/ms	95% 分位耗时/ms	平均加速比
FP32 OM	62.43%	82.68%	0.00 pp	0.00 pp	3.4808	3.5081	1.00x
FP16 OM	62.43%	82.68%	0.00 pp	0.00 pp	3.1410	3.1659	1.11x
INT8 OM	62.44%	82.67%	+0.01 pp	-0.01 pp	1.4999	1.5196	2.32x

这组结果说明：

1. FP16 与 FP32 的 Top1/Top5 完全一致，说明 FP16 转换对分类结果没有可见损失。
2. INT8 的 Top1 与 FP32 基本持平，Top5 只低 0.01 个百分点。
3. INT8 的平均推理耗时相对 FP32 约 2.32x，加速收益明显。

INT8 Top1 比 FP32 高了 0.01 个百分点，这个现象不能理解成 “INT8 精度更好”。验证集是 10000 张图片，0.01 个百分点只对应 1 张图片。具体计数是 FP32/FP16 Top1 正确 6243 张，INT8 Top1 正确 6244 张；同时 Top5 是 FP32/FP16 正确 8268 张，INT8 正确 8267 张。也就是说，量化后的量化比例、零点、取整和截断改变了少量边界样本的 logits 排序，有的样本从错变对，也有的样本从 Top5 内掉出。这种微小正负波动应视为量化噪声范围内的排序抖动，而不是稳定收益。

图 8.6. 校准样本数 sweep 流程

8.12. 如何编写 sweep 实验：用最少胶水代码重复完整流程

如果 INT8 的完整验证集精度已经满足项目要求，这一节可以不做。sweep 是一个可选的分析实验，用来回答更细的问题：校准图片到底需要多少张，精度曲线什么时候进入平台区，以及某次 INT8 精度波动是否与校准样本数量有关。

```
bash tools/sweep_calibration_samples.sh 20 50 100 200 400
```

sweep 程序会为每个样本数单独执行图中的流程：

这里故意让 shell 只做实验编排，不在 shell 里嵌入 Python 逻辑。每个 sweep 点都必须完整经历下面六步：

1. 从完整校准清单中按固定随机种子抽取 N 张图片。
2. 根据当前 N 生成一份新的 AMCT 量化配置。
3. 用这 N 张图片重新统计激活范围，并冻结当前 N 对应的 deploy ONNX。
4. 用 ATC 把当前 deploy ONNX 转成独立的 `int8_sN.om`。
5. 在同一份验证集上比较 FP32、FP16 和当前 INT8。
6. 从验证报告中提取 Top1、Top5 和推理耗时，追加到 `summary.csv`。

`calibration_sweep_helper.py` 里有两个子命令：

子命令	作用
`make-subset`	用固定随机种子打乱 `calib_list.txt`，取前 N 张，保证实验可复现
`append-summary`	从每个 `accuracy_compare.json` 中提取 Top1/Top5 和 INT8 推理耗时，写入 CSV

常用环境变量如下：

变量	默认值	说明
`SAMPLE_COUNTS`	`20 50 100 200 400`	不传命令行参数时使用的样本数列表
`VAL_SAMPLES`	`0`	每个点评估多少张验证图，`0` 表示全部验证集
`OUT_DIR`	`outputs/calibration_sweep`	sweep 输出目录
`SOC_VERSION`	`Ascend310B4`	ATC 目标芯片
`SWEEP_SEED`	`2024`	固定校准子集随机顺序

`summary.csv` 中最关键的列是：

列	含义
`calibration_samples`	当前 INT8 模型使用的校准图片数量
`int8_top1_pct` / `int8_top5_pct`	当前 INT8 模型验证集精度
`int8_minus_fp32_top1_pp` / `int8_minus_fp32_top5_pp`	INT8 相对 FP32 的精度变化
`int8_mean_ms` / `int8_p95_ms`	当前 INT8 模型平均推理耗时和 95% 分位耗时

本轮 sweep 结果如下：

校准图片数	INT8 Top1	INT8 Top5	Top1 相对 FP32	Top5 相对 FP32	平均推理耗时/ms	95% 分位耗时/ms
20	62.44%	82.73%	+0.01 pp	+0.05 pp	1.4977	1.5195
50	62.59%	82.73%	+0.16 pp	+0.05 pp	1.5198	1.5441
100	62.53%	82.73%	+0.10 pp	+0.05 pp	1.4995	1.5236
200	62.60%	82.78%	+0.17 pp	+0.10 pp	1.5055	1.5290
400	62.44%	82.67%	+0.01 pp	-0.01 pp	1.5125	1.5351

这轮单次实验的经验拐点大约出现在 50 到 200 张之间：20 张已经没有明显掉点，50 张后 Top1/Top5 基本进入平台区，200 张达到本轮最高 Top1/Top5；继续增加到 400 张没有继续提升，反而回到与主 INT8 模型一致的 62.44%/82.67%。

这不是矛盾，而是 PTQ 的正常现象。校准范围估计要在 “覆盖极端值” 和 “保持常见值分辨率” 之间折中，再叠加分类边界样本的 logits 排序抖动，accuracy 曲线不一定随样本数单调上升。工程上不要假设图片越多越好，而应该用固定验证集观察平台区；若要把 200 张作为正式结论，还应换不同随机种子、增加每类校准图片数量再重复实验。

把这个实验迁移到其他模型时，需要保持两点不变。第一，所有 sweep 点必须使用同一份验证集，否则曲线变化可能来自验证样本变化。第二，每个样本数都要重新执行 AMCT 校准和 ATC 转换，不能只替换清单后复用旧 INT8 OM，因为量化参数已经固化在 deploy ONNX 和 OM 中。

8.13. 如何设计报告和故障回退

量化验证代码不能只把结果打印在终端。终端输出适合人看，但不适合复查、画图、写报告或比较多轮实验。更好的做法是让每个程序都写结构化报告：输入模型是谁、模型 hash 是什么、用了多少样本、Top1/Top5 是多少、推理耗时统计是什么。

本章参考实现中，报告设计遵循下面的原则：

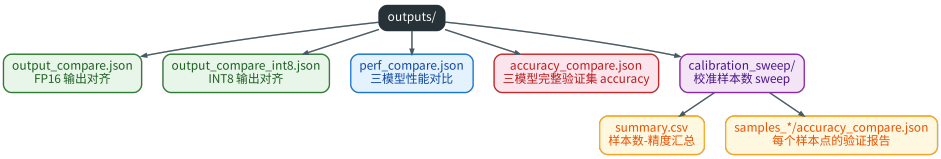


图 8.7. 第 8 章结果文件结构

报告内容	编写原因
保存模型路径、大小和 hash	防止后来分不清报告对应哪一个 OM
保存样本数量和清单路径	防止把 200 张调试结果当成完整验证集结果
保存 Top1/Top5 和 logits 差异	同时观察 “是否预测正确” 和 “输出是否接近”
保存平均值、中位数、95% 分位、最小值、最大值	平均值看趋势，95% 分位看少数慢帧
sweep 使用 CSV 汇总	方便直接画 “校准样本数 - accuracy” 曲线
每个 sweep 点保留独立 JSON	当某个点异常时，可以回到原始报告排查

常见失败模式如下：

现象	可能原因	处理建议
ATC 转 FP16 失败	ONNX 算子或 shape 不满足转换要求	先检查 FP32 OM 是否能转换
FP16 输出明显漂移	输入 layout、shape 或预处理不一致	回到 `01_compare_outputs.py` 小样本定位
`import amct_onnx` 失败	AMCT 与 CANN/Python/架构不匹配	按 CANN 版本重新安装 AMCT
INT8 accuracy 大幅下降	校准集不代表真实输入分布	增加代表性校准图片，检查预处理
INT8 性能收益很小	算子回退或模型太小	查看 ATC 日志和 Profiling
少数帧耗时明显偏高	预热不足、资源复用不稳定	增加 warmup，固定运行环境

量化失败时不要先调参数。建议按图中的顺序回退检查：

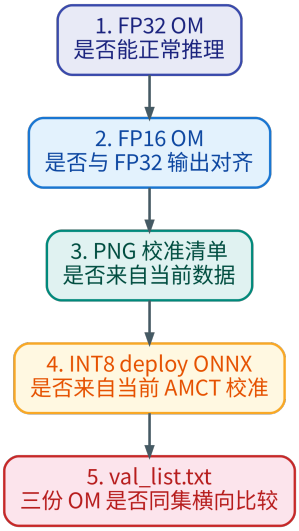


图 8.8. 量化失败时的回退检查顺序

8.14. 小结

这个案例的重点不是“把 ResNet18 转成 INT8”这么简单，而是建立一套可复用的量化评估方法。先用 FP32 建立精度参考，再用 FP16 做温和优化，最后用 INT8 PTQ 追求更高性能。每一步都要回答三个问题：输出是否对齐，速度是否提升，验证集精度是否仍然可接受。

从本轮结果看，FP16 与 FP32 精度一致但性能收益有限；INT8 的完整验证集精度基本持平，平均推理耗时从 3.4808 ms 降到 1.4999 ms。对这个模型和这套 PNG 校准数据而言，INT8 PTQ 是值得部署的优化方向。更重要的是，读者可以把本章的代码结构和判断方法迁移到自己的 ONNX 模型上：替换模型、输入 shape、预处理和数据清单，然后重新执行同一套对齐、性能、accuracy 和 sweep 实验。

迁移时最容易漏改的是四类内容：ATC 的 `--input_shape`、预处理函数、类别数和标签映射、校准集/验证集清单格式。只要这四类内容保持一致，本章的五个脚本就能继续作为一个完整的量化评估模板使用。

9. AI 项目交付闭环

9.1. 章节总览

本章以 “基于 COCO SSD300 的实验室实时检测告警系统” 为贯穿案例，建立从需求澄清-> 指标体系-> 评测集-> 迭代节奏-> 资产沉淀-> 交付与回归的一套闭环方法论，让技术决策基于指标与风险敞口，而非经验臆测。核心理念：可量化、可比较、可复用、可追溯。

案例默认使用 Hugging Face 仓库 `zhouxzh/SSD300` 中的 `ssd300_resnet50.onnx` / `ssd300_resnet50.om`。其中 `ssd300_` 表示输入尺寸为 300×300，和后续 `ssd320_` SSDLite320 模型区分。实验室告警逻辑只基于 COCO 检测结果做后处理，例如 ROI 闯入、人数统计、背包/电脑/水杯等物品遗留提醒，不重新训练模型，也不额外构建专用数据集。

9.2. 需求澄清 Canvas

维度	要素	问题提示	示例
场景	输入源/运行环境	摄像头？批处理？	实验室入口或设备区摄像头
目标	功能/业务价值	用户希望看到什么结果？	SSD300 实时检测 + 规则告警
指标	Latency/FPS/精度	哪些分位数重要？	<80ms / >=25FPS / mAP>=0.6
约束	能耗/内存/带宽	上限是多少？	功耗 <=15W 内存 <=3GB
风险	数据/硬件/算法	失败模式有哪些？	低光/遮挡/误报/摄像头抖动
合规	隐私/许可	是否需要脱敏？	仅上传事件元数据
成本	硬件/云	ROI 衡量？	10 台板卡预算

输出：`requirement.yaml`（版本化），后续所有评审基于此文档。

9.3. 指标分层与优先级

层级	类别	指标	说明	失败后果
SLO A	体验	p95 延迟	端到端	体验差/丢帧
SLO A	性能	FPS	稳态吞吐	处理拥堵
SLO B	质量	mAP/F1/Top1	任务正确性	无法满足业务
SLO B	稳定	Crash/小时	可靠性	运维成本高
SLO C	资源	内存峰值/功耗	成本约束	设备异常/降频
SLO C	带宽	上行 kbps	成本/合规	费用/拥塞

优先级：先保障 A（体验 + 功能可用），再稳定 B（质量/稳定），最后优化 C（资源效率）。

9.4. Baseline 策略与控制变量法

Baseline 目标：建立 “最小改动可运行” 标尺。原则：

1. 不提前做微优化；
2. 记录所有关键参数：模型版本、输入尺寸、预处理策略、硬件温度范围；
3. 一次仅改变单个变量（batch、精度、线程数）。基线存档：`baseline/<date>-<commit>/metrics.json`；对比脚本生成差异报告。

9.5. 评测集设计原则

原则	内容
代表性	涵盖主流场景/光照/角度
覆盖边界	极端尺寸、模糊、遮挡
可再现	文件命名规范 + 固定清单
可扩展	新增样本不破坏旧索引
标注一致	标注工具/规范/审校流程

目录示例：

```
dataset_eval/
  images/
    day/*.jpg
    night/*.jpg
    occlusion/*.jpg
  annotations/
    instances_train.json
    instances_val.json
```

```
meta/
  README.md
  version.txt
```

提供 Hash 列表，防止样本被替换而影响回归可信度。

9.6. 迭代计划与看板

四阶段：

Sprint	目标	核心产出	风险控制
0	环境/基线	baseline metrics	依赖清单齐全
1	精度与功能稳固	精度报告	数据问题快速反馈
2	性能与稳定	性能对比表/监控上线	Watchdog 验证
3	工程交付包装	Release Notes/脚本	灰度计划制定

看板列：Backlog -> Doing -> Review -> Bench -> Done；性能/精度任务需进入 Bench 列执行对比脚本通过后才可 Done。

9.7. 资产沉淀文档体系

文档	内容	更新频率
README	快速启动	版本变化时
ARCHITECTURE	架构图/模块说明	结构调整
MODEL_CARD	模型来源/许可/精度/限制	模型更新
EVAL_REPORT	数据与评测方法/指标	每次发布
PERF_REPORT	基线/优化对比	优化后
CHANGELOG	可见版本差异	每次版本
RISK_LOG	已知风险列表	动态

MODEL_CARD 需包含：数据来源、训练超参摘要、输入契约、已知局限、许可（如 Apache-2.0）、安全与偏见说明（若涉及识别敏感属性声明避免用途）。

9.8. 交付目录与不可变产物

```
release/
```

```
v1.0/
  manifest.json     # 产物 hash / 版本矩阵
  models/
    ssd300_resnet50.om
    classify.om
    signature.json
  scripts/
    run.ps1
    run.sh
    watchdog.sh
  configs/
    default.yaml
  docs/
    model_card_ssd300.md
    model_card_classify.md
    QUICKSTART.md
  reports/
    perf.json
    accuracy.json
```

manifest.json 字段：`{version, commit, build_time, model_hashes, dependencies}`。

9.9. 上线前综合 Checklist

类别	检查项	通过标准
功能	核心用例 100%	自动化用例通过
性能	p95 < 目标 +5%	连续 30min 稳定
精度	mAP/Top1 回归差 < 阈值	与基线对比
资源	内存峰值 < 75%	1h 稳态无泄漏
稳定	Crash=0, 重启 =0	守护日志清洁
安全	日志无敏感泄露	关键字段脱敏
配置	签名校验一致	Hash 匹配
回滚	验证上一版本可用	切换 < 30s

9.10. 验收、回归与漂移监测

交付后 7 天加密监控：记录时延、精度漂移（采样对比模型输出变化）。漂移检测：相同输入集合（Shadow Set）每天抽样跑一次 -> 统计 logits KL 散度/TopK 变化率，高于阈值（如 KL > 0.05）触发报警（潜在数据分布变化或模型文件损坏）。回归集版本化：`eval_set_vX`；若需替换样本 -> 新增版本，不覆盖旧数据。

9.11. 风险管理与决策日志

风险登记表：`risk_log.md` 每条包含：ID、描述、影响、概率、缓解、当前状态。决策日志（Decision Record, ADR）：记录架构/模型/精度策略选择及备选方案放弃理由，以便新成员快速建立上下文。

9.12. 章节小结

方法论的核心不是流程文档堆砌，而是 “指标驱动 + 资产沉淀 + 可回滚” 三支柱。通过契约化需求、标准化 Baseline、规范化评测与回归体系，使团队协作更高效、风险暴露更透明、交付结果更可信。

9.13. 实践任务

1. 输出 `requirement.yaml`（含指标与约束）。
2. 构建 30 张代表图像的 mini 评测集并附 Hash 列表。
3. 生成 baseline `metrics.json` 与后一次优化对比 diff 报告。
4. 制作一个 MODEL_CARD 模板并填写一个模型示例。
5. 编写上线 Checklist 并模拟一项未通过情形与处置方案。

Part II.

实践案例

1. 案例 1：智能考勤机

1.1. 教程定位

考勤系统是边缘人工智能中一个典型的端到端问题：摄像头产生连续图像，模型在设备侧完成检测和特征提取，主机程序负责比对与记录，浏览器则提供管理和观察入口。它同时涉及模型合同、主机—设备数据传输、线程生命周期、网络接口和个人数据边界，因而适合作为本书的第一个综合案例。

本案例的目标不是构造可直接用于真实身份认证的产品，而是建立一条可以观察、拆解和复现实验的技术链路。系统在昇腾 310B 开发板上运行两个离线模型，将人脸图像转换为可比较的特征向量，再由 SQLite 保存用户记录和考勤事件。FastAPI 提供类型明确的 HTTP 接口，React 界面负责用户注册、浏览器抓拍、手动打卡和记录查看。板端摄像头产生的预览帧通过 MJPEG 接口送往浏览器。

自动登记陌生人是本案例保留的教学演示策略：它用于展示 “检测到人脸—提取特征—写入数据库” 的完整分支，不代表经过同意的身份注册，也不构成生产级认证方案。真实部署必须增加身份确认、访问控制、活体检测、审计和数据删除机制；本章不把演示结果表述为识别准确率或考勤有效性结论。

完成本案例后，读者应能够：

1. 根据输入、输出和前处理合同检查一个 ONNX/OM 模型，而不是只依据文件名判断模型是否可用；
2. 解释 PyACL 初始化、Host/Device 拷贝、模型执行和资源释放的顺序；
3. 设计一个由 FastAPI API、React 浏览器界面和单一硬件工作线程组成的边缘服务；
4. 区分模型转换检查、数值烟测、接口烟测、用户体验检查和性能测量等不同证据；
5. 讨论生物特征数据的保存范围、访问边界和教学示例与生产系统之间的差异。

1.2. 问题背景与系统边界

在云端系统中，摄像头数据通常上传到远程服务进行处理；在边缘设备上，图像、特征和考勤记录可以留在设备本地，从而减少网络依赖并缩短数据离开现场的路径。代价是设备同时承担推理、采集、存储和服务任务，资源释放、并发控制和故障可见性必须由应用显式管理。

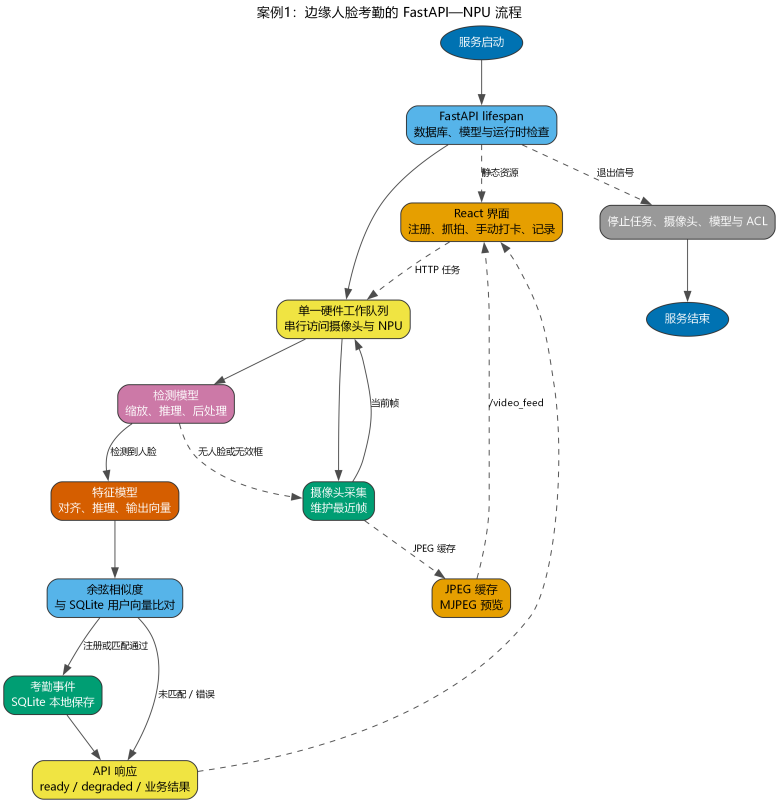

案例1：边缘人脸考勤的 FastAPI—NPU 流程
服务启动
FastAPI lifespan
数据库、模型与运行时检查
静态资源
退出信号
React 界面
注册、抓拍、手动打卡、记录
停止任务、摄像头、模型与 ACL
HTTP 任务
服务结束
单一硬件工作队列
串行访问摄像头与 NPU
检测模型
缩放、推理、后处理
当前帧
/video_feed
检测到人脸
无人脸或无效框
特征模型
对齐、推理、输出向量
摄像头采集
维护最近帧
JPEG 缓存
余弦相似度
与 SQLite 用户向量比对
JPEG 缓存
MJPEG 预览
注册或匹配通过
考勤事件
SQLite 本地保存
未匹配 / 错误
API 响应
ready / degraded / 业务结果

图 1.1. 案例 1 系统流程

本案例的边界如下：

边界	本案例承担的工作	不在本案例结论范围内
图像采集	板端 USB 摄像头采集；浏览器也可提交单张图像	摄像头质量、光照和姿态的全面评测
模型推理	在昇腾 NPU 上执行已转换的检测和特征模型	重新训练模型或证明某一公开基准上的精度
身份比对	在主机内存中计算余弦相似度并选取最高候选	法律意义上的身份认证或活体认证
数据保存	SQLite 保存用户、特征和事件元数据	云端同步、多人权限和长期档案管理
Web 交互	FastAPI API 与 React 界面	互联网暴露、生产级认证和高可用集群

系统默认只监听本机地址。若在可信的实验局域网内访问开发板，操作者必须显式指定监听地址，并意识到照片、特征和考勤接口仍属于敏感资源。

1.3. 系统架构

系统由四个相互约束的层次构成：

1. **采集层**：板端 `VideoCamera` 读取摄像头帧并维护最近一帧；React 可以使用浏览器摄像头生成上传图像。两种来源都进入同一图像处理接口。
2. **硬件工作层**：一个受控的运行线程顺序调用检测、裁剪和特征提取；注册、打卡等推理任务经同一队列执行，避免多个 HTTP 请求同时使用同一 ACL context、Device buffer 或摄像头句柄。SQLite 另以进程内锁保护读写。
3. **服务层**：FastAPI 将注册、抓拍、手动打卡、记录查询和 MJPEG 预览暴露为 HTTP 资源。API 只负责校验请求、提交硬件任务和组织响应，不在请求函数中复制一套推理流程。
4. **界面层**：React 调用 API 并呈现状态。界面中的“自动登记”标签必须说明其教学属性，不能暗示这是经过人工授权的生产注册。

1.3.1. 一次图像请求的逻辑顺序

对于注册或手动打卡，一次请求通常经历以下阶段：

1. FastAPI 读取 multipart 文件或 base64 图像，并限制请求大小；
2. OpenCV 解码图像，检查图像非空且位于允许的处理范围；
3. 检测模型定位候选人脸，选择面积最大的候选作为本案例的简化策略；

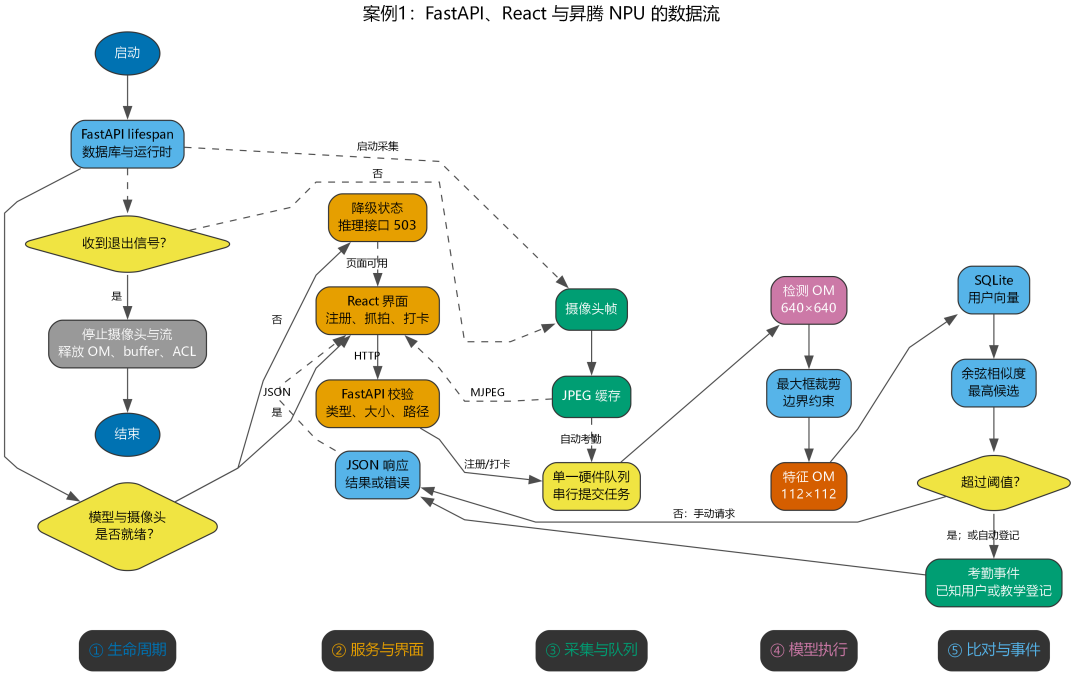


图 1.2. 案例 1 详细流程

4. 根据边界框裁剪人脸并送入特征模型；
5. 将输出向量与数据库中的向量逐一计算余弦相似度；
6. 注册请求写入用户和头像，打卡请求在通过阈值时写入事件；
7. API 返回结构化结果，界面根据 `success`、`match` 或错误状态更新视图。

这是一种便于教学的串行流程。它没有声称对多人脸、遮挡、姿态或大规模用户库具有最佳性能；这些问题应在后续实验中单独设计协议。

1.4. 模型与张量合同

1.4.1. 文件角色

样例脚本使用以下逻辑文件名：

逻辑角色	ONNX 文件（准备阶段）	OM 文件（运行阶段）	处理结果
人脸检测	`models/det_500m.onnx`	`models/face_detection.om`	候选框与分数，供主机后处理
特征提取	`models/w600k_mbf.onnx`	`models/face_recognition.om`	人脸特征向量

文件名是工程约定，不足以证明网络的论文名称或训练来源。检测网络的输出解析必须以实际 ONNX 图、`check_onnx.py/check_onnx_out.py` 的结果以及板端数值烟测共同确认。本实现采用三尺度分数、边界框距离和关键点输出的解析路径；若转换得到的图结构不同，应先修正后处理合同，不能以 “模型能加载” 替代验证。

1.4.2. 输入和前处理

当前实现编码了两组固定输入形状：检测输入为 `1×3×640×640`，特征输入为 `1×3×112×112`，数据类型为浮点数组。摄像头帧通常以 BGR 排列进入 OpenCV，再缩放、归一化并转换为 `NCHW`。这些尺寸和归一化常数属于运行时合同，必须与板端 OM 的描述和模型准备记录一致；改变其中任一项都应重新执行输入输出检查和数值烟测。

特征模型的输出在当前实现中被解释为 `float32` 向量并写入 SQLite BLOB。向量维度、字节序和归一化方式是数据库数据的隐含类型信息，迁移或更换模型时不能只保留文件名而忽略这些字段。

1.4.3. 从 ONNX 到 OM

ONNX 是准备和检查阶段的交换格式，OM 是昇腾运行时加载的离线模型。模型准备脚本负责下载或定位源资产、检查 ONNX 输入输出，并在板端调用 ATC 生成 OM。ATC、PyACL 和 OM 执行依赖 CANN 环境，开发机上的纯 Python 测试不能替代这些步骤。模型、照片、数据库和板端报告属于运行资产，均不复制到本书源码。

推荐把证据分成四个独立门槛：

1. **文件检查**：路径存在、文件可读、ONNX 能被解析；
2. **合同检查**：输入形状、输出数量、数据类型与后处理实现一致；
3. **转换检查**：ATC 成功生成目标 OM，并保留命令和日志；
4. **运行检查**：在目标开发板上完成 ACL 加载、一次推理和资源释放。

只有完成相应门槛，文档才可以使用 “已验证” 这一表述。没有报告支撑的 FPS、延迟、准确率和召回率不应写入案例结论。

1.5. PyACL 运行时与资源生命周期

1.5.1. 初始化

应用启动时由运行时管理器完成以下工作：

1. 调用 `acl.init`，设置目标设备并创建 context 与 stream；
2. 加载检测和特征 OM，读取模型描述中的输入输出信息；

3. 为输入、输出数据集和 Device buffer 分配资源；
4. 初始化 SQLite，并启动摄像头采集线程或延迟到首次使用。

FastAPI 的 lifespan 是组织这些步骤的合适位置。它使服务启动失败能够显式呈现为 `degraded` 健康状态，而不是让请求在模型不存在时悄悄生成没有语义的向量。

1.5.2. 一次执行

对每个模型，主机把连续的 NumPy 数组拷贝到 Device buffer，调用 `acl.mdl.execute`，再将输出拷回主机并按模型合同解释。检测和特征提取应由同一个硬件工作队列串行调度；数据库查询和余弦相似度计算在主机侧完成，SQLite 读写由进程内锁与事务边界保护。

1.5.3. 释放

服务停止时应先停止接收新任务，再停止 MJPEG 生成器和摄像头线程，随后释放模型数据集、data buffer、Device buffer、模型句柄、stream、context 和设备，最后调用 `acl.finalize`。显式的 `stop()` 与线程 `join()` 比依赖 Python 析构函数更容易测试；若线程未在期限内退出，运行时必须报告失败并禁止再次创建硬件 owner。

1.6. FastAPI 与 React 的协作边界

1.6.1. API 资源

下表描述稳定的教学接口。字段由请求校验和运行时合同共同约束；错误响应使用可读的 `error` 字段，而不是把 Python 异常堆栈返回给浏览器。

方法	路径	语义
`GET`	`/api/health`	返回服务、模型和摄像头的就绪状态
`GET`	`/api/users`	返回用户列表，不包含特征向量
`POST`	`/api/users`	接收姓名与图像，提取特征并注册用户
`PUT`	`/api/users/{id}`	修改用户显示名称
`DELETE`	`/api/users/{id}`	删除用户及其关联事件
`POST`	`/api/camera/capture`	从板端摄像头抓取一张图像
`POST`	`/api/clockin`	接收上传或浏览器图像并尝试手动打卡
`GET`	`/api/attendance`	返回当天每位用户的最近一条记录

方法	路径	语义
GET	/video_feed	返回缓存帧组成的 multipart/x-mixed-replace 流
GET	/uploads/{resource}	返回已授权的头像或抓拍资源

模型缺失、加载失败或输出合同不满足时，健康接口应报告 degraded，注册和推理接口返回 HTTP 503。这比返回随机特征或空的成功响应更容易定位部署错误，也不会把无效结果写入考勤数据。

1.6.2. 浏览器界面

React 页面通过 fetch 调用上述资源，使用文件上传或浏览器 getUserMedia 产生图像。离开页面、重新抓拍或请求失败时，前端应释放媒体轨道并清理对象 URL。FastAPI 托管构建后的静态资源；/、/users_page 和 /attendance_page 作为兼容入口回退到同一个 SPA，使旧书签仍然可用。

MJPEG 预览只读取硬件线程维护的最近 JPEG，不在每个 HTTP 客户端中重新执行检测。这样可以把 “采集与推理” 和 “网络传输” 分开，避免浏览器数量改变硬件工作顺序。它仍然是教学级流式方案，不提供 WebRTC、鉴权和带宽自适应。

1.6.3. 同步与异步的选择

FastAPI 采用 ASGI，但 PyACL、OpenCV 和标准库 sqlite3 都是阻塞调用。请求处理函数不能因为使用 FastAPI 就全部改写为 `async def`；硬件操作应进入单一工作队列或受控线程池，异步层只负责等待结果和发送响应。多 worker 启动会复制模型和摄像头句柄，本案例固定单 worker。

1.7. 检测、特征与比对

1.7.1. 检测与裁剪

检测输出先经过分数阈值筛选、坐标缩放和 NMS，再得到候选框。当前示例在存在多张人脸时选择面积最大的框，作为单人考勤演示的确定性策略。空框、越界框或裁剪后尺寸为零时，应返回可解释的业务错误，不应把整张无脸图像当作有效身份输入。

1.7.2. 特征比对

设查询向量为 q，数据库向量为 v_i，余弦相似度为

$$s(q, v_i) = \frac{q \cdot v_i}{\|q\|_2 \|v_i\|_2 + \epsilon}.$$

系统在候选用户中取最大相似度，并与配置阈值比较。示例阈值是工程参数，不是经过特定数据集校准的生物识别决策边界；改变模型、摄像头、光照或用户群体后，应以独立的正负样本协议重新估计阈值，并报告误接受与误拒绝情况。

1.7.3. 两种考勤分支

- **板端自动分支**：摄像头线程定期取当前帧，检测最大人脸，完成特征比对；已匹配用户可以写入 `camera_auto` 事件，陌生人按教学策略自动登记并标注为演示事件。
- **浏览器手动分支**：React 上传文件或浏览器抓拍，API 完成同样的检测、特征和比对过程；只有匹配通过时才写入 `manual` 事件，未匹配结果返回 `match=false`。

`GET /api/attendance` 的当前语义是“当天每位用户最近一次记录”，不是完整历史审计。若研究需要多次事件、跨日期查询或导出，应另行设计带日期、用户、分页和时区参数的接口，不应仅修改页面标题。

1.8. 数据存储与隐私边界

SQLite 表保存用户名称、特征 BLOB、头像资源标识和考勤事件。特征是敏感的生物特征数据，即使转换为 BLOB 也没有自动获得加密或匿名性。教学部署应至少做到：

1. 上传文件由服务端生成不透明资源名，限制大小和图像类型，校验规范化路径位于 `data/uploads` 内；
2. API 列表不返回 embedding，资源下载经过明确的访问边界，不把上传目录直接作为公开静态目录；
3. 数据库、照片、模型和报告保留在设备本地，测试结束按实验协议删除；
4. 日志只记录必要的事件标识，不输出完整特征、原始图像或调试堆栈；
5. 自动登记仅用于教学观察，正式系统必须有知情同意、人工审核、撤销和删除流程。

这些措施是最低限度的工程边界，不等同于完整的隐私影响评估或合规认证。

1.9. 实验步骤与证据记录

1.9.1. 开发机上的纯 Python 检查

开发机不具备 CANN、PyACL、OM 和摄像头运行条件。可执行的检查包括目录合同、Python 语法、FastAPI 请求合同和使用 fake runtime 的生命周期测试：

```
cd samples/case1
python -m unittest discover -s tests -p 'test_layout.py'
python -m py_compile app.py face_attendance/*.py
```

模型检查脚本可以解析已提供的 ONNX 文件，但解析成功不等于转换成功或推理正确。测试输出应标明“静态检查”，不能写成板端性能结论。

1.9.2. 开发板上的硬件检查

在已加载 CANN 环境的 Ascend 310B 上依次完成：

1. 检查设备状态、摄像头节点和 Python `acl` 导入；
2. 检查 ONNX 输入输出合同，按目标 SoC 运行 ATC（若 OM 尚未存在）；
3. 加载两个 OM，执行一次检测和一次特征提取，确认输出形状；
4. 启动 FastAPI，检查 `/api/health`、静态首页、用户 API 和 MJPEG 首帧；
5. 用合成或经同意的测试图像完成注册、手动打卡和自动分支；
6. 发送停止信号，检查摄像头、模型、ACL context 和服务进程均已释放。

每项结果都应保留命令、环境、模型文件名、输入协议和原始日志路径。只有在具有明确测量脚本和报告时，才可增加延迟、帧率、识别率或资源占用数字。

1.9.3. API 合同示例

以下示例只说明接口形状，不包含真实图像或生产凭据：

```
curl http://127.0.0.1:5000/api/health
curl http://127.0.0.1:5000/api/users
curl -F 'name=example' -F 'image=@synthetic-face.jpg' \
  http://127.0.0.1:5000/api/users
curl -F 'image=@synthetic-face.jpg' \
  http://127.0.0.1:5000/api/clockin
```

板端服务若需要局域网访问，应显式传入受控的 `--host`，并在实验结束后恢复本机监听。默认命令不启用调试模式、自动重载或多 worker。

1.10. 局限与可扩展方向

本案例的简化点包括：最大人脸策略、全库线性检索、固定阈值、SQLite 单机写入、MJPEG 预览和教学性的陌生人自动登记。它们有助于暴露基本数据流，但不能直接推导出大规模、多摄像头、跨网络或高安全场景的结论。

合理的后续实验包括：

- 用独立的同人/异人样本校准阈值，并报告 ROC、误接受率和误拒绝率；
- 增加活体检测和人工审核，将 “未知人脸” 与 “新用户注册” 分离；
- 将向量检索从全库遍历扩展为带版本和重建策略的 ANN 索引；
- 以任务队列和限流机制研究多摄像头、多个浏览器客户端的资源竞争；
- 比较 MJPEG 与其他传输协议的带宽和延迟，但不得把协议比较结果归因于 NPU 推理速度。

1.11. 小结

案例 1 把一个看似简单的 “摄像头打卡” 问题拆解为模型合同、硬件运行时、数据比对、服务接口和隐私边界五个可以单独验证的对象。FastAPI 与 React 负责清晰地表达服务和交互，单一硬件工作线程负责稳定地使用摄像头与 NPU，SQLite 则提供可观察的本地事件存储。学习重点不是记住某个未经测量的延迟数字，而是建立从资产检查到板端证据的完整推理链，并能说明每个结论的适用范围。

1.12. 参考文献

1. Deng, J., Guo, J., Xue, N., and Zafeiriou, S. *ArcFace: Additive Angular Margin Loss for Deep Face Recognition.* CVPR, 2019. DOI: 10.1109/CVPR.2019.00482.
2. FastAPI documentation, Request files and uploads and Lifespan events.

2. 案例 2：目标跟踪检测

2.1. 教程定位

本案例面向昇腾 310B 平台的开发入门，目标不只是把程序跑起来，而是建立一条完整、清晰、可扩展的目标跟踪知识主线：

1. 为什么边缘设备上要优先选择轻量级检测网络。
2. MobileNet 的结构为什么适合做骨干网络。
3. SSD 的检测思想为什么适合实时场景。
4. MobileNet-SSD 作为检测前端，在目标跟踪任务中有哪些优点和局限。
5. 为什么在检测结果之上叠加 DeepSORT，是一个很好的多目标跟踪入门方案。
6. 本案例中的 ssdlite 与 tracking 代码，分别承担了什么角色。

从整体结构看，本案例对应一条标准的视频目标分析流水线：
本案例选择的实现路线是：

- 使用 MobileNet-SSD 完成实时目标检测。
- 使用一个简化版 DeepSORT 风格跟踪器完成多目标跟踪。
- 使用 CPU 与 Ascend NPU 统一接口，方便读者同时理解算法与部署。

这条路线适合作为昇腾 310B 开发教程中的案例章节，因为它兼顾了三点：

- 算法结构足够完整。
- 推理链路足够直观。
- 工程复杂度可控，适合实验训练和工程实践。

2.2. 实验硬件与运行条件

本案例既可以在普通 CPU 环境下运行，也可以在昇腾 NPU 环境下运行。为了完成实时检测与实时跟踪实验，建议准备如下硬件条件：

- 一台 Linux 主机或昇腾开发环境
- 一只 USB 摄像头，用于实时视频采集

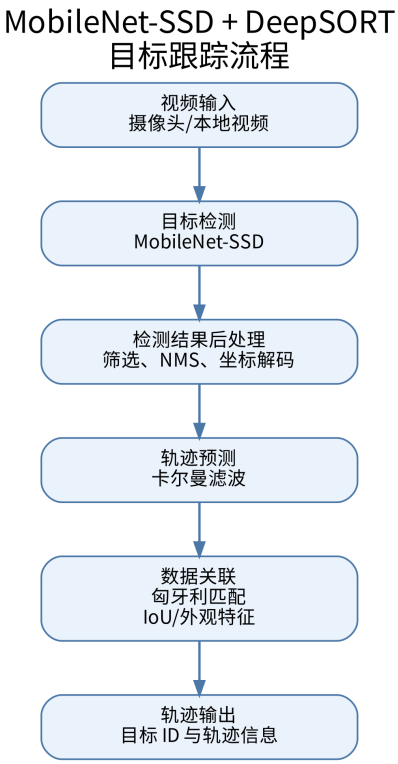

MobileNet-SSD + DeepSORT
目标跟踪流程
视频输入
摄像头/本地视频
目标检测
MobileNet-SSD
检测结果后处理
筛选、NMS、坐标解码
轨迹预测
卡尔曼滤波
数据关联
匈牙利匹配
IoU/外观特征
轨迹输出
目标 ID 与轨迹信息

图 2.1. 程序流程图

- Ascend 310B 或兼容昇腾 NPU 设备，运行 `npu` 模式时需要

运行方式可以分为两类：

- 实时演示：接入 USB 摄像头，使用 `--source 0` 作为输入
- 离线演示：直接读取本地视频文件，例如 `--source demo.mp4`

如果只运行 `cpu` 模式，可以没有昇腾 NPU；如果运行 `npu` 模式，则还需要本机正确安装 Ascend ACL Python 运行时，并准备对应的 `.om` 模型文件。当前模型下载脚本默认使用 `zhouxzh/SSDLite320`，该仓库发布的是 `.onnx` 模型；在 Ascend 310B 上运行 NPU 模式前，需要使用 `scripts/convert_onnx_to_om.py` 将 ONNX 转成 OM。

常用准备流程如下：

```
python scripts/download_models.py --onnx
source /usr/local/Ascend/ascend-toolkit/set_env.sh
python scripts/convert_onnx_to_om.py --soc-version Ascend310B4
```

转换脚本默认读取 `models/*.onnx`，并把同名 `.om` 文件写回 `models/`。如果板端 SoC 版本不是 `Ascend310B4`，需要按实际环境修改 `--soc-version`。

2.3. 本案例支持的骨干网络

当前仓库里的 SSD 模型已经覆盖了两类主干网络：

- MobileNet 系列：`mobilenetv1`、`mobilenetv2`、`mobilenetv3`、`mobilenetv3_large_100`、`mobilenetv4`
- ResNet 系列：`resnet18`、`resnet34`、`resnet50`、`resnet101`、`resnet151`

其中：

- `ssd320_*` 主要对应 MobileNet 系列，输入尺寸通常为 `320 x 320`
- `ssd300_*` 主要对应 ResNet 系列，输入尺寸通常为 `300 x 300`

从工程角度看，MobileNet 系列更强调轻量化和实时性，ResNet 系列更强调特征表达能力和检测稳定性。两类骨干网络共同构成了本案例中 “轻量实时” 和 “更强表达” 两种不同取向的实验基础。

2.4. 目标跟踪对时序关联的要求

目标检测解决的是单帧图像中的两个问题：

- 目标在哪里。

- 目标属于什么类别。

例如，在一帧图像里，检测器可能输出三个框，其中两个是行人，一个是汽车。但仅靠检测器，我们无法回答下面这些时间维度上的问题：

- 当前这一帧左侧的行人，是不是上一帧的 3 号目标。
- 被遮挡后重新出现的车辆，是否还是之前那一辆。
- 两个目标交叉通过时，系统该如何保持它们的身份不交换。

上述问题正是目标跟踪所要解决的核心任务。多目标跟踪系统不仅需要检测目标的存在，还需要为同一目标在连续帧中维持稳定的身份编号，即稳定的 ID。

因此，目标跟踪可以理解为“目标检测在时间维度上的延伸”。检测负责看见目标，跟踪负责维持目标身份。

2.5. MobileNet-SSD 作为检测前端的依据

2.5.1. 边缘侧场景的基本要求

昇腾 310B 面向边缘智能计算场景。在这类场景中，算法设计通常要同时考虑以下问题：

- 实时性是否足够。
- 模型规模是否可控。
- 推理链路是否简单稳定。
- 是否便于部署到 NPU 或 CPU 回退环境。

如果直接选择参数量庞大、结构复杂、后处理繁重的检测器，理解成本与工程调试成本都会显著上升，系统主线反而不易把握。因此，本案例采用轻量、经典且工程路径清晰的 MobileNet-SSD 组合作为检测前端。

2.5.2. MobileNet 的网络结构

MobileNet 的核心设计思想，是用深度可分离卷积替代标准卷积，从而显著降低参数量与计算量。

标准卷积把空间卷积和通道融合同时完成，而 MobileNet 将其拆分成两步：

1. Depthwise Convolution：对每个输入通道分别做空间卷积。
2. Pointwise Convolution：使用 1×1 卷积完成通道之间的信息融合。

如果标准卷积的输入通道数为 M，输出通道数为 N，卷积核大小为 $K \times K$，特征图大小为 $D \times D$，那么标准卷积的计算量大致为：

$$D^2 \cdot K^2 \cdot M \cdot N$$

而深度可分离卷积的计算量大致为：

$$D^2 \cdot K^2 \cdot M + D^2 \cdot M \cdot N$$

当 $K = 3$ 时，这种拆分能够显著降低计算成本。这一特性对于边缘设备尤为重要。
从结构上看，MobileNet 可以理解为由以下几类模块反复堆叠而成：

- 普通卷积层，用于最初级的特征提取。
- 深度卷积层，用于逐通道提取空间信息。
- 逐点卷积层，用于跨通道融合特征。
- 下采样层，用于逐步扩大感受野，获得更强的语义信息。

MobileNet 的主要价值体现在以下几个方面：

- 它很好地体现了轻量化网络的设计思想。
- 它适合作为边缘侧模型部署的入门例子。
- 它与 SSD 结合后，能够形成一条结构清晰、速度较快的检测链路。

2.5.3. MobileNet v1 至 v4 的版本演进

MobileNet 并非单一网络，而是一个持续演进的轻量级网络家族。v1 至 v4 的演进，本质上体现为在移动端与边缘侧约束下，对精度、速度与结构表达能力的持续平衡与优化。

MobileNet v1

MobileNet v1 是这个家族的起点。它最核心的贡献，就是把深度可分离卷积系统化地应用到整个网络主干中。

结构特点：

- 以 depthwise convolution 加 pointwise convolution 为基本单元
- 网络结构规整，堆叠方式简单直接
- 参数量和计算量相比标准卷积网络大幅下降

优点：

- 结构最直观，最容易理解
- 轻量化效果明显
- 在边缘设备上容易部署

局限：

- 特征表达能力相对有限
- 在相同算力下，精度通常不如后续版本

MobileNet v2

MobileNet v2 的核心改进是引入了 inverted residual 和 linear bottleneck。
结构特点：

- 先通过 1×1 卷积扩张通道
- 再做 depthwise convolution
- 最后通过线性 bottleneck 压缩回较低维度
- 在满足条件时使用残差连接

它与传统残差块的设计思路存在明显差异。传统残差块通常采用 “宽输入到宽输出” 的结构，而 MobileNet v2 更强调 “窄输入、宽中间层、窄输出”，因此被称为 inverted residual。

这一版本的优势在于：

- 在轻量化前提下提升了表达能力
- 残差连接帮助梯度传播更稳定
- 在线性 bottleneck 中减少非线性带来的信息损失

MobileNet v3

MobileNet v3 在 v2 基础上继续优化，它融合了神经网络结构搜索和轻量注意力机制的思想。

结构特点：

- 延续 inverted residual 主体框架
- 引入 SE 模块增强通道注意力
- 使用 h-swish 等更适合移动端的激活函数
- 网络结构由精度与延迟共同驱动优化

与 v2 相比，v3 的一个重要特征在于更加关注真实硬件上的执行效率，而不仅仅是理论 FLOPs 的下降。换言之，v3 的优化目标并非单纯减少计算量，而是在移动端与边缘端设备上取得更优的实际速度与精度平衡。

MobileNet v4

MobileNet v4 是更后续的轻量骨干版本，它进一步强化了硬件感知设计，更强调不同算子组合在实际设备上的性能收益。

结构特点：

- 延续轻量主干的整体方向
- 更强调不同卷积模块和块结构的组合效率
- 进一步面向硬件友好的延迟优化
- 在不同配置下兼顾速度、参数量和表达能力

从本仓库的模型组织可以看出,模型目录中既包含 `mobilenetv4`,也包含 `mobilenetv4_conv_large.onnx`、`mobilenetv4_hybrid_medium.onnx` 这样的变体文件。这表明 v4 系列本身已经具备不同配置路径，用于在更强表达能力与更低计算开销之间进行权衡。

2.5.4. MobileNet v1-v4 的结构对比总结

如果按结构演进主线来总结，四个版本的差异可以概括为：

- v1：用深度可分离卷积解决 “怎么把网络做轻”
- v2：用 inverted residual 和 linear bottleneck 解决 “轻量化下如何保留表达能力”
- v3：用 SE、h-swish 和结构搜索解决 “如何进一步优化真实设备上的速度与精度平衡”
- v4：进一步走向硬件感知和模块组合优化，强调在实际部署环境中的综合收益

如果按本案例的应用视角看，这些版本的差异意味着：

- v1 更适合讲轻量化网络的最基础结构
- v2 更适合讲轻量网络中的残差和瓶颈设计
- v3 更适合讲轻量网络与注意力机制、激活函数优化的结合
- v4 更适合讲面向部署性能的后续演化方向

2.5.5. MobileNet 与 ResNet 骨干在本案例中的取向差异

本案例同时提供 MobileNet 和 ResNet 两条骨干路线，这样可以更直观地对比不同网络家族的使用取向。

MobileNet 系列的特点是：

- 参数量更小
- 推理速度更快
- 更适合实时演示和边缘部署

ResNet 系列的特点是：

- 特征提取能力更强
- 网络层次更深，残差结构更成熟
- 在一些场景下更容易得到稳定的检测效果

因此，在本案例中：

- 如果更关注实时性和轻量部署，可以优先尝试 MobileNet v1-v4
- 如果更关注骨干网络深度和表达能力，可以尝试 ResNet18 到 ResNet151 的多个版本

2.5.6. SSD 的检测特点

SSD 是 Single Shot MultiBox Detector 的缩写，属于典型的一阶段检测器。它的核心特点是：不需要像两阶段检测器那样先生成候选区域，再做分类与回归，而是直接在不同尺度的特征图上预测目标位置和类别。

SSD 的关键思想包括：

- 在多个尺度特征图上进行检测，兼顾大目标和小目标。
- 在每个位置预先定义一组 default boxes，也叫 prior boxes。
- 网络直接回归 default box 的位置偏移量，同时预测各类别分数。
- 经过解码和 NMS 后，输出最终的检测框。

SSD 的优点主要有：

- 推理速度快，适合实时场景。
- 结构相对直接，便于理解和部署。
- 检测头设计清晰，容易解释 “分类” 和 “回归” 两个分支的作用。

SSD 的局限也需要明确说明：

- 对小目标和密集目标场景通常不如更新的检测器稳定。
- 检测效果较依赖先验框设计与输入尺寸。
- 在复杂遮挡场景下，单帧检测质量会波动，从而影响后续跟踪。

2.5.7. MobileNet-SSD 用于目标跟踪的优缺点

在目标跟踪任务中, 检测器不是孤立存在的, 它会直接影响轨迹质量。MobileNet-SSD 作为跟踪前端，有以下优点：

- 轻量高效，适合边缘侧实时处理。

- 输出结构标准，便于接入后续跟踪器。
- 检测速度较快，可以为多目标跟踪提供稳定的帧级观测。
- 工程复杂度低，便于构建完整案例。

但它也存在明显不足：

- 对小目标和远距离目标的检测精度有限。
- 遇到密集遮挡时，漏检和框抖动会增多。
- 当检测框位置不稳定时，跟踪器更容易发生 ID switch。
- 仅靠检测框几何信息，难以应对长时间遮挡或重识别问题。

因此，MobileNet-SSD 的价值不在于 “它是最强的检测前端”，而在于 “它能以较低复杂度，把检测到跟踪这条主线讲清楚”。

2.6. DeepSORT 作为跟踪后端的选择依据

2.6.1. 从检测到跟踪，还缺少什么

检测器每一帧都输出一组目标框，但这些框之间没有时间关联。要把检测结果变成稳定轨迹，还需要以下机制：

- 轨迹状态表示。
- 帧间运动预测。
- 检测与轨迹之间的匹配。
- 轨迹的创建、保留和删除。

DeepSORT 恰好提供了这样一套结构完整的解决框架。

2.6.2. DeepSORT 为什么适合作为入门算法

完整的 DeepSORT 在 SORT 基础上增加了外观特征建模，因此在目标遮挡、交叉和重识别问题上通常具有更好的稳定性。尽管本案例并未实现完整的工业级 DeepSORT，但保留了其最核心、最适合入门阶段把握的基本思路：

- 用卡尔曼滤波做运动预测。
- 用匈牙利算法做全局匹配。
- 用 IOU 作为主要几何相似度。
- 用轨迹生命周期参数管理目标的出现与消失。

这种简化版方案有三个突出优点：

- 思路完整，已经覆盖多目标跟踪中的核心环节。
- 数学和工程难度适中，不需要先掌握 ReID 网络训练。
- 便于直接观察检测质量、IOU 阈值、轨迹寿命等因素对最终效果的影响。

因此，将 MobileNet-SSD 与 DeepSORT 组合使用，是一条结构清晰、实现成本适中且便于展开分析的入门路径。

2.7. 本案例的代码结构与总体流程

本案例的工程代码主要分为三层：

2.7.1. 入口层

- scripts/detection_app.py：只做实时检测。
- scripts/tracking_app.py：先做检测，再做跟踪。

2.7.2. 检测层

- ssdlite/backend_base.py：统一检测后端基类，负责预处理、推理调度与输出解码。
- ssdlite/cpu_backend.py：ONNXRuntime CPU 推理实现。
- ssdlite/npu_backend.py：Ascend ACL NPU 推理实现。
- ssdlite/decoder.py：SSD prior boxes、位置解码、类别概率与 NMS。

2.7.3. 跟踪层

- tracking/deepsort.py：轨迹对象、匹配逻辑、轨迹生命周期管理。
- tracking/kalman_filter.py：卡尔曼滤波状态预测与观测更新。
- utils/postprocessing.py：检测结果转换为跟踪器输入，并将轨迹画回图像。

整条链路的执行顺序可以概括为：

1. 读取摄像头或视频流。
2. 对输入帧执行 MobileNet-SSD 检测。
3. 解码输出张量并得到边界框、类别和分数。
4. 将检测结果整理成跟踪器统一输入格式。
5. 对已有轨迹进行预测。
6. 将当前检测结果与历史轨迹做关联。

7. 更新匹配轨迹、创建新轨迹、删除失效轨迹。
8. 在图像上绘制检测结果或轨迹结果。

2.8. 检测部分代码解析

2.8.1. detection_app.py 如何组织检测主流程

在 scripts/detection_app.py 中，主程序首先解析参数，然后根据 device 选择后端，再进入逐帧推理循环。这一设计体现了很清晰的工程结构：入口脚本只负责流程编排，把模型相关细节下沉到 ssdlite 模块。

主流程的示意代码如下：

```
labels = load_labels(args.labels)
model_path = resolve_model_path(args.model, args.backbone, model_dir, args.device)
backend = CpuBackend(model_path) or NpuBackend(model_path)

while True:
    ok, frame = cap.read()
    detections, profile_ms = backend.infer_with_profile(
        frame,
        args.score_threshold,
        args.nms_threshold,
        args.max_detections,
    )
    annotated = draw_detections(...)
```

这里需要关注的重点并非某一行语法细节，而是模块职责的划分方式：

- 入口脚本负责参数、视频流、显示和保存。
- backend 负责模型推理。
- decoder 负责把原始张量变成检测框。
- postprocessing 负责可视化。

这是一种很清晰的工程拆分方式。

2.8.2. backend_base.py 如何统一检测后端

ssdlite/backend_base.py 是检测部分最核心的代码之一。它把 CPU 与 NPU 两种后端统一到同一个抽象接口下。这样设计后，入口脚本就不用关心底层到底是 ONNXRuntime 还是 ACL。

这个基类做了三件关键事情：

- 根据模型名推断输入尺寸。

- 统一图像预处理。
- 统一输出解码流程。

其中，infer_with_profile 的核心逻辑如下：

```
input_tensor = preprocess_frame(frame, self.input_hw)
outputs = self._run_model(input_tensor)
detections = decode_detections(
    outputs,
    frame.shape,
    score_threshold,
    nms_threshold,
    max_detections,
    self.ssd_decoder,
    self.strict_ssd,
)
```

这段代码清晰地对应了检测系统中的三个阶段：

1. 预处理：把原始图像缩放、归一化，并变成模型要求的 NCHW 张量。
2. 推理：调用不同后端执行前向计算。
3. 解码：把输出张量还原成真实的目标框与类别分数。

这里需要明确的一点是，模型推理本身并不等于最终检测结果。真正可用于后续跟踪的目标框，需要由模型输出经过解码与 NMS 后处理后才能得到。

2.8.3. preprocess_frame 体现了部署输入规范

在 backend_base.py 中，preprocess_frame 完成了以下操作：

- BGR 转 RGB。
- resize 到固定输入尺寸。
- 像素值归一化到 0 到 1。
- 按 ImageNet 风格均值与方差做标准化。
- 从 HWC 转为 CHW。
- 增加 batch 维度。

这说明了一个重要的部署原则：模型输入的预处理流程必须与训练阶段的约定保持一致，否则推理结果将发生明显偏移。

2.8.4. decoder.py 如何体现 SSD 的核心思想

ssdlite/decoder.py 直接对应了 SSD 的理论结构，是整条检测链路中的关键部分。

default boxes

在 DefaultBoxes 类中，代码根据不同特征图尺度和长宽比生成先验框。这正是 SSD 的基础：网络并不是直接从零开始生成框，而是在预定义框基础上学习偏移量。

位置回归的反变换

在 SSDDecoder.scale_back_batch 中，网络输出的并不是最终框坐标，而是相对于默认框的偏移量。代码中先利用 scale_xy 和 scale_wh 对偏移量进行还原，再把中心点形式转换为左上角和右下角坐标。

更直接地说：

- 网络先预测相对于先验框的中心偏移和宽高缩放。
- 解码阶段再把这些相对量恢复成图像上的实际边界框。

分类分数与 NMS

当前工程版本中的 decode_single 已经不是 “对每个类别都扫描全部 prior” 的基础写法，而是先对每个 prior 选出一个最佳前景类别，再进入后续筛选和 NMS。其主线思路更接近下面这样：

```
foreground_scores = scores_in[:, class_ids]
best_class_indices = np.argmax(foreground_scores, axis=1)
best_scores = foreground_scores[np.arange(foreground_scores.shape[0]), best_class_indices]
keep_score = best_scores >= score_threshold

candidate_boxes = bboxes_in[keep_score, :]
candidate_scores = best_scores[keep_score]
candidate_labels = class_ids[best_class_indices[keep_score]]
```

随后，代码再按类别分别执行 NMS。这样做体现了 SSD 后处理中的三个关键工程点：

- 先为每个 prior 找到最可信的前景类别。
- 再根据阈值筛掉低置信度候选框。
- 最后只对保留下来的候选框按类别做 NMS。

如果仍然采用 “每个类别都扫全部 prior” 的方式，那么在 COCO 这类多类别场景里，解码阶段会产生大量无效扫描和重复 NMS，实时性会明显受影响。

NMS 的作用是什么

NMS 是 Non-Maximum Suppression，即非极大值抑制。它的核心作用可以概括为一句话：当多个候选框同时指向同一个目标时，只保留其中最可信的候选框，并删除其余高度重叠的框。

这是因为 SSD 会在每个特征图位置、每种先验框形状上产生候选框。对于一个真实目标，模型往往不会只输出一个框，而是会输出一组位置接近、分数相近的候选框。如果不执行 NMS，画面中就会出现大量相互覆盖的检测框，例如同一行人周围同时叠加 5 个到 10 个候选框。

NMS 的作用主要有四点：

- 去除重复检测，让一个目标尽量只保留一个主框。
- 降低后续显示和统计的混乱程度。
- 为跟踪器提供更稳定、更稀疏的观测输入。
- 避免同一目标在跟踪阶段被误认为多个目标。

对本案例尤其要强调最后一点。如果检测器把同一目标输出成多个高重叠框，那么 tracking 模块在创建轨迹时，可能会把这些框错误地当成多个独立目标，从而造成重复轨迹、ID 混乱和后续匹配不稳定。

NMS 的处理逻辑可以概括成下面 4 步：

1. 按置信度从高到低排序所有候选框。
2. 取当前分数最高的框作为保留框。
3. 计算其余框与该保留框的 IOU。
4. 删除所有 IOU 超过阈值的框，然后继续处理剩余框。

因此，NMS 本质上是在做 “去重”。它不是提高分类能力，而是在后处理阶段把重复候选框压缩成更干净的检测结果。

ssdlite 中 _nms 的实现解析 {#src-experiment-case2-h33}

当前工程版本中的 NMS 函数如下：

```
def _nms(boxes, scores, iou_threshold, presorted=False):
    if len(boxes) == 0:
        return np.empty((0,), dtype=np.int64)

    if presorted:
        order = np.arange(scores.size, dtype=np.int64)
    else:
        order = np.argsort(scores)[::-1]

    keep = []
    while order.size > 0:
        index = order[0]
        keep.append(index)
        if order.size == 1:
            break

        remaining = order[1:]
```

```
        iou = _calc_iou_with_box(boxes[remaining], boxes[index])
        order = remaining[iou <= iou_threshold]

    return np.array(keep, dtype=np.int64)
```

这段实现虽然短，但几乎完整体现了经典 NMS 的核心流程。

第一步，处理空输入：

```
if len(boxes) == 0:
    return np.empty((0,), dtype=np.int64)
```

这一步是工程上必不可少的防御式处理。如果当前类别一个候选框都没有，就直接返回空索引，避免后续排序和 IOU 计算报错。

第二步，确定候选框顺序：

```
if presorted:
    order = np.arange(scores.size, dtype=np.int64)
else:
    order = np.argsort(scores)[::-1]
```

这里多了一个 `presorted` 参数。其作用是：如果外部已经按分数把候选框排好了，就不必在 NMS 内部重复排序。这样可以减少一次多余的排序开销。

第三步，循环保留最高分框：

```
keep = []
while order.size > 0:
    index = order[0]
    keep.append(index)
```

这里的含义至关重要：每一轮循环都将当前最高分框作为 “代表框” 保留下来。NMS 的基本原则是，在一组高度重叠的候选框中，分数最高的框通常最应优先保留。

第四步，只剩一个框时直接结束：

```
if order.size == 1:
    break
```

如果当前只剩一个候选框，那么它已经被保留，没有必要再做 IOU 比较。

第五步，计算其余候选框与当前最高分框的 IOU：

```
remaining = order[1:]
iou = _calc_iou_with_box(boxes[remaining], boxes[index])
```

这一步可以这样理解：

- `boxes[index]` 是当前保留框。
- `boxes[remaining]` 是除了它以外剩余的候选框。

- `_calc_iou_with_box(...)` 直接计算 “多框对单框” 的 IOU。

如果某个候选框与保留框的 IOU 很高，就说明这两个框大概率描述的是同一个目标，此时就没有必要同时保留。

第六步，删除 IOU 过高的重复框：

```
order = remaining[iou <= iou_threshold]
```

这一行代码是整个 NMS 实现中最关键的语句之一。它表示：

- 只保留那些与当前主框重叠不太严重的候选框。
- 不再通过 `np.where(...)+1` 做额外索引映射，而是直接对 `remaining` 做布尔筛选。

执行完这一行后，所有与当前主框高度重叠的框都会被移除，剩下的框继续进入下一轮循环。

第七步，返回保留框索引：

```
return np.array(keep, dtype=np.int64)
```

最终返回的不是框本身，而是被保留框在原数组中的索引。后续代码再用这些索引去提取真正的框、分数和类别。

calc_iou 为什么是 NMS 的基础

NMS 能否正确工作，取决于 IOU 计算是否准确。本案例在 decoder.py 中通过 calc_iou 计算两个框集合之间的交并比。其核心思想是：

$$IOU = \frac{\text{交集面积}}{\text{并集面积}}$$

在代码里，这一过程分为三步：

- 先求两个框相交区域的左上角和右下角。
- 再计算交集面积。
- 最后用交集面积除以并集面积。

如果 IOU 接近 1，说明两个框几乎重合；如果 IOU 接近 0，说明两个框几乎没有重叠。NMS 正是利用这一数值来判断 “两个候选框是不是在描述同一个目标”。

NMS 阈值对结果的影响

NMS 中最重要的超参数是 `iou_threshold`。它决定了 “多大程度的重叠应被视为重复”。

如果阈值较小，例如 0.3，那么算法会更严格：

- 只要两个框稍微重叠得多一点，就可能删除其中一个。

- 输出框更少，更干净。
- 但也可能误删本来相邻但不是同一目标的框。

如果阈值较大，例如 0.6 或 0.7，那么算法会更宽松：

- 允许更多重叠框同时保留。
- 对密集目标场景可能更友好。
- 但重复检测也更容易残留。

因此，NMS 阈值本质上是在 “去重强度” 和 “保留相邻目标” 之间做权衡。

NMS 对跟踪效果的直接影响

NMS 容易被视为检测模块内部的技术细节，但在目标跟踪系统中，它实际上会直接影响跟踪质量。

原因是跟踪器把检测器输出当作观测输入。如果检测结果中存在大量重复框，就会产生以下问题：

- 同一目标可能在同一帧中生成多条新轨迹。
- 轨迹关联时会出现多对一竞争。
- 轨迹数目虚高，画面看起来混乱。
- 目标 ID 容易抖动，甚至频繁切换。

从这个角度看，NMS 不是孤立的后处理技巧，而是整个 “检测到跟踪” 流水线稳定运行的重要保障。

decode_single 中 NMS 的完整上下文

在本案例中，NMS 不是单独执行的，而是被放在 decode_single 的类别循环中。其完整语义是：

1. 针对每一个类别单独处理。
2. 先按分数过滤掉置信度过低的框。
3. 再保留该类别中得分最高的一部分候选框。
4. 最后在这个类别内部做 NMS。

这意味着本案例采用的是 “按类别分别做 NMS” 的策略。这样设计的优点是：不同类别之间不会互相抑制。例如一个行人框和一个自行车框即使重叠较大，也不应该因为彼此重叠就被删除。

2.8.5. CPU 与 NPU 后端如何被统一接入

本案例中的 ssdlite/cpu_backend.py 与 ssdlite/npu_backend.py 都继承自 DetectionBackend，这意味着它们对外暴露的是同一套接口。这种设计有两层价值：

- 从算法角度看，检测逻辑不依赖具体硬件后端。
- 从工程角度看，同一套上层代码可以在 CPU 和 Ascend NPU 之间切换。

这也体现了一个重要的工程原则：算法主线尽量稳定，硬件适配尽量下沉到后端实现层。

2.9. 从检测到跟踪：tracking_app.py 的桥接作用

如果说 detection_app.py 用于建立检测流程，那么 tracking_app.py 的作用就在于将 “检测结果如何转化为轨迹” 这一过程完整串联起来。

在 scripts/tracking_app.py 中，检测模块与跟踪模块之间的桥接代码如下：

```
detections, profile_ms = backend.infer_with_profile(...)
tracker_inputs = detections_to_tracker_inputs(detections)
tracks = tracker.update(tracker_inputs)
annotated = draw_tracks(...)
```

这一过程可以分为四个连续步骤：

1. 先由检测器输出本帧的目标集合。
2. 再把字典形式的检测结果转成跟踪器统一使用的数组格式。
3. 调用跟踪器更新轨迹状态。
4. 把最新轨迹结果绘制到图像上。

其中，utils/postprocessing.py 中的 detections_to_tracker_inputs 函数具有关键作用，因为它完成了检测模块与跟踪模块之间的数据接口统一。转换后的输入格式为：

$$[x_1, y_1, x_2, y_2, score, class_id]$$

这正是后续轨迹匹配与分类约束所依赖的输入格式。

2.10. 跟踪部分代码解析

2.10.1. Track 类如何表示一条轨迹

tracking/deepsort.py 中的 Track 类代表单个目标的历史状态。每条轨迹至少维护了以下信息：

- track_id：目标编号。
- bbox：当前边界框。
- score：当前检测置信度。
- class_id：目标类别。
- trail：历史中心点序列，用于绘制轨迹拖尾。
- kalman_filter：用于状态预测与更新。
- time_since_update：距离上次匹配成功已经过去多少帧。
- hits：这条轨迹累计被成功匹配了多少次。

这表明轨迹并不是静态边界框的简单记录，而是一个具有时间属性的状态对象。因此，“检测框” 和 “轨迹” 必须明确区分。

2.10.2. predict 和 update 对应卡尔曼滤波两阶段

Track 类中最核心的两个方法是 predict 和 update。

predict 的作用是：即使当前帧还没有拿到新的检测结果，也先根据上一时刻状态预测目标现在大概会出现在哪里。

update 的作用是：一旦当前帧有新的检测框匹配到这条轨迹，就用新的观测值修正状态估计。

这正对应卡尔曼滤波的两步：

1. Predict：根据上一时刻状态外推当前状态。
2. Update：用当前观测修正预测误差。

本案例里，状态量采用了一个比较直观的简化形式：

$$x = [x, y, v_x, v_y]^T$$

其中 x, y 表示目标中心位置，v_x, v_y 表示速度。这样做的好处是：数学含义直观，代码实现也容易读懂。

2.10.3. kalman_filter.py 如何实现线性状态估计

tracking/kalman_filter.py 中定义了最基础的卡尔曼滤波器。其状态转移矩阵为：

$$F = \begin{bmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

这表示系统假设目标在相邻帧之间近似满足匀速运动。观测矩阵为：

$$H = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix}$$

这表示我们能够直接观测到的是目标中心位置，而不是速度。

这一实现虽然经过简化，但其价值十分明确，因为它将 “运动预测” 与 “观测修正” 这两个概念直接落实到了矩阵运算之中。

2.10.4. 卡尔曼滤波器的发展历史与作者信息

卡尔曼滤波器的提出者是鲁道夫·埃米尔·卡尔曼，英文名 Rudolf Emil Kalman。他是匈牙利裔美国数学家和控制理论学者，1960 年发表了著名论文 A New Approach to Linear Filtering and Prediction Problems，系统建立了线性动态系统状态估计的现代形式。

从历史背景看，卡尔曼滤波器并不是突然出现的。它的形成与 20 世纪中叶的控制理论、随机过程、航空航天导航和军事工程需求密切相关。第二次世界大战后，工程界越来越需要解决这样一类问题：

- 系统状态不能被直接完整观测。
- 传感器观测存在噪声。
- 系统本身还在不断运动和变化。

例如，雷达跟踪飞机、导弹制导、航天器轨道估计，本质上都属于这一类问题。

卡尔曼滤波器之所以具有里程碑意义，是因为它把 “预测” 和 “校正” 统一进了一个递推框架中：

- 先根据上一时刻状态和系统模型预测当前状态。
- 再根据当前观测对预测结果进行修正。

这一思想后来在 20 世纪 60 年代迅速应用到阿波罗计划的导航系统中，也因此被广泛传播到航空航天、自动控制、机器人、计算机视觉和金融工程等领域。

在发展过程中，卡尔曼滤波形成了多个重要分支：

- 标准卡尔曼滤波：用于线性高斯系统。
- 扩展卡尔曼滤波 EKF：用于非线性系统的一阶近似。
- 无迹卡尔曼滤波 UKF：通过采样点传播非线性分布。
- 粒子滤波：进一步放宽线性和高斯假设。

本案例采用最基础的标准卡尔曼滤波器。采用这一形式，并非因为它代表当前最复杂的状态估计方案，而是因为它在入门阶段具有最清晰的解释路径：

- 数学形式清晰。
- 与目标跟踪中的位置预测问题高度匹配。
- 代码实现短小，便于直接对应矩阵运算理解。

2.10.5. 卡尔曼滤波器到底在解决什么问题

在目标跟踪里，卡尔曼滤波器要解决的问题可以表述为：当检测器每一帧给出的目标框存在抖动、漏检和噪声时，如何根据历史状态更平滑地估计目标当前位置，并在短时没有观测的情况下继续维持轨迹。

这意味着卡尔曼滤波并不是在做 “目标分类”，也不是在做 “目标匹配”，它做的是状态估计。

在本案例里，这个状态估计问题被简化成了二维平面上的匀速运动模型：

$$x_k = Fx_{k-1} + w_k$$

$$z_k = Hx_k + v_k$$

其中：

- x_k 表示第 k 帧的隐藏状态，也就是目标真实但不可完全直接观测的状态。
- z_k 表示第 k 帧的观测，也就是检测器给出的目标中心位置。
- F 是状态转移矩阵。
- H 是观测矩阵。
- w_k 是过程噪声。
- v_k 是观测噪声。

从工程角度看，这个模型表达的是一个朴素但有效的假设：目标会以相对平滑的方式运动，而检测器给出的观测值只是对真实运动状态的带噪声测量。

2.10.6. 本案例中的 predict 与 update 如何对应算法公式

tracking/kalman_filter.py 中的两个核心函数正好对应卡尔曼滤波的两个阶段。

第一阶段是预测：

```
def predict(self):
    self.x = self.F @ self.x
    self.P = self.F @ self.P @ self.F.T + self.Q
    return self.x
```

该阶段完成以下两项操作：

- 用状态转移矩阵 `F` 预测新的状态向量 `x`。
- 用协方差传播公式预测新的不确定性 `P`。

第二阶段是更新：

```
def update(self, z):
    measurement = np.asarray(z, dtype=np.float32).reshape((2, 1))
    innovation = measurement - self.H @ self.x
    innovation_covariance = self.H @ self.P @ self.H.T + self.R
    kalman_gain = np.linalg.solve(innovation_covariance.T, (self.P @ self.H.T).T).T
    self.x = self.x + kalman_gain @ innovation

    correction = self._identity - kalman_gain @ self.H
    self.P = correction @ self.P @ correction.T + kalman_gain @ self.R @ kalman_gain.T
```

这段代码中几个量的意义需要讲清楚：

- `innovation` 是残差，表示 “观测值与预测值之间的差”。
- `innovation_covariance` 是残差协方差，表示当前误差的不确定性。
- `kalman_gain` 是卡尔曼增益，决定 “应该更相信模型预测还是更相信当前观测”。

当前实现和基础版相比还多了两点工程优化：

- 用 `np.linalg.solve` 代替直接求逆，数值更稳定。
- 用 Joseph form 更新协方差矩阵，更容易保持协方差的对称性和正定性。

这里最重要的量仍然是卡尔曼增益。它并非经验性设定的固定权重，而是由当前状态不确定性与观测噪声共同决定的自适应权重。

可以直观地理解为：

- 如果观测噪声很大，滤波器会更信任模型预测。
- 如果观测噪声较小，滤波器会更信任当前检测结果。

这也是卡尔曼滤波器的重要优势之一。它并不是简单的滑动平均，而是在概率意义上进行最优线性估计。

2.10.7. 卡尔曼滤波在本案例中的适用性

对本案例而言，卡尔曼滤波器之所以合适，并不在于目标运动一定严格符合线性高斯模型，而在于它在工程上实现了较好的综合平衡：

- 算法复杂度低。
- 推理和更新速度快。
- 能明显改善检测框抖动。
- 在短时遮挡和漏检场景下能维持轨迹连续。

如果在入门阶段直接引入 EKF、UKF 或粒子滤波，虽然理论上能够处理更复杂的运动模型，但理解成本会显著上升，主线结构反而更不易把握。因此，本案例采用标准卡尔曼滤波具有充分的合理性。

2.10.8. DeepSORT.update 如何体现多目标跟踪主线

tracking/deepsort.py 中的 DeepSORT.update 是整条跟踪主线最核心的函数，其执行顺序如下：

```
for track in self.tracks:
    track.predict()

matched, unmatched_detections, _ = self._associate(detections)

for track_idx, detection_idx in matched:
    self.tracks[track_idx].update(detections[detection_idx])

for detection_idx in unmatched_detections:
    self._create_track(detections[detection_idx])

self.tracks = [track for track in self.tracks if track.time_since_update <= self.max_age]
```

这一段代码几乎完整体现了多目标跟踪的标准流程：

1. 对所有旧轨迹做预测。
2. 把当前检测结果与旧轨迹做关联。
3. 用匹配到的检测结果更新旧轨迹。
4. 对未匹配检测创建新轨迹。
5. 删除长期未更新的轨迹。

换言之，跟踪并不是对检测框进行简单编号，而是在连续帧之间持续执行预测、匹配、更新与清理。

2.10.9. 数据关联为什么使用 IOU + 匈牙利算法

在 _associate 中，当前工程版本会先构造轨迹与检测之间的 IOU 矩阵和类别兼容矩阵，再调用匈牙利算法获得全局最优分配。对应代码思路如下：

```
iou_matrix = self._calculate_iou_matrix(detections)
class_mask = self._calculate_class_compatibility_matrix(detections)
assignment_scores = np.where(class_mask, iou_matrix, -1.0)
matched_indices = self._linear_assignment(assignment_scores)
```

之所以这样设计，是因为当场景中有多条轨迹和多个检测框时，不能只做局部贪心匹配。匈牙利算法的作用在于：

- 从全局角度寻找一一对应的最优匹配。
- 避免一个检测框同时分配给多条轨迹。
- 避免一条轨迹同时匹配多个检测框。

本案例还加入了类别兼容矩阵：

```
class_mask = self._calculate_class_compatibility_matrix(detections)
assignment_scores = np.where(class_mask, iou_matrix, -1.0)
```

这说明系统不仅看几何位置，还会在进入全局匹配之前，先利用类别信息屏蔽明显不合理的配对关系，避免 “行人轨迹匹配到车辆框” 这类明显错误。

2.10.10. 匈牙利算法的发展历史与作者信息

匈牙利算法是解决二分图最优匹配和指派问题的经典算法。它最早由美国数学家哈罗德·W·库恩，英文名 Harold W. Kuhn，在 1955 年系统提出。之所以命名为 “匈牙利算法”，是因为库恩的工作建立在两位匈牙利数学家的理论成果之上：

- 德奈什·克尼格，英文名 Dénes Kőnig。
- 耶诺·埃格瓦里，英文名 Jenő Egerváry。

他们在图论和指派问题上的研究，为后来算法化求解最优匹配提供了重要理论基础。之后，美国数学家詹姆斯·芒克雷斯，英文名 James Munkres，在 1957 年进一步改进了该算法，使其在计算复杂度和工程实现上更加完善。因此，这一算法也常被称为 Kuhn-Munkres 算法。

从历史背景看，匈牙利算法主要服务于一类经典的组合优化问题：

- 一组任务要分配给一组工人。
- 每一种分配都有代价或收益。
- 目标是在一一对应约束下，使总代价最小或总收益最大。

这个问题后来被称为 assignment problem，即指派问题。它在生产调度、资源分配、路径规划、视觉跟踪等场景中都较为常见。

在多目标跟踪中，匈牙利算法的角色就是把 “轨迹” 和 “当前检测框” 看成两侧节点，把相似度或代价矩阵看成边权，然后求出全局最优的一一匹配关系。

2.10.11. 匈牙利算法到底解决什么问题

在目标跟踪里，如果只有一条轨迹和一个检测框，那么匹配很简单。但一旦场景中存在多条轨迹和多个检测框，问题就会迅速复杂起来。

例如，假设当前有 3 条旧轨迹和 3 个新检测框。此时系统不能只看 “哪两个最像就先配对”，因为局部最优未必能得到全局最优。匈牙利算法解决的就是这个问题：

- 它在满足一一对应约束的前提下，寻找整体代价最小或整体相似度最大的匹配方案。

这正是多目标跟踪里数据关联的核心。

2.10.12. 本案例中匈牙利算法是如何被调用的

在 tracking/deepsort.py 中，匈牙利算法的调用被封装在 `_linear_assignment` 中：

```
def _linear_assignment(self, cost_matrix):
    if cost_matrix.size == 0:
        return np.empty((0, 2), dtype=np.int64)

    row_ind, col_ind = linear_sum_assignment(-cost_matrix)
    return np.array(list(zip(row_ind, col_ind)), dtype=np.int64)
```

这里调用的是 scipy.optimize.linear_sum_assignment，它是匈牙利算法在科学计算库中的标准实现。

该函数默认求解最小化问题，而本案例构造的是 IOU 相似度矩阵，IOU 越大表示匹配越优。因此，代码中采用了一个典型的等价变换：

```
linear_sum_assignment(-cost_matrix)
```

即将 “最大化相似度” 转化为 “最小化负相似度”。

这里需要强调的一点是，同一个优化算法既可以用于最小代价问题，也可以通过简单变换用于最大收益问题。

2.10.13. 匈牙利算法在本案例中的完整上下文

在 `_associate` 中，数据关联的整体过程是：

1. 先计算每一条轨迹与每一个检测框之间的 IOU。

2. 再构造类别兼容矩阵，屏蔽明显不合理的类别组合。
3. 调用匈牙利算法得到全局最优分配。
4. 对主匹配结果应用 IOU 阈值过滤。
5. 对未匹配目标再执行一轮基于中心距离的补充匹配。

对应代码主线是：

```
iou_matrix = self._calculate_iou_matrix(detections)
class_mask = self._calculate_class_compatibility_matrix(detections)
assignment_scores = np.where(class_mask, iou_matrix, -1.0)
matched_indices = self._linear_assignment(assignment_scores)

for track_idx, detection_idx in matched_indices:
    if assignment_scores[track_idx, detection_idx] < self.iou_threshold:
        continue

fallback_matches = self._match_by_center_distance(...)
```

这段代码说明一个重要事实：匈牙利算法本身只负责 “给出最优一一分配”，但它并不直接理解哪些匹配在语义上是合理的。因此，本案例又叠加了两层约束：

- 类别必须兼容。
- IOU 必须高于阈值。
- 当 IOU 不足但中心运动关系合理时，还允许进入中心距离补充匹配。

由此可见，匈牙利算法提供的是全局分配框架，而不是完整的业务规则集合。

2.10.14. 匈牙利算法相对于贪心匹配的优势

一个常见问题在于，为什么不直接选择当前最大 IOU 的一对进行匹配。该贪心策略看似简单，但存在明显局限：

- 先做出的局部选择可能破坏后续整体最优。
- 当多个轨迹都与同一个检测框接近时，容易造成冲突。
- 在目标密集场景下，局部贪心更容易导致 ID 交换。

匈牙利算法的价值就在于它能从全局角度处理这类一一匹配问题。虽然本案例的匹配度量比较简单，只用了 IOU，但只要场景中存在多目标竞争，使用全局分配通常都比局部贪心更稳健。

2.10.15. 匈牙利算法在多目标跟踪中的局限

需要说明的是，匈牙利算法并不是万能的。它解决的是 “在给定代价矩阵时，如何求最优匹配”，但如果代价矩阵本身构造得不好，算法仍然可能得到错误匹配。

在本案例中，主匹配代价矩阵主要由 IOU 构成，因此它会受到以下问题影响：

- 快速运动目标与旧轨迹重叠变小，IOU 不足。
- 相邻目标位置太近，几何信息不够区分。
- 遮挡后重新出现时，仅靠 IOU 难以恢复原身份。

这也是为什么当前工程版本又额外加入了 “类别兼容矩阵” 和 “中心距离补充匹配”。匈牙利算法负责的是 “最优分配”，但 “什么样的代价矩阵才合理” 仍然是跟踪算法设计的关键。

2.10.16. 轨迹生命周期参数如何影响最终效果

本案例中的三组参数具有较强的实验分析价值：

- max_age：轨迹最长允许失配多少帧。
- min_hits：轨迹至少命中多少次后才输出。
- iou_threshold：检测与轨迹关联所需的最小 IOU。

这些参数分别控制三类问题：

- 轨迹是否容易过早消失。
- 新轨迹是否容易过快出现。
- 匹配是否保守还是激进。

通过调参可以直接观察以下现象：

- max_age 过小，目标短时遮挡后容易断轨。
- min_hits 过小，误检容易变成短暂轨迹。
- iou_threshold 过高，快速移动目标更容易失配。
- iou_threshold 过低，邻近目标更容易错配。

2.11. 可视化代码如何帮助理解算法结果

在 utils/postprocessing.py 中，draw_detections 与 draw_tracks 分别负责绘制检测结果和跟踪结果。尽管这部分并不直接构成算法主体，但其作用十分重要，因为可视化能够将抽象状态转化为可直接观察的现象。

其中，draw_tracks 中有两点尤其值得关注：

- 每条轨迹使用固定颜色，帮助观察 ID 是否稳定。
- trail 轨迹拖尾可以直观看出运动路径与预测连续性。

因此，在调节参数时，不仅能够观察检测框是否正常输出，还能够观察 ID 是否发生跳变，以及轨迹是否保持连续。

2.12. 如何评价这个案例方案

2.12.1. 这个方案的优点

作为昇腾 310B 开发教程中的案例，本方案有很强的适配性：

- 模型轻量，适合边缘侧实时实验。
- 检测和跟踪两个阶段边界清晰，便于分层讲解。
- 代码结构整洁，便于把入口、后端、解码、跟踪拆开讲。
- DeepSORT 采用简化实现，避免课程一开始就陷入 ReID 细节。
- 既能跑 CPU，也能跑 NPU，便于体现部署层面的思维。

2.12.2. 这个方案的不足

需要明确的是，本案例并不是为了追求最强跟踪精度：

- 检测前端仍是经典 SSD，精度不是当前最先进水平。
- 跟踪器没有使用外观特征，因此遮挡恢复能力有限。
- 数据关联主要依赖 IOU，对交叉目标和密集目标不够鲁棒。
- 卡尔曼状态建模较简化，更偏向实现可解释性而不是工业最优效果。

2.12.3. 作为入门案例的合理性

正因为它没有引入过多复杂部件，目标跟踪的基本问题反而更容易被清晰呈现：

- 检测结果是如何来的。
- 检测结果如何进入跟踪器。
- 为什么需要预测。
- 为什么需要全局匹配。
- 为什么轨迹管理会决定最终 ID 稳定性。

当这些基础概念建立起来后，再引入完整 DeepSORT、ByteTrack、OC-SORT 或更复杂的 ReID 模块，学习曲线会平缓很多。

2.13. 实验设计

为了把本案例组织成一章完整教程，可以安排以下实验任务：

2.13.1. 只运行检测链路

目标：理解 MobileNet-SSD 的输入输出与后处理。
可观察：

- 不同模型输入尺寸对速度和效果的影响。
- score_threshold 对检测框数量的影响。
- NMS 阈值对重复框抑制效果的影响。

推荐命令：

```
python scripts/detection_app.py --device npu --source 0
python scripts/detection_app.py --device npu --source 0 --backbone mobilenetv2_100
python scripts/detection_app.py --device npu --source 0 --backbone resnet18
```

实验提示：

- 对比 `ssd320_*` 和 `ssd300_*` 两类模型时，不仅要看 FPS，还要同时观察检测框数量和画面稳定性。
- 当 `score-threshold` 较低时，解码后的候选框数会明显增加，后处理负担也会增大。
- 当 `nms-threshold` 较大时，重复框更容易残留，后续 tracking 的输入质量也会受到影响。

2.13.2. 检测链路的性能拆分实验

目标：学会区分摄像头读取、预处理、推理、解码和绘制的性能瓶颈。
可观察：

- `Read`：摄像头或视频流读取时间。
- `Pre`：模型输入预处理时间。
- `Infer`：模型推理时间。
- `Decode`：后处理时间。
- `Draw`：检测框或轨迹绘制时间。

推荐命令：

```
python scripts/detection_app.py --device npu --source 0
python scripts/detection_app.py --device npu --source 0 --camera-mjpeg
python scripts/detection_app.py --device npu --source 0 --camera-profile 1280x720@60
```

实验提示：

- 如果 Read 明显高于 Pre，瓶颈更可能在摄像头采集或驱动路径，而不是模型预处理本身。
- 如果 Decode 偏高，可以优先降低 max-detections 或缩小关注类别范围。
- 如果启用 --camera-mjpeg 后 Read 下降，说明当前 USB 摄像头和驱动组合更适合 MJPEG 模式；如果反而上升，则应恢复默认模式。
- 如果请求的分辨率或帧率不是摄像头原生档位，驱动可能发生额外缩放或格式转换，Read 往往会明显上升。

2.13.3. 在检测基础上启用跟踪

目标：观察轨迹是如何从检测结果中产生的。
可观察：

- 同一目标在连续帧中是否保持稳定 ID。
- 当目标短时遮挡时，轨迹是否断裂。
- 当多个目标接近时，是否发生 ID 交换。

推荐命令：

```
python scripts/tracking_app.py --device npu --source 0
python scripts/tracking_app.py --device npu --source 0 --track-classes person
python scripts/tracking_app.py --device npu --source 0 --track-classes person,bus
```

实验提示：

- 使用 --track-classes 后，可以明显减少无关类别进入解码和跟踪流程，便于观察指定目标的轨迹稳定性。
- 对实时摄像头场景，建议优先用 person 这类高频类别做实验，更容易观察目标进出画面、遮挡和交叉等现象。

2.13.4. 类别过滤与解码开销实验

目标：理解 “只跟踪指定类别” 不仅影响画面内容，也影响后处理负载。
可观察：

- 指定类别前后，Decode 时间是否变化。
- 指定类别前后，检测框数量和轨迹数量是否更稳定。
- 不同类别组合对解码开销和跟踪干扰的影响。

推荐命令：

```
python scripts/tracking_app.py --device npu --source 0
python scripts/tracking_app.py --device npu --source 0 --track-classes person
python scripts/tracking_app.py --device npu --source 0 --track-classes person,bus
```

实验提示：

- 当只关心少数类别时，优先在解码阶段就做类别过滤，比“先解出全部类别再过滤”更符合实时场景。
- 如果场景中本来就几乎只有行人，那么 `--track-classes person` 的主要收益会体现在画面更干净和干扰更少，而不是极大的速度变化。

2.13.5. 调节 DeepSORT 参数

目标：理解生命周期管理与关联阈值。
可调节：

- track_max_age
- track_min_hits
- track_iou_threshold

推荐命令：

```
python scripts/tracking_app.py --device npu --source 0 --track-max-age 120
python scripts/tracking_app.py --device npu --source 0 --track-min-hits 3
python scripts/tracking_app.py --device npu --source 0 --track-iou-threshold 0.4
```

实验中应重点解释参数变化背后的原因，而不是只记录结果。
观察重点：

- `track-max-age` 过小，目标短时遮挡或漏检后容易断轨。
- `track-min-hits` 过小，误检更容易形成短暂轨迹。
- `track-iou-threshold` 过高，快速运动目标更容易匹配失败。

2.13.6. 调节中心距离与平滑参数

目标：理解补充匹配与轨迹平滑对稳定性的影响。
可调节：

- track-center-distance-threshold
- track-size-smoothing
- track-score-smoothing

推荐命令：

```
python scripts/tracking_app.py --device npu --source 0 --track-center-distance-threshold
    ↪ 2.0
python scripts/tracking_app.py --device npu --source 0 --track-size-smoothing 0.85 --track
    ↪ -score-smoothing 0.8
python scripts/tracking_app.py --device npu --source 0 --track-center-distance-threshold
    ↪ 1.4 --track-size-smoothing 0.6
```

观察重点：

- `track-center-distance-threshold` 增大后，快速运动和短时失配场景下的轨迹更容易续上，但如果过大，也可能提升误匹配概率。
- `track-size-smoothing` 增大后，框宽高更稳定，但目标尺度变化的响应会更慢。
- `track-score-smoothing` 增大后，显示分数更平滑，但对置信度突变的响应也更迟缓。

2.13.7. 组合实验建议

为了让实验更有层次，可以按下面顺序组织：

1. 先做 detection 基础实验，理解检测、解码和 NMS 的作用。
2. 再做性能拆分实验，判断瓶颈是在 `Read`、`Pre`、`Infer` 还是 `Decode`。
3. 然后启用 tracking，观察轨迹生成与 ID 稳定性。
4. 最后再做 `track-classes`、中心距离阈值和平滑参数的组合对比。

这样安排的好处是：

- 先把单帧检测问题讲清楚。
- 再把实时性能问题拆清楚。
- 最后把时序跟踪问题和参数调优联系起来。

2.14. 当前工程版本的优化说明

前文很多内容是按“教学上的基础实现思路”来讲解的，便于先把检测与跟踪主线讲清楚。当前仓库中的代码在此基础上又做了一轮面向实时性的工程优化，因此实际实现相比前文的基础版描述更进一步。阅读源码时，建议把这一节与前文结合起来看。

2.14.1. 检测链路中的工程优化

解码器不再对所有类别逐类扫描全部 prior

在基础版 SSD 后处理中，一个直观但开销较大的实现方式是：对每个类别都遍历全部 prior，然后分别做阈值筛选和 NMS。这样做虽然容易理解，但在 COCO 这类多类别场景里，后处理开销会明显增大。

当前仓库中的 ssdlite/decoder.py 已经改成了更适合实时场景的实现：

- 对每个 prior 先在候选前景类别中选出一个最佳类别。
- 只保留分数超过阈值的候选框。
- 再按类别分别做 NMS。

这样做的优点是：

- 大幅减少无效类别扫描。
- 降低候选框数量。
- 避免同一个 prior 在多个类别上重复进入 NMS。

这类优化尤其适合边缘设备，因为它直接减少了解码阶段的 CPU 开销。

支持在解码阶段按指定类别过滤

在当前工程版本中，tracking_app.py 增加了按类别跟踪的命令行参数：

```
python scripts/tracking_app.py --track-classes person
python scripts/tracking_app.py --track-classes person,bus
```

这里并不是 “先把所有检测框都解出来，再在跟踪前简单过滤”，而是把允许的类别一直传递到 decoder 内部，让解码阶段只关注这些目标类别。这样做的意义在于：

- 可以减少不关心类别的后处理开销。
- 可以降低多类别场景中的误检测干扰。
- 可以让跟踪器专注于任务相关目标，例如只跟踪行人或公交车。

default boxes 的生成做了向量化改写

基础版的 default boxes 生成通常会使用多层 Python 循环去枚举中心点和长宽比，这种写法直观，但初始化时效率较低。

当前实现已经把 DefaultBoxes 中的中心点网格生成和尺寸组合改写成 NumPy 向量化方式，同时拆分成更清晰的辅助函数：

- `_build_layer_sizes`：负责生成当前特征层的 anchor 尺寸集合。
- `_build_center_coordinates`：负责生成当前特征层的网格中心点。

这样既提升了代码可读性，也减少了初始化阶段的 Python 循环开销。

SSD300 与 SSDLite320 使用不同的 default boxes 设计

当前 decoder.py 中保留了两个明确区分的入口：

- `dboxes300_coco()`：对应原始 SSD300 论文风格的 default boxes 设计。
- `dboxes320_coco()`：对应 torchvision SSDLite / MobileNet-SSD 路径使用的 default boxes 设计。

这并不只是输入尺寸从 `300` 换成 `320`，而是两种不同模型家族在先验框配置上的区别。把这两个入口拆开写清楚，有助于读者理解 “不同 SSD 变体的 prior boxes 设计并不完全相同”。

IOU 与 NMS 的实现做了进一步优化

在当前工程版本中，decoder.py 里的 `calc_iou`、`_calc_iou_with_box` 和 `_nms` 已经做了两类优化：

- 用更直接的广播写法计算 IOU，提高可读性并减少中间变量。
- 在 NMS 中区分 “多框对单框” 的专用 IOU 计算路径，避免重复走更通用但更重的函数。
- 对已经按分数排序的候选框，不再重复做一次完整排序。

这说明同一个算法模块往往可以分成两种层次来理解：

- 原理层：为什么需要 IOU 和 NMS。
- 工程层：怎样把同样的逻辑写得更适合实时设备。

预处理与读帧时间被明确分开统计

在早期实验中，容易把摄像头读取时间和图像预处理时间混在一起看，从而误以为 “预处理最慢”。当前版本已经在 detection_app.py 和 tracking_app.py 中把以下时间分开统计：

- `Read`：摄像头或视频流读取时间。
- `Pre`：真正的模型输入预处理时间。
- `Infer`：模型推理时间。
- `Decode`：后处理时间。
- `Draw`：绘制与显示时间。

这对边缘设备非常重要，因为摄像头读取、视频解码、模型预处理和模型推理往往处于不同瓶颈路径上，必须分开分析。

摄像头实时输入增加了更适合实时场景的参数

当前 detection_app.py 和 tracking_app.py 都支持：

- `--camera-profile`
- `--camera-mjpeg`
- `--no-camera-mjpeg`

其中 `--camera-profile` 用一个参数统一表达摄像头原生采集档位，例如 `1280x720@60`、`1024x576@30` 或 `@30`。这种写法比同时暴露宽、高、FPS 三个独立参数更适合教学，因为读者更容易把 “摄像头采集模式” 理解成一个完整组合。

其中 `--camera-mjpeg` 适用于部分 USB 摄像头。如果摄像头和驱动支持 MJPEG，启用后有时可以降低实时读取延迟。但要注意，这类参数是否有效，与具体摄像头、驱动和板端 OpenCV 构建方式密切相关，因此需要实测。

此外，当前工程版本已经把 OpenCV 运行时辅助逻辑合并到 `utils/opencv_runtime.py` 中，这个文件现在同时负责：

- OpenCV Qt 字体目录修复
- 摄像头启动日志与阶段计时
- V4L2 优先打开实时摄像头
- 请求较小缓冲区以减轻实时场景中的采集积压

在一组 OrangePi AI Pro / Ascend 310B 的实测中，针对一只支持 `MJPG 1280x720@60` 的 USB 摄像头，启用 V4L2 优先后端和 `buffer=1` 请求后，tracking 的显示 FPS 从约 `20` 提升到了约 `26`。这个结果并不是固定值，但足以说明：在边缘端实时链路中，摄像头读帧路径本身就值得单独优化。

2.14.2. 进一步提高帧率的建议

如果在完成当前这轮采集优化后，FPS 仍然没有达到目标，可以继续按下面顺序做进一步优化：

- 优先选择摄像头原生支持的 `camera-profile`，避免非原生分辨率触发驱动缩放或格式转换。
- 如果目标是稳定跟踪帧率，而不是单纯追求更大画面，优先尝试 `1024x576@30`、`800x448@30` 这类更均衡的原生档位。
- 保留 `MJPEG` 用于高分辨率高帧率场景，但要注意 `Read` 中也包含解码时间，因此是否更快必须实测。
- 进一步把摄像头采集改成后台线程，只保留最新一帧，减少主线程推理和绘制阻塞导致的帧积压。

- 如果 Draw 时间偏高，可以降低显示窗口尺寸、减少拖尾绘制长度，或在无界面场景下使用 --no-display。
- 如果 Decode 时间偏高，可以减小 max-detections，或者优先使用 --track-classes 限制关注类别。
- 如果 Infer 仍是主要瓶颈，则应从模型侧继续优化，例如切换更轻量骨干、减小输入尺寸，或根据任务需要降低采集分辨率。

2.14.3. 跟踪链路中的工程优化

关联前先做类别兼容性约束

基础版多目标跟踪常见的写法是：先对所有轨迹和检测框计算 IOU，再在匹配完成后排除类别不兼容的结果。当前版本把类别兼容性提前到了关联阶段本身，也就是说：

- 不兼容的类别对在进入匈牙利匹配前就会被屏蔽。
- 这样可以减少无效匹配占用全局分配机会。

这种处理对多类别场景尤其重要，因为它可以有效降低 “行人轨迹误匹配到车辆框” 这类明显错误。

IOU 矩阵改成向量化计算

tracking/deepsort.py 中轨迹和检测之间的 IOU 矩阵，已经从双层 Python 循环改成了 NumPy 向量化实现。这样做有两个好处：

- 匹配阶段速度更快。
- 代码更容易与 “矩阵形式的数据关联” 这一概念对应起来。

增加基于中心距离的补充匹配

仅靠 IOU 做关联时，如果目标移动较快、检测框尺寸波动较大，或者短时遮挡后重新出现，轨迹容易断开。当前工程版本在 IOU 关联失败后，又增加了一轮基于中心距离的补充匹配：

- 先计算轨迹中心和检测框中心的归一化距离。
- 再结合类别约束，做一次补充关联。

这样可以提高短时丢检和快速移动场景下的轨迹连续性。

轨迹框尺寸与分数做了平滑

在 Track.update 中，当前版本不再每一帧都直接用新检测框完全替换轨迹框，而是对宽高和分数做指数平滑：

- `size_smoothing`：控制框宽高变化的平滑程度。
- `score_smoothing`：控制置信度显示的平滑程度。

这样做的效果是：

- 画面中的框抖动更小。
- 轨迹框大小变化更连续。
- 显示分数不容易大起大落。

轨迹类别不再只看最近一帧

当前 Track 类中维护了 `class_scores`，会对每个轨迹在历史匹配过程中累计类别分数，再选择当前最可信的类别。这比简单采用 “最近一帧的 class_id” 更稳定，尤其适合检测框偶尔分类波动的场景。

2.14.4. 卡尔曼滤波器中的工程优化

初始协方差更加符合目标跟踪场景

在基础讲解中，常把卡尔曼滤波器写成最简单的形式，例如：

- `P = I`
- `Q = 0.05 * I`
- `R = 0.5 * I`

这很适合教学起点，但在实际跟踪里往往过于理想化。当前工程版本已经改成更合理的初始化方式：

- 位置初始不确定性较小。
- 速度初始不确定性较大。

这更符合新轨迹刚创建时的实际情况：我们大致知道目标在哪里，但一开始并不知道它运动得有多快。

过程噪声和观测噪声被显式区分

当前 KalmanFilter 中把位置过程噪声、速度过程噪声和观测噪声分开定义，而不是用单个常量矩阵草草代替。这样做的好处是：

- 参数意义更清楚。
- 后续调参更方便。
- 更能贴合 “位置观测可信，但速度需要逐步估计” 的目标跟踪特点。

更新阶段使用更稳定的数值形式

当前实现中：

- 卡尔曼增益计算使用 `np.linalg.solve`，而不是直接显式求逆。
- 协方差更新使用 Joseph form，而不是简单的 `P = P - KHP`。

这种改法在数学上更稳定，尤其适合连续多帧递推的视觉跟踪任务，因为协方差矩阵的对称性和正定性更容易保持。

2.14.5. tracking_app.py 当前支持的实用参数

当前 tracking_app.py 在基础版参数之外，还支持以下跟踪专有参数：

- `--track-max-age`
- `--track-min-hits`
- `--track-iou-threshold`
- `--track-center-distance-threshold`
- `--track-size-smoothing`
- `--track-score-smoothing`
- `--track-classes`

这些参数分别对应：

- 轨迹生命周期控制。
- IOU 关联阈值。
- 中心距离补充匹配阈值。
- 轨迹框和分数的平滑强度。
- 指定只跟踪哪些类别。

例如，只跟踪行人时，可以使用：

```
python scripts/tracking_app.py --device npu --source 0 --track-classes person
```

如果需要同时跟踪行人和公交车，可以使用：

```
python scripts/tracking_app.py --device npu --source 0 --track-classes person,bus
```

如果希望在快速运动场景中增强补充关联，可以适当提高中心距离阈值，例如：

```
python scripts/tracking_app.py --device npu --source 0 --track-center-distance-threshold
    ↪ 2.0
```

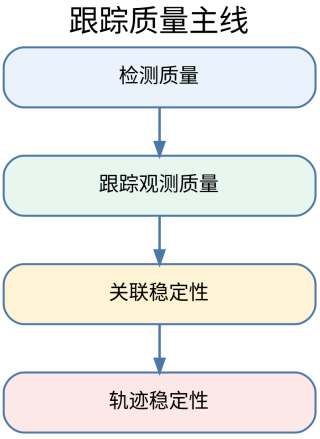


图 2.2. 主线图

2.14.6. 如何理解“教材版”和“工程版”的关系

本案例的一个教学特点是：前文先用更直观的形式讲清算法主线，再在当前工程代码中逐步引入更适合实时边缘设备的优化实现。两者并不是相互冲突，而是两个层次：

- 教材版：帮助读者看懂为什么需要这些模块。
- 工程版：帮助读者理解怎样让这些模块在真实设备上跑得更稳、更快。

因此，建议阅读顺序是：

1. 先按前文把 MobileNet-SSD、NMS、卡尔曼滤波和数据关联的原理主线理清。
2. 再结合当前仓库代码，对照本节总结理解这些优化为什么能够改善实时效果。

2.15. 章节总结

本案例展示了一条较为典型的目标跟踪实现路线：

- 用 MobileNet 作为轻量骨干网络，降低边缘侧计算成本。
- 用 SSD 作为一阶段检测器，直接输出类别与边界框。
- 用统一检测后端把 CPU 与 Ascend NPU 推理过程封装起来。
- 用简化版 DeepSORT 完成轨迹预测、数据关联与轨迹维护。

从知识结构上看，本章真正要让读者掌握的，不只是某个脚本的使用方法，而是下面这条主线：

理解这条主线之后，读者就能明白：目标跟踪不是一个独立黑盒，而是一套由检测、预测、匹配和管理共同组成的时序识别系统。

对于昇腾 310B 教程而言，这个案例的价值正在于此。它既保留了真实部署场景中需要考虑的实时性与工程组织问题，又把目标跟踪最核心的算法逻辑清楚地呈现出来，适合作为后续学习更复杂视觉任务的基础章节。

3. 案例 3：智能电子琴

数字乐器的音色生成既要描述音高、力度、起音和衰减等音乐控制信息，也要在有限计算资源下连续产生具有稳定听感的音频波形。传统采样合成依赖预先录制的声音素材，而直接生成波形的神经网络通常需要在音频采样率上执行高密度计算。可微分数字信号处理（Differentiable Digital Signal Processing，DDSP）将谐波振荡器、滤波噪声和混响等声学结构引入神经网络训练，使模型能够预测具有物理含义的控制量，再由数字信号处理模块完成波形合成。DDSP 论文给出了这一方法的系统论述。

将 DDSP 从研究模型迁移到边缘计算设备，并不只是完成模型格式转换。实时演奏需要维持毫秒级控制周期和连续循环状态，MIDI 文件渲染需要处理完整乐曲的时序上下文，麦克风音色转换还涉及音高估计、噪声门、采样率转换和双工音频路由。这些任务具有不同的时间尺度和资源需求，能够集中体现神经网络推理、数字信号处理、实时调度与人机交互之间的协同关系。

基于上述背景，本案例以 Ascend 310B 为计算平台，选取复音钢琴演奏、MIDI 文件音色渲染和单音音色转换三类任务，构建一套完整的 DDSP 实验系统。该案例既可用于理解 Piano-DDSP、MIDI-DDSP 与 DDSP-VST 的网络结构和控制机制，也可用于研究 NPU 推理与 CPU 音频合成的任务划分，以及模型在交互式数字乐器、自动音乐制作和边缘音频处理中的应用方式。

配套代码位于 `samples/case3`。

3.1. 案例目标与学习成果

3.1.1. 系统功能

该 Web 工作站支持以下三类处理流程：

工作流	输入	神经网络	输出	适用场景
实时演奏	触摸屏钢琴或标准 MIDI 键盘	Piano-DDSP OM	实时扬声器声音	触控输入与具有力度、踏板和弯音控制的实体演奏
MIDI-DDSP	`.mid` 或 `.midi` 文件	Expression 与 Synthesis OM 组件	WAV、开发板播放或浏览器播放	离线多声部音色渲染

工作流	输入	神经网络	输出	适用场景
DDSP-VST	摄像头或独立麦克风的实体音频输入	Feature OM 与 Control OM	实时音色转换	把单音哼唱或乐器声转换为 11 种音色

设备页不参与音乐生成，它负责展示开发板、NPU、模型、音频、MIDI、蓝牙和 Python 环境状态，并提供独占的扬声器与麦克风测试。

3.1.2. 学习成果与边界

读者将理解并复现以下内容：

1. DDSP 以可解释控制量连接神经网络与音频合成模块的基本原理。
2. Piano-DDSP、MIDI-DDSP 和 DDSP-VST 三套网络在输入、状态、时序和用途上的区别。
3. 自训练模型导出与上游 TFLite 模型迁移两类来源，以及 ONNX、OM 和板端推理之间的关系。
4. FastAPI、WebSocket、音频线程、MIDI 状态机、资源互斥和文件产物如何协作。
5. React 界面如何在触摸屏、桌面和移动视口中保持良好的可读性、可操作性与渲染效率。

本案例关注声音生成与实时音频处理，摄像头仅作为麦克风载体，不涉及图像分析。DDSP-VST Effect 的研究对象限定为基频相对稳定的单音输入，复音分离与复杂声场建模不在讨论范围内。运行指标用于描述计算链路和资源状态；音色质量、声压与声学特性仍需通过独立的听音和测量实验评价。

3.2. 器件与材料清单

3.2.1. 最小配置

最小配置用于完成触控演奏、浏览器操作和基本声音输出。

器件	数量	接口	用途	替代要求
Ascend 310B 开发板	1	以太网、HDMI、USB、音频	运行 FastAPI、PyACL、OM 推理和 CPU DSP	必须具备可用 CANN/PyACL 和对应 SoC 的 OM 支持

器件	数量	接口	用途	替代要求
原配电源与启动存储	1 套	开发板专用	启动和稳定供电	电压、电流和启动介质必须符合板卡说明
HDMI 触摸显示器	1	HDMI、USB 触控	本地显示与触摸演奏	应支持浏览器全屏显示和稳定的多点触控
HDMI 线与 USB 触控线	各 1	HDMI、USB	视频与触控回传	线材需支持屏幕分辨率和持续供电
音频输出	1	USB、板载接口或 Bluetooth A2DP	播放合成音频	支持有线音箱、蓝牙音箱和蓝牙耳机；实时演奏验收推荐使用有线输出
局域网与开发电脑	1 套	以太网或 Wi-Fi	构建、部署和浏览器访问	开发电脑与开发板需位于可信局域网

本案例支持通过 Bluetooth A2DP 连接蓝牙音箱和蓝牙耳机。蓝牙链路中的音频编码、无线传输和播放缓冲通常会增加端到端延迟，因此实时演奏时可能出现较明显的声音滞后，影响演奏反馈。该部分延迟来自蓝牙音频链路，不应归因于 NPU 推理性能；评价模型推理和实时合成能力时，应优先采用有线输出，或将蓝牙链路延迟作为独立分量测量。

3.2.2. 完整功能配置

以下型号构成本案例的验证配置，但不代表唯一可用品牌。

器件	数量	验证型号	接口	在实验中的作用
Ascend 310B 开发板	1	Orange Pi AIpro	USB、HDMI、网络	OM 推理、音频、后端和触摸屏主机
触摸显示器	1	HDMI 触摸显示器	HDMI + USB	四工作区本地操作
USB 音箱	1	EDIFIER M16 Pro	USB Audio	Piano-DDSP、MIDI-DDSP 和 DDSP-VST 的默认输出
摄像头麦克风	1	UGREEN Camera 1080P	USB Audio Capture	DDSP-VST 输入与麦克风测试；图像数据不参与本案例
MIDI 键盘	1	MIDIPLUS TINY, 32 键	USB MIDI	物理按键、力度和控制器输入

器件	数量	验证型号	接口	在实验中的作用
路由器	1	普通局域网路由器	以太网或 Wi-Fi	分配 IPv4 地址并连接开发电脑与开发板
网线	1	Cat5e 或更高	RJ45	推荐的稳定部署和调试链路
开发电脑	1	Windows 或 Linux	USB、网络	Python 测试、Node 前端构建和模型发布下载
USB 线材与供电线	若干	与设备匹配	USB-A、USB-C 等	音箱、摄像头、MIDI、触控和供电连接

3.2.3. 可选扩展

扩展	用途	使用条件
蓝牙 A2DP 音箱	无线播放	先由开发板系统完成配对，并在 PulseAudio 中出现可用 sink；蓝牙延迟不作为低延迟验收路径
USB DAC 或 USB 声卡	提供更稳定的线路输出	必须能被开发板枚举并显示为可用输出
独立单音麦克风	改善 DDSP-VST 输入信噪比	必须是实体 capture source，不能使用 PulseAudio monitor
延音踏板	控制实体 MIDI 的 CC64	MIDI 控制器需要上报标准控制变化消息

3.3. 系统总体架构与完整流程

3.3.1. 三条业务链

三条链路共享 NPU 和音频设备，但采用不同的时间模型。Piano-DDSP 每 `4 ms` 更新一次控制量；MIDI-DDSP 利用完整乐曲上下文离线生成 WAV；DDSP-VST 每 `20 ms` 处理一帧音高和响度控制。因此，三条链路需要分别设计状态管理、调度和缓冲策略。

3.3.2. 硬件连接

应按设备标识选择输入输出。程序不会在摄像头或音箱断开后静默切换到其他设备，也不会把 PulseAudio 输出监听源用作麦克风输入。显式路由可以避免意外回放系统声音、产生反馈或将测试信号发送到错误的音频设备。

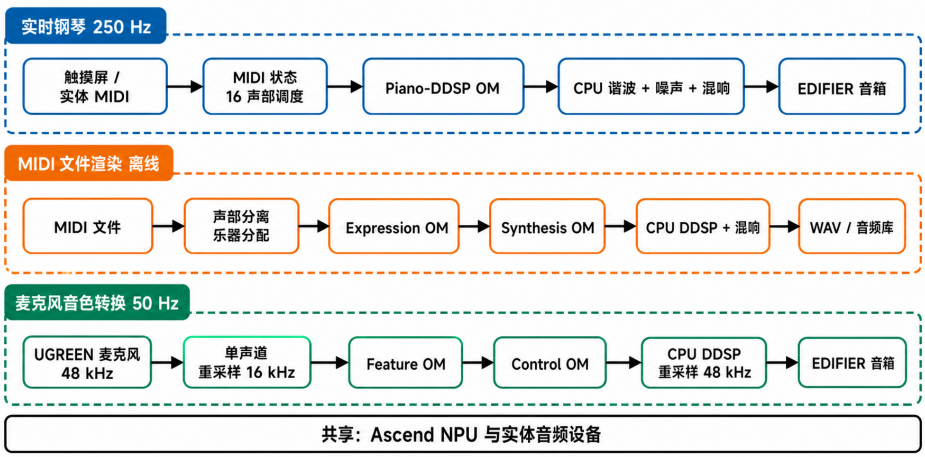


图 3.1. Case 3 的实时钢琴、MIDI 文件渲染和麦克风音色转换三条业务链

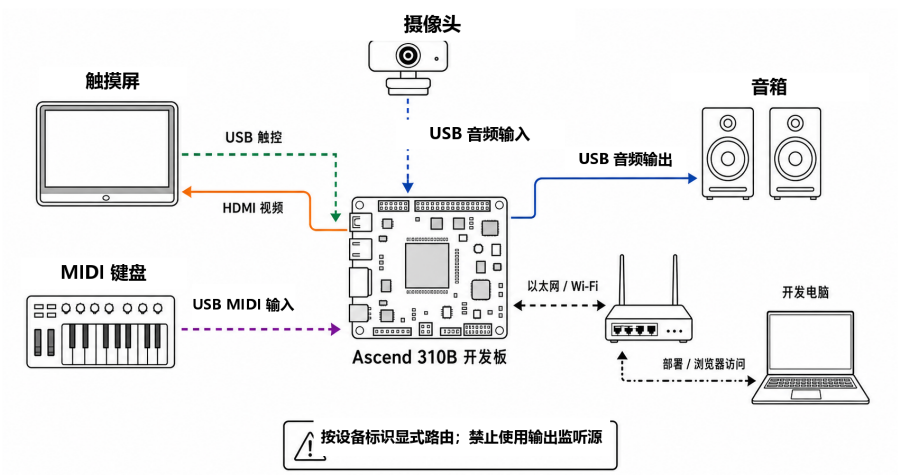


图 3.2. Ascend 310B 开发板与触摸屏、MIDI 键盘、USB 麦克风、音箱和开发电脑的硬件连接

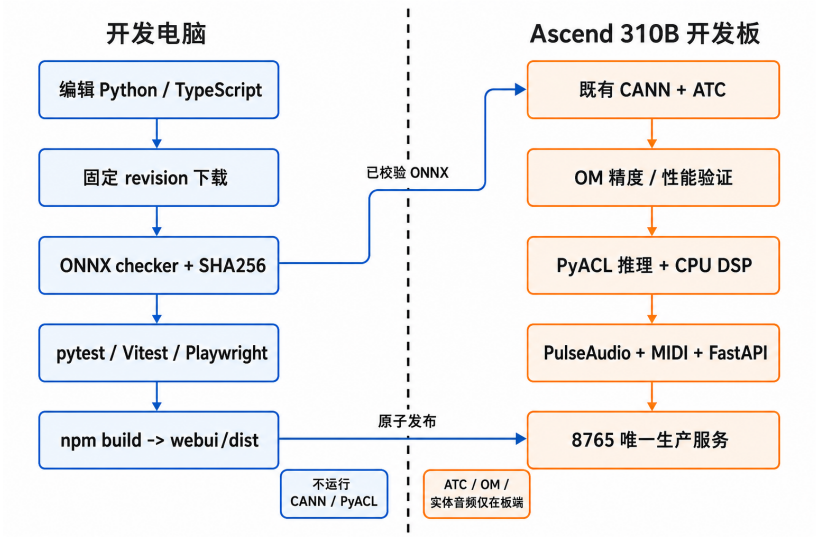


图 3.3. 开发电脑与 Ascend 310B 开发板的职责边界

3.3.3. 开发电脑与开发板的职责边界

操作	开发电脑	Ascend 310B
编辑源码、运行普通 Python 单元测试	是	否
`npm ci`、`npm run test`、`npm run build`	是	否
下载并校验发布模型	是	可接收已校验资产
ATC、PyACL、OM 推理、`npu-smi`	否	是
PulseAudio 实体路由和真实听音	否	是

3.4. DDSP 原理

3.4.1. 从采样合成到可微分 DSP

传统采样合成器把大量录音映射到音高和力度区域，音色逼真但资产大，跨音高和连续控制依赖插值。端到端波形神经网络可以直接预测音频，却需要在音频采样率上输出大量数据，实时部署的计算量、状态和可解释性都更困难。

DDSP 将振荡器、滤波器、包络和混响等数字信号处理模块写成可微形式，使这些模块能够参与梯度优化；部署阶段则由神经网络预测低维控制量，再由信号处理模块合成波形。DDSP 论文提出了这一建模框架，官方代码给出了相应实现。

3.4.2. 基频、谐波和相位

设采样率为 f_s，第 n 个采样的基频为 $f_0[n]$。第 k 个谐波的相位按下式累积：

$$\phi_k[n+1] = \phi_k[n] + \frac{2\pi k f_0[n]}{f_s}.$$

网络预测总幅度 $A[n]$ 和未归一化谐波参数 $z_k[n]$。使用非负映射和归一化得到谐波比例：

$$c_k[n] = \frac{\mathrm{softplus}(z_k[n])}{\sum_{j=1}^{K} \mathrm{softplus}(z_j[n]) + \varepsilon}.$$

谐波信号为：

$$x_h[n] = A[n] \sum_{k=1}^{K} c_k[n] \sin(\phi_k[n]).$$

超过奈奎斯特频率的谐波必须被抑制，即 $k f_0[n] < f_s/2$。Piano-DDSP 还预测 `inharmonicity`，用于表达钢琴弦的部分音偏离整数倍基频的现象。

3.4.3. 滤波噪声、包络、重采样和混响

噪声支路把白噪声 $w[n]$ 变换到频域，并用网络预测的频带幅度 $H_b[t]$ 构成时变滤波器：

$$x_n[n] = \mathrm{ISTFT}\left(\mathrm{STFT}(w[n]) \odot H[t, f]\right).$$

最终干声是谐波和噪声之和：

$$x_{dry}[n] = g_h x_h[n] + g_n x_n[n].$$

控制网络工作在 `50 Hz` 或 `250 Hz`，远低于 `16 kHz` 音频采样率，因此幅度、基频和频带系数需要插值到音频时间轴。混响可以是学习到的冲激响应 $r[n]$，也可以是 FreeVerb 风格反馈延迟网络：

$$x_{wet}[n] = x_{dry}[n] * r[n], \qquad y[n] = (1-m)x_{dry}[n] + m x_{wet}[n].$$

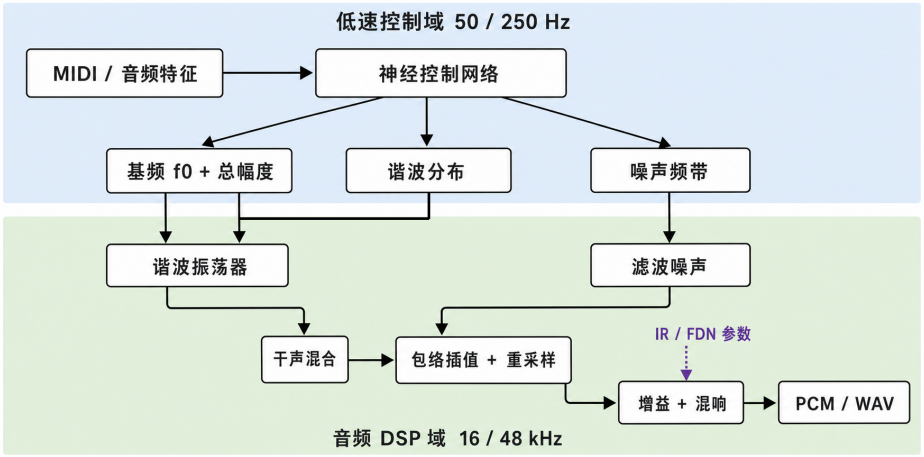


图 3.4. DDSP 从低速神经控制量到音频 DSP 输出的信号流程

3.4.4. 参数如何改变听感

控制量	物理或听觉含义	常见异常
`f0`	基频，决定音高和滑音轨迹	噪声被误检为基频时会出现无意义的低音或抖动
幅度或响度	控制音符包络和整体强弱	过大会削波，过小会让音色模型看不到有效输入
谐波分布	决定明亮度、共鸣和乐器主体音色	高频谐波过多会尖锐，归一化错误会改变总能量
噪声频带	表达吹气、摩擦、击弦和瞬态	过高会产生嘶声，全部关闭会让音色不自然
混响	表达空间和尾音	全湿声会掩盖起音，实时链路中也会增加主观拖尾
噪声门	阻止环境底噪触发模型	阈值过低会持续触发，过高会吞掉轻声起音

3.5. 三套神经网络架构

3.5.1. Piano-DDSP

模型来源

本案例所用的 Piano-DDSP 不是对上游 TensorFlow 权重的格式转换，而是作者在独立的 piano-ddsp-pytorch 训练项目中完成的 PyTorch 实现与训练。网络设计以 DDSP-Piano 的复音钢琴建模方法为基础，发布权重均由该项目在 MAESTRO 数据集上的训练过程产生，并由训练所得 checkpoint 导出为 ONNX。因而，Piano-DDSP 的研究链路是“结构复现与修改、PyTorch 训练、ONNX 导出、OM 转换”，与本章另外两类模型的迁移来源有本质区别。

网络实现

模型以 MIDI 音高、力度、踏板状态和钢琴录音域索引为条件，在 16 kHz 采样率下以 250 Hz 控制率工作，每个控制帧对应 64 个音频采样，并行描述最多 16 个声部。网络由全局上下文分支和共享权重的单声部分支构成。全局分支综合所有活动音符、踏板和钢琴嵌入，通过循环网络形成跨声部的上下文表示；单声部分支则为每个声部分别结合当前音符、延音阶段与全局上下文，预测幅度、谐波分布和噪声频带。音高支路进一步估计基频、弦间微小失谐与非谐性，从而表达钢琴弦的刚度及同音弦拍频。

这一分层结构兼顾了复音耦合与逐声部独立性：全局上下文反映踏板和同时发声音符对整体音色的影响，单声部循环状态保持各音符起音、持续和释放阶段的连续性。神经网络只预测声学控制量，谐波振荡、滤波噪声、相位累积与混响卷积由宿主端 DSP 完成，避免在高采样率上直接生成整段波形。

训练策略

MAESTRO 提供时间对齐的钢琴音频与高精度 MIDI。训练时，MIDI 音符、击键力度和踏板控制构成条件序列，配对音频构成声学监督；录音年份映射为钢琴录音域索引，使模型能够学习不同乐器与录音环境之间的系统差异。训练、验证和测试按照数据集给定的乐曲划分组织，避免同一作品的不同演奏跨越数据子集而造成信息泄漏。

训练过程将音色控制与音高细节分阶段优化。首先关闭失谐支路，学习上下文网络、单声部控制网络、钢琴嵌入和混响，使模型建立稳定的幅度、谐波与噪声表征；随后固定主要音色通路，训练失谐、非谐性及其录音域嵌入；最后在音高支路生效的条件下微调控制网络，使频率结构与音色包络共同收敛。分阶段训练减少了幅度谱重建与细微音高偏差之间的相互干扰，也使不同结构方案能够在一致的数据和控制合同下比较。

训练系统保留干声与混响声两条评价路径。干声约束主要考察振荡器控制和瞬态是否合理，混响声约束则用于评价最终听感及空间响应。对全湿声候选结构，还加入能量、

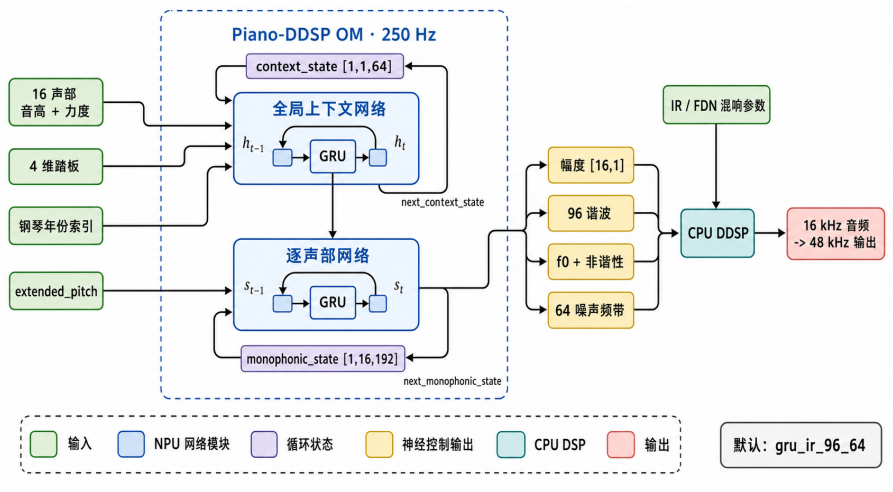


图 3.5. Piano-DDSP 的上下文网络、逐声部网络、循环状态和 CPU 合成边界

起音和力度相关的感知约束，以抑制仅凭频谱损失获得低数值误差、却削弱击弦瞬态的情况。模型选择因此不能只依据单一损失值，还需要结合未见曲目的渲染结果和听音比较。

面向实时部署的结构修改

为适应流式推理，训练实现将原本隐含在序列计算中的上下文状态和逐声部状态改为显式输入输出，使每次推理只处理一个控制帧，同时保持帧间递归关系。音符释放状态由宿主端维护，谐波相位、噪声重叠缓存和混响历史也位于模型之外；这种边界既缩小了 ONNX 计算图，也使状态复位、声部回收和确定性重放具有明确语义。面向 Ascend 转换时，GRU 进一步等价展开为矩阵乘、门控激活和逐元素运算，以规避循环算子的精度与算子支持限制。展开只改变计算图的表示方式，不改变训练权重和递推方程。

在共同的输入、状态和输出合同下，本案例训练并比较了四类结构：

结构方案	主要修改	研究目的
GRU 与学习型冲激响应	复现基准上下文网络和单声部 GRU，预测 96 个谐波与 64 个噪声频带	建立与 DDSP-Piano 结构相近的因果基线
FiLM 与反馈延迟网络	用钢琴嵌入调制全局特征，引入更深的单声部网络、联合非谐性模型和 128/96 维控制	考察条件调制、模型容量和参数化混响的作用

结构方案	主要修改	研究目的
GRU 全湿声校准	保留基准控制网络，强化混响声、能量、起音和力度约束	研究感知目标对击弦瞬态和空间感的影响
FiLM 全湿声校准	结合 FiLM、深层单声部网络、联合非谐性和学习型冲激响应	比较结构扩展与感知校准的联合效果

这些结构名称表示建模假设，而不是预设的音质排序。不同方案在网络容量、谐波分辨率和混响表示上各有侧重，最终选择需要同时考虑闭环数值误差、实时预算和主观听音结果。

ONNX 导出合同

训练后的 PyTorch 控制网络以 FP32、固定批量和单控制帧形式导出为 ONNX。下表以 GRU 与学习型冲激响应结构为例；FiLM 与反馈延迟网络结构保持相同的输入和循环状态语义，但将谐波、噪声及混响输出分别扩展为相应的控制维度。

方向	张量	形状	含义
输入	`conditioning`	`[1,1,16,2]`	16 声部的音高与力度条件
输入	`pedal`	`[1,1,4]`	踏板与连续控制状态
输入	`piano_model`	`[1]`	MAESTRO 钢琴年份索引
输入	`extended_pitch`	`[1,1,16,1]`	扩展音高条件
输入	`context_state`	`[1,1,64]`	全局循环状态
输入	`monophonic_state`	`[1,16,192]`	每声部循环状态
输出	`amplitudes`	`[1,1,16,1]`	声部总幅度
输出	`harmonic_distribution`	`[1,1,16,96]`	96 个谐波比例
输出	`inharmonicity、f0_hz`	`[1,1,16,1]`	非谐性和基频
输出	`noise_magnitudes`	`[1,1,16,64]`	64 个噪声频带
输出	`reverb_ir`	`[1,24000]`	学习到的混响冲激响应
输出	`next_context_state`	`[1,1,64]`	下一帧全局状态
输出	`next_monophonic_state`	`[1,16,192]`	下一帧声部状态

导出验证不仅比较单帧输出，还将上一帧状态连续反馈给下一帧，逐项对照 PyTorch 与 ONNX Runtime 的控制量和循环状态。这样可以发现单帧误差较小、但在长序列中逐步累积的偏差。通过该检验的 ONNX 才进入后续 OM 转换；此处的数值一致性仍不等价于音质评价或端到端音频延迟。

3.5.2. MIDI-DDSP

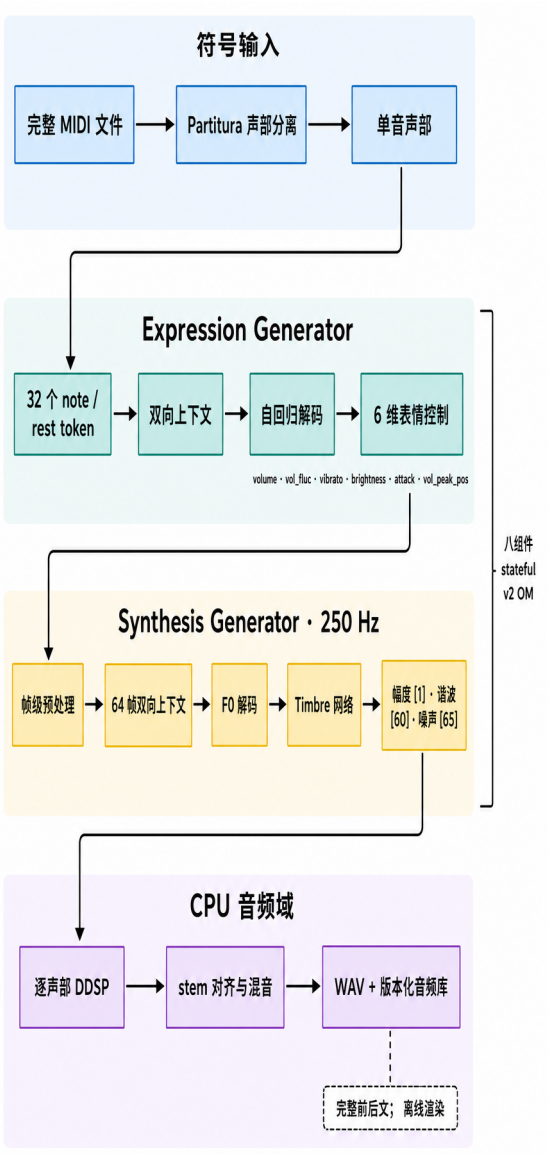


图 3.6. MIDI-DDSP 从符号输入、Expression Generator、Synthesis Generator 到 CPU 音频域的层级结构

MIDI-DDSP 为已知完整乐曲的离线层级模型，不是物理 MIDI 键盘的低延迟模型。Expression Generator 先从最长 32 个 note/rest token 的上下文预测 6 维表情控制：volume、

vol_fluc、vibrato、brightness、attack 和 vol_peak_pos。Synthesis Generator 再用 64 帧窗口、250 Hz 控制率生成音频控制量。MIDI-DDSP 论文和官方仓库解释了这种符号控制与音色合成的层级结构。

stateful v2 把原始图拆成八个可独立转换和批处理的组件：

组件	关键输入输出
expression_context_forward	前向 token 上下文与状态
expression_context_backward	反向 token 上下文与状态
expression_decode	上下文、上一控制与下一自回归状态
synthesis_precondition	帧级音高、表情和乐器条件
synthesis_context_forward	前向帧上下文与状态
synthesis_context_backward	反向帧上下文与状态
synthesis_f0_decode	基频控制与解码状态
synthesis_timbre	amplitude[1]、harmonics[60]、noise_amps[65]

模型为单声部建模。程序先把 MIDI 分成单音声部，再按静态 batch 1/2/4/8 选择最小可容纳 OM，生成每个 stem，最后对齐并混音。双向上下文需要看到完整前后文，因此不能把这条链路直接放到按键后的 4 ms 实时路径中。

3.5.3. DDSP-VST Feature 与 Control

DDSP-VST Effect 参考上游 DDSP-VST 实现。其计算过程分为 Feature 与 Control 两级：Feature 模型从滑动音频窗口估计基频和响度表征，Control 模型结合循环状态生成谐波、噪声与幅度控制量。两级模型的分离使音频特征提取与乐器音色建模能够采用不同的状态更新方式。

模型	输入	输出	时率与作用
Feature OM	float32[1024] 音频	f0_scaled[1]、pw_scaled[1]、f0_hz[1]、pw_db[1]	16 kHz，步长 320，即 50 Hz
Control OM	state[512]、f0_scaled[1]、pw_scaled[1]	amplitude[1]、harmonics[60]、noise_amps[65]、state_out[512]	每 20 ms 更新音色控制与 GRU 状态

Control 模型覆盖巴松、单簧管、长笛、口风琴、萨克斯、西塔琴、长号、小号、大号、小提琴和人声音色，共 11 类。不同音色共享相同的输入与状态合同，但通过各自的模型参数形成不同的谐波和噪声分布。

3.5.4. 三套网络对比

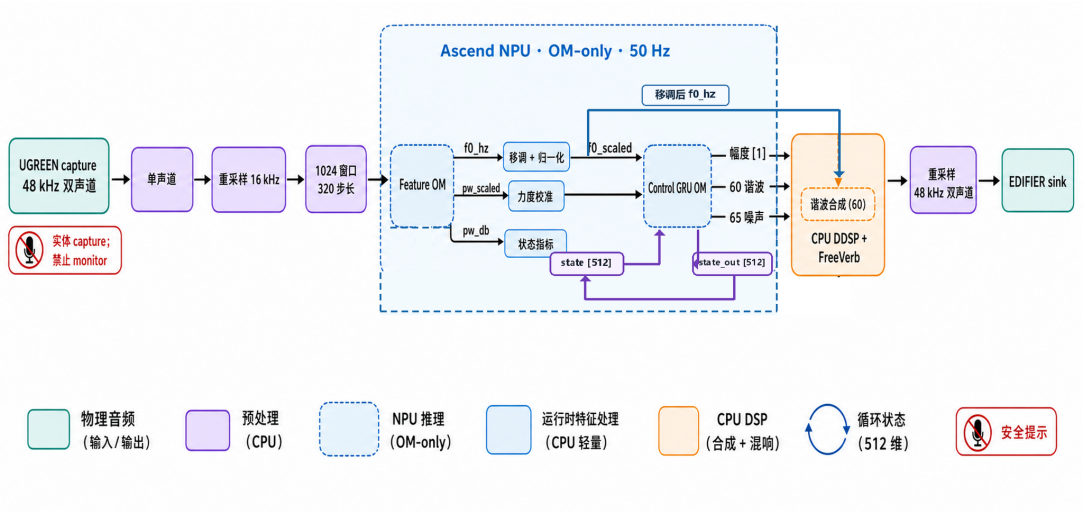


图 3.7. DDSP-VST Effect 的实体音频输入、Feature OM、Control GRU 状态回路和 CPU 音频输出

特性	Piano-DDSP	MIDI-DDSP	DDSP-VST Effect
输入	MIDI 帧、踏板、钢琴索引	完整 MIDI 文件与乐器分配	实体麦克风音频
主要用途	实时复音钢琴	离线高质量 MIDI 渲染	实时单音音色转换
复音能力	固定 16 声部	分离后逐单音声部渲染并混音	单音输入
控制率	250 Hz	250 Hz	50 Hz
时序状态	显式全局与逐声部状态	八组件双向和自回归状态	512 维 Control GRU 状态
前后文	只依赖过去和当前帧	使用完整乐曲的双向上下文	滑动音频窗口与过去状态
NPU 输出	96 谐波、64 噪声等控制	60 谐波、65 噪声等控制	Feature 特征和 60/65 控制
DSP 边界	CPU 合成、重采样和 IR/FDN	CPU 合成、混响、stem 混音和 WAV	CPU 合成、重采样、增益和 FreeVerb

3.6. 模型转换与 Ascend 部署

3.6.1. 两类模型来源

三套网络进入 Ascend 部署链之前具有两种不同来源，不能将其笼统表述为同一种“模型下载” 过程。

模型族	原始来源	ONNX 的形成方式	本案例中的作用
Piano-DDSP	作者基于 DDSP-Piano 结构完成的 PyTorch 训练模型	从本地训练 checkpoint 导出因果、显式状态的 ONNX	实时复音钢琴控制
MIDI-DDSP	Google MIDI-DDSP 项目的 TFLite 模型	按层级生成过程重构静态 ONNX 组件，并显式化双向与自回归状态	完整 MIDI 的表情与音色渲染
DDSP-VST	Google DDSP-VST 项目的 TFLite 模型	将 Feature 与 Control 分别转换为流式 ONNX，并显式化 Control 的循环状态	实时单音特征提取与音色转换

Piano-DDSP 的训练代码与模型修改见 piano-ddsp-pytorch，导出的 ONNX 模型发布在 Piano-DDSP 模型仓库。MIDI-DDSP 与 DDSP-VST 的网络语义分别以 Google MIDI-DDSP 和 Google DDSP-VST 为依据。固定模型来源的意义在于使后续差异可以归因于图变换、精度配置或硬件执行，而不是上游权重的变化。

3.6.2. 从 TFLite 到 ONNX 的图语义重构

TFLite 到 ONNX 的迁移不能只追求算子逐项替换，还必须保持原模型的时序语义。DDSP-VST 的 Control 模型原本包含循环控制流；转换时将其改写为单步计算图，把上一帧 GRU 状态作为输入、下一帧状态作为输出，并将幅度非线性、谐波归一化和奈奎斯特频率屏蔽纳入同一图中。Feature 模型则保持滑动音频窗口与固定步长，使基频和响度估计与上游实时处理的时间尺度一致。验证时需要连续输入多帧信号，因为循环状态的误差不会在孤立单帧测试中充分显现。

MIDI-DDSP 的转换更为复杂。Expression Generator 与 Synthesis Generator 同时包含双向上下文和自回归解码，若简单按窗口截断，会破坏完整乐曲的前后文。本案例将其分解为上下文编码、条件预处理、自回归解码、基频解码和音色控制等八类组件，显式传递各级状态，并通过有效帧掩码保持填充区域不参与统计。该设计既满足 ONNX 静态形状约束，又保留完整 MIDI 序列上的双向语义和逐帧递推关系。

具体转换遵循以下方法：

1. 以 TFLite 模型和一组连续参考输入为起点，记录初始状态、逐帧输出及状态演化，而非只保存孤立样本。

2. 识别 TFLite 子图中的常量权重、张量布局、门控顺序和后处理操作，建立与上游模型一致的逻辑张量合同。
3. 将动态循环和张量列表改写为固定形状的单步或分段 ONNX 图，把所有跨调用信息显式表示为状态输入输出。
4. 对广播、归一化、填充掩码和非线性后处理逐项对齐，避免框架默认语义差异改变声学控制量。
5. 使用相同输入连续运行 TFLite 与 ONNX，比较控制量、离散采样结果和循环状态；通过闭环对照后再执行 ONNX 结构检查。

两类转换均以 TFLite 输出为参考，比较连续控制量、离散采样结果和状态演化；只有当 ONNX 在相同输入下保持规定误差范围内的一致性，才能进入硬件编译阶段。这种做法把 “模型可以被转换” 与 “转换后仍实现同一算法” 区分开来。

3.6.3. ONNX 到 OM 的转换与验证

ONNX 首先按照 ONNX 官方规范检查计算图、张量名称、形状和数据类型，再由与开发板匹配的 CANN 工具链转换为 OM；ATC 参数及目标芯片配置应以 Ascend CANN 官方文档为准。精度模式不是孤立的编译选项，而会改变循环状态和声学控制量的数值行为，因此不同网络应依据参考误差与实时预算分别确定，不能仅以转换成功或模型文件大小作为选择标准。

验证分为四个相互衔接的层次：

验证层次	核心问题	主要判据
结构一致性	OM 是否实现预期的输入输出接口	张量名称、形状、类型和状态维度完全对应
数值一致性	图变换和精度选择是否改变模型行为	ONNX 与 OM 在固定输入及连续状态回路上的误差受控，无 NaN 或 Inf
时间可行性	推理是否能在控制周期内稳定完成	预热后统计延迟分布，并以高百分位而非单次最小值判断
系统一致性	神经控制量能否形成连续可听的音频	检查非静音输入输出、音频缓冲、削波、设备路由与长时间状态增长

其中，Piano-DDSP 尤其需要考察全局状态与逐声部状态的长期累积误差；MIDI-DDSP 需要保持双向上下文、自回归采样和整曲统计；DDSP-VST 则需要联合验证 Feature 与 Control 的控制周期。详细转换命令、张量夹具和报告格式见模型与 OM 部署文档。

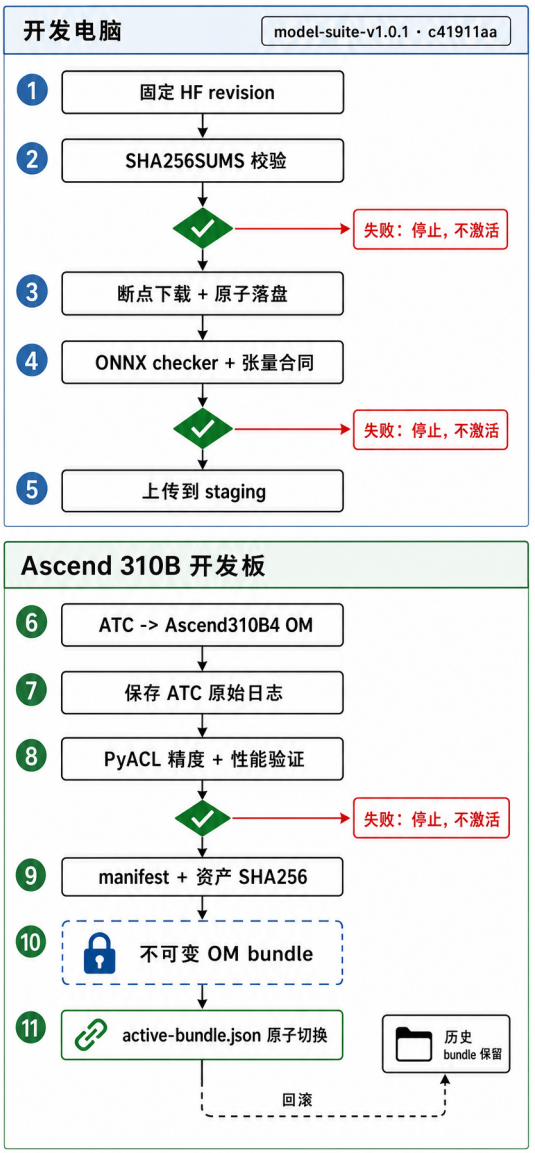


图 3.8. 模型从来源确认、ONNX 语义验证、板端 ATC 转换到 PyACL 闭环检验的流程

3.6.4. 运行时模型边界

实验系统在板端以 OM 作为统一的神经网络执行形式，使三套模型共享相同的 NPU 调用和资源管理边界。若模型合同、循环状态或音频设备条件不满足，运行会话应显式终止并释放资源，避免不同推理后端或隐式设备切换混入同一次实验。ONNX 与 TFLite 在这里承担来源对照和数值参考的角色，性能评价则以真实 OM 推理和完整音频链路为对象。

3.7. 后端设计

3.7.1. 服务分层与作业模型

后端以 FastAPI 为服务框架，将静态界面、请求响应、事件推送、文件产物和硬件控制组织在同一应用边界内。查询类请求用于获得模型、设备和会话快照，命令类请求负责启动或停止具有副作用的操作，WebSocket 则传递音符边沿、作业进度和实时指标。三类通信承担不同的时间语义，可避免用高频轮询近似短暂事件，也能防止将耗时渲染阻塞在一次 HTTP 请求中。

MIDI-DDSP 渲染采用异步作业模型。输入 MIDI、声部分配、模型选择与合成参数共同确定一次渲染上下文，输出 WAV 与任务元数据构成可追溯的结果版本。关系数据库只承担索引和检索功能，原始 MIDI、音频产物及其元数据仍是实验记录的主体；这种安排既支持同一乐曲的多方案比较，也避免界面状态成为唯一的数据来源。

3.7.2. 资源互斥与实时钢琴

资源协调层对以下会占用 NPU 或物理音频设备的流程实行互斥：

- Piano-DDSP 触控与实体 MIDI 共享实时会话；
- DDSP-VST Effect 双工链路；
- MIDI-DDSP 板端 WAV 播放；
- 麦克风输入测试；
- 扬声器左右声道测试。

冲突请求会被显式拒绝，而不是让多个任务同时打开同一硬件。会话在正常停止、异常退出或设备断开时均需释放资源；设备和模型的实际路径由后端管理，浏览器只表达选择意图。由此可以把一次实验的输入、模型与输出路由固定在明确的资源组合上。

实时钢琴在独立常驻进程中维持神经状态和音频状态。MIDI 按下事件立即进入状态机，过早到达的释放事件只延长合成内部的最小发声门限，从而避免一次浏览器刷新周期内的快速点按退化为零长度声音。重复音符、延音踏板、声部替换和异常断开都在同一状态机中处理，以保证有限声部槽位能够被确定地占用和回收。

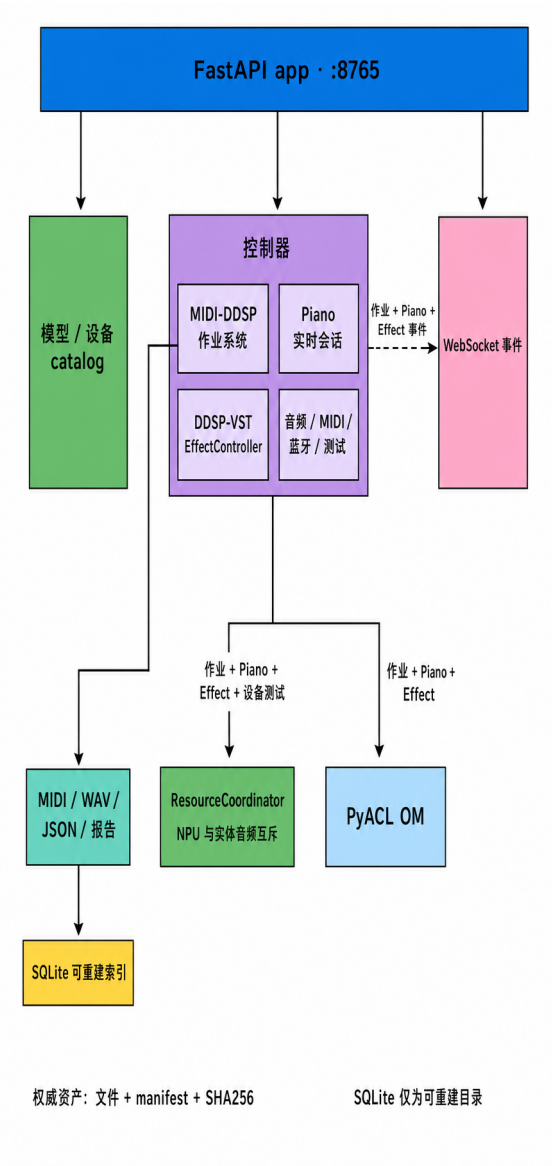


图 3.9. Case 3 FastAPI 后端的控制器、资源互斥、PyACL、事件和存储模块边界

合成结果先写入有界队列，再依据交互延迟与抗抖动需求组合为不同长度的音频块。较短的缓冲提高按键响应，但对推理抖动和设备调度更敏感；较长的缓冲能够提高连续性，却增加演奏反馈延迟。合成增益、系统混音器音量和音箱硬件音量属于三个独立控制层，评价时应分别记录，避免把音量变化误判为模型幅度变化。

3.7.3. DDSP-VST 音频线程与安全控制

Effect 的固定链路为：`48 kHz 双声道摄像头输入 -> 单声道 -> 16 kHz -> 1024 窗/320 步长 -> Feature OM -> Control OM -> DDSP 合成 -> 48 kHz 双声道输出`。输入和输出使用 `parec`、`paplay` 对应的 PulseAudio source/sink，不使用 monitor，也不在设备丢失后自动改路。

噪声门先校准环境底噪，再用开启阈值、迟滞、保持、开启和关闭时间平滑门增益。默认只输出转换后的声音，输出增益为 `-18 dB`。持续过载会进入明确的安全静音，并在状态和 WebSocket 中报告；不会继续输出可能削波的声音。

3.7.4. 通信语义

通信形式	适用信息	设计依据
REST 查询	模型目录、设备目录、系统状态和历史作业	信息具有快照语义，可重复读取且不改变运行状态
REST 命令	会话启停、参数修改、渲染创建和设备测试	操作具有明确副作用，需要校验资源条件并返回执行结果
WebSocket 事件	音符边沿、踏板变化、作业进度、设备变化和实时指标	信息短暂且连续，轮询可能遗漏事件或引入额外延迟
文件访问	MIDI、WAV、录音与评价报告	大型产物采用独立传输，避免混入控制消息

通信接口的核心约束是：客户端提交目录标识和有界参数，服务端解析实际模型与设备；实时事件负责同步变化，状态快照负责断线后的重新对齐。具体端点和字段属于实现合同，见WebUI 与 API 指南。由于服务直接控制音频与模型资源，部署范围应限定在可信网络中。

3.8. 前端设计

3.8.1. 信息架构与状态同步

前端以 React、TypeScript 和 Canvas 构成交互层，按照实时演奏、MIDI 文件渲染、麦克风音色转换和设备管理四类任务组织顶层信息。触摸琴键与实体 MIDI 键盘共享同一

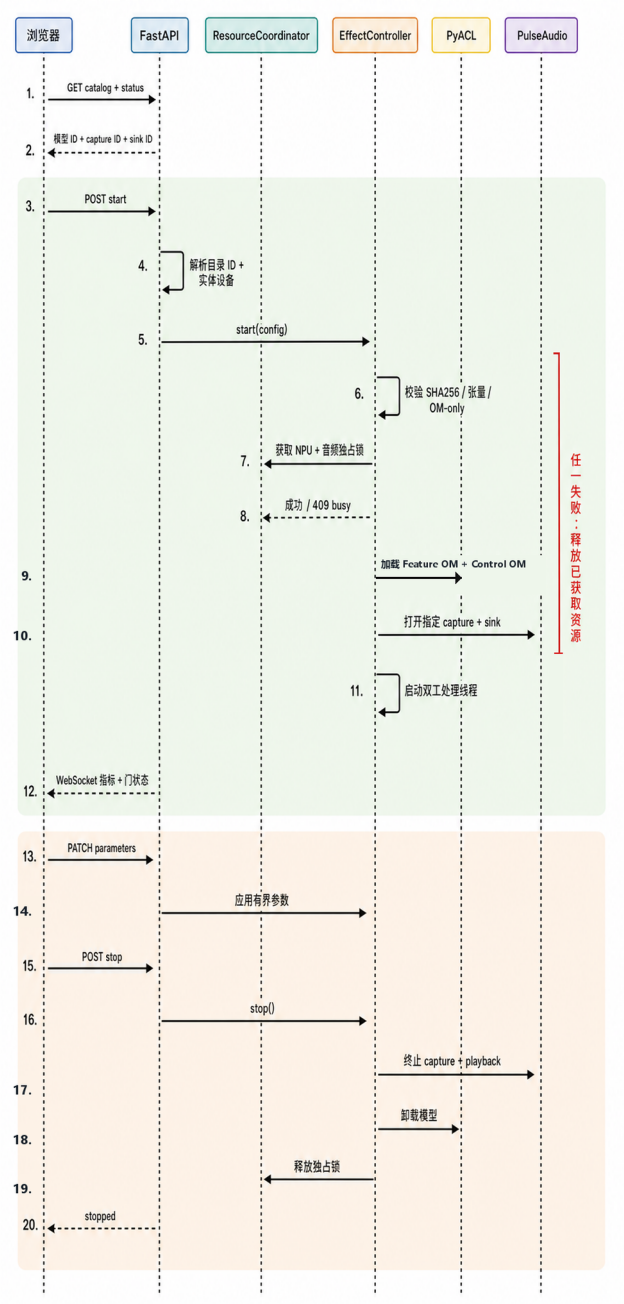


图 3.10. DDSP-VST Effect 从目录查询、资源加锁和模型启动到有序停止释放的时序

实时演奏工作区和后端会话，只在输入方式上切换，避免把同一模型和参数拆成两套彼此漂移的界面。

界面状态分为三类：用户偏好、服务端快照和实时事件。音色显示方式、键盘范围等偏好可以保存在浏览器中；模型可用性、设备选择和会话生命周期以服务端为准；音符、作业进度和音频指标则由事件流更新。恢复页面时先读取快照，再接续事件，能够在刷新或短暂断线后重新建立一致状态。会话运行期间锁定会改变模型或设备归属的选项，以免界面选择与实际音频链路脱节。

低频状态采用周期性查询，短暂变化采用 WebSocket 推送。工作区按需加载，包含连续动画的视图在不可见时暂停绘制，使界面更新不会与音频线程争夺不必要的 CPU 时间。运行、不可用、故障、输入门和安全静音等状态同时使用文字、图标和颜色表达，保证信息不依赖单一视觉通道。

3.8.2. 两个钢琴卷帘的时间模型

实时演奏和 MIDI 文件浏览都使用钢琴卷帘，但二者具有不同的时间模型：

卷帘类型	时间来源	绘制策略	性能边界
实时卷帘	WebSocket 的音符边沿	缓存键位与网格，仅更新新增轨迹和当前按键	限制刷新率、像素密度与历史长度，空闲时停止绘制
文件卷帘	完整 MIDI 时间轴与播放位置	分离静态网格、音符层和动态光标	以分层 Canvas 承载长序列，避免为每个音符创建界面节点

实时卷帘保留短音符的最小可见高度；状态快照只用于重新同步当前按键，不能反推已错过的短音符。文件卷帘支持声部颜色、拍号网格、缩放、拖动、进度光标和活动音符，但不是 MIDI 编辑器。

3.8.3. 触摸屏、桌面和手机布局

响应式设计不以某一种屏幕尺寸作为成立条件，而以内容优先级、可用空间和输入方式决定布局。宽视口并排显示控制面板与主要可视化区域，空间收窄时依次换行或折叠次要信息；钢琴键盘和卷帘保持自身比例并在容器内缩放。触控目标应留有足够间距，连续滑块与离散命令采用不同控件，文本标签不得因缩放而被截断。

移动布局将高频工作流保留在可达位置，并处理浏览器安全区；桌面布局则强调横向比较和键盘焦点。所有布局都应避免文档级横向滚动、控件重叠和状态跳动，同时保留页签语义、可访问名称与清晰的焦点顺序。由此，界面可以适应不同显示设备，而不把案例能力绑定到某一分辨率或物理尺寸。

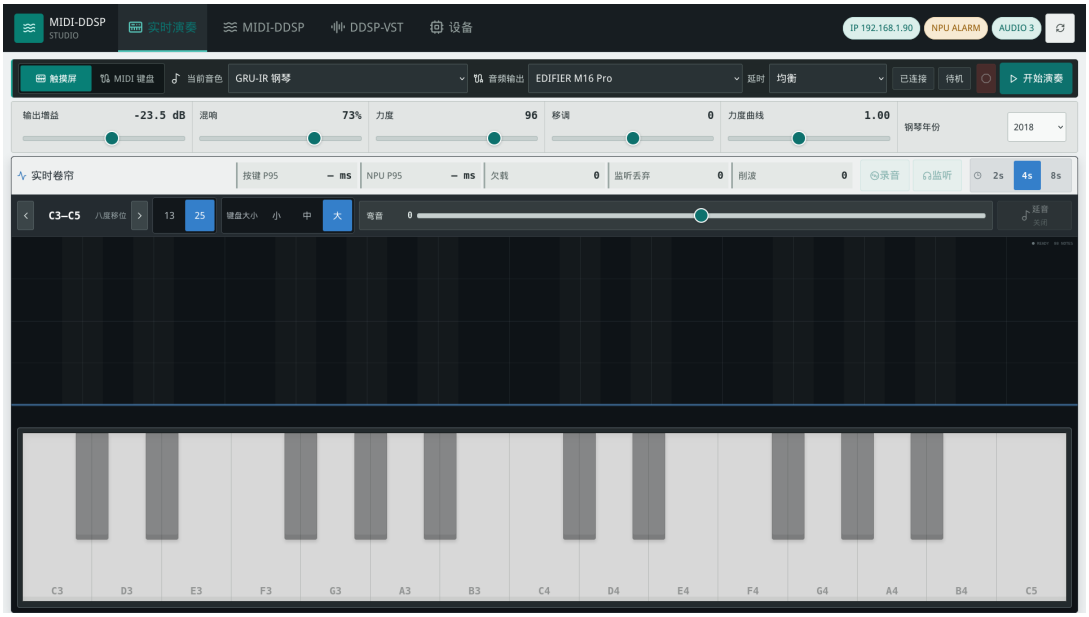


图 3.11. 实时演奏工作区的触摸屏模式

3.9. 逐页界面与操作说明

本节截图采集自 Ascend 310B 开发板实际运行的服务，用于说明界面结构、交互关系和典型状态，不作为性能测量证据。界面中的 NPU 诊断警告应与设备可见性、模型加载状态和 OM 推理结果联合解释，不能由单一提示直接推断推理失败。

3.9.1. 实时演奏 / 触摸屏

区域	控件	作用	正常状态	注意事项
统一会话栏	输入方式、当前音色、音频输出、延时档、开始/停止、Panic	管理共享 Piano-DDSP 会话	截图为已连接、待机	会话运行、录音或切换时不能更换输入方式；Panic 用于释放悬挂音符
参数带	输出增益、混响、触控力度、移调、力度曲线、钢琴年份	实时改变当前钢琴音色参数	数值稳定，不改变布局	输出增益不是系统音量，参数值受服务端目录约束
卷帘工具栏	按键 P95、NPU P95、欠载、监听丢弃、削波、录音、监听、2/4/8 秒	查看会话指标和实时 note edge	待机指标为空或为 0，运行后更新	短音符保留最小可见轨迹，监听和录音只在会话运行时可用

区域	控件	作用	正常状态	注意事项
触控命令栏	13/25 键、小/中/大、八度、弯音、延音	调整屏幕琴键范围并演奏	截图为 25 键与大键盘	无缓存时键盘大小默认为 “中”；弯音松开自动回中，88 键只属于 MIDI 键盘模式
底部钢琴	触控白键和黑键	手指直接发送 `note_on`/`note_off`	待机可查看音域，开始后发声	快速松开仍由后端保证 16 ms 门长；失焦、取消触摸或停止会全部停音

3.9.2. 实时演奏 / MIDI 键盘

区域	控件	作用	正常状态	注意事项
统一会话栏与参数带	当前钢琴音色、音频输出、延时档、输出增益、混响、移调和力度曲线	与触摸屏模式共享 Piano-DDSP 会话	截图为已连接、待机	本页没有第二套音色参数，也不加载 DDSP-VST Synth 或 MIDI 文件抽屉
键盘范围	常见键数与八度	匹配实体控制器与可视范围	键数与已连接键盘一致	非全音域键盘可逐八度移动；全音域范围固定为 A0-C8
实体 MIDI 输入	服务端端口下拉框	绑定板端枚举的输入端口	已连接键盘被选中	浏览器 Web MIDI 不是输入来源；运行中不能换端口，断开时释放该来源音符
卷帘工具栏	会话指标、录音、监听、2/4/8 秒	观察真实 WebSocket 音符边沿和运行状态	短音符仍有可见标记	不用轮询替代边沿事件；录音、监听和指标与触摸屏模式共享
可视键盘	只读键位高亮	对照实体 MIDI 的当前音符和音域	32 键范围为 F2-C5	只用于反馈，不在此处提供鼠标或触控演奏

3.9.3. MIDI-DDSP 音频库

区域	控件	作用	正常状态	注意事项
主标题	音频库/新建渲染	切换浏览与创建流程	音频库选中	两种流程不混在一个长页面
曲目与版本	版本下拉、设为默认	一首 MIDI 管理多次渲染	版本、音色和 WAV 同步变化	缺失文件会标记不可用，不删除历史

区域	控件	作用	正常状态	注意事项
文件卷帘	缩放、跟随、全屏	查看完整 MIDI 和播放光标	仅一个可见卷帘	页面不恢复第二个波形动画
播放区	开发板/浏览器、设备、增益	选择实际播放目标	默认开发板喇叭	浏览器播放不改变系统 mixer
右侧曲目列表	曲目、版本数、时长	快速切换音频库来源	选中项有文字和底色	选择不可用版本时返回错误

3.9.4. MIDI-DDSP 新建渲染

区域	控件	作用	正常状态	注意事项
曲目栏	MIDI 选择和上传	选择现有文件或上传受限格式	显示时长、音符、轨道和声部	只接受 `.mid/.midi` 和大小限制
文件卷帘	声部颜色与视图控制	检查声部分离结果	声部数与表格一致	和弦轨需先拆成单音声部
分配表	每声部合成音色	修改自动建议	每个声部都有有效目录 ID	浏览器不提交模型文件路径
右侧设置	模型包、方案、随机种子、增益、尾音	固定可复现的渲染配置	OM 已验证、声部已就绪	相同配置再次渲染仍创建新版本
底部操作	开始渲染、状态、产物	创建异步任务并显示进度	成功后可下载	运行时保留取消、心跳、预计完成时间和报告

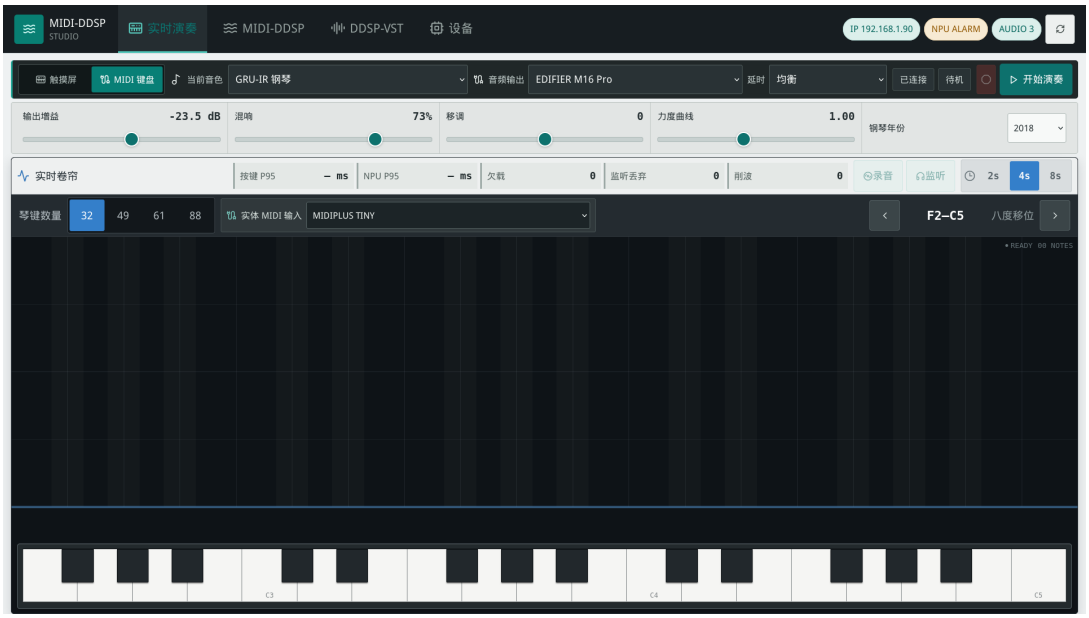


图 3.12. 实时演奏工作区的 MIDI 键盘模式

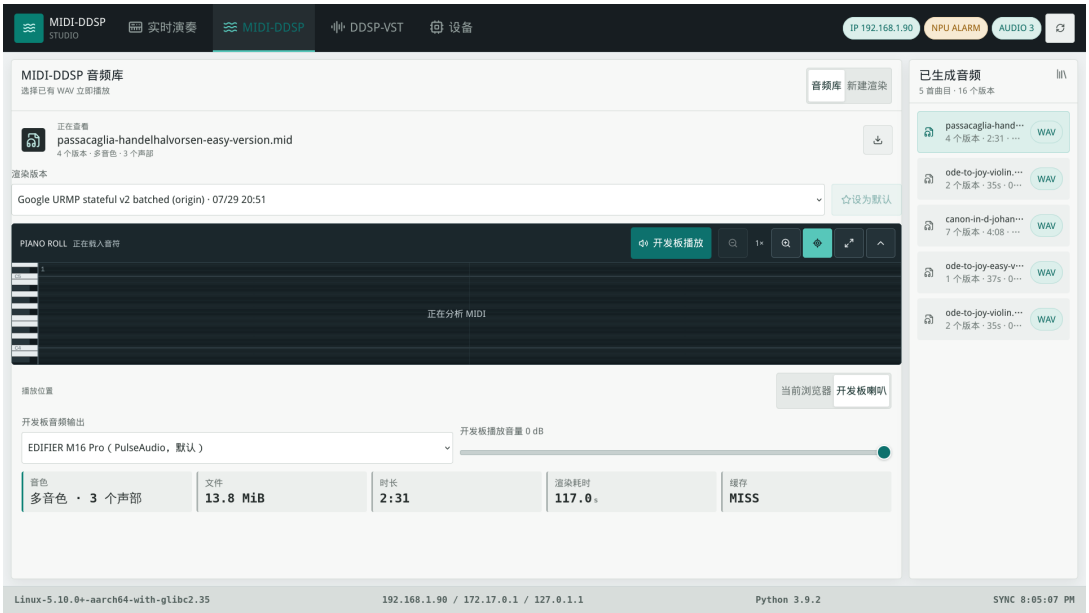


图 3.13. MIDI-DDSP 音频库页面

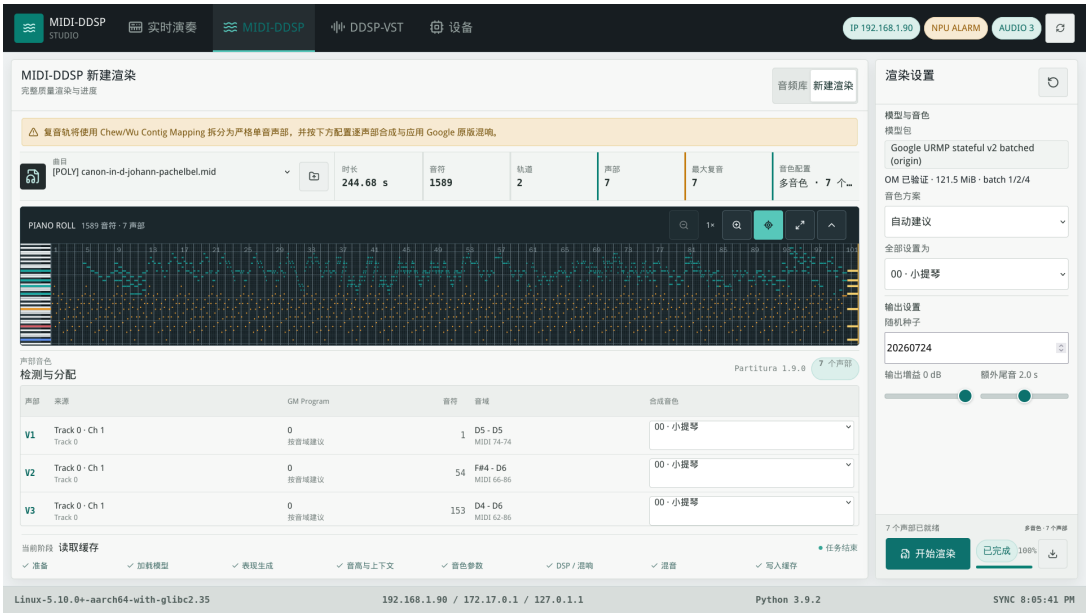


图 3.14. MIDI-DDSP 新建渲染页面

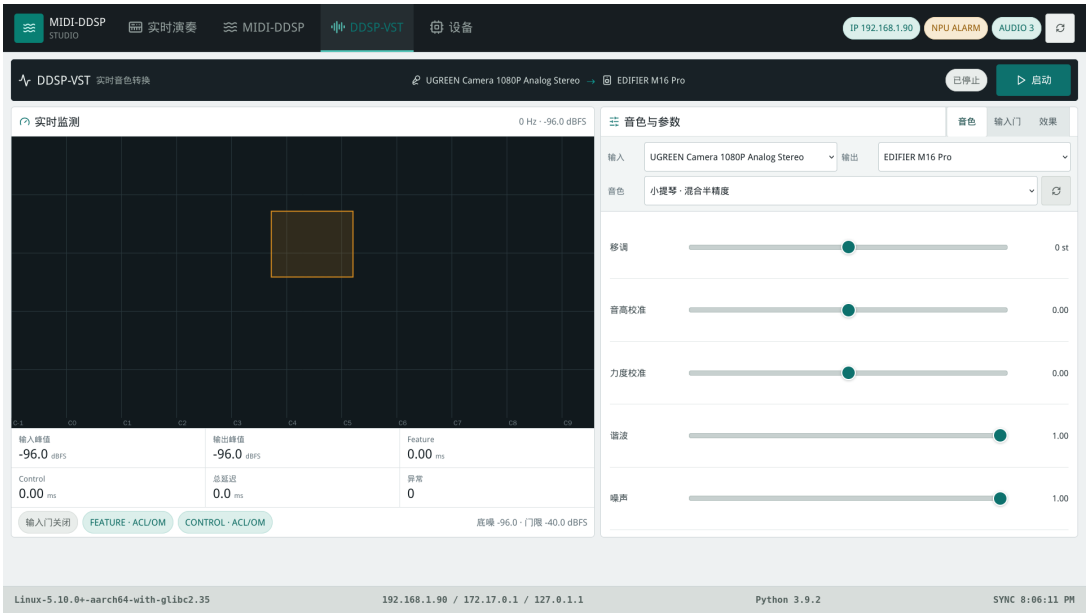


图 3.15. DDSP-VST 音色页面

3.9.5. DDSP-VST 音色

区域	控件	作用	正常状态	注意事项
顶部链路	输入、输出、运行/停止	明确真实设备路径	实体输入到所选音频输出，运行中	运行时不允许静默换设备
轨迹 Canvas	音高和响度点	观察单音输入与门限	Canvas 非空，异常为 0	环境噪声也可能有音高估计，不等于已输出
模型区	11 种中文音色	选择 Control OM	已选择小提琴音色	后端使用稳定的模型 ID
音色参数	移调、音高校准、力度、谐波、噪声	调整转换控制	有边界值即时更新	谐波和噪声全关会失去主要声源
指标	Feature、Control、总延迟、后端	检查 OM 链路和线程状态	`ACL/OM` 且异常 0	指标不能代替听感评价

3.9.6. DDSP-VST 输入门

区域	控件	作用	正常状态	注意事项
校准	重新校准	采集环境底噪并给出建议门限	显示底噪和门限	校准时应保持安静，不要讲话或播放声音
开启门限	dBFS 滑块	达到阈值后允许音频进入	底噪低于开启门限	太低会被噪声触发，太高会吞轻声
迟滞	dB 滑块	让关闭阈值低于开启阈值	开关不频繁抖动	迟滞不是额外增益
保持/开启/关闭	毫秒滑块	平滑门状态	输入门状态稳定	关闭过慢会拖尾，过快会切断音头

区域	控件	作用	正常状态	注意事项
混响	0 到 1	控制湿声比例	默认 0	本实现没有原声 Dry/Wet 直通
安全状态	异常和安全静音	防止持续过载	异常 0、安全静音关闭	触发后应停止并排查输入和增益

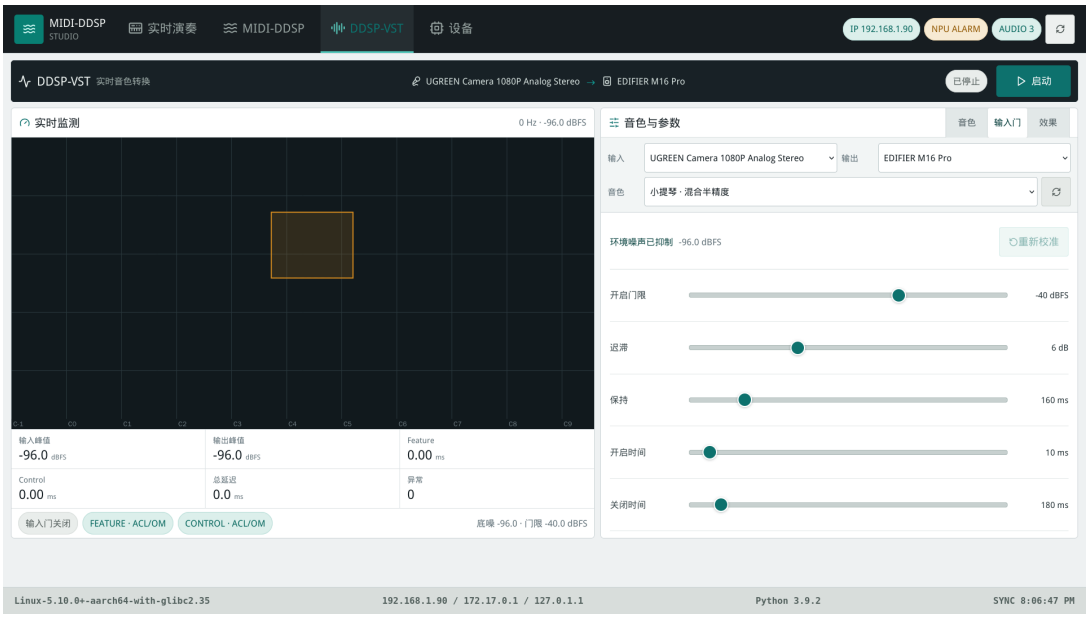


图 3.16. DDSP-VST 输入门页面

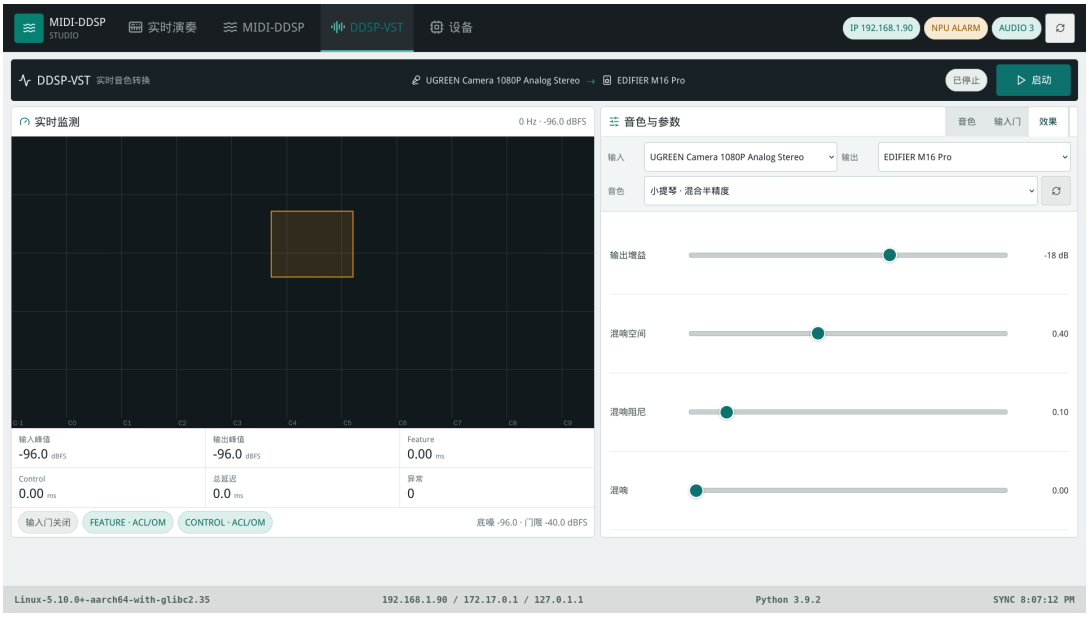


图 3.17. DDSP-VST 效果页面

3.9.8. 设备概览

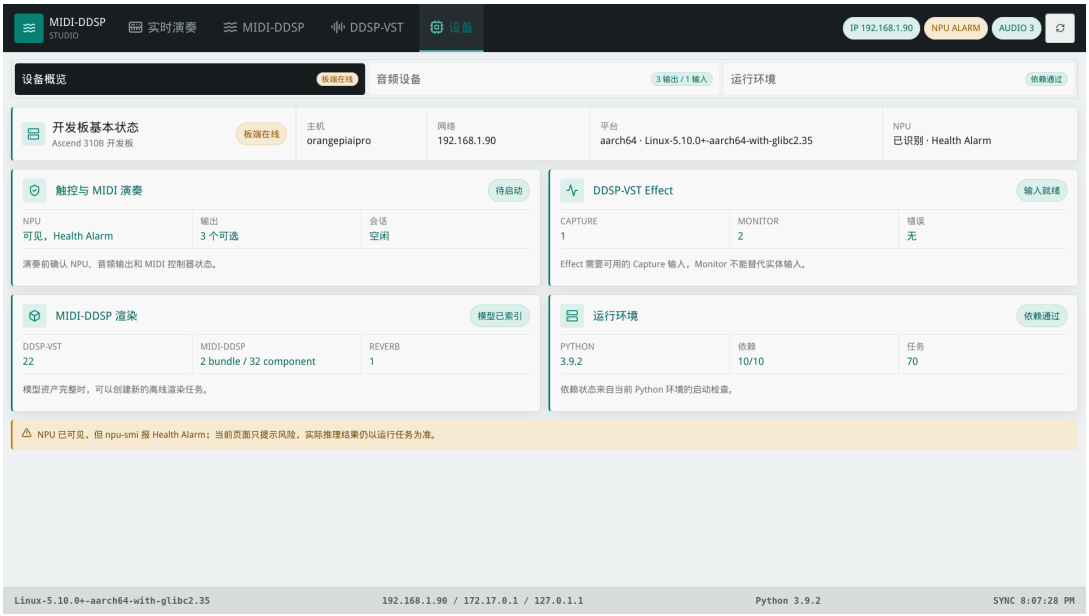


图 3.18. 设备概览页面

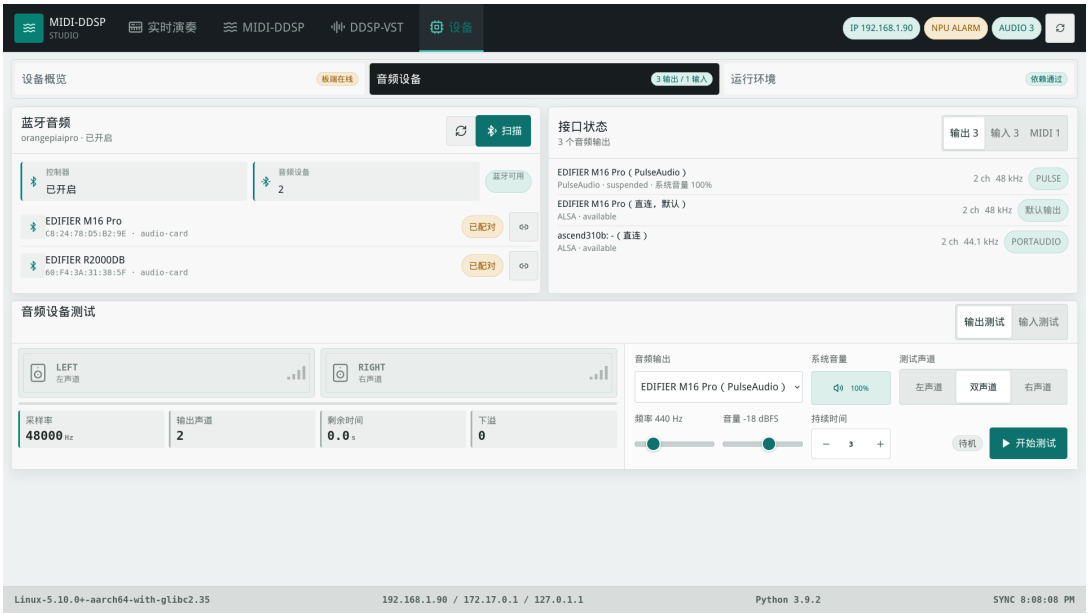


图 3.19. 音频输出与扬声器测试页面

区域	正常状态	验收要点
开发板状态	板端在线、IP 可见、NPU 状态显示	Health Alarm 是警告，真实推理结果优先
触控与 MIDI	输出可选、会话无错误	概览只提示准备度，不重复提供跳转按钮
DDSP-VST	至少一个 capture、错误为 0	monitor 数量不代表麦克风数量
MIDI-DDSP	bundle 和组件已索引	发现资产不等同于已听音验收
运行环境	Python、依赖、任务状态可见	详细内容放在运行环境页

3.9.9. 音频输出与扬声器测试

区域	控件	作用	正常状态	注意事项
蓝牙音频	刷新、扫描、连接	管理板端已有蓝牙能力	控制器开启、设备可见	不在板端安装缺失的 `bluetoothctl`
接口状态	输出/输入/MIDI	切换设备清单	系统音频输出和直接 ALSA 设备可见	“检测到” 不等于已听见
声道表	Left、Right 和电平	展示测试声道和状态	空闲时电平归零	测试会独占输出资源
输出设置	音频输出、系统音量	选择输出并显示系统音量	已选择有线输出且未静音	音箱硬件按键应通过设备事件更新显示

区域	控件	作用	正常状态	注意事项
测试设置	左/双/右声道、频率、增益、时长	播放受控正弦测试	状态从 IDLE 进入运行	从低增益开始，避免突然高声压

3.9.10. 音频输入与麦克风测试

区域	控件	作用	正常状态	注意事项
输入清单	capture 与 monitor	区分实体采集和回放监视	实体输入标为 CAPTURE	Effect 和输入测试都拒绝 monitor
实时电平	dBFS 表和峰值	观察麦克风输入	讲话时电平变化、无溢出	`-96 dBFS` 表示空闲或未开始测试
输入选择	capture 下拉	固定真实麦克风 ID	实体输入被选中	设备断开后不会自动替换
检测阈值	dBFS 滑块	控制输入测试是否判为有效	高于底噪、低于说话峰值	与 DDSP-VST 噪声门是两个不同设置
时长与开始	步进器、开始输入测试	运行独占采集检查	IDLE 或完成结果	测试期间不能启动 Effect

3.9.11. MIDI 设备

区域	控件	作用	正常状态	注意事项
MIDI 清单	MIDI 页签	枚举服务端输入端口	键数与可用状态可见	浏览器 Web MIDI 不是板端输入来源
设备与端口	型号、输入端口	识别正确设备	端口状态正常	重新插拔后设备 ID 可能变化，应刷新
MIDI 输入状态	全宽设备状态卡	显示键数、型号、输入端口和可用状态	实体键盘已连接	切换到 MIDI 后不显示音频输入输出测试；MIDI 枚举本身不会产生声音
蓝牙区	已配对音频设备	展示另一类外设状态	与 MIDI 清单分开	蓝牙音频不能替代 USB MIDI

3.9.12. 运行环境

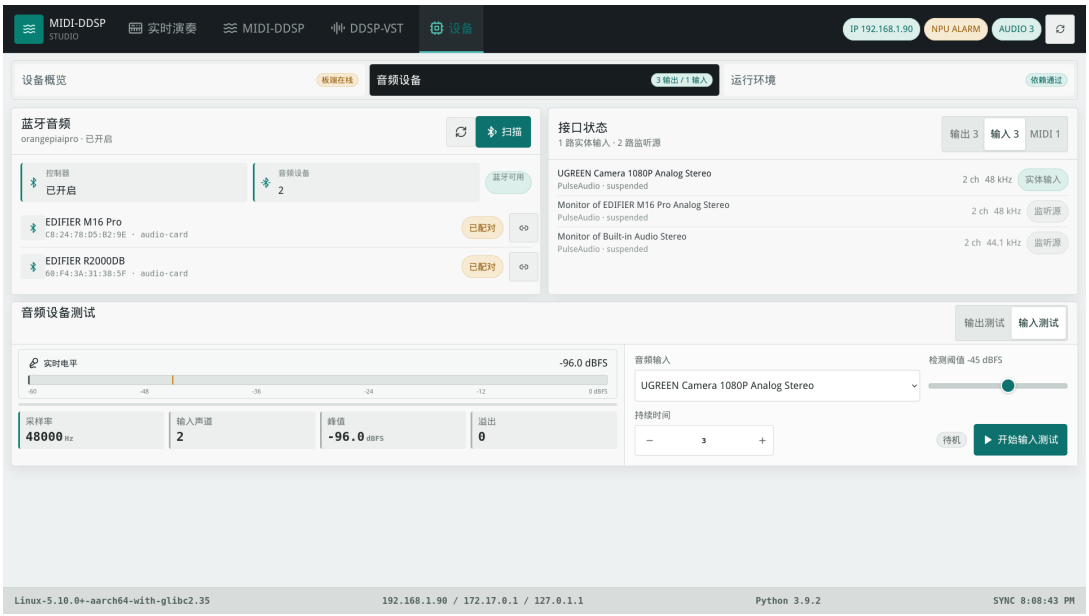


图 3.20. 音频输入与麦克风测试页面

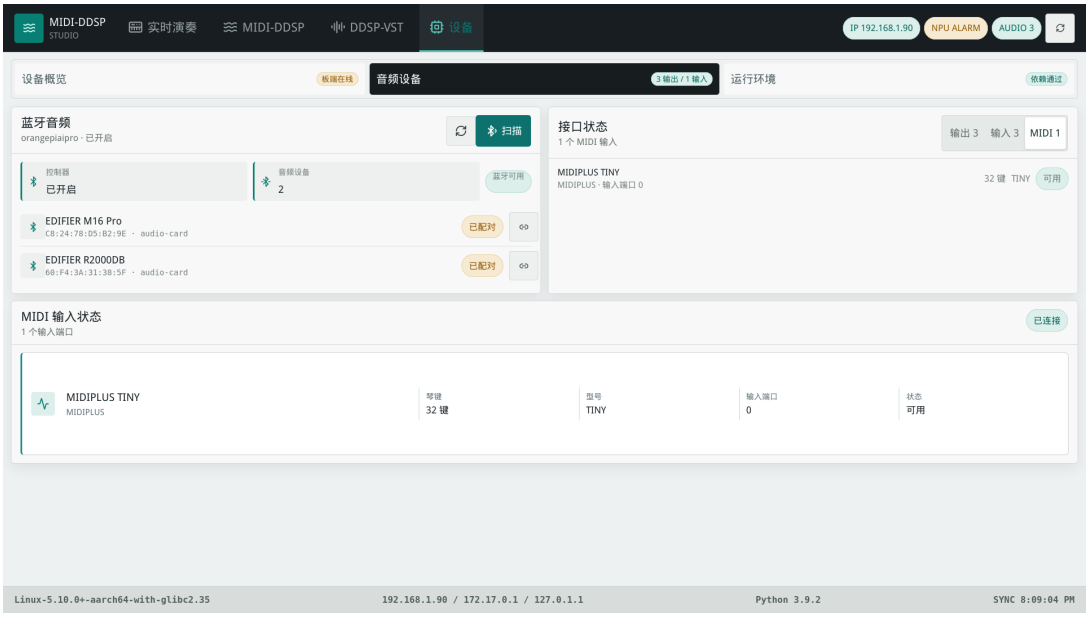


图 3.21. MIDI 设备页面

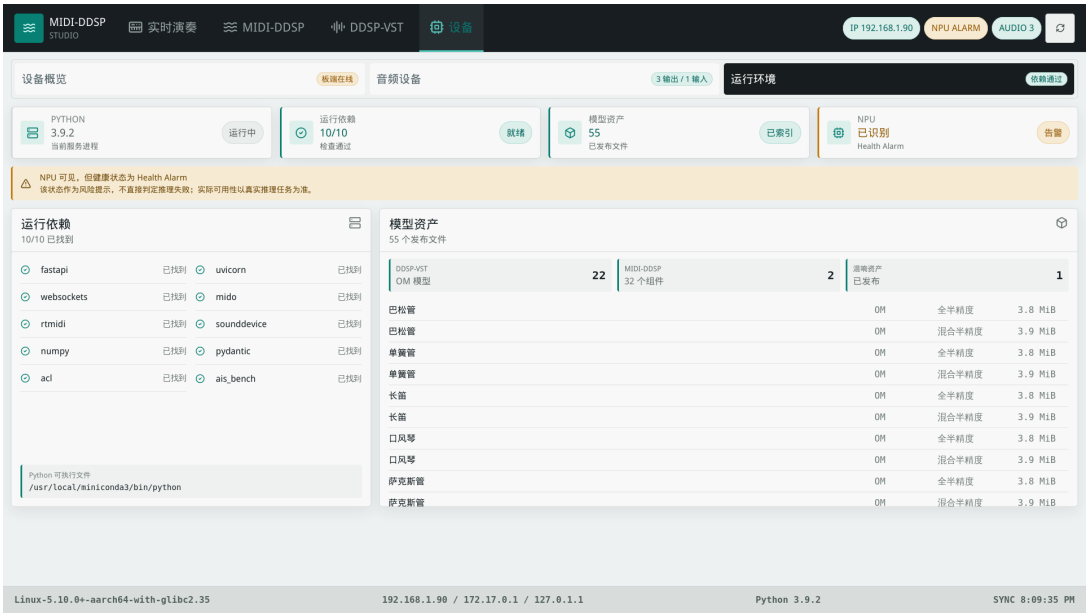

MIDI-DDSP
STUDIO
实时演奏
MIDI-DDSP
DDSP-VST
设备
IP 192.168.1.90
NPU ALARM
AUDIO 3
设备概览
板端在线
音频设备
3 输出 / 1 输入
运行环境
依赖通过
PYTHON
3.9.2
当前服务进程
运行中
运行依赖
10/10
检查通过
就绪
模型资产
55
已发布文件
已索引
NPU
已识别
Health Alarm
告警
NPU 可见，但健康状态为 Health Alarm
该状态作为风险提示，不直接判定推理失败；实际可用性以真实推理任务为准。
运行依赖
10/10 已找到
fastapi 已找到
uvicorn 已找到
websockets 已找到
mido 已找到
rtmidi 已找到
sounddevice 已找到
numpy 已找到
pydantic 已找到
acl 已找到
ais_bench 已找到
Python 可执行文件
/usr/local/miniconda3/bin/python
模型资产
55 个发布文件
DDSP-VST
OM 模型
22
MIDI-DDSP
32 个组件
2
已发布
1
巴松管 OM 全半精度 3.8 MiB
巴松管 OM 混合半精度 3.9 MiB
单簧管 OM 全半精度 3.8 MiB
单簧管 OM 混合半精度 3.9 MiB
长笛 OM 全半精度 3.8 MiB
长笛 OM 混合半精度 3.9 MiB
口风琴 OM 全半精度 3.8 MiB
口风琴 OM 混合半精度 3.9 MiB
萨克斯管 OM 全半精度 3.8 MiB
萨克斯管 OM 混合半精度 3.9 MiB
Linux-5.10.0+-aarch64-with-glibc2.35
192.168.1.90 / 172.17.0.1 / 127.0.1.1
Python 3.9.2
SYNC 8:09:35 PM

图 3.22. 运行环境页面

区域	控件	作用	正常状态	注意事项
摘要	Python、依赖、模型、NPU	一屏判断服务准备度	版本号与依赖状态均可见	版本应与部署清单和板端环境一致
NPU 警告	Health Alarm 文本	如实展示硬件诊断	警告可见	不直接粘贴整个终端输出
运行依赖	包名与找到状态	定位 Python 启动缺包	全部找到	板端只允许唯一 requirements 的 pip 安装例外
模型资产	类型、精度、大小	检查目录索引结果	OM 数量和精度可见	资产存在仍需清单和真实推理验证
Python 环境	解释器与环境名称	核对服务使用的板端运行环境	环境与依赖合同一致	不在运行中隐式切换解释器

3.9.13. 布局验收场景

使用场景	导航与布局策略	重点验收
横向触摸显示器	完整导航，主控制与可视化并排	实际点按、琴键可达、无裁切和滚动冲突
桌面浏览器	完整导航，强调信息对照	参数区紧凑但不重叠，Canvas 与表格无横向溢出
平板或窄窗口	控制组换行，次要信息折叠	页签语义、触控间距和文本完整性
移动浏览器	采用易触达导航并处理安全区	核心命令可达，无文档级横向滚动

3.10. 从零复现实验

3.10.1. 接线与启动检查

1. 断电状态连接开发板电源、启动存储、HDMI 屏幕和 USB 触控线。
2. 连接音频输出、摄像头或独立麦克风以及 MIDI 键盘；高功耗 USB 设备较多时使用合规的独立供电 Hub。
3. 连接网线或 Wi-Fi，启动开发板，确认触摸和桌面正常。
4. 在开发板查看 IPv4、USB、ALSA/PulseAudio 和 MIDI；这些是板端只读诊断，不在开发电脑执行。

```
ip -4 address
lsusb
pactl list short sinks
```

```
pactl list short sources
cat /proc/asound/cards
cat /proc/asound/seq/clients
```

在 `pactl list short sources` 中，麦克风应显示为实体 capture source。名称以 `.monitor` 结尾的条目是输出回放监视源，不能用于 DDSP-VST 或麦克风输入测试。

3.10.2. 开发电脑准备与本地测试

```
# 从 Ascend310 仓库根目录执行
cd samples\case3
python -m pip install -r requirements.txt
python -m pytest -q

cd webui
npm ci
npm run test
npm run build
npm run test:e2e
```

各 npm 命令的职责不同：

命令	作用	何时必须执行
`npm ci`	严格按 `package-lock.json` 安装前端依赖	第一次准备环境、删除过 `node_modules` 或锁文件变化后
`npm run test`	用 Vitest 检查组件、状态和交互回归	前端逻辑、组件或相关样式改变后；纯低风险文字修改可酌情跳过
`npm run build`	TypeScript 检查并生成部署所需 `webui/dist`	任何前端源码变化后必须执行
`npm run test:e2e`	用 Playwright 检查生产页面布局和交互	发布前或影响工作流、响应式、Canvas 时

开发板不安装 Node 或 npm，也不运行 Vite 生产服务器。它只接收开发电脑生成的 `webui/dist/`。

3.10.3. 模型准备与转换边界

模型准备应遵循第 6 节给出的两类来源：Piano-DDSP 使用作者训练并验证的 ONNX，MIDI-DDSP 与 DDSP-VST 使用由上游 TFLite 语义重构并完成对照的 ONNX。进入 ATC 之前，应确认来源、输入输出合同和参考结果彼此对应，避免把不同结构或音色的模型混入同一次转换。

ONNX 到 OM 的转换和 PyACL 验证必须在真实 Ascend 开发板上完成，开发电脑不安装或模拟 CANN。各模型族的输入形状、循环状态和精度判据不同，应依据模型与 OM 部署文档与Piano-DDSP 合同执行，并保留足以复核图变换和数值对照的实验记录。

3.10.4. 部署、原子切换和启动

在开发电脑上执行仓库提供的部署脚本：

```
# 从 Ascend310 仓库根目录执行
cd samples\case3
powershell -ExecutionPolicy Bypass -File tools/deploy_midi_ddsp_webui.ps1
```

部署过程先在暂存位置完成程序、前端资源和模型元数据的一致性检查，再原子切换运行版本。该策略使正在运行的服务不会读取到半同步状态，并将程序升级与实验产生的 MIDI、WAV 和评价记录分离。

在板端已有 conda `base` 中，仅允许安装本仓库唯一 requirements 并确认 pytest：

```
cd /home/HwHiAiUser/Documents/case3
/usr/local/miniconda3/bin/python -m pip install -r requirements.txt
/usr/local/miniconda3/bin/python -m pytest -q
/usr/local/miniconda3/bin/python scripts/check_webui_env.py
/usr/local/miniconda3/bin/python scripts/run_webui.py
```

服务启动后检查本机页面和状态接口：

```
curl -fsS http://127.0.0.1:8765/
curl -fsS http://127.0.0.1:8765/api/v1/status
```

还应核对进程状态、事件连接、生产静态资源和触摸操作。由于服务能够直接控制模型和音频设备，不应将其暴露到不可信网络。

3.10.5. 功能验收顺序

1. **设备**：确认网络、NPU、Python 环境、实体音频输入输出和 MIDI 键盘均可用。
2. **触控演奏**：先以低音量测试单音，再测试快速点按、多指、延音、移调和 panic。
3. **MIDI 键盘**：测试力度、重复音、快速音阶、CC64、断开重连和无悬挂音符。
4. **MIDI-DDSP**：选择 MIDI，检查声部分离和分配，渲染 WAV，切换版本并测试开发板/浏览器播放。
5. **DDSP-VST**：停止钢琴会话，选择实体输入与输出，安静校准噪声门，再以独立单音声源测试各类音色。

真实音频验收至少记录推理耗时、队列延迟、总延迟、PCM 非零、削波、capture overflow 和 playback underrun。仅有 HTTP 200 不能证明发声正确。

3.11. 性能评价与故障排查

3.11.1. DDSP-VST 板端长稳评价方法

为避免把接口连通性、单模型推理或短时运行误写成实时音频性能，本案例采用长时间双工测试评价 DDSP-VST Effect。评价协议包括：

- 固定实体音频输入、待测 Control OM 和实体音频输出；
- 使用独立单音声源，禁止把扬声器到麦克风的声学反馈当作输入；
- 持续至少 `600 s`，状态每 `10 s` 采样一次，处理帧数持续增长；
- 存在非静音输入、非静音输出和有效 F0；
- Feature 与 Control 的合计 p95 小于 `20 ms`，软件总延迟小于 `150 ms`；
- capture overflow、playback underrun、clipped samples 均为 `0`，且 safety mute 未触发。

其中，p95 表示采样期间耗时分布的第 95 百分位数，时间单位均为毫秒。`20 ms` 和 `150 ms` 是本案例的验收阈值，不是未执行测试时的性能结论。

只有同时满足上述条件的报告，才能作为板端性能结论的证据。接口响应、单独的 OM 冒烟测试或页面截图均不能替代双工长稳评价；未通过的实验也应保留原始指标，以免在事后选择性呈现结果。

3.11.2. 常见问题

现象	优先检查	处理原则
页面正常但没有声音	会话状态、输出 ID、系统静音、合成增益、PCM 非零、underrun	先用扬声器测试确认路由，再检查模型和 MIDI；不要同时提高系统音量和合成增益
环境噪声触发 DDSP-VST	capture 是否正确、底噪、开启门限、迟滞和校准环境	安静时重新校准；提高开启门限，保留合理迟滞和保持时间
DDSP-VST 始终没有输出	输入门、`pw_db`、安全静音、Feature/Control 后端	确认讲话峰值高于门限且两个后端都是 `acl/om`
快速 MIDI 声音抖动	重复 note edge、声部复用、FIFO、设备 underrun	不增加前端 hold delay；检查后端 16 ms 最小门、WebSocket 边沿和 MIDI 来源释放
触摸键抖动或多指缺失	浏览器 pointer 事件、触控硬件、多指取消	确保 `pointerdown/up/cancel` 成对，禁用浏览器手势冲突，保留 `panic` 回收

现象	优先检查	处理原则
悬挂音符或踏板	`note_off`、CC64、`release_source`、断线清理	执行 panic；修复事件边界，不能靠高频轮询重建短音符
开发板没有 IPv4	路由器 DHCP、网线、接口状态、地址冲突	先恢复网络基础设施；不要修改应用代码来掩盖路由器故障
NPU 显示 Health Alarm	NPU 是否可见、真实 OM 是否能加载和推理、CANN 日志	保留警告；真实推理成功时不自动阻断，失败时保存诊断
模型资产验证失败	发布版本、模型清单、暂存文件和同步完整性	重新下载损坏的暂存文件；不能跳过校验或修改清单以迎合错误文件
摄像头或音箱中途断开	固定设备 ID、PulseAudio source/sink、线程退出和资源锁	立即停止并释放资源；重新枚举后由用户显式选择，禁止静默改路
蓝牙设备可见但不可播放	`bluetoothctl`、PulseAudio A2DP sink、连接 profile	使用板端已有设施；缺少系统组件时报告，不在部署中安装

更多板端日志、ATC/OOM、音频和兼容性案例见测试故障排查记录与音频输出说明。

3.11.3. 测试矩阵

层级	工具	主要覆盖
Python 单元与 API	pytest	MIDI 状态边沿、最小门长、资源冲突、索引重建、模型后端边界、监视源拒绝、参数边界和线程清理
React 组件	Vitest	加载、不可用、运行、故障、安全静音、触摸事件和音色中文显示
页面回归	Playwright	多类视口、顶层导航、实时演奏双输入模式、Canvas 非空、无溢出、快速触摸和 MIDI-DDSP 单卷帘
板端模型	PyACL 与参考 NPZ	张量合同、1,000/10,000 帧精度、p95/p99 和 NaN/Inf
板端音频	真实双工与听音	PCM 非零、路由、总延迟、overflow、underrun、clipping 和设备断开

本地测试不得运行 PyACL、ATC、OM 推理或 `npu-smi`；这些结果必须来自真实 Ascend 310B。

3.12. 系统分层、总结与参考资料

3.12.1. 软件分层

层次	核心职责	与相邻层的边界
交互层	呈现演奏、渲染、音色转换和设备状态	只提交用户意图，不持有模型文件或物理设备路径
服务与协调层	校验请求、管理作业、同步事件并仲裁共享资源	不直接实现神经网络或音频合成算法
神经推理层	维护模型输入输出和循环状态，调用 OM 产生控制量	不承担高采样率波形生成
信号处理层	完成谐波、噪声、混响、重采样、缓冲与设备输出	不决定模型或界面状态
实验记录层	关联输入、配置、音频产物和评价结果	不参与实时控制路径

这种分层对应不同的时间尺度：交互事件要求及时响应，神经控制按固定帧率递推，音频设备持续消费采样块，离线作业则以完整乐曲为单位。将这些时间尺度分离，是在同一设备上维持实时性与可复现性的基础。

3.12.2. 数据与控制边界

系统的数据流遵循单向责任关系：浏览器表达操作意图，后端确定模型与设备资源，OM 预测低维声学控制量，CPU DSP 将控制量转换为波形，实验记录层保存可复核的输入与结果。状态流则沿相反方向返回：音频线程和模型运行时产生指标，服务端汇总为快照与事件，界面据此更新显示。

这一边界有两个方法论意义。其一，模型推理耗时与端到端音频延迟可以分别测量，避免把缓冲或无线传输延迟归因于 NPU；其二，界面刷新、设备切换和文件索引不会直接改变正在执行的神经状态，从而降低交互层扰动实时链路的风险。

3.12.3. 继续阅读

- Case3 代码总览
- 系统分层设计
- WebUI、操作与 API
- Piano-DDSP 模型与实时合同
- MIDI-DDSP 与 DDSP-VST 对比
- MIDI-DDSP 实时与离线边界
- 模型与 OM 部署

- 音频输出与设备边界
- 测试故障排查

3.12.4. 总结

本案例展示了三类 DDSP 模型在边缘设备上的不同研究路径。Piano-DDSP 由作者基于 PyTorch 实现并在 MAESTRO 数据上训练，通过显式循环状态支持低延迟复音演奏；MIDI-DDSP 与 DDSP-VST 则从 Google 项目的 TFLite 模型出发，在保持层级上下文和流式状态语义的前提下迁移到 ONNX 与 OM。三者共享 “神经网络预测控制量、宿主 DSP 生成波形” 的基本范式，却分别对应因果复音控制、完整乐曲建模和实时单音转换三种时间问题。

案例的普遍意义不在于把若干模型集中到同一界面，而在于建立清晰的模型来源、状态边界和评价层次。只有将图语义一致性、NPU 推理预算、CPU 音频处理、设备路由与人机交互分别建模并联合验证，才能对实时数字乐器的性能与局限作出可靠判断。

3.12.5. 参考资料

4. 案例 4：掌纹识别

4.1. 案例目标

本案例在 Ascend 310B4 / 8T 开发板上运行 FastAPI + React 掌纹识别工作台。生产推理固定使用 NPU `mixed_fp16`；CPU、EDCC、origin 和离线候选比较不进入在线接口。

本版本是冻结的人工测试发布。CCNet 为默认/回滚模型；`manual_test` profile 可逐一测试五个 CompNet。五个 CompNet 的状态为 `manual_test_pending`，不能把人工测试前的状态写成稳定验收通过。

4.2. 手动环境

在开发板同一个 shell 中执行：

```
source /usr/local/miniconda3/etc/profile.d/conda.sh
conda activate base
source /usr/local/Ascend/ascend-toolkit/set_env.sh
cd <release-root>
export PALMPRINT_ROOT="$PWD"
export ACL_PYTHON_SITE=/usr/local/Ascend/ascend-toolkit/latest/python/site-packages
export PYTHONPATH="$PALMPRINT_ROOT:$ACL_PYTHON_SITE${PYTHONPATH:+:$PYTHONPATH}"
export PALMPRINT_PROFILE=manual_test
unset PALMPRINT_REQUIRE_TEMPLATE_ENCRYPTION
unset PALMPRINT_TEMPLATE_KEY_FILE
unset PALMPRINT_TEMPLATE_DIR
python -c "import sys, acl; print(sys.executable); print(acl.__file__)"
python -m palmprint_workbench.tools.verify_assets --strict
python -m palmprint_workbench.api --host 0.0.0.0 --port 7860
```

源码不会自动安装依赖、下载数据、构建 EDCC、转换模型或修改系统环境。OM 从固定来源取得后，必须按 `om_manifest.json` 核对字节数和 SHA-256。

4.3. API 和界面

启动后检查 `/api/health`、`/api/bootstrap`、`/api/candidates` 和 `/`。manual_test profile 的 bootstrap 应返回 CCNet 与五个 canonical CompNet，均为 `npu/mixed_fp16`。

触摸屏逐页检查：系统状态、上传识别、模板注册、候选审计。测试 1280×720、1024×768 和 375×812；按钮不重叠，状态文本不溢出，API 不可用时显示空服务状态。

4.4. 六模型人工测试

canonical ID 为：

- `ccnet`
- `compnet_tongji_600`
- `compnet_iitd_460`
- `compnet_rest_358`
- `compnet_xjtu_flash_200`
- `compnet_xjtu_natural_200`

每个模型执行：上传合成 ROI 或匿名测试图、识别、注册三个模板、查询、命中识别、删除、重启后再查询。模板 namespace 固定为 `<model_id>__npu__mixed_fp16`。

每次记录模型 ID、OM SHA、请求时间、HTTP 状态、模型耗时、模板状态和服务 PID。当前内测版本会把用户主动触发的原图、ROI 和 JSON 索引自动保存到发行目录的 `data/captures/`，把明文 `PWST1` embedding 模板保存到 `data/templates/`；连续预览帧不保存。删除模板不会删除图片证据。真实掌纹图像、姓名、embedding 和完整报告不得进入源码或截图，也不得同步到 GitHub/HF。

查看和清理内测图片由操作者手动执行：

```
find "$PALMPRINT_ROOT/data/captures" -type f
rm -rf "$PALMPRINT_ROOT/data/captures"
rm -rf "$PALMPRINT_ROOT/data/templates"
```

4.5. 摄像头测试

先检查 `/api/cameras`，优先选择 1280×720 MJPG。连续预览 100 帧，记录首帧时间、稳定帧间隔和实际输出尺寸；拍照识别后关闭预览，确认设备释放，再重新打开一次。若画面滞后，先降低预览分辨率并确认没有其他进程占用设备。

4.6. 常见问题

现象	处理
`import acl` 失败	重新加载同一 shell 的 conda/CANN，确认 Python 和 `acl.__file__` 来自同一环境
模型不在列表	确认 `PALMPRINT_PROFILE=manual_test`、registry、OM SHA 和 bootstrap
模板不存在	确认注册和识别使用同一模型、精度和当前发行目录的 `data/templates`
模板文件损坏	停止服务，备份 `data/templates`，再使用 `.pstore.bak/.previous` 恢复
摄像头卡顿	检查 MJPG、实际分辨率和设备占用，先使用 960×540 或 1280×720
服务健康但识别失败	执行真实上传或摄像头 smoke；health 不主动加载 NPU
ACL 异常退出、设备 reset 或进程意外退出	停止测试，保存时间/PID/模型/摘要，重启当前版本或回退 CCNet；不要升级版本
`Health: Alarm`	只记录设备状态，不单独判断模型或程序结果

4.7. 版本冻结

人工测试期间不升级 Python、依赖、驱动、CANN 或应用版本。发现问题时停止当前模型、记录复现步骤、回退 CCNet，并在人工测试结束后单独规划下一版本。人工测试通过后，再更新准入证据、release manifest、版本号和完整验收记录。

4.8. 全候选模型测试

工作台运行通道只保留六个已有 OM 且契约可验证的 NPU embedding。候选清单仍保留完整 28 项审计记录；其余 22 个分类器、ROI、掌静脉、EDCC、SDK 或缺少可复现资产的项目已从 API/UI 候选矩阵归档，不会被服务加载，也不能被强制转换成掌纹 512 维 embedding。

在源码环境中执行静态审计：

```
python -m tools.offline.candidate_campaign inventory --all
python -m tools.offline.candidate_campaign local --all
python -m tools.offline.candidate_campaign report --all
```

当前板端地址是 `192.168.8.178`。在板端激活 conda/CANN 后，对已有 OM 的候选逐一执行：

```
python tools/board/collect_npu_trace.py --label <candidate-id> --interval 1 -- \
  python -m tools.offline.candidate_campaign board \
```

```
  --candidate <candidate-id> --image <roi-image>
python tools/board/acl_lifecycle_probe.py --model <candidate-id> \
  --image <roi-image> --cycles 10
```

每项记录来源 revision、许可、资产字节数/SHA、输入输出契约、实际 backend、耗时、退出码、温度、功耗、内存、大页、dmesg 和生命周期。`139`、`err_ret=-512`、设备 reset、LPM/AICore/RAS 增量或清理失败会阻断该项；缺权重、不可复现来源或闭源 SDK 则记录为 `blocked_*` 或 `not_applicable`，不是 “测试通过”。完整日志留在板端私有目录，源码只提交脱敏状态和哈希。

5. 案例 5：智能数据采集分析仪

示波器把连续电压画成波形，却没有回答 “这一段信号在什么时间、什么频率出现了什么现象”。这个问题把一个看似简单的采集任务分成四层：可靠地取得样本，把样本组织成可解释的窗口，从窗口构造频域或时频表示，再把结果与原始证据和实际计算后端绑定起来。

本章围绕两种物理信号学习这一方法。Hantek 路线以 CH1 的 CAL 方波为已验证实例，用固定 Hann 窗 DFT 说明确定性信号处理如何部署到 NPU；Little Bee B1 的双通道电流测量属于尚待标定的扩展任务。RTL-SDR 路线把连续复数 IQ 变成时频图，再由 YOLO11s 检测网络形成候选区域。两条路线都运行在 Ascend 310B 上，但它们不是同一种 “模型”：固定 DFT 是数值计算图，YOLO11s 才是经过训练的神经网络。

本章采用 “核心实验加扩展实验” 的组织方式，而不是要求读者一次完成全部硬件操作。Hantek 路线是核心实验，读者先学习第 3、4 节，再完成第 6 至 8 节；RTL-SDR 路线是扩展实验，在此前基础上学习第 5 节和第 9、10 节。涉及 CANN、ATC、ACL、OM、USB 和 RTL-SDR 的命令只能在真实 310B 开发板上验证，本地开发机只承担代码和数值参考检查。

路线	教学定位	可以确认的结论
Hantek 固定 DFT	核心实验	CAL 连通、两通道数据帧合同、固定窗口、真实 OM 频谱和会话追溯
RTL-SDR 时频检测	扩展实验	CU8 采集、时频预处理、OM 张量合同和主机流水线预算；不包括调制类别准确率

5.1. 从波形到可解释事件

5.1.1. 本章要回答的学习问题

设采集系统在一段时间内获得样本 $x[n]$。仅显示 $x[n]$ 时，读者只能观察幅值随时间变化；若希望说明 “何时出现了某一频率成分” 或 “哪一段频带包含一个候选信号”，还必须定义分析窗口、频率坐标、模型输入和结果回溯关系。本章用下列问题组织学习：

1. 采样率与窗口长度怎样同时限制时间尺度和频率分辨率?

2. 为什么实值波形适合单边频谱，而复数 IQ 必须保留正、负频移?
3. 固定 DFT 和训练得到的目标检测网络分别解决什么问题?
4. 一个时频检测框怎样还原成时间区间和名义频带?
5. 为什么 “OM 可以运行” 仍不能替代模型准确率、测量标定或连续实时性证据?

完成本章后，读者应能用自己的话说明输入窗口、CPU/NPU 分工、模型输出、实时预算和证据边界；能完成一条安全的 Hantek 固定 DFT 实验，并能在已有 `accepted` manifest 的开发板上开展 RTL-SDR 扩展实验。

5.1.2. 证据等级与实验边界

教学实验中的结论必须与证据相称。当前 Hantek 记录证明了 CH1 的 `CAL` 方波连通与固定 DFT OM 界面；Little Bee 的真实 CH2 接入、幅相标定和双通道测量验收仍待完成。RTL-SDR 记录证明了接天线条件下的连续采集、时频预处理与 NPU 推理流水线；无标签真实 IQ 不能推出调制识别准确率。

结论类型	本章允许的证据	本章不能据此声称的结论
采集连通	`CAL` 波形、USB 诊断、原始帧或 CU8 归档	幅度、带宽、相位或频率校准
NPU 执行	`aclruntime` 实际加载 OM、后端状态和推理时延	CPU 结果等价于 NPU 结果
流水线实时性	完整窗口、零丢批、逐阶段时延和连续运行记录	模型准确率或射频测量校准
检测性能	有来源的标签、独立验证集与指标	仅凭无标签检测框断言准确率

本地开发电脑用于 `pytest`、ONNX 图和 NumPy 参考；Ascend 310B 才用于 ATC、OM、真实采集和 NPU 延迟；Hantek/Little Bee 与 RTL-SDR 才提供真实模拟波形或 IQ。这个边界不是流程负担，而是防止把模拟测试写成硬件验收的前提。

5.2. 实验器材、接线与安全

5.2.1. Hantek 采集与扩展测量

器材	本案例中的用途
昇腾 310B 开发板	采集控制、CPU 预处理、OM 推理和触摸显示
Hantek 6022BE	提供两个物理采集通道；本章已验证 CH1 的 `CAL` 连通
Hantek 电压探头	连接 CH1，首轮实验接 6022BE 的 `CAL` 输出

器材	本案例中的用途
Little Bee B1	后续双通道扩展所用电流探头；不纳入本章当前验收
SMA-BNC 连接线	把 Little Bee 输出连接到 CH2
经确认隔离的教学回路	后续双通道扩展实验的被测对象

Hantek 硬件与接线说明记录了 6022BE 两个通道共地的边界，不应把它当成隔离示波器。Little Bee 的操作说明也明确限定其为实验室探头，不具备 CAT 测量等级，不得夹持未绝缘导线。[1] 因此，本章的第一次真实采集只使用 6022BE 自带的 `CAL` 输出；接入 Little Bee 前，必须由设备手册、课程实验规范或负责教师确认被测回路的隔离和允许测量范围。本章不以“低压”一词替代这一确认，也不把磁耦合或电池供电误解为任意共模电压下的隔离保证。

> **安全边界：**未确认探头量程、接地、隔离、带宽和校准参数之前，禁止接入市电、裸露导体或打开后的市电设备。软件中的量程选择不能弥补错误接线带来的电气风险。

这张图把两条硬件路线放在同一页中，目的是先把接线关系和证据边界分开：左侧的 `CAL -> CH1` 是最小连通检查；Little Bee 只允许接入已确认隔离且适合该探头的单根绝缘导线，并由 CH2 接收；右侧的中心频率、增益和 PPM 只是 RTL-SDR 的运行配置。

图 5-1 两条实验路线的硬件连接与安全边界。这是教学结构图，不是实机验收照片；它特别标出 6022BE 的共地、非隔离输入，以及 `CAL`、接收配置分别能够和不能够证明什么。图源见 `img5/case5-hardware-connections.dot`。

下图来自探头接触 6022BE `CAL` 标准测试输出。它证明 PulseView、sigrok 驱动、USB 和 CH1 探头能够获得真实方波，但不能证明幅值准确、带宽达标、CH1/CH2 同步或 NPU 已经通过验收。

图 5-2 `CAL` 连通时的 PulseView 界面示例；CH2 当时没有作为已标定电流通道使用。该图只说明最小采集基线，不能替代会话记录或计量标定。

5.2.2. RTL-SDR 实验

RTL-SDR 主线至少需要一只受 `rtl_sdr` 工具支持的接收机，以及与实验频段匹配的天线或受控线缆。当前板端记录使用 RTL-SDR Blog V4，但程序的核心合同是 CU8 复数 IQ，并不把部署包绑定到某一接收机销售型号。

真实运行必须如实记录射频输入状态：

- `antenna_connected` 表示已连接天线；
- `lab_cabled` 表示连接到受控实验线缆；
- `disconnected` 表示没有接入射频输入；

[1] Weston Braun. “Little Bee B1 Operating Instructions.” 固定审查提交：9157c50a5d6e。

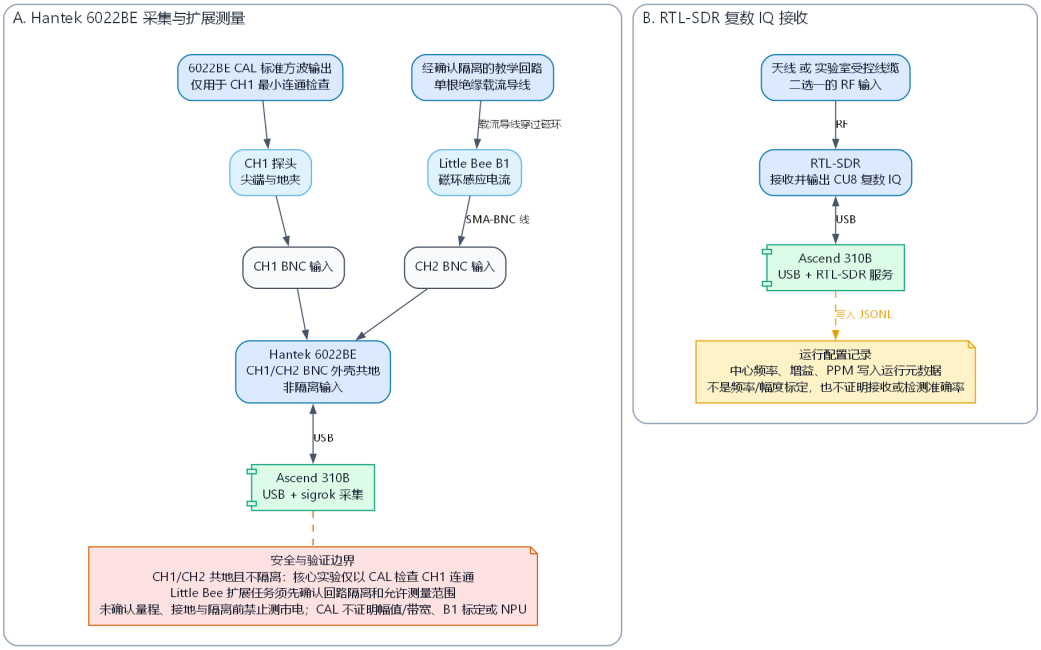


图 5.1. Hantek 采集与 RTL-SDR 连接和安全边界

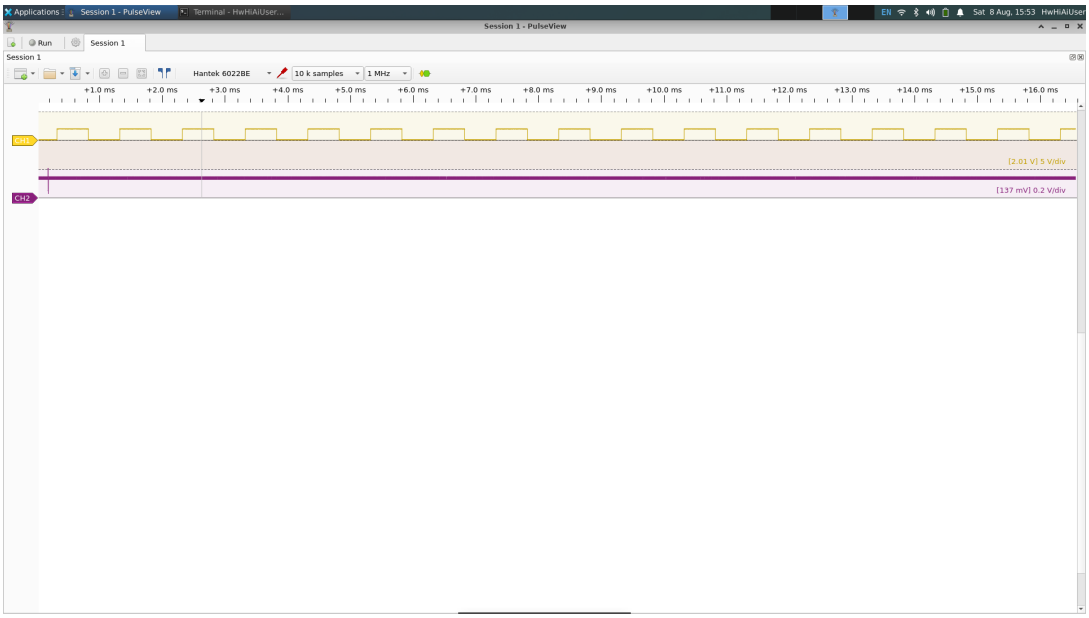


图 5.2. Hantek 6022BE 在 PulseView 中采集 CAL 方波

- `unknown` 只能表示状态未确认，不能作为硬件验收声明。

中心频率、增益和 PPM 都是接收配置，不是校准结论。某个地点在 100 MHz 使用 40.2 dB 增益没有明显字节级削顶，并不意味着其他天线、频率和环境也应采用相同增益。

5.3. 读懂时域、频域和复数 IQ

5.3.1. 从一段波形得到什么

设一个窗口包含 N 个样本 $x[n]$。均值描述波形的直流偏置：

$$\bar{x} = \frac{1}{N}\sum_{n=0}^{N-1} x[n]$$

原始均方根值描述包含直流分量的窗口有效幅值：

$$x_{\mathrm{RMS}} = \sqrt{\frac{1}{N}\sum_{n=0}^{N-1} x^2[n]}$$

峰峰值是最大值与最小值之差：

$$x_{\mathrm{pp}} = \max(x) - \min(x)$$

若关心去直流后的交流幅值，应使用

$$x_{\mathrm{AC,RMS}} = \sqrt{\frac{1}{N}\sum_{n=0}^{N-1} \left(x[n] - \bar{x}\right)^2}.$$

均值可用于观察偏置，原始 RMS 适合比较窗口整体幅值，AC RMS 则对应去直流后的交流成分，峰峰值对脉冲和削顶更敏感。当前 `CaptureProcessor` 已经对最新采集块计算每路的原始 `mean`、`rms` 和 `peak_to_peak`，而送入 OM 的完整分析窗口会另行去直流。Hantek 工作区和 `analysis.jsonl` 尚未持久化这些统计量，因此本章不把它们列为已经完成的端到端功能。

5.3.2. 窗口长度决定观察尺度

采样率为 f_s、窗口包含 N 点时，窗口时间跨度和 DFT 频率间隔分别为：

$$T = \frac{N}{f_s}, \qquad \Delta f = \frac{f_s}{N}$$

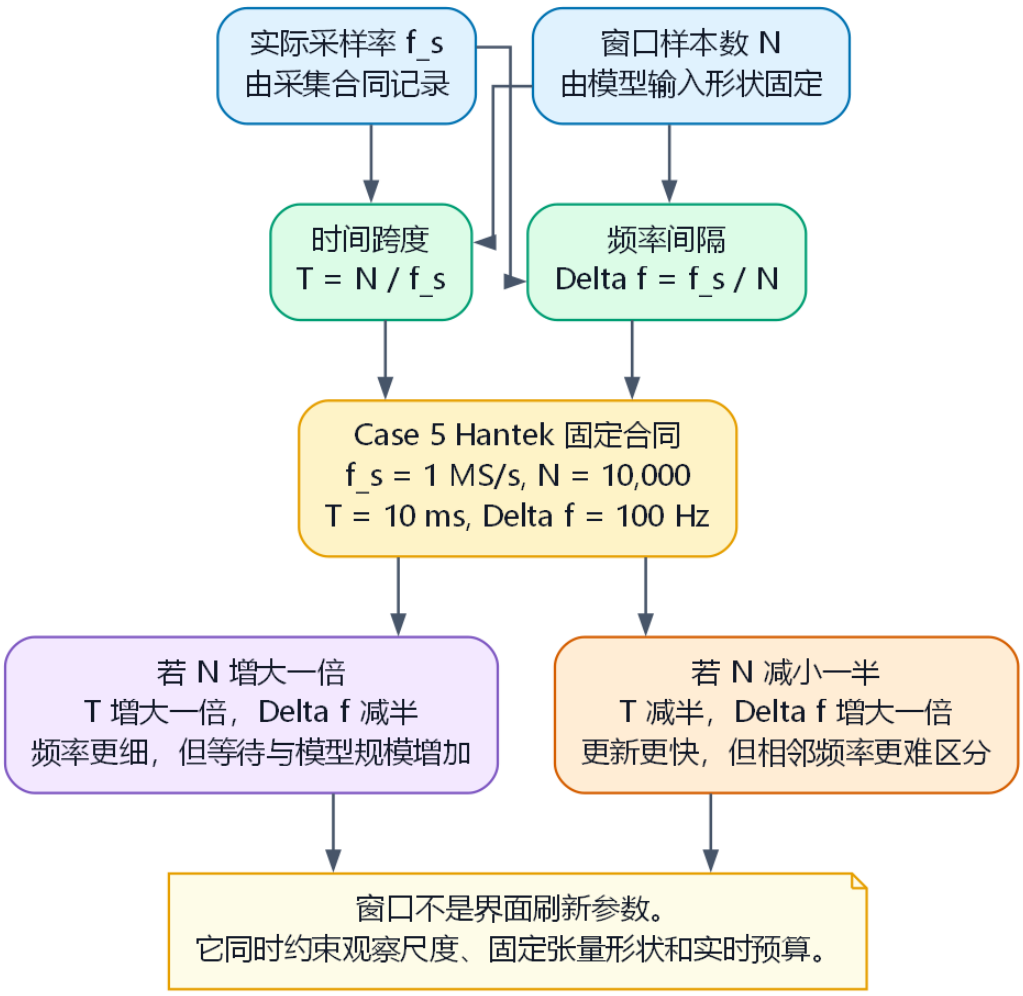


图 5.3. 采样率、窗口长度、时间跨度和频率分辨率的关系

Hantek 主线固定为由驱动读回并与 OM 合同一致的 $f_s = 1\,000\,000$ S/s、$N = 10\,000$，因此每个窗口覆盖 10 ms，频率间隔为 100 Hz。模型只保留 0 到 20 kHz，共 201 个频点。第 11 节的“有效速率”是主机收到回调的吞吐量，不是频率轴所用的采样率，不能用它替换 f_s。

这个例子揭示了一个常见取舍：扩大窗口可以减小 Δf，让相邻频率更容易分开，但也会增加单次分析等待时间和固定模型输入规模。缩短窗口可以更快更新界面，却会降低频率分辨率。窗口参数不能只由界面刷新速度决定，还必须与模型形状和实时预算一致。

这张图把公式变成一个可操作的设计关系：在采样率固定时，窗口点数同时决定等待时间和频率 bin 的间隔；把点数翻倍不会免费增加信息，而是把每次分析所需的样本和时间一并翻倍。

图 5-3 采样窗口的尺度取舍。Hantek 当前合同把 $f_s = 1$ MS/s 和 $N = 10\,000$ 固定为 10 ms 与 100 Hz；改变任一量都要求重新审查窗口预算和模型合同。图源见 `img5/case5-sampling-window-resolution.dot`。

5.3.3. 为什么要去直流和加窗

Hantek 数据在进入 OM 前先执行逐通道去直流：

$$x'[n] = x[n] - \bar{x}$$

OM 内部再把 $x'[n]$ 与固定 Hann 窗相乘，并投影到正弦、余弦基函数。对第 k 个频点，可以把计算理解为：

$$X[k] = \sum_{n=0}^{N-1} x'[n] w[n] e^{-j2\pi kn/N}, \qquad P[k] = |X[k]|^2$$

加窗不是为了“让频谱更漂亮”，而是为了减小有限窗口边界不连续造成的频谱泄漏。Hantek 固定 DFT 使用 Hann 窗；正式 RTL-SDR 频谱检测模型则使用 manifest 锁定的 FFTW Blackman 时频预处理。二者服务于不同模型合同，不能随意互换。

Hantek OM 返回线性功率。显示层才进行如下转换：

$$P_{\mathrm{dB}} = 10 \log_{10} \left(\frac{\max(P, 10^{-12})}{1\ \mathrm{V}^2} \right)$$

界面将它标注为“相对 $1\ \mathrm{V}^2$，未校准”。它不是 dBm、dBFS，也不应在未完成测量链标定时称为标准 dBV。

5.3.4. 实值波形与复数 IQ

Hantek 的每个通道都是实值时间序列。对实值信号，正频率和负频率互为共轭，因此仪表盘只显示单边频谱。

RTL-SDR 输出的是交错 CU8：一个字节表示 I，下一个字节表示 Q。程序按模型合同把它转换为近似位于 $[-1, 1]$ 的复数样本：

$$z[n] = \frac{I_{u8}[n] - 127.5}{127.5} + j \frac{Q_{u8}[n] - 127.5}{127.5}$$

复数 IQ 保留了相对于中心频率的正、负频移。旧固定 DFT 教学模型因此输出从负频率到正频率的完整频谱；正式检测模型则把一段连续 IQ 构造成时频图，再交给 NPU 检测器。

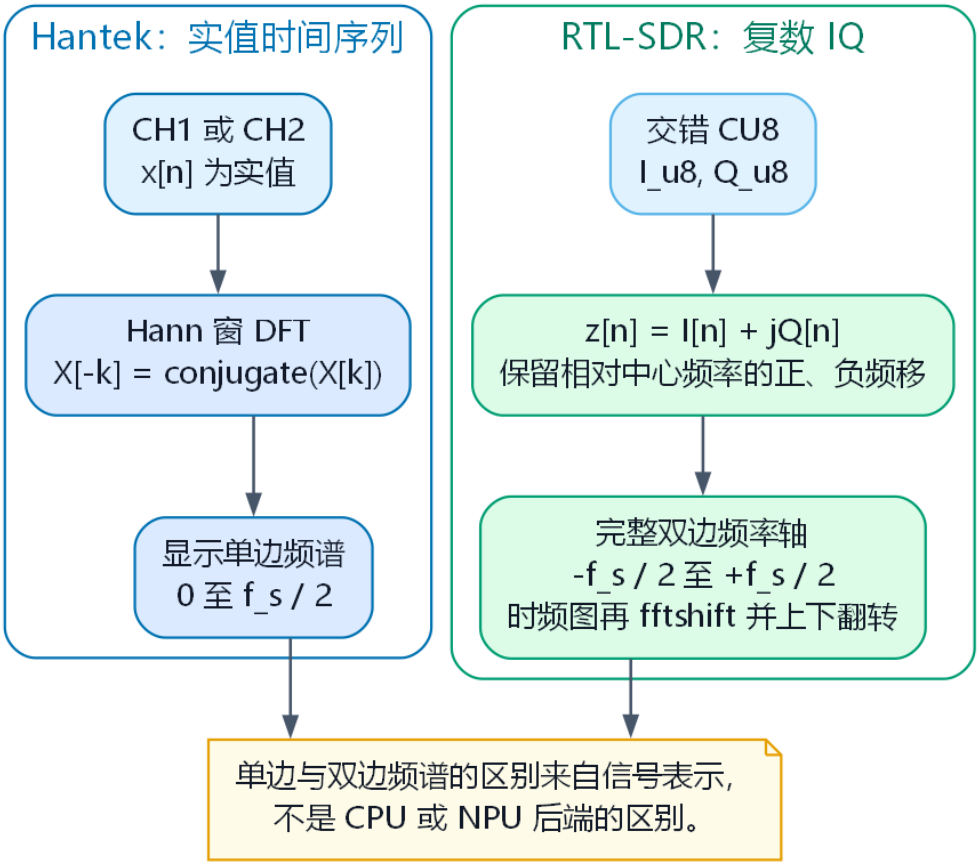


图 5.4. 实值波形的单边频谱与复数 IQ 的双边频谱对照

这张对照图解决“为什么两类谱图的横轴范围不同”的问题。单边或双边显示首先由信号表示决定，不是 CPU 与 NPU 的性能选择；Hantek 与 RTL-SDR 的窗函数也分别由各自的模型合同锁定。

图 5-4 实值与复数 IQ 的频谱表示。实值信号的负频率由共轭对称性决定；复数 IQ 保留中心频率两侧的独立频移。图源见 `img5/case5-real-vs-iq-spectrum.dot`。

这张图解决“像素与物理量怎样对应”的问题。它把一个模型图像还原为连续 IQ、时间行和频率 bin；图中的箭头是数据变换，不是神经网络层。

图 5-5 IQ 到时频图的映射。2.048 MS/s 下，1024 点的一行覆盖 0.5 ms，频率 bin 间隔为 2 kHz；纵轴经过 `fftshift` *和上下翻转，图像顶端对应中心频率上方约* $f_s/2$*。这是当前实现的名义频率坐标，尚不构成射频频率标定。图源见* `img5/case5-iq-to-spectrogram-mapping.dot`。

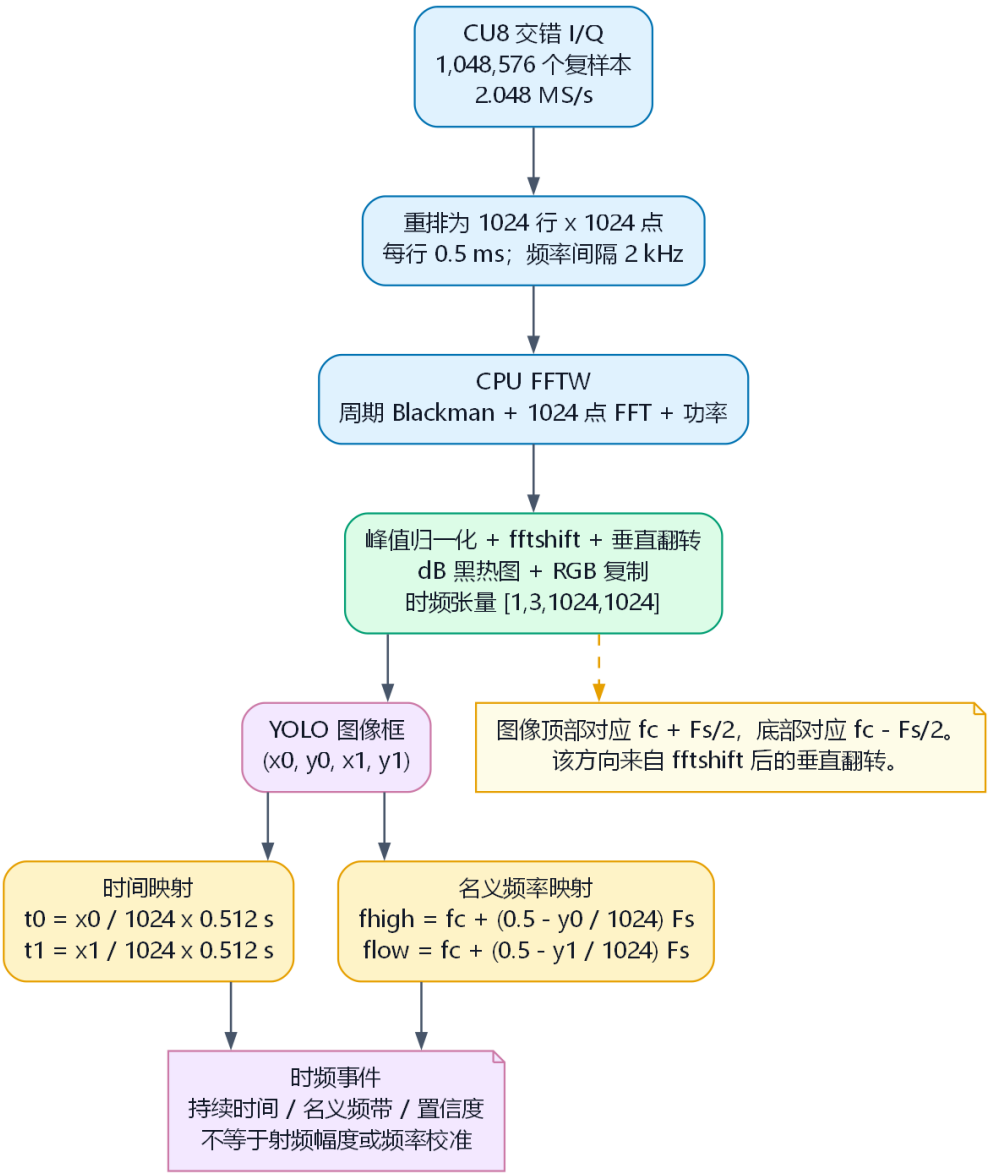


图 5.5. RTL-SDR IQ 到 1024 乘 1024 时频图和物理坐标的映射

5.4. 双主线系统架构

5.4.1. 先划清两种 OM 的计算边界

这张图解决 “同样部署为 OM 是否就同样属于人工智能” 的问题。两条主线都把固定形状张量交给 Ascend 310B，但 Hantek 的图只实现预先写死的 DFT 投影；RTL-SDR 的图才包含由训练确定的卷积、特征融合和检测参数。

图 5-6 两种 NPU 任务的计算边界。固定 DFT 是确定性数值分析，不含训练得到的可学习参数；YOLO11s 是学习模型。两条路径都要求 OM 真正运行：初始化不可用时显式报告 `NPU unavailable`*，运行期错误显式失败且不以 CPU 结果替代。图源见* `img5/case5-computation-boundary.dot`。

NPU 是执行位置，不是算法类别。把固定 DFT 编译为 ONNX/OM 的目的，是练习静态张量合同、数值校验和后端追溯；它并不使 DFT 变成神经网络。相反，YOLO11s 的卷积权重和检测头参数来自上游训练过程，即使在部署阶段它们同样固定，也必须单独评价其任务语义、数据分布和精度证据。

Hantek 路径首先把具体驱动返回值转换为公共 `BridgeFrameV1`。从这一边界开始，处理、NPU、存储和界面都不需要了解 `hantek-6xxx` 的私有结构。未来更换示波器时，只需要在 `time_frequency_dashboard/acquisition/` 中增加适配器，并继续输出相同帧合同。

RTL-SDR 路径不复用双通道电压帧，因为它面对的是复数 IQ 和第三方模型合同。它使用 `CapturedIqBatch` 保留批次序号、原始样本偏移、主机接收时间和分阶段耗时，再由 `RtlSdrService` 统一服务 CLI 和 Qt 工作区。

5.4.2. 三种模型合同不能混用

路径	输入	核心 NPU 任务	输出与用途
Hantek 主线	`[1,2,10000]` float32	Hann 窗固定实值 DFT	`[1,2,201,1]` 线性频谱功率
RTL 固定 DFT 教学	`[16,2,1024]` float32 I/Q	Hann 窗固定复数 DFT	`[16,1024]` 完整频谱功率
RTL 正式检测	`[1,3,1024,1024]` float32 图像	TorchSig YOLO11 时频目标检测	`output0 [1,55,21504]` 的框参数与评分

第一种合同驱动 Hantek 频谱与瀑布。第二种合同只用于理解复数 IQ、验证 NPU DFT 和比较 CPU/NPU 算法，不被正式 SDR 工作区调用。第三种合同才是当前 RTL-SDR 正式界面的检测路径，其中 CPU FFTW 生成模型规定的 Blackman 时频输入，NPU 执行神经网络检测。它的 51 个评分通道来自部署合同；该 checkpoint 的训练记录采用单类监督，因此这些通道不能被解释为 51 种调制类别，不能由张量维度或 UI 标签表自行推出。

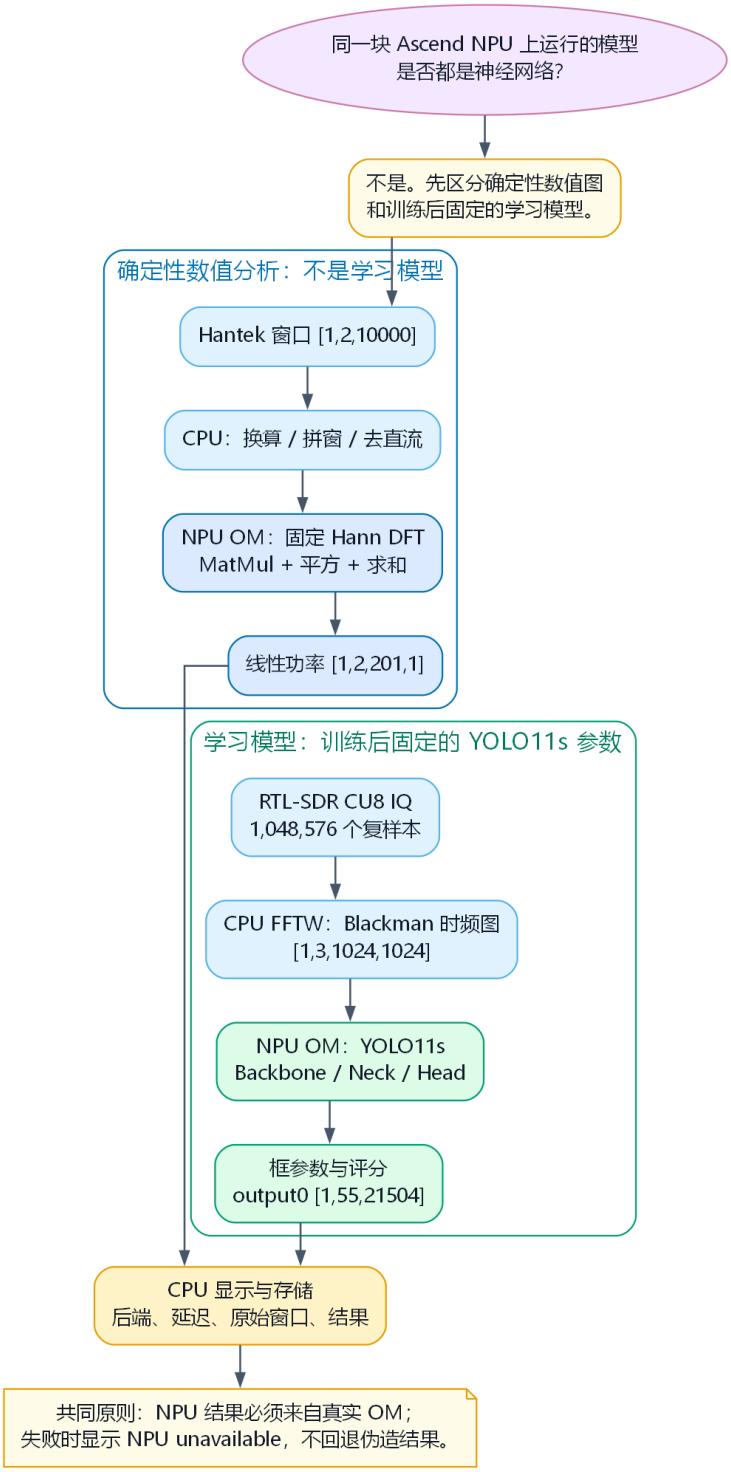


图 5.6. Case 5 固定 DFT 与 YOLO11s 的计算边界

5.4.3. 实时系统为什么需要有界队列

采集、NPU 和绘图不会永远保持相同速度。如果输入队列无限增长，短暂的推理或界面卡顿最终会耗尽内存，而且读者看到的仍是几秒前的旧结果。

Case 5 的分析输入队列默认容量为 2。队列满时，`LatestQueue` 丢弃最旧任务并保留最新窗口；结果队列和瀑布历史同样有固定上限。丢弃计数会进入状态和会话记录。这个策略适合实时观察，但不等于原始数据也可以无声丢失：原始帧由独立的有界会话写入器保存，并单独记录存储丢帧和写入错误。

`InstrumentCoordinator` 还保证 Hantek 与 RTL-SDR 不会同时占用 NPU 和设备控制路径。切换页面不会自动打开硬件；旧来源、队列和 NPU runner 完全停止后，另一条链才能启动。

5.5. 从时频图到神经网络检测

本节讨论的不是 “怎样运行一个 OM 文件”，而是一个更基础的问题：模型看到的是什么、网络怎样把像素组织成候选事件、以及什么证据才足以支持模型结论。后面的板端命令只是在这条认识链完成后，把已经定义清楚的合同部署到 NPU。

5.5.1. 模型看到的是标准化的时频形态

正式 RTL-SDR 模型不直接读取原始 I/Q。设复数序列为 $z[n]$，Blackman 窗为 $w[n]$，每行长度为 $N = 1024$，当前实现不重叠地构造时频图：

$$X[m,k] = \sum_{n=0}^{N-1} z[n+mN]w[n]\exp\left(-j\frac{2\pi kn}{N}\right), \qquad P[m,k] = |X[m,k]|^2.$$

其中 m 是图像的时间行，k 是频率 bin。程序对功率执行 `fftshift`、上下翻转、dB 变换和**按单幅图自身最大值归一化**，再以黑热色标形成一个通道并复制为 RGB 三通道。因此，`[1,3,1024,1024]` 不是自然相机图像，而是一个具有固定轴向约定的张量合同。

在 $f_s = 2.048$ MS/s 下，这个合同可以直接换算为观察尺度：

$$T_{\text{row}} = \frac{N}{f_s} = 0.5\ \text{ms}, \qquad \Delta f = \frac{f_s}{N} = 2\ \text{kHz}, \qquad T_{\text{image}} = 1024T_{\text{row}} = 0.512\ \text{s}.$$

图 5-5 中，检测框的横坐标是该 0.512 s 批次内的时间比例；纵坐标则因图像坐标向下增长而反向映射到频率。对于像素框 (x_0, y_0, x_1, y_1)，界面使用

$$t_i = \frac{x_i}{1024} T_{\text{image}}, \qquad f_i = f_c + \left(0.5 - \frac{y_i}{1024}\right) f_s$$

把它还原为名义时间和频率范围。这个映射使检测结果可以与 CU8 批次、中心频率和采样率关联；但由于每幅图独立归一化、接收链增益未校准，它不能给出绝对接收功率，也不自动消除本振偏差或天线误差。

5.5.2. 先限定模型证据能够说明什么

本章选用经审查的 `gr-spectrumdetect` 候选作为部署对象。部署端不引入 PyTorch，只接收审查后的静态 ONNX、OM 和 manifest；来源版本、工件哈希、输入名和准入记录保存在本章的配套资料中。这里关注的是可复查的证据类型，而不是某一次构建的工件编号。

这里存在一项必须说明的训练语义限制：上游 README 与训练 YAML 将该权重描述为宽带射频能量区域检测，并采用 `single_cls=True`。Ultralytics 的这一路径会将训练数据的类别数收束为 1，验证时也按单一预测类别处理。虽然同一权重的网络合同仍保留 51 个评分通道，部署代码也为它们提供标签名称，但这些软件映射没有获得该 checkpoint 的 51 类监督语义。本章因此采用下表的证据表述。

论断	本章证据状态	可以得出的结论
固定输入可形成 `[1,3,1024,1024]`，OM 输出可读取	已由 manifest、数值校验和板端运行记录支持	预处理与 NPU 部署边界连通
输出有 51 个评分通道	已由 ONNX 静态形状 `[1,55,21504]` 与部署代码支持	只能确认 4 个框参数加 51 个评分的张量布局，不能由此推出类别数
模型将时频区域作为检测目标	有上游单类训练说明支持	可以把框视为候选区域；现有兼容解码的框数和通道索引仍需与单类后处理分开解释
51 项软件标签是此权重的调制类别	不受当前 `single_cls=True` 训练语义支持	不能把任何标签当作该 checkpoint 的多类识别；需要另一个多类监督训练的模型

上游材料将该实验权重描述为从 YOLO11s 出发、使用 Wideband level-2 损伤数据的检测训练；记录中还包含冻结第一层、SGD 学习率 10^{-4} 和单个 epoch 等运行参数。它们用于说明权重的来源和任务背景，不构成本项目对训练过程的独立复现。更重要的是，`single_cls=True` 改变了监督标签的语义，因此不能把 checkpoint 自报的检测指标改写为“51 类调制识别精度”。[2]

若要研究 51 类调制识别，首先需要获得或重新训练一个具有多类监督语义的模型，而不是只为当前 checkpoint 追加一次本地评测。随后才应建立带控制发射源、注入路径

[2]TorchDSP. “gr-spectrumdetect.” 固定审查版本：868cb381e1fdd7d13ad70ecaf271e5060c43308d. https://github.com/TorchDSP/gr-spectrumdetect/blob/868cb381e1fdd7d13ad70ecaf271e5060c43308d/README.md#training

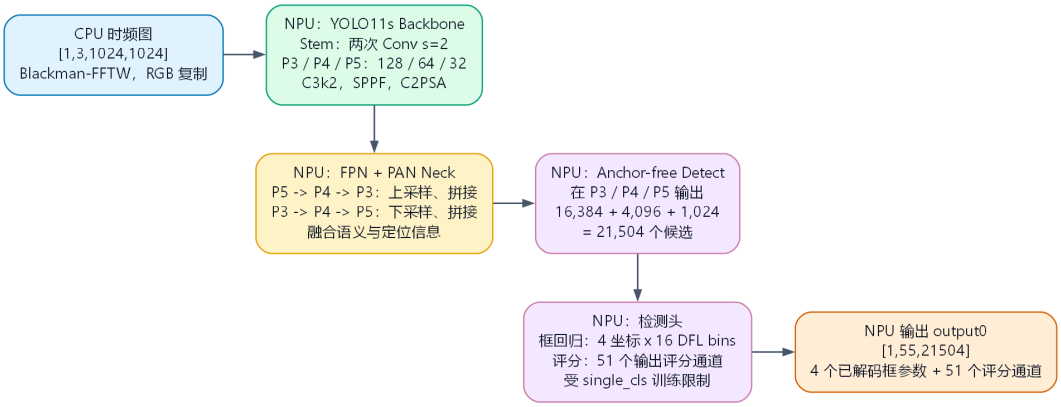


图 5.7. TorchSig YOLO11s 时频目标检测网络结构

或可靠标注的训练/验证/测试集，并按独立采集会话划分数据，报告召回率、精确率、IoU 阈值下的 mAP 以及混淆矩阵。本文记录的无标签真实 IQ 只用于评估采集、预处理、NPU 推理和实时预算，不能替代多类模型的任务精度实验。[3] [4]

5.5.3. YOLO11s 如何从图像形成候选框

这张图解决 “`[1,55,21504]` 从哪里来” 的问题。它是按输入输出合同和上游 YOLO11s 架构整理的教学拓扑，而不是由图像生成模型猜测的网络示意。候选模型的可执行合同见 `candidate_catalog.py`，工件准入与数值复核过程见配套资料 07。

图 5-7 当前 TorchSig YOLO11s 部署图的教学化结构。输入是 CPU 预处理后的时频图；Backbone 提取多尺度特征，FPN/PAN Neck 融合尺度，Detect Head 在 P3、P4、P5 上输出候选。图在 `output0` 处结束；阈值与 NMS 的 CPU 确定性后处理由图 5-8 单独说明。它不是逐节点的可执行 ONNX 报告，也不构成模型准确率证据。图源见 `img5/case5-yolo11s-architecture.dot`。

固定 DFT 的投影矩阵由公式预先确定；卷积网络的核 W 和偏置 b 则由训练数据优化。抽象地说，一个卷积层在位置 (i, j) 为输出通道 c_o 计算

$$Y_{c_o,i,j} = \sigma\left(b_{c_o} + \sum_{c_i,u,v} W_{c_o,c_i,u,v} X_{c_i,i+u,j+v}\right).$$

在时频图中，i 和 j 对应时间、频率的局部邻域。训练后的卷积核会响应某些局部纹理、边缘、带状区域或它们的组合；它并没有直接 “读到调制名称”。这也是为什么模型输出必须结合训练标签和独立实验解释。

[3] Luke Boegner et al. “Large Scale Radio Frequency Wideband Signal Detection & Recognition.” arXiv:2211.10335, 2022. https://doi.org/10.48550/arXiv.2211.10335

[4] TorchDSP. “gr-spectrumdetect.” 固定审查版本：868cb381e1fdd7d13ad70ecaf271e5060c43308d. https://github.com/TorchDSP/gr-spectrumdetect/blob/868cb381e1fdd7d13ad70ecaf271e5060c43308d/README.md#training

可以从输入到输出分五步阅读图 5-7：

1. **Stem 与 Backbone**。两个步长为 2 的卷积先把 1024×1024 图像压缩到 512×512、256×256；随后 `C3k2` 块提取局部模式。P3、P4、P5 的代表性特征图分别为 `[1,256,128,128]`、`[1,256,64,64]` 和 `[1,512,32,32]`。
2. **深层上下文**。P5 上的 `SPPF` 扩展感受野，`C2PSA` 在深层特征上引入注意力式上下文建模。它们处理的是频谱图形态，不是直接从 IQ 中估计相位或绝对功率。
3. **FPN/PAN Neck**。自顶向下的上采样和拼接把高层语义传给较细的 P3/P4 特征；自底向上的路径再把定位信息传回较粗尺度。这样持续很短、带宽很窄的痕迹和持续较长、带宽较宽的痕迹都有对应的候选尺度。
4. **三尺度 Detect Head**。三个输出网格的候选数为 $128^2 + 64^2 + 32^2 = 21\,504$。因此，输出的最后一个维度不是 21,504 个时间样本，而是 21,504 个候选位置。
5. **DFL 框解码**。DFL 用离散分布的期望解码连续框参数。当前导出合同的回归分支为 $4 \times 16 = 64$ 个离散值，即 `reg_max=16`；它随后与 51 个评分通道拼接，形成已解码的 $4 + 51 = 55$ 个输出值，而不是原始的 115 个头部值。前述固定数值来自当前导出合同，DFL 的一般原理见文献[5]；对当前单类监督 checkpoint，这些通道不能被解释为经过训练的 51 类语义。

下表把图中的模块名称与它们在本实验里可观察的职责对应起来。它解释结构，并不把模块名当作性能结论。

模块	在张量上的作用	对时频图任务的意义
步长 2 卷积	降低空间分辨率并增加通道数	逐步把局部时间–频率纹理汇聚为更大感受野的特征
`C3k2`	在通道变换与瓶颈路径间组织局部特征	在不同分辨率上保留可用于定位的局部带状或突发形态
`SPPF`	用池化组合扩展深层上下文	为持续时间长或带宽宽的候选补充更大范围的上下文
`C2PSA`	在深层特征上建立注意力式关系	让相隔较远的时频结构可以共同参与深层表示；不等于获得了绝对功率或物理因果关系
FPN/PAN	自顶向下融合语义，自底向上回传定位信息	让 P3、P4、P5 同时覆盖不同尺度的时频区域
Detect + DFL	分别形成评分与离散框分布，再解码为连续坐标	产生候选框参数和评分，随后由确定性后处理筛选

训练阶段通常把定位、类别和分布回归误差组成目标函数，可抽象写成

$$\mathcal{L} = \lambda_{\text{box}}\mathcal{L}_{\text{box}} + \lambda_{\text{cls}}\mathcal{L}_{\text{cls}} + \lambda_{\text{dfl}}\mathcal{L}_{\text{dfl}}.$$

[5]Xiang Li et al. “Generalized Focal Loss: Learning Qualified and Distributed Bounding Boxes for Dense Object Detection.” NeurIPS 2020. https://doi.org/10.48550/arXiv.2006.04388

这个公式说明检测网络为何同时学习 “框在哪里” 和 “该框的评分是什么”；它是 YOLO 检测训练的概念说明，不是对本项目上游训练超参数或精度的重新复现。对当前 checkpoint，类别项在单类监督语义下服务于候选信号区域，`single_cls=True` 限制优先于部署侧的标签表。

5.5.4. 从 NPU 张量到候选区域

这张图解决 “神经网络输出后还发生了什么” 的问题。ONNX 没有嵌入 NMS；OM 输出先在 CPU 上完成确定性的解码和抑制，才成为界面叠加框和 JSONL 记录。

图 5-8 当前 OM 兼容解码流程。`output0 [1,55,21504]` *转置为候选行后，前四列按* `xywh` *解释；现有实现从后 51 列取最大评分和通道索引，并按该索引分组做 IoU 0.70 的 NMS，最多保留 300 个框。该索引只是张量布局的兼容信息，不是调制类别标签；因此本图只说明输出解码和流水线连通，不能说明候选区域的类别、数量、召回率或误报率。图源见* `img5/case5-yolo-detection-decode-flow.dot`。

这一段 CPU 后处理不构成 CPU 推理回退：卷积、特征融合和头部计算已经由 OM 完成，CPU 只执行框坐标换算、阈值筛选、NMS 和物理坐标映射。当前实现保留按评分通道分组的兼容逻辑，但它不能被表述为语义正确的 class-aware NMS；单类部署的正式后处理应改用单一候选得分和 class-agnostic NMS。在该源代码完成相应修订前，本章只把当前输出用于验证张量合同、数值有限性和流水线连通。若初始化时 OM 无法加载，实时入口显示或记录 `NPU unavailable`；若已运行的 OM 返回 NaN、Inf 或不满足 manifest 的形状，RTL 服务会终止本次运行并写入失败记录。两种失败都不会用另一个 CPU 检测器伪造 NPU 成功。

5.6. 核心实验：从 CAL 方波到固定分析窗口

5.6.1. 先确认 USB，而不是直接启动仪表盘

USB 诊断程序只枚举已知设备，不打开示波器，也不上传固件。这样可以先区分 “没有识别到设备” “当前用户没有权限” 和 “设备被其他程序占用”。在开发板的项目根目录执行：

```
cd ~/Documents/case5
python -m time_frequency_dashboard.acquisition.usb_diagnostics
```

你应该看到：输出中列出 Hantek 对应的 USB 标识，并显示当前用户是否可写。如果显示 `writable=False`，应按 `scripts/udev/60-case5-hantek6022.rules` 配置权限，而不是使用 `sudo` 启动整个仪表盘。

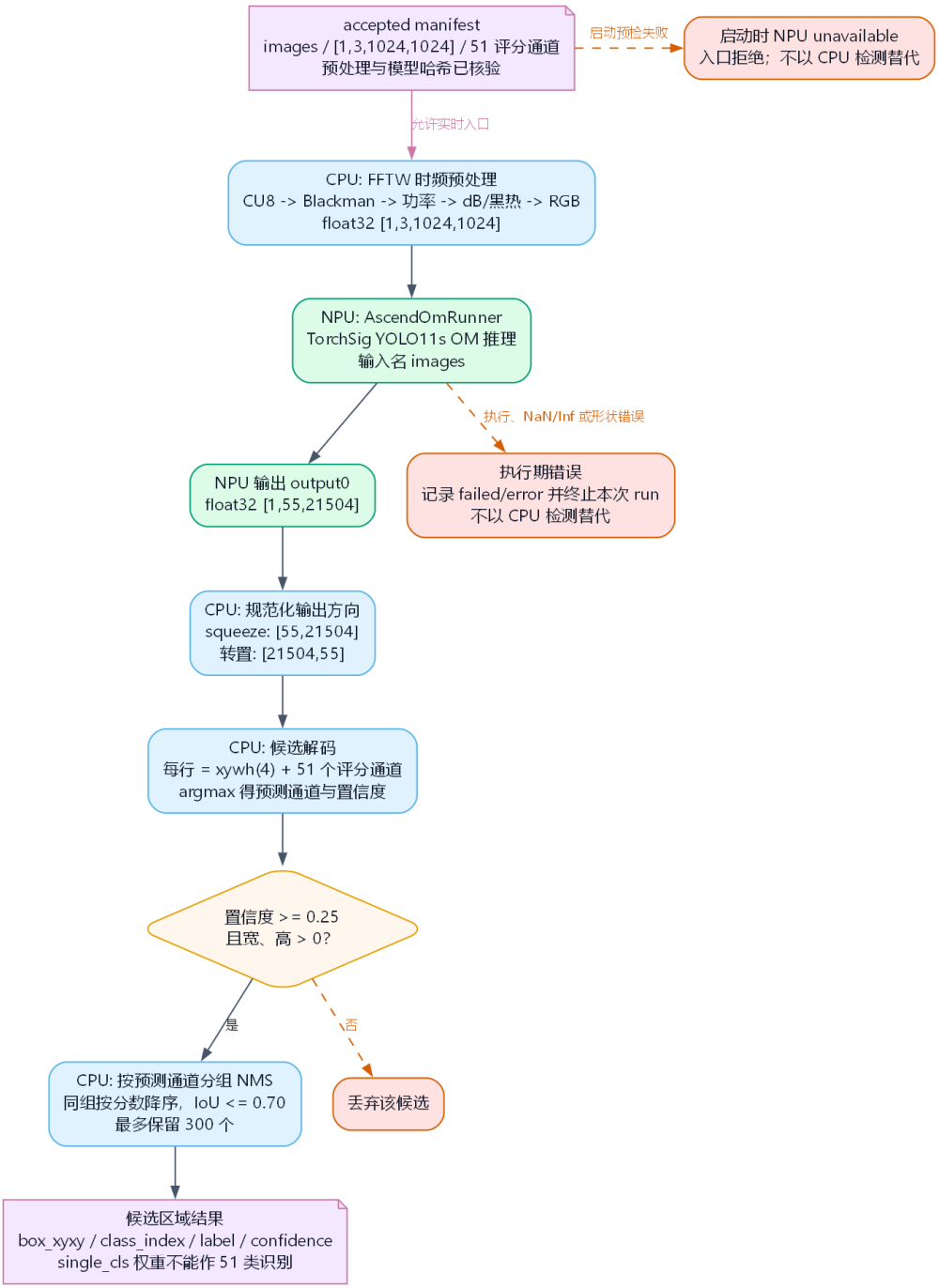


图 5.8. YOLO11 输出张量到时频检测事件的解码流程

检查点：只有设备已识别且当前用户可访问时才继续。诊断阶段没有方波并不表示采集失败，因为它根本没有打开 ADC。

5.6.2. 用 PulseView 建立最小真实基线

首次接线时，将 CH1 探头接到 6022BE 的 `CAL` 输出，用 PulseView 选择 Hantek 6022BE、1 MHz 和 10 k samples，观察类似图 5-2 的方波。这个步骤把问题限制在设备、驱动和探头三项，不涉及 Case 5 的 Python、OM 或界面。

确认方波后关闭 PulseView。PulseView、`sigrok-cli` 和仪表盘都需要独占同一台 6022BE。如果设备仍显示为 `1d50:608e` 的 `fx2lafw` 固件状态，关闭占用程序后物理拔插设备，再启动下一次 sigrok session。

检查点：能看到真实 CAL 方波，并且切换到 Case 5 前已经关闭 PulseView。不要把这一步写成 NPU 验收。

5.6.3. 编译长期运行的 sigrok 采集桥

Case 5 不要求安装 `pyhantek6022`，也不再编译 OpenHantek 桥。板端使用系统 `libsigrok` 的 `hantek-6xxx` 驱动，由一个小型 C 程序把回调转换成稳定的二进制帧。

系统包由用户手动安装完成后，执行：

```
cd ~/Documents/case5
bash scripts/build_sigrok_capture_bridge.sh
```

你应该看到：生成 `build/sigrok_capture_bridge`。脚本从自身位置解析项目根目录，不依赖临时工作目录。

桥接程序把二进制帧写到 stdout，把诊断写到 stderr。不要把 stdout 当成文本输出重定向到终端。Python 看到的公共对象可以简化为下面这段代码：

```
@dataclass(frozen=True)
class BridgeFrame:
    sequence: int
    host_receive_ns: int
    sample_rate_hz: float
    flags: int
    samples: np.ndarray  # [samples, 2], float32
```

`sequence` 用于发现用户态帧缺口，`host_receive_ns` 是主机接收时间，`sample_rate_hz` 是驱动实际返回的采样率，`flags` 当前至少包含削顶标志。设备驱动没有提供 FIFO 溢出、ADC 序号或跨 USB 回调缺口元数据，所以桥序号连续只能证明桥到 Python 的帧没有丢失，不能证明物理 ADC 绝对无间隙。

5.6.4. 从连续帧组成固定分析窗口

采集回调的块大小不应成为模型输入形状。`WindowAssembler` 将连续帧拼成 10,000 点窗口；一旦发现序号跳变、主机时间倒退或采样率变化，就清空未完成窗口，从下一帧重新开始：

```
if self._last_sequence is not None and frame.sequence != self._last_sequence + 1:
    self.reset()
if self._sample_rate_hz is not None and frame.host_receive_ns < self._last_host_receive_ns
    ↪ :
    self.reset()
if self._sample_rate_hz is not None and not np.isclose(
    self._sample_rate_hz, frame.sample_rate_hz, rtol=0.0, atol=0.0
):
    self.reset()
```

这段处理体现了一个重要原则：宁可放弃跨越缺口的半个窗口，也不能把不连续样本拼成一段看似正常的频谱。

在启用 Little Bee 扩展配置时，`CaptureProcessor` 会按声明参数把 CH2 电压换算为趋势性电流，并计算最新采集块的统计量：

```
current = self.config.current_conversion.to_current(frame.samples[:, 1])
converted = np.column_stack((frame.samples[:, 0], current)).astype(np.float32)
self.latest_statistics = (
    signal_statistics(converted[:, 0]),
    signal_statistics(converted[:, 1]),
)
```

这里的 CH2 换算只是配置驱动的显示变换，不是对 Little Bee 的本机标定。在核心 `CAL` 实验中，CH2 仍保留在两通道帧和固定形状张量中，但不据其数值作出电流测量结论。固定窗口形成后，每个通道再减去自身均值。CPU 只做单位换算、窗口整理和去直流，不计算供 Hantek 界面使用的 FFT 频谱。

这张图把 “帧缺口为什么必须丢掉半个窗口” 和 “队列为什么丢弃旧分析任务” 放在同一条流程上。它对应的是运行时数据完整性规则，而不是一次性的 USB 连接检查。

图 5-9 Hantek 实时窗口形成流程。任何帧序号、主机时间或采样率连续性错误都会重置未完成窗口；形成 `[1,2,10000]` 后才进入容量为 2 的最新任务队列。队列满时丢弃旧分析任务并计数，不能把跨越缺口的样本拼成正常频谱。图源见 `img5/case5-hantek-acquisition-flow.dot`。

5.6.5. 扩展任务：Little Bee B1 的接入准备

如需开展后续双通道扩展实验，才将 Little Bee 输出经 SMA-BNC 接入 CH2。开始采集前，应在探头本体上完成 `Reset/Zero`，记录工作模式、匝数、电池状态和连接方式；被测回路的隔离和允许测量范围必须已按第 2 节确认。

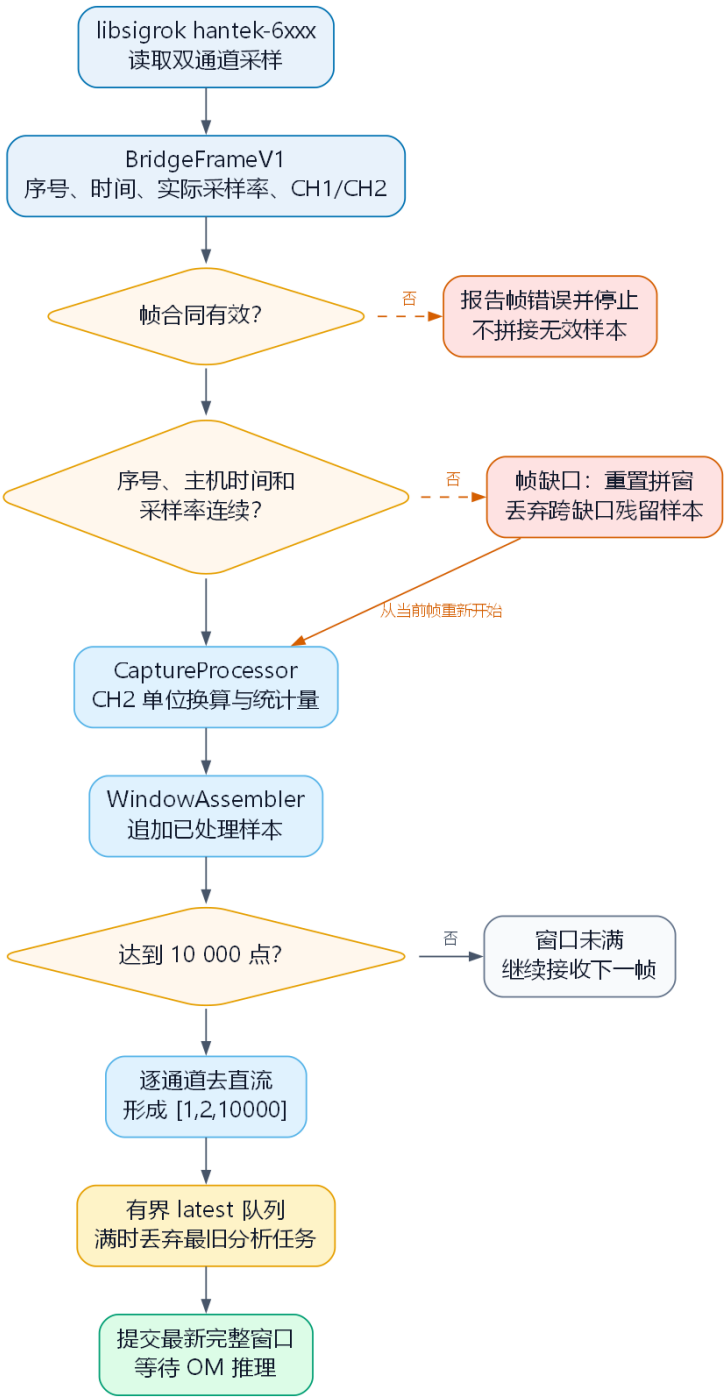


图 5.9. Hantek 桥接帧到固定分析窗口的流程

首版软件为这一扩展任务预设青色模式、1 MHz 声明带宽、1.00 V/A 灵敏度和单匝导线。换算关系为：

$$I = \frac{V_{\text{out}}}{S\,N_{\text{turns}}}$$

默认参数下，$S = 1.00\ \text{V/A}$、$N_{\text{turns}} = 1$。这只是按公开说明进行的声明换算，配置版本为 `declared-cyan-1.0V-per-A-1turn`。当前成品探头尚未完成本机幅值、零点漂移和相位标定，因此 CH2 只能作为趋势性电流，不用于输出计量级功率、相位角或功率因数，也不属于本章核心实验的验收条件。

5.7. 在 310B 上完成固定 DFT

5.7.1. 把 DFT 写成静态 ONNX 图

传统 FFT 算法并不是本实验要部署的图。这里预先用 NumPy 生成带 Hann 窗的正弦、余弦投影矩阵，再用标准 ONNX 算子表达固定 DFT。输入窗口与矩阵相乘后，实部和虚部平方并成对求和，得到线性频谱功率。

`export_npu_spectrum.py` 中的核心节点如下：

```
nodes = [
    helper.make_node("MatMul", ["waveforms", "dft_projection_weights"], ["projected"]),
    helper.make_node("Mul", ["projected", "projected"], ["squared_projection"]),
    helper.make_node("Reshape", ["squared_projection", "pair_shape"], ["squared_pairs"]),
    helper.make_node("AveragePool", ["squared_pairs"], ["pair_average"],
                     kernel_shape=[2, 1], strides=[2, 1]),
    helper.make_node("Mul", ["pair_average", "pair_sum_scale"], ["spectrum_power"]),
]
```

输入是 `[1,2,10000]`，输出是 `[1,2,201,1]`。权重、频率轴、窗口类型、采样率和输入输出形状同时写入 JSON 元数据。部署程序不依赖 PyTorch、`torch_npu` 或 `torch.onnx`。

这张图解决“生成 ONNX 后为什么还不能直接宣布 NPU 可用”的问题。ATC 只完成格式转换；真实 OM 的加载、数值与有限值检查才决定仪表盘能否发布频谱。

图 5-10 固定 DFT 的板端验证与发布流程。只有真实 OM 输出形状正确、全为有限值并满足数值门限时，频谱和瀑布才接收线性功率；失败路径明确显示 `NPU unavailable`*，没有 CPU FFT 回退。图源见* `img5/case5-hantek-om-flow.dot`。

5.7.2. 在开发板生成并验证 OM

以下命令只能在 Ascend 310B 开发板执行。`prepare_models` 先生成 ONNX，再调用板端 ATC 转为 OM；`verify_npu_model` 使用同一测试输入比较数值参考与真实 OM 输出。

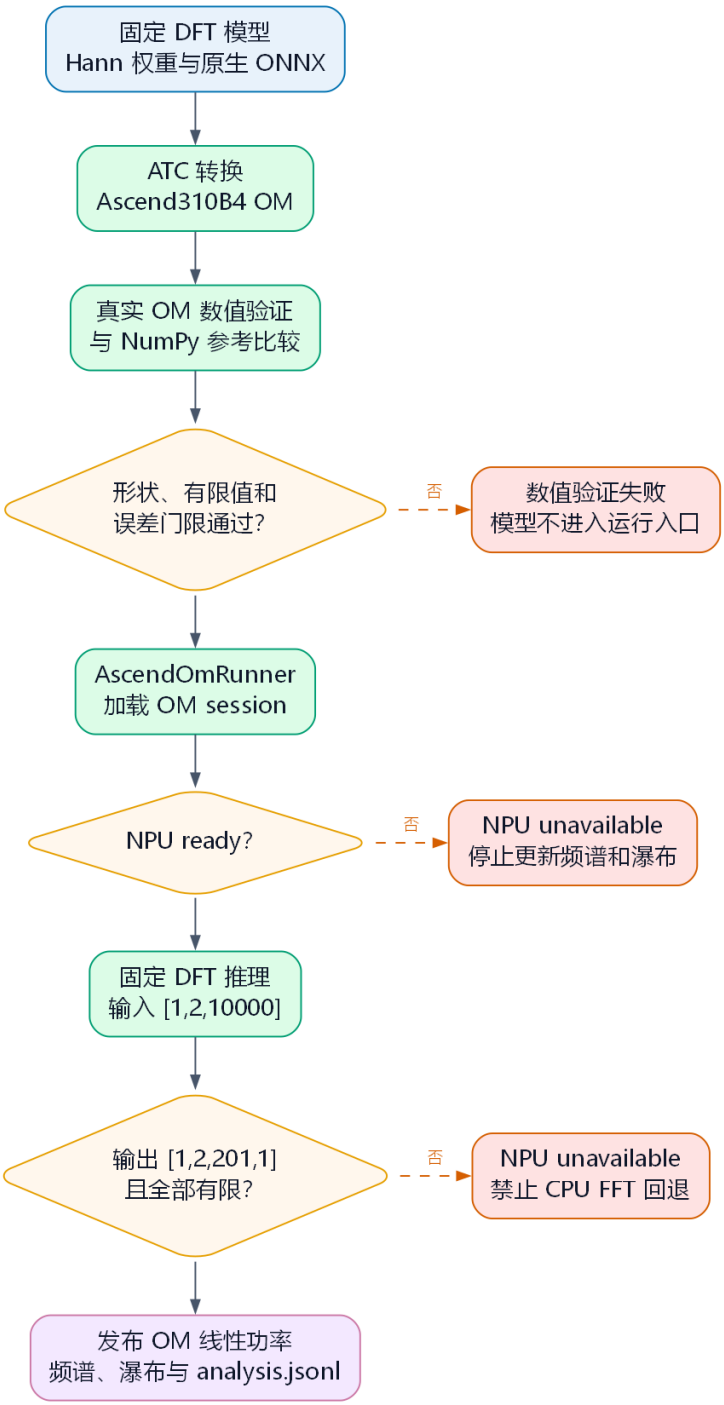


图 5.10. Hantek 固定 DFT 从 ONNX 到 OM 发布的流程

```
cd ~/Documents/case5
source /usr/local/Ascend/ascend-toolkit/set_env.sh
source /usr/local/miniconda3/etc/profile.d/conda.sh
conda activate base

python -m time_frequency_dashboard.model.prepare_models
python -m time_frequency_dashboard.model.verify_npu_model
```

你应该看到： `models/generated/` 中出现 ONNX、OM 和元数据文件，验证程序报告真实 OM 已加载、输出形状正确、数值有限且误差满足设定门限。

检查点：生成 ONNX 成功不等于 NPU 验证成功。只有 `aclruntime.InferenceSession` 实际加载 OM 并完成推理，才能记录 `NPU (Ascend 310B)`。

5.7.3. 启动仪表盘并读懂结果

确认 PulseView 和 `sigrok-cli` 已退出，运行：

```
cd ~/Documents/case5
bash scripts/run_dashboard.sh --sigrok-bridge build/sigrok_capture_bridge
```

连接 Hantek 后，顶部后端区域应显示 `NPU (Ascend 310B)`，而不是 `NPU unavailable`。图 5-11 是 2026-08-15 在 `ascend8t` 的 HDMI `1920x1080` 输出上取得的实机记录：CH1 探头接 6022BE 的低压 `CAL`，采样合同为 1 MS/s、10,000 点窗口。它解决的是 “真实采集、OM 和触摸界面是否同时工作” 的问题，不是电流标定或射频性能验收。

图 5-11a Hantek 波形工作区，完整显示区为 1920x1080。绿色状态块证明本次频谱后端为真实 `NPU (Ascend 310B)`；底部给出帧、USB 块、丢弃计数、延迟和会话目录。波形来自 CH1 的 `CAL`，不代表 Little Bee 的 CH2 幅相已标定。

图 5-11b 同一会话的频谱与瀑布页。上图和瀑布均由固定 DFT OM 的功率输出绘制，右侧色标显示的是相对 1 V² 的未校准 dB，不是 dBV、dBFS 或 dBm。图中的离散谱线说明方波谐波结构，不能单独用于估计探头带宽或精确幅值。

界面可以按从上到下、从左到右的顺序阅读：

1. 顶部状态栏给出当前设备、实际 NPU 后端和错误信息；
2. 左侧参数区显示量程、探头倍率、分析通道、瀑布历史和色标；
3. 波形页显示当前采集块，暂停显示只冻结绘图，不会释放 USB；
4. 频谱与瀑布页显示同一批 OM 功率结果，瀑布只保存有界历史；
5. 底部状态栏给出分析帧、USB 块、丢帧、NPU 延迟、处理延迟和会话路径。

CAL 方波的基频约为 1 kHz，理想方波还会出现奇次谐波。在频谱页观察这些离散成分，比只看时域方波更容易理解频域为什么能描述波形边沿。但本次显示仍是未校准的相对功率，不应根据峰值直接推断探头带宽或精确电压。

图 5.11. Hantek CAL 方波在 Case 5 仪表盘中的实时显示

图 5.12. Hantek CAL 方波的 NPU DFT 频谱与瀑布

5.7.4. NPU 不可用时系统怎样表现

Hantek 的 `AscendOmRunner` 只有在 OM 文件存在、`aclruntime` 可导入、session 建立成功且输出名称可读取后才进入 ready 状态。初始化失败时，Hantek 顶部明确显示 `NPU unavailable`；已经开始的 Hantek 分析若返回 NaN、Inf、错误形状或运行异常，分析 worker 停止发布新频谱行并转入同一不可用状态。

RTL-SDR 的状态语义应单独阅读：初始化或 OM 合同预检失败时入口显示或记录 `NPU unavailable`；服务已经启动后若遇到 NaN、Inf、错误形状或运行异常，则本次 run 进入 `failed`，并在 JSONL footer 中写入 `error` 和 `completion_status=failed`。后者不能笼统写成一个成功后又 “降级” 的状态。

系统没有 Hantek CPU FFT fallback。模拟源可以帮助调试采集、队列和界面，但没有 OM 时不会生成一条 CPU 频谱冒充 NPU 结果。这个失败状态是可验证设计的一部分，而不是缺少功能。

5.8. 保存并复核一次会话

5.8.1. 会话目录记录了什么

每次开始 Hantek 采集时，程序都会在 `data/hantek_sessions/` 下创建一个新的 UTC 时间目录，不覆盖旧证据。

文件	作用
`manifest.json`	保存配置、模型元数据、原始帧格式和时间基准
`raw_XXXX.c5raw`	分块保存原始 `BridgeFrameV1`，默认每块不超过 64 MiB
`raw_index.jsonl`	记录每帧的序号、时间、采样率、块文件、偏移和字节数
`analysis.jsonl`	记录窗口范围、后端、形状、线性功率和推理延迟
`summary.json`	汇总原始字节、帧数、分析数、丢帧和写入错误

原始数据默认最多写入 1 GiB。达到上限或写队列满时，程序记录存储丢帧，不会让内存无限增长。分析 JSON 使用 `allow_nan=False`，因此 NaN 或 Inf 不能悄悄进入可追溯记录。

5.8.2. 从一条结果追到原始窗口

假设 `analysis.jsonl` 中某条结果记录了 `first_sequence=120`、`last_sequence=121`。复核时可以按下面的顺序追踪：

1. 在 `analysis.jsonl` 读取窗口序号范围、主机起止时间、模型后端和输出；

2. 在 raw_index.jsonl 查找序号 120、121 对应的块文件和字节偏移；
3. 根据索引从 raw_XXXX.c5raw 读取原始 BridgeFrameV1；
4. 在 manifest.json 核对采样率、探头倍率、Little Bee 换算版本和模型元数据；
5. 在 summary.json 检查本次会话是否发生存储丢帧或写入错误。

显示层使用 dB 便于阅读，但会话保存的是 OM 返回的线性功率。这样复核者可以重新选择显示下限，不会因为界面色标改变而丢失原始分析值。

这张图解决 “一个漂亮的频谱图怎样被独立复核” 的问题。它从结果记录反向回到原始帧，再让配置与会话完整性证据参与判断。

*图 5-12 单条 Hantek 分析结果的追溯路径。*analysis.jsonl *给出窗口范围和 OM 结果，*raw_index.jsonl *定位原始块；*manifest.json *约束采样率、探头和模型，*summary.json *报告会话完整性。缺少任一环节时，不能声称该窗口完全可复核。图源见* img5/case5-session-trace-flow.dot。

5.8.3. 当前仍未完成的部分

当前代码已经计算最新采集块的均值、RMS 和峰峰值，但 Hantek UI 与 analysis.jsonl 尚未持久化这些字段。Little Bee 的真实双通道接入、幅值标定、相位延迟和同步验收也尚未完成。因此，本章能够验收的是两通道数据帧合同、固定窗口、NPU 频谱、瀑布、后端状态和结果追溯；不能验收计量级功率、功率因数或故障分类。

5.9. 扩展实验：从 RTL-SDR IQ 到时频检测

5.9.1. 先用固定 DFT 理解 IQ 频谱

正式检测模型包含时频预处理、目标检测和模型准入，第一次接触时不容易判断错误来自哪一层。旧固定复数 DFT Demo 提供了一条更小的教学路径：先生成一个频率已知的复数正弦波，确认 OM 的峰值频点正确，再接入真实接收机。

在开发板生成并验证模型：

```
cd ~/Documents/case5
source /usr/local/Ascend/ascend-toolkit/set_env.sh
source /usr/local/miniconda3/etc/profile.d/conda.sh
conda activate base

command -v rtl_sdr
python -m time_frequency_dashboard.model.prepare_rtl_iq_model
python -m time_frequency_dashboard.model.verify_rtl_iq_model
```

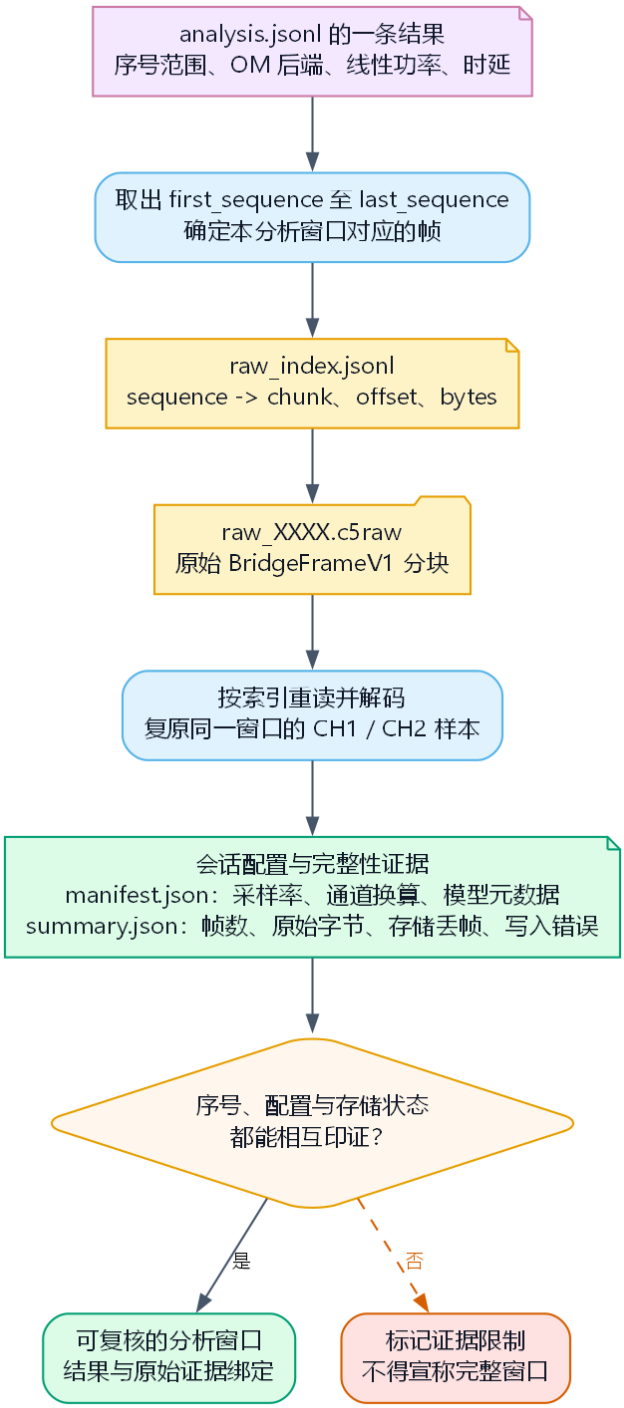


图 5.13. Hantek 分析结果到原始证据的追溯流程

模型输入为 `[16,2,1024]`，一次推理覆盖 16 个窗口。采样率为 2.048 MS/s 时，单个 1024 点窗口为 0.5 ms，整个批次为 8 ms，频率间隔为 2 kHz。输出按照 `fftshift` 从 -1.024 MHz 排列到接近 +1.024 MHz。

先运行不依赖天线和射频环境的合成测试：

```
bash scripts/run_rtl_sdr_npu_demo.sh --source tone --batches 2
```

你应该看到： JSONL 中记录 `NPU (Ascend 310B)`、输入输出形状、推理耗时和与合成音调一致的峰值频率。

检查点： 这个 Demo 的频谱来自 OM，但它不被 PySide6 SDR 工作区使用，也不能代表正式检测模型的性能。

关闭 GQRX、GNU Radio、SDR++、`rtl_test` 等可能占用接收机的程序，确认天线或受控线缆状态后，可以运行有限真实采集：

```
bash scripts/run_rtl_sdr_npu_demo.sh \
  --source rtl \
  --center-frequency 100000000 \
  --gain-db 40.2 \
  --batches 8
```

40.2 dB 只是当前板卡、天线和 100 MHz 环境的实测起点，不是通用增益标定。若改用其他频段，应先通过独立短采集检查 CU8 端点削顶和 I/Q 方差。

5.9.2. 正式 RTL-SDR 服务的数据流

正式 CLI 与仪表盘共同调用 `RtlSdrService`。一次运行按以下顺序发生：

1. 读取并复核 `accepted` manifest、ONNX/OM 哈希和固定输入合同；
2. 根据模型形状和采样率规划完整窗口，预检磁盘容量；
3. 启动 `rtl_sdr`，精确读取一个完整模型批次所需的 CU8 字节；
4. 归档原始 CU8，解码 I/Q，并构造模型输入；
5. 调用 `AscendOmRunner`，校验输出并执行 Top-K 或 NMS；
6. 将批次记录写入 JSONL，并把最新结果发布给 Qt；
7. 停止时写入 footer，绑定 CU8 字节数、SHA256、完成状态和时延汇总。

输入队列满时丢弃最旧批次，并在记录中增加 `queue_dropped_batches`。显示也只保留最新不可变快照，因此绘图变慢不会改写已经归档的模型输入或无限占用内存。

这张图解决 “时频图和检测框分别在哪里计算” 的问题。CPU 的 FFTW 只构造 manifest 规定的模型输入；神经网络检测必须来自 OM。

图 5-13a RTL-SDR 实时服务流程。原始 CU8 先归档并绑定 SHA256；输入队列积压时丢弃最旧待推理批次；FFTW 生成时频图后由 OM 执行检测。初始化失败时入口拒绝

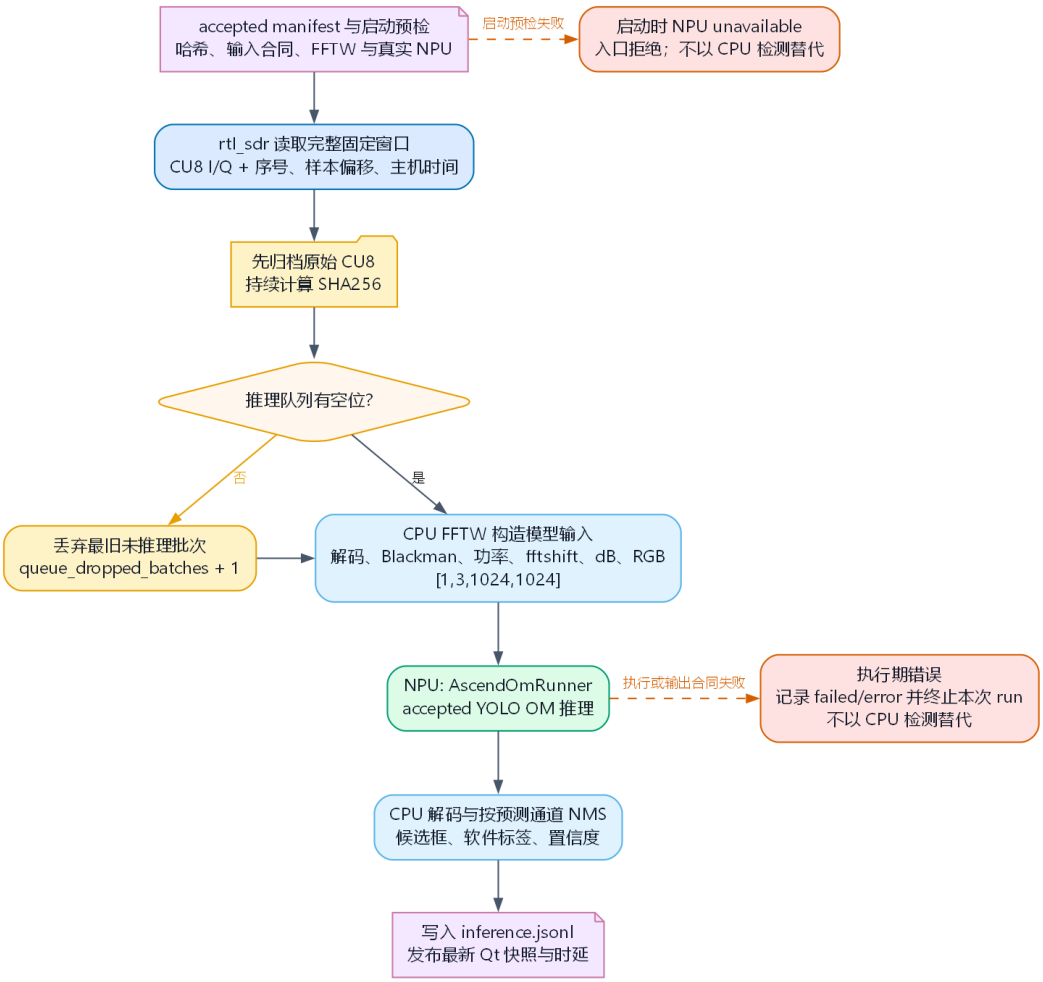


图 5.14. RTL-SDR 实时服务从 CU8 到 OM 检测与 JSONL 的流程

NPU 运行；执行期输出异常则以失败记录终止本次 run，二者都不会改用 CPU 检测器。图源见 `img5/case5-rtl-service-flow.dot`。

5.9.3. 谁计算时频图，谁完成检测

当前正式模型是频谱图目标检测器。CPU 使用 FFTW 按 manifest 中的结构化合同执行 CU8 解码、1024 点无重叠 Blackman 窗、功率、`fftshift`、dB 归一化、黑热图和 RGB 复制，得到真正送入 OM 的 `[1,3,1024,1024]` 输入。

这个时频图可以在界面中预览，但必须标注为 “CPU FFTW 模型输入”。NPU 执行 YOLO 并返回框参数与评分，CPU 再按第 5.4 节进行确定性解码和显示。当前 checkpoint 的输出只可作为候选区域链路的技术性证据；现有兼容解码中的通道索引和框数均不是类别或真实事件数量。把 CPU 模型输入称为 “NPU FFT”，或者用 CPU 检测结果代替 OM，都会破坏后端可解释性。

对于 raw-IQ 分类模型，CPU 只按 manifest 进行去直流和归一化，NPU 返回类别 logits，界面再显示 Top-K。两类模型共用服务，但输入形状和后处理不能混用。

图 5-13b 和图 5-13c 是 2026-08-15 在 `ascend8t` 的 HDMI `1920x1080` 输出上获得的一次真实短运行。运行选择已准入的 `torchsig-yolo11s-v1.1.0-b1` OM，记录中心频率 100 MHz、采样率 2.048 MS/s、手动增益 40.2 dB、`antenna_connected` 和 10 s 请求。服务将请求扩展为 20 个完整窗口；本次完成 20/20 批、推理队列丢批为 0，并对同一 CU8/JSONL 的只读 QC 报告端到端 P95 为 902.59 ms。该记录证明数据流与后端，不构成天线、频率轴、信号类别或检测准确率的计量结论。

图 5-13b RTL-SDR I/Q 与星座页，完整显示区为 1920x1080。中间两图是已归档 CU8 解码后的未校准 I/Q 和显示用星座，右侧表是 OM 输出经确定性解码后的最近候选。顶部绿色块明确给出 `NPU (Ascend 310B)`*；表中省略的长时间和频率值可由悬浮提示读取。*

图 5-13c RTL-SDR 时频检测页。灰度图是 CPU FFTW/Blackman 预处理后实际送入 OM 的 `[1,3,1024,1024]` *图像；绿色框来自 OM 神经网络输出的 CPU 确定性解码。为保证触摸屏可读性，界面保留全部框和结果表，但只标注一组不重叠的高置信度框。*`bpsk` *是当前单类 checkpoint 的部署映射文字，不能解释为已验证的 BPSK 识别或 51 类调制识别。名义 RF 轴尚未校准。*

5.10. 模型准入与真实板端运行

5.10.1. 为什么一个 OM 文件还不够

ATC 能够生成 OM，只说明图在当前工具链下可编译。要进入实时入口，还必须回答：这个模型来自哪里？权重是否与审查版本一致？ONNX 与 OM 是否被修改？NPU 输出是

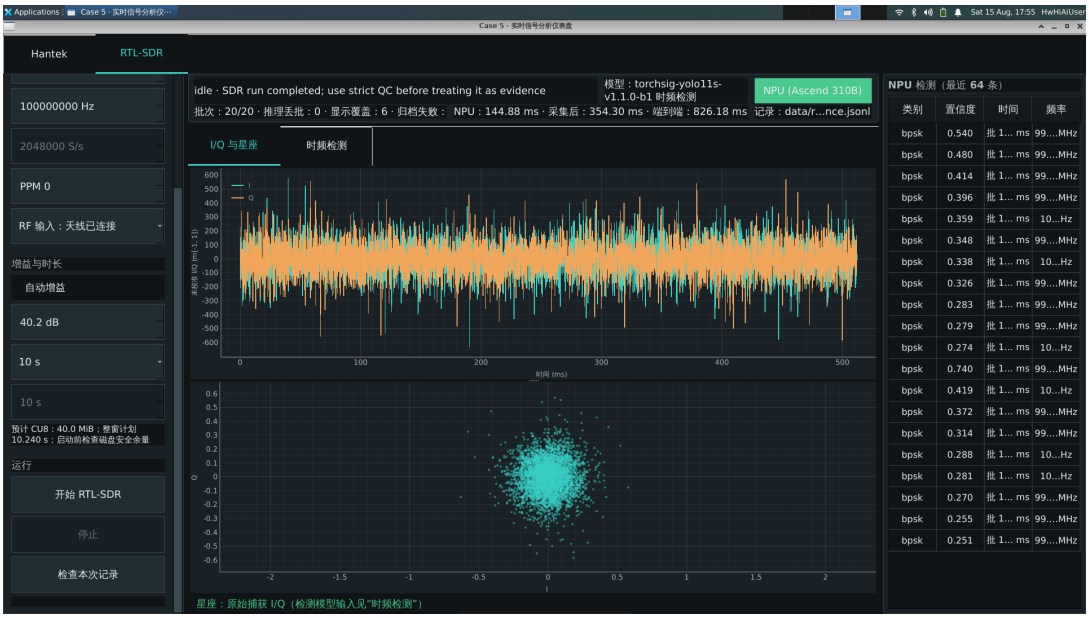


图 5.15. RTL-SDR 真实 I/Q 与星座工作区

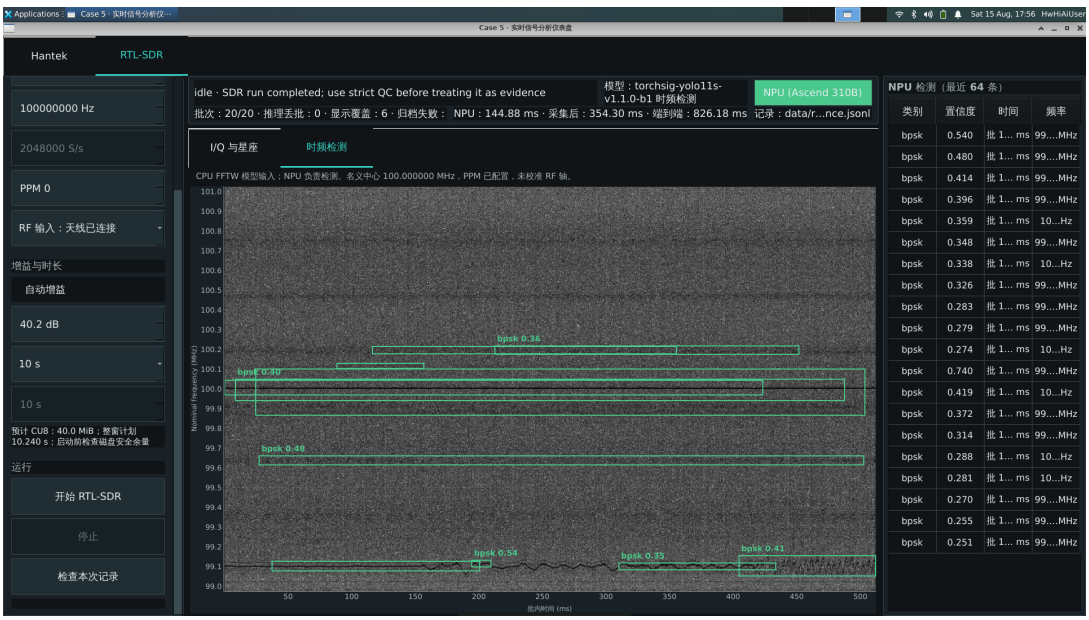


图 5.16. RTL-SDR 真实时频检测工作区

否与参考数值一致? P95 推理时间是否小于一个输入窗口? 预处理是否与模型训练和验证时一致?

Case 5 用版本化 manifest 保存这些答案。正式入口只展示同时满足以下条件的模型:

- `admission.status` 为 `accepted`;
- ONNX、OM 和上游权重 SHA256 与记录一致;
- 输入形状、采样率和结构化预处理合同完整;
- 数值比较通过，输出有限;
- NPU P95 满足固定输入窗口预算;
- 模型被标记为可用于实时演示。

`accepted` 只表示模型和 NPU 边界通过准入，不代表采集、预处理、排队和后处理组成的整条主机流水线也已经实时通过。后者需要一次独立运行记录。

这张图区分 “离线准入” 与 “在线连续运行证据”。前者证明一个固定模型合同可部署，后者才说明在明确输入状态下的完整流水线没有超过窗口预算。

图 5-14 模型准入与证据附着流程。来源、哈希、数值比较和窗口预算共同决定 `accepted` *；真实运行的 CU8、JSONL 和只读 QC 形成新的 sibling manifest 证据，而不是改写原始 manifest。图源见* `img5/case5-model-admission-flow.dot`。

5.10.2. 从模型原理到板端候选

第 5 节已经从数据表示、YOLO11s 结构和后处理解释了这个候选。进入板端前，需要依次完成来源审查、ONNX 与 OM 工件核对、输入预处理复核、数值比较和窗口预算测量；这些信息由 manifest 绑定。部署仓库不包含训练环境和 PyTorch，上游权重只在隔离的准备环境中读取，部署侧只接收经过审查的 ONNX、OM 和 manifest。

若基础混合精度不能满足数值门限，板端准备流程可以采用 CANN 的局部精度保护机制。它只处理已发现的数值敏感边界，不改变模型训练语义，也不能把单类训练权重的通道布局变成多类识别。具体参数、工件名和重建命令随工具链演进，应以配套资料 07为准；不得通过手工修改 `status` 把 candidate 变成 accepted。[6] [7]

5.10.3. 运行一次完整的真实窗口实验

当 `models/generated/inference/` 中已经存在通过准入的 manifest 时，先进行 10 秒级短运行:

[6] Huawei. “CANN 8.0 ATC precision_mode.” https://www.hiascend.com/document/detail/en/canncommercial/800/devaids/atc/atlasatcparam_16_0068.html

[7] Huawei. “CANN 8.0 ATC keep_dtype.” https://www.hiascend.com/document/detail/en/canncommercial/800/devaids/atc/atlasatcparam_16_0074.html

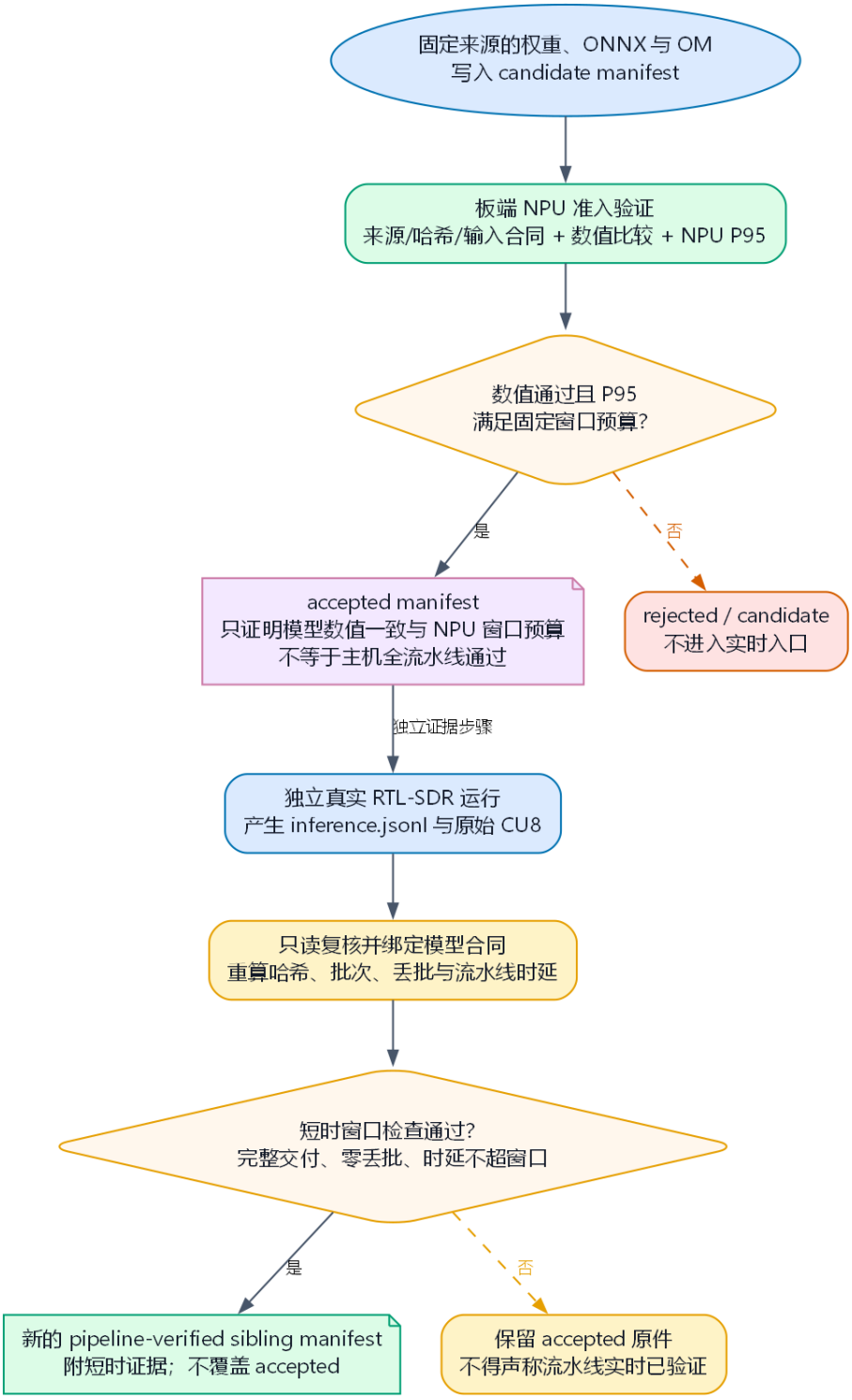


图 5.17. 模型准入与连续流水线证据的流程

```
cd ~/Documents/case5
bash scripts/run_rtl_sdr_npu_inference.sh \
  --source rtl \
  --manifest models/generated/inference/candidates/<accepted-manifest.json> \
  --center-frequency 100000000 \
  --gain-db 40.2 \
  --rf-input-context antenna_connected \
  --duration-seconds 10
```

当前检测模型的一个输入需要 $1024 \times 1024 = 1\,048\,576$ 个复样本。在 2.048 MS/s 下，一个完整窗口正好是 0.512 s。10 秒请求需要向上取整为 20 个窗口，因此实际计划为 10.240 s：

$$20 \times 0.512\ \text{s} = 10.240\ \text{s}$$

每个复样本使用两个 CU8 字节，所以计划归档大小为 40 MiB。服务在打开设备前使用同一公式检查磁盘空间，JSONL 同时记录请求时长、计划时长、样本数和批次数。

你应该看到： metadata、每个批次和 footer 都记录 `NPU (Ascend 310B)`；生产批次等于完成批次；短运行没有队列丢批；`post_capture_pipeline_max_ms` 不超过 512 ms 窗口预算。

检查点： 当前 UI 的通道索引或调制名称都不能作为类别输出；它们至多说明采集、预处理、OM 输出和兼容解码链路连通。由于该 checkpoint 采用 `single_cls=True`，现有框数也不能作为真实时频事件数量。要开展类别识别或候选区域性能评价，必须先修订为语义一致的单类后处理，或换用具有多类监督语义的模型并进行独立评测。

5.10.4. 对同一份证据执行只读复核

运行完成后，用只读汇总器重新读取 JSONL 所绑定的同一份 CU8：

```
python -m time_frequency_dashboard.rtl_sdr_run_report \
  --inference-jsonl data/rtl_sdr_npu_inference/<run>/inference.jsonl \
  --output data/rtl_sdr_npu_inference/<run>/qc_summary.json
```

汇总器复核 CU8 字节数和 SHA256，要求 metadata、每批和 footer 都记录真实 NPU 后端，并统计 I/Q 直流偏置、端点削顶率和逐阶段 P50/P95/max 时延。它不会重新调用接收机、FFTW 或 NPU，也不会覆盖原始 JSONL、CU8 或已有 QC 文件。

若需要把一次运行附着为新的流水线证据，应从原 manifest 生成 sibling manifest，而不是改写采集时使用的文件：

```
python -m time_frequency_dashboard.model.attach_pipeline_realtime_evidence \
  --manifest models/generated/inference/candidates/<accepted-v3.manifest.json> \
  --inference-jsonl data/rtl_sdr_npu_inference/<run>/inference.jsonl \
  --output models/generated/inference/candidates/<pipeline-verified-v4.manifest.json>

```

```
python -m time_frequency_dashboard.model.attach_pipeline_realtime_evidence \
  --manifest models/generated/inference/candidates/<pipeline-verified-v4.manifest.json> \
  --verify-attached
```

至少 600 秒、输入状态为 `antenna_connected` 或 `lab_cabled`、零丢批且每批处理不超过窗口预算，才会被标记为连续管线通过。短运行只完成窗口级 smoke test。

5.11. 性能结果与工程判断

5.11.1. Hantek 采集吞吐说明了什么

表 5-1 给出一次板端采集回调测量的结果。它只统计主机从 `libsigrok 0.5.2` 收到的模拟样本，不包含 Qt 绘制、会话存储或 NPU 推理；完整试验条件见配套资料 01。

请求采样率	单通道有效速率	双通道每通道有效速率
1 MS/s	0.993 MS/s	0.991 MS/s
8 MS/s	7.769 MS/s	7.635 MS/s
16 MS/s	15.169 MS/s	14.742 MS/s
24 MS/s	20.843 MS/s	20.011 MS/s

30 MS/s 和 48 MS/s 请求没有继续提高有效速率。Case 5 主线因此使用驱动读回并与 OM 合同一致的 1 MS/s 作为频率轴采样率，而不按设备宣传值推断运行能力。表中的“有效速率”是主机回调吞吐量，不能代替频率轴采样率；这些数字也不能证明跨 USB 回调的 ADC 样本绝对无间隙。

5.11.2. RTL-SDR 连续流水线

表 5-2 摘录一次已接天线的连续板端运行：中心频率 100 MHz、采样率 2.048 MS/s、固定请求增益 40.2 dB、模型窗口 512 ms。除非另有说明，时延均从主机获得完整 IQ 批次后开始计时；“NPU 调用边界”包括张量准备、主机到设备传输、OM 执行、设备到主机传输和输出复制，不是射频首个样本到界面显示的端到端时延。完整环境与原始记录见配套资料 07。

指标（一次板端记录）	结果
连续运行时间	600.145 s
计划 / 完成批次	1,170 / 1,170
队列丢批	0
后采集流水线 P50 / P95 / max	252.440 / 255.858 / 282.768 ms
主机至主机 NPU 调用边界 P50 / P95 / max	139.425 / 139.881 / 141.935 ms

指标（一次板端记录）	结果
FFTW 预处理 P50 / P95	76.404 / 79.410 ms
CU8 归档大小	2,453,667,840 bytes

后采集最大处理时间小于 512 ms，且生产批次与完成批次相等、零丢批，因此这次记录满足本章定义的连续管线条件。它只证明该板卡、这组参数和这次输入条件下的主机流水线，不证明射频幅值已经校准；当前兼容解码产生的框数和显示标签也不能作为类别结论或真实事件计数，因为 checkpoint 的 `single_cls=True` 训练语义不支持 51 类调制识别，且这次真实 IQ 没有地面真值。

5.11.3. NPU 并不天然快于最佳 CPU 算法

旧固定复数 DFT 教学路径使用同一份真实 CU8，对 1024 点、batch 16 的不同实现进行了 50 次预热和 300 次重复测量。计时从主机输入数组可用开始，到主机可读取输出为止；初始化时间不计入，OM 数据包含主机与设备之间的数据传输：

实现	P50	P95	计算含义
ARM FFTW3	0.136 ms	0.141 ms	优化后的 CPU FFT
NumPy FFT	1.208 ms	1.270 ms	Python/NumPy CPU 基线
Ascend OM	1.618 ms	1.641 ms	主机到主机固定稠密 DFT
CPU 稠密 DFT	6.168 ms	6.421 ms	与 OM 相同矩阵算法

OM 比相同算法的 CPU 稠密 DFT 快，但明显慢于 FFTW。原因不是 “NPU 不适合信号处理”，而是小型 FFT 有成熟的低复杂度 CPU 实现，而固定矩阵 DFT 需要更多乘加和主机设备搬运。当前工程选择因此是：轻量 FFT、解码和归约留在 ARM/FFTW，大型固定形状卷积、矩阵和神经网络优先评估 NPU。

是否迁移到 NPU，应同时检查数值、P95、输入窗口预算、最佳 CPU 基线和 CPU 释放价值，而不是只看一次推理是否成功。

5.12. 从零复现、验收与故障排查

5.12.1. 本地开发检查

本地开发机只验证 Python、NumPy、ONNX 和 Qt 隔离逻辑：

```
cd samples/case5
python -m pytest -q
python -m compileall -q time_frequency_dashboard
```

pytest 覆盖帧协议、窗口缺口、单位换算、有界队列、ONNX 与 NumPy 数值对照、会话存储、NPU 不可用状态、仪表盘资源互斥、RTL manifest 和 JSONL/QC 合同。测试跳过硬件项并不表示硬件已经通过。

5.12.2. 开发板准备

在开发板上进入固定目录，由用户手动安装系统依赖：

```
cd ~/Documents/case5
sudo apt-get update
sudo apt-get install -y libsigrok-dev sigrok-cli gcc pkg-config libfftw3-single3 rtl-sdr
```

如果只运行 Hantek 路径，可以不安装 `rtl-sdr`；如果还要编译独立 FFTW C 基准，需要额外安装 `libfftw3-dev`。项目脚本不会自行执行 `sudo`。

随后安装用户态 Python 依赖：

```
source /usr/local/miniconda3/etc/profile.d/conda.sh
conda activate base
python -m pip install -r requirements-board.txt
```

`requirements-board.txt` 不包含 CANN、ACL、系统 sigrok 包或 PyTorch。CANN 使用开发板已有环境。

5.12.3. Hantek 验收顺序

1. 运行 USB 诊断，确认设备存在且当前用户可写；
2. 用 PulseView 和 `CAL` 完成最小真实基线，随后关闭 PulseView；
3. 编译 `build/sigrok_capture_bridge`；
4. 生成并验证 Hantek 固定 DFT OM；
5. 启动仪表盘，确认实际采样率为 1 MS/s；
6. 确认后端为 `NPU (Ascend 310B)`，频谱和瀑布随 OM 结果更新；
7. 停止采集，复核会话目录中的原始帧、分析记录和 summary；

Little Bee 接入属于扩展任务。只有完成零点、模式、匝数、幅值、相位和同步标定后，CH2 才能作为电流测量通道解释；它不属于上述核心验收。

真实 USB 与 OM 集成测试只能在板端显式启用：

```
cd ~/Documents/case5
export CASE5_RUN_HARDWARE_TESTS=1
python -m pytest -q tests/test_hardware_capture_and_inference.py
```

该测试必须先取得至少两个包含 CH1/CH2 的真实 Hantek 帧窗口，再把窗口送入 OM，检查输出形状 `[1,2,201,1]` 和有限值。它验证两通道帧合同和 NPU 频谱链路，不验证 CH2 的 Little Bee 电流标定。没有设置环境变量时的跳过不能写成“硬件测试通过”。

5.12.4. RTL-SDR 验收顺序

1. 用 `command -v rtl_sdr` 和 `rtl_test -t` 确认工具与接收机可用；
2. 生成并验证旧固定 IQ DFT OM，先运行 `--source tone`；
3. 使用有限真实 IQ 检查中心频率、增益、PPM 和 CU8 削顶；
4. 准备带来源、哈希、数值和预算证据的 accepted manifest；
5. 运行 10 秒正式服务，检查完整窗口、真实 NPU 后端和零丢批；
6. 对同一 JSONL/CU8 生成只读 QC；
7. 需要连续验收时，再执行至少 600 秒、已接天线或受控线缆的运行。

Hantek 与 RTL-SDR 在仪表盘中互斥。切换前必须先停止当前来源，等待采集进程、队列和 NPU runner 完全退出。互斥只证明应用层不会并行分配设备，不自动证明某一 CANN 版本可以在同一 Python 进程中完成任意 OM 的反复重初始化；这项行为仍需按板端环境单独验收。

5.12.5. 常见故障

现象	原因	处理
`Resource busy` / `LIBUSB_ERROR_BUSY`	PulseView、`sigrok-cli` 或旧仪表盘仍占用 6022BE	关闭占用程序；必要时物理拔插
`writable=False`	当前用户没有 USB 写权限	安装项目 udev 规则，不要 sudo 启动 UI
`sigrok capture bridge not found`	尚未编译桥或路径错误	运行 `scripts/` 下的 `build_sigrok_capture_bridge.sh`
实际采样率不是 1 MS/s	采集合同与固定 OM 不匹配	检查驱动、量程和启动参数
`NPU unavailable`	Hantek runner 在初始化或输出合同检查中失败，或 RTL 服务在启动预检时无法建立 OM	先区分 Hantek 与 RTL 路径，再运行对应 `verify_*_model` 并查看具体消息
RTL run `failed` / `error`	runner 已启动后输出出现 NaN/Inf 或违反形状合同	保留 JSONL，检查失败记录；不要把运行期错误写成 CPU 或 NPU 成功
`GeExecutor has not been initialized` 或 `[ACL: invalid parameter]`	本次板端审查中，Hantek 停止后同一 Python 进程重新启动 RTL-SDR OM 时出现的 CANN 生命周期错误	保留日志和失败 JSONL，确认 USB 已释放后关闭并重新启动仪表盘；不得改用 CPU 检测器。该跨来源重初始化问题不应被写成已通过的无缝切换验收

现象	原因	处理
Hantek 频谱不更新	NPU 没有 ready 或未形成完整窗口	检查后端、采样率、帧缺口和队列计数
CH2 数值异常	Little Bee 模式、匝数、去零或削顶不正确	回到探头说明和经确认隔离的教学回路重新检查；标定前不把 CH2 数值当作计量结果
SDR 页面没有模型	没有通过准入和哈希复核的 manifest	按 `docs/07` 完成模型准入
RTL 后采集延迟超过 512 ms	CPU 预处理、排队或 NPU 超出窗口预算	检查逐阶段 P95/max 和丢批，不得宣称实时通过
检测框很多但无法解释	当前 checkpoint 为单类区域检测，且真实 IQ 没有地面真值或接收链过载	检查 QC；只能报告候选区域链路连通，不能把软件标签写成多类准确率

5.13. 思考题、总结与参考资料

5.13.1. 思考与改进任务

1. 如果 Hantek 窗口从 10,000 点扩大到 20,000 点，时间跨度、频率间隔、OM 输入形状和更新率分别怎样变化?
2. `LatestQueue` 满时为什么丢弃最旧分析窗口？如果应用目标改为离线证据采集，队列策略应怎样改变?
3. 为 Little Bee 设计幅值和相位标定实验。需要哪些参考仪器、频点、负载和元数据，才能开始计算有功功率与功率因数?
4. 在 Hantek UI 和 `analysis.jsonl` 中加入 RMS、峰值和峰峰值时，怎样保证这些统计量与同一原始窗口关联，而不是最新显示块?
5. 当前 RTL 检测窗口为 512 ms。如果换成更小输入模型，哪些性能指标必须重新测量，旧 manifest 中哪些证据会失效?
6. 为什么固定 DFT OM 比 CPU 稠密 DFT 快，却比 FFTW 慢？这个结果对 “NPU 化” 逐点运算有什么启示?
7. 设计一个带已知标签的受控射频实验，使检测准确率与链路实时性能够分别评价。
8. 为什么 `[1,55,21504]` 的张量合同不足以证明 51 类调制识别？说明为什么必须先换用具有多类监督语义的模型，再建立训练、标注和独立评测证据。

5.13.2. 可迁移的方法

Case 5 的两条实验链面对不同物理信号，却使用了相同的工程方法。第一，先建立稳定的采集合同，把具体设备格式隔离在适配层。第二，把连续数据整理成固定窗口，让

时间、序号和原始文件始终可追溯。第三，只把适合固定形状和较高计算量的任务交给 NPU，并用最佳 CPU 实现作为比较基线。第四，后端状态、模型版本、延迟和错误必须进入界面和记录，不能在失败时悄悄替换结果。第五，把模型标签视为可检验的科学假设：对于本章这个 `single_cls=True` checkpoint，当前软件标签和框数仅服务于兼容性显示，不能代表类别或事件数量。要进行多类调制识别，必须替换为多类监督模型，并给出独立标签与适当指标。

这套方法比某一个示波器或某一个模型更重要。更换采集设备时，公共处理和存储合同仍然成立；更换模型时，manifest 和固定窗口预算仍然约束运行；增加新分析任务时，读者也能明确回答 “输入是什么、谁计算、输出是什么、怎样验证、结果依据在哪里”。

5.13.3. Case 5 专项文档

- 01 Hantek 6022BE：sigrok 驱动和实机采集
- 02 Little Bee B1：开源电流与磁场探头
- 03 架构与 sigrok 数据流
- 04 实时信号分析仪表盘前端
- 05 第三方代码与许可证
- 06 RTL-SDR 实时 NPU 服务与旧 DFT 教学路径
- 07 昇腾 310B 异构信号处理评估与 SDR-NPU 指引
- Case 5 代码入口与板端命令

5.13.4. 外部资料

6. 案例 6：智能小车

6.1. 项目简介

本项目基于昇腾 310B 平台，构建一个智能小车的视觉感知系统。系统结合 **经典计算机视觉**和**深度学习**两种方法：使用 OpenCV 的 Canny 边缘检测 + Hough 变换进行车道线检测，使用 ResNet18 卷积神经网络在 NPU 上进行驾驶场景分类。两种方法互补——几何特征（车道线）用经典 CV 提取，语义理解（场景类型）交给深度学习。

与旧版案例 6 不同，本项目不涉及机器人硬件（底盘、电机、激光雷达等），而是聚焦于昇腾 310B 最擅长的部分：**摄像头画面的实时 AI 理解**。这也是真实自动驾驶系统中"感知层" 的核心功能。

6.1.1. 与已有案例的关系

案例	领域	NPU 任务	方法特点
案例 2	目标检测 + 跟踪	SSD 检测	纯深度学习
案例 6	**智能小车感知**	**ResNet18 场景分类**	**经典 CV + 深度学习混合**
案例 7	智能相册	ResNet50 特征提取	批量索引 + FAISS 检索
案例 8	手势识别	MobileNetV3 分类	训练 -> 转换 -> 部署

案例 6 是全书第一个**显式结合经典 CV 与深度学习**的案例，展示了在实际工程中如何选择合适的技术方案——不是所有问题都需要神经网络。

6.2. 内容大纲

6.2.1. 硬件准备

本项目只需要：

- **昇腾 310B 开发者套件**：核心计算单元，运行场景分类模型
- **USB 摄像头**（可选）：用于实时画面采集，也可以用测试图片
- **显示屏**（可选）：用于浏览器访问 Gradio 界面

无需任何机器人硬件。车道线检测和场景分类都是纯视觉任务，在 310B + 摄像头组合上即可完整运行。

硬件架构如图所示：

```
flowchart LR
    subgraph INPUT["输入"]
        CAM["USB 摄像头\n或测试图片"]
    end

    subgraph COMPUTE["计算平台"]
        CPU["CPU\n• 车道线检测 (OpenCV)\n• 图像预处理"]
        NPU["昇腾 310B NPU\n• ResNet18 场景分类\n• OM 离线模型"]
    end

    subgraph OUTPUT["输出"]
        UI["Gradio Web 界面\n• 车道线叠加显示\n• 场景标签 + 驾驶建议"]
    end

    CAM -->|"图像帧"| CPU
    CPU -->|"车道线结果"| UI
    CPU -->|"预处理张量"| NPU
    NPU -->|"场景分类结果"| UI
```

6.2.2. 软件环境

操作系统与框架

层级	软件	版本	用途
操作系统	Ubuntu	20.04 / 22.04 LTS	运行环境
CANN	Ascend Toolkit	7.0+	NPU 驱动与 ATC 转换工具
Python	CPython	3.8 ~ 3.10	应用开发语言
深度学习	PyTorch + torchvision	2.0+	ResNet18 模型 / CPU 推理回退
图像处理	OpenCV	4.8+	车道线检测 + 图像预处理
Web	Gradio	4.0+	感知结果展示界面

环境准备

```
# 一键安装依赖并导出模型
bash setup.sh
```

`setup.sh` 依次完成：系统包安装 -> Python 依赖安装 -> ONNX 模型导出（如有 CANN 则继续转换为 OM）。

如果只想在开发机上导出 ONNX：

```
python3 prepare_models.py --onnx-only
```

Python 依赖说明

包	用途	必需?
`gradio`	Web 界面框架	是
`torch` + `torchvision`	ResNet18 模型定义、CPU 推理回退	是
`opencv-python`	车道线检测（Canny/Hough）、图像缩放	是
`numpy`	数值计算、softmax	是
`Pillow`	Gradio 图像格式转换	是
`onnx`	ONNX 模型校验（仅 prepare_models.py）	仅转换
`acl` (CANN 自带)	NPU 推理，随 CANN 安装	仅推理

6.2.3. 车道线检测原理

车道线检测是一个经典的计算机视觉问题。车道线本质上是图像中的**直线**或**平滑曲线**，具有明显的边缘特征和固定的几何约束（位于画面下半部分，大致呈八字形）。这些特征使得经典 CV 方法——Canny 边缘检测 + Hough 直线变换——比深度学习更直接有效。

为什么不用深度学习做车道线检测

方法	优势	劣势
经典 CV (Canny + Hough)	无需训练数据、速度快、可解释、参数可调	对复杂场景（阴影、弯道）敏感
深度学习 (UNet / LaneNet)	对复杂场景鲁棒	需要标注数据、模型大、OM 转换复杂

对于昇腾 310B 上的智能小车应用，经典 CV 是更合理的选择：不需要额外的标注数据，不占用 NPU 资源（NPU 留给场景分类），处理速度极快。

检测流水线

```
flowchart TD
```

```
    INPUT["输入图像\nBGR (H, W, 3)"] --> GRAY["cv2.cvtColor\nBGR -> Grayscale"]
    GRAY --> BLUR["cv2.GaussianBlur\n降噪 (5×5)"]
    BLUR --> CANNY["cv2.Canny\n边缘检测\n阈值 50/150"]
    CANNY --> ROI["ROI Mask\n保留画面下部 40%\n(道路区域)"]
    ROI --> HOUGH["cv2.HoughLinesP\n概率 Hough 变换\n提取线段"]
    HOUGH --> FILTER["斜率过滤\n|slope| >= 0.4\n分离左右车道"]
    FILTER --> FIT["加权线性回归\n拟合完整车道线"]
    FIT --> DRAW["cv2.addWeighted\n叠加到原图"]
```

每一步的实现都在 lane_detector.py 的 `LaneDetector` 类中。关键步骤说明：

1. **Canny 边缘检测**：检测图像中亮度剧烈变化的位置（车道线通常是白色或黄色的，与深色路面形成强烈对比）
2. **ROI 蒙版**：只保留图像下半部分（通常是路面区域），过滤掉天空、建筑物等干扰
3. **Hough 直线变换**：将边缘图中的点映射到参数空间，找到共线的点群——即直线段。概率 Hough 变换 (`HoughLinesP`) 直接返回线段端点，比标准 Hough 变换更高效
4. **斜率过滤**：根据斜率的正负号分离左右车道线（左车道斜率为负，右车道斜率为正），并过滤掉接近水平的线段
5. **车道线拟合**：用加权线性回归将多个短线段合并为一条完整的车道线。以 y 为自变量（车道线接近垂直），以线段长度为权重
6. **叠加绘制**：用 `cv2.addWeighted` 将车道线半透明叠加到原图上，并在两条车道线之间填充绿色区域标记可行驶车道

6.2.4. 驾驶场景分类

车道线检测告诉小车” 路在哪里”，但还需要知道” 是什么路”——高速公路还是城市街道？前方是否有交叉口？这需要语义层面的理解。

本项目使用 ResNet18 对 5 种典型驾驶场景进行分类：

#	场景	特征	驾驶建议
0	高速公路	多车道、护栏、路牌	保持车道，注意车速
1	城市道路	建筑、行人、路边停车	注意行人和交叉口
2	交叉路口	路口、交通灯、斑马线	减速观察
3	停车场	车位线、密集车辆	低速行驶
4	隧道	昏暗、墙壁、灯光	开启车灯

为什么选择 ResNet18

模型	参数量	310B 推理	适用性
ResNet18	11.7M	~12ms	推荐：轻量、torchvision 内置

模型	参数量	310B 推理	适用性
ResNet50	25.6M	~25ms	更准但更慢，案例 7 已使用
MobileNetV3-Small	2.5M	~5ms	案例 8 已使用，精度略低
EfficientNet-B0	5.3M	~10ms	也可用

选择 ResNet18 的理由：- **与已有案例差异化**：案例 7 用 ResNet50 做特征提取，案例 8 用 MobileNetV3-Small 做手势分类，这里用 ResNet18 做场景分类——三种不同的模型架构 - **5 分类任务足够**：不需要 ResNet50 的 2048 维特征，ResNet18 的 512 维全连接层足够 - **torchvision 内置**：与案例 7 相同，权重自动从 PyTorch CDN 获取

场景分类流程

```
flowchart TD
    INPUT["输入图像\nBGR (H, W, 3)"] --> RESIZE["cv2.resize\n-> 224×224"]
    RESIZE --> RGB["cv2.cvtColor\nBGR -> RGB"]
    RGB --> NORM["归一化\npixel/255 -> (pixel-mean)/std\nImageNet 统计值"]
    NORM --> TENSOR["-> NCHW (1, 3, 224, 224)"]
    TENSOR --> CHECK{"NPU 可用?"}

    CHECK -->|"是 OM 模型"| NPU_INFER["NPU 推理\nacl.mdl.execute()"]
    CHECK -->|"否 回退"| CPU_INFER["CPU 推理\ntorch model()"]

    NPU_INFER --> SOFTMAX["Softmax -> 概率分布"]
    CPU_INFER --> SOFTMAX

    SOFTMAX --> OUTPUT["输出\n场景类别 + 置信度 + 驾驶建议"]
```

预处理步骤与案例 7、案例 8 完全一致：缩放、色彩转换、ImageNet 归一化、维度重排。

6.2.5. 模型转换与昇腾部署

```
flowchart TD
    subgraph EXPORT["1. ONNX 导出（开发机 / 昇腾设备）"]
        TORCH["torchvision\nresnet18(IMAGENET1K_V1)\nmodel.fc = Linear(512, 5)"]
        ONNX["resnet18_scene.onnx\n输入 (1, 3, 224, 224)\n输出 (1, 5)"]
    end

    subgraph CONVERT["2. ATC 转换（昇腾设备）"]
        OM["resnet18_scene.om\nAscend 离线模型 ~45MB"]
    end

    subgraph DEPLOY["3. 推理部署"]
        APP["app.py -> SceneClassifier\n-> AscendModel.execute()"]
    end
```

```
    TORCH -->|"torch.onnx.export()\nopset=11"| ONNX
    ONNX -->|"atc --framework=5\n--soc_version=Ascend310B4"| OM
    OM -->|"acl.mdl.execute()"| APP
```

步骤 1：导出 ONNX

```
python3 prepare_models.py --onnx-only
```

这一步构建 ResNet18 模型，将原本 1000 类的 ImageNet 分类头替换为 5 类驾驶场景分类头，然后用固定输入形状 `(1, 3, 224, 224)` 导出 ONNX。

步骤 2：ATC 转换

```
# 在昇腾设备上运行
python3 prepare_models.py
```

等效的 atc 命令：

```
atc --model=models/resnet18_scene.onnx \
    --framework=5 \
    --output=models/resnet18_scene \
    --soc_version=Ascend310B4 \
    --input_format=NCHW \
    --input_shape=input:1,3,224,224
```

关于训练权重

本项目的 `prepare_models.py` 会生成一个基线 `.pth` 文件（ImageNet 骨干 + 随机初始化的 5 类分类头）。**这个基线模型的预测是随机的**——要获得有意义的场景分类结果，需要在驾驶场景数据集上训练。

这是因为：- 手动标注 5 类驾驶场景数据集超出了本书的范围 - 训练流程已在案例 8 中完整演示（HaGRID + MobileNetV3-Small）- 读者可以参考案例 8 的 `train.py` 模式，用驾驶场景数据集（如 BDD100K、KITTI）自行训练

CPU 回退模式下，`SceneClassifier` 会自动检测 `.pth` 文件，加载训练好的权重。如果只有基线模型，会在启动时打印警告。

6.2.6. Web 界面

```
flowchart TD
```

```
    subgraph UI["Gradio Blocks 界面"]
        TAB1[" 道路感知 (Tab 1)"]
        TAB2[" 系统信息 (Tab 2)"]
    end

    subgraph TAB1_DETAIL["感知 Tab"]
        UPLOAD["gr.Image\n上传道路图像"] --> PERCEIVE["perceive()"]
        PERCEIVE --> LANE["LaneDetector.detect()\n车道线检测 (CPU)"]
        PERCEIVE --> SCENE["SceneClassifier.classify()\n场景分类 (NPU/CPU)"]
        LANE --> OVERLAY["车道线叠加图"]
        SCENE --> REPORT["Markdown 感知报告\n场景 + 置信度 + 驾驶建议"]
    end

    subgraph TAB2_DETAIL["信息 Tab"]
        SYS["系统信息\n模型状态 / 后端 / 参数"]
    end

    TAB1 --> TAB1_DETAIL
    TAB2 --> TAB2_DETAIL
```

两个页签

1. **道路感知**：上传道路图像（或使用示例图片），系统同时执行车道线检测和场景分类。左侧显示输入图像，右侧显示感知报告（车道检测状态、场景标签、置信度进度条、Top-3 预测、驾驶建议），下方显示车道线叠加效果图。
2. **系统信息**：展示当前推理后端（NPU/CPU）、模型文件状态、5 类场景列表及驾驶建议、车道线检测参数（Canny 阈值、Hough 阈值等）。

双后端

与案例 7、案例 8 相同，`SceneClassifier` 使用懒加载 + 自动后端检测：

```
_classifier = None

def get_classifier():
    global _classifier
    if _classifier is None:
        _classifier = SceneClassifier()  # 自动检测 NPU/CPU
    return _classifier
```

6.2.7. 用户手册

部署

1. 将项目代码拷贝到昇腾 310B 设备

2. 运行 `bash setup.sh` 安装依赖并导出模型
3. 启动服务：`python3 app.py`
4. 浏览器打开 `http://<设备IP>:7860`

使用

1. 在「道路感知」页签上传一张包含道路的图像
2. 观察车道线检测结果——绿色线条应覆盖路面上的车道标线
3. 查看右侧的场景分类结果和驾驶建议
4. 如果车道线检测不理想，可以在 config.py 中调整参数

调整车道线参数

参数	默认值	调高效果	调低效果
`CANNY_LOW/HIGH`	50 / 150	减少边缘噪点	检测更多边缘
`HOUGH_THRESHOLD`	50	只保留强直线	检测弱直线
`HOUGH_MIN_LINE_LEN`	40	过滤短线段	保留短线段
`HOUGH_MAX_LINE_GAP`	100	容忍更大断连	不连接断裂线
`ROI_TOP`	0.6	ROI 更小（忽略远处）	ROI 更大（包含远处）

故障排除

问题	可能原因	解决方法
车道线检测不到	图像中无清晰车道线	使用有标线的道路图像，调整 Canny 阈值
车道线位置偏差大	弯道 / 虚线	Hough 直线模型对弯道敏感，可以降低 `HOUGH_THRESHOLD`
场景分类随机	未加载训练权重	需要在驾驶场景数据集上训练，或使用案例 8 的模式训练
NPU 初始化失败	CANN 环境未配置	`source /usr/local/Ascend/ascend-toolkit/set_env.sh`
ATC 转换失败	soc_version 不匹配	`npu-smi info` 查看版本

如何训练场景分类模型

参考案例 8 的 `train.py` 模式：1. 准备驾驶场景数据集（按 5 个类别分文件夹存放图像）2. 修改案例 8 的 `train.py`，将模型替换为 ResNet18，类别数改为 5 3. 训练完成后将 `.pth` 文件放到 `models/resnet18_scene.pth` 4. 运行 `python3 prepare_models.py --force` 重新导出 ONNX/OM

与案例 2 的配合

案例 2 提供了完整的 SSD 目标检测 + DeepSORT 跟踪。读者可以将案例 2 的 `ssdlite/` 模块集成到本案中，实现：

1. **车道线检测**（本案，经典 CV）
2. **目标检测**（案例 2，SSD on NPU）——检测行人、车辆、交通标志
3. **场景分类**（本案，ResNet18 on NPU）——判断道路类型
4. **目标跟踪**（案例 2，DeepSORT）——跟踪移动物体

这四项共同构成一个完整的智能小车视觉感知栈。

6.3. 源代码结构

```
case6/
├── app.py                  # Gradio Web 界面入口
├── lane_detector.py         # 经典 CV 车道线检测
├── scene_classifier.py      # NPU/CPU 双后端场景分类
├── config.py                # 配置常量
├── prepare_models.py        # ONNX 导出 & ATC OM 转换
├── setup.sh                 # 一键环境安装
├── requirements.txt         # Python 依赖列表
├── data/
│   └── .gitkeep
├── models/                  # 模型文件目录 (.pth / .onnx / .om)
└── README.md                # 快速开始指南
```

模块间的调用关系：

```
flowchart TB
    APP["app.py\nGradio 界面入口"] --> LD["lane_detector.py\nLaneDetector\n· detect()\n· draw_overlay()"]
    APP --> SC["scene_classifier.py\nSceneClassifier\n· classify()\n· preprocess()"]
    APP --> CFG["config.py\n· SCENE_CLASSES\n· IMAGE_SIZE\n· 车道线参数"]

    SC --> AR["AscendResource\nacl.init / device / context"]
    SC --> AM["AscendModel\nacl.mdl.execute()"]
    SC --> TORCH["torchvision\nresnet18(IMAGENET1K_V1)"]

    LD --> CV["cv2\nCanny / Hough / GaussianBlur"]

    PREP["prepare_models.py\n模型转换"] --> TORCH
    PREP --> CFG

    AR --> ACL["acl (CANN)"]
    AM --> ACL
```

与之前案例的继承关系：

- `AscendResource` / `AscendModel` 沿用案例 8 的同一套模式
- `config.py` 的结构与案例 7/案例 8 一致（`BASE_DIR` + `os.path.join`）
- `app.py` 的 Gradio 懒加载模式与案例 7/案例 8/案例 9 一致
- `prepare_models.py` 的 ONNX -> ATC 流程与案例 7/案例 8 一致
- `LaneDetector` 是本书第一个纯经典 CV 类——无需任何模型文件，纯 OpenCV 实现
- 本项目**不需要**训练脚本——场景分类头尚未训练，但推理框架完整，参考案例 8 即可完成训练

6.4. 效果演示

6.4.1. 预期效果

功能	预期表现	备注
车道线检测	清晰标线检测率 > 90%	对弯道、虚线、阴影敏感
场景分类 (已训练)	5 类准确率 > 85%	需要训练，基线模型随机
端到端处理时间	车道线 <5ms + 推理 ~12ms (NPU)	总计 ~20ms
车道线检测范围	画面下部 40% 区域	ROI 参数可调

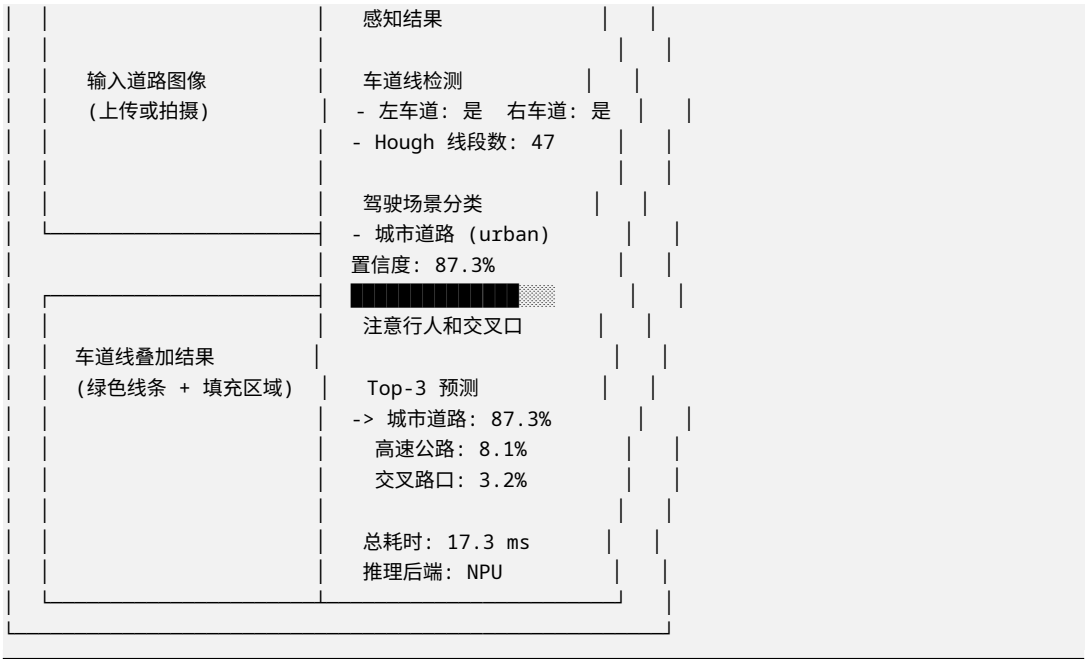
6.4.2. 性能指标

指标	NPU (Ascend 310B)	CPU (PyTorch)
车道线检测	<5 ms	<5 ms
场景分类	10-15 ms	25-40 ms
模型大小 (.om)	~45 MB	N/A

6.4.3. 浏览器中的效果

Gradio 界面在浏览器中的预期布局：

```
┌──────────────────────────────────────────────────────────┐
│  智能小车视觉感知                                        │
│ 车道线检测 (经典 CV) + 驾驶场景分类 (ResNet18 on NPU) │
├──────────────────────────────────────────────────────────┤
│ [ 道路感知]  [ 系统信息]                                 │
│                                                          │
│  ┌──────────────────────┬──────────────────────────┐     │
```



6.4.4. 如何验证系统正常工作

1. 打开浏览器访问 `http://127.0.0.1:7860`
2. 切换到「道路感知」页签
3. 上传一张包含清晰车道线的道路图像
4. 确认车道线叠加图中出现绿色车道线标记
5. 确认右侧感知报告中显示场景分类结果（如未训练则显示随机结果，这是预期行为）
6. 切换到「系统信息」页签，确认后端状态和参数显示正确
7. 在昇腾设备上，确认后端显示”NPU (Ascend 310B)”

7. 案例 7：智能相册

本教程从问题建模、模型迁移、NPU 运行时、相册服务器到触摸屏和电子纸输出，完整解释 Case7 的设计与实现。

7.1. 案例目标与问题定义

传统电子相册只会按文件名或固定顺序轮播。Case7 要解决的是一个更完整的问题：照片由手机上传到局域网服务器，服务器在 Ascend 310B 上建立语义索引，再根据中文/英文查询、日期、天气和最近显示历史选择合适照片；同一份选择结果既服务开发板上的 10 寸触摸屏，也服务已登记的 Waveshare ESP32-S3-PhotoPainter 和 Seeed Studio reTerminal E1002 两类 ESP32 电子纸终端。当前已有的实机记录按设备分别标注，不能把一台设备的历史结果当作另一台已验收。Orange Pi 的 SPI/GPIO 直连微雪 E6 是独立的本机输出路径，不是第三种可注册 ESP32 相册。

因此，Case7 不是单独的 “图片分类脚本”，而是一条端到端链路：

1. **照片进入**：手机上传或扫描目录，经过内容 hash、解码、尺寸和路径安全检查；
2. **照片理解**：NPU 图像编码器生成归一化 embedding，SQLite 保存元数据，FAISS 保存逐模型检索结构；
3. **照片选择**：中文/英文文本查询、相似图查询和日期/天气智能选图使用相应模型空间；
4. **照片显示**：FastAPI 静态页面、JPEG 设备接口和 E6 dry-run 使用同一个选择状态；
5. **设备协同**：ESP32 通过能力握手、manifest、ETag 和按需 content 获取照片。

生产服务只接受已准入的 Ascend OM，不回退 CPU/PyTorch 推理。CPU 负责图像解码、OpenCV Haar 人数计数、FAISS、JPEG/E6 准备和离线 ONNX 数值参考。

7.2. 系统总体架构

下面的架构图展示手机、触摸屏和远程终端如何共享 310B 上的 FastAPI、索引和 NPU 资源。

```
flowchart LR
    accTitle: 智能相册系统架构
    accDescr: 手机、触摸屏和远程显示设备通过 FastAPI 访问 SQLite/FAISS 索引、Ascend 310B NPU 和天气服务

    phone[手机浏览器]
    touch[10 寸触摸屏]
    esp32[Waveshare PhotoPainter / Seeed E1002]
    e6[Orange Pi SPI/GPIO 直连微雪 E6]

    subgraph server[Orange Pi AIpro / Ascend 310B]
        api[FastAPI 服务]
        selector[智能选图器]
        runtime[PyACL NPU 运行时]
        storage[(SQLite 元数据)]
        faiss[(逐模型 FAISS 索引)]
        renderer[按需渲染器]
    end

    weather[Open-Meteo]
    models[MobileCLIP / Chinese-CLIP / ResNet50 OM]

    phone -->|上传、搜索、配置| api
    touch -->|图库、控制、设置| api
    esp32 -->|握手、manifest、ETag| api
    api -->|未来 SPI/GPIO| e6
    api --> storage
    api --> selector
    selector --> runtime
    runtime --> models
    runtime --> faiss
    selector --> storage
    api --> renderer
    renderer -->|JPEG 或 E6 帧| esp32
    renderer -->|浏览器 JPEG| touch
    selector -->|定时刷新| weather
    api --> faiss
```

7.2.1. PDF 版结构图

下面两张图由 Graphviz DOT 源文件生成。Markdown 中保留 Mermaid 版本便于在线阅读，PNG 版本用于 VuePress/Pandoc 电子书排版；修改结构时先改 DOT，再重新生成 PNG。

图 7-1 网络结构：手机、触摸屏和 PhotoPainter 通过 FastAPI 访问 SQLite/FAISS、Ascend 310B NPU、按需渲染器和天气服务。图源：`img7/network-architecture.dot`。

图 7-2 程序流程：服务启动完成 CANN、模型 hash 和索引恢复后，分别处理上传、语义搜索、显示控制以及 ESP32/PhotoPainter 的 ETag 拉图请求。图源：`img7/program-flow.dot`。

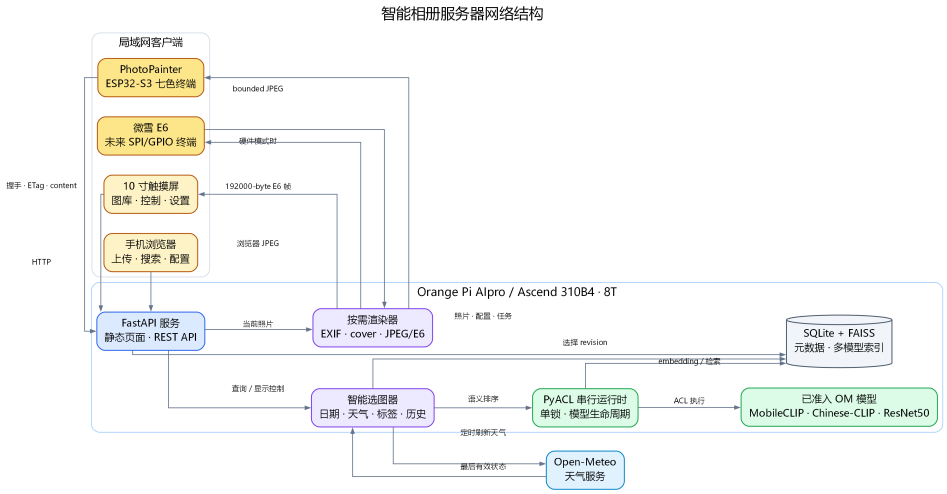


图 7.1. 局域网相册服务器网络结构图

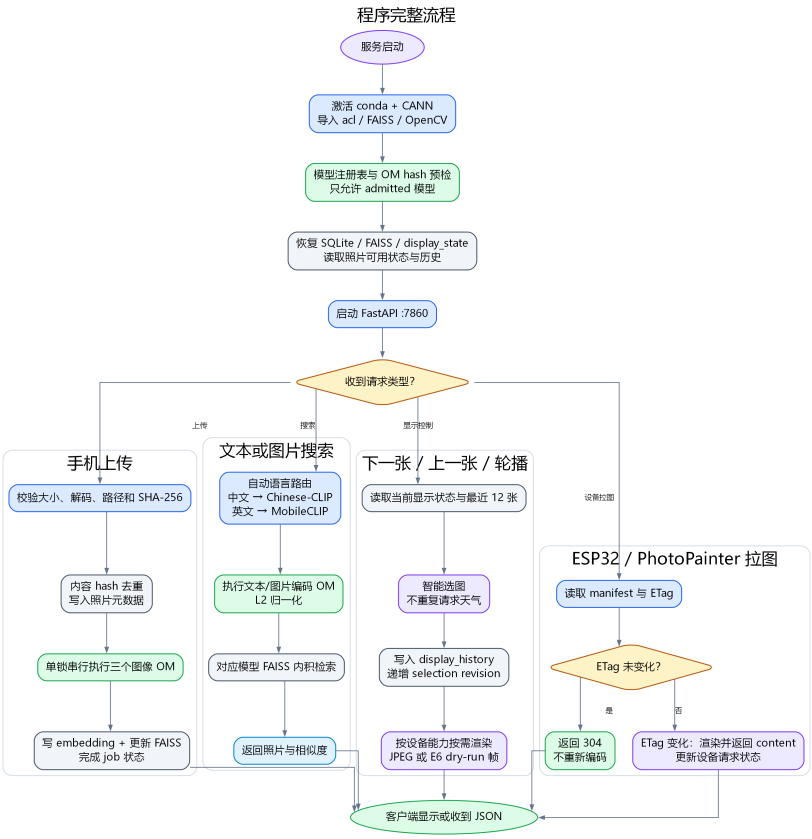


图 7.2. 程序完整流程图

7.2.2. 数据流

```
flowchart TB
    accTitle: 相册数据处理流程
    accDescr: 照片从上传和校验开始，经过三模型索引，最终被文本检索或智能选择并按设备能力渲染

    input([上传或扫描照片]) --> validate[校验路径、解码、大小和 SHA-256]
    validate --> metadata[写入 EXIF、时间、尺寸；保留兼容元数据]
    metadata --> encode[三模型 NPU 图像编码]
    encode --> vector[(SQLite embedding + FAISS)]
    query[文本、图片或轮播时隙] --> route{选择模型或策略}
    route --> search[归一化向量内积检索]
    route --> select[日期、天气、兼容元数据和历史评分]
    search --> result[当前照片 revision]
    select --> result
    result --> render[尺寸、EXIF、叠加和 JPEG/E6 渲染]
    render --> output[触摸屏、手机、ESP32 或 E6]
```

7.2.3. 服务启动与请求时序

```
sequenceDiagram
    accTitle: NPU 服务启动时序
    accDescr: 服务启动先加载 CANN 和注册表，再恢复 SQLite/FAISS 状态，最后接受手机或设备请求

    participant launcher as 启动脚本
    participant cann as CANN/PyACL
    participant registry as 模型注册表
    participant index as SQLite/FAISS
    participant api as FastAPI
    participant client as 客户端

    launcher->>cann: import acl 与设备初始化
    cann-->>launcher: Ascend 310B 可用
    launcher->>registry: 校验 admitted OM 与 SHA-256
    registry-->>launcher: 三个模型准入
    launcher->>index: 恢复照片、embedding 和显示状态
    index-->>launcher: 500 张 COCO-CN 图库
    launcher->>api: 启动 7860
    client->>api: GET /api/health 或 POST /api/search/text
    api->>cann: 单锁执行图像/文本 OM
    cann-->>api: 归一化 embedding
    api->>index: FAISS 检索或读取选择状态
    index-->>api: 结果与 revision
    api-->>client: JSON、JPEG 或 304
```

7.3. 硬件与软件基础

层	组件	在本案例中的职责
计算	Orange Pi AIpro / Ascend 310B4 / 8T	执行图像和文本 OM
工具链	CANN 8.x、ATC、PyACL	转换、加载和执行模型；准入报告记录板端 `version.cfg` 的实际版本
服务	FastAPI、Uvicorn、原生 HTML/CSS/JavaScript	手机、触摸屏和设备 HTTP 接口
存储	SQLite、faiss-cpu 1.7.4	元数据真源和逐模型向量检索
图像	Pillow、OpenCV	解码、EXIF、尺寸检查和 Haar 人数计数
显示	QDtech MPI1001、PhotoPainter、微雪 E6	本机管理、六色终端和 E6 协议输出

310B 的 NPU 只负责已准入 OM 的推理。相册数据、FAISS 文件、设备状态和配置都留在板端；手机和 ESP32 只通过 HTTP 获取受控结果。

7.4. 核心数据模型

`photos` 保存原图生命周期，关键字段包括 **`id`**、`filepath`、`filename`、`sha256`、`size_bytes`、`mtime`、`available`、`width`、`height`、`mime_type`、`capture_time`、`capture_time_source`、`upload_time`、`tags`、`face_count` 和 `deleted_at`。普通上传原图位于系统用户的 `~/Pictures/ai-album/imports/` ↪，临时 multipart 文件位于 `~/Pictures/ai-album/.upload-tmp/`；仓库 `shared/photos/` 仅承载 COCO-CN 和旧数据兼容读取，避免个人照片进入发布目录。其中 `tags` 只保留为旧数据库迁移兼容元数据；常规上传不要求或写入手工标签，当前工程也没有部署自动图像标注模型。

`embeddings` 使用 `(photo_id, model_id)` 作为主键，保存维度和归一化向量。每个模型有独立的 `IndexIDMap2(IndexFlatIP)`，FAISS 的 ID 就是 SQLite 的 `photo_id`，因此不会把不同模型空间混合。

`display_state` 和 `display_history` 保存本机触摸屏、两个 PhotoFrame profile 与本机 E6 输出的当前照片、暂停状态、时隙、策略 revision、selection revision 和最近 12 张历史。`jobs` 保存手机上传后的串行索引进度。

7.5. 多模型语义检索方法

7.5.1. CLIP 的共同向量空间

CLIP 使用图像编码器和文本编码器把两个模态投影到同一个向量空间。图像查询先编码成向量，文本查询也编码成向量，二者做 L2 归一化后使用内积近似余弦相似度。相似度高表示模型训练中认为图像和文本语义更接近。

Case7 不把三个模型的数值直接相加：不同训练数据、投影层和维度形成不同坐标空间。模型 ID 是索引隔离和 API 路由的边界。

7.5.2. 参考实现与工程取舍

本案例参考了 Andy 的个人图像搜索实践和 `atarss/clip-image-search`。博客展示了一个很重要的工程事实：图片检索不是“把图片文件名交给搜索框”，而是先递归导入照片、提取图像向量和尺寸等元数据，再用归一化向量做相似度排序；同模态图像查询的分数通常高于跨模态文字查询，分数只能用于同一模型空间内排序，不能跨模型直接比较[1]。上游仓库的实现把文件导入、CLIP 编码、元数据保存和查询服务拆成两个阶段，并在查询时先按 `width`、`height`、扩展名等字段过滤 MongoDB，再按批次计算余弦相似度[2]。

Case7 保留这条数据流，但替换了不适合 310B 服务器的组件：

参考项目做法	Case7 实现	取舍原因
OpenAI CLIP 常驻 PyTorch	已准入 MobileCLIP/Chinese-CLIP OM，经 PyACL 串行执行	NPU 是生产推理边界，CPU 不作 fallback
MongoDB 保存图片和向量	SQLite 保存照片元数据与 embedding，FAISS 保存检索缓存	减少常驻服务，保留可审计的单文件真源
导入时可复制到 hash 目录	上传原图放在系统 `Pictures` 受管目录，SHA-256 去重	防止个人照片混入发布目录，原图生命周期可控
查询时拉取全部向量并分块计算	无筛选时使用每模型 `IndexIDMap2(IndexFlatIP)`；有筛选时先 SQLite 预筛选再计算候选内积	兼顾常用路径延迟和元数据过滤正确性
早期网页 Demo	原生 FastAPI 与触摸屏/手机页面	当前服务同时支持本机显示和 ESP32，不依赖额外 UI 运行时
OCR 作为后续实验	本版本不自动 OCR、不生成手工标签	不把未准入的 CPU 模型混入 NPU 相册主链路

因此，本案例借鉴的是可验证的检索分层和元数据预筛选思想，不复制 MongoDB 或未经准入的 OCR。`photo_index.py` 的 `search_vector()` 在带有尺寸、格式或人数条件时先构

[1] Andy. *基于 CLIP 模型特征搭建简易的个人图像搜索引擎*. https://andy9999678.me/blog/archives/239

[2] atarss. *clip-image-search*. https://github.com/atarss/clip-image-search

造候选集合，再进行归一化内积；没有条件时仍直接查询 FAISS。两条路径都返回相同的 `SearchResult` 合同，前端只需要消费照片 ID、文件名和分数。

> **边界：**上游仓库的 README 将 FAISS、EXIF 和多语言列为 TODO；Case7 已经分别用 FAISS、SQLite/EXIF 和双语模型实现，但这不表示上游项目已经提供这些能力，也不表示两个项目的模型权重或分数可以互换。

7.5.3. MobileCLIP-S0

MobileCLIP-S0 面向英文和通用语义，图像输入固定为 `1x3x256x256`，文本输入固定为 `1x77`，输出 embedding 为 512 维。图像和文本组件分别导出 ONNX，再分别转换成 OM。

```
flowchart LR
    accTitle: MobileCLIP 编码结构
    accDescr: MobileCLIP-S0 分别编码图像和英文文本，再投影到 512 维归一化空间进行检索

    image[RGB 图像 256x256] --> image_encoder[MobileCLIP 图像编码器]
    text[英文文本 77 tokens] --> text_encoder[MobileCLIP 文本编码器]
    image_encoder --> image_projection[图像投影层]
    text_encoder --> text_projection[文本投影层]
    image_projection --> image_vector[512 维 L2 归一化]
    text_projection --> text_vector[512 维 L2 归一化]
    image_vector --> similarity[FAISS 内积或余弦相似度]
    text_vector --> similarity
```

7.5.4. Chinese-CLIP RN50

Chinese-CLIP RN50 面向中文语义，图像输入固定为 `1x3x224x224`，文本输入固定为 `1x52`，输出 embedding 为 1024 维。文本 tokenizer 使用 BERT 风格的 `vocab.txt`、`[CLS]`、`[SEP]`、`[PAD]` 和 `[UNK]` 合同；tokenizer 的物理行读取规则必须和固定上游实现一致。

```
flowchart LR
    accTitle: Chinese-CLIP 编码结构
    accDescr: Chinese-CLIP RN50 使用 ResNet 图像分支和 BERT 风格中文文本分支，输出 1024 维归一化
    ↪ 向量

    image[RGB 图像 224x224] --> rn50[ResNet50 图像编码器]
    text[中文文本 52 tokens] --> bert[BERT 风格文本编码器]
    rn50 --> image_projection[图像投影层]
    bert --> text_projection[文本投影层]
    image_projection --> image_vector[1024 维 L2 归一化]
    text_projection --> text_vector[1024 维 L2 归一化]
    image_vector --> similarity[Chinese-CLIP FAISS 内积]
    text_vector --> similarity
```

7.5.5. ResNet50 经典相似图

ResNet50 输出 2048 维视觉特征，用于兼容传统“以图搜相似图”模式。它没有文本编码器，也不提供中文/英文语义能力。它必须使用独立 FAISS 文件，不能和任意 CLIP embedding 混合。

```
flowchart LR
    accTitle: ResNet50 相似图结构
    accDescr: ResNet50 只把图像映射到 2048 维视觉特征空间，用于图像到图像检索

    image[RGB 图像 224x224] --> backbone[ResNet50 卷积骨干]
    backbone --> feature[去除分类头]
    feature --> vector[2048 维 L2 归一化]
    vector --> search[ResNet50 独立 FAISS 检索]
```

7.5.6. 自动路由

文本自动模式检测中日韩字符时选择 Chinese-CLIP，否则选择 MobileCLIP；图片自动模式默认选择 MobileCLIP。用户手动指定模型时，服务检查模型已准入且查询输入合同正确。

7.6. 昇腾 310B 模型迁移方法

模型迁移分为四个可审计阶段：

```
checkpoint / tokenizer
        ↓ 离线导出
FP32 ONNX（固定 batch=1）
        ↓ 板端 ATC
Ascend310B4 mixed-FP16 OM
        ↓ PyACL / ACL
NPU embedding + FAISS 检索
```

导出阶段记录输入名称、shape、dtype、输出维度和 ONNX SHA-256。转换阶段使用 `--soc_version Ascend310B4` 和 `--precision_mode allow_fp32_to_fp16`。MobileCLIP 图像组件的当前生产 OM 已通过选择性精度扫描，使用 C0 的零节点 FP32 白名单；任何未来的例外仍必须写入 ATC 证据，而不能更换模型。

310B 板端内存有限，ATC 强制单线程：`MAX_COMPILE_CORE_NUMBER=1`、`TBE_PARALLEL_COMPILER=0`、`ASCENDC_PAR_COMPILE_JOB=0`、`TILINGKEY_PAR_COMPILE=0`、`OMP_NUM_THREADS=1` 和 `OPENBLAS_NUM_THREADS=1`。不添加 swap 或编译缓存，不开启并行图编译；转换器只在旧版确实提供该参数时选择 `--enable_graph_parallel=0`。CANN 8.x 的 `--ac_parallel_enable` 是动态 shape 执行阶段的 AI CPU/Core 并行选项，不等价于图编译开关，不能把它写成 ATC 并行门禁。

MobileCLIP 和 Chinese-CLIP 文本图目前登记为 `int64` 输入。这个类型是否能被目标 CANN 版本的 ONNX 到 ATC 链路接受，必须用板端最小文本图预检；若失败，应重新导出并验证 `int32` 合同，同时更新 tokenizer、ACL 输入检查、hash 和准入报告，而不是只改模型清单。

准入脚本先验证 ACL 初始化，再验证输入字节数、dtype、输出字节数、维度和有限值，最后比较 ONNX 与 OM 的归一化向量，余弦相似度门槛为 `0.995`。只有 `models/registry.json` 中 hash 一致且 `status=admitted` 的 OM 才会被服务加载。

7.6.1. 目标 SoC 与运行时版本

MobileCLIP 的 OM 绑定 ATC 转换时指定的 `--soc_version`，不能默认跨不同 310B SoC 通用。当前服务只使用与目标板卡匹配、已完成准入的 OM。若更换板卡或 CANN 运行时，必须在新环境重新执行 ONNX 合同、ATC、ACL 数值和检索验证；未取得当前报告时，不在教程中写入旧板卡数字。

`npu-smi info` 用于记录板卡、驱动和固件字段；固件字段无法读取时原样记录，不从驱动版本推断。CANN Toolkit/ATC、板端 Runtime/PyACL、驱动和 NPU 固件必须按厂商兼容矩阵配套。版本升级应先查官方说明，再在隔离目录验证，不覆盖生产 OM、注册表或索引。

7.6.2. 选择性 FP32/FP16 与误差定位

`allow_fp32_to_fp16` 是一种转换许可，不是“整张图强制 FP16”。图像浮点输入和 embedding 输出仍按 FP32 ONNX 合同处理；文本 token 输入是 `int64`。ATC 会在允许的节点上选择 FP16 实现，未被转换或被 `keep_dtype` 白名单保护的节点仍可沿用 FP32。这个区别很重要：MobileCLIP 图像分支当前生产 OM 的组件级策略是 `allow_fp32_to_fp16` ↪，并已推广 C0 的零节点 FP32 白名单。推广前的 `must_keep_origin_dtype` OM 及其 `mobileclip_s0_image_keep_dtype.cfg` 仅保留为诊断和候选节点清单。MobileCLIP 文本、Chinese-CLIP 两个分支和 ResNet50 图像分支使用模型级 `allow_fp32_to_fp16`。详细的命令、清单和证据字段见 模型流水线与 NPU 准入。

诊断的基本方法是比较同一输入在 ONNX 参考执行器和候选 OM 各中间输出的方向，而不是只看最终 embedding。现有诊断报告在 `network.4` 的加法节点记录余弦 `0.9994069` ↪，在 `network.5/proj/proj.0/lkb_reparam/Conv` 记录 `0.7932538`；同一分组还出现 SE `Sigmoid` `0.7223154`、主乘法 `0.2903332` 和 `proj.1` 卷积 `0.5836687`。另一次诊断的 `network.5/proj/proj.1/` ↪ `activation/Mul_1` 为 `0.2216825`，最终 embedding 为 `0.2551974`。这些数字来自诊断图的单个输入，是误差传播线索而不是因果证明。一次整组件 `force_fp32` 候选也记录了 `0.4368718` 的失败，因此“全部 FP32”并不能被当作自动修复。

选择性精度扫描按从小到大的白名单逐级验证：

候选	FP32 保留范围	本次结果
C0	空白名单	通过并推广为生产 OM
C1	network.5/.../lkb_reparam/Conv	通过，因 C0 更快而未推广
C2	C1 加 SE 的池化、两层卷积、激活、门控和乘法	C0 已满足门槛，未执行
C3	C1 加 GELU/Erf、主乘法链和 proj.1 卷积	C0 已满足门槛，未执行
C4	C2 + C3，再保护两组注意力 MatMul/Softmax 和 head MatMul	C0 已满足门槛，未执行

候选模型的流程如下。候选 OM、keep-dtype 文件和报告必须位于独立的 reports/precision_sweep/mobileclip_s0_image_precision/<candidate>/ 目录，不能覆盖生产 OM、注册表、SQLite、FAISS 或既有准入报告。

```
flowchart TB
    accTitle: 选择性精度准入流程
    accDescr: MobileCLIP 图像候选从单线程 ATC 转换开始，经过数值、检索和性能门禁后才可能替换生产模型

    source([生产 ONNX]) --> candidate[生成 C0 到 C4 白名单]
    candidate --> convert[单线程 ATC 隔离转换]
    convert --> numeric{36 个输入余弦 >= 0.995?}
    numeric -->|否| reject[保留日志并拒绝候选]
    numeric -->|是| recall{500 张图库 Recall 不下降?}
    recall -->|否| reject
    recall -->|是| latency{P50 <= 生产基线 90%?}
    latency -->|否| reject
    latency -->|是| admit[候选可推广]

    classDef source_style fill:#ede9fe,stroke:#7c3aed,stroke-width:2px,color:#3b0764
    classDef process_style fill:#dbeafe,stroke:#2563eb,stroke-width:2px,color:#1e3a5f
    classDef decision_style fill:#fef9c3,stroke:#ca8a04,stroke-width:2px,color:#713f12
    classDef reject_style fill:#fee2e2,stroke:#dc2626,stroke-width:2px,color:#7f1d1d
    classDef admit_style fill:#dcfce7,stroke:#16a34a,stroke-width:2px,color:#14532d

    class source source_style
    class candidate,convert process_style
    class numeric,recall,latency decision_style
    class reject reject_style
    class admit admit_style
```

每个候选需要 32 张按 image_id 固定的 COCO-CN 图片和种子 310 至 313 生成的 4 张确定性 uint8 BGR 合成图；合成图也必须经过生产的 NpuEmbeddingBackend.preprocess_image，而不是直接把标准正态张量送入模型。输出必须是 512 维有限向量，36 次归一化余弦全部达到 0.995。随后在临时索引中重建 500 张图库，以同轮生产 OM 比较既有 20 条英文查询的 Recall@1/3/5；临时向量和索引在报告完成后删除。最后在 Ascend310B4 板端以单线程、20 次预热、100 次计时、3 轮重复测量 P50/P95。

每次候选实验都必须把板卡型号、CANN 版本、ONNX/OM SHA-256、ATC 日志、36 个数值样本、500 张图库 Recall@1/3/5 和三轮 P50/P95 写入独立报告。只有当前报告同时通过数值、检索和性能门槛，候选才可以进入注册表；没有报告支撑的数字在教程中一律不预填。`Health: Alarm` 只作为诊断字段，不能单独改变准入结论。候选实验的临时向量和索引在报告完成后删除。

7.7. NPU 运行时与串行资源管理

`embedding_backend.py` 用一个 ACL 资源管理器拥有 device、context、stream 和 model。`ModelManager` 按需加载图像/文本 OM；切换模型前释放旧 model，避免多个大模型同时占用内存。获取模型、构造输入、执行和释放由同一把锁保护。

服务的索引任务、ATC 转换和单次 embedding 都是串行的。这样牺牲并发换取可预测的 NPU 内存峰值和板端稳定性。CPU/PyTorch 不作为生产 fallback，因为 fallback 会掩盖 OM 缺失、ACL 初始化失败或模型合同错误；这些错误必须直接返回并写入健康状态。

7.8. 智能选图与天气方法

候选照片必须可用且未删除。默认总分为：

```
总分 = 0.60 * Chinese-CLIP 语义相似度
     + 0.25 * 日期、月份、星期和时段匹配
     + 0.15 * 天气与兼容元数据匹配
```

照片时间优先 EXIF `DateTimeOriginal`，缺失时使用上传时间。天气默认来自 Open-Meteo；请求失败时保留最后有效状态。历史迁移照片若包含兼容 `tags`，可参与天气匹配；新上传照片不接受手工标签，缺少该元数据时该项得分为零。最近显示的 12 张照片进入重复抑制窗口，无匹配时回退到最新照片。

天气在后台按 `weather.refresh_seconds` 更新；**`next`**、`previous`、图库点选和相似图查询复用已有天气状态，不主动访问天气服务。服务器在空闲调度周期中用已准入 Chinese-CLIP 文本编码器查询语义空间，并把 FAISS 排序后的照片 ID 放入易失的内存候选队列；手动 **`next`** 消费这份 NPU 语义计划，因此不是按文件名绕过模型。队列因天气、配置、图库变化而失效或耗尽时，下一次操作会在缓存天气下补做一次串行 NPU 排序。这样连续点击不会重复请求天气或反复加载模型，同时每个候选顺序仍由 NPU 语义分数参与决定。触摸屏自动选图默认每 60 秒执行，普通电脑网页每 30 秒轮询显示元数据；网页比较 ETag，未变化时不重新下载 JPEG。电子纸物理刷新使用独立的 30 分钟周期，也可明确设置为 10 分钟，暂停相册会暂停这些自动动作。

手动切图的控制请求和图像显示是两个阶段：前者提交 NPU 预选队列中的照片 ID（冷队列时先完成一次 NPU 文本推理和 FAISS 排序），浏览器随后按实际视口请求一次性 JPEG。高分辨率原图的 EXIF 旋转、缩放和 JPEG 编码通常比温热队列的选择状态更新更耗时；服务器使用 `Image.draft` 作为当前 JPEG 解码的尺寸提示，但不写入缩略图或其他派生缓存。首页不预加载整个图库，图库面板打开后才创建网格，网格图片使用浏览器懒加载和异步解码，避免原图请求与主屏渲染争用板端资源。

`selection_revision` 在上传、天气变化、配置变化和轮播时隙到达时递增。每个 profile 的 current photo、pause 状态和 history 写入 SQLite，重启后恢复而不是重新随机选择。

7.9. 服务器化设计

7.9.1. 统一端口约定

Case7 的 310B 服务统一使用 `7860`：手机、10 寸触摸屏、ESP32 的 PhotoFrame 图片 URL、`curl` 示例和教学部署命令均写作 `http://<BOARD_IP>:7860/`。这是普通用户进程可直接监听的非特权端口，避免为了 80 端口修改 Linux 权限、CANN 环境或额外部署代理。

ESP32 与 310B 是两个不同的 HTTP 服务：ESP32 自身的控制页面为 `http://<ESP32-IP>/`，其默认端口是 80；310B 相册服务器仍为 7860。发布脚本可能短暂使用一个仅回环可见的 smoke 端口验证候选版本，但该端口不写入设备、不出现在用户操作流程中。

7.9.2. 上传任务状态机

手机的 `POST /api/photos/upload` 只负责接收和保存受管原图，然后返回 `job_id`。单线程任务依次完成解码、元数据写入、三模型 embedding 和 FAISS 更新；手机轮询 `/api/jobs/{job_id}` 获取 `queued`、`running`、`completed` 或 `failed`。

7.9.3. 配置与显示状态

`GET/PATCH /api/config` 使用 revision 乐观锁和原子 JSON 写入。手机可以修改时区、轮播、天气坐标、重复抑制、JPEG 参数、E6 参数和文件名水印开关，但不能执行 shell、修改任意系统路径或改变 CANN 环境。显示控制 API 持久化 `select`、**`next`**、`previous`、`pause` 和 `resume`。

7.9.4. ESP32 与 PhotoPainter

设备先发送显示能力，服务器返回 device ID、轮询周期和 manifest。后续以 ETag 条件请求 manifest/content；JPEG profile 按设备尺寸、方向和字节上限按需编码，E6 profile 输出固定 800x480、六色、192000-byte 帧。PhotoPainter profile 返回 bounded JPEG，由上游固件继续完成六色校准、抖动和电子纸刷新。

Case7 固定记录两种 7.3 英寸设备 profile：Waveshare ESP32-S3-PhotoPainter 官方产品页 / Wiki 为 E6 六色（黑、白、绿、蓝、红、黄）800x480；Wiki Mode 1 接受 800x480 或 480x800 图像，因此内容可标记为 `landscape` 或 `portrait`；其上游板级 profile 固定 `hardware_rotation_deg=180`。Seeed Studio reTerminal E1002 官方 Wiki 为 ACeP / Spectra 6 全彩 800x480，Case7 将它固定为 `landscape`，板级补偿为 `0`。方向字段只允许 `landscape`、`portrait`，不提供用户可调的 360°、90°/270° 或安装角度；服务器 JPEG `rotation` 保持 `0`，配对和 URL Rotation 同步时才把 profile 补偿写入固件的 `display_rotation_deg`。E1002 的 `portrait` 请求必须拒绝。Seeed 的资料只确认面板规格，横屏限制是本项目的设备策略；厂商资料不替代固件识别和真实面板刷新证据。

PhotoPainter 竖屏倒置的排查

`display_orientation` 表示逻辑内容方向，`display_rotation_deg` 表示板级物理坐标补偿。官方 v2.18.0 的 Waveshare 板级头文件将补偿定义为 180 度；若服务器把该字段错误地写成 0，竖屏画面会整体上下倒置。Case7 不在 JPEG 上再次旋转，而是在注册和 `X-Config-Payload` 中发送固定补偿，并以 `X-Album-Hardware-Rotation` 回显。读取设备 `/api/config` 时应看到 Waveshare 的 `portrait/180` 或 `landscape/180` 组合；E1002 则为 `landscape/0`。这不是 NPU、EXIF 或图片内容错误，不能通过增加 90/270 度用户选项解决。

新设备在管理 API 和低层握手 API 中都必须显式携带这两个 `profile_id` 之一，JPEG 能力固定为 `["jpeg"]`；服务不会因为 800x480、设备名称或 IP 地址相同而猜测型号。缺少 profile 的记录会被标记为待确认，仍可在管理页查看，但不能取图或推进轮播，直到操作者按实物型号完成确认。

当前 LAN 部署的设备注册和取图都是 URL-only：不生成、不显示、也不要求设备令牌。服务设置私有、重新验证缓存语义和 `Vary`，不把真实文件路径暴露给设备。禁用设备返回 `404`；访问边界依赖可信局域网和服务器端设备启停状态，而不是旧固件令牌。

7.9.5. 从串口启动日志取得 PhotoPainter 的 IPv4 地址

设备网页地址由路由器 DHCP 分配，不能从设备名称、MAC 或旧的租约记录推断。实际操作时只连接当前 PhotoPainter，打开串口监视器并按一次 **BOOT** 唤醒，再按复位键观察完整启动过程：

```
$idfPython = 'C:\Espressif\tools\python\v6.0.2\venv\Scripts\python.exe'
& $idfPython -m serial.tools.miniterm COM17 115200
```

日志中出现 `No WiFi credentials found - Starting AP mode` 表示设备还在临时配网 AP；连接形如 `PhotoFrame - XXXXX` 的 2.4 GHz AP 后配置家庭网络。配网成功必须同时看到 `sta ip: ...`、`HTTP server started` 和 `Web interface available at: http://...`。保存完整串口日志，地址以本次启动打印的 IPv4 为准：

```
profile: waveshare_photopainter_73
serial: <SERIAL_PORT>
firmware: <FIRMWARE_VERSION>
sta ip: <ESP32_IP>
web: http://<ESP32_IP>/
```

在同一局域网电脑上用这个 IPv4 地址验证，而不是登录 `photoframe.local`：

```
$photoIp = '<ESP32_IP>'
curl.exe --noproxy "*" "http://$photoIp/api/system-info"
curl.exe --noproxy "*" -I "http://$photoIp/"
```

`photoframe.local` 只是可选 mDNS 别名，Windows、VPN 或路由器不支持解析时仍属正常；它不是账号、密码或必须的登录入口。310B 相册服务器地址使用 `<BOARD_IP>:7860`；旧租约记录不能代替当前串口日志中的地址。每次重启、换路由器或重新配网后都要重新读取 `sta ip`。完整的端口枚举、日志保存、DHCP 变化和故障排查步骤见仓库中的 ESP32 电子相册设备管理。

7.9.6. ESP32 深度休眠后的唤醒与 310B 发现

两类终端的“联网”“可发现”和“已配对”是三个独立状态。Waveshare PhotoPainter 从深度休眠唤醒使用 **BOOT**，Seeed reTerminal E1002 使用顶部绿色 **Wake/Refresh**；醒来后先等待 Wi-Fi、HTTP 和 mDNS 服务恢复，再在 310B 上调用 `GET /api/admin/devices/discover`。该接口只查询 `_esp32-pframe._tcp`、读取每个候选的 `/api/system-info`，并保留字面 IPv4、硬件 ID、板型和固件版本供操作者选择。它不扫描网段，也不能唤醒设备。深度休眠时 Wi-Fi、mDNS 和 HTTP 均关闭，310B 不存在可发送的网络唤醒包。

Case7 对这两类 ESP32 终端固定采用深度休眠：注册、策略更新和设备成功拉图时都写入 `deep_sleep_enabled=true`，前端不提供关闭或“调试常亮”选项。这样可以保持电池续航和实体唤醒键的语义一致；重新刷写固件后重新注册会再次修复该配置。设备必须先由实体按键或固件定时器唤醒，310B 才能发现并服务下一次 URL Rotation 请求。

发现为空时，可以在 310B 操作页面的 **设置 > ESP32 唤醒与发现**中输入该地址并点击 **读取并验证 IP**；它调用只读 `POST /api/admin/devices/probe`。也可以从串口 `sta ip:` 或路由器 DHCP 租约取得当前 IPv4，直接读取 `http://<ESP32-IP>/api/system-info`；只有 `project_name=esp32-photoframe`、`device_id` 和正确 profile 均核对后，才提交 `POST /api/admin/devices/register`。注册返回 `202/awaiting_pull` 只代表控制面配置完成；设备随后

主动请求 `http://<BOARD_IP>:7860/api/devices/<device_id>/photoframe`，才算观察到真实拉图。两台设备可能都显示 `photoframe.local`，因此不能按主机名或列表第一项自动配对。原厂 SenseCraft/Xiaozhi 固件若没有上述 API 或 mDNS 服务，必须先完成固件适配，不能仅凭外壳和二维码判定兼容。可执行的逐步命令、故障表和证据模板见 ESP32 电子相册设备管理。

E1002 的 MicroSD 兼容性由设备固件和硬件说明决定，不是 Case7 网络传输的前提。服务器不把 SD 卡作为照片缓存；存储卡异常与深度休眠策略是两个独立条件。

7.9.7. 横竖屏图像方向

照片显示先执行 EXIF Orientation 校正，再应用设备 profile 的 `orientation`（`landscape` 或 `portrait`）。默认 `orientation_mode=auto` 保持照片本身的横竖构图，只将其按 `cover` 或 `fit` 放进目标画布；这是人像照片的安全默认值。需要让最终编码帧严格匹配显示器横竖方向时，可启用 `orientation_mode=match_display`，服务会按 profile 选择 `landscape` 或 `portrait` 目标；E1002 的 `portrait` 请求直接拒绝，不通过隐式交换宽高放行。

设备可用 `X-Display-Width` 和 `X-Display-Height` 在**其已登记 profile 允许范围内**协商当前能力：只有 PhotoPainter 可以使用 `480x800` 竖屏，E1002 的这类请求会拒绝；不可沿用旧的方向标签。JPEG 响应中的 `X-Album-Orientation` 是实际像素方向，`X-Album-Target-Orientation` 是目标显示方向，因此 `auto` 模式下两者可以不同。尺寸、方向模式和安装 profile 的固定硬件补偿也会进入 JPEG variant/ETag，但不旋转 JPEG 像素：Waveshare PhotoPainter 为 `hardware_rotation_deg=180`，E1002 为 `0`，并分别写入固件的 `display_rotation_deg`。触摸屏 `display.*` 与 E6 `epaper.*` 的方向设置相互独立；E6 仍是固定 `800x480`、192000-byte 线协议，天气刷新或触摸屏方向变化不会让同一 E6 图帧失效并触发额外刷新。

7.10. 触摸屏交互设计

触摸屏首页把照片作为第一视觉层，天气卡片和状态信息使用高对比度实体背景。五个首页动作收拢为一个紧凑工具栏；文件名作为同一工具栏内的非交互文本，设置页有“显示文件名水印”开关。8 秒无操作时工具栏和文件名一起淡出，触摸照片唤醒。

底部导航打开图库、智能搜索、上传、设备和设置五个全屏面板。按钮最小高度为 56px，图片网格固定宽高比；1920x1080、1280x800、1024x600 和 400x900 视口均禁止页面横向滚动。

如果开发板同时连接两个 HDMI 输出，X11 会把它们合成一个更宽的虚拟桌面；触摸设备若仍映射到整个桌面，会表现为照片页面没有偏移但点击位置向一侧偏离。Case7 的 kiosk 启动脚本在打开浏览器前将 `QDtech MPI1001` 映射到主输出 `HDMI-1`，并在重启或热插拔后重新执行。该输入映射属于显示会话配置，与网页 CSS、照片 EXIF 方向和 NPU 推理无关。

设备面板内部再按任务拆成四个互斥视图，而不是把本机屏幕、远端终端和配对表单连续堆叠在同一长页面：

- **总览**显示登记记录、型号确认、最近拉图和启用数量，并提供进入其他视图的快捷入口；
- **本机触摸屏**只管理开发板 HDMI 显示设备的轮播、方向和文件名水印；
- **已注册设备**按 Waveshare PhotoPainter、Seeed reTerminal E1002 和待确认型号分组，集中处理远端设备状态、策略、推进、启停和删除；
- **发现与配对**只负责唤醒后的局域网发现、IP 验证和身份登记。

标签使用 `tablist`/`tabpanel` 语义，同一时间仅激活一个视图；窄屏时标签排成两列，设备数量变化只更新标签副标题，不改变操作位置。这种分区使本机触摸屏成为可单独设置的设备，也避免把 “发现候选设备” 和 “已经注册的设备” 混为一谈。

7.11. E6 图像处理方法

`epaper_display.py` 在 CPU 上执行：

1. 读取 EXIF 方向并旋转；
2. 将图像居中裁剪到 800x480；
3. 量化到 E6 六色调色板；
4. 使用官方稀疏颜色编号：黑 0、白 1、黄 2、红 3、保留黑 4、蓝 5、绿 6；
5. 两个像素按高半字节优先打包，输出 `800*480/2 = 192000` bytes；
6. dry-run 验证初始化、刷新、BUSY 和休眠命令序列。

真实硬件模式必须显式提供 SPI 设备、gpiochip 和 DC/RST/BUSY line offset。当前驱动板型号和接线尚未确认，PNG/帧 dry-run 只能证明协议编码，不能宣称微雪实屏刷新通过。PhotoPainter 六色路径同样不能由 E6 六色帧结果替代。

7.12. 工程代码导读

模块	职责
`app.py`	FastAPI 入口、上传、搜索、配置、显示和设备 API
`server_config.py`	配置默认值、类型校验、revision 和原子写入
`model_registry.py`	候选清单、生产注册表、文件 hash 和准入状态
`embedding_backend.py`	ACL 资源、固定输入输出合同、tokenizer 和模型切换锁

模块	职责
`photo_index.py`	SQLite 元数据、旧索引迁移、逐模型 FAISS 和增量更新
`smart_selector.py`	日期、天气、兼容元数据、语义排序、时隙和显示历史
`display_policy.py`	PhotoPainter 策略、cron、渲染、ETag 组成和 JPEG 限制
`epaper_display.py`	E6 旋转、裁剪、量化、打包和 periphery 协议
`web/`	原生静态触摸屏/手机 UI，不需要额外前端构建运行时
`scripts/`	模型、COCO-CN、部署、服务、kiosk 和性能脚本

7.13. 测试、性能与验收方法

测试把不同证据分开：

- 单元测试覆盖模型合同、ACL 模拟、索引隔离、上传安全、配置 revision、显示历史、设备握手/ETag 和 E6 帧；
- 模型准入覆盖 ONNX 合同、ATC 日志、OM hash、ACL 数值和有限值；
- COCO-CN 检索覆盖 20 条中文和 20 条英文查询，Recall@3 门槛均为 80%；
- 性能使用单线程、20 次预热、100 次计时、3 轮重复，报告图像/文本编码、FAISS 和 API P50/P95；
- UI 验收覆盖 1920x1080、1280x800、1024x600 和 400x900，无横向滚动、控件重叠或文字溢出；
- E6 dry-run 覆盖 EXIF、裁剪、六色、颜色编号、192000 bytes、BUSY 超时和无 GPIO 拒绝。

当前教程只定义验收协议，不预填具体 Recall、P50/P95、photo ID 或 hash。执行时把实际 JSON 报告路径、板卡/运行时版本、输入数量和结果写入 `samples/case7/reports/`，并在工程验证文档中注明证据类型。模型准入、接口健康、触摸屏、设备 URL 拉图和 E6 dry-run 必须分别报告；其中 API 成功或 ACL 数值通过不能替代真实电子纸刷新结论。

7.14. 限制、许可证与后续工作

- MobileCLIP 使用 Apple Machine Learning Research Model License，按清单中的非商业研究/教育边界使用；Chinese-CLIP 代码和权重许可应以固定上游仓库为准；ResNet50 使用 torchvision/ImageNet 研究教育资产。

- COCO-CN 是唯一公开测试集；图像来源于 COCO-CN/MS-COCO，使用时必须保留 manifest 的来源和许可证字段。
- Open-Meteo 是默认天气服务，网络失败时只能使用最后有效状态。
- PhotoPainter 使用上游 `v2.18.0` 固件；Case7 不维护自定义 ESP32 固件分支。
- 服务无管理员登录，只适合可信局域网，禁止公网暴露。
- 微雪 E6 驱动板型号和 GPIO/SPI 接线尚未确认，真实实屏刷新是独立后续验收。
- 通用 ESP-IDF 客户端需要具体 ESP32 变体、显示控制器、总线、GPIO、电源、Flash/P-SRAM 和 IDF 版本后才能安全实现；当前先使用 HTTP 协议和上游 PhotoFrame。

7.15. 参考资料

- Apple MobileCLIP
- OFA-Sys Chinese-CLIP
- COCO-CN 论文
- Waveshare E6 Python 驱动
- Ascend samples
- ESP32 PhotoFrame
- Andy：基于 CLIP 模型特征搭建简易的个人图像搜索引擎
- atarss/clip-image-search

8. 案例 8：实时手势识别

8.1. 项目简介

本案例在昇腾 310B 上实现实时手势识别。系统从 USB 摄像头读取视频，使用 HaGRID YOLOv10 模型在 NPU 上完成目标检测，把检测框和标签绘制到原始画面，再使用 H.264/WebRTC 将结果发送到浏览器。

案例代码位于 `samples/case8`。其中 `hagrid_yolo` 保存模型预处理、推理后端和后处理，`webrtc_app` 保存摄像头、DVPP 和 CANN VENC 适配，`scripts` 保存模型转换、OM 推理和 WebRTC 服务入口，`web` 保存浏览器页面。

完整的工程说明位于 `samples/case8/docs/`：

- `docs/README.md`：工程文档入口；
- `docs/01-project-design.md`：项目设计；
- `docs/02-program-analysis.md`：程序解析；
- `docs/03-model-dataset-and-conversion.md`：模型、数据集与转换；
- `docs/04-deployment-and-testing.md`：部署与测试；
- `docs/05-known-issues-and-version-compatibility.md`：问题与版本兼容性。

8.2. 学习目标

完成本案例后，你应能够：

- 说明 HaGRID 手势检测任务和 YOLOv10 输出含义；
- 将 PyTorch 权重导出为 ONNX，并使用 ATC 转换为 Ascend OM；
- 使用 PyACL 加载 OM，完成摄像头帧的预处理、推理和后处理；
- 解释 NV12、CANN VENC、CPU `libx264` 和 WebRTC 的关系；
- 在开发板上完成 OM 冒烟、摄像头测试和远程 WebRTC 验收。

8.3. 摄像头权限

官方昇腾 310B 系统通常将摄像头设备设置为 `root:video`、`0660`。如果使用 `HwHiAiUser` 启动 case8，该用户必须属于 `video` 组，否则 WebRTC 会报告 `Cannot open video source: /dev/video0`，即使设备列表中能看到 `/dev/video0`。

使用 root 或具有用户管理权限的账户执行：

```
sudo usermod -aG video HwHiAiUser
```

执行后退出当前 SSH 会话并重新登录。官方 Atlas 开发板的实际验证还要求随后拔下并重新插入 USB 摄像头，使 UVC 设备重新初始化；否则即使用户组已经更新，旧设备会话也可能导致首帧等待超时。重新插拔后再检查：

```
id HwHiAiUser
ls -l /dev/video0 /dev/video1
```

`id` 输出中应包含 `video`。`/dev/video0` 通常是真实的视频采集节点，`/dev/video1` 可能是 UVC metadata 节点，case8 应使用 `/dev/video0`。不要使用 `chmod 666` 代替用户组配置。

8.4. 官方 Atlas 开发板验证

目前的 case8 已在官方 Huawei Atlas 开发板上完成端到端验证。此次验证的板端软件栈为：`npu-smi` 产品标识 `Atlas 200I A2`、芯片 Ascend 310B4；Ubuntu 22.04.5 LTS；内核 `6.6.0-72.0.0.76.h914.eulerosv2r15.ascend.aarch64`；固件 `7.8.0.5.216`；驱动包 `25.5.0`（内部版本 `V100R001C23SPC005B219`）；CANN `9.2.0-beta.2`；`npu-smi 25.5.0`。板端没有单独暴露可核对的 `HDK` 版本字段，因此这里记录实际可查询的固件、驱动和 CANN 版本，不把其中任何一个版本冒充 HDK 版本。

完成 `HwHiAiUser` 加入 `video` 组、重新登录并重新插拔摄像头后，程序可以从 `/dev/video0` 采集 MJPG，使用 NPU 执行 OM 推理，使用 CANN VENC 进行 H.264 硬件编码，并通过 WebRTC 连续推流到浏览器。这是当前官方 Atlas 平台上的 `observed-pass` 结果；CANN VENC 失败时仍保持明确报错并停止，不自动回退到 CPU 编码。

8.5. 系统流程

```
flowchart LR
    accTitle: 实时手势识别流程
    accDescr: 摄像头图像经过预处理和 OM 推理，检测结果绘制回原图后编码为 H.264，通过 WebRTC 发送到浏览器。
    cam[USB 摄像头] --> pre[预处理]
```

```
    pre --> om[ACL 加载 OM]
    om --> post[后处理与绘制]
    post --> enc[CANN VENC 或显式选择的 CPU 编码]
    enc --> rtc[WebRTC]
    rtc --> browser[浏览器]
```

原始画面可以是 1280x720 等采集尺寸，但模型输入固定为静态 `1x3x640x640`。程序使用 letterbox 保持宽高比，再转成 RGB、NCHW 和归一化张量。推理输出经过置信度过滤、坐标恢复和 NMS 后绘制回原图。

8.6. 模型与数据集

本案例使用 HaGRID 提供的 YOLOv10 手势检测权重，不在开发板上训练模型。导出脚本同时生成 ONNX、标签文件和元数据；ATC 在目标 Ascend 310B 上生成 OM。模型输入尺寸、输入名称和类别顺序以元数据及 OM 描述为准，不能在网页中动态修改模型输入尺寸。

模型导出示例：

```
cd samples/case8
python weights/export_yolo_to_onnx.py \
  --weights weights/YOLOv10n_gestures.pt \
  --output-dir models --imgsz 640 --batch 1 --opset 13 --device cpu
```

开发板转换示例：

```
cd /home/HwHiAiUser/Documents/case8
source /usr/local/Ascend/ascend-toolkit/set_env.sh
SOC_VERSION=Ascend310B4 bash scripts/atc_convert.sh
```

8.7. OM 推理

模型转换成功后，先执行不接摄像头的 benchmark：

```
python scripts/infer_om_camera.py \
  --benchmark-runs 20 --warmup-runs 5 --print-model-info
```

再执行摄像头推理：

```
python scripts/infer_om_camera.py \
  --source /dev/video0 --camera-width 1280 --camera-height 720 \
  --camera-fps 30 --max-frames 60 --no-window
```

benchmark 只验证 ACL 能够加载和执行 OM；摄像头测试还会验证采集、预处理、后处理和资源释放。两者都通过后再进入 WebRTC 测试。

8.8. WebRTC 推流

启动服务：

```
python scripts/webrtc_om_app.py
```

浏览器访问开发板打印的局域网地址，默认是 `http://192.168.1.100:8080`。服务通过 `/models` 列出 OM，通过 `/offer` 完成 SDP 协商，通过 `/stats` 提供采集、推理和编码状态。

网页的编码器选项有两个：

- `CANN VENC`：推理仍在 NPU，H.264 使用 Ascend VENC；通道创建或编码失败时明确报错并停止，不自动切换 CPU。
- `CPU libx264`：仅在用户明确选择时使用 CPU 编码，OM 推理仍在 NPU。

因此 CPU 编码可以作为明确的对照路径，但不能作为 CANN VENC 失败后的隐藏回退。

8.9. 验收顺序

1. 检查 CANN 环境和 `import acl`。
2. 运行 OM benchmark。
3. 运行有限帧摄像头推理。
4. 检查摄像头实际 MJPG/FourCC 和帧率。
5. 启动 WebRTC，选择 CANN VENC，确认浏览器持续收到 H.264。
6. 若 VENC 失败，保存错误码和设备日志，不用 CPU 结果替代硬件验收。

香橙派 8T 的系统版本差异会影响 VENC。当前已验证的旧系统、新系统和 `iommu_map failed -34` 证据集中记录在 `samples/case8/docs/05-known-issues-and-version-compatibility.md`。

9. 案例 9：智能聊天机器人

本教程面向 Ascend310B4/8T 与 Ascend310B1/20T 的文本 LLM 实验。当前同时维护两条彼此独立的路线：Qwen2.5 静态 KV 的 ONNX -> OM -> 原生 ACL 路线，以及 MindSpore/MindNLP 候选路线。20T 当前地址为 192.168.1.101；.95 与 .210 是同一块板的历史地址。8T 当前地址为 192.168.1.90；.11.14 与报告采集地址 .178 是同一块板的历史别名。2026-10-15 的 20T 完整机器验收确认：Qwen1.5-0.5B-Chat 的 MindSpore 路线是当前最适合文字聊天的实验性候选；Qwen2.5-0.5B 静态 KV OM 的 ACL 路线功能完整但明显偏慢。TinyLlama 中文质量未通过；DeepSeek、Qwen3、MiniCPM3 等未在本次形成完整当前验收，不列为可用。本教程只写入有原始证据的结果；not-run 和 blocked 不等同于对硬件或模型的普遍否定。

9.1. 当前目标和已暂停范围

本案例的当前目标是把聊天模型以 OpenAI-compatible JSON/SSE 服务运行在昇腾 NPU 上，再通过 Case9 网关和文字页面访问。当前只验收文字 LLM，不把文字服务当作 XiaoZhi 设备后端的完整证明。

Profile 或链路	当前状态
Qwen2.5 静态 KV ACL：8080 -> 7861 -> 7865	现有正式基线不在本次切换；20T 当前 .101 的 B1 OM 完整验收通过，但短问候首事件 p50 约 4.73 s、约 0.422 token/s，不作为当前流畅文字入口
qwen1.5-0.5b-mindspore：.90 B4/8T 与 .101 B1/20T	8T 保持 blocked；20T 当前批次全机器门通过，首 token p50 约 655.5 ms、约 1.544 token/s，当前标记为 experimental_dirty_base，人工质量/正式准入待签字
tinyllama-1.1b-mindspore：.90 B4/8T 与 .101 B1/20T	两板均保留 blocked；历史 .95 缺口批次 8/9，32/48-token 输出出现 U+FFFD，中文机器质量 7/10
deepseek-r1-qwen-1.5b-mindspore：.90 B4/8T 与 .101 B1/20T	8T 历史 .11.14 缺口批次 9/9 机器门和性能通过；历史 .95 隔离 API 机器门通过但中文质量/dirty-base 准入未完成，保持 blocked
音频、麦克风、ASR/TTS、PTT、XiaoZhi、OTA、设备 WebSocket	暂停，不安装、不启动、不验收

机器门只表示协议、资源和运行检查通过，不表示回答正确或适合生产。正式入口、音频链路和 XiaoZhi 均须另行批准。

9.2. 当前 20T 已验收模型与选择

本节的每一项都来自 `192.168.1.101` 的 Ascend310B1/20T 实测，而不是框架文档或控制机推断。测试环境为 CANN `8.0.0`、Python `3.9.2`；`Health: Alarm` 仅保留为诊断信息。统一验收包含工件哈希、目标 SoC、实际 NPU 生成、JSON/SSE、`8/16/32/64` token 长输出、10 轮稳定性、中文机器探测、协议边界和 2 次预热后的 30 次性能测量。

模型	路线	完整机器门	p50 首 token/首事件	p50 总耗时	token/s p50	当前选择
Qwen1.5-0.5B-Chat	MindSpore 2.4.10 + MindNLP 0.4.1	通过	`655.5 ms`	`1295.4 ms`	`1.544`	当前首选文字聊天候选
Qwen2.5-0.5B-Instruct 静态 KV 1024 FP32 OM	原生 ACL/Py-ACL	通过	`4727.8 ms`	`4741.3 ms`	`0.422`	功能验证候选，不作为当前流畅入口

Qwen1.5 的 64-token 请求实际生成 55 token 后正常 `stop`，10 轮稳定性为 `10/10`；长中文输出连贯且 UTF-8 完整。它的健康接口同时记录 `device_target=Ascend` 与 `npu-smi=Ascend310B1`，因此本节不把它写成 CPU fallback。该板使用共享 `base`，其中已有 Torch 相关包；适配器不导入这些包，但环境仍须标记为 `experimental_dirty_base`。

Qwen2.5 OM 的字节数为 `1,266,009,438`,SHA-256 为 `6bca884fbce746efdb02f8c9294cad5b2faa6c8b96cac9`。其 48 个 KV cache 张量、tokenizer 锁、ACL descriptor 与 B1 SoC 均通过核对；运行中 NPU AICore 约 `75-80%`，设备内存约 `9.0 GB`，证明该路线确实执行在 NPU。它慢的原因是当前静态图及逐 token ACL 调用的延迟，不是模型无法运行或退回到 CPU。

因此，当前应优先在候选链使用 Qwen1.5 MindSpore；OM/ACL 路线继续保留作可复现的低层功能验证和优化对象。两者均未自动替换正式 `8080 -> 7861 -> 7865` 入口。

当前 `.90` 的只读设备快照还显示周期性 `DRV_LPM_FAULT 0x80E3A203`，NPU 内存约 `13598/15610 MB`；在用户确认设备恢复前，暂停新的重模型加载和长压测。`Health: Alarm` 仅作为诊断记录，不单独替代模型门禁；快照、哈希和处理边界见仓库验收记录。

9.3. MindSpore 候选模型范围

候选注册表同时保存已测试、失败和未执行的 Profile。下表是本教程当前固定的首批范围；条件候选只用于记录研究状态，不下载、不安装，也不能由 `case9-modelctl` 启动。

Profile	8T B4	20T B1	首轮结论
`qwen1.5-0.5b-mindspore`	已测但 B4 `blocked`	已测	B4 身份门待补；B1 共享 `base`,技术通过后为 `experimental_dirty_base`
`tinyllama-1.1b-mindspore`	已测	已测	长输出/中文质量失败，保持 `blocked`；英文单独报告
`deepseek-r1-qwen-1.5b-mindspore`	兼容性已测	优先已测	中文质量与正式准入独立审核
`qwen2.5-0.5b-mindspore`	`blocked`：context-only 诊断有输出，但严格 placement 证据缺失	**not**-run	不能用诊断输出启动服务
`qwen2.5-1.5b-mindspore`	`blocked`：七个文件已同步并校验；污染 CANN/PYTHONPATH 的初次 SIGSEGV 仅作历史证据，正确 CANN v4 已完成加载和两 token context-only 诊断	**not**-run	placement、生成/API/质量/性能均未完成；内存门原始记录为 `docs/37-qwen25-1.5b-8t-memory-gate-20260904.md`
`qwen3-0.6b-mindspore`	`blocked`：MindNLP 0.4.1 无 Qwen3 loader	**not**-run	不升级环境
`qwen3-1.7b-mindspore`	`blocked`：MindNLP 0.4.1 无 Qwen3 loader	**not**-run	不升级环境
`minicpm3-4b-mindspore`	不执行	**not**-run	仅使用官方 20T/24G Modelers FP16 示例

OpenPangu、EE-Model、TeleChat、BitCPM、MiniCPM5、MiniMind 和 Haidass 等条件候选若依赖 Torch/Torch-NPU、vLLM、MindIE、新版 CANN 或缺少 310B 证据，则登记为 `blocked`/**not**-run，不得作为失败回退。模型来源、证据等级和具体阻断原因见仓库中的候选清单与验收记录；框架支持不能替代当前板端 NPU 证据。

9.4. 浏览器、网关、ACL 和 NPU 架构

候选链一次只运行一个 MindSpore worker。浏览器没有管理接口，Profile 切换只能在板端执行 `case9-modelctl`；网关密钥只存在板端环境。ONNX -> OM 路线和 MindSpore

路线共享 NPU，但不共享进程、模型实例或验收结论；同一时刻只启动明确记录的那一条路线。

```
flowchart LR
    accTitle: Case9 candidate text architecture
    accDescr: A trusted browser reaches the read-only candidate text UI and authenticated gateway; the gateway reaches one active MindSpore profile worker, while the formal Qwen2.5 ONNX-to-OM ACL route remains a separate NPU path.
    browser[浏览器文字 UI :7868] --> gateway[候选网关 :7867]
    gateway --> worker[活动 Profile 服务 :8090]
    worker --> ms[MindSpore]
    ms --> npu[Ascend NPU]
    ctl[板端 modelctl] --> worker
    formal[正式网关 :7861] --> acl[Qwen2.5 ACL :8080]
    acl --> npu
```

用途	服务	网关	页面
MindSpore 候选	`127.0.0.1:8090`	`127.0.0.1:7867`	`0.0.0.0:7868`
正式 Qwen2.5 基线	`127.0.0.1:8080`	`127.0.0.1:7861`	`0.0.0.0:7865`

候选 UI 是实验页面，任何同网段主机都可能提交文字；只允许在可信实验网络使用。

9.5. 两种板卡和无 Torch 边界

`.90` 是 Ascend310B4/8T（`.11.14` 是此前活动地址，报告采集地址为 `.178`）；`.101` 是 Ascend310B1/20T。`.11.14`、`.178`、`.95` 和 `.210` 都是历史地址，不应再用于新批次。两块板的 CANN、Python、MindSpore、MindNLP、驱动和 `npu-smi` 版本必须在每个批次重新快照，不能把一块板的环境结论复制给另一块板。Qwen1.5、TinyLlama 和 DeepSeek 使用板端已有 `base`，因此逐板机器门全部通过后也只能标记 `experimental_dirty_base`；Qwen1.5 B4 当前身份门不完整而保持 `blocked`，TinyLlama 的既有失败证据仍保持 `blocked`，DeepSeek 的质量和准入在双板数据补齐前保持未决。

每次在板端同一个 shell 中准备环境：

```
source /usr/local/miniconda3/etc/profile.d/conda.sh
conda activate base
source /usr/local/Ascend/ascend-toolkit/set_env.sh
# 版本以本批次 environment snapshot 为准；不要从另一块板复制版本结论。
# PYTHONNOUSERSITE=1 只用于负向诊断，不能作为服务启动环境。
unset PYTHONNOUSERSITE
cd ~/case9-mindspore-chat
export PYTHONPATH="$PWD${PYTHONPATH:+:$PYTHONPATH}"
```

MindSpore 路线直接调用 MindSpore/MindNLP；现有 `torch`、`torch_npu`、`torchaudio` 和其他污染包只记录，不删除、不升级，适配代码不得导入它们。ONNX -> OM 路线使用独立的 `case9-acl-om` 环境和原生 ACL，板端不安装 Torch、Transformers、ONNX Runtime、vLLM 或 MindIE。两条路线都不修改系统 CANN、驱动、kernel 或 OPP。控制机 `sci-agent` 只用于静态检查、前端构建和外部参考，不把其运行时复制到板端。

9.6. 模型工件与哈希校验

注册表是 `configs/chat_model_profiles.json`。两条路线的工件必须分开登记：ONNX -> OM 路线记录 ONNX、每块 SoC 对应的 OM、ATC 日志和 ACL descriptor；MindSpore 路线记录 checkpoint、tokenizer、配置、源码 revision 及环境锁。已有历史工件位于本地忽略目录：

```
samples/case9/repro/qwen25-kv1024-dual-board-20260827/
samples/case9/repro/mindspore-chat-20260829/
```

补齐批次由 `scripts/sync_case9_gap_bundle.sh` 写入新的时间戳目录，不覆盖上述历史证据。每个目录应含板端环境快照、原始验收报告、源码快照、`bundle-manifest.json` 和 `SHA256SUMS.txt`；清单记录来源 IP、SoC、字节数和 SHA-256，模型文件不提交 Git。DeepSeek 的当前目标板是 `.101`；`.95` 和 `.210` 上的旧路径或旧缓存不能作为当前工件证明。

Qwen2.5 静态 KV 的已登记哈希(用于启动前核验)为：ONNX `b4870df5da9c8cbef4163ceb65d4dc13433f2`，B4 OM `f6650e52ff3908288763ef7957832ade606b0e554fa8fde986932f1ca1140eb8`，B1 OM `6bca884fbce746efdb02f8c9294cad5b2faa6c8b96cac9ec8c83730126298609`，tokenizer `c0382117ea329cdf097041132f`。哈希只证明工件完整性，不证明目标板上的 ATC、ACL 或中文质量门通过。

启动前只读核验 Profile 声明的每个文件：

```
python scripts/verify_mindspore_profile_artifacts.py \
  --profile qwen1.5-0.5b-mindspore \
  --root "$PWD" \
  --output reports/mindspore-chat/qwen1.5-0.5b-mindspore/artifact-verify.json
```

返回 `passed` 才能记录 `artifact_verified`。下载或同步必须使用临时文件、远端字节数、本地 SHA-256 和原子改名；不使用未验证的 LFS 指针、缓存目录或可变 `main` 作为模型证明。

9.7. 新板环境检查和模型门禁

在 `.90` 和 `.95` 分别记录 SoC、CANN、Python、磁盘、内存、HugePages、`npu-smi` 和污染包；`.178`/`.210` 只用于查阅历史报告：

```
python - <<'PY'
import importlib.metadata, sys
print("python", sys.version)
for name in ("mindspore", "mindnlp", "numpy", "pytest"):
    try:
        print(name, importlib.metadata.version(name))
    except importlib.metadata.PackageNotFoundError:
        print(name, "missing")
PY
npu-smi info
bash scripts/case9-modelctl.sh list
bash scripts/case9-modelctl.sh status
```

MindSpore Profile 没有 OM 转换步骤；其门禁是工件核验、MindSpore Ascend context、tokenizer 导入、单 token 生成、JSON/SSE 契约、长输出、稳定性、质量和性能。Qwen2.5 的 ONNX -> ATC -> OM -> ACL 门禁属于独立正式基线，必须逐板确认 `Ascend310B4` 或 `Ascend310B1` 的 descriptor、输入输出 byte size、ACL smoke 和 NPU 采样，不能用 MindSpore 结果替代。任何门失败都记录为 `failed`、`blocked` 或 `not-run`，不自动切换 CPU、云端或其他推理框架；没有当前 `.101` 的完整环境和工件快照前，不得把 `.210` 的历史报告当作 20T 通过证据。

ONNX -> OM 路线的候选端口先使用 `127.0.0.1:8084`；通过完整门禁后才有资格替换正式 `127.0.0.1:8080`。MindSpore 路线使用 `127.0.0.1:8090`，两者不能同时占用同一 NPU 模型实例。

9.8. 候选 Profile 服务和 OpenAI API

`case9-modelctl` 负责停止旧 worker、启动新 worker、轮询健康状态和记录活动状态；失败时尝试回滚，回滚也失败则 fail-closed：

```
CASE9_ALLOW_EXPERIMENTAL=1 bash scripts/case9-modelctl.sh switch qwen1.5-0.5b-mindspore
# 正式注册表中的 blocked/not-run Profile 会被 CLI 拒绝；测试批次只能使用明确标记
# 的临时注册表，不能用参数绕过工件、环境或质量门禁。
```

DeepSeek 的当前目标板是 `.101`；`.95` 与 `.210` 只保留为历史地址。服务固定监听 `127.0.0.1:8090`，内部模型名为 `case9-active`，请求限制如下：

- `messages` 只接受 `system`、`user`、`assistant`，使用 Profile tokenizer/chat template；
- batch 为 1，单进程单请求串行，实际 token 化后检查 context `1024`；
- `max_tokens` 默认 `32`、上限 `64`，只接受 `temperature=0`、`top_p=1`；
- 请求体上限 `256 KiB`，支持普通 JSON 和 OpenAI SSE；

- SSE 只发送前缀差量；实现包含超时 watchdog 和客户端中断清理，但 watchdog 实际触发、自动回滚及浏览器会话清空仍未获得本轮正向验收；
- 客户端不能指定 Profile、权重路径、后端或任意 Python 表达式。

接口为 `GET /health`、`GET /v1/models` 和 `POST /v1/chat/completions`。`/health` 返回 Profile、revision、环境指纹、NPU 型号、worker PID、busy、缓存清理和 admission 状态。

9.9. 网关和文字 UI 验证

Profile 服务健康后，在另一个板端 shell 启动候选链：

```
export GATEWAY_API_KEY="<板端内部 token>"
bash scripts/run_mindspore_chat_gateway.sh
bash scripts/run_mindspore_chat_text.sh
```

候选网关固定转发到 `http://127.0.0.1:8090/v1`，对外模型名是 `case9-rag`，并关闭 RAG 注入：

```
UPSTREAM_BASE_URL=http://127.0.0.1:8090/v1
UPSTREAM_MODEL=case9-active
RAG_ENABLED=false
MAX_CONCURRENT_REQUESTS=1
PUBLIC_MODEL_ID=case9-rag
```

浏览器按实际运行板卡访问 `http://192.168.1.90:7868/`(8T)或 `http://192.168.1.101:7868/` (20T)。已归档的候选链 smoke 报告包含未授权 `401`、授权 `/v1/models`、JSON/SSE 和 UI health 的历史响应；这些是协议/UI 证据，不是双板缺口批次的自动通过。每次新批次都必须保存请求、响应、服务 PID 和板卡 IP，不能把 `.90` 的 UI 结果复制给 `.95`。

正式 `8080 -> 7861 -> 7865` 由本教程和候选脚本保持不变。

9.10. 长输出、稳定性、中文质量和性能测试

验收脚本只访问已启动的 loopback 服务，不管理 PID：

```
python scripts/mindspore_chat_acceptance.py \
  --profile qwen1.5-0.5b-mindspore --execute \
  --run-id qwen-full-20260829b --timeout 600 \
  --probe-file tests/fixtures/qwen25_chinese_probe.json \
  --output reports/mindspore-chat/qwen1.5-0.5b-mindspore/qwen-full-20260829b
```

已有可复核的历史数据如下；它们用于说明基线，不等于当前双板缺口批次已经通过：

路线/Profile	板卡和采集地址	协议/批次	已测指标或门结果	当前解释
Qwen2.5 静态 KV ACL	B4/8T（当前 .90；历史采集 .11.14/.178）	2 warmup + 30 SSE, max_tokens=2	总耗时 p50/p95 8,693.731/8,707.133 ms；首事件 p50/p95 8,563.591/8,576.954 ms；约 0.230/0.231 token/s	正式基线历史证据；本轮只补身份、工件和 descriptor 索引
Qwen2.5 静态 KV ACL	B1/20T（旧采集 .210；当前 .101）	同协议	总耗时 p50/p95 6,486.422/6,506.085 ms；首事件 p50/p95 6,364.634/6,383.111 ms；约 0.308/0.310 token/s	只作为 B1 历史对照，不能与 B4 合并排名
qwen1.5-0.5b-mindspore	B4/8T（当前 .90；历史报告 .11.14）	2 warmup + 30 SSE, max_tokens=2	历史报表机器门 9/9；历史身份字段缺失按 8/9，当前 G0 已补齐但严格 placement 仍失败	历史输出和性能保留，但 B4 当前 blocked；人工质量、重复故障定位和准入未完成
tinyllama-1.1b-mindspore	B4/8T（当前 .90；历史报告 .11.14）	同协议	总耗时 p50/p95 3,114.857/3,185.731 ms；首事件 p50/p95 3,110.338/3,181.192 ms；约 0.642/0.651 token/s；机器门 8/9	32-token 输出含 U+FFFD，保持 blocked
deepseek-r1-qwen-1.5b-mindspore	B1/20T（.95）	隔离 API 2 warmup + 30 SSE, max_tokens=2	总耗时 p50/p95 2,484.751/2,557.242 ms；首事件 p50/p95 2,478.495/2,551.004 ms；约 0.805/0.815 token/s	仅隔离实验；双板统一门、中文质量和准入仍未完成

qwen2.5-0.5b-mindspore	B4/8T（当前 .90；旧 .11.14/.178）	loader 诊断 + 并发 npu-smi 采样	固定权重 988,097,824 bytes；加载 48.223862 s；16 token 中文生成 27.194196 s；AICore 非零峰值约 37%、设备内存约 38% -> 91% -> 46%	context-only 活动旁证；无显式 placement metadata，严格 worker blocked；未启动 API/网关，不是正式候选通过

缺口批次必须在两块板分别执行，状态只能引用相应原始报告：

缺口门	当前 .90 B4/8T（历史报告 .11.14/.178）	.101 B1/20T
Qwen2.5 ONNX/OM 当前身份、descriptor、ACL smoke	passed（identity-only；历史性能不重跑）	blocked（当前 ONNX/OM/contract/lock 缺失，未执行 ACL load）
Qwen1.5 MindSpore 长输出、稳定性、质量、性能	历史报表 9/9，但严格身份门 8/9，当前 blocked；共享 base，人工质量待审	passed，9/9 机器门，人工质量待审，experimental_dirty_base
TinyLlama MindSpore 长输出、稳定性、质量、性能	blocked（历史长输出/中文质量失败）	failed，8/9；32/48 token UTF-8 失败，保持 blocked
DeepSeek MindSpore API、稳定性、质量、性能	passed，9/9 机器门，人工质量待审，experimental_dirty_base	已有隔离 API 机器门；中文质量和正式准入 blocked
Qwen2.5-0.5B MindSpore loader/placement	blocked；固定工件和 NPU 活动诊断已记录，严格 placement 缺失	not-run；当前 20T 尚未运行该 Profile
Qwen2.5-1.5B MindSpore 配置/权重加载	blocked；初次污染环境 SIGSEGV（exit 139）保留为历史证据，正确 CANN v4 已加载并生成 2 token，但 placement/API/质量/性能未运行	not-run；未执行
Qwen3-0.6B/1.7B MindSpore loader	blocked；MindNLP 0.4.1 无 Qwen3 loader，未下载权重	not-run；未执行

上述状态直接对应本地复现包中的原始报告；not-run 只保留有意未执行的跨 SoC 兼容性组合，不是对板端能力的推断。历史两批错误边界为 6/6，协议边界包含超上下文、超大正文和客户端中断后的 health 检查；这些结果仍需在双板批次中分别复核。早期 Qwen 日志曾有 TBE 进程、leaked semaphore、dtype、top_p 和 attention mask 告警；post-mask n 批次已固定 top_p=1.0 并显式传入 mask。Health: Alarm 只作为诊断记录，不单独阻断结果。

在 post-mask n 批次后，8T 内核记录了重复 DRV_LPM_FAULT 0x80E3A203，blackbox 描述为 lpm get current error。原 worker 退出后，受控启动的新 worker 15897 完成了缩小

恢复批次（2/4 token 长输出、1 次稳定性、1 次性能、错误与协议边界）和候选链复核；这只证明服务恢复，不证明硬件稳定。恢复时 NPU 内存约 14.9/15.6 GB、HugePages 547/547，应先解决或定位该诊断事件，再重新执行完整稳定性和性能批次。

机器有效不等于中文正确。Qwen 的历史硬件探测存在事实错误，TinyLlama 主要面向英文；正式中文/英文人工评分、quality_reviewed 和 admitted 仍未完成。任何一块板的性能数字都必须带 SoC、环境、warmup、循环数、百分位方法和报告路径，不能只抄到教程表格。

9.11. 失败处理、回滚和证据保存

每次批次使用唯一 UTC 目录，保存命令、PID、服务日志、环境指纹、模型哈希、原始响应和 npu-smi before/during/after 快照。失败时：

1. 只停止本批次明确记录且命令行匹配的 worker PID；
2. 保留失败日志、哈希和报告，不删除系统 CANN、conda 缓存、其他模型或正式服务；
3. 维持 blocked、not-run 或具体失败状态，不自动切换 CPU、云端、Torch、vLLM、MindIE 或其他模型；
4. 切换失败先回滚上一个已验证 Profile，回滚失败则保持候选链不可用；
5. 新板复现时重新做环境、工件、加载、API、稳定性、质量和性能门，不能因相同模型名或 IP 别名自动继承正式状态。

若出现 DRV_LPM_FAULT、设备重置、worker 非正常退出或 NPU 内存持续增长，先保存 dmesg -T、npu-smi info、modelctl status、监听端口和本批次 PID，再停止扩大测试；不得把一次受控恢复写成模型稳定性通过。

详细原始文件位于 repro/mindspore-chat-20260829/、repro/qwen25-kv1024-dual-board-20260827/ 以及每次生成的 repro/case9-dual-board-gap-<timestamp>/（本地忽略）；板端对应 ~/case9-mindspore-chat/run/、reports/。TinyLlama 的 U+FFFD、.210 历史地址的连接失败记录和任何新的 .95 门禁失败都必须保留，不能改写成当前地址的成功结果。

9.12. XiaoZhi 后续边界

本轮不安装或启动 xiaozhi-esp32-server，不提供 OTA、设备 WebSocket、Opus、ASR/TTS 或真实设备命令。文字 LLM 的 JSON/SSE 成功不能证明 XiaoZhi 语音闭环可用。恢复该阶段前，需单独审核无 Torch 的语音组件、设备协议、鉴权、音频资源和真实设备端到端验收。

模型候选的来源、证据分级、逐板执行顺序和验收记录由仓库工程文档维护；本教程只保留可独立复现的命令、边界和已测结果，不依赖其他仓库 Markdown 文件。

参考：MindSpore Orange Pi 在线推理目录、Qwen1.5 模型卡、TinyLlama 模型卡、DeepSeek 模型卡、昇腾 ATC 文档。

附录

附录 1. 开发板与基础环境

1.1. 昇腾 310B 开发板介绍

OrangePi AIpro(8T) 开发板是由香橙派联合华为精心打造的高性能 AI 开发板。它采用昇腾 AI 技术路线，搭载昇腾 310B 处理器（4 核 64 位处理器 + AI 处理器），集成图形处理器，支持 8TOPS INT8 的 AI 算力。板载 8GB/16GB LPDDR4X 内存，并支持外接 32GB 至 256GB 的 eMMC 模块，同时支持双 4K 高清输出。

OrangePi AIpro(8T) 拥有丰富的接口资源, 包括两个 HDMI 输出、GPIO 接口、Type-C 电源接口、支持 SATA/NVMe SSD 2280 的 M.2 插槽、TF 插槽、千兆网口、两个 USB3.0、一个 USB Type-C 3.0、一个 Micro USB（用于串口打印调试）、两个 MIPI 摄像头接口以及一个 MIPI 屏幕接口等。此外，还预留了电池接口。

该开发板广泛适用于 AI 边缘计算、深度视觉学习、视频流 AI 分析、自然语言处理、智能小车、机械臂、无人机、云计算、AR/VR、智能安防及智能家居等领域，覆盖 AIoT 的各个行业。在软件方面，OrangePi AIpro(8T) 支持 Ubuntu 和 openEuler 操作系统, 能够满足大多数 AI 算法原型验证及推理应用开发的需求。在这个教程中，我们只介绍基于 Ubuntu 操作系统的昇腾 310B 开发流程。

1.1.1. 开发板详细视图

1.1.2. 开发板硬件规格

1.2. 配件准备

为了顺利进行开发，请准备以下配件：

1. **TF 卡** 建议使用容量 64GB 及以上、速率为 Class10 级以上的闪迪（SanDisk）品牌 TF 卡。虽然最小支持 32GB，但为了避免开发过程中出现磁盘空间不足的问题，推荐使用更大容量。
2. **TF 卡读卡器** 用于在电脑上读写 TF 卡以刷写系统镜像。建议选择 USB 3.0 速率的读卡器，以减少系统刷写的等待时间。

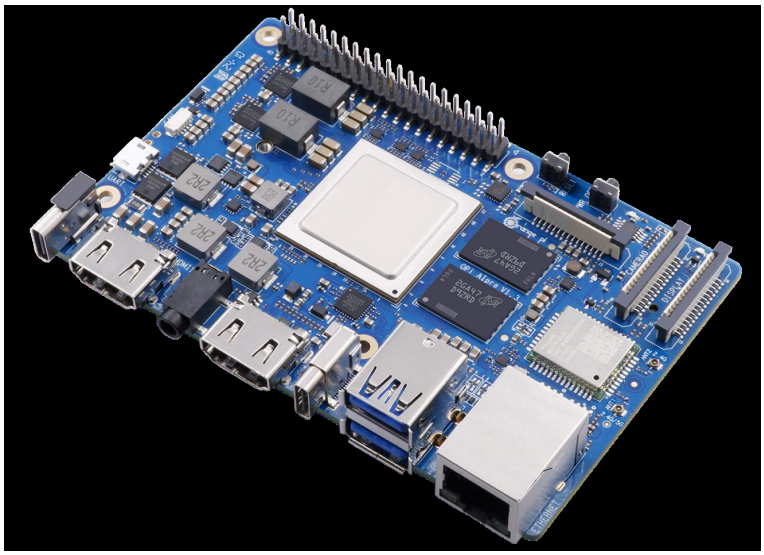


图 1.1. 产品图

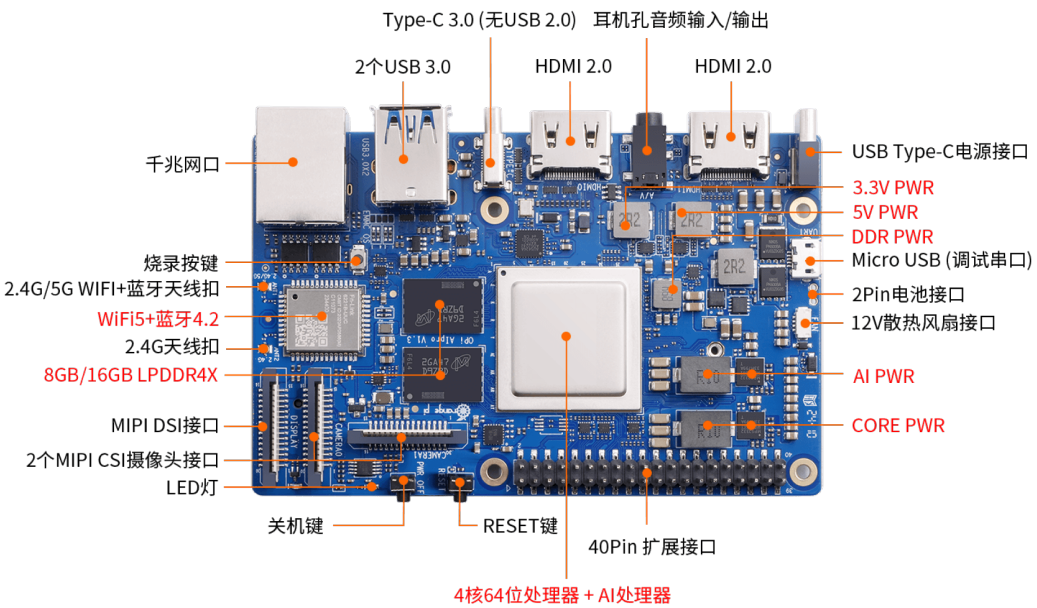


图 1.2. 正面标注视图

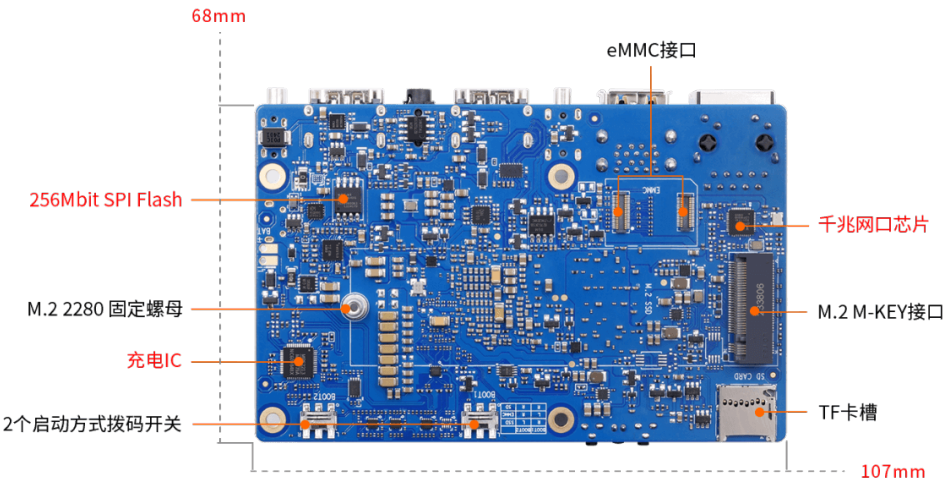


图 1.3. 背面标注视图

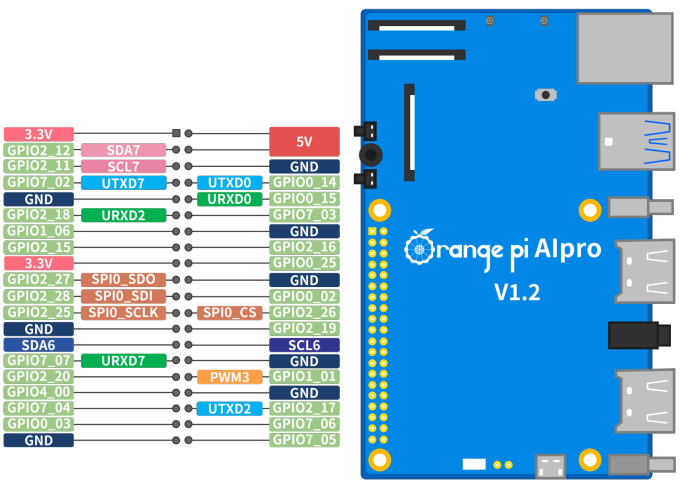


图 1.4. GPIO 接口定义

图 1.5. tf 卡

图 1.6. 读卡器

3. **HDMI 线** 开发板配备标准 HDMI 接口。请根据您的显示器接口类型，准备标准的 HDMI 线或 HDMI 转 Mini-HDMI/Micro-HDMI 线。
4. **电源适配器** 开发板采用 PD 协议供电（20V 挡位），功率需求为 65W。请务必使用支持 PD 协议 20V 输出的 65W Type-C 电源适配器。
5. **USB 鼠标和键盘** 用于在本地桌面环境下对开发板进行操作和调试。
6. **金属外壳** 用于保护开发板硬件，防止意外短路或物理损伤。

图 1.7. HDMI

图 1.8. Mini HDMI

图 1.9. PD 电源

图 1.10. 外壳

图 1.11. 风扇

图 1.12. 转接线

7. **散热风扇及散热鳍片** 由于昇腾 310B 处理器在高负载下发热量较大，强烈建议安装主动散热设备。开发板提供 2pin 风扇接口（12V），支持 PWM 调速。
8. **Type-C 转 USB 3.0 转接线（可选）**开发板的一个 Type-C 接口支持 USB 3.0 协议（不兼容 USB 2.0），可通过转接线连接 USB 3.0 外设。
9. **M.2 NVMe SSD（可选）**开发板背部的 M.2 接口支持 PCIe 协议（2280 规格），可安装 NVMe SSD 作为系统盘或扩展存储。
10. **M.2 SATA SSD（可选）**该 M.2 接口同时也支持 SATA 协议，因此也可以使用 M.2 SATA（NGFF）接口的 SSD。
11. **eMMC 模块（可选）** eMMC 是一种高性能、低功耗的嵌入式存储方案。相比 TF 卡，eMMC 读写速度更快（100-400MB/s）且更稳定。开发板预留了 eMMC 接口，需额外购买香橙派专用 eMMC 模块。
12. **USB 摄像头（可选）**用于图像识别、视频通话等应用开发。
13. **网线（可选）**虽然开发板板载 Wi-Fi 模块，但在需要更稳定网络连接的场景下，建议使用千兆网口连接有线网络。
14. **树莓派 IMX219 摄像头（MIPI-CSI）（可选）**开发板提供两个 MIPI-CSI 接口，兼容树莓派 IMX219 摄像头，可直接连接而无需占用 USB 接口。

图 1.13. nvme ssd

图 1.14. ngff ssd

图 1.15. emmc 正面

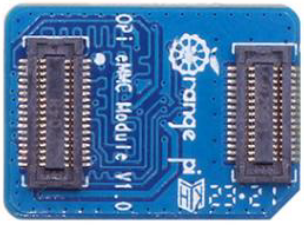


图 1.16. emmc 背面

图 1.17. 摄像头

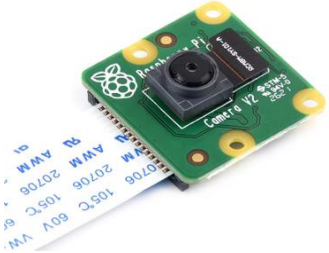


图 1.18. MIPI-CSI 摄像头

图 1.19. MIPI 显示器

图 1.20. Micro USB 数据线

15. **MIPI LCD 显示屏（可选）**开发板配备 MIPI-DSI 显示接口，可直接驱动兼容的 MIPI 显示屏（如树莓派 5 寸屏），无需外接 HDMI 显示器。
16. **Micro USB 数据线（可选）**开发板板载 CH343P 芯片，将 UART 调试串口转换为 Micro USB 接口。使用 Micro USB 数据线连接电脑，即可进行串口调试。

1.3. 操作系统安装

作为华为昇腾生态的重要成员，OrangePi AIpro(8T) 支持 Ubuntu 和 openEuler 两种操作系统。由于开发板板载无预装系统，我们需要通过电脑将系统镜像刷写到 TF 卡中。建议使用 Windows 11 或 Ubuntu 22.04 及以上版本的 PC 进行操作。

首先，访问香橙派官网的技术支持页面。

在页面下方找到 “官方镜像” 区域。官方提供了预装昇腾 NPU 应用环境及软件的 Ubuntu 和 openEuler 镜像，极大地方便了开发者快速上手。

1.3.1. 镜像下载

Ubuntu

1. **点击下载**：点击 Ubuntu 镜像对应的下载按钮。
2. **获取提取码**：复制弹出的百度网盘提取码，并点击跳转链接。
3. **选择镜像文件**：在网盘中找到名为`Ubuntu`的文件夹并进入。
4. **文件说明**：
    - `.xz`后缀文件为镜像压缩包。
    - `.sha`后缀文件为 MD5 校验码，用于验证下载文件的完整性。

图 1.21. 技术支持页面

图 1.22. 官方镜像

图 1.23. 下载

图 1.24. 跳转

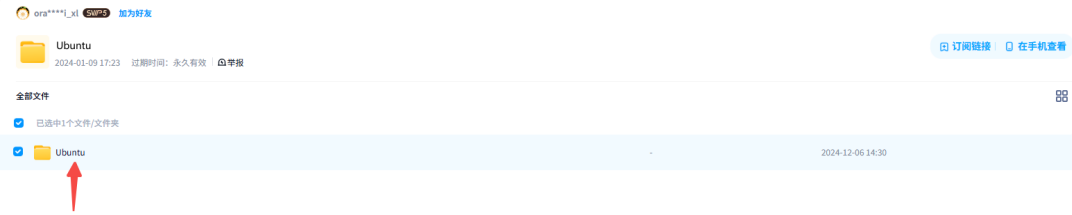


图 1.25. 文件夹

5. **选择版本**：
   - **Desktop**：包含 GUI 图形化界面，适合初学者和需要桌面环境的用户。
   - **Minimal**：仅包含命令行界面，适合高级用户或服务器用途。

   建议初学者下载带有`Desktop`字样的镜像。
6. **下载与解压**：下载完成后，请先校验文件完整性，再解压`.xz`压缩包得到`.img`镜像文件。

1.3.2. 校验下载文件 (MD5)

为了确保下载的镜像文件未损坏，建议在解压前进行 MD5 校验。

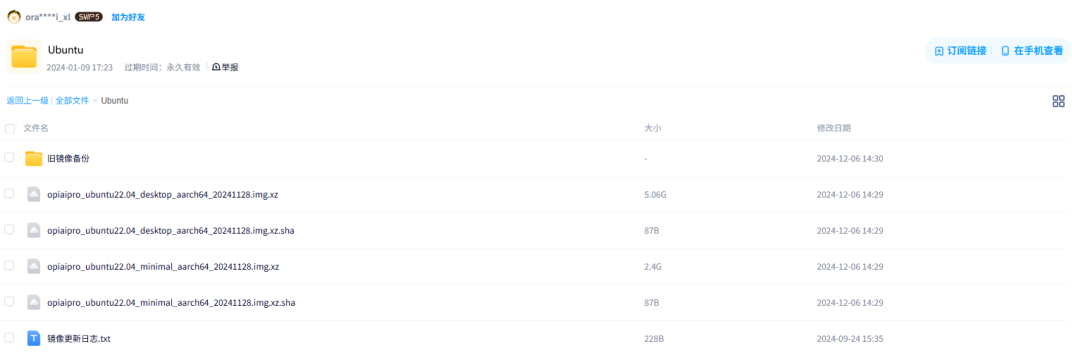


图 1.26. 选择镜像

图 1.27. 终端

图 1.28. md5 校验

- **Windows**：在镜像文件所在文件夹内按住 **Shift** 键并点击鼠标右键，选择 “在终端中打开” 或 “在此处打开 Powershell 窗口”。输入以下命令（文件名请根据实际情况修改）：

```
certutil -hashfile opiaipro_ubuntu22.04_desktop_aarch64_20241128.img.xz md5
```

  将输出的 MD5 值与同目录下.sha文件中的内容进行比对。

- **Ubuntu**：

```
md5sum <filename>
```

- **macOS**：

```
md5 <filename>
```

若校验值一致，说明文件完整，可进行解压；若不一致，请重新下载。

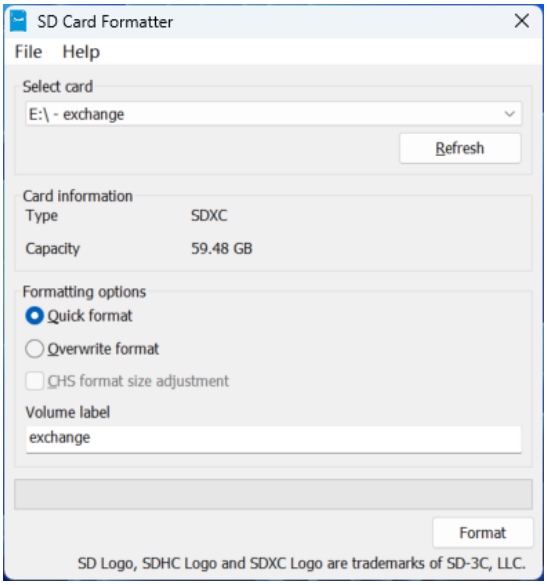


图 1.29. TF 卡格式化

1.3.3. 刷写系统到 TF 卡

工具准备

- **下载链接**：官网 | 百度网盘

1. **SD Card Formatter**：用于格式化 TF 卡，确保存储卡状态良好。
2. **balenaEtcher**：用于将`.img`镜像文件烧录到 TF 卡。**注意：建议使用 1.19.25 及以下版本，以避免兼容性问题**。

格式化 TF 卡

1. 将 TF 卡插入读卡器，连接至电脑。
2. 打开 **SD Card Formatter** 软件。
3. 确认选中了正确的盘符，点击右下角的 **Format** 按钮。
   警告：格式化将清除 TF 卡上所有数据，请确认无误后点击 “是”。
4. 等待格式化完成，点击确定。

烧录镜像（以 Ubuntu 为例）

1. 打开 **balenaEtcher**，选择 “Flash from file”（从文件烧录）。
2. 选择解压后的镜像文件（`.img`格式），然后点击 “Select target” 选择您的 TF 卡。
3. 点击 “Flash!” 开始烧录。
4. 烧录完成后软件会自动进行校验，请耐心等待。

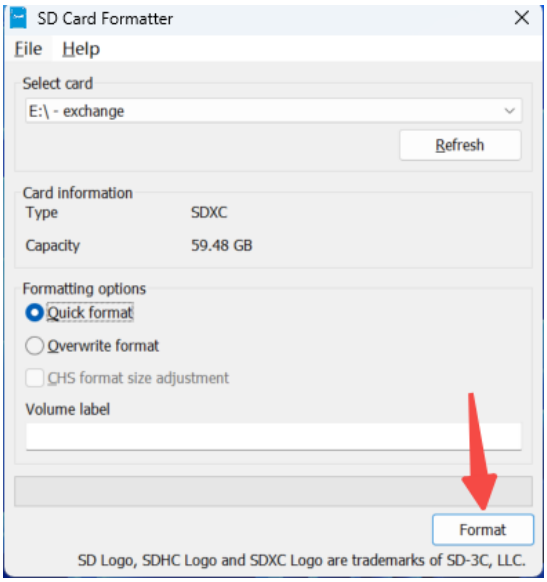


图 1.30. 格式化

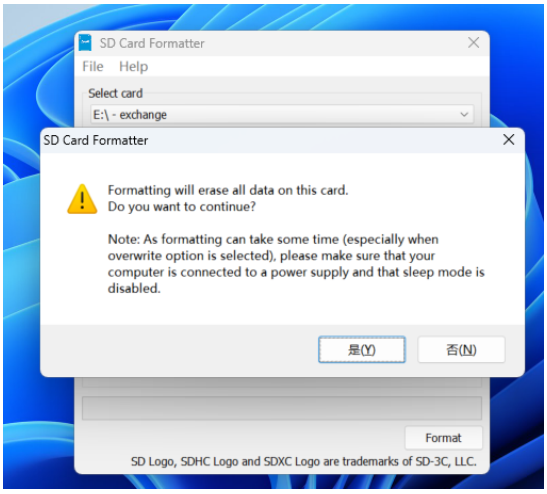


图 1.31. warning

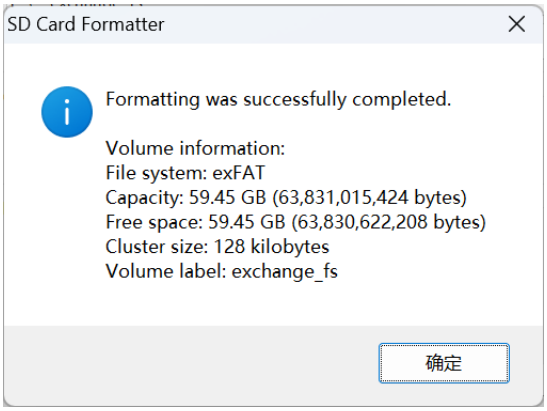


图 1.32. 格式化完成

图 1.33. balenaEther

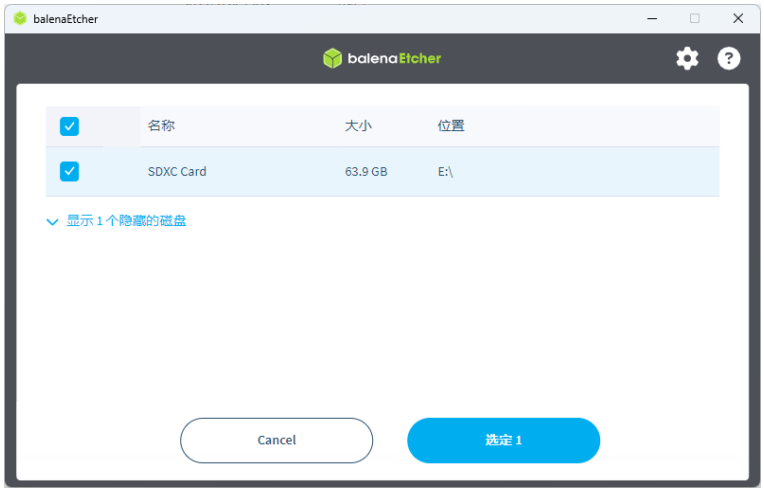


图 1.34. 选择磁盘

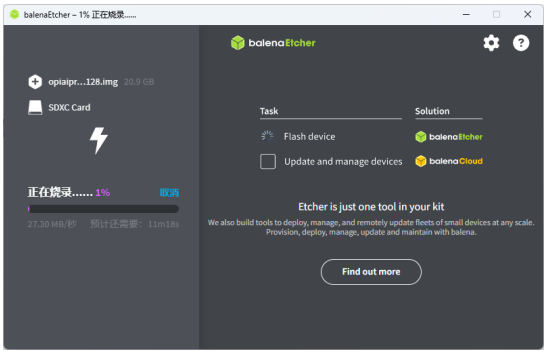


图 1.35. 烧录过程

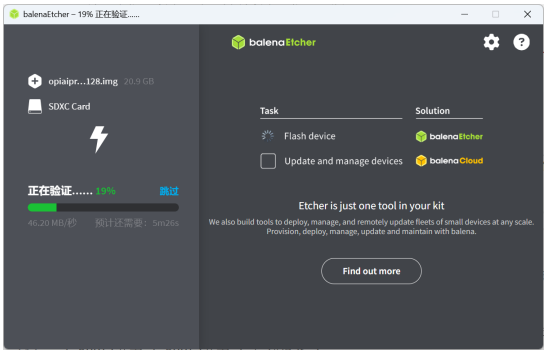


图 1.36. 校验过程

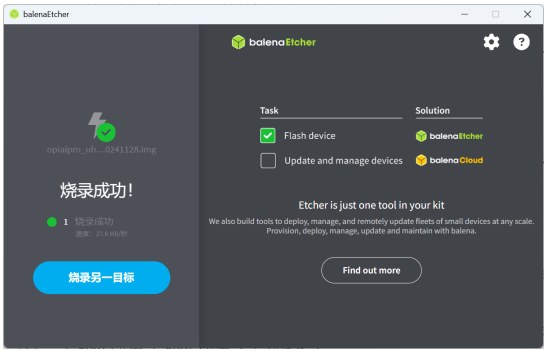


图 1.37. 完毕

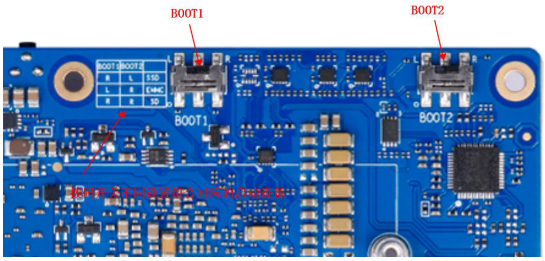


图 1.38. boot 开关

5. 校验通过后，关闭软件并安全弹出 TF 卡。

其他存储介质说明

- **eMMC**：板载无 eMMC，需购买专用模块。刷写方法请参考香橙派用户手册。
- **SSD**：支持 M.2 SSD 启动，但兼容性有限（仅支持特定品牌型号），且需自行准备。初学者不推荐直接使用 SSD 作为系统盘。

设置启动模式

开发板支持从 TF 卡、eMMC 或 M.2 SSD 启动。当连接了多种存储设备时，需通过背面的拨码开关（BOOT 开关）指定启动设备。

拨码开关状态说明（ON 方向为 1/右，相反为 0/左，具体请参考板上丝印或下表）：

Boot1	Boot2	启动设备
左	左	(未使用)
右	**右**	**TF 卡**
左	右	eMMC
右	左	M.2 SSD

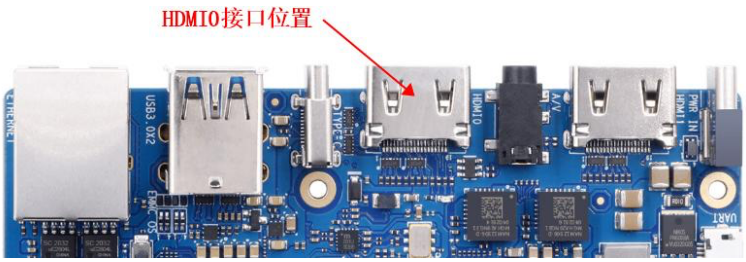


图 1.39. HDMI0

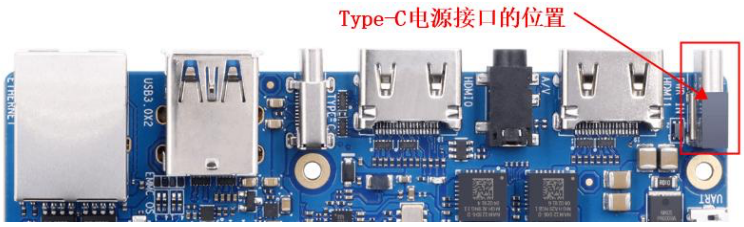


图 1.40. TYPE-C Power

注意：切换拨码开关后，必须**完全断电**（拔掉电源线）再重新上电，新的启动配置才会生效。仅按 RESET 键重启无效。

1.4. 启动开发板

1.4.1. 方式一：图形化界面启动

1. **硬件连接**：
   - 将刷写好的 TF 卡插入开发板插槽。
   - 确认拨码开关均拨至**右侧**（TF 卡启动模式）。
   - 将 HDMI 线连接至 **HDMI0** 接口（靠近 USB 3.0 接口的那个）。
   - 连接鼠标和键盘。
   - 最后，接入 Type-C 电源。
2. **系统启动**：
   - 上电后，风扇会全速旋转，随后声音变小，屏幕显示启动画面。
   - 稍候进入登录界面。
3. **登录系统**：
   - 默认普通用户：`HwHiAiUser`，密码：`Mind@123`
   - 默认 Root 用户：`root`，密码：`Mind@123`

图 1.41. 登录

图 1.42. 桌面

输入密码登录进入桌面环境。
若无法登陆请检查输入的密码是否正确，大小写以及符号是否正确
默认账户表格：| 用户名 | 密码 | | :—: | :—: | | root | Mind@123 | | HwHiAiUser | Mind@123 |

1.4.2. 方式二：串口登录

如果暂时没有显示器、键盘和鼠标，也可以通过串口登录开发板。建议初学者使用开发板自带的 Micro USB 接口，该方法不需要额外接线，只需要一根 Micro USB 数据线，接入电脑后打开设备管理器查询对应的串口，然后使用 PUTTY 进行连接即可。

1. **使用 Micro USB 数据线连接开发板和电脑，此时请不要给开发板上电。**
2. **打开电脑的设备管理器，选择端口，寻找开发板对应的串口端口号**

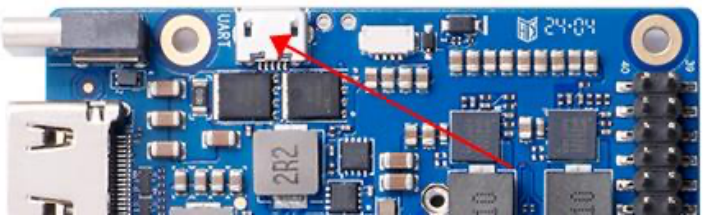


图 1.43. MicroUSB 串口

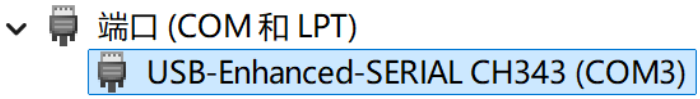


图 1.44. 端口号

3. **打开串口调试软件（PUTTY）**
   将 Connection Type 选择为`Serial`，然后在 Serial Line 处将端口号修改为设备管理器中查到的端口号，如作者此处端口号为`COM3`，此外，还需要将 Speed 从 9600 修改为 115200，最后点击 Open 打开串口。
4. **给开发板上电，等待出现`Ubuntu 22.04.3 LTS orangepiaipro ttyAM0`字样，输入登录的用户名 HwHiAiUser 并回车，然后输入密码 Mind@123 并回车，注意在输入密码的时候屏幕并不会显示任何东西，登陆后的界面如图所示。**

1.5. 网络连接

1.5.1. 无线网络连接 (WiFi)

开发板板载了 WiFi 模块，可以通过命令行工具`nmcli`轻松连接无线网络。

1. **扫描 WiFi**

```
nmcli dev wifi list
```

2. **连接 WiFi** 将`<SSID>`替换为你的 WiFi 名称，`<PASSWORD>`替换为密码。

```
nmcli dev wifi connect <SSID> password <PASSWORD>
```

3. **查看连接状态**

```
nmcli connection show
```

如果不熟悉命令参数，也可以使用`nmtui`提供的终端图形界面连接 WiFi：

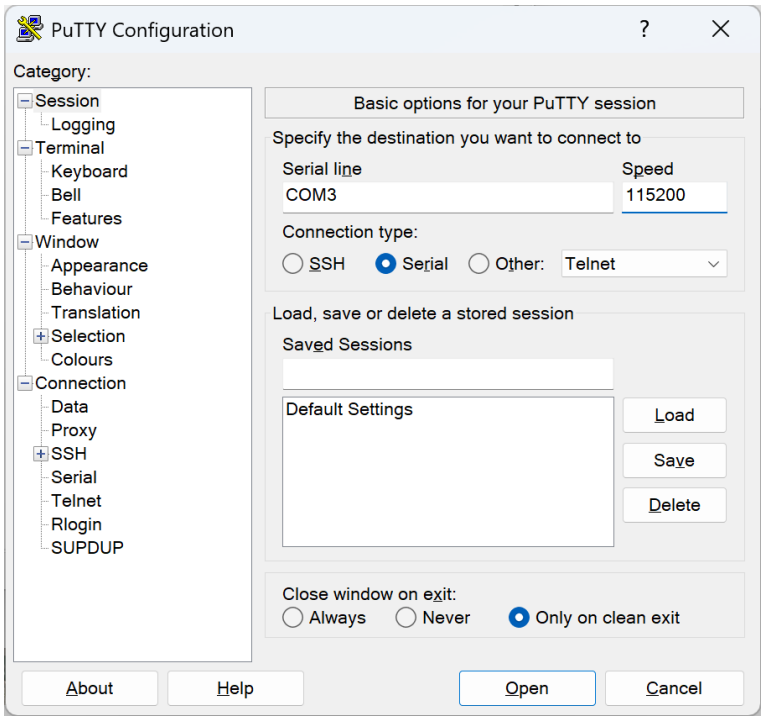


图 1.45. PUTTY

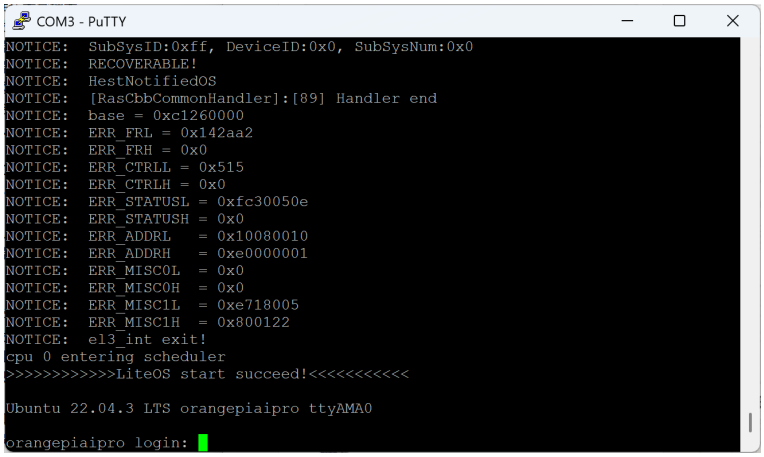


图 1.46. 串口

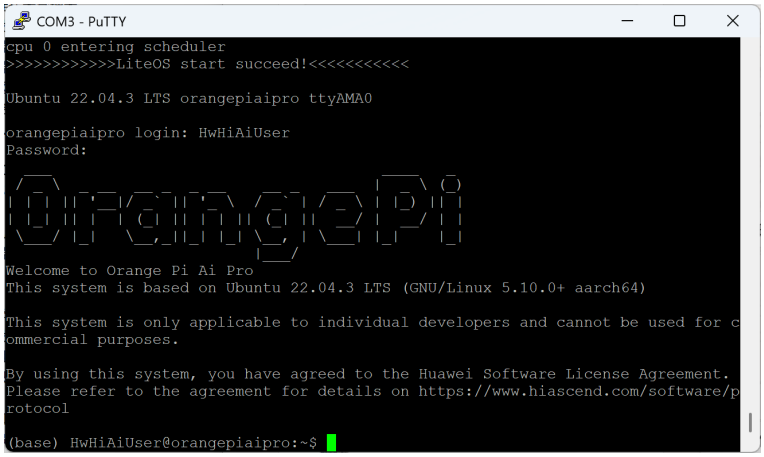
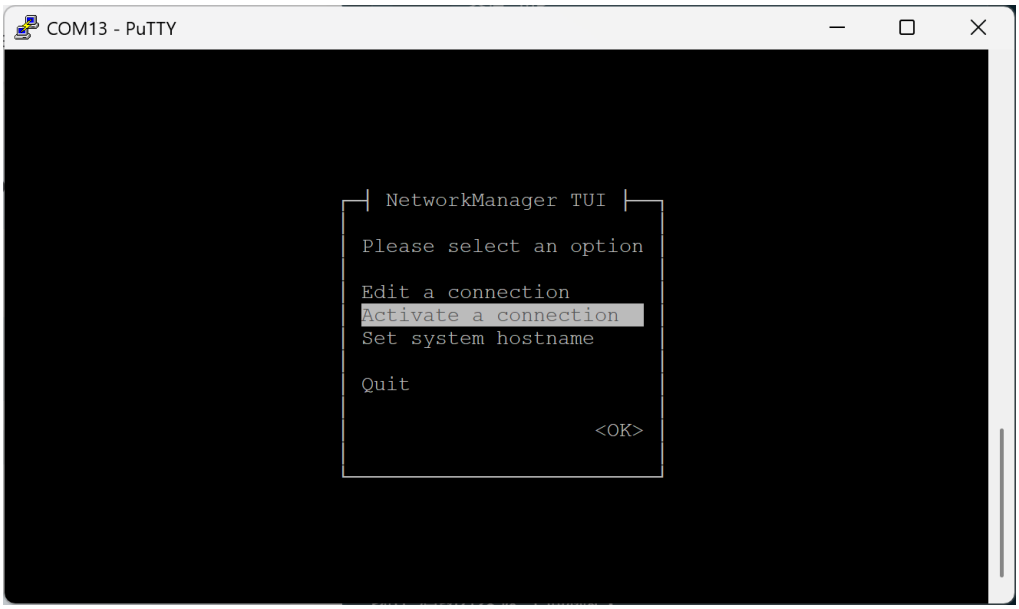


图 1.47. 登录成功

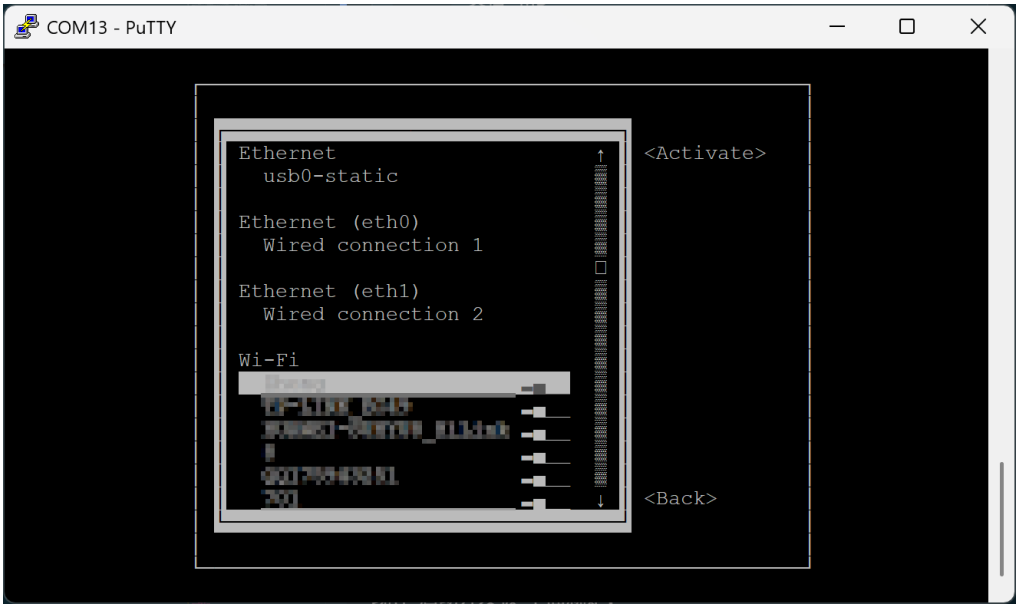
1. **打开 NetworkManager 终端界面**

```
sudo nmtui
```

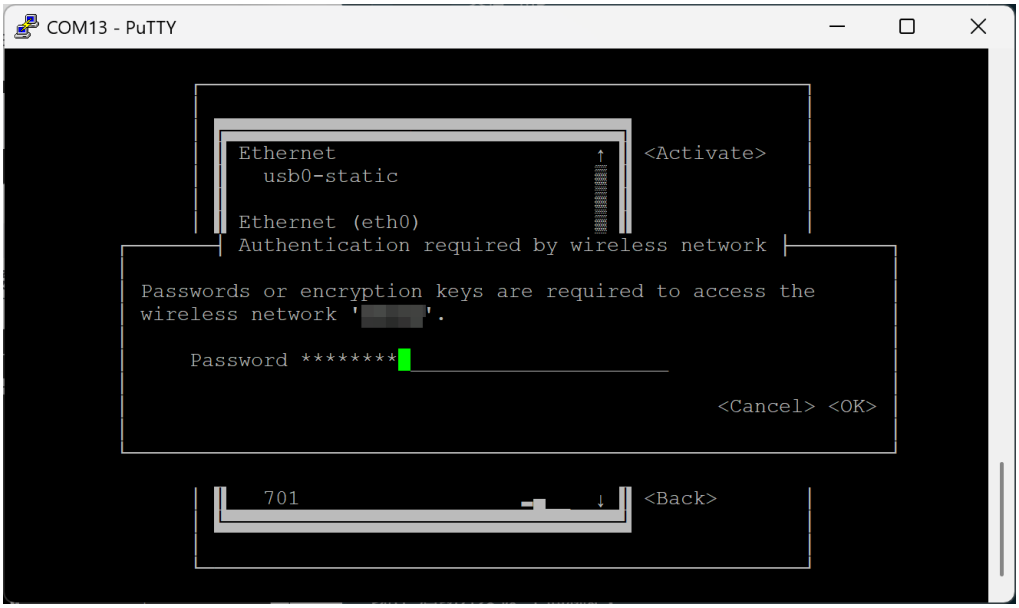
接着就会出现如下的界面：



2. **选择无线网络** 在界面中选择`Activate a connection`，进入连接列表后选择需要连接的 WiFi 名称。



3. **输入密码并连接** 按提示输入 WiFi 密码，确认后等待连接状态变为已连接。完成后选择`Back`返回，再选择`Quit`退出。



4. **确认网络状态**

```
nmcli connection show
ip addr
```

1.5.2. 有线网络连接

如果有线网络可用，直接插入网线即可。可以通过以下命令查看 IP 地址：

```
ip addr
```

或者

```
ifconfig
```

1.6. 远程连接 (SSH)

为了方便开发，通常我们会使用个人电脑通过 SSH 远程连接到开发板。

1. **获取开发板 IP 地址** 使用上述`ip addr`命令获取开发板的 IP 地址（通常在`wlan0`或`eth0`接口下）。
2. **使用 SSH 客户端连接** 在你的个人电脑终端（Windows 可以使用 CMD、PowerShell 或 Putty，Mac/Linux 使用 Terminal）中输入：

   ```
   ssh HwHiAiUser@<开发板IP地址>
   ```

   例如：

   ```
   ssh HwHiAiUser@192.168.1.100
   ```

   默认密码为：`Mind@123`

1.6.1. Windows PowerShell 配置 SSH 密钥免密码登录

首次使用密码登录确认开发板身份后，可以把开发机的公钥写入开发板账户。以后使用相同的开发机账户连接同一块板时，SSH 会使用私钥认证，不再要求输入开发板密码。密钥登录只省去认证步骤；它不替代对开发板 IP、用户名和主机密钥指纹的核对。

1. **先核对主机身份**
   第一次连接会显示开发板的主机密钥指纹。应在开发板本地终端核对后再接受，而不是因为连接方便就直接输入 `yes`：

   ```
   ssh-keygen -lf /etc/ssh/ssh_host_ed25519_key.pub
   ```

   若某个 IP 曾属于另一台板，SSH 可能报告主机密钥变化。先确认该 IP 确实是目标板，再只删除该目标的旧记录：

```
ssh-keygen -R <开发板IP地址>
```

不要删除整个 known_hosts 文件。

2. **在 Windows PowerShell 生成 Ed25519 密钥**
   以下命令只在本机尚未创建该密钥时生成它。私钥文件不得发送给开发板、其他电脑或 Git 仓库；只有 .pub 公钥可以安装到开发板。

```
$key = Join-Path $env:USERPROFILE '.ssh\id_ed25519'

if (-not (Test-Path -LiteralPath $key)) {
  ssh-keygen -t ed25519 -a 64 -f $key
}
```

根据提示设置私钥口令。口令保护的是开发机上的私钥，不是开发板账户密码。若需要自动化执行部署，可通过受保护的本机 SSH agent 管理私钥；不要为了省事把私钥提交到项目中。

3. **使用一次开发板密码安装公钥**
   将 <开发板IP地址> 替换为实际地址。此命令会要求输入一次 HwHiAiUser 的开发板密码，并创建权限正确的 .ssh 目录和 authorized_keys 文件。${key}.pub 使用花括号明确指出变量边界，避免 PowerShell 将 $key.pub 误解析为名为 pub 的属性。

```
Get-Content -Raw -LiteralPath "${key}.pub" | ssh HwHiAiUser@<开发板IP地址> `
  'umask 077; mkdir -p ~/.ssh; cat >> ~/.ssh/authorized_keys; chmod 700 ~/.ssh;
    ↪ chmod 600 ~/.ssh/authorized_keys'
```

对同一把密钥只需执行一次。若误重复执行，只会在 authorized_keys 中留下重复的同一公钥，不会生成新的访问权限；可在开发板上人工去重，但不要修改其他用户的条目。

4. **验证免密码登录**
   BatchMode=yes 会禁止 SSH 回退到交互式密码输入，因此该命令成功才说明公钥认证已生效：

```
ssh -o BatchMode=yes -o IdentitiesOnly=yes -i $key `
  HwHiAiUser@<开发板IP地址> 'hostname; whoami'
```

正常情况下会输出开发板主机名和 HwHiAiUser，且不再要求输入开发板密码。若仍然要求密码，先在开发板本地终端检查目录、文件权限和归属：

```
ls -ld ~/.ssh
ls -l ~/.ssh/authorized_keys
```

`.ssh` 应为 `700`，`authorized_keys` 应为 `600`，两者均应归 `HwHiAiUser` 所有。还应确认当前 PowerShell 会话中的 `$key` 指向实际私钥文件，公钥路径写法为 `"${key}.pub"`，而不是未定义变量时会指向当前目录 `.pub` 的 `"$key.pub"`。

5. **可选：为常用开发板设置别名**

   在 `C:\Users\<Windows用户名>\.ssh\config` 中添加一段配置后，可以用别名连接。不要把真实 IP、用户名、私钥或口令写入公开仓库。

```
Host ascend310b
  HostName <开发板IP地址>
  User HwHiAiUser
  IdentityFile ~/.ssh/id_ed25519
  IdentitiesOnly yes
```

   之后可使用：

```
ssh ascend310b
scp .\example.txt ascend310b:~/
```

1.7. 验证开发环境

系统启动并连接网络后，我们需要验证昇腾 AI 处理器的状态以及开发环境是否正常。

1. **查看 NPU 状态** 使用`npu-smi`工具查看 NPU 的详细信息，包括温度、功耗、算力利用率等。

```
npu-smi info
```

   如果能看到类似以下的输出，说明 NPU 工作正常：

```
+------------------------------------------------------------------------------------------------+
| npu-smi 23.0.0                       Version: 23.0.0                                           |
|------------------------------------------------------------------------------------------------|
| NPU     Name                         Health   Power(W)     Temp(C)                             |
| Chip    Device                       Bus-Id   AICore(%)    Memory-Usage(MB)                    |
|================================================================================================|
| 0       310B4                        OK       12.8         45                0                 |
| 0       0                            0000:00:00.0 0            2433 / 7564                     |
```

```
|===========================================================================================================|
```

2. **检查 CANN 环境** 官方镜像通常已预装 CANN（Compute Architecture for Neural Networks）。可以通过检查环境变量或运行简单命令来确认。通常环境变量设置在`.bashrc`中，尝试执行：

```
echo $ASCEND_HOME_PATH
```

或者检查编译器版本：

```
c++ --version
```

1.8. 结语

至此，你的昇腾 310B 开发板（OrangePi AIpro）已经完成了基本的环境搭建。接下来，你可以进入案例 1：智能考勤机的学习，开始你的第一个 AI 应用开发之旅。

附录 2. 基于昇腾 310B 的 Ubuntu 教程

本附录把 Ubuntu 基础操作、远程访问、网络配置和昇腾 310B 板端命令整理为一条可复用的学习路径。它面向三类位置：负责编辑代码和文档的**开发机**、负责连接网络和发起远程操作的 **Ubuntu 主机**、真正运行 CANN、PyACL、OM 推理和摄像头程序的 **310B 板端**。同一个命令在不同位置执行，得到的结果和证明范围不同，因此每一步都要先确认命令在哪台机器、哪个用户、哪个 shell 和哪个软件环境中运行。

本附录的命令来自 Ubuntu 基础教程中的文件结构、终端编辑器、SSH、VNC、NFS、Docker、热点与 WiFi、开机自启动、静态 IP 和无屏幕联网资料，并结合现有 310B 材料重新编排。外部资料中的 ROS 小车、树莓派、RDK X5、Orin 等名称只作为操作背景出现于命令示例的原始路径中，不能据此推断 310B 板的实际设备名、IP 地址、接口编号或运行结果。

2.1. 1. 教程定位与证据边界

2.1.1. 三类操作位置

位置	用途	能做什么	不能据此推断什么
开发机	编辑正文、样例代码和配置文件	文本编辑、Python 语法检查、纯 Python 测试、前端构建、静态文件检查	不能推断板端 CANN、PyACL、OM、摄像头、音频和 NPU 结果
Ubuntu 主机	作为 SSH、VNC、NFS、SCP 和 rsync 的客户端	发起远程连接、挂载共享目录、传输文件、运行普通 Ubuntu 命令	不能因为客户端命令成功就认定板端推理或硬件测试成功
310B 板端	运行目标 Ubuntu 系统与昇腾运行时	加载 CANN、执行 ATC、导入 acl、运行 OM、检查设备、服务和日志	不能把一次导入成功写成完整模型精度或端到端性能

开发机和 Ubuntu 主机可以共享普通 Linux 操作，例如查看路径、编辑文本、管理网络和传输文件。板端操作则需要额外的证据控制：`source /usr/local/Ascend/ascend-toolkit ↪ /set_env.sh`、`npu-smi info`、`atc`、`python -c 'import acl'`、OM 加载、V4L2 摄像头访问和真实 NPU 推理都应在 310B 板上执行。

2.1.2. 板端专属操作

以下操作默认只能在已经安装好驱动、CANN 和目标 Python 环境的 310B 板端执行：

```
source /usr/local/Ascend/ascend-toolkit/set_env.sh
command -v atc
python -c 'import acl; print("PyACL import: ok")'
npu-smi info
ls -l /dev/video*
v4l2-ctl --device=/dev/video0 --list-formats-ext
```

`command -v atc` 检查实际调用的 ATC 路径，`import acl` 检查当前 Python 是否能够加载 PyACL，`npu-smi info` 读取 NPU 设备摘要。三者属于不同的证据层：ATC 存在不代表转换成功，PyACL 可导入不代表模型能运行，`npu-smi` 能输出也不代表某个 OM 已经通过数值和性能验证。

2.1.3. 证据状态

记录结果时，使用下面五类标签，避免把先验知识写成已经观测到的事实：

标签	含义
`documented`	官方文档、厂商文档或教程中明确写出的能力
`inferred`	根据已有资料推断出的可能路径，尚未在目标板上验证
`observed-pass`	在当前板端、当前环境和当前命令下实际通过
`observed-fail`	在当前组合下实际失败，并保留了完整错误和命令
`untested`	尚未执行，不能归入支持或不支持

一条命令的通过只能证明它对应的最小验证门。语法检查、HTTP 健康检查、ATC 转换、ACL 数值烟测、任务精度、性能测试和界面烟测必须分别记录。没有执行的板端测试不能写成“已验证”，没有采集到的日志不能补写。

2.1.4. 执行命令前的四个问题

执行任何会改变文件、网络、用户、软件包或设备状态的命令前，先回答：

1. 命令在哪台机器、哪个 shell 和哪个 conda 环境中运行?
2. 输入文件、输出目录、日志和远端路径是否已经解析为明确的绝对路径?
3. 这一步证明的是语法、路径、网络、转换、数值一致性、性能，还是硬件现象?
4. 命令是否安装软件、修改权限、覆盖文件、删除数据或暴露网络服务?

四个问题没有答案时，先执行只读检查，不要直接执行写入命令。

2.2. 2. Ubuntu 与 Linux 基础

2.2.1. 终端、用户和提示符

Ubuntu 的图形界面之外，终端是与 Linux 交互的主要入口。提示符 `$` 通常表示普通用户，`#` 通常表示 root 用户。进入 root 状态以后，所有命令的权限都会提高，误操作的后果也会放大。

```
whoami
id
hostname
pwd
printf 'shell=%s\n' "$SHELL"
```

`whoami` 显示当前用户名，`id` 同时显示用户、组和补充组，`hostname` 显示主机名，`pwd` 显示当前目录，`$SHELL` 显示当前 shell 路径。远程登录后先确认这些信息，避免把命令发到另一台同名设备。

退出临时 root 状态或当前 shell 可使用：

```
exit
```

也可以按 `Ctrl+D` 发送文件结束符。不要为了省事长期保留 root 终端，更不要在 root 终端中编辑项目文件。

2.2.2. 第一条环境检查命令

```
uname -a
cat /etc/os-release
date --iso-8601=seconds
ip addr
ip route
```

`uname -a` 查看内核和架构，`/etc/os-release` 查看 Ubuntu 版本，`date` 生成带时区含义的时间戳，`ip addr` 查看接口地址，`ip route` 查看默认路由。把这几条命令的输出和后续测试放在同一份记录中，才能说明测试环境。

2.2.3. 可审计 shell

脚本或排障会话可以使用严格模式：

```
set -euo pipefail
pwd
whoami
```

```
hostname
printf 'shell=%s\n' "$SHELL"
```

`set -e` 在未处理的命令失败后停止脚本，`-u` 让未定义变量直接报错，`pipefail` 保留管道前段的失败状态。某个命令允许失败时，应显式处理返回值，不能为了继续执行而全局关闭严格模式。

交互式排查时可以先只运行环境检查命令。脚本中如果必须临时关闭 `-u` 来加载厂商环境脚本，应在加载完成后恢复原状态：

```
load_cann() {
  if [[ $- == *u* ]]; then
    set +u
    source /usr/local/Ascend/ascend-toolkit/set_env.sh
    set -u
  else
    source /usr/local/Ascend/ascend-toolkit/set_env.sh
  fi
}
```

2.2.4. 命令权限与执行位置

同一句命令在开发机、Ubuntu 客户端和板端的结果可能不同。使用 `command -v` 查看实际可执行文件：

```
command -v python
command -v python3
command -v atc || true
command -v npu-smi || true
```

`|| true` 只适合让检查流程继续，不代表后面的命令可以跳过环境问题。若 `atc` 或 `npu-smi` 不存在，应记录当前机器和当前环境，而不是修改代码来掩盖缺失的板端依赖。

2.3. 3. 文件系统、路径与常用命令

2.3.1. Ubuntu 目录结构

目录	主要用途
`/bin`	重要的二进制应用程序
`/boot`	启动配置文件和内核相关文件
`/dev`	设备文件，以文件形式表示设备和接口
`/etc`	系统配置文件、启动脚本和服务配置

目录	主要用途
/home	普通用户主目录
/lib	系统库文件
/mnt	临时挂载文件系统的位置
/opt	可选应用程序安装目录
/usr/local	用户自行安装的软件和运行环境

/dev 中的文件不是普通文档。/dev/video0、/dev/snd 和 /dev/bus/usb 对应真实设备，访问它们会改变设备状态或与硬件交互。/etc 下的文件通常需要 root 权限，修改前应备份并确认目标主机。

2.3.2. 绝对路径、相对路径和工作目录

绝对路径从 / 开始，例如 /home/<user>/Documents/ascend310。相对路径从当前工作目录开始，例如 models/example.om。使用 .. 返回上一层：

```
cd ~/Documents
pwd
cd ..
pwd
cd -
pwd
```

cd 不带参数时返回用户主目录，cd - 返回上一个工作目录。进入服务、转换或同步命令之前，先执行 pwd，确认没有在错误目录中运行。

解析实际路径：

```
realpath .
realpath models/example.om
readlink -f frontend/dist/index.html
```

realpath 和 readlink -f 适合在复制、删除、挂载和部署前确认符号链接真实指向。路径中包含空格时，变量和参数应使用双引号：

```
target_dir="$HOME/Documents/board assets"
mkdir -p "$target_dir"
cp -a config.example.yaml "$target_dir/"
```

2.3.3. 创建、查看和复制

```
mkdir -p reports/board/$(date --iso-8601=date)
touch notes.txt
```

```
ls -la
ls -lh models
cp -a config.example.yaml config.yaml
cp -r templates templates-backup
mv old-name.txt new-name.txt
```

`mkdir -p` 创建多级目录，`cp -a` 尽量保留文件属性，`cp -r` 复制目录，`mv` 同时用于移动和重命名。覆盖已有文件之前，先用 `ls`、`test -e` 或 `stat` 检查目标：

```
test -e config.yaml && echo 'target exists'
stat -c '%A %U:%G %s %n' config.yaml
file config.yaml
```

`stat` 显示权限、所有者、字节数和名称，`file` 只根据文件内容给出格式线索，不能证明模型可以由 ACL 加载。

2.3.4. 查看文本和日志

```
head -n 40 service.log
tail -n 80 service.log
tail -f service.log
wc -l service.log
sed -n '1,80p' service.log
awk '{print NR ":" $0}' service.log | tail -n 20
grep -n 'ERROR\|WARN' service.log
```

`tail -f` 持续跟踪日志，适合前台排障；批量检查应使用 `head`、`tail`、`wc`、`sed` 和 `grep`，并把输出保存到报告。仓库内搜索可使用 `rg`：

```
rg -n 'atc|acl|sha256|127\.0\.0\.1|0\.0\.0\.0' README.md docs scripts
```

2.3.5. 查找、容量和文件类型

```
find . -maxdepth 2 -type f -print | sort
du -sh models data reports
df -h .
lsblk
```

`find` 先用 `-maxdepth` 限制范围，避免扫描整个家目录或挂载盘；`du` 查看目录占用，`df` 查看文件系统剩余空间，两者含义不同。`lsblk` 只用于查看块设备树，不能未经确认就挂载或格式化设备。

2.3.6. 归档和校验

```
run_date="$(date --iso-8601=date)"
tar -czf "reports/board/smoke-$run_date.tar.gz" reports/board/smoke.log
tar -tzf "reports/board/smoke-$run_date.tar.gz" | head
unzip -l artifact.zip
sha256sum models/example.om
```

压缩前先确认内容。不要把照片、生物特征模板、数据库、模型二进制、密钥和 token 放入公开归档。`sha256sum` 只能证明字节一致，不能证明模型正确、可加载或数值正确。

2.4. 4. 用户、组、权限、sudo 与安全删除

2.4.1. Linux 用户和组

Ubuntu 安装时会创建一个普通用户，同时保留具有更高权限的 root 用户。普通用户在自己的主目录中通常不需要 `sudo`，访问 `/etc`、安装软件包或修改系统服务时才需要提权。

每个文件都有三组权限：

权限组	适用范围
user	文件所有者
group	文件所属组
other	其他用户

每组权限包含读 `r`、写 `w`、执行 `x`：

```
ls -l script.sh
stat -c '%A %a %U:%G %n' script.sh
id
groups
```

`ls -l` 的权限字符串分为三段。`rwxr-xr-x` 表示所有者可读写执行，组和其他用户可读可执行。`stat` 的 `%a` 给出八进制权限，`%U:%G` 给出所有者和组。

2.4.2. 修改权限

给脚本增加所有者执行权限：

```
chmod u+x scripts/run_service.sh
bash -n scripts/run_service.sh
./scripts/run_service.sh --help
```

`chmod u+x` 只给文件所有者增加执行权限。若脚本需要其他用户执行，应先确认运行用户和目录权限，再选择精确的组权限或 ACL。不要对整个家目录、模型目录或系统目录执行 `chmod -R 777`。

修改所有者和组需要管理员权限：

```
sudo chown "$USER":"$USER" path/to/file
sudo chgrp developers path/to/directory
```

`chown -R` 和 `chgrp -R` 会递归改变整棵树，执行前必须用 `find` 预览目标，并确认没有系统文件和共享资产混在其中。

2.4.3. sudo

`sudo` 表示以超级用户权限执行单条命令：

```
sudo apt-get update
sudo systemctl status ssh --no-pager
sudo -i
```

`sudo -i` 进入 root 环境，`sudo su` 也可能加载 root 环境或保留部分当前环境。两者都不是日常编辑项目文件的默认方式。退出 root 状态使用：

```
exit
```

频繁使用系统管理命令时，应通过明确的 sudo 策略和组权限管理，而不是把普通用户长期改成 root。

2.4.4. 设备组权限

摄像头和音频设备常通过 `video`、`audio` 等组授权：

```
groups
ls -l /dev/video* /dev/snd
sudo usermod -aG video "$USER"
```

执行 `usermod` 后需要重新登录，组变更才会进入新会话。不要用 `sudo python` 启动整个服务来绕开设备权限，因为这会改变服务的数据目录所有者、环境变量和日志归属。

2.4.5. 安全删除

`rm` 不提供回收站。删除测试数据时，先把目标解析为绝对路径并验证：

```
target_dir="$(realpath -- "$PALMPRINT_ROOT/data/captures")"
```

```
case "$target_dir" in
  "$PALMPRINT_ROOT"/data/captures) ;;
  *) printf 'refuse to remove: %s\n' "$target_dir" >&2; exit 1 ;;
esac
find "$target_dir" -maxdepth 1 -type f -print
rm -f -- "$target_dir"/*.jpg
```

先运行 `find` 检查文件列表，再由操作者确认后执行删除。`rm -rf` 只应在脚本创建的隔离临时目录中使用，并且变量必须非空、路径必须已经解析。不要把变量为空、路径未验证的命令变成 `rm -rf "$HOME"`、`rm -rf .` 或对整个部署目录的递归删除。

2.5. 5. nano 与 vim 编辑器

2.5.1. nano

打开文件：

```
nano config.yaml
sudo nano /etc/exports
```

nano 进入后可以直接编辑文本。保存时按 `Ctrl+O`，屏幕会显示文件名；不修改文件名就按回车。退出按 `Ctrl+X`。如果文件有未保存修改，nano 会询问是否保存，按屏幕提示选择。

编辑系统文件前先备份：

```
sudo cp /etc/apt/sources.list /etc/apt/sources.list.bak
sudo nano /etc/apt/sources.list
```

2.5.2. vim

打开文件：

```
vim config.yaml
sudo vim ~/.config/autostart/myprogram.desktop
```

vim 初始处于普通模式。按 `i` 或 `a` 进入插入模式，按 `Esc` 回到普通模式。常用移动和编辑键：

按键	作用
`k` / `j`	光标上移 / 下移
`h` / `l`	光标左移 / 右移
`i` / `I`	当前光标处插入 / 行首插入

按键	作用
a / A	光标后插入 / 行末插入
o / O	当前行后新建一行 / 当前行前新建一行
x / X	删除当前字符 / 删除前一个字符
dd	剪切整行，也可作为删除
dw	删除一个单词
d^	删除到行首
dG / d1G	删除到文档末尾 / 删除到文档开头
gg / Shift+g	移到第一行 / 最后一行
yy / p / P	复制整行 / 粘贴到光标后 / 粘贴到光标前

保存和退出前先按 Esc 回到普通模式，再输入冒号命令：

命令	作用
:q	退出
:q!	强制退出，不保存
:w	保存
:wq	保存并退出
:wq!	强制保存并退出
:w <文件路径>	另存为

编辑器中的 :q! 会丢失未保存内容，只在确认不需要修改时使用。需要编辑 root 文件时使用 sudo nano 或 sudo vim，不要把来源不明的 tee 保存技巧直接粘贴到终端。保存后使用 head、grep 或 git diff 检查文件内容。

2.6. 6. APT、换源、Conda、pip 与 Python 路径

2.6.1. APT 软件包管理

APT 操作会改变系统状态。执行前记录 Ubuntu 版本和计划安装的包：

```
cat /etc/os-release
sudo apt-get update
sudo apt-get install -y package-name
sudo apt-get install -f
```

apt-get update 获取软件包列表，install 安装软件包，install -f 修复依赖。安装完成后记录命令、版本和输出：

```
dpkg -s python3-dev python3-pip 2>/dev/null | grep -E '^(Package|Status|Version):'
apt-cache policy package-name
```

不要为了通过文档示例而重复安装或升级 CANN 相关包。板端已经验证过的 CANN、驱动和 Python 环境应保持原样，变更前先建立回滚方案。

2.6.2. 换源

Ubuntu 的软件源配置通常位于 `/etc/apt/sources.list`，部分新版本还会使用 `/etc/apt/sources.list.d/` 下的 `.sources` 文件。先备份，再根据 Ubuntu 版本替换为合适的国内镜像：

```
sudo cp -a /etc/apt/sources.list /etc/apt/sources.list.bak
sudo nano /etc/apt/sources.list
sudo apt-get update
```

换源时确认发行版代号与镜像仓库匹配。不要混用不同 Ubuntu 版本的仓库，不要把第三方 PPA 当作系统基础源。更新失败时先恢复备份并检查错误，不要把错误源继续保留。

2.6.3. Conda 环境

先加载 Conda shell 函数，再激活环境：

```
source /usr/local/miniconda3/etc/profile.d/conda.sh
conda activate base
python --version
python -c 'import sys; print(sys.executable)'
```

若脚本开启了 `set -u`，而 Conda 或 CANN 环境脚本读取未定义变量，可以使用第 2 节的 `load_cann` 模式，只在加载厂商脚本时临时关闭 `-u`，加载完成后恢复。

创建独立环境：

```
conda create -n case9-acl-om python=3.9
conda activate case9-acl-om
python -c 'import sys; print(sys.executable)'
```

环境名称和 Python 版本必须来自项目的版本要求。不要用其他版本的同名环境替换已经验证的板端环境。

2.6.4. pip 与 Python 解释器

优先使用 `python -m pip`，让 pip 对应当前解释器：

```
python -m pip --version
python -m pip install -r requirements.txt
```

```
python -m pip install -r requirements-board.txt
python -m pip list
```

不要使用 `sudo pip install` 或 `sudo python`。前者会把包安装到系统目录并绕过 Conda，后者会改变脚本的解释器和文件所有权。安装失败时先记录 `python -m pip --version`、`which python`、`sys.executable` 和完整错误，再决定是否变更环境。

2.6.5. Python 路径和动态库路径

```
export PYTHONNOUSERSITE=1
export PYTHONPATH="$PWD:${PYTHONPATH:-}"
export LD_LIBRARY_PATH="/usr/local/Ascend/ascend-toolkit/latest/lib64:${LD_LIBRARY_PATH:-}"
python -c 'import sys; print("\n".join(sys.path))'
python -c 'import os; print(os.environ.get("LD_LIBRARY_PATH", ""))'
```

`PYTHONNOUSERSITE=1` 用于避免用户目录中的包遮蔽板端环境，但可能让实际启动环境中存在的用户站点包不可见。判断导入问题时，应比较隔离环境和真实启动环境的 `sys.executable` 与 `sys.path`。`PYTHONPATH` 和 `LD_LIBRARY_PATH` 只加入已经确认的项目目录或 CANN 目录。持久化修改应经过部署评审，不能临时写入 `/etc/profile` 后让所有用户和所有服务继承。

2.7. 7. 网络、静态 IP、WiFi、热点与同网段检查

2.7.1. 网络状态

```
ip addr
ip route
nmcli connection show
nmcli -t -f NAME connection show
ping -c 4 <target-ip>
```

`ip addr` 查看接口地址，`ip route` 查看默认路由和网段，`nmcli connection show` 查看 NetworkManager 连接配置。`ping` 成功只说明网络层可达，不代表 SSH、VNC、NFS 或 NPU 服务已经可用。

2.7.2. 一般静态 IP 概念

Ubuntu 图形桌面可以通过 NetworkManager 配置静态 IP。打开连接编辑器：

```
nm-connection-editor
```

在对应连接中进入 IPv4 Settings，将方法设为 Manual，再填写地址、子网掩码、网关和 DNS。静态地址留在目标设备所在网段内，且不能与其他设备冲突。修改后重新启用该连接，再验证：

```
ip -4 addr show
ip route
ping -c 4 <同网段设备IP>
```

静态 IP 不等于端口已经开放。SSH 服务、VNC 共享开关、NFS 导出和防火墙都需要单独检查。不要在不确定目标网段和网关的情况下直接覆盖原配置，先在连接编辑器中记录原值，或保存一份配置备份。

2.7.3. 同网段判断

IPv4 地址由网络地址和主机地址组成，子网掩码决定两部分的分界。以 255.255.255.0 为例，前 24 位是网络地址，后 8 位是主机地址。192.168.1.10 和 192.168.1.20 与同一子网掩码进行按位与运算后，网络地址都是 192.168.1.0，因此属于同一网段。192.168.0.100 与 192.168.1.100 在同一掩码下不属于同一网段。

排障时检查客户端和板端是否拿到同一网段的地址、是否连接到同一个热点或有线网络、网关是否一致。若客户端和板端存在多个网卡，应使用 ip route get <目标IP> 确认实际出口接口，不要只看某一个网卡的地址。

2.7.4. 创建 WiFi 热点

图形界面通常从右上角网络图标进入 Edit Connections，点击加号，选择 WiFi，再点击 Create。命令行方式可以打开连接编辑器：

```
nm-connection-editor
```

在 Wi-Fi 选项卡中设置：

- SSID：热点名称，和连接配置名称 connection name 不是一回事。
- Mode：选择 Hotspot。
- Band：可选自动、2.4 GHz 或 5 GHz。频段拥挤时固定频段可能更稳定。
- Device：多网卡主机需要指定实际使用的无线网卡。
- Wi-Fi Security：选择 WPA&WPA2 Personal 并设置密码。
- IPv4 Settings：按需填写静态地址、掩码和网关。
- IPv6：不需要时可选择忽略。

一个无线网卡通常不能同时作为热点又连接外部 WiFi。需要同时提供热点和连接上游网络时，应使用两个独立网卡，并分别确认设备名和驱动状态。

2.7.5. 热点与 WiFi 模式切换

图形界面中先断开当前热点或 WiFi，再选择要连接的网络。重新开启热点时，从网络设置菜单选择热点配置并启用。切换后检查：

```
ip addr
iw dev
nmcli device status
nmcli connection show --active
```

`iw dev` 是否可用取决于系统是否安装相关工具；即使命令不存在，也应记录 `nmcli device status` 和界面状态，而不是直接推断网卡损坏。切换过程中，已经连接的 SSH 会话可能短暂断开。使用无屏幕板端时，先准备好重新发现的 IP 的方法，再修改网络模式。

2.7.6. 自动连接优先级

热点和 WiFi 配置可以在 `General` 选项卡中设置 `Connect automatically with priority`。不同版本的教程对数值方向描述不一致：有的资料写 “数值越大优先级越高”，有的资料写 “数值越小优先级越高”。不要只照抄数字，应查看当前 NetworkManager 的实际配置并做一次重启验证：

```
nmcli -f connection.id,connection.autoconnect,connection.autoconnect-priority connection show
```

记录哪一个连接在重启后实际被选中，再据此调整优先级。修改网络配置后，若板端没有屏幕，先确认仍能通过已有热点或串口获取 IP，防止失去远程入口。

2.7.7. 无屏幕联网的基本流程

无屏幕板端的基本顺序是：默认热点可用，客户端连接热点，通过 SSH 登录，扫描可用 WiFi，选择目标 SSID 并输入密码，获取新的 IP，然后让客户端连接到同一 WiFi。切换时可能短暂断开当前热点，需要准备重新连接或通过串口查看 IP。

```
ssh -Y <user>@<board-ip>
nmcli device wifi list
nmcli connection show
ip addr
```

具体热点名称、默认 IP 和脚本路径必须来自当前板端的实际配置。不要把其他产品的示例热点名或脚本路径当作 310B 板的默认值。成功连接 WiFi 后，IP 可能改变；应记录 DHCP 分配的地址或为板端设置不冲突的静态地址。

2.8. 8. SSH 远程命令行控制

2.8.1. 客户端与服务端

SSH 使用客户端和服务端模型。310B 板端作为服务端运行 `sshd`，Ubuntu 主机作为客户端发起连接。客户端连接后看到的 shell 就是板端 shell，因此文件路径、权限、设备和运行时环境都以板端为准。

2.8.2. 安装和启动服务端

在板端确认 SSH 状态：

```
sudo service ssh status
```

未安装时执行：

```
sudo apt update
sudo apt install openssh-server
sudo service ssh start
sudo systemctl enable ssh
```

再次检查：

```
systemctl status ssh --no-pager
ss -ltnp | grep ':22'
```

Ubuntu 主机通常自带 SSH 客户端。需要单独安装时执行：

```
sudo apt-get install openssh-client
ssh -V
```

2.8.3. 网络连通性检查

先连接板端 WiFi 或有线网络，再检查同网段和可达性：

```
ip addr
ip route
ping -c 4 192.168.0.100
```

若无法 ping 通，依次检查：两台设备是否在同一网段、客户端是否连接了错误的网络、板端 IP 是否已经变化、路由出口是否正确、无线热点是否仍处于开启状态。`ping` 不通时不要直接反复执行 `ssh`，先把网络层问题定位清楚。

2.8.4. 登录

源文档中的典型命令是：

```
ssh -Y wheeltec@192.168.0.100
```

其中 `-Y` 启用受信任的 X11 转发，`wheeltec` 是服务端用户名，`192.168.0.100` 是服务端地址。实际使用时替换为已经确认的值，不要复制示例密码。仅需要命令行时可以使用普通 SSH：

```
ssh <user>@<board-ip>
ssh <user>@<board-ip> 'hostname; uname -a; npu-smi info'
```

登录后提示符中的用户名和主机名会变化。若用户名仍是本地用户，说明没有进入远端 shell。需要执行单条板端命令时，把命令放在引号中，并确认引号内的变量是在远端还是本地展开。

2.8.5. known_hosts 冲突

第一次连接某台设备或设备身份发生变化时，可能出现主机密钥冲突。源文档给出的处理命令是：

```
ssh-keygen -f "/home/wheeltec-client/.ssh/known_hosts" -R "192.168.0.100"
ssh -Y wheeltec@192.168.0.100
```

第一条命令只删除指定主机在 `known_hosts` 中的记录，第二条重新连接。执行删除前先确认 IP 确实属于目标板端，并核对新的主机密钥指纹。不要为了省事直接删除整个 `known_hosts`，也不要在未确认设备身份时盲目确认新密钥。

2.8.6. 同一网段排障

如果 SSH 连接失败，检查客户端和板端的 IPv4 地址与子网掩码：

```
ip -4 addr show
ip route
ping -c 4 <board-ip>
```

静态 IP 必须位于同一网段，且不能与板端地址或其他设备冲突。若修改的是客户端地址，保留原配置；若修改的是板端地址，先确认自己有第二条访问通道。路由器、热点、交换机和防火墙都可能影响可达性。

2.8.7. SSH 安全注意事项

- 使用强密码或 SSH 密钥，优先使用密钥登录。
- 只允许可信网络访问 22 端口，不要把 SSH 直接暴露到公网。
- 需要远程管理时，先确认服务端主机密钥指纹，再建立连接。
- `ssh -Y` 会转发 X11 连接，信任边界较宽，只在受信网络中使用。
- 多人共用板端时，为每个用户分配账号，不要共享 root 密码。
- 修改防火墙或 SSH 配置后，保留一个已登录会话，再从新会话验证，避免把自己锁在系统外。

2.9. 9. VNC 远程桌面控制

2.9.1. 客户端与服务端

VNC 也采用客户端和服务端模型。板端提供桌面服务，Ubuntu 主机或 Windows 主机运行 VNC 客户端。远程桌面适合查看图形窗口、图像预览和需要鼠标交互的程序；纯命令行任务优先使用 SSH。

2.9.2. Ubuntu 客户端安装 Remmina

源文档给出的 Remmina 安装命令是：

```
sudo apt-add-repository ppa:remmina-ppa-team/remmina-next
sudo apt update
sudo apt install remmina remmina-plugin-rdp remmina-plugin-secret
```

PPA 是否适合当前 Ubuntu 版本需要先确认。若系统仓库已经提供合适版本，也可以只安装 `remmina`。安装完成后从应用菜单启动，选择 VNC 协议，填写板端 IP 地址，再输入 VNC 密码。源文档示例 IP 为 `192.168.0.100`，实际使用时必须替换。

2.9.3. Windows 客户端使用 MobaXterm

在 Windows 主机上启动 MobaXterm，点击左上角 `Session`，选择 `VNC`，填写板端 IP 地址，确认客户端已经连接到板端所在网络，然后点击 `OK` 并输入 VNC 密码。不同版本的菜单名称可能略有变化，但客户端、目标 IP、密码和网络可达性这几项不变。

2.9.4. 分辨率和画面质量

板端没有连接显示器时，VNC 分辨率可能较低。源文档给出的调整命令是：

```
xrandr --fb 1024x768
```

`xrandr --fb` 设置当前 X 显示的帧缓冲尺寸。执行前先运行 `xrandr` 查看可用输出和模式，确认目标分辨率不会超出显示能力。若画面卡顿，可以在 Remmina 中降低画质、颜色深度或更新频率。图像质量和实时性需要权衡，不要把降低画质后的流畅度当作原始编码性能。

2.9.5. 共享开关

部分 Ubuntu 桌面镜像使用共享设置提供 VNC。修改网线、IP 或网络模式后，VNC 可能断开，需要检查桌面设置中的 `Sharing` 和 `Screen Sharing` 是否仍为活动状态。带有远程桌面协议的版本可能还需要启用兼容 VNC 协议的选项。共享开关关闭时，SSH 可能仍正常，但 VNC 无法连接。

2.9.6. 屏幕和显示要求

部分板端镜像在没有连接显示器时不会生成可用的桌面帧缓冲。VNC 连接前可能需要连接真实屏幕或使用显示欺骗器。无屏幕场景应先确认 VNC 服务、共享开关、显示输出和分辨率，再判断是否为客户端问题。

2.9.7. VNC 中打开终端

大部分 VNC 客户端无法让 `Ctrl+Alt+T` 正确传递到远端桌面。源文档建议使用鼠标右键菜单中的终端入口。打开终端后，先用 `whoami`、`hostname` 和 `pwd` 确认会话确实运行在板端。

2.9.8. SSH 与 VNC 的对比

对比项	SSH	VNC
传输内容	终端文本和可选 X11 转发	桌面图像、鼠标和键盘事件
带宽占用	通常较低	通常较高，受分辨率和画质影响
适合任务	命令、日志、脚本、服务管理	图形程序、图像窗口、桌面交互
无屏幕要求	不要求桌面帧缓冲	部分镜像要求显示器或显示欺骗器
安全边界	可使用密钥和严格访问控制	需要保护 VNC 密码和端口
断线影响	前台命令可能随会话结束	前台图形程序可能随会话结束

VNC 服务不要直接暴露到不可信网络。需要通过 SSH 隧道访问时，可以把本地端口转发到板端 VNC 端口，但应确认 VNC 监听地址和端口，并使用唯一强密码。

2.10. 10. NFS、SCP 与 rsync

2.10.1. NFS 的角色

NFS 把服务端目录映射到客户端挂载点。典型的板端作为服务端，Ubuntu 主机作为客户端。挂载后，客户端编辑文件就像编辑本地目录，但底层文件仍属于远端文件系统，断网、服务端关机或网络切换会造成挂载失效。

2.10.2. 服务端安装和导出

板端安装服务端：

```
sudo apt-get install nfs-kernel-server
```

编辑导出配置：

```
sudo nano /etc/exports
```

文件内容示例：

```
/srv/ascend310-share *(rw,sync,no_root_squash)
```

第一列是共享目录，第二列是允许访问的客户端范围。生产或多人环境中应把 `*` 收窄为具体网段或主机地址，并根据需要改为 `ro`。`no_root_squash` 会让客户端 root 保留 root 权限，安全风险较高；只有在隔离实验网络中经过评估后才使用。

源文档还出现过下面的旧示例：

```
/home/wheeltec/wheeltec_ros2 *(rw,sync,no_root_squash)
```

该路径只是示例，不是 310B 板的默认路径。使用时必须替换为实际共享目录，并确认目录存在、内容范围和客户端权限。

启动服务：

```
sudo /etc/init.d/nfs-kernel-server start
sudo /etc/init.d/nfs-kernel-server restart
systemctl status nfs-kernel-server --no-pager
exportfs -v
```

`exportfs -v` 查看当前导出，`systemctl status` 查看服务状态。修改 `/etc/exports` 后通常需要重新加载或重启服务。

2.10.3. 客户端安装和挂载

Ubuntu 客户端安装：

```
sudo apt-get install nfs-common
```

创建挂载点：

```
sudo mkdir -p /mnt/mount_nfs
```

先确认可达，再挂载：

```
ping -c 4 <server-ip>
sudo mount -t nfs <server-ip>:/srv/ascend310-share /mnt/mount_nfs
mount | grep /mnt/mount_nfs
```

需要指定 NFS 版本或锁选项时，使用明确的参数：

```
sudo mount -t nfs -o nolock <server-ip>:/srv/ascend310-share /mnt/mount_nfs
```

卸载：

```
sudo umount -t nfs <server-ip>:/srv/ascend310-share /mnt/mount_nfs
```

若服务端曾经关机，挂载可能变成失效状态，重新挂载前先卸载。不要对正在写入的共享目录执行 `umount -f`，除非已经确认没有进程使用并且接受数据风险。

2.10.4. NFS 权限和安全

源文档中的旧配置使用了宽泛权限。下面的命令只作为说明，不建议直接照抄：

```
sudo chmod -R 777 /path/to/share
sudo chown -R 777 nobody /path/to/share
```

`chmod -R 777` 会向所有用户开放读写执行；`chown -R 777 nobody` 不是通用安全的用户和组写法。更合理的做法是创建专用组、设置 `u+rwX` 或组权限、把客户端限制在同一网段，并在不需要写入时使用只读导出。模型、数据、模板和报告目录应按实际所有者设置权限，不要用一次宽泛递归命令处理整个家目录。

2.10.5. SCP 定向传输

SCP 使用 SSH 传输文件：

```
board_user="replace-with-board-user"
board_ip="replace-with-board-ip"
ssh_target="$board_user@$board_ip"
scp demo/vtest.avi "$ssh_target:Documents/ascend310/demo/"
scp "$ssh_target:Documents/ascend310/reports/result.json" .
```

下载和上传前都先检查目标目录和磁盘空间：

```
ssh "$ssh_target" 'df -h "$HOME"; test -d "$HOME/Documents/ascend310/demo"'
```

2.10.6. rsync 同步

rsync 适合目录同步和断点续传。先预览：

```
rsync -av --dry-run samples/case8/ \
  "$ssh_target:Documents/ascend310/samples/case8/"
```

确认输出后再执行：

```
rsync -av --protect-args \
  samples/case8/ \
  "$ssh_target:Documents/ascend310/samples/case8/"
```

默认不要使用 `--delete`。它会删除远端已有文件，可能误删模型、数据或报告。需要排除大资产时：

```
rsync -av --protect-args \
  --exclude 'data/' --exclude 'models/' --exclude '*.om' \
  samples/case8/ \
  "$ssh_target:Documents/ascend310/samples/case8/"
```

同步后分别在本地和远端执行 `wc -c`、`sha256sum` 或清单检查。模型、OM、数据集和真实运行报告不应因为一次同步就进入 Git。

2.11. 11. Docker 基础与 ROS2 容器命令

2.11.1. Docker 基本概念

Docker 使用镜像和容器组织应用。镜像包含文件系统和运行环境，容器是镜像的一次隔离运行实例。容器与宿主系统共享内核，但可以使用独立的文件系统、网络和进程空间。不要把容器当作可以随意获得所有硬件权限的沙箱。

检查 Docker 是否安装和运行：

```
docker --version
systemctl status docker --no-pager
sudo systemctl start docker
sudo systemctl enable docker
```

查看容器和镜像：

```
docker ps
docker ps -a
docker images
```

`docker ps` 只显示运行中的容器，`docker ps -a` 显示包括已停止的容器，`docker images` 显示本地镜像。

2.11.2. 通用容器生命周期命令

```
docker start <container-name>
docker exec -it <container-name> bash
exit
docker stop <container-name>
docker restart <container-name>
docker update --restart=always <container-name>
docker inspect <container-name>
```

`docker exec -it` 进入正在运行的容器，`exit` 退出容器 shell 返回宿主。`docker update --restart=always` 会让容器随 Docker 服务自动启动，设置前应确认应用具备断电恢复和安全停机策略。

2.11.3. 教程中的 ROS2 容器命令

部分镜像提供了封装脚本，树莓派 5 的 ROS2 示例命令名称为：

```
ros2_start
ros2
ros2_stop
```

```
ros2_restart
```

这些名称是镜像提供的帮助命令或别名，不是 Docker 自带子命令。执行 `ros2_start` 启动容器，`ros2` 进入容器环境，`ros2_stop` 停止容器，`ros2_restart` 重启容器。使用前先检查它们是脚本、别名还是函数：

```
type ros2_start
type ros2
type ros2_stop
type ros2_restart
```

进入容器后看到的用户提示符可能变化。容器状态异常时先查看 `docker ps -a`、`docker logs <container-name>` 和 `docker inspect`，不要直接删除容器或镜像。ROS2 节点、话题、launch 和工作空间编译属于 ROS2 内容，应放在附录 5 中讲解。

2.11.4. 源码目录、挂载和编译

容器可以把宿主目录挂载到容器路径。源文档提示容器内的源码可能已经映射到宿主目录，因此可以在宿主或容器中修改文件，但编译必须在安装了对应构建系统的容器中完成。使用前确认挂载关系：

```
docker inspect -f '{{json .Mounts}}' <container-name>
docker exec -it <container-name> pwd
docker exec -it <container-name> ls -la
```

不要把宿主 `/`、`/etc`、Docker socket 或整个家目录挂载到不可信容器。需要摄像头、串口、USB 或 NPU 设备时，逐项确认设备节点和权限，不要直接使用 `--privileged`。

2.11.5. 图形窗口和 DISPLAY

容器中的图形程序需要显示服务器。源文档中的常见传递方式是：

```
DISPLAY=${DISPLAY:-unix:0}
docker exec -it -e DISPLAY=$DISPLAY <container-id> bash
```

`DISPLAY` 是图形显示目标，`-e` 把它传给容器。使用 X11 转发时还要确认宿主端的 `xhost` 和认证设置。源教程使用：

```
xhost +
```

该命令允许所有客户端连接显示服务器，范围过宽。受信多用户环境中应改为只允许当前本地用户，并在使用后恢复限制。SSH 会话的 `DISPLAY` 可能是 `localhost:10.0` 等转发地址，容器启动后新建的 SSH 会话不一定继承原有显示权限。

2.12. 12. 程序开机自启动：桌面项、启动应用和 systemd

2.12.1. 桌面自启动项

桌面环境可以读取 `~/.config/autostart/*.desktop` 自动启动程序：

```
mkdir -p ~/.config/autostart
vim ~/.config/autostart/myprogram.desktop
```

文件内容：

```
[Desktop Entry]
Encoding=UTF-8
Type=Application
Name=myprogram
Exec=/home/wheeltec/script.sh
Terminal=true
```

`Exec` 必须指向已经确认的脚本或程序，`Terminal=true` 表示需要打开终端。实际使用时把 `/home/wheeltec/script.sh` 替换为当前用户可执行的路径，并设置文件权限：

```
chmod a+r ~/.config/autostart/myprogram.desktop
```

用户级桌面项只适用于图形登录用户。服务账号、无桌面会话和系统级程序应使用 systemd，而不是依赖桌面自启动。

2.12.2. 启动脚本

一个最小脚本应明确 shebang、工作目录和环境：

```
#!/bin/bash
set -euo pipefail
cd /home/<user>/Documents/ascend310
exec /home/<user>/Documents/ascend310/scripts/run_service.sh
```

赋予所有者执行权限即可：

```
chmod u+x script.sh
bash -n script.sh
./script.sh
```

源教程中的桌面脚本会调用终端模拟器并加载应用程序环境。若脚本只做桌面显示，`Terminal=true` 可以保留终端；若脚本管理后台服务，使用 systemd 并在服务单元中显式设置环境变量。

2.12.3. Startup Applications

部分 Ubuntu 桌面提供 `Startup Applications` 图形工具。搜索 `startup` 打开该工具，点击 `Add`，填写名称和启动命令，保存后重启验证。它本质上仍是用户会话级自启动，不等价于系统服务。

2.12.4. systemd 服务

无桌面或需要自动重启的程序使用 systemd。创建单元文件：

```
sudo vim /etc/systemd/system/ascend310-demo.service
```

示例内容：

```
[Unit]
Description=Ascend 310B demo service
After=network-online.target
Wants=network-online.target

[Service]
Type=simple
User=<board-user>
WorkingDirectory=/home/<board-user>/Documents/ascend310
Environment=PYTHONNOUSERSITE=1
ExecStart=/home/<board-user>/Documents/ascend310/scripts/run_service.sh
Restart=on-failure
RestartSec=5

[Install]
WantedBy=multi-user.target
```

加载并启用：

```
sudo systemctl daemon-reload
sudo systemctl enable --now ascend310-demo.service
systemctl status ascend310-demo.service --no-pager
journalctl -u ascend310-demo.service -b --no-pager | tail -n 80
```

systemd 不会自动继承交互式 shell 中临时 `source` 的环境。CANN、Conda、`PYTHONPATH` 和 `LD_LIBRARY_PATH` 应由包装脚本或服务单元显式设置。修改单元后重新执行 `daemon-reload`，停止或重启前先确认服务 PID 和影响范围。

2.12.5. 旧式 rc.local

部分旧教程使用 `/etc/rc.local`：

```
sudo nano /etc/rc.local
```

在文件末尾添加脚本路径，例如：

```
/home/wheeltec/nfs.sh
```

然后重启：

```
reboot
```

新版本 Ubuntu 默认不一定启用 `rc.local`，而且它不提供服务依赖、重启策略和日志单位。除非目标系统已有明确要求，优先使用 systemd。

2.13. 13. 昇腾 310B 的 CANN、PyACL、NPU、ATC 与 OM

2.13.1. 在同一 shell 加载 Conda 和 CANN

```
source /usr/local/miniconda3/etc/profile.d/conda.sh
conda activate base
source /usr/local/Ascend/ascend-toolkit/set_env.sh
python --version
python -c 'import sys; print(sys.executable)'
command -v atc
python -c 'import acl; print("PyACL import: ok")'
```

`set_env.sh` 设置 ACL、运行时和工具链相关路径，`import acl` 验证当前解释器能否加载 PyACL。另一个终端中执行 `source` 不会影响当前服务进程。启动 ATC 或 NPU 服务时，Conda 和 CANN 必须在同一个 shell 或同一个包装脚本中加载。

2.13.2. NPU 设备状态

```
npu-smi info
```

`npu-smi info` 输出设备、芯片和健康状态。310B4 / 8T 与 310B1 / 20T 属于不同算力层级，结果不能混合排名。已知板端出现 `Health: Alarm` 时，应把它作为诊断背景保存，但不能仅凭告警判定 ATC、ACL、精度或性能失败；相反，应继续检查 `acl` 导入失败、设备不存在、ATC 非零退出、OM 缺失、推理不一致、段错误和资源泄漏等具体现象。

2.13.3. ATC 转换

ATC 把 ONNX 等前端模型转换为 OM。下面是最小示例：

```
atc \
  --model=models/resnet18_scene.onnx \
  --framework=5 \
  --output=models/resnet18_scene \
  --soc_version=Ascend310B4 \
  --input_format=NCHW \
  --input_shape=input:1,3,224,224
```

`--framework=5` 表示 ONNX，`--soc_version=Ascend310B4` 必须与目标板匹配，输入布局和 `--input_shape` 必须来自该模型的输入合同。不要从另一个模型复制形状、算子参数或 `--soc_version`。转换前先阅读项目脚本的 `--help`：

```
python scripts/convert_onnx_to_om.py --help
SOC_VERSION=Ascend310B4 bash scripts/atc_convert.sh
```

脚本可能写入日志、manifest 和中间目录。执行前确认输出路径，执行后保存完整 ATC 命令、CANN 版本、模型输入合同、退出码和日志。

2.13.4. OM 工件检查

转换完成后检查文件存在、字节数和摘要：

```
stat -c '%s %n' models/*.om
sha256sum models/example.om
```

若复现包提供了 `SHA256SUMS.txt`，只能在生成该清单的根目录执行：

```
bundle_root=/path/to/repro-bundle
cd "$bundle_root"
sha256sum -c SHA256SUMS.txt
```

摘要一致只证明文件没有发生变化。OM 是否能够由 PyACL 加载、输入输出是否与合同一致、数值是否可接受，需要分别执行 ACL 烟测和数值比较。不同 CANN 版本、不同 SoC、不同精度和不同模型 ID 的 OM 不能混用。

2.13.5. PyACL 与 OM 推理

PyACL 是 ACL 的 Python 接口。服务启动前执行最小导入检查：

```
python -c 'import acl; print("PyACL import: ok")'
```

推荐使用项目包装脚本，因为包装脚本可以在启动前检查 `import acl`：

```
bash scripts/run_palmprint_service.sh --host 127.0.0.1 --port 7860
```

若 `import acl` 失败，记录 Python 路径、CANN 路径和完整错误，修复环境后重新启动服务。不要添加 CPU 推理回退，也不要用随机输出或模拟结果让健康检查通过。服务健康接口可以证明 HTTP 进程存活，不能证明 NPU 推理链已加载。

2.13.6. 环境变量边界

```
export PYTHONNOUSERSITE=1
export PYTHONPATH="$PWD:${PYTHONPATH:-}"
export LD_LIBRARY_PATH="/usr/local/Ascend/ascend-toolkit/latest/lib64:${LD_LIBRARY_PATH:-}
    ↪ "
```

这些变量只针对当前 shell 和子进程。需要长期一致的环境应写入项目包装脚本或 systemd 单元，并经过部署评审。不要全局修改 `/etc/profile` 来让所有用户继承未经验证的 CANN 和 Python 路径。

2.14. 14. 板端设备、服务、日志、测试与证据边界

2.14.1. 系统状态报告

```
uname -a
cat /etc/os-release
hostname
date --iso-8601=seconds
free -h
swapon --show
df -h
uptime
ps -p 1 -o pid,comm,args
npu-smi info
```

这份状态报告用于记录测试环境，不是性能测试。把它和后续 ATC、ACL、精度和性能命令放在同一报告目录中，便于追溯。

2.14.2. 摄像头和 V4L2

```
ls -l /dev/video*
groups
```

```
v4l2-ctl --device=/dev/video0 --list-formats-ext
v4l2-ctl --device=/dev/video0 --all
```

`/dev/video0` 不一定是目标摄像头，设备编号可能随枚举顺序变化。先记录实际节点、格式、分辨率和帧率，再运行采集程序。`--source 0` 的含义由 V4L2 枚举顺序决定，不能仅凭一次运行推断为固定硬件。

无权限时检查组：

```
groups
sudo usermod -aG video "$USER"
```

重新登录后再验证。不要用 root 启动整个图形程序或 Python 服务来绕开设备权限。

2.14.3. USB 和音频设备

```
lsusb
lsusb -t
cat /proc/asound/cards
aplay -l
arecord -l
ls -l /dev/snd
```

音频设备编号和名称必须以当前板端输出为准。`aplay -l` 只列出播放设备，`arecord -l` 列出采集设备。播放测试会改变音量或产生声音，执行前确认扬声器、耳机和实验环境。

USB 采集设备通常需要独占。确认设备节点后查看占用进程：

```
lsusb
usb_node="/dev/bus/usb/001/002"
test -e "$usb_node"
fuser -v "$usb_node"
```

`fuser -v` 只用于定位占用者，确认进程属于哪个程序后再处理。不要按模糊名称批量结束进程。

2.14.4. 服务和端口

```
systemctl status ssh --no-pager
systemctl --failed
systemctl list-units --type=service --state=running
ss -ltnp
ss -lunp
```

`systemctl --failed` 显示失败单元，`ss -ltnp` 显示 TCP 监听及进程，`ss -lunp` 显示 UDP 监听。服务绑定到 `127.0.0.1` 时只接受本机连接，绑定到 `0.0.0.0` 则对所有网卡开放。只有可信实验网络中的明确需求才使用 `0.0.0.0`。

2.14.5. 进程和日志

```
pgrep -af 'uvicorn|python|docker|npu'
ps -ef | grep -E 'python|uvicorn|docker' | grep -v grep
journalctl -b -p err --no-pager | tail -n 100
journalctl -u ascend310-demo.service -b --no-pager | tail -n 80
sudo dmesg -T | tail -n 200
```

`journalctl -b` 查看本次启动，`-u` 查看指定 systemd 单元，`-p err` 过滤错误级别。`dmesg` 查看内核消息，涉及 USB、摄像头、网络和驱动问题时尤其有用。公开报告只保留与故障相关的行，并删除用户名、IP、token、真实图像路径等隐私信息。

2.14.6. 安全停止服务

只停止已经确认的 PID：

```
service_pid=12345
case "$service_pid" in
  ''|*[!0-9]*) printf 'invalid PID\n' >&2; exit 2 ;;
esac
test "$service_pid" -gt 1
ps -fp "$service_pid"
readlink -f "/proc/$service_pid/cwd"
kill -TERM "$service_pid"
for _ in $(seq 1 20); do
  kill -0 "$service_pid" 2>/dev/null || break
  sleep 1
done
```

先确认 PID 对应的命令、工作目录和用户，再发送 `TERM`。不要执行 `pkill python`、`killall python` 或按模糊名称批量结束进程，板端可能同时运行多个实验。

2.14.7. 本地测试

开发机可以执行语法检查和纯 Python 测试：

```
python -m py_compile app.py
python -m compileall -q package_or_directory
python -m pytest -q
python -m unittest discover -s tests -v
git diff --check
```

前端项目按锁文件执行：

```
npm ci
npm test
npm run build
npm run test:e2e
```

真实板端硬件测试必须有显式开关，且只在板端执行。例如项目定义了硬件测试变量时：

```
BOARD_HARDWARE_TESTS=1 python -m pytest -q tests/test_board_hardware.py
```

变量名应以项目实际脚本为准。普通单元测试不应隐式访问 NPU、摄像头、串口、音频设备或真实个人数据。

2.14.8. 证据门

证据门	能证明	不能证明
Python 语法检查	文件可解析	运行时依赖、板端设备、模型正确性
HTTP 200	路由和服务进程可达	NPU 推理成功、精度和性能
`npu-smi info`	设备可被管理工具读取	某个 OM、模型或应用程序已经验证
`import acl`	PyACL 可在当前解释器导入	ACL context、具体模型和算子链通过
ATC 退出码为 0	转换流程返回成功	OM 数值和任务精度正确
OM 摘要一致	文件字节未变化	模型可加载或输入输出合同正确
ACL 数值烟测	指定输入的数值关系在容差内	数据集精度、延迟和吞吐
任务精度测试	指定数据集和协议上的指标	其他数据集和硬件层级的结果
性能测试	指定参数下的延迟或吞吐	精度和端到端业务正确性
UI 烟测	页面或控件可以交互	后端真实硬件推理已经完成

每份报告至少记录硬件型号和算力层级、CANN 版本、模型 ID 和精度、输入合同、预热次数、循环次数、重复次数、百分位方法、输入来源、输出路径和原始日志。

2.15. 15. 故障排查与禁止命令模式

2.15.1. 常见故障

现象	先执行	判断重点
SSH 无法连接	`ping`、`ip addr`、`ip route`、`systemctl status ssh`	是否同网段、服务是否启动、22 端口是否监听
known_hosts 冲突	`ssh-keygen -R <host>`	只删除目标主机记录，并核对新指纹
VNC 无法连接	检查共享开关、板端 IP、VNC 服务和显示输出	VNC 与 SSH 的端口和权限边界不同
VNC 分辨率低	`xrandr`、`xrandr --fb 1024x768`	是否超过显示能力，是否影响画面质量
WiFi 切换后失联	检查热点、网卡、连接优先级和新 IP	一个网卡通常不能同时做热点和连接上游 WiFi
无法 ping 通板端	`ip addr`、`ip route`、`ping`	同网段、路由、热点和地址冲突
NFS 挂载失败	`ping`、`showmount`、`exportfs -v`、检查路径	导出范围、目录路径、客户端权限和失效挂载
NFS 服务端关机	先 `umount` 再重新 `mount`	失效挂载不能直接覆盖
Docker 容器不在运行	`docker ps -a`、`docker logs`	容器状态、退出码和重启策略
容器中无图形窗口	检查 `DISPLAY`、X11 认证、容器环境变量	图形显示与业务推理是两个问题
`import acl` 失败	`command -v python`、`source set_env.sh`、重新导入	解释器和 CANN 是否来自同一环境
ATC 找不到算子	`atc --version`、检查 framework、soc_version、输入形状和日志	保存失败命令和模型合同，不生成伪 OM
`npu-smi` 显示 Alarm	保存状态报告并检查具体错误	告警是诊断背景，不能单独作为测试结论
摄像头无图像	`/dev/video*`、`v4l2-ctl`、`groups`	设备节点、格式、权限和实际尺寸
USB 仪器被占用	`lsusb`、`fuser -v <device>`	确认占用者，不用宽泛 kill
音频无声音	`aplay -l`、`pactl list short sinks`	设备、默认 sink 和配置 profile
端口被占用	`ss -ltnp`、`ps -fp <pid>`	只停止确认属于目标服务的 PID
磁盘不足	`df -h`、`du -sh <dir>`	先归档报告，再清理隔离临时目录

2.15.2. 禁止或必须审查的命令模式

以下命令模式不应未经确认直接执行：

- `rm -rf` 指向家目录、仓库根目录、变量未验证的路径或正在运行的部署目录。
- `rsync --delete` 同步到未确认的远端目录，它可能删除模型、数据和报告。
- `pkill python`、`killall` 或模糊 `kill`，它们可能终止其他案例和系统服务。
- `sudo pip install`、`sudo python`，它们会绕过 Conda 并改变系统包所有权。
- `chmod -R 777` 和未核实所有者的递归 `chown`，它们会放宽整个目录树的权限。

- 在开发机运行 CANN、ATC、PyACL、OM、摄像头或 NPU 命令，并把失败归因于代码。
- 在一个终端加载 CANN 或 Conda，却在另一个未加载环境的终端启动服务。
- 启动 `0.0.0.0`、公网隧道、未鉴权网关或宽泛 X11 授权后再处理个人数据。
- 使用 `xhost +` 允许所有客户端访问显示服务器，而不是限制到当前用户。
- 直接使用 `--privileged` 给容器开放全部宿主能力，而不是逐项授权设备。
- 把 `npu-smi` 摘要、HTTP 200、软件 tone、CPU 回退或界面可达写成完整硬件验收。
- 未确认路径就复制真实照片、生物特征模板、数据库、模型、密钥或 token 到 Git、截图和公开报告。

安全的操作顺序是：确认机器和环境，解析绝对路径，预览输入与目标，执行最小范围命令，保存原始输出，再把经过审查的结论写入报告。这样才能让 Ubuntu 命令成为可复现的操作步骤，而不是无法追溯的部署记录。

附录 3. Python 编程基础

> **本章导读** 目标：用 Python 检查昇腾 310B 模型部署中常见的配置、清单和结果数据。数据：正文内联示例，不依赖外部数据集。标准库：`json`、`statistics`、`math`。输出：可直接在 Python REPL 中运行，也可保存为 `.py` 文件复现。

本附录按实际使用顺序讲解 Python。模型 ID、输入形状、预处理均值、精度模式和准入状态会反复出现在样例代码中，因此本章直接用这些对象编写示例。

`for`、`if`、`dict`、`set`、异常处理等基础语法会在前几节展开，第 4 节补充 `samples/` 中高频出现的文件、JSON、命令行和异步写法。所有示例都只使用标准库，不引入 `pandas`，也不需要板端运行环境。

3.1. 1. 数据、类型与容器

3.1.1. 变量与基本类型

变量是给值起的名字。Python 中每个值都有类型，类型决定它可以参与哪些运算。检查类型用 `type()`，查看结果用 `print()`。

```
model_id = "mobileclip_s0"
feature_dim = 512
input_scale = 0.00392156862745098
is_admitted = True

print(type(model_id))
print(type(feature_dim))
print(type(input_scale))
print(type(is_admitted))
```

输出：

```
<class 'str'>
<class 'int'>
<class 'float'>
<class 'bool'>
```

`str` 是文本，`int` 是整数，`float` 是小数，`bool` 是布尔值。模型 ID 适合存成 `str`，嵌入维度适合存成 `int`。

3.1.2. 数字运算与单位换算

模型文件大小经常需要在字节和 MiB 之间换算。`/` 是普通除法，`//` 是向下取整，`%` 是取余数，`round()` 控制显示精度。

```
bytes_size = 131904474
mib_size = bytes_size / 1024 / 1024

print(mib_size)
print(round(mib_size, 2))
print(bytes_size // 1024)
print(bytes_size % 1024)
```

输出：

```
125.79369640350342
125.79
128812
986
```

`round()` 只改变显示结果，不改变原始变量。

3.1.3. 列表与元组

模型输入形状、预处理均值和标准差通常是有序数值。`list` 保存可增删的有序数据，`tuple` 保存创建后不修改的固定组合。

```
input_shape = [1, 3, 224, 224]
image_mean = (0.485, 0.456, 0.406)

print(input_shape[0])
print(input_shape[-1])
print(len(input_shape))
print(image_mean[0])
```

输出：

```
1
224
4
0.485
```

列表索引从 0 开始，`-1` 表示最后一个元素。`len()` 返回元素个数。

3.1.4. 字典

模型清单由多个字段组成，每个字段用一个键和对应的值表示。`dict` 保存键值对，适合描述一个模型条目。

```
model = {
    "model_id": "mobileclip_s0",
    "input_shape": [1, 3, 256, 256],
    "precision_mode": "mixed_fp16",
    "status": "admitted",
}

print(model["model_id"])
print(model["input_shape"])
```

输出：

```
mobileclip_s0
[1, 3, 256, 256]
```

键通常使用字符串，值可以是字符串、数字、列表或另一个字典。

3.1.5. 集合

需要知道模型清单中出现过哪些精度模式时，用 `set` 去重。

```
precision_modes = {"mixed_fp16", "allow_fp32_to_fp16", "mixed_fp16"}
print(precision_modes)
```

输出：

```
{'allow_fp32_to_fp16', 'mixed_fp16'}
```

集合不保留插入顺序，也不能通过索引取值。顺序不重要的去重任务才适合使用集合。

3.2. 2. 控制流

3.2.1. 条件分支

`if` 处理一个条件，`elif` 处理“否则如果”，`else` 是兜底。判断从上往下执行，先命中的分支生效。

```
status = "admitted"
```

```

if status == "admitted":
    print("可进入 NPU 服务")
elif status == "candidate":
    print("仅用于离线评估")
else:
    print("状态未知")
```

输出：

```
可进入 NPU 服务
```

比较运算符 `==` 判断是否相等，结果是一个布尔值。

3.2.2. for 循环

`for` 循环依次取出列表中的元素，重复执行缩进块。累加前先设置 `total = 0`，这个初始值必须写在循环外。

```
feature_dims = [512, 1024, 2048]

total = 0
for dim in feature_dims:
    total = total + dim

print(total)
print(min(feature_dims))
print(max(feature_dims))
```

输出：

```
3584
512
2048
```

3.2.3. range、enumerate 与 zip

`range(n)` 生成从 0 到 `n - 1` 的整数序列。`enumerate()` 同时返回序号和元素，`zip()` 把多个序列按位置配对。

```
for index, dim in enumerate([512, 1024, 2048], start=1):
    print(index, dim)
```

输出：

```
1 512
```

```
2 1024
3 2048
```

```
model_ids = ["mobileclip_s0", "resnet50_feature"]
dims = [512, 2048]

for model_id, dim in zip(model_ids, dims):
    print(model_id, dim)
```

输出：

```
mobileclip_s0 512
resnet50_feature 2048
```

3.2.4. while、break 与 continue

while 在条件仍为真时重复执行。循环内必须修改条件变量，否则会无限循环。**continue** 跳过本次循环，**break** 立即结束整个循环。

```
attempt = 0

while attempt < 3:
    print(f"attempt {attempt + 1}")
    attempt += 1
```

输出：

```
attempt 1
attempt 2
attempt 3
```

```
dims = [512, 1024, 2048]

for dim in dims:
    if dim == 1024:
        continue
    print(dim)

for dim in dims:
    if dim > 1024:
        break
    print(dim)
```

输出：

```
512
```

```
2048
512
1024
```

3.2.5. 列表与字典推导式

推导式用一行代码生成新列表或字典，适合结构简单的过滤和转换。

```python
dims = [512, 1024, 2048]
large_dims = [dim for dim in dims if dim >= 1024]

print(large_dims)
```

输出：

```
[1024, 2048]
```

```python
sizes = {"mobileclip_s0": 131904474, "resnet50_feature": 102400000}
sizes_mib = {model: round(size / 1024 / 1024, 2) for model, size in sizes.items()}

print(sizes_mib)
```

输出：

```
{'mobileclip_s0': 125.79, 'resnet50_feature': 97.66}
```

3.3. 3. 函数与模型清单

3.3.1. 函数封装

函数把重复逻辑打包成可复用单元。下面函数接收模型列表，返回按精度模式统计的字典。

```python
def count_by_precision(models):
    result = {}
    for model in models:
        mode = model["precision_mode"]
        result[mode] = result.get(mode, 0) + 1
    return result

models = [
    {"precision_mode": "mixed_fp16"},
    {"precision_mode": "allow_fp32_to_fp16"},
```

```
    {"precision_mode": "allow_fp32_to_fp16"},
]

summary = count_by_precision(models)
for mode, count in sorted(summary.items()):
    print(f"{mode}: {count}")
```

输出：

```
allow_fp32_to_fp16: 2
mixed_fp16: 1
```

`dict.get(key, 0)` 在键不存在时返回默认值 0。`sorted()` 让输出顺序稳定。

3.3.2. 默认参数

默认参数让调用更简洁，同时允许按名字覆盖默认值。

```
def format_model(model_id, precision="mixed_fp16"):
    return f"{model_id}: {precision}"

print(format_model("mobileclip_s0"))
print(format_model("resnet50_feature", precision="allow_fp32_to_fp16"))
```

输出：

```
mobileclip_s0: mixed_fp16
resnet50_feature: allow_fp32_to_fp16
```

3.3.3. *args 与 **kwargs

`*values` 收集位置参数，`**options` 收集关键字参数。

```
def summarize(title, *values, **options):
    print(title, values, options)

summarize("models", 512, 1024, status="admitted")
```

输出：

```
models (512, 1024) {'status': 'admitted'}
```

3.3.4. lambda

`lambda` 适合写一个短小的临时函数。

```
models = [
    {"model_id": "mobileclip_s0", "embedding_dim": 512},
    {"model_id": "resnet50_feature", "embedding_dim": 2048},
]

models.sort(key=lambda model: model["embedding_dim"], reverse=True)
print([model["model_id"] for model in models])
```

输出：

```
['resnet50_feature', 'mobileclip_s0']
```

3.3.5. f-string

f-string 用 `f"..."` 开头，大括号内插入变量。

```
model_id = "mobileclip_s0"
embedding_dim = 512

print(f"{model_id}: {embedding_dim} dim")
```

输出：

```
mobileclip_s0: 512 dim
```

3.3.6. 类型注解

类型注解帮助阅读代码，也方便工具检查错误。它不会改变运行结果。

```
def admitted_models(models: list[dict]) -> list[dict]:
    return [model for model in models if model.get("status") == "admitted"]
```

3.3.7. dataclass

`dataclass` 自动生成初始化、比较和打印方法，适合保存结构清晰的记录。

```
from dataclasses import dataclass

@dataclass
```

```python
class ModelRecord:
    model_id: str
    embedding_dim: int
    status: str = "admitted"

record = ModelRecord("mobileclip_s0", 512)
print(record)
```

输出：

```
ModelRecord(model_id='mobileclip_s0', embedding_dim=512, status='admitted')
```

3.3.8. class 与 self

`__init__` 是实例创建时执行的方法，`self` 指向当前实例。

```python
class ModelChecker:
    def __init__(self, status="candidate"):
        self.status = status

    def is_admitted(self):
        return self.status == "admitted"

checker = ModelChecker("admitted")
print(checker.is_admitted())
```

输出：

```
True
```

3.4. 4. 文件、JSON、命令行与异步

3.4.1. 缺失键与类型转换

模型清单可能来自不同版本的脚本，字段并不总是一致。直接读取缺失字段会触发 `KeyError`，用 `dict.get()` 可以提供默认值。

```python
model = {
    "model_id": "mobileclip_s0",
    "precision_mode": "mixed_fp16",
}

print(model.get("status", "unknown"))
```

输出：

```
unknown
```

把字符串转成数字时，输入内容决定转换是否成功。`int("fp16")` 会触发 `ValueError`，可以用 `try/except` 捕获。

```
value = "fp16"

try:
    numeric_value = int(value)
except ValueError:
    numeric_value = None

print(numeric_value)
```

输出：

```
None
```

`None` 表示没有有效数值。统计前应先检查它，不要把 `None` 当成 0。

3.4.2. pathlib 与 with open

`Path` 提供跨平台的路径操作，`with open` 在读取或写入结束后自动关闭文件。

```
from pathlib import Path

path = Path("tmp/appendix3/notes.txt")
path.parent.mkdir(parents=True, exist_ok=True)
path.write_text("mixed_fp16\n", encoding="utf-8")

with path.open(encoding="utf-8") as f:
    print(f.read().strip())
```

输出：

```
mixed_fp16
```

3.4.3. json.loads 与 json.dumps

模型配置常以 JSON 保存。`json.loads()` 把 JSON 文本转回 Python 对象，`json.dumps()` 把 Python 对象转成 JSON 文本。

```
import json
```

```

text = '{"model_id": "mobileclip_s0", "status": "admitted"}'
data = json.loads(text)

print(data["model_id"])
print(data.get("status", "unknown"))
```

输出：

```
mobileclip_s0
admitted
```

```
config = {"model_id": "mobileclip_s0", "precision_mode": "mixed_fp16"}
print(json.dumps(config, ensure_ascii=False, indent=2))
```

输出：

```
{
  "model_id": "mobileclip_s0",
  "precision_mode": "mixed_fp16"
}
```

解析失败时，`json.loads()` 会抛出 `json.JSONDecodeError`。处理外部配置或日志时，先确认它确实是合法 JSON，再读取字段，不要把解析失败和字段缺失混为一谈。

3.4.4. try / except / else / finally

`else` 在没有异常时执行，`finally` 无论是否异常都会执行。

```
try:
    dim = int("512")
except ValueError:
    dim = None
else:
    print("converted")
finally:
    print("done")
```

输出：

```
converted
done
```

3.4.5. argparse

`argparse` 用来解析命令行参数，样例脚本中常用来接收模型 ID、精度和路径。

```python
import argparse

parser = argparse.ArgumentParser()
parser.add_argument("--model-id", default="mobileclip_s0")
args = parser.parse_args(["--model-id", "resnet50_feature"])

print(args.model_id)
```

输出：

```
resnet50_feature
```

3.4.6. if name == “main”

被直接运行时会执行 `main()`，被其他模块导入时不会自动执行。

```python
def main():
    print("check complete")

if __name__ == "__main__":
    main()
```

输出：

```
check complete
```

3.4.7. logging

`logging` 比 `print()` 更适合记录运行过程，样例服务中常用来区分普通日志和错误日志。

```python
import logging

logging.basicConfig(level=logging.INFO)
logger = logging.getLogger("appendix3")
logger.info("model check complete")
```

输出：

```
INFO:appendix3:model check complete
```

3.4.8. subprocess

`subprocess.run()` 启动外部命令，`capture_output=True` 捕获标准输出。

```
import subprocess

result = subprocess.run(["python", "--version"], capture_output=True, text=True)
print(result.stdout.strip())
```

输出内容取决于本机 Python 版本。

3.4.9. async / await

`async def` 定义异步函数，`await` 等待异步结果。样例中的 WebRTC 和聊天服务大量使用这种写法。

```
import asyncio

async def check():
    return "admitted"

async def main():
    print(await check())

asyncio.run(main())
```

输出：

```
admitted
```

3.5. 5. 调试与验证

3.5.1. 最小调试手段

出错时先确认对象的真实内容，不要只看报错文字。`repr()` 可以显示字符串中的空格和引号。

```
status = " admitted"

print(status)
print(repr(status))
```

输出：

```
admitted
' admitted'
```

如果逻辑复杂，可以在可疑位置使用 `breakpoint()` 进入调试器，或把中间变量打印出来。脚本保存后，先在仓库根目录执行：

```
python -m py_compile 脚本路径.py
```

`py_compile` 只检查语法，不运行板端逻辑。昇腾 310B 的 CANN、ATC 和 ACL 检查仍应在真实开发板上完成。

3.5.2. 验证清单

- ☐ 能说出 `int`、`float`、`str`、`bool` 的区别。
- ☐ 能用列表保存输入形状或特征维度，并用循环完成求和。
- ☐ 能用字典表示一个模型条目，按字段名读取值。
- ☐ 能写函数统计模型清单中的精度模式。
- ☐ 能区分 `KeyError`、`ValueError` 和 `json.JSONDecodeError`。
- ☐ 能阅读 `samples/` 中常见的列表推导、`enumerate`、`argparse` 和 `pathlib` 写法。
- ☐ 示例只使用标准库，没有 `pandas` 或板端运行时依赖。

3.5.3. 常见错误与坑

现象	可能原因	处理方式
读取缺失字段报 `KeyError`	模型清单字段不一致	先审计字段，再用 `get()`
`int("fp16")` 报错	字符串不是数字	用 `try/except` 捕获并记录
JSON 解析失败	文本不是合法 JSON	检查完整输入，捕获 `JSONDecodeError`
统计结果比预期少	把字符串当成数字	先看 `type()`，必要时转换
输出顺序每次不同	遍历集合或字典	需要稳定顺序时用 `sorted()`

3.6. 6. 作业

1. 新建一个脚本，统计 `feature_dims = [512, 1024, 2048, 512]` 的元素个数、最小值、最大值和去重后的维度集合。
2. 写一个函数 `count_admitted(models)`，接收模型字典列表，返回 `status == "admitted"` 的模型数量。

3. 写一个 `safe_get(data, key)` 函数：键存在时返回值，不存在或类型错误时返回 `None`。
4. 用列表推导式筛出 `[512, 1024, 2048, 512]` 中大于等于 1024 的元素。
5. 用 `argparse` 写一个脚本，接收 `--model-id`，并输出该参数值。
6. 把上面的脚本都交给 DSH 或人工审查一次，逐行解释每个分支和异常处理。

AI 辅助审查时仍遵循先理解计划、检查命令、小步执行、验证结果。DSH 生成代码后，必须自己运行并解释输出，不能只看 “已完成” 的结论。

附录 4. Vibe Coding 基础

> **本章导读** 目标：掌握 Vibe Coding 的基本流程，用 DSH 生成、运行并审查 Python 代码。环境：昇腾 310B 自带 Anaconda，本附录不安装 Python 环境。数据：`samples/case7/models/registry.json` 输出：`tmp/appendix4/<姓名>/vibe_models.py`

Vibe Coding 是用自然语言描述目标，由 AI Agent 写代码并运行，人负责审查和验证。DSH 是这套流程中的执行环境，DeepSeek 负责理解任务和生成方案。本附录只讲基础流程，不要求提前掌握完整 Python 语法。

4.1. 1. Vibe Coding 是什么

Vibe Coding 的基本流程是：人负责目标、边界和结果，Agent 负责代码生成、运行和修改。它的完整循环是：

```
定义问题 → 要求最小版本 → 运行 → 反馈真实结果 → 追问 → 验证
```

每一步都有明确负责人：

工作	谁负责	检查重点
描述目标	你	要读什么、做什么、输出到哪里
制定计划	Agent	计划是否只动指定文件
生成代码	Agent	代码是否使用允许的工具和依赖
运行代码	Agent	是否先展示命令，再执行
解释代码	Agent	每一行是否都能讲清楚
审查结果	你	输出是否来自真实文件，结论能否复现
验证安全	你	是否修改了不该改的文件，是否泄露 Key

Vibe Coding 适合探索代码、生成初稿、解释报错和快速迭代。它不适合在完全看不懂代码、不了解文件边界或没有验证方法时直接信任结果。

参考教程：

- datawhalechina/easy-vibe：从零开始学 AI 编程，用 “先看示例 + 自己动手做” 组织课程。

- datawhalechina/vibe-vibe：系统化的 Vibe Coding 教程，强调从 “写代码” 转向 “对话式创作”。

4.2. 2. Agent 与 DSH 基础

AI Agent 不是普通的聊天机器人。聊天机器人只负责生成回答，Agent 还可以读取文件、运行命令、修改代码和搜索网页。DSH（DeepSeek Harness）是一个能执行这些操作的 Agent 工作环境。

一个 Agent 通常由四部分组成：

组成部分	作用	DSH 中的例子
大模型	理解任务，生成计划和代码	DeepSeek 大模型
工具	执行文件、命令和网络操作	读写文件、运行命令、搜索网页
记忆	记录目标和上下文	当前会话、长期目标
工作区	Agent 的活动范围	当前仓库根目录

常见 Agent 可以按权限从低到高分类：

Agent	类型	是否直接操作本地文件	适合用途
ChatGPT	网页/App 助手	通常不会	问答、写作、代码解释
AI IDE	代码编辑器	会，但通常在打开的项目内	代码补全、修改当前项目
DSH	本地 Agent	会，可以运行命令	读取文件、生成脚本、执行验证

DSH 的工作方式是一个循环：理解目标，制定计划，调用工具，看到结果，再调整计划。下面是一个简化的流程：

```
flowchart LR
    A[你提出目标] --> B[Agent 制定计划]
    B --> C[Agent 调用工具]
    C --> D[工具返回真实结果]
    D --> E{结果符合目标吗}
    E -- "不符合" --> B
    E -- "符合" --> F[Agent 汇报结果]
    F --> G[你审查并验证]
```

本教程使用 DeepSeek + DSH，因为 DeepSeek 的中文理解、代码和推理表现适合课程任务，DSH 提供了计划模式、工具调用和安全确认机制。模型 API 成本会随厂商调整，实际以官网为准。

参考资料：

- Hannibal046/Awesome-LLM：持续更新的大语言模型论文、工具和教程清单。
- WangRongsheng/awesome-LLM-resources：中文友好的 LLM、Agent、多模态与训练推理资料汇总。
- luban-agi/Awesome-AIGC-Tutorials：面向入门学习的 LLM 和 AIGC 教程合集。
- e2b-dev/awesome-ai-agents：AI Agent 框架、论文、工具和项目清单。
- luo-junyu/Awesome-Agent-Papers：以论文为主的 LLM Agent 综述和进展。
- Shubhamsaboo/awesome-llm-apps：Agent、RAG 和应用案例合集。

4.3. 3. DSH 配置与安全检查

4.3.1. 确认环境

昇腾 310B 自带 Anaconda，本附录不安装 Python 环境。先确认基础命令可用：

```
python --version
conda --version
dsh web
```

能看到 `python` 和 `conda` 版本即可继续。`dsh web` 启动后，浏览器打开：

```
http://127.0.0.1:3080
```

DSH 的官方仓库在：
https://github.com/deepseek-ai/deepseek-harness

4.3.2. 配置模型与工作区

DSH 需要两样配置才能开始工作：模型 API Key 和工作区。先到 DeepSeek 开放平台申请 API Key：

https://platform.deepseek.com/

申请后打开 DSH 的 `Settings → Models`，粘贴 API Key 并保存。然后在主界面点击 `Choose workspace`，选择本仓库根目录。昇腾 310B 上常见路径为：

```
~/Documents/Ascend310
```

Windows 开发机上类似：

```
D:\Github\Ascend310
```

API Key 只保存在本地设置里，不要写进教程、代码、聊天内容或 GitHub。

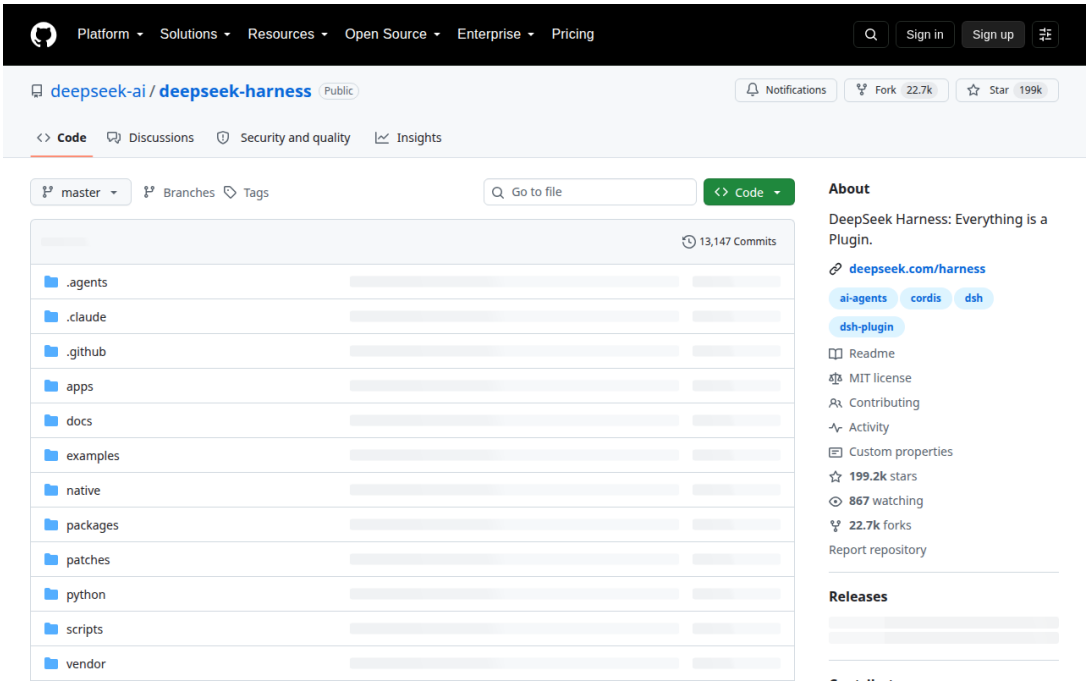


图 4.1. DSH 官方仓库

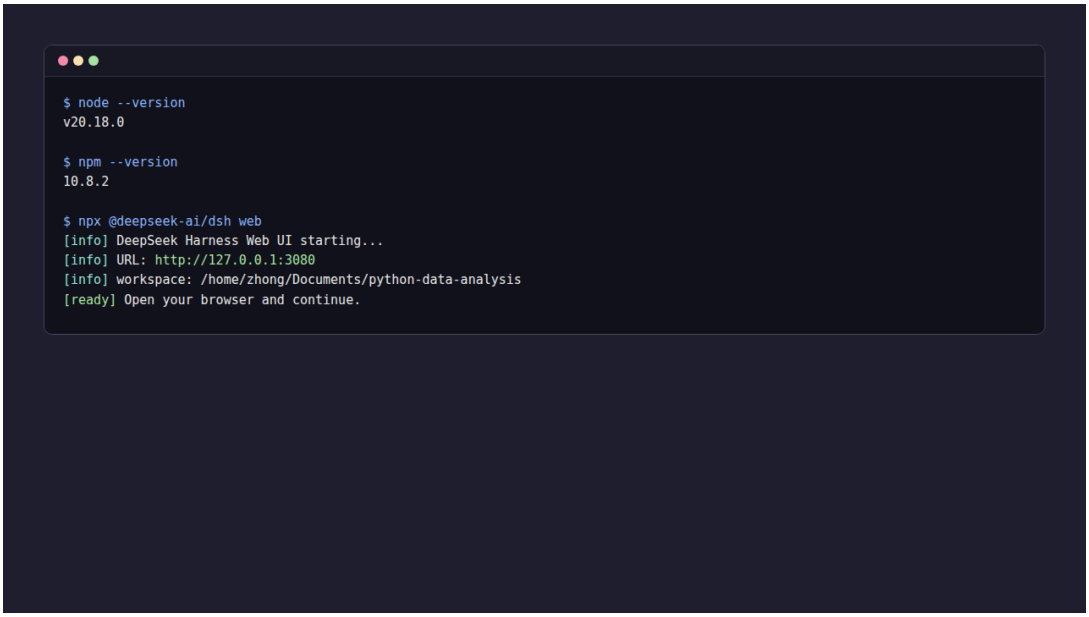


图 4.2. DSH 启动与验证示例

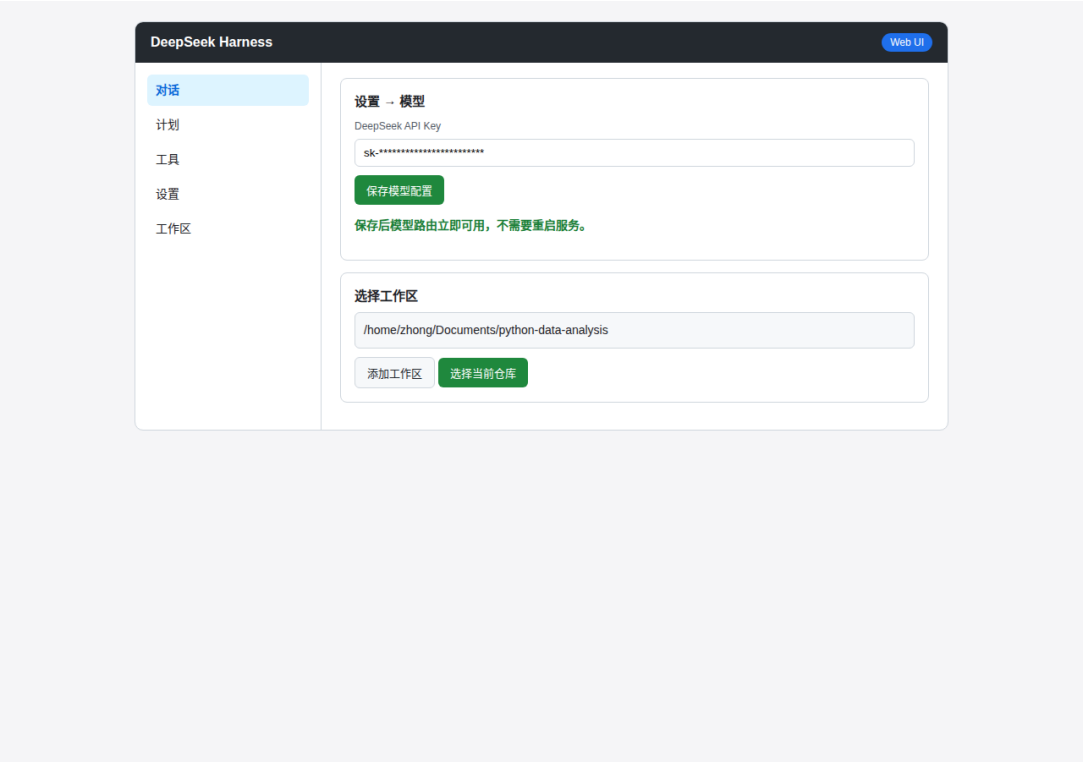


图 4.3. DSH 模型与工作区配置示意图

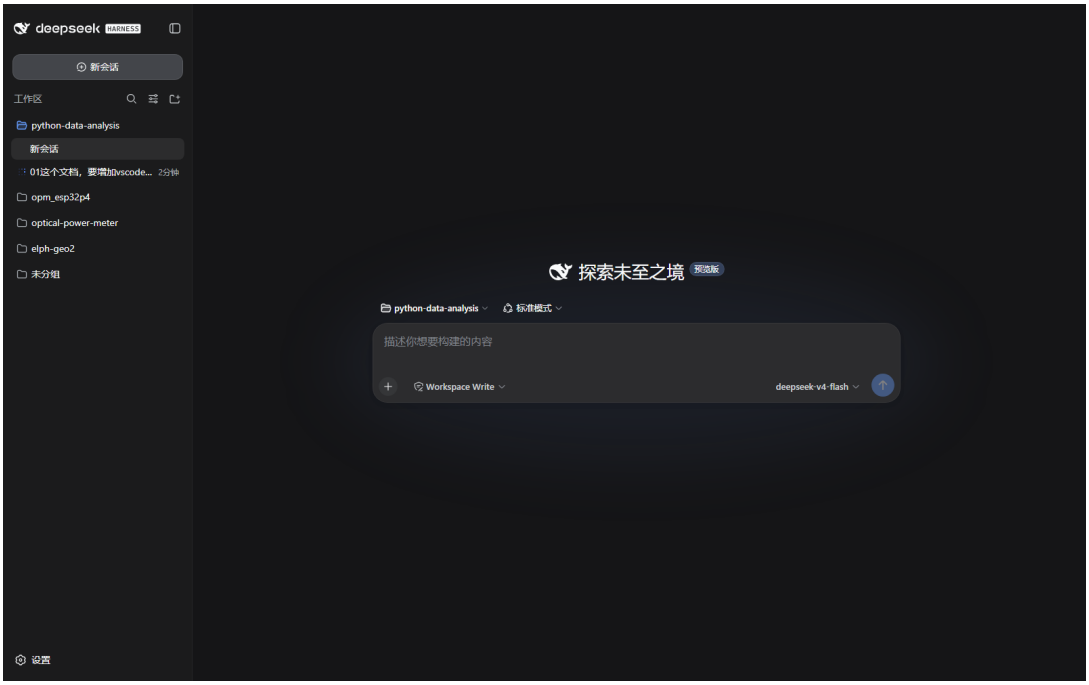


图 4.4. DSH Web 界面

4.3.3. DSH 界面

区域	作用	常见操作
左侧会话栏	新建会话、切换历史会话	点 “新会话” 开始一个干净任务
中部消息区	你与 Agent 的对话记录	滚动查看每一步
底部输入框	输入自然语言任务	回车发送
工具行	显示 Agent 正在读取、编辑、运行什么	点击查看工具执行详情
右上角状态	当前工作区、模型、安全模式	确认工作区是否正确
设置	API Key、模型、工作区配置	只配置一次，之后不要随意修改

每次开始任务前，先问一句：

```
请告诉我当前工作区路径。
```

如果 DSH 回答的路径不是本仓库，先改回本仓库再继续。

4.3.4. 安全规则

Agent 能做的事情和真人操作电脑一样有后果。看不懂的命令、不确定的文件修改，都可能破坏课程项目或系统环境。使用 DSH 时遵守五条规则：

1. 先看计划，再批准执行。
2. 只修改工作区里的指定文件。
3. 不执行删除、格式化、上传、系统级安装等危险命令。
4. 不把 API Key 写进代码、文档或 GitHub。
5. 结论必须能从文件内容或命令输出中验证。

常见风险与应对：

风险	表现	应对
选错工作区	DSH 找不到文件，或修改了别的目录	先问当前工作区路径，再重新选择
危险命令	Agent 要删除文件、安装依赖或改系统配置	停下来，要求解释，确认后再继续
生成看不懂的代码	代码能运行，但自己讲不清每一行	让 DSH 逐行解释，能复述后再验收
Key 泄露	API Key 出现在代码或文档中	删除 Key，重新生成，绝不提交
长任务失控	Agent 连续修改多个文件	拆成小任务，每步检查一次结果

4.4. 4. 第一次 Vibe Coding 任务

本附录的第一个任务不使用外部数据，改为读取本仓库中的模型清单：

```
samples/case7/models/registry.json
```

文件里包含 `models` 列表，每个模型有 `model_id`、`status`、`embedding_dim` 和 `precision_mode` 等字段。

4.4.1. 提示词

在 DSH 中输入：

```
请先告诉我当前工作区路径，确认后开始。
请读取 samples/case7/models/registry.json，不要修改原文件。
用标准库 json 输出：
1. models 数量；
2. 每个模型的 model_id、status、embedding_dim、precision_mode。
代码保存到 tmp/appendix4/<姓名>/vibe_models.py。
只允许读取，不安装任何依赖。
```

提示词包含了目标、数据路径、输出位置、约束和验收标准。DSH 应该先制定计划，再读取文件，再生成脚本。

4.4.2. 参考脚本

DSH 生成的代码可能不完全相同，下面是一个可运行的参考写法：

```python
import json
from pathlib import Path

path = Path("samples/case7/models/registry.json")
data = json.loads(path.read_text(encoding="utf-8"))
models = data["models"]

print("models:", len(models))
for model in models:
    print(model["model_id"], model["status"], model["embedding_dim"], model["
    ↪ precision_mode"])
```

4.4.3. 预期输出

当前仓库实测输出：

```
models: 3
mobileclip_s0__npu__mixed_fp16 admitted 512 allow_fp32_to_fp16
chinese_clip_rn50__npu__mixed_fp16 admitted 1024 allow_fp32_to_fp16
resnet50_feature__npu__mixed_fp16 admitted 2048 allow_fp32_to_fp16
```

如果 DSH 的输出和上面不一致，先检查工作区路径、文件内容和代码，不要直接接受结果。

4.4.4. 验收步骤

DSH 完成任务后，按以下顺序验收：

1. 检查计划：是否只读取指定 JSON，不修改原文件。
2. 检查命令：是否只运行 Python，不执行安装或删除命令。
3. 检查代码：是否使用标准库 `json` 和 `pathlib`，没有 pandas。
4. 自己运行一次脚本，确认输出与文件内容一致。
5. 用一句话说明脚本读取了哪个字段、计算了什么、输出了什么。

4.5. 5. 提示词与审查方法

4.5.1. 提示词结构

要素	要求	示例
目标	说明要做什么	读取模型清单并输出摘要
数据路径	给出明确路径	`samples/case7/models/registry.json`
输出位置	说明代码保存到哪里	`tmp/appendix4/<姓名>/vibe_models.py`
约束	说明允许和禁止的操作	只读、只用标准库、不装依赖
验收标准	说明怎样算完成	输出模型数量、ID、状态、维度

好的提示词让 Agent 少猜测。坏的提示词只有一句话，例如 “帮我分析一下”，这时 Agent 可能选错文件、改变工作区或生成无法验证的结论。

4.5.2. 审查清单

项目	检查内容
计划	Agent 是否先说明将读取什么、运行什么、写入什么
命令	每条命令是否看得懂，是否修改原文件
代码	是否能用标准库完成，是否每一行都能解释
输出	是否来自真实文件，能否复现
安全	是否没有删除、上传、安装系统级依赖，是否没有泄露 Key
记录	是否保留了 DSH 修改代码的过程

4.6. 6. 参考资料

- datawhalechina/easy-vibe：从零开始学 AI 编程，用 “先看示例 + 自己动手做” 组织课程。
- datawhalechina/vibe-vibe：系统化的 Vibe Coding 教程，强调从 “写代码” 转向 “对话式创作”。
- Hannibal046/Awesome-LLM：持续更新的大语言模型论文、工具和教程清单。
- WangRongsheng/awesome-LLM-resources：中文友好的 LLM、Agent、多模态与训练推理资料汇总。
- luban-agi/Awesome-AIGC-Tutorials：面向入门学习的 LLM 和 AIGC 教程合集。
- e2b-dev/awesome-ai-agents：AI Agent 框架、论文、工具和项目清单。
- luo-junyu/Awesome-Agent-Papers：以论文为主的 LLM Agent 综述和进展。
- Shubhamsaboo/awesome-llm-apps：Agent、RAG 和应用案例合集。
- DSH 官方仓库

- DeepSeek 开放平台

4.7. 7. 验证清单

- ☐ 能说出 Vibe Coding 的完整循环。
- ☐ 能区分 Agent、大模型、工具和工作区。
- ☐ `python --version` 和 `conda --version` 已确认可用。
- ☐ DSH 已启动，浏览器可以打开 `http://127.0.0.1:3080`。
- ☐ 已配置 API Key 和本仓库工作区。
- ☐ 能背出 5 条安全规则。
- ☐ 第一次任务只读取 `samples/case7/models/registry.json`，没有修改原文件。
- ☐ 能解释生成脚本中的 `json`、`Path` 和 `for` 循环。
- ☐ 脚本输出与文件内容一致。

4.8. 8. 常见错误与坑

现象	可能原因	处理方式
DSH 找不到文件	工作区不在本仓库	先问当前工作区路径，再重新选择
DSH 修改了原文件	文件边界不清晰	明确只读路径，检查计划后再批准
DSH 使用了 pandas	提示词没有限制依赖	明确只用标准库 `json` 和 `pathlib`
代码能运行但讲不清	直接接受了生成结果	让 DSH 逐行解释，自己复述后再保存
API Key 出现在代码中	配置时把 Key 当数据传给了 Agent	删除 Key 并重新生成，绝不提交
输出与文件不一致	没有自己运行脚本	重新运行，检查路径、字段和输出
DSH 页面打不开	启动命令还在运行，或端口被占用	等显示 `URL: http://127.0.0.1:3080` 后再刷新

4.9. 9. 作业

1. 用自己的话写 300 字以内的 “Vibe Coding 安全说明”，至少包含：Vibe Coding 是什么、5 条安全规则、3 个绝对不执行的操作、为什么每条结论都要自己验证。把说明写入 `tmp/appendix4/<姓名>/safety.md`。
2. 写一份 150 字以内的 “提示词设计说明”，解释为什么提示词要包含目标、路径、输出位置和约束。把说明整理成 `tmp/appendix4/<姓名>/prompt-notes.md`。
3. 让 DSH 读取 `samples/case7/models/registry.json`，输出 `models` 数量、每个模型的 `model_id`、`status`、`embedding_dim` 和 `precision_mode`。把脚本保存为 `tmp/appendix4/<姓名>/vibe_models.py`。
4. 让 DSH 增加一个统计维度，例如统计 `status == "admitted"` 的模型数量，或者按 `embedding_dim` 排序。
5. 把 DSH 生成的代码逐行审查一遍，说明哪几行最可能出错，为什么。

4.10. 10. 评分要点

项目	要求
认知	能解释 Vibe Coding 和 Agent 的基本概念
安全	能背出红线，看不懂的命令会停下来解释
提示词	提示词包含目标、路径、输出位置、约束和验收标准
代码审查	能逐行解释生成脚本，能指出风险和修改点
结果验证	输出与 `samples/case7/models/registry.json` 一致
AI 协作	保留 DSH 修改代码的记录，并说明每处修改为什么安全

附录 5. ROS2 基础教程

本附录以 ROS2 Humble 为主要示例版本，介绍机器人软件框架的定位、DDS 通信底层、工作空间与功能包、四种通信模型、命令行工具、可视化与数据记录、Python launch、TF2 与 URDF、传感器与底盘接口、SSH/VNC 远程操作、多机通信、程序自启动，以及一条明确标记为未验证的 ROS2 与昇腾 310B 推理服务集成路径。

主要内容取自 WHEELTEC R550 ROS2 系列教程中的《ROS2 入门教程》《ROS2 与 STM32 运动底盘通信》《ROS2 环境配置更换 URDF 模型教程》《ROS2 小车上手操作》《ROS2 查看小车底盘数据教程》《ROS2 查看小车雷达数据教程》，并按本文档所属教材的语境重新组织。概念性内容可对照教程中列出的 ROS2 官方文档入口。

5.1. 教程定位与证据边界

在开始使用命令之前，先固定本附录的证据边界。以下四条是全文的前提，后续任何章节都不与之冲突。

- `samples/case1` 至 `samples/case9` 当前不包含经过验证的 ROS2 运行时链路。仓库中检索不到 `rclpy`、`rclcpp`、`colcon`、`ros2 run` 等运行时实现与验证记录，因此本书已有案例与 ROS2 之间不存在可直接引用的集成结果。
- 本附录没有在昇腾 310B 开发板上验证过 ROS2。文中命令来自厂商教程与 ROS2 官方文档语境，用于建立基础概念与操作习惯，不等价于板端实测结论。
- 第 12 节的 ROS2 与昇腾 310B 推理服务集成路径明确标记为 proposed 与 untested。只有完成板端实测、留存日志与输入协议之后，才能把该路径升级为已验证结论。
- 本附录不给出延迟、吞吐、精度等性能数字。这类数字必须绑定模型、精度、硬件算力档位、数据集、预热、统计方法与原始报告路径，缺少任何一项都不能作为教材结论。

章节中的 `documented`、`inferred`、`observed-pass`、`observed-fail`、`untested` 标记沿用本书统一的证据分级方式。厂商教程给出的接口与命令属于 `documented`；跨平台迁移判断属于 `inferred`；本文中的集成路径属于 `untested`。

5.1.1. 命令来源与参考资料

来源	用途	证据等级
ROS2 入门教程.pdf	体系框架、通信模型、命令行、rqt 与 rviz2、bag、launch、TF2、URDF	documented
ROS2-WHEELTEC 机器人的 ROS 和 STM32 通信过程	底盘串口协议、设备别名、话题与服务接口	documented
ROS2 环境配置更换 urdf 模型教程	URDF 替换流程与常见位置偏差	documented
ROS2 小车上手操作.pdf	远程登录、编译选项、硬件检查顺序	documented
ROS2 查看小车底盘数据教程.pdf	/odom、/imu/data_raw 与 IMU 可视化	documented
ROS2 查看小车雷达数据教程.pdf	/scan、rviz2 LaserScan 与固定坐标系	documented
ROS2 官方文档 https://docs.ros.org/en/humble/index.html	DDS、发现机制、环境变量与工具行为的背景说明	documented

教程资料中出现的车型、雷达型号、相机型号与设备别名属于 WHEELTEC 默认配置。更换硬件或修改配置后，话题名、串口别名与坐标系可能变化，必须以实际 `ros2 topic list` 与 `ros2 node info` 的输出为准。

5.2. 1. 定位、DDS、去中心化发现与教材边界

5.2.1. ROS2 在机器人软件中的位置

ROS2 是一个开源的机器人软件框架，用于把机械结构、嵌入式控制、传感器驱动与上层算法组织成可复用的软件模块。教程把其生态划分为四个部分：

- 通信（Plumbing）：在节点之间传递消息的中间件，是整个框架的核心实现。
- 工具（Tools）：launch、调试、可视化、绘图、数据记录与回放等开发工具。
- 功能（Capabilities）：传感器驱动、运动规划、导航等可复用能力集合。
- 社区（Community）：贡献功能包、文档与经验的组织与个人。

教程归纳的 ROS2 特性包括开源免费、面向多机器人系统、跨平台（Linux、Windows、macOS、RTOS，乃至无操作系统的微控制器）、面向实时控制需求、适应不同网络环境、可用于产品化部署，以及覆盖设计到部署的项目管理机制。这些是框架的设计目标，是否在具体平台上成立需要单独验证。

5.2.2. DDS 与去中心化发现

ROS2 没有自行实现通信底层，而是直接采用 DDS 作为消息传递机制。教程给出的两点结论是：节点之间不再通过中心管理节点关联，节点可以直接相互发现并建立对等通信；通信的实时性、可靠性与连续性由 DDS 的传输质量策略承担。

从使用角度看，需要记住三个可观察事实：

- 发现是分布式的。两个节点只要在同一 DDS 域、网络可达且话题名与消息类型一致，就可以自动发现对方并开始通信，不需要启动额外的调度程序。
- 域由环境变量分组。`ROS_DOMAIN_ID` 相同的节点才会参与同一发现范围，多组实验可以用不同域号相互隔离。
- 发现依赖网络条件。默认使用组播，跨网段、NAT 或禁用组播的网络可能让同一域内的节点互相看不见。

这三个事实解释了第 10 节的多机配置，也解释了为什么排障时第一步总是确认域号、网络与接口类型，而不是先怀疑话题代码。

5.2.3. 与昇腾 310B 教材的边界

本教材的主线是在昇腾 310B 上完成模型部署与应用构建。ROS2 在本教材中的定位是机器人侧的软件组织方式，而不是已经接入的运行时依赖：

- 现有案例的推理入口、HTTP 接口与前端工作台都不经过 ROS2，也不需要 ROS2 才能运行。
- 若要把案例中的推理能力接入机器人系统，需要单独设计适配节点，并接受第 12 节列出的全部未验证前提。
- 教材正文中的 CANN、ATC、OM、ACL 证据边界继续有效，ROS2 的引入不会放宽这些边界。

5.3. 2. 工作空间、功能包与构建系统

5.3.1. 工作空间目录结构

教程给出的 ROS2 工作空间结构如下：

```
WorkSpace --- 自定义的工作空间
    |--- build：存储中间文件的目录，该目录下会为每一个功能包创建一个单独子目录
    |--- install：安装目录，该目录下会为每一个功能包创建一个单独子目录
    |--- log：日志目录，用于存储日志文件
    |--- src：用于存储功能包源码的目录
        |-- C++ 功能包
```

```
            |-- package.xml：包信息，比如包名、版本、作者、依赖项
            |-- CMakeLists.txt：配置编译规则，比如源文件、依赖项、目标文件
            |-- src：C++ 源文件目录
            |-- include：头文件目录
            |-- msg / srv / action：消息、服务、动作接口目录
        |-- Python 功能包
            |-- package.xml：包信息，比如包名、版本、作者、依赖项
            |-- setup.py：与 C++ 功能包的 CMakeLists.txt 类似
            |-- setup.cfg：功能包基本配置文件
            |-- resource：资源目录
            |-- test：存储测试相关文件
            |-- 功能包同名目录：Python 源文件目录
```

`src` 是唯一需要手工维护的目录。`build`、`install`、`log` 由构建与运行过程生成，可以删除后重新编译，也可以在版本控制中忽略。手工修改 `build` 或 `install` 中的内容不会带来稳定结果，重新编译就会覆盖。

5.3.2. 创建工作空间并编译

```
mkdir -p ~/ros2_ws/src
cd ~/ros2_ws
colcon build
```

第一条命令创建源码目录，第二条命令进入工作空间根目录，第三条命令在根目录执行编译。`colcon` 按当前目录作为工作空间根，因此 `colcon build` 必须在包含 `src` 的目录中执行，在 `src` 内执行不会得到预期结果。

```
source /opt/ros/humble/setup.bash
echo "source /opt/ros/humble/setup.bash" >> ~/.bashrc
```

第一行为当前终端加载 ROS2 环境，之后 `ros2`、`rviz2`、`rqt` 等命令才能被找到；第二行把同样的加载动作写入 `~/.bashrc`，使新终端自动生效。官方文档也建议在需要时单独 `source` 工作空间的 `install/setup.bash`，让工作空间中的功能包覆盖系统同名包。

```
colcon build --packages-select <package1> <package2>
```

`--packages-select` 只构建指定的一个或多个功能包，不处理其他包的依赖关系，适合功能包改动后的快速迭代。

```
colcon build --symlink-install
colcon build --symlink-install --packages-select <package_name>
```

厂商教程推荐使用 `--symlink-install`。它通过符号链接安装资源文件，编译后 `src` 中的 `yaml` 配置与 `launch.py` 文件修改可以直接生效，不需要每次都重新编译。只使用普通 `colcon build` 时，任意文件改动后都需要重新编译。

5.3.3. 创建功能包

```
ros2 pkg create --build-type ament_python <pkg_name> rclpy std_msgs sensor_msgs
ros2 pkg create --build-type ament_cmake cpp_pubsub
ros2 pkg create --build-type ament_cmake cpp_pubsub --dependencies rclcpp std_msgs
```

第一行创建 Python 功能包并声明 `rclpy`、`std_msgs`、`sensor_msgs` 依赖；第二行创建 C++ 功能包；第三行在创建时直接声明 `rclcpp` 与 `std_msgs` 依赖，避免之后再手工补依赖。

5.3.4. `package.xml`：包元信息与依赖声明

`package.xml` 描述包名、版本、描述、维护者以及构建与运行依赖。下列内容取自教程的 Python 功能包示例：

```
<?xml version="1.0"?>
<package format="3">
  <name>py_pubsub</name>
  <version>0.0.0</version>
  <description>TODO: Package description</description>
  <maintainer email="noah@todo.todo">noah</maintainer>
  <license>TODO: License declaration</license>
  <depend>rclpy</depend>
  <depend>std_msgs</depend>
  <test_depend>ament_copyright</test_depend>
  <test_depend>ament_flake8</test_depend>
  <test_depend>ament_pep257</test_depend>
  <test_depend>python3-pytest</test_depend>

  <export>
    <build_type>ament_python</build_type>
  </export>
</package>
```

`<depend>` 声明运行与构建都需要的能力，`<test_depend>` 声明仅测试阶段使用的能力，`<export>` 中的 `build_type` 决定构建方式。Python 功能包使用 `ament_python`，C++ 功能包使用 `ament_cmake`。源文件中用到的每一个消息包都应在依赖中登记，否则编译或运行阶段会报找不到类型。

5.3.5. CMakeLists.txt：C++ 功能包的编译规则

教程的 C++ 话题通信功能包配置如下：

```
cmake_minimum_required(VERSION 3.8)
project(cpp_pubsub)

find_package(ament_cmake REQUIRED)
find_package(rclcpp REQUIRED)
find_package(std_msgs REQUIRED)

add_executable(talker src/publisher_member_function.cpp)
ament_target_dependencies(talker rclcpp std_msgs)

add_executable(listener src/subscriber_member_function.cpp)
ament_target_dependencies(listener rclcpp std_msgs)

install(TARGETS
talker
listener
DESTINATION lib/${PROJECT_NAME})

ament_package()
```

`add_executable` 把源文件编译为目标可执行文件，`ament_target_dependencies` 把该目标与消息库、客户端库链接起来，`install(TARGETS ... DESTINATION lib/${PROJECT_NAME})` 决定 `ros2 run` 能通过哪个路径找到它。三个环节缺少任意一个，`ros2 run` 都会找不到可执行文件。

5.3.6. setup.py：Python 功能包的入口点

```
from setuptools import setup

package_name = 'py_pubsub'

setup(
    name=package_name,
    version='0.0.0',
    packages=[package_name],
    data_files=[
        ('share/ament_index/resource_index/packages',
            ['resource/' + package_name]),
        ('share/' + package_name, ['package.xml']),
    ],
    install_requires=['setuptools'],
    zip_safe=True,
    maintainer='noah',
    maintainer_email='noah@todo.todo',
    description='TODO: Package description',
```

```
    license='TODO: License declaration',
    tests_require=['pytest'],
    entry_points={
        'console_scripts': [
            'talker = py_pubsub.publisher_member_function:main',
            'listener = py_pubsub.subscriber_member_function:main',
        ],
    },
)
```

`data_files` 中的 `resource` 与 `package.xml` 让 ROS2 的包索引能够发现该功能包；`entry_points` 的 `console_scripts` 把 Python 模块映射为可执行名，因此 `ros2 run py_pubsub talker` 实际调用的是 `publisher_member_function` 中的 `main` 函数。功能包同名目录中还必须存在 `__init__.py`，否则模块无法被正确导入。

5.4. 3. 节点、通信模型、接口与计算图

5.4.1. 节点与话题

节点（Node）是一切通信对象的载体。教程的表述是：雷达驱动节点负责发布雷达消息，摄像头驱动节点负责发布图像消息，一个完整的机器人系统由许多协同工作的节点组成，并且单个可执行文件可以包含一个或多个节点。

话题（Topic）把使用同名、同类型消息的节点关联起来，构成通信的基础。话题的发布方与订阅方是多对多关系，适合持续更新、逻辑处理较少的数据传输场景，例如传感器数据流。

5.4.2. 四种通信模型

模型	交互方式	关系	典型用途
话题	发布方发布数据，订阅方接收数据，数据流单向	多对多	持续更新的传感器数据与状态流
服务	客户端发送请求，服务端返回结果	一对多	偶然发生、有逻辑处理、需要应答的操作
动作	客户端发送目标，服务端持续反馈并给出最终结果，可取消	一对多	耗时的请求响应，需要连续反馈
参数	服务端保存数据，客户端读取与修改	共享	节点配置项与运行期可调参数

选择依据可以简化为三个问题：数据是否持续产生，调用是否需要等待结果，过程是否需要中间反馈。持续产生用话题，需要等待单次结果用服务，需要等待长时间任务并观察进度用动作，需要调整节点配置用参数。

5.4.3. 消息、服务与动作接口

通信需要数据载体，ROS2 把载体称为接口（interface），接口文件有 `msg`、`srv`、`action` 三种。教程用激光雷达说明统一接口的价值：不同厂家雷达的驱动方式与扫描速率各不相同，但都把自己的数据转换成 `sensor_msgs/msg/LaserScan`，于是导航程序不必为每种雷达单独适配。

这个例子也说明了接口的排障意义。话题名相同但消息类型不同，两个节点不会成功通信；反过来，只要接口类型一致，发布方与订阅方可以用不同编程语言实现。

5.4.4. 计算图

把运行中的节点、话题、服务、动作与参数作为整体观察，就得到系统的计算图。计算图是排障的主要视角：

- `ros2 node list` 与 `ros2 node info` 给出节点与其连接关系。
- `ros2 topic list`、`ros2 service list`、`ros2 action list` 给出通信端点。
- `rqt` 的 Node Graph 插件把计算图以图形方式呈现。

定位问题时先看计算图是否符合预期，再进入具体话题的数据内容，可以避免在错误方向上反复读取数据。

5.5. 4. 命令行工具

教程给出的常用命令包括 `ros2 pkg`、`ros2 run`、`ros2 node`、`ros2 topic`、`ros2 interface`、`ros2 service`、`ros2 action`、`ros2 param`。所有子命令都可以用 `-h` 或 `--help` 查看帮助，例如 `ros2 node -h` 与 `ros2 node list --help`。

5.5.1. 运行节点与启动文件

```
ros2 run pkg_name node_name
ros2 run cpp_pubsub listener
ros2 run cpp_pubsub talker
```

`ros2 run` 的第一个参数是功能包名，第二个参数是该包中的可执行文件名。上面的两行分别启动订阅方与发布方，通常需要打开两个终端，先启动订阅方再启动发布方可以立即看到数据。教程中 Python 功能包的用法一致，只需把包名换成 `py_pubsub`。

```
ros2 launch turn_on_wheeltec_robot turn_on_wheeltec_robot.launch.py
```

`ros2 launch` 的第一个参数是功能包名，第二个参数是 launch 文件名。这条命令一次启动底盘通信、IMU 处理、EKF 状态估计、TF 变换与机器人模型等多个节点，是厂商教程中启动小车底层的标准入口。

5.5.2. 节点

```
ros2 node list
ros2 node info /wheeltec_robot
```

`ros2 node list` 输出当前运行中的节点；`ros2 node info` 输出指定节点发布与订阅的话题、提供的服务及其消息类型。厂商教程建议在启动底层后用第二条命令确认节点运行状态，这一步能同时验证串口节点是否上线、话题接口是否与文档一致。

5.5.3. 话题

```
ros2 topic list
ros2 topic echo /odom
ros2 topic echo /imu/data_raw
ros2 topic echo /scan
```

第一条命令列出当前话题。后续三条分别读取底盘里程计、IMU 原始数据与激光雷达扫描数据。`ros2 topic echo` 会持续打印消息内容，确认数据正常后应使用 `Ctrl+C` 结束，避免长期占用终端。

```
ros2 topic hz /scan
ros2 topic bw /scan
ros2 topic info /scan
ros2 topic type /scan
ros2 topic find sensor_msgs/msg/LaserScan
ros2 topic pub /cmd_vel geometry_msgs/msg/Twist "{linear: {x: 0.1}, angular: {z: 0.0}}"
```

`hz` 输出发布频率，`bw` 输出带宽占用，`info` 输出发布方与订阅方数量，`type` 输出接口类型，`find` 按类型反查话题。`pub` 向指定话题发布消息，上例向底盘速度话题发送一个前进指令；在真实机器人上执行前必须确认车体周围安全，并准备好停止手段。

5.5.4. 服务

```
ros2 service list
ros2 service find turtlesim/srv/Spawn
ros2 service type /spawn
```

服务列表与类型查询用于确认服务端点是否存在、接口是否符合预期。`list` 输出运行中的服务，`find` 按服务类型查找，`type` 输出指定服务使用的接口类型。

```
ros2 service call /spawn turtlesim/srv/Spawn "{x: 1.0,y: 2.0,theta: 0.3,name: turtle2}"
```

`ros2 service call` 向服务发送请求，参数为服务名、接口类型与请求内容。教程用海龟仿真器的 `/spawn` 服务演示了这种调用方式，请求字段采用 YAML 形式的字典。

5.5.5. 动作

```
ros2 action list
ros2 action info /turtle1/rotate_absolute
ros2 action send_goal /turtle1/rotate_absolute turtlesim/action/RotateAbsolute theta:\ 0.0
```

`ros2 action list` 输出运行中的动作，`info` 输出动作的客户端与服务端信息，`send_goal` 发送动作目标。动作与服务的差别在于是否有中间反馈与取消能力，因此 `send_goal` 的输出通常包含中间反馈与最终结果两部分。

5.5.6. 参数

```
ros2 param list
ros2 param describe turtlesim background_b
ros2 param get turtlesim background_b
ros2 param set turtlesim background_b 10
```

`param list` 输出可用参数，`describe` 输出参数的描述信息，`get` 与 `set` 分别读取和修改参数值。教程用海龟仿真器的背景色参数演示了完整流程。参数属于运行期配置，修改后一般不需要重新编译，但重新启动节点时是否保留取决于节点实现与参数文件。

```
ros2 param dump /turtlesim
ros2 param load /turtlesim /tmp/turtlesim_params.yaml
```

`dump` 把节点参数写入文件，`load` 从文件把参数加载到节点。这一对命令常用于保存可复现的实验配置，把当前参数集作为记录的一部分留存。

5.5.7. 接口

```
ros2 interface list
ros2 interface packages
ros2 interface package sensor_msgs
ros2 interface show sensor_msgs/msg/LaserScan
```

```
ros2 interface proto sensor_msgs/msg/LaserScan
```

`list` 输出所有可用接口，`packages` 输出包含接口消息的功能包，`package` 输出指定功能包下的接口，`show` 输出接口定义格式，`proto` 输出接口消息原型。读取 `show` 的输出是理解话题内容最快的方式，字段含义、单位与嵌套结构都在其中。

5.5.8. 功能包与记录回放

```
ros2 pkg list
ros2 pkg executables rqt_gui
ros2 pkg prefix turn_on_wheeltec_robot
ros2 pkg xml turn_on_wheeltec_robot
```

`pkg list` 输出可用功能包，`pkg executables` 输出指定包的可执行文件列表，`pkg prefix` 输出包的安装前缀路径，`pkg xml` 输出包的清单内容。`ros2 pkg executables` 与 `ros2 run` 配合使用，可以在不读源码的情况下确认节点名称。

```
ros2 bag record /scan
ros2 bag info rosbag2_lidar
ros2 bag play rosbag2_lidar
```

`ros2 bag` 用于记录与回放机器人数据，详细用法见第 5 节。这三条命令分别完成录制、查看元信息与回放。

5.6. 5. rqt、rviz2 与 ros2 bag

5.6.1. rqt

```
rqt
ros2 run rqt_gui rqt_gui
```

两种方式都可以启动 rqt。rqt 是基于 Qt 的图形界面框架，以插件形式提供节点图、话题监视、图像查看与 TF 树等工具。启动后通过 plugins 菜单添加所需插件。

教程列出的四个常用插件与用途：

插件	用途
Node Graph	查看节点之间的连接关系，等价于图形化的计算图
Topic Monitor	监视话题的发布状态与数据
Image View	查看图像话题的数据流
TF Tree	查看坐标变换树的连接与完整性

5.6.2. rviz2

```
rviz2
```

`rviz2` 是 ROS2 的数据可视化工具，用于把抽象数据以图形方式呈现。教程中的典型用法包括：

- 添加 LaserScan 显示组件，选择 `/scan` 话题，把 Fixed Frame 设为 `laser`，即可看到雷达点云。Fixed Frame 定义可视化数据的基准坐标系，雷达数据会按照它做坐标转换。
- 添加 Imu 显示组件，选择 `/imu/data_raw` 话题，观察姿态数据。厂商教程说明镜像中默认已安装该插件，若没有找到，可按发行版安装：

```
sudo apt install ros-humble-rviz-imu-plugin
```

在 rviz2 中看不到数据时，最常见的原因是 Fixed Frame 选错、坐标变换缺失或话题尚未发布。排查顺序是先确认 `ros2 topic list` 中存在目标话题，再用 `ros2 topic echo` 确认有数据，最后检查 TF 树是否包含所需坐标系。

5.6.3. ros2 bag

教程给出的 bag 子命令包括 `record`、`info`、`play`、`convert`、`list` 与 `reindex`。

```
ros2 bag record /topic-name
ros2 bag record topic-name1 topic-name2
ros2 bag record -a
```

第一行记录单个话题，第二行记录多个话题，第三行记录所有话题。录制话题较多时数据量增长很快，建议先明确需要记录的话题集合，并预先确认磁盘剩余空间。

```
ros2 bag info rosbag2_lidar
ros2 bag play rosbag2_lidar
ros2 bag play rosbag2_lidar -r 10
ros2 bag play rosbag2_lidar -l
ros2 bag play rosbag2_lidar --topics /topic-name
```

`info` 输出 bag 文件的相关信息，`play` 回放数据，`-r 10` 表示 10 倍速播放，`-l` 表示循环播放，`--topics` 指定只回放某个话题。回放期间可以用 `ros2 topic echo /topic-name` 验证数据是否按预期出现。

bag 文件属于实验记录，可能包含传感器原始数据。是否纳入版本控制取决于数据规模与隐私边界；本教材建议把 bag 文件留在本地产物目录，只在必要时保留小型、脱敏的样例。

5.7. 6. Python launch 文件与参数传递

5.7.1. launch 文件的定位

一个机器人系统往往需要同时启动多个节点，并且每个节点都要按场景配置参数。launch 文件通过统一的语法与规则组织这些启动动作。教程指出 ROS2 的 launch 文件支持 xml、yaml 与 python 三种格式，其中 Python 格式可以借助语言特性实现条件判断、参数化配置与更复杂的启动逻辑。

launch 文件的基本结构是把每个节点、文件或脚本抽象成一个 action，再由 `LaunchDescription` 统一返回：

```python
def generate_launch_description():
    return LaunchDescription([
        action_1,
        action_2,
        # ...
        action_n
    ])
```

`generate_launch_description` 是启动器的入口函数，返回值是所有待执行动作的描述列表。缺少这个函数名或返回值类型不正确，`ros2 launch` 会直接报错。

5.7.2. 组合节点与包含其他 launch 文件

教程给出的示例展示了包含子 launch 文件与启动节点的组合方式：

```python
import os
from pathlib import Path
from ament_index_python.packages import get_package_share_directory
from launch import LaunchDescription
from launch.actions import IncludeLaunchDescription
from launch.launch_description_sources import PythonLaunchDescriptionSource
import launch_ros.actions

def generate_launch_description():
    launch_dir = os.path.join(
        get_package_share_directory('turn_on_wheeltec_robot'),
        'launch'
    )
    imu_config = Path(
        get_package_share_directory('turn_on_wheeltec_robot'),
        'config', 'imu.yaml'
    )

    wheeltec_robot = IncludeLaunchDescription(
        PythonLaunchDescriptionSource(
            os.path.join(launch_dir, 'base_serial.launch.py')),
```

```
        launch_arguments={'akmcar': 'false'}.items(),
    )

    imu_filter_node = launch_ros.actions.Node(
        package='imu_filter_madgwick',
        executable='imu_filter_madgwick_node',
        parameters=[imu_config]
    )

    return LaunchDescription([
        wheeltec_robot,
        imu_filter_node
    ])
```

`get_package_share_directory` 取得功能包安装后的共享目录路径,`IncludeLaunchDescription` 复用已有 launch 文件，`launch_arguments` 把参数传给被包含的文件，`launch_ros.actions.Node` 启动单个节点，`parameters=[imu_config]` 从 YAML 文件加载节点参数。用参数文件而不是把数值写死在代码里，可以让同一份代码适配不同车型与传感器。

5.7.3. 真实底盘启动文件的结构

厂商的 `turn_on_wheeltec_robot.launch.py` 是这一模式的实际应用，其结构可以概括为：

```
carto_slam = LaunchConfiguration('carto_slam', default='false')
carto_slam_dec = DeclareLaunchArgument('carto_slam', default_value='false')

wheeltec_robot = IncludeLaunchDescription(
    PythonLaunchDescriptionSource(os.path.join(launch_dir, 'base_serial.launch.py')),
    launch_arguments={'akmcar': 'false'}.items(),
)

robot_ekf = IncludeLaunchDescription(
    PythonLaunchDescriptionSource(os.path.join(launch_dir, 'wheeltec_ekf.launch.py')),
    launch_arguments={'carto_slam': carto_slam}.items(),
)

base_to_link = launch_ros.actions.Node(
    package='tf2_ros',
    executable='static_transform_publisher',
    name='base_to_link',
    arguments=['0', '0', '0', '0', '0', '0', 'base_footprint', 'base_link'],
)

ld = LaunchDescription()
ld.add_action(carto_slam_dec)
ld.add_action(wheeltec_robot)
ld.add_action(base_to_link)
ld.add_action(robot_ekf)
return ld
```

`DeclareLaunchArgument` 声明一个可以在命令行覆盖的启动参数，`LaunchConfiguration` 读取该参数的当前值。`carto_slam` 因此成为一个开关：默认取 `false`，使用 Cartographer 的功能包可以把它改成 `true`，被包含的 EKF launch 文件据此选择不同的参数文件。

这种参数回传写法在阅读大型启动文件时很常见，理解 `DeclareLaunchArgument`、`LaunchConfiguration` 与 `launch_arguments` 三者关系，就能沿着启动链条找到最终生效的配置。

```
ros2 launch turn_on_wheeltec_robot turn_on_wheeltec_robot.launch.py carto_slam:=true
ros2 launch turn_on_wheeltec_robot turn_on_wheeltec_robot.launch.py --show-args
```

命令行使用 `参数名:=值` 的形式覆盖启动参数，`--show-args` 列出该 launch 文件接受的参数及默认值。修改参数前先查看参数清单，可以避免拼写错误导致的静默失效。

5.7.4. 参数传递的三种方式

- YAML 参数文件：把参数集中写在 `config/*.yaml`，由 `parameters=[<path>]` 加载。厂商的 `imu.yaml`、`ekf.yaml`、`ekf_carto.yaml` 都属于这一类。
- 命令行覆盖：用 `参数名:=值` 覆盖 launch 参数，适合临时切换车型或功能开关。
- 启动文件内直接传字典：在 `Node` 的 `parameters` 中直接给出参数键值，适合少量、与场景强绑定的配置。

参数来源越多，越需要在启动日志中确认最终生效值。ROS2 节点在参数不符时会给出警告或使用默认值，这两类信息都应当保留在日志中。

5.8. 7. TF2 坐标变换与 URDF

5.8.1. TF2 的作用

移动机器人上的传感器与执行器分布在不同位置，测量数据必须转换到统一坐标系才能组合使用。TF2 提供管理坐标系相对关系与查询变换的工具。

完整的坐标变换由广播方与监听方两部分组成：每个广播方发布一组坐标系相对关系，监听方把多组关系融合成一棵坐标树。ROS 中的坐标变换基于右手坐标系，变换关系可以分解为平移与旋转两部分。

5.8.2. 静态坐标变换

两个坐标系相对位置固定时使用静态变换，例如雷达与 `base_link` 之间的安装位置。教程给出的命令行用法：

```
# 发布 A 到 B 的位姿
# 格式: ros2 run tf2_ros static_transform_publisher <x偏移> <y偏移> <z偏移> <偏航角> <俯仰角> <
    ↪ 横滚角> <父坐标系> <子坐标系>
ros2 run tf2_ros static_transform_publisher 0 0 3 0 0 3.14 A B

# 监听/获取 TF 关系
ros2 run tf2_ros tf2_echo A B
```

`static_transform_publisher` 按固定顺序接收平移、旋转与父子坐标系参数，`tf2_echo` 打印两个坐标系之间的实时变换。参数顺序写错会得到看起来“有数据但方向不对”的结果，因此修改安装参数后应同时用 `tf2_echo` 与 rviz2 交叉确认。

教程给出的 launch 写法把雷达安装位置写成静态变换：

```
launch_ros.actions.Node(
    package='tf2_ros',
    executable='static_transform_publisher',
    name='base_to_laser',
    arguments=['0.048', '0', '0.18', '0', '0', '0', 'base_footprint', 'laser'],
)
```

这组数值表示雷达相对 `base_footprint` 的 x、y、z 偏移与零旋转。不同车型的安装位置不同，厂商的 `robot_mode_description.launch.py` 为每种车型分别给出激光雷达与相机的平移参数。

5.8.3. 坐标树与可视化

```
ros2 run tf2_tools view_frames
rqt
```

`view_frames` 生成坐标树的报告文件，rqt 的 TF Tree 插件以图形方式显示同样的结构。检查坐标树时关注三件事：是否只有一个根坐标系、每个子坐标系是否只有一条到根的路径、传感器坐标系是否都已挂接到机器人本体。

5.8.4. URDF 模型描述

URDF 用 XML 描述机器人的连杆（link）与关节（joint）。教程把小车简化为车体、左右轮子、激光雷达与相机、计算平台等部分，建模过程就是把这些部分逐一描述并组合起来。

连杆描述刚体的外观与物理属性：

```
<link name="left_wheel_link">
    <visual>
        <origin xyz="0 0 0" rpy="0 0 0" />
        <geometry>
```

```
            <cylinder radius="0.0325" length="0.025"/>
        </geometry>
        <material name="black">
            <color rgba="0 0 0 1"/>
        </material>
    </visual>
</link>
```

origin 设置连杆的偏移与旋转，geometry 设置形状，示例使用圆柱体表示车轮，material 设置视觉颜色。name 必须在文件中唯一，否则模型树无法正确解析。

关节描述两个刚体之间的连接方式：

```
<joint name="left_wheel_joint" type="continuous">
    <origin xyz="0 0.08 0.0325" rpy="1.57 0 0"/>
    <parent link="base_link"/>
    <child link="left_wheel_link"/>
    <axis xyz="0 1 0"/>
</joint>
```

parent 与 child 定义连接方向，origin 描述子连杆相对父连杆的偏移与旋转，axis 给出关节运动轴的单位向量。关节类型共有六种，常见的有 continuous（连续旋转）、revolute（有角度限制的旋转）、prismatic（滑动）、fixed（固定）与 planar、floating。选择错误会让模型在可视化中表现异常，或在运动学计算中给出与实际不符的自由度。

所有 link 与 joint 最终放在一个 robot 标签内，构成完整的机器人模型。模型文件由 robot_state_publisher 读取并发布到 TF 树，关节状态由 joint_state_publisher 提供，厂商的 robot_mode_description.launch.py 中可以看到这两个节点的配置方式。

5.8.5. 替换 URDF 模型的流程

教程给出的模型替换流程可以归纳为四步：

1. 把导出模型包中的 meshes 目录复制到 wheeltec_robot_urdf 功能包的 meshes 目录，并按车型重命名。
2. 把导出的 urdf 文件复制到该功能包的 urdf 目录。
3. 打开 urdf 文件，把所有网格文件路径替换为新车型的 meshes 目录路径。
4. 在 robot_mode_description.launch.py 中新增该车型的配置块，车型名与 urdf 名保持一致，并按实测调整坐标参数，然后单独编译相关功能包。

模型替换后最常见的两个问题是相机模型脱离车体、车体部分出现在地面以下。教程给出的处理方式是：修改 turn_on_wheeltec_robot.launch.py 中 base_footprint 的 z 坐标调整整体高度；修改 robot_mode_description.launch.py 中对应车型的 xyz 参数调整相机与雷达位置；若仍有偏移，再检查父坐标系应使用 base_footprint 还是 base_link。

单独编译相关功能包时使用：

```
colcon build --symlink-install --packages-select wheeltec_robot_urdf
colcon build --symlink-install --packages-select turn_on_wheeltec_robot
```

模型文件与启动文件分属两个功能包，改动哪个就重新编译哪个。使用 `--symlink-install` 时，仅修改 launch 文件或 YAML 参数可以省去重新编译步骤。

5.9. 8. 相机、雷达、IMU 与串口底盘接口

本节整理厂商教程中出现的话题与接口。这些接口属于 WHEELTEC 平台的 documented 配置，本教材没有在昇腾 310B 上运行过这些节点。

5.9.1. 接口总览

接口	消息类型	方向	说明	来源
`/odom`	`nav_msgs/msg/Odometry`	底盘节点发布	里程计位姿与速度	底盘数据教程
`/imu/data_raw`	`sensor_msgs/msg/Imu`	底盘节点发布	板载 IMU 原始数据	底盘数据教程
`/scan`	`sensor_msgs/msg/LaserScan`	雷达驱动发布	单线激光雷达扫描数据	雷达数据教程
`/cmd_vel`	`geometry_msgs/msg/Twist`	上层节点发布	目标线速度与角速度	STM32 通信文档
`/red_vel`	速度消息	上层节点发布	红外对接速度	STM32 通信文档
`/robot_recharge_flag`	标志消息	上层节点发布	回充命令标志	STM32 通信文档

电池电压、超声波与自动回充的其余话题在文档中以功能描述出现而未给出完整话题名，使用时以 `ros2 topic list` 的实际输出为准。

5.9.2. 激光雷达

雷达驱动的统一输出接口是 `sensor_msgs/msg/LaserScan`。厂商教程给出的确认流程是：先启动雷达，再读取话题，最后用 rviz2 可视化。

```
ros2 launch turn_on_wheeltec_robot wheeltec_lidar.launch.py
ros2 topic echo /scan
rviz2
```

第一条命令启动雷达驱动，第二条读取扫描数据，第三条启动可视化工具。在 rviz2 中添加 LaserScan 显示组件并选择 `/scan` 话题，把 Fixed Frame 设为 `laser`，即可观察到点云。

雷达型号在 `wheeltec_param.yaml` 中选择；较旧版本代码的型号设置位置可能在 `wheeltec_lidar.launch.py`。型号配置错误时驱动可能启动但不发布数据，因此启动后必须用话题输出确认，而不能只看终端是否停止报错。

5.9.3. IMU

```
ros2 launch turn_on_wheeltec_robot turn_on_wheeltec_robot.launch.py
ros2 topic echo /imu/data_raw
```

底盘节点启动后会持续发布 IMU 数据。厂商教程说明镜像中已包含 rviz 的 IMU 可视化插件，安装命令见第 5.2 节。

底盘节点同时运行 `Quaternion_Solution.cpp` 中的轻量级姿态解算模块，把加速度计与陀螺仪数据融合为四元数姿态。上层若需要滤波后的姿态，厂商的启动文件会额外运行 `imu_filter_madgwick` 节点并加载 `config/imu.yaml`。

5.9.4. 串口底盘

ROS 主控通过 USB 与 STM32 控制器通信，设备在系统中会被识别为 `/dev/ttyCH343USB0` 或 `/dev/ttyUSB0` 等名称。教程指出这些名称在每次连接时可能变化，且默认可能没有读写权限，因此使用固定的 udev 别名单一入口。

传感器	设备号	别名
串口版雷达	0001	`/dev/wheeltec_lidar`
STM32 控制器	0002	`/dev/wheeltec_controller`
惯导 IMU	0003	`/dev/wheeltec_IMU`
语音传感器	0004	`/dev/wheeltec_mic`
GNSS 接收机	0005	`/dev/wheeltec_gnss`

别名脚本存放在 `turn_on_wheeltec_robot` 功能包的 `wheeltec_udev.sh` 文件中：

```
sudo chmod 777 wheeltec_udev.sh
sudo sh wheeltec_udev.sh
```

执行后重新插拔设备，再用 `ll /dev` 确认别名出现。此后无论接入哪个 USB 接口，代码都可以使用固定别名访问设备。注意 `chmod 777` 只用于厂商脚本的首次赋权，长期使用应改为更严格的权限设置。

底盘节点 `wheeltec_robot_node` 的数据流如下：

- 通过串口接收 STM32 上报的编码器、IMU、超声波与电池电压数据，按 24 字节数据帧接收并校验帧尾与校验和，解析后转换为国际单位。
- 把速度积分得到位姿，发布里程计、IMU 与电池电压话题。
- 订阅 `/cmd_vel`，把目标速度换算为整数并按协议打包成 11 字节发送帧，计算校验值后写入固定位置，帧尾为 `0x7D`。
- 节点退出时在析构函数中向串口发送一帧目标速度为零的指令，再关闭串口。

退出时自动下发零速度是重要安全设计。上层程序异常退出后如果仍保持旧速度，底盘会继续运动，因此集成新节点时应保留这条停止路径，并避免用强制结束方式绕过析构。

```
ros2 launch turn_on_wheeltec_robot turn_on_wheeltec_robot.launch.py
ros2 node info /wheeltec_robot
```

第二条命令可以一次性确认该节点发布与订阅的话题列表及其消息类型，是核对底盘接口最直接的方式。

5.9.5. 相机

厂商教程把相机作为可选的传感器功能包启动，`turn_on_wheeltec_robot` 功能包中包含 `wheeltec_camera.launch.py`，并为 Astra 相机提供 `config/camera_info.yaml` 内参文件。

```
ros2 launch turn_on_wheeltec_robot wheeltec_camera.launch.py
ros2 topic list
```

相机驱动通常以 `sensor_msgs/msg/Image` 发布图像、以 `sensor_msgs/msg/CameraInfo` 发布内参，订阅方需要同时使用两者才能完成几何还原。具体话题名取决于驱动实现，以上命令中的 `ros2 topic list` 输出才是最终依据。使用 `ros2 topic info` 与 `ros2 interface show` 可以进一步确认消息类型与字段。

相机内参文件与图像话题的对应关系必须在集成时核对。用错内参会让测距、跟随与视觉伺服出现系统性偏差，而且这种偏差在画面上通常不易察觉。

5.10. 9. SSH 与 VNC 远程操作 ROS2

5.10.1. 何时使用 SSH

SSH 适合命令行操作：编译功能包、启动 launch 文件、读取话题、查看节点信息、观察日志。它的带宽占用低、可脚本化、输出可以完整复制到记录中，是远程调试机器人的默认选择。

```
ssh -Y wheeltec@192.168.0.100
```

`-Y` 启用可信 X11 转发，使远程的图形程序可以显示在本机。转发 rviz2 与 rqt 的延迟通常明显高于本地运行，因此只有在无法直接使用 VNC 或只做少量图形检查时才值得使用。若本机没有运行 X 服务，图形程序无法启动，但命令行部分不受影响。

```
ping 192.168.0.100
scp local_file wheeltec@192.168.0.100:~/
```

第一条命令确认网络可达，第二条把本地文件复制到远端用户目录。厂商教程说明小车的默认 IP 为 `192.168.0.100`，默认热点名称包含 WHEELTEC 字样；热点密码以随车资料为准，首次使用时应当修改。修改网络配置后 IP 可能变化，连接前先确认当前地址。

5.10.2. 何时使用 VNC

VNC 适合需要完整桌面环境的场景：持续观察 rviz2 的点云与模型、使用 rqt 的图形插件、比对窗口中的模型与真实机器人。它以图像传输为主，带宽占用高于 SSH，刷新率与延迟受网络质量影响明显。

```
vncserver :1 -geometry 1280x720 -depth 24
vncviewer 192.168.0.100:1
```

第一条命令在远端启动显示号为 1 的 VNC 会话，第二条在本机连接该会话。VNC 服务端实现取决于板端安装（TigerVNC、x11vnc 等），参数与启动方式可能不同，使用前应确认远端已安装并配置对应服务。厂商教程中远程登录以 SSH 为主，VNC 属于需要时启用的补充手段。

5.10.3. 选择建议

任务	推荐方式	原因
编译功能包、查看构建日志	SSH	输出为文本，可复制进记录，带宽占用低
`ros2 topic echo`、`ros2 param` 调试	SSH	命令行工具即可完成，无需图形界面
启动 launch 并观察终端输出	SSH	可以直接看到节点错误与警告
长时间观察 rviz2 点云或模型	VNC	图像持续刷新，X11 转发的延迟难以接受
rqt 插件组合调试	VNC	多窗口交互在完整桌面下更可靠

批量文件同步	SSH 与 `scp`	可以配合校验与定向复制

两种方式可以同时使用：用 SSH 启动节点并读取日志，用 VNC 观察图形结果。混用时注意不要在两侧重复启动同一节点，重复的节点名会让话题出现多个发布方，从而给出难以解释的数据。

5.10.4. 远程操作的安全边界

- 厂商默认账号与默认密码应在首次部署后修改，不要在公开文档中留存口令。
- 只在受控网络内开放 SSH 与 VNC 端口，避免直接暴露到公网。
- 远程发送底盘速度指令前确认车体周围无人，并保持一个可用的停止终端。
- 远程会话断开不等于节点停止。断开前用 `Ctrl+C` 有序结束节点，或确认已配置的服务状态。

5.11. 10. 多机通信与 ROS_DOMAIN_ID

5.11.1. 域号与发现范围

ROS2 节点通过 DDS 在同一域内相互发现。`ROS_DOMAIN_ID` 相同的节点才会参与同一发现范围，不同域之间默认互不可见。这为多组实验提供了隔离手段：同一网络中可以并行运行若干互不干扰的 ROS2 系统。

```
export ROS_DOMAIN_ID=42
echo "export ROS_DOMAIN_ID=42" >> ~/.bashrc
```

第一条命令为当前终端设置域号，第二条写入 `~/.bashrc` 使其对新终端生效。参与通信的所有机器与终端必须使用相同域号。官方文档建议在常用范围内选择域号并避免冲突；本教材建议把域号写入实验记录，避免复现时遗漏导致“节点互相看不见”。

```
ros2 daemon stop
ros2 daemon start
```

修改域号或网络配置后，先停止再启动 daemon 可以避免旧发现信息残留。daemon 只服务于命令行工具的查询，不参与节点之间的业务通信。

5.11.2. 同网段与网络检查

多机通信通常要求主机与机器人在同一子网，且网络允许组播。跨网段、NAT、禁用组播的无线网络都会影响发现过程。

```
ip addr
ip route
ping 192.168.0.100
```

`ip addr` 查看本机地址与子网掩码，`ip route` 查看路由，`ping` 确认基础可达性。三者的组合可以判断两台机器是否真正处于同一网段，而不只是“看起来在同一个网络里”。

```
echo $ROS_DOMAIN_ID
echo $ROS_LOCALHOST_ONLY
echo $RMW_IMPLEMENTATION
```

第一条确认当前域号，第二条确认是否被限制在本地回环，第三条确认当前使用的 DDS 实现。`ROS_LOCALHOST_ONLY=1` 会把通信限制在本机，跨机调试时它必须为未设置或 `0`。两台机器上的 `RMW_IMPLEMENTATION` 应保持一致，混用不同实现会带来发现与类型支持差异。

```
ros2 node list
ros2 topic list
ros2 topic info /scan
```

在多机场景中，这三条命令是最直接的验证方式：如果本机能看到对方的节点与话题，说明域号、网络与发现配置一致；如果只看到本机节点，问题在发现层而不是话题内容层。

```
ros2 doctor --report
```

`ros2 doctor` 汇总环境变量、网络与中间件的检查结果，可作为多机排障的起点。它是 ROS2 官方工具，报告的条目较多，应按“域号与本地限制、网络地址、中间件实现”的顺序阅读。

5.11.3. 常见发现失败原因

- 两台机器域号不同，或其中一台把域号写进了不同的 shell 配置。
- 网络使用 NAT 或客户端隔离，组播无法到达对端。
- 防火墙阻止了组播与 DDS 使用的端口。
- 主机名无法解析，发现过程无法完成地址交换。
- 一端启用了本地回环限制。
- 两端使用不同的 DDS 实现或类型支持配置。

发现失败时的处理顺序是先确认域号与环境变量，再确认网段与组播，最后才考虑配置初始对等点或发现服务器。基础网络问题没有排除之前，改中间件参数通常只会增加变量。

5.12. 11. 自启动、日志与进程管理

5.12.1. 日志位置

ROS2 的日志分为三层：

- 构建日志：`colcon build` 的输出，工作空间的 `log` 目录中按功能包与时间保存。
- 节点运行日志：默认写入 `~/.ros/log`，可通过环境变量覆盖到指定目录。
- 终端输出：launch 启动的节点会把警告与错误打印到终端，这是排障时最先查看的位置。

```
ls -lt ~/.ros/log | head
ros2 launch turn_on_wheeltec_robot turn_on_wheeltec_robot.launch.py --ros-args --log-level
    ↪ info
```

第一条命令列出最近的运行日志目录。第二条为通过 launch 启动的节点设置日志级别，出现异常时可以临时提高到 `debug` 获取更详细的信息。日志级别是诊断手段，不应长期使用 `debug`，否则会掩盖真正的错误信息。

5.12.2. 进程查看与有序停止

```
pgrep -af ros2
pgrep -af wheeltec_robot
ps -ef | grep ros2
```

`pgrep` 按名称查找进程并显示完整命令行，`ps` 与 `grep` 的组合用于交叉确认。停止节点前必须先确认目标 PID 属于本次启动的进程，避免误停其他会话中的节点。

```
kill <PID>
```

终止单个进程时优先使用 `kill` 发送默认信号，让节点执行回调与析构流程。底盘节点在析构函数中会下发零速度指令，强制结束会跳过这一步，因此只有在进程无响应时才考虑更强的方式，并在之后确认底盘已停止。

```
ss -ltnp
```

这条命令列出本机监听端口及其进程。机器人上常同时运行 ROS2 节点、Web 服务与调试工具，确认端口占用可以避免不同服务相互冲突。

5.12.3. 自启动方案

ROS2 节点本身没有内建开机自启机制，需要借助系统服务或桌面会话配置。推荐使用 systemd 服务，因为它可以定义依赖、重启策略与日志去处。下面是一个模板，属于 proposed 方案，需要在目标平台上实际验证后再启用：

```
[Unit]
Description=WHEELTEC ROS2 bringup
After=network-online.target

[Service]
Type=simple
User=wheeltec
ExecStart=/bin/bash -lc 'source /opt/ros/humble/setup.bash && source /home/wheeltec/
    ↪ wheeltec_ros2/install/setup.bash && ros2 launch turn_on_wheeltec_robot
    ↪ turn_on_wheeltec_robot.launch.py'
Restart=on-failure
RestartSec=5
Environment=ROS_DOMAIN_ID=42

[Install]
WantedBy=multi-user.target
```

模板中的关键点有四个：`ExecStart` 用登录 shell 加载 ROS2 与工作空间环境；`User` 指定运行账号；`ROS_DOMAIN_ID` 显式声明域号；`Restart` 只对异常退出重启，避免正常停止后又被拉起。保存为服务单元后可用下列命令启用：

```
sudo systemctl daemon-reload
sudo systemctl enable --now wheeltec-ros2.service
systemctl status wheeltec-ros2.service
journalctl -u wheeltec-ros2.service -n 100
```

前两条命令重新加载单元并启用服务，后两条查看服务状态与最近日志。启用前必须确认串口设备别名已经建立、网络已就绪、启动文件不需要交互输入，否则服务会在开机后反复失败。

桌面会话下的自启动可以写入 `~/.config/autostart`，适合需要图形界面的场景，例如开机后自动启动 rviz2。无论采用哪种方式，都应保证同一节点只被启动一次，多个实例会产生重复发布方，给数据解释带来困难。

5.12.4. 运行状态检查清单

- 服务或终端中确认 launch 已启动且没有报错。
- `ros2 node list` 中节点数量与预期一致。
- `ros2 topic echo` 目标话题有数据。
- 出现异常时先读日志，再考虑重启。

- 重启前确认旧进程已经退出，避免遗留进程占用串口。

5.13. 12. proposed / untested：ROS2 与昇腾 310B 推理服务的集成路径

证据标记：proposed + untested。本节描述的是设计路径，不是已验证方案。本教材没有在昇腾 310B 上运行过 ROS2，也没有把本仓库的推理代码接入任何 ROS2 节点。文中不包含性能、精度与延迟结论；这些数据必须在板端按完整协议测量并留存日志后才能给出。

5.13.1. 目标形态

集成目标可以用一条数据流概括：

```
ROS2 传感器节点  -->  图像或点云话题  -->  推理适配节点  -->  结果话题 / 服务
                                                |
                                                +--> 现有 OM + ACL 运行时
```

推理适配节点承担三件事：订阅标准传感器话题、把消息转换为现有运行时接受的输入、把推理结果发布回 ROS2。现有案例的推理代码与其 HTTP 接口保持不变，ROS2 只作为机器人侧的组织方式接入。

5.13.2. 前置条件

以下每一条都是未验证前提，必须在板端逐项确认：

- 板端系统发行版与所选 ROS2 发行版的官方支持列表匹配，且 Python 版本与运行时依赖不冲突。
- 板端可以正常加载 CANN 环境并完成一次既有模型的推理，作为接入前的基线。
- 传感器驱动能够在 ROS2 中发布标准消息类型。
- 串口设备别名、权限与 udev 规则已经生效。
- 计算资源余量足以同时运行传感器驱动、推理进程与上层节点，这一点需要实测而不能估算。

5.13.3. 建议的实现步骤

1. 建立独立工作空间，避免与案例目录混放。

```
mkdir -p ~/ros2_npu_ws/src
cd ~/ros2_npu_ws
colcon build
source install/setup.bash
```

工作空间独立后，ROS2 的构建产物与案例中的 Python 环境互不干扰；`install/setup.bash` 需要为每个新终端加载。

2. 用一个最小节点验证运行时可达性，只做订阅与日志输出，不加载模型。这一步把 ROS2 环境问题与模型问题分开。
3. 增加模型加载，复用现有 OM 与 ACL 执行路径，进程启动时完成一次预热推理，避免在每帧回调中重复初始化。
4. 把转换与推理放在明确的边界内：图像消息先转为数组，按既有输入契约完成预处理，输出按既有后处理逻辑转换为 ROS2 消息。不要在这一层发明新的数值约定。
5. 用 launch 文件组合传感器驱动、适配节点与上层节点，模型路径、输入尺寸与精度作为参数传入，便于复现与切换。

```
from launch import LaunchDescription
from launch.actions import DeclareLaunchArgument
from launch.substitutions import LaunchConfiguration
from launch_ros.actions import Node

def generate_launch_description():
    model_path = LaunchConfiguration('model_path')
    return LaunchDescription([
        DeclareLaunchArgument('model_path', default_value=''),
        Node(
            package='npu_bridge',
            executable='inference_node',
            parameters=[{'model_path': model_path}],
        ),
    ])
```

这段启动文件把模型路径参数化，便于在同一节点上比较不同模型，而不需要修改代码。参数留空时必须由节点给出明确错误，避免加载到错误模型后静默运行。

6. 结果消息应携带模型标识与输入来源，便于后续核对每个输出对应哪个模型与哪一帧输入。缺少这些字段会让记录无法回溯。

5.13.4. 风险与失败模式

风险	表现	建议处理
Python 版本或依赖冲突	导入失败、ABI 不匹配	先在隔离环境中确认依赖组合，再接入 ROS2

风险	表现	建议处理
中间件与网络配置冲突	节点互相不可见	按第 10 节顺序检查域号、网络与本地限制
串口权限或别名缺失	底盘节点启动失败	检查 udev 规则与设备别名
回调阻塞	话题频率下降、延迟累积	把推理放在独立线程或进程，并限制队列长度
内存增长	长时间运行后分配失败	复用缓冲，避免在每帧创建大对象
环境变量缺失	健康检查通过但推理失败	在启动脚本中显式加载 CANN 与工作空间环境
时间戳不一致	多传感器数据难以对齐	统一时间源并检查消息头的时间戳

任何一项在板端复现失败，都应记录为对应层的失败，而不是直接推导为 “ROS2 不可用” 或 “NPU 不支持”。分层记录是本书既有证据规则在 ROS2 场景下的延续。

5.13.5. 验收门槛

集成路径只有在下列门槛全部通过后，才能从 untested 升级为已验证：

- 板端完成 ROS2 环境导入与节点启动，日志完整保存。
- 适配节点在真实传感器数据下连续运行，无未处理异常。
- 推理输出与离线基线在同一输入上的数值对比通过，对比协议与允许误差记录在案。
- 进程退出时串口停止指令正常下发，机器人可靠停止。
- 断开与重连传感器后，节点行为符合预期。
- 所有结论绑定模型版本、精度、输入契约与日志路径。

5.14. 13. 课堂练习与验收标准

5.14.1. 练习任务

1. 工作空间与功能包：创建 `~/ros2_ws/src`，新建一个 Python 功能包与一个 C++ 功能包，执行 `colcon build`，并确认 `ros2 pkg list` 能列出它们。
2. 话题通信：实现发布方与订阅方节点，用 `ros2 run` 分别启动，用 `ros2 topic echo` 观察消息内容，并记录 `ros2 topic info` 的发布方与订阅方数量。
3. 服务与动作：用海龟仿真器练习 `ros2 service call` 与 `ros2 action send_goal`，说明两者在调用形式与反馈方式上的差别。
4. 参数：读取并修改一个节点的参数，用 `ros2 param dump` 保存参数文件，再用 `ros2 param load` 恢复，比较前后状态。
5. 数据记录：录制一个话题的 bag 文件，用 `ros2 bag info` 查看元信息，再回放并验证数据可用。

6. launch 文件：编写一个包含两个节点并传入参数的 Python launch 文件，用 `--show -args` 查看参数列表。
7. 坐标变换：发布一个静态变换，用 `tf2_echo` 验证结果，再在 rviz2 中观察。
8. URDF 检查：在一份 URDF 中找出所有 `link` 与 `joint`，说明关节类型的选择依据；修改一个坐标参数并观察可视化结果。
9. 接口核对：对一个实际运行的话题，用 `ros2 topic type` 与 `ros2 interface show` 输出其完整字段，并解释每个字段的含义。
10. 远程操作：分别用 SSH 与 VNC 完成一次远程任务，记录两种方式在延迟、可读性和操作便利性上的差异。
11. 多机通信：在两台机器上设置相同的域号，验证节点可见性；再改成不同域号，记录观察结果。
12. 集成设计：为第 12 节的集成路径写一份设计说明，明确节点职责、话题接口、参数与验收门槛。该设计不要求运行，但必须标注为未验证方案。

5.14.2. 验收标准

项目	验收信号	证据形式	未通过时的处理
工作空间	`colcon build` 成功，`source install/setup.bash` 后功能包可被检索	终端输出记录	检查目录结构与依赖声明
话题通信	订阅方收到发布方消息，主题名与类型一致	两个终端的输出	检查话题名、消息类型与域号
服务调用	服务调用返回结果，字段符合接口定义	调用命令与返回值	检查接口类型与请求字段
参数操作	参数修改生效并可保存恢复	参数文件与前后取值	检查参数名与节点是否支持运行期修改
bag 记录	录制文件可查看、可回放	`ros2 bag info` 输出	检查录制话题与磁盘空间
launch 文件	一条命令启动多个节点，参数按预期生效	启动日志与参数列表	检查 action 组织与参数传递路径
坐标变换	`tf2_echo` 输出与设计参数一致，坐标树完整	命令输出与坐标树报告	检查父子坐标系与参数顺序
底盘接口	节点信息中的话题与文档一致，速度指令可被安全停止	`ros2 node info` 输出	检查串口别名、权限与协议版本
远程操作	SSH 与 VNC 均能完成各自适用任务	操作记录与说明	检查网络、认证与服务配置
多机通信	同域可见、异域不可见，结论可重复	两次对照记录	检查域号、网段与本地限制

项目	验收信号	证据形式	未通过时的处理
集成设计	设计标注为未验证，字段与门槛完整	设计文档	补齐接口定义或证据边界

验收时只记录实际观察到的结果。命令没有执行就不要填写为通过，输出没有留存就不要写成结论。ROS2 相关的板端集成结论必须同时说明运行平台、模型版本、接口类型与日志路径，缺一项都只能作为待验证记录。

附录 6. 工具箱、FAQ 与参数模板

6.1. 内容概览

本附录聚焦“查得快、用得稳”：常见报错速查、转换参数模板、性能/质量 Checklist、术语字典、推荐资源与社区贡献规范。可作为日常开发随手翻阅的工具章节。

6.2. 常见报错速查

分类	报错/现象	可能原因	排查步骤	解决建议
ATC	`E19001: Op Not Supported`	新算子/版本落后	确认 CANN 版本 + onnxsim 简化	升级/替换结构/自定义算子
ATC	Shape 推断失败	动态维度不明确	检查 `--input_shape`/动态参数	固定关键维度或提供范围
ACL	`aclmdlLoadFromFile failed`	权限/路径/模型损坏	校验文件 hash/权限	修正权限/重新生成 OM
Runtime	OOM / alloc 失败	Batch 或分辨率过大	统计输入分布	降 batch/分桶/复用内存
运行	推理输出 NAN	数值溢出/量化尺度错误	Dump 中间 Tensor	调整量化/保留 FP32 层
性能	Timeline 大量 gap	Host 阻塞/小算子	Profiling 分析	合并算子/异步预取
性能	H2D 高占比 >25%	多次小拷贝	合并缓冲	AIPP 下沉/批量化
精度	Top1 下降 >1%	预处理不匹配	对比 ONNX 输出	统一 Normalize & Layout
精度	mAP 不稳定	阈值或 NMS 误差	调整阈值/比对中间框	校准 NMS 公式/尺度
稳定	间歇 Crash	悬空指针/并发访问	启用 ASAN/日志回溯	修订生命周期/加锁
部署	模型加载慢	冷启动/IO 慢	预热/缓存	预加载 + 固态存储
安全	日志泄露敏感路径	直接 print	grep 审计	结构化日志脱敏

6.3. 模型转换参数模板合集

6.3.1. 分类模型 (ResNet)

```
atc --model=resnet50.onnx \
    --framework=5 \
    --output=resnet50_fp16 \
    --input_format=NCHW \
    --input_shape="input:1,3,224,224" \
    --soc_version=Ascend310B \
    --precision_mode=allow_fp32_to_fp16 \
    --log=info
```

6.3.2. YOLO 动态分辨率

```
atc --model=yolov5s.onnx \
    --framework=5 \
    --output=yolov5s_640_768 \
    --dynamic_image_size="640,640;768,768" \
    --input_format=NCHW \
    --soc_version=Ascend310B \
    --op_select_implmode=high_performance \
    --precision_mode=allow_fp32_to_fp16
```

6.3.3. INT8 量化（示例）

```
atc --model=resnet50.onnx \
    --framework=5 \
    --output=resnet50_int8 \
    --input_format=NCHW \
    --input_shape="input:1,3,224,224" \
    --soc_version=Ascend310B \
    --precision_mode=allow_mix_precision \
    --insert_op_conf=aipp.cfg \
    --enable_small_channel=true
```

6.4. 性能与质量 Checklist（执行勾项）

性能：

□ Profiling 无明显 Idle gap > 10%

□ H2D + D2H 占比 < 25%

□ Postprocess 占比 < 20%

□ Stream 利用率平衡（无单流饱和）

□ 使用内存池减少频繁 alloc/free

精度：

□ ONNX vs OM Top1 差异 < 0.2%

□ L1 平均误差 < 1e-3（FP16）

□ NMS 输出框数量与基线差异 < 1 框/图（平均）

□ INT8 校准集覆盖多场景

稳定性：

□ 1h 稳态无 Crash / OOM

□ 温度在安全区间 < 85°C

□ 看门狗重启次数 = 0

安全：

□ 日志无明文密钥

□ 模型文件 hash 校验通过

6.5. 术语表（扩展）

术语	说明
OM	Ascend 离线模型二进制格式
ACL	Ascend 计算语言 API 层
ATC	模型转换/编译工具
AIPP	自动图像预处理模块
Stream	异步任务调度通道
Profiling	性能采样分析工具体系
Fallback	算子未匹配优化实现退回通用实现
Quant Calibration	量化尺度统计过程
Baseline	初始标准对照性能/精度集
Drift	指标随时间未经预期的漂移

6.6. 推荐资源与外部引用

- Ascend 官方文档入口（安装/算子列表/最佳实践）
- CANN Release Notes：版本兼容与已知问题。
- ONNX Operator 列表与语义说明。
- Open Model Zoo / ModelScope：获取预训练模型与许可信息。
- 学术资源：算子融合、低比特量化、蒸馏相关论文列表。

6.7. 贡献指南摘要

流程：Fork -> 新分支 -> 修改/新增 -> 本地 lint & 生成脚本 -> PR（描述动机/影响面/验证方式）。PR 要求：| 要素 | 说明 | | —- | —- | | 标题 | 简明说明改动作用 | | 描述 | 背景 + 修改点 + 风险 | | 验证 | 性能/精度/功能截图或数据 | | 回滚 | 若失败如何恢复 | | 关联 Issue | 追踪链接 |

6.8. FAQ

问题	回答
模型转换慢怎么办?	使用 SSD，关闭调试日志，检查不必要动态 shape。
精度下降如何定位?	离线脚本层级 Dump 比对，逐层二分。
如何减少内存占用?	启用内存池 + 减少中间冗余张量 + 固定 batch。
量化后收益不明显?	检查是否 Compute-bound，或激活分布集中导致尺度相近。
NMS 很慢?	合并小框批量处理/降低候选阈值/考虑 Device 版 NMS。

6.9. License 与引用

本书内容遵循 Apache 2.0 许可证。引用：> 《昇腾 310B 实战：从入门到精通边缘计算与人工智能》（GitHub: zhouxzh/Ascend310）

6.10. 版本路线回顾

版本	内容	目标
v0.1	结构框架	验证框架可行
v0.3	核心部署链路	形成可用主线
v0.6	案例与工程化	贴近实战
v1.0	全面审校发行	正式发布

6.11. 实践任务

1. 为你的项目添加 1 条本地常见错误记录（含根因与解决）。
2. 复制分类 ATC 模板并改写为检测模型版本（含动态尺寸）。
3. 在术语表补充 3 个任务相关术语（并验证唯一性）。
4. 选取 FAQ 一条，写出更深入排查脚本思路。

附录 7. CANN 升级与版本配套

7.1. 本附录解决的问题

本附录按教程顺序组织两条独立升级路线：

- **路线 A：香橙派 AI Pro 8T**：`20241128`、Ascend 310B4 / 8T、驱动 `23.0.0`、B309，CANN 7.0 升级到 CANN 8.0；驱动和固件保持不变。
- **路线 B：华为官方 Atlas 310B**：原始 CANN 7.0、驱动/HDK 23.0，升级到固件 `7.8.0.5.216`、驱动/HDK 25.5.0 和 CANN `9.2.0-beta.2`；升级后必须恢复 HDMI/DRM。

两条路线共用版本采集、安装包校验、回滚材料和验收原则，但硬件、内核、驱动、固件、CANN 包及测试结果不可混用。Atlas 的 HDMI/DRM 未恢复时，升级不算完成。

昇腾软件环境经常被简单地称为 “CANN 版本”。这种说法对于初次安装可以帮助理解，但不足以支撑一次可复现的升级。一个能够运行的 310B 环境至少包含操作系统与内核、NPU 驱动、NPU 固件、CANN 用户态软件、Python 运行时以及可能存在的框架适配层。它们由不同安装包提供，也有不同的生命周期。只替换其中一个组件，可能使 `npu-smi` 仍能显示设备，却让 ATC、PyACL 或已有 OM 模型在运行时失败。

本附录以昇腾官方版本配套查询助手的 310B 系列查询结果为当前版本配套参考，并以 CANN 8.0、CANN 9.0.X 和 CANN 9.2 发布线的配套说明补充升级信息，建立一套面向昇腾 310B 的版本审查和升级方法。文中沿用本书的证据状态：`documented` 表示厂商版本说明或安装指南已明确给出；`inferred` 表示由配套关系推得、尚未在目标板验证；`observed-pass` 和 `observed-fail` 分别表示在记录的硬件、版本和输入条件下实测通过或失败；`untested` 表示尚未执行。命令示例属于项目操作建议，不等于厂商对所有设备的兼容性承诺。

本附录不是 “看到新版本就升级” 的建议。若现有案例已经通过了模型转换、ACL 烟测和长稳测试，升级应被视为一次新的变更实验；只有在回归证据完整后，才可以把新环境写入案例的默认配置。

7.2. 教程路线与已验证组合

路线	已验证基线	目标组合	必须通过的结果
A：香橙派 AI Pro 8T	Ubuntu 22.04.3，`20241128`，310B4，驱动 23.0，B309，CANN 7.0	CANN 8.0，驱动和固件不变	ACL、最小 VENC、OM、CANN VENC、case8 WebRTC、回归
B：华为官方 Atlas 310B	Ubuntu 22.04.5，310B4，CANN 7.0，驱动/HDK 23.0	固件 `7.8.0.5.216`、驱动/HDK 25.5.0、CANN 9.2.0-beta.2	HDMI/DRM、ACL、OM、摄像头、CANN VENC、WebRTC、回归

先选择路线，再执行后文的共用准备、专项步骤和统一验收。不要把香橙派的 B309/5.10.0+ 条件与 Atlas 的 7.8.0.5.216/6.6 内核条件拼接。

7.3. 软件栈中的版本层次

7.3.1. 驱动、固件、CANN 和框架适配层

层次	主要职责	常见查询或目录	升级时的判断
操作系统与内核	提供用户态、内核模块、设备节点和系统调用	系统信息与发行版文件	内核、发行版、架构必须在硬件和驱动支持范围内
NPU 驱动	让操作系统和用户进程访问 NPU，管理设备、内存和运行时接口	`npu-smi` 与驱动目录	驱动包必须匹配目标 SoC、内核和固件；它不是 CANN Toolkit
NPU 固件	运行在设备侧的底层固件，负责设备启动及硬件协同	固件目录与厂商升级工具	覆盖升级时通常先固件后驱动；升级过程中不能断电或复位
CANN	用户态编译器、Runtime、AscendCL、PyACL、算子和工具链	`set_env.sh`、`atc` 与 Toolkit 目录	必须满足 CANN 与 Ascend HDK 的配套关系
框架适配层	例如 `torch_npu`、MindSpore 插件和第三方库	Python 导入检查	需单独核对框架、Python、CANN 和驱动的兼容矩阵

这里的 “固件版本” 和 “驱动版本” 不是同义词。固件主要在设备侧运行，驱动主要在主机操作系统中运行；二者常由同一套 Ascend HDK 下载页面提供，但安装包、升级命令和生效方式不同。版本说明中的 **Ascend HDK** 是厂商用于表达硬件开发套件配套关系的版本标识，不能把它直接当作板上某一个可执行文件的版本号。实际安装时仍应以目标硬件页面列出的驱动包和固件包全名为准。

CANN 也不是一个单一二进制。CANN 9.0.X 的版本说明把组合包分为 Toolkit、算子包（ops）和加速库（NNAL）；Toolkit 中还包含 Runtime、AscendCL 扩展、编译器和 PyACL 等子包。因而 “升级 CANN” 可能指升级完整 Toolkit，也可能只升级一个允许独

立升级的子包。两种操作的风险和验收范围不同。后续命令块给出默认目录的检查方式；表中使用目录类别，是为了避免把默认路径误读为唯一安装路径。

7.3.2. 310B 的产品边界

本仓库的主要目标板是 Ascend 310B4 / 8T。CANN 9.0.X 版本说明中出现的 Ascend 950PR、Atlas A2 或 Atlas A3 特性，首先证明这些特性在相应产品族上被发布；它们不自动证明 310B 支持。对 310B 的结论必须同时满足：

1. 目标硬件的配套页面列出该驱动、固件和 CANN 组合；
2. 当前操作系统、架构和 Python 环境满足安装要求；
3. 代表性模型或 API 在板端通过了运行时和任务级验证。

如果只满足前两项，结论应写成 “配套关系已记录，板端功能未验证”，而不是 “已支持”。

7.4. 官方配套查询的使用方法

版本配套查询助手用于升级前按目标板卡、操作系统、CPU 架构和产品筛选实际安装包。查询结果是版本选择依据，不是板端验收结果；最终记录仍需保存实际驱动、固件、Toolkit、ops/NNAL 包名、摘要和对应测试结果。PyTorch 等查询项也不能直接等同于本地 `torch` 或 `torch_npu` 已安装。

7.5. 历史版本线与 Ascend HDK 配套关系

7.5.1. CANN 与 Ascend HDK

下面的关系用于理解 CANN 8.0、CANN 8.5.X、CANN 9.0.X 和 CANN 9.2.X 发布线与 Ascend HDK 的配套边界。它们是版本选择和升级前核对的参考，不替代下载页面针对具体 310B 产品、操作系统和架构的筛选结果。

CANN	版本说明中列出的 Ascend HDK
CANN 8.0	以对应 8.0 发布包和目标产品下载页列出的 HDK 为准；本项目香橙派路线固定使用原有驱动/固件组合，不在该路线中升级 HDK
CANN 8.5.X	26.0.RC1、25.5.X、25.3.X、25.2.X、25.0.X 等版本线，具体以对应小版本说明为准
CANN 9.0.X	25.7.RC1、26.0.RC1/RC3、25.5.X、25.3.X、25.2.X、25.0.X 等版本线，具体以对应小版本说明为准

CANN	版本说明中列出的 Ascend HDK
CANN 9.2.X	本项目 Atlas 路线实测使用 HDK 25.5.0、固件 `7.8.0.5.216` 和驱动 `25.5.0`；其他 9.2.X 发布包仍需按目标产品下载页重新核对

表中的 HDK 是版本说明使用的配套标识，不是可以替代驱动包或固件包名称的独立版本号。CANN 8.0 的具体配套必须以实际 8.0 发布包和目标产品下载页为准；CANN 8.5.X 和 CANN 9.0.X 的小版本差异不能被忽略；CANN 9.2.X 在本项目中已经以 HDK 25.5.0/固件 `7.8.0.5.216`/驱动 `25.5.0` 的 Atlas 组合完成实测。正式记录仍需保存包名、实际版本、摘要和板端验收结果。

7.5.2. 组合包与独立子包

CANN 9.0.X 的组合包可按下表理解；CANN 8.0 和 CANN 9.2 也应遵循相同的 Toolkit、ops、NNAL 分层原则：

组合包	代表内容	独立升级边界
Toolkit	Runtime、AscendCL 扩展、编译器、工具、PyACL 等	先确认基础 Toolkit 已安装；子包路径必须与组合包一致
ops	DVPP、通用算子、数学算子、Transformer 算子等	版本说明列出的若干 ops 子包支持独立升级
NNAL	`atb`、`sip` 等加速库	仅在业务确实使用时纳入变更范围

同一发布线内的 ops 与 Toolkit 通常具有更明确的配套关系；跨发布线组合必须以对应版本说明和下载页为准。该表不能替代完整兼容性检查：框架插件、模型算子、驱动固件和操作系统仍需单独验证。对于本仓库的案例，优先采用同一发布线的 Toolkit 与 ops，只有在有明确需求和回归报告时才使用跨版本组合。

7.5.3. 版本特性与 310B 结论的关系

CANN 9.0.X 说明包含低比特量化、确定性计算、通信合并、SIMD/SIMT、Runtime 查询接口等变化，也列出了若干废弃接口和已知问题。这些条目适合用来制定迁移检查表，但不能直接写成 “本项目已获得性能提升”。本仓库不把发布说明中的理论加速倍数、算子覆盖或修复列表当作案例测量结果。

升级前至少搜索代码和配置中是否使用了下列废弃方向：

- `aclnnGroupedMatmul` 的旧版本接口，迁移目标为版本说明列出的 `aclnnGroupedMatmulV5`；
- `aclnnPromptFlashAttention`、`aclnnIncreFlashAttention` 等旧版本接口；

- 模型压缩工具中被标记为废弃的非均匀量化、自动混合精度、近似校准、INT4 量化感知训练和 `amct_mindspore` 特性。

这只是静态迁移筛查。每一项都还要在目标板上完成编译、加载、数值一致性和任务级回归。

7.6. 升级前的准备与门禁

7.6.1. 固定目标与维护窗口

升级前写下以下信息，并把它们与本次报告绑定：

项目	需要记录的内容
设备	产品型号、SoC、算力等级、序列号或经脱敏的设备标识
系统	发行版、版本、内核、`aarch64` 或 `x86_64`
软件	驱动、固件、CANN Toolkit、ops/NNAL、Python、框架插件
业务	正在运行的服务、端口、模型和数据目录
证据	当前健康检查、代表性 ACL 烟测、性能基线和回滚包位置

升级会造成业务中断。应先停止案例服务和自动启动单元，通知使用者，确保串口或本地控制台可用，并确认设备有稳定供电。安装或升级过程中不要对主机或设备执行断电、复位、热插拔或并行维护操作。

7.6.2. 采集当前版本

以下命令只读，不会修改系统；应在板端目标环境中执行：

```
uname -a
uname -m
cat /etc/os-release
hostname
date --iso-8601=seconds
npu-smi info
command -v atc || true
python --version
python -c 'import sys; print(sys.executable)'
python -c 'import acl; print("PyACL import: ok")'
readlink -f /usr/local/Ascend/ascend-toolkit/latest 2>/dev/null || true
readlink -f /usr/local/Ascend/driver 2>/dev/null || true
readlink -f /usr/local/Ascend/firmware 2>/dev/null || true
df -h /usr/local/Ascend
```

`npu-smi info` 能显示设备并不等于 CANN、PyACL 和已有模型全部可用；它只构成设备层检查。`import acl`、`atc --version`、模型加载和任务输出应分别记录。若 `import acl` 来自用户目录或另一个 Conda 环境，必须把解释器路径和 `sys.path` 一并保存，避免把环境混用误判为版本问题。

7.6.3. 保存可回退的材料

至少保存：当前版本输出、安装包文件名和摘要、CANN 安装配置、服务启动参数、模型清单、环境变量、测试输入、原始日志和回滚顺序。模型、用户数据和个人信息不要复制到公开报告。

```
report_root="$HOME/upgrade-reports/$(date -u +%Y%m%dT%H%M%SZ)"
mkdir -p "$report_root"
npu-smi info > "$report_root/npu-smi-before.txt"
uname -a > "$report_root/uname-before.txt"
python -m pip list > "$report_root/pip-before.txt"
env | sort | sed -E 's/(TOKEN|KEY|PASSWORD|SECRET)=.*/\1=[redacted]/I' \
  > "$report_root/env-before.txt"
```

如果需要保存 `/usr/local/Ascend` 下的配置，只复制经审查的文本配置，不要把整个安装目录打包进 Git。报告目录应位于隔离位置，并明确其保留期限。

7.6.4. 检查安装权限、空间和架构

驱动和固件通常需要 root 权限；CANN `.run` 包可以按安装指南由安装用户执行。安装用户、运行用户和驱动固件用户组的关系必须符合目标版本指南。升级前检查：

```
id
groups
df -h /usr/local/Ascend /tmp
du -sh /usr/local/Ascend/ascend-toolkit \
       /usr/local/Ascend/driver \
       /usr/local/Ascend/firmware 2>/dev/null || true
mount | sort
```

空间要求以当前下载页面和安装指南为准，并为解包、日志和回滚包预留余量。

7.6.5. 核对安装包而不是文件名

下载页同时提供不同产品和架构的包；文件名相似不代表可以互换。先记录所选安装包的本地摘要，再以厂商发布的对应摘要清单和签名材料完成验证：

```
package_dir="$HOME/packages/cann-upgrade"; cd "$package_dir" || exit 1
sha256sum ./selected-*.run > package-sha256.txt || exit 1
```

将 `selected-*.run` 替换为实际选择的文件名；`.deb` 或 `.rpm` 包应按其包管理器和版本指南核验。若厂商提供的 `SHA256SUMS` 包含这些精确文件，应提取对应条目形成专用校验清单，再执行 `sha256sum -c hashlist`；摘要不匹配或文件缺失即停止。签名校验必须使用该版本发布说明指定的材料和工具，不能把“下载页提供签名”写成“签名已验证”。必要时再用 `file -- *.run` 核对安装包类型。不要使用未核验的镜像链接、旧板卡包或其他 SoC 的包。若下载页没有明确列出目标 310B 的配套关系，状态应记为 `unknown`，先做小范围兼容性核对，不得直接覆盖生产环境。

7.7. 共用安装与升级操作

7.7.1. 首次安装：驱动后固件

首次安装是指设备尚未安装驱动和固件，或两者都已卸载。官方安装指南的顺序是先驱动、后固件；安装完成后按提示重启，再安装 CANN。先将下载页筛选出的真实文件名写入变量，示例不预设 310B 的固定包名：

```
cd "$package_dir" || exit 1
driver_package="./selected-driver.run"; firmware_package="./selected-firmware.run"
chmod +x "$driver_package" "$firmware_package" || exit 1
sudo "$driver_package" --full --install-for-all || exit 1
sudo "$firmware_package" --full || exit 1
sudo reboot
```

安装方式、`--full` 参数和 `--install-for-all` 是否适用取决于包类型和版本指南；不要把上面的示例当作任意设备的固定命令。每一步均在失败时退出；只有驱动和固件安装都成功后才重启。重启后先执行 `npu-smi info`，确认设备层正常，再安装或加载 CANN。

7.7.2. 覆盖升级：固件、驱动、CANN

覆盖升级是指现有驱动和固件仍在系统中，准备换到新的配套版本。官方升级指南要求同时升级时遵循：

先升级固件，再升级驱动，最后升级 CANN 软件。

命令结构如下：

```
cd "$package_dir" || exit 1
driver_package="./selected-driver.run"; firmware_package="./selected-firmware.run"
chmod +x "$firmware_package" "$driver_package" || exit 1
"$firmware_package" --check || exit 1
"$driver_package" --check || exit 1
sudo "$firmware_package" --upgrade || exit 1
sudo "$driver_package" --upgrade || exit 1
sudo reboot
```

如果安装路径不是默认路径，必须把路径参数替换为当前安装路径。升级过程中不要同时启动服务、运行 ATC 或访问摄像头。升级成功消息只是安装器层面的结果，重启后还要完成设备、运行时和任务级验收。

7.7.3. 升级 CANN Toolkit

驱动和固件已经在配套状态时，可以单独升级 CANN。对 `.run` 包，常见的检查顺序是“增加执行权限—校验安装包—升级”：

```
cd "$package_dir" || exit 1
toolkit_package="./selected-toolkit.run"
chmod +x "$toolkit_package" || exit 1
"$toolkit_package" --check || exit 1
"$toolkit_package" --upgrade || exit 1
```

升级命令通常从安装配置中读取原有路径；如果使用了自定义安装路径，应按安装指南显式指定。升级后检查 `latest` 软链接指向的版本，并在同一 shell 加载环境：

```
source /usr/local/Ascend/ascend-toolkit/set_env.sh
readlink -f /usr/local/Ascend/ascend-toolkit/latest
command -v atc
atc --version
python -c 'import acl; print("PyACL import: ok")'
```

7.7.4. 子包独立升级

CANN 9.0.X 说明允许若干子包独立升级。独立升级不是“随便替换一个 `.so`”：

1. 组合包 `Ascend-cann-toolkit` 和 `Ascend-cann-ops` 必须已经安装；
2. 子包版本必须在当前 CANN 版本说明的配套表中；
3. 子包的安装路径必须与组合包一致；
4. 升级后要重新运行使用该子包的案例和模型回归。

以 `cann-hixl` 为例，变量中应填写下载页提供的实际文件名，例如带有版本号和架构标识的 `.run` 包。默认路径时省略 `--install-path`；自定义路径时使用下例：

```
hixl_package="./selected-cann-hixl.run"
chmod +x "$hixl_package" || exit 1
"$hixl_package" --upgrade --install-path="/home/custom_path" || exit 1
```

上例中的包名和版本号只是命令形状。执行前必须先查看该包的 `--help` 或对应安装指南；如果安装器提供独立的校验选项，应先完成校验，失败时停止，不要用 `--upgrade` 强行覆盖。

7.8. 板卡专项教程

7.8.1. 路线 A：香橙派 AI Pro 8T

本路线使用前文固定的 20241128、驱动 23.0、B309、CANN 7.0 基线。先完成 “升级前的准备与门禁”，然后只升级 CANN 用户态软件：

1. 使用下载中心取得的同一发布号 Toolkit 和 kernels-310b 包，完成 SHA-256 与 --check。
2. 对已安装的 CANN 7.0 Toolkit 使用 --upgrade。
3. 加载 /usr/local/Ascend/ascend-toolkit/set_env.sh 后安装目标 Kernels；若板端登记方式不同，以该发布包安装指南为准。
4. 不刷写驱动或固件，不修改内核、IOMMU 和启动参数，不覆盖旧 OM。
5. 执行统一验收中的门 2 至门 5，并额外完成 case8 的最小 ACL/VENC、OM、摄像头、CANN VENC 和 WebRTC 流程。

本项目已实际验证 20241128 + CANN 7.0 -> CANN 8.0 的 ACL、CANN VENC 和 case8 WebRTC 结果。该结论不外推到 20250925 系统或 Atlas 板。

7.8.2. 路线 B：华为官方 Atlas 310B

本路线的升级前基线是 CANN 7.0、驱动/HDK 23.0；目标是固件 7.8.0.5.216、驱动/HDK 25.5.0、CANN 9.2.0-beta.2。执行顺序固定为：

1. 按共用准备保存 CANN 7.0、驱动/HDK 23.0 的版本、配置、模型和日志。
2. 按 “覆盖升级” 先升级固件，再升级驱动，重启后执行设备门检查。
3. 按 “升级 CANN Toolkit” 配置 CANN 9.2.0-beta.2，执行 PyACL/ATC 检查。
4. 完成本节后面的 HDMI/DRM 恢复；HDMI 未恢复时不得进入最终业务验收。
5. HDMI 通过后执行统一验收门 2 至门 5，以及 Atlas 摄像头、OM、CANN VENC 和 WebRTC 回归。

本路线的升级结果必须记录为完整组合，不得只写 “CANN 9.2.0 环境”。

Atlas HDMI/DRM 必做恢复

本节针对 Atlas 200I A2 / Ascend 310B4、Ubuntu 22.04.5、内核 6.6.0-72.0.0.76.h914.eulerosv2r15.ascend.aarch64、驱动 25.5.0。先固定启动介质、DTB 和当前显示状态：

```
uname -a
findmnt -no SOURCE /
cat /proc/cmdline
```

```
npu-smi info
ls -l /dev/dri 2>&1 || true
lsmod | grep -E 'ascend_vdp_drm|drm|drv_osal|ascend_qos' || true
dmesg -T | grep -Ei 'drm|hdmi|vdp|hifb' | tail -n 200
```

本次板标识为 M100-B51，根分区为 /dev/mmcblk1p2。将 DTB 显示包含从 hi1910B-vdp.dtsi 切换到 hi1910B-drm.dtsi，并保留：

```
console=tty1 cma=256M rootfstype=ext4 root=/dev/mmcblk1p2
```

DTB 写入 SD 卡实际启动位置并重启。使用驱动 25.5 SDK 的精确 KERNEL_OBJ、内核配置和 Module.symvers 构建 drm_dma_helper.ko；构建源文件为 drm_gem_dma_helper.c、drm_fbdev_dma
↪ .c、drm_fb_dma_helper.c。通用 Linux 6.6 构建会因厂商 ABI 不匹配失败，不能替代 SDK 构建。目标 ABI 为：

```
6.6.0-72.0.0.76.h914.eulerosv2r15.ascend.aarch64 SMP mod_unload modversions aarch64
```

drv_osal.ko 和 ascend_qos.ko 由 start-davinci.service 加载，显示模块必须排在该服务之后。建立 /etc/systemd/system/ascend-hdmi-drm.service，保留 drm、drm_kms_helper、drm_dma_helper
↪ 、ascend_vdp_drm 和元数据补丁的加载顺序。其关键单元内容为：

```
[Unit]
Description=Load Ascend HDMI DRM stack
Wants=systemd-modules-load.service
Requires=start-davinci.service
After=systemd-modules-load.service local-fs.target start-davinci.service
Before=display-manager.service
ConditionPathExists=/var/davinci/driver/ascend_vdp_drm.ko

[Service]
Type=oneshot
ExecStart=/usr/sbin/modprobe drm
ExecStart=/usr/sbin/modprobe drm_kms_helper
ExecStart=/usr/sbin/modprobe drm_dma_helper
ExecStart=/bin/sh -ec 'grep -qw ascend_vdp_drm /proc/modules || /usr/sbin/insmod /var/
    ↪ davinci/driver/ascend_vdp_drm.ko'
ExecStart=/bin/sh -ec 'grep -qw ascend_drm_metadata_fix /proc/modules || /usr/sbin/insmod
    ↪ /lib/modules/6.6.0-72.0.0.76.h914.eulerosv2r15.ascend.aarch64/updates/ascend-hdmi/
    ↪ ascend_drm_metadata_fix.ko target_data=$(cat /sys/module/ascend_vdp_drm/sections/.
    ↪ data)'
RemainAfterExit=yes

[Install]
WantedBy=multi-user.target
```

本次 DRM_IOCTL_VERSION 的 name/date/description 为空，导致 libdrm 的 drmGetVersion ↪ () 在 Xorg 初始化期间中止。必须使用同一 SDK 构建 ascend_drm_metadata_fix.ko。完整源码如下：

```
// SPDX-License-Identifier: GPL-2.0-only
#include <linux/errno.h>
#include <linux/init.h>
#include <linux/kernel.h>
#include <linux/module.h>
#include <linux/moduleparam.h>
#include <linux/types.h>
#include <drm/drm_drv.h>

#define ASCEND_VDP_DRM_DRIVER_OFFSET 0xd0UL

static unsigned long target_data;
module_param(target_data, ulong, 0400);

static char driver_name[] = "ascend_vdp_drm";
static char driver_desc[] = "atlas-200a2 DRM driver";
static char driver_date[] = "20261007";

static int __init ascend_drm_metadata_fix_init(void)
{
    struct drm_driver *driver;

    if (!target_data)
        return -EINVAL;

    driver = (struct drm_driver *)(target_data + ASCEND_VDP_DRM_DRIVER_OFFSET);
    if (!driver->fops || driver->name || driver->desc || driver->date)
        return -EINVAL;

    driver->major = 1;
    driver->minor = 0;
    driver->patchlevel = 0;
    driver->name = driver_name;
    driver->desc = driver_desc;
    driver->date = driver_date;
    pr_info("ascend_drm_metadata_fix: populated ascend_vdp_drm DRM metadata\\n");
    return 0;
}

module_init(ascend_drm_metadata_fix_init);

MODULE_LICENSE("GPL");
MODULE_DESCRIPTION("Populate omitted Ascend VDP DRM driver metadata");
```

ASCEND_VDP_DRM_DRIVER_OFFSET=0xd0 是本次 ascend_vdp_drm.ko 中本地 ext_drm_driver 符号相对于模块 .data 段的偏移量。驱动、内核、编译器或结构布局改变后必须重新分析，不

能复制到其他版本。模块中的 driver->fops 和空字段检查只用于阻止明显错误的重复补丁，不构成跨版本兼容性保证。

本次构建产物及验证输出为：

```
/lib/modules/6.6.0-72.0.0.76.h914.eulerosv2r15.ascend.aarch64/updates/ascend-hdmi/drm_dma_helper.ko
SHA-256: e0847ac4c53b471a8d28bee372fef44547217e8a94ce2c89471e7f349359fa0b

/lib/modules/6.6.0-72.0.0.76.h914.eulerosv2r15.ascend.aarch64/updates/ascend-hdmi/ascend_drm_metadata_fix.ko
SHA-256: ce8f96929d7ad5d198d1019d2c7fc0da4b47ea40e83186f130ca9c3e833106ed

DRM_IOCTL_VERSION: 1.0.0 name=ascend_vdp_drm date=20261007 desc=atlas-200a2 DRM driver
```

恢复原系统的 SDDM + LXQt + xfwm4 图形方案。HDMI/DRM 修复不需要额外安装其他桌面环境；显示管理器只需要在 DRM 设备初始化之后启动：

```
# 停用当前显示管理器；原系统保留的 SDDM/LXQt 不需要重新安装。
current_dm="$(basename "$(readlink -f /etc/systemd/system/display-manager.service 2>/dev/null || true)")"
if [ -n "$current_dm" ]; then sudo systemctl disable --now "$current_dm" || true; fi
sudo systemctl disable --now lightdm.service || true
sudo systemctl enable sddm.service
# SDDM 的启动前检查要求这个文件指向 /usr/bin/sddm。
printf '%s\n' /usr/bin/sddm | sudo tee /etc/X11/default-display-manager
sudo systemctl set-default graphical.target
sudo systemctl daemon-reload
sudo systemctl enable ascend-hdmi-drm.service
sudo systemctl restart sddm.service
```

如果 sddm.service 因 ExecStartPre 失败，先检查 /etc/X11/default-display-manager 是否仍指向旧的显示管理器路径，修正后执行：

```
sudo systemctl reset-failed sddm.service
sudo systemctl start sddm.service
```

/var/lib/sddm/state.conf 应保留 Session=/usr/share/xsessions/lxqt.desktop，登录 HwHiAiUser 后进入 LXQt。若换系统时使用了不含原桌面包的最小镜像，只恢复原镜像对应的 SDDM/LXQt 包即可，不要为了验证 HDMI 额外引入新的桌面环境。

验收必须同时包括 /dev/dri/card0、三个显示模块、HDMI-A-1 的 connected/enabled/DPMS、EDID、KMS active CRTC、SDDM 登录界面和 npu-smi。登录后再确认会话为 LXQt；最后在显示器前确认实际画面。HDMI 通过后才能继续 Atlas 的 PyACL、OM、摄像头、CANN VENC、WebRTC、回归和性能测试。

7.9. 升级后的统一验收顺序

升级验收分成五个门，不应把一个门的通过写成全链路通过。

7.9.1. 门 1：设备和驱动

```
npu-smi info
lsmod | grep -E 'drv|ascend|hisi' || true
ls -l /dev/davinci* /dev/devmm* 2>/dev/null || true
dmesg -T | tail -n 120
```

记录设备数量、型号、驱动显示的版本和异常日志。`Health: Alarm` 只能作为诊断背景；是否阻断升级要由设备缺失、驱动加载失败、复位循环或可复现的任务错误决定。

对于支持该工具的 310B 驱动包，可进一步读取设备侧系统版本。命令的实际路径和参数应以当前驱动版本的升级指南为准：

```
upgrade_tool="/usr/local/Ascend/driver/tools/upgrade-tool"
if [ -x "$upgrade_tool" ]; then
  "$upgrade_tool" --device_index -1 --system_version
else
  echo "upgrade-tool is not available; record the version from the board's release files"
fi
```

`npu-smi` 和 `upgrade-tool` 的输出分别属于设备状态查询和版本查询；二者都不能替代 CANN、框架插件和代表性模型的验证。

7.9.2. 门 2：CANN 和 PyACL

```
source /usr/local/Ascend/ascend-toolkit/set_env.sh
command -v atc
atc --version
python -c 'import acl, sys; print(sys.executable); print("acl: ok")'
python -c 'import numpy; print(numpy.__version__)'
```

如果只在当前交互 shell 中成功，不能说明 systemd、Conda 或案例启动脚本也能成功。应使用与服务完全相同的启动入口再次执行。

7.9.3. 门 3：代表性模型

选择每个业务最小但有代表性的模型，记录输入合同、模型摘要、SoC、精度模式和输出。对 ONNX 到 OM 的案例，至少执行一次模型检查和一次 ACL 加载；对已经固定的 OM，不要因为 `npu-smi` 正常就跳过加载。

```
model_path="models/accepted-model.om"
test -r "$model_path"
stat -c '%s %n' "$model_path"
sha256sum "$model_path"
```

模型加载通过仍然只是数值烟测。图像识别、音频合成、时频分析和 RAG 生成必须分别使用各自的输入协议与验收脚本。

7.9.4. 门 4：服务和前端

```
ss -ltnp
systemctl --type=service --state=running
service_port=7860  # 按案例实际服务端口修改
curl --fail "http://127.0.0.1:${service_port}/health"
```

把 `service_port` 替换为实际服务端口；对没有 `/health` 的案例，应使用其 README 中规定的根页面或 API 路由。服务返回 HTTP 200 不能代替 NPU 推理和任务级结果。

7.9.5. 门 5：回归与性能

升级后重新运行与旧版本相同的输入、预热次数、循环次数、线程数和输出检查。只有当结果与旧版本在预先定义的容差内一致，且没有新的资源泄漏、崩溃、超时、溢出或延迟回归，才可以更新案例报告。未经重新测量的旧数字不得标为新版本结果。

7.10. 已知变化与迁移检查

7.10.1. 废弃接口

CANN 9.0.X 说明把若干 Transformer 相关接口标记为废弃，并给出替代接口；说明中还指出部分特性将在后续版本移除。升级项目应先用静态搜索找出旧接口，再按实际调用路径迁移。不要因为编译仍然通过就认为废弃接口可以长期保留。

```
rg -n "aclnn(GroupedMat[Mm]ul|PromptFlashAttention|IncreFlashAttention|
    ↪ FusedInferAttentionScore)" \
  src samples
```

搜索结果需要由熟悉该模型和算子合同的人审阅。若接口来自第三方二进制或生成代码，必须在运行时日志和版本清单中标记为待验证。

7.10.2. 版本说明中的已知问题

CANN 9.0.X 版本说明记录了通信域中 `int64` 集合通信算子可能影响断链后的快速恢复，也记录了跨超场景 `sendrecv` 偶发卡住的问题。它们属于版本说明中的风险提示，不是本仓库 310B 已观察到的故障。若案例使用相关通信路径，应在目标设备和实际拓扑上设计单独的故障恢复测试；不使用该路径时，在报告中写明 “不适用”。

7.10.3. CANN 9.0.X 小版本补丁的使用方式

同一 9.0.X 发布线的小版本补丁可能更新配套关系和修复，但不能替代目标产品下载页对 310B 的筛选。采用任一 9.0.X 小版本时应保留：

- 目标 9.0.X 小版本的包名、架构和摘要；
- 驱动、固件、Toolkit、ops/NNAL 的完整组合；
- 旧版本基线报告和新版本回归报告；
- 任何因接口废弃、模型算子或框架插件变化产生的适配提交。

不要只升级 Toolkit 而把驱动、固件和 `torch_npu` 的旧版本写成 “9.0.X 环境”。准确写法应列出每个组件的实际版本和证据。

7.11. 回退与故障处理

7.11.1. 回退原则

官方升级说明指出，从高版本回退到低版本通常需要卸载高版本后重新安装目标低版本，不能假设 `--upgrade` 会自动完成降级。因此升级前要保存旧安装包和配置，并准备与当前业务隔离的回退窗口。回退命令必须按目标版本安装指南执行，不能凭经验组合卸载脚本。

7.11.2. 失败时的最小记录

```
failure_root="$HOME/upgrade-reports/failed-$(date -u +%Y%m%dT%H%M%SZ)"
mkdir -p "$failure_root"
npu-smi info > "$failure_root/npu-smi.txt" 2>&1 || true
dmesg -T | tail -n 200 > "$failure_root/dmesg-tail.txt" 2>&1 || true
journalctl -b --no-pager -n 200 > "$failure_root/journal-tail.txt" 2>&1 || true
ps auxww > "$failure_root/processes.txt"
ss -ltnp > "$failure_root/listeners.txt" 2>&1 || true
```

保存日志后，先停止受影响的服务，确认设备状态和安装目录，再决定回退或提交支持工单。不要在失败后连续尝试不同版本包，也不要删除原始日志。任何 `rm -rf`、卸载脚本或覆盖安装都必须先解析绝对路径并获得维护授权。

7.12. 面向本仓库案例的升级清单

阶段	必做事项	对应案例影响
变更前	固定板卡、SoC、CANN/HDK 组合和 Python 环境	所有案例
资产前	保存 ONNX/OM 清单、输入合同和旧报告	案例 2、3、4、5、7、8、9 尤其重要
设备层	固件、驱动升级并重启，确认 `npu-smi`	所有使用 NPU 的案例
软件层	加载 CANN，确认 ATC、PyACL 和框架插件	案例 1、3、4、5、7、9
模型层	重新检查或转换模型，执行 ACL 数值烟测	所有模型案例
外设层	重新枚举摄像头、USB、ALSA、MIDI 和网络端口	案例 2、3、5、8、9
任务层	使用固定输入重跑功能、性能和长稳测试	对应案例的验收报告
文档层	更新版本清单和证据路径，不覆盖旧报告	教材、README、工程文档

升级完成前，案例文档中的 “已验证” 不应自动继承。最小可接受结论是：设备层、运行时层、模型层和任务层分别有可定位的输出；缺少其中任一层时，结论写为 `untested` 或 `unknown`。

7.13. 版本记录模板

```
路线：A 香橙派 AI Pro 8T / B 华为官方 Atlas 310B
设备：
SoC/算力等级：
操作系统/内核/架构：
升级前驱动：
升级前固件：
升级前 CANN Toolkit/ops/NNAL：
升级前 HDK：
升级前 Python/框架插件：
目标驱动：
目标固件：
```

```
目标 CANN Toolkit/ops/NNAL:
目标 HDK:
安装包摘要与签名位置:
升级顺序: 首次安装 / 覆盖升级 / 仅 CANN / 子包独立升级
维护窗口:
回滚包与配置位置:
设备门结果:
Atlas HDMI/DRM 门结果:
运行时门结果:
模型门结果:
服务门结果:
任务和性能门结果:
结论: observed-pass / observed-fail / untested / unknown
原始报告目录:
```

7.14. 资料来源与使用边界

- 昇腾版本配套查询助手：310B 系列产品的当前软件版本配套查询。本文表格于 2026-10-03 查询，官网将 2026/06/30 标为推荐。
- CANN 9.0.X 版本说明：版本配套、组合包、特性变化、废弃项和已知问题。
- 昇腾资源下载中心：按产品、操作系统、架构筛选驱动、固件和 CANN 包。
- 升级前必读：升级影响、回退限制和固件—驱动—CANN 的升级顺序。具体版本应以对应版本的安装指南为准。
- 安装 NPU 驱动和固件：安装包形式、权限、默认路径和用户组注意事项。
- CANN 下载中心：按具体产品筛选驱动、固件、Toolkit 和配套版本。
- Ascend 310B 驱动升级指南：使用 `upgrade-tool` 查询设备侧系统版本的示例。

本附录中的配套关系来自上述官方查询页和各发布线版本说明。正式升级前，应在昇腾下载中心重新确认页面、包名、签名、适用产品和当前版本的安装指南；本附录不替代厂商发布的安装手册，也不构成对未测试硬件组合的兼容性承诺。